

Оглавление

Предисловие переводчика.....	25
ПРЕДИСЛОВИЕ.....	26
Что нового во второй редакции.....	27
Цели.....	31
Что нового в современном подходе?.....	33
Вопросы и ответы (Q&A).....	36
Другие особенности.....	37
Программы.....	38
Аудитория.....	39
Организация.....	40
Упражнения и проекты по программированию.....	42
Ошибки, недостатки (?).....	43
Выражение признательности.....	43
1 Введение в Си.....	46
1.1 История Си.....	46
Происхождение.....	47
Стандартизация.....	48
Языки на основе Си.....	50
1.2 Сильные и слабые стороны Си.....	52
Сильные стороны.....	53
Недостатки.....	55
Эффективное использование Си.....	58
Q&A (Вопросы и ответы).....	61

2

2 Основы Си.....	65
2.1 Создание простой программы.....	66
ПРОГРАММА Печать каламбура.....	66
Компиляция и связывание (компоновка).....	68
Интегрированные среды разработки.....	70
2.2 Общая форма простой программы.....	71
Директивы.....	72
Функции.....	73
Инструкции.....	76
Печать строк.....	77
2.3 Комментарии.....	78
2.4 Переменные и присваивание.....	82
Типы.....	83
Объявления (Декларации).....	84
Присваивание.....	86
Печать значения переменной.....	88
ПРОГРАММА Вычисление габаритного веса коробки... ..	89
Инициализация.....	92
Печать выражений.....	93
2.5 Чтение ввода.....	94
ПРОГРАММА Вычисление габаритного веса коробки (Пересмотренная).....	95
2.6 Определение имён для констант.....	97
ПРОГРАММА Преобразование градусов Фаренгейта в градусы Цельсия.....	99

3

2.7 Идентификаторы.....	101
Ключевые слова.....	103
2.8 Схема программы на Си.....	104
Вопросы и ответы.....	109
Упражнения.....	117
Проекты по программированию.....	119
3 Форматированный ввод/вывод.....	122
3.1 Функция printf.....	123
Спецификации преобразования.....	126
ПРОГРАММА Использование printf для форматирования чисел.....	128
Эскейп-последовательности.....	131
3.2 Функция scanf.....	132
Как работает scanf.....	134
Обычные символы в форматной строке.....	138
Перепутывание printf со scanf.....	140
ПРОГРАММА Сложение дробей.....	141
Вопросы и ответы.....	142
Упражнения.....	146
Проекты по программированию.....	147
4 Выражения.....	150
4.1 Арифметические операторы.....	152
Определяемое реализацией поведение.....	154
Приоритет операторов и ассоциативность.....	155
ПРОГРАММА Вычислить контрольную цифру UPC.....	156

4

4.2 Операторы присваивания.....	160
Простое присваивание.....	160
Побочные эффекты.....	161
lvalue (л-значения).....	163
Составное присваивание.....	163
4.3 Операторы инкремента и декремента.....	165
4.4 Вычисление выражений.....	168
Порядок вычисления подвыражений.....	171
Неопределённое поведение.....	173
4.5 Инструкции-выражения.....	174
Вопросы и ответы.....	175
Упражнения.....	180
Проекты по программированию.....	185
5 Инструкции выбора.....	187
5.1 Логические выражения.....	189
Реляционные операторы.....	189
Операторы равенства.....	190
Логические операторы.....	192
5.2 Инструкция if.....	194
Составные инструкции.....	195
Компонент else.....	197
Каскадированные инструкции if.....	200
ПРОГРАММА Расчёт комиссии брокера.....	202
Проблема "висячего else".....	205

Условные выражения.....	207
Логические значения в C89.....	209
Логические значения в C99.....	211
5.3 Инструкция switch.....	212
Роль инструкции break.....	217
ПРОГРАММА Программирование печати даты в юридической форме.....	219
Вопросы и ответы.....	221
Упражнения.....	228
Проекты по программированию.....	232
6 Циклы.....	237
6.1 Инструкции while.....	238
Бесконечные циклы.....	242
ПРОГРАММА Печать таблицы квадратов.....	242
ПРОГРАММА Суммирование ряда чисел.....	244
6.2 Инструкция do.....	245
ПРОГРАММА Вычисление числа цифр в целых числах	247
6.3 Инструкция for.....	249
Идиомы инструкции for.....	251
Опускание выражений в инструкции for.....	252
Инструкции for в C99.....	254
Оператор запятая.....	255
ПРОГРАММА Печать таблицы квадратов (повторение)	257
6.4 Выход из цикла.....	259

Инструкция break.....	260
Инструкция continue.....	262
Инструкция goto.....	264
ПРОГРАММА Составление баланса чековой книжки....	266
6.5 Инструкция Null.....	271
Вопросы и ответы.....	275
Упражнения.....	280
Проекты по программированию.....	283
7. Основные типы.....	288
7.1. Целочисленные типы.....	290
Знаковые и беззнаковые целые числа.....	290
Целочисленные типы в C99.....	295
Целочисленные константы.....	296
Целочисленные константы в C99.....	298
Переполнение целых чисел.....	300
Чтение и запись целых чисел.....	301
ПРОГРАММА суммирования ряда чисел (повторение). 303	
7.2 Типы чисел с плавающей точкой.....	305
Стандарт чисел с плавающей точкой IEEE.....	306
Константы из чисел с плавающей точкой.....	308
Чтение и запись чисел с плавающей точкой.....	309
7.3 Типы символов.....	310
Наборы символов.....	311
Операции над символами.....	312

Знаковые и беззнаковые символы.....	314
Арифметические типы.....	315
Управляющие последовательности (Escape- последовательности).....	316
Функции для работы с символами.....	321
Чтение и запись символов с помощью scanf и printf.....	322
Чтение и запись символов с помощью getchar и putchar	323
ПРОГРАММА Длина сообщения.....	326
7.4 Преобразование типов.....	329
Обычные арифметические преобразования.....	330
Преобразование при присваивании.....	335
Неявные преобразования в C99.....	336
Операции приведения типов.....	338
7.5 Определения типов.....	341
Преимущества определений типов.....	342
Определения типов и переносимость.....	343
Оператор sizeof.....	345
Вопросы и ответы.....	347
Упражнения.....	354
Проекты по программированию.....	358
8 Массивы.....	364
8.1 Одномерные массивы.....	365
Индексация массива.....	366
ПРОГРАММА Запись ряда чисел в обратном порядке...	369
Инициализация массива.....	371

Назначенные инициализаторы.....	372
ПРОГРАММА Проверка числа на наличие в нем повторных цифр.....	374
Использование оператора sizeof с массивами.....	376
ПРОГРАММА Расчёт процентов.....	378
8.2 Многомерные массивы.....	381
Инициализация многомерного массива.....	384
Константные массивы.....	386
ПРОГРАММА Сдача карт.....	387
8.3 Массивы переменной длины (C99).....	391
Вопросы и ответы.....	394
Упражнения.....	398
Проекты по программированию.....	401
9 Функции.....	411
9.1 Определение и вызов функции.....	412
ПРОГРАММА Вычисление средних значений.....	412
ПРОГРАММА Печать обратного отсчёта.....	416
ПРОГРАММА Печать каламбура (повторение).....	418
Определения функций.....	420
Вызовы функций.....	423
ПРОГРАММА Проверка: является ли число простым...	425
9.2. Объявление функций.....	428
9.3 Аргументы.....	432
Преобразования аргументов.....	435
Массивы как аргументы.....	437

Параметры - массивы переменной длины.....	444
Использование static в объявлениях параметров-массивов	448
Сложные (составные) литералы.....	449
Инструкция return.....	451
Завершение программы.....	453
Функции exit.....	455
9.6. Рекурсия.....	456
Алгоритм быстрой сортировки.....	458
ПРОГРАММА Quicksort.....	464
Вопросы и ответы.....	468
Упражнения.....	480
Проекты по программированию.....	486
10 Организация программы.....	492
10.1 Локальные переменные.....	493
Статические локальные переменные.....	495
Параметры.....	496
10.2 Внешние переменные.....	496
Пример: использование внешних переменных для реализации стёка.....	497
Плюсы и минусы внешних переменных.....	499
ПРОГРАММА Угадывание числа.....	501
10.3 Блоки.....	512
10.4 Область действия (видимости).....	513
10.5. Организация программы на Си.....	516

ПРОГРАММА Классификация покера.....	518
Вопросы и ответы.....	531
Упражнения.....	533
Проекты по программированию.....	535
11 Указатели.....	539
11.1 Переменные-указатели.....	540
Объявление переменных-указателей.....	542
11.2 Операторы взятия адреса и разыменования.....	543
Оператор адреса (взятия адреса).....	544
Оператор разыменования.....	545
Присваивание указателя.....	547
11.4 Указатели в качестве аргументов.....	550
ПРОГРАММА Поиск наибольших и наименьших элементов в массиве.....	554
Использование const для защиты аргументов.....	557
11.5 Указатели как возвращаемые значения.....	558
Вопросы и ответы.....	560
Упражнения.....	565
Проекты по программированию.....	568
12 Указатели и массивы.....	569
12.1 Арифметика указателей.....	571
Прибавление целого числа к указателю.....	572
Вычитание целого числа из указателя.....	573
Вычитание одного указателя из другого.....	574
Сравнение указателей.....	575

Указатели на составные литералы.....	576
12.2 Использование указателей для обработки массива....	577
Комбинация операторов * и ++.....	579
12.3 Использование имени массива в качестве указателя..	583
ПРОГРАММА Реверсирование последовательности чисел (Пересмотренная).....	585
Массивы как аргументы (пересмотрено).....	587
Использование указателя в качестве имени массива.....	590
12.4 Указатели и многомерные массивы.....	591
Обработка элементов многомерного массива.....	592
Обработка строк многомерного массива.....	594
Обработка столбцов многомерного массива.....	595
Использование имени многомерного массива в качестве указателя.....	596
Указатели и массивы переменной длины (C99).....	598
Вопросы и ответы.....	601
Упражнения.....	605
Проекты по программированию.....	612
13 Строки.....	614
13.1 Строковый литерал.....	616
Escape-последовательности в строковых литералах.....	616
Продолжение строкового литерала.....	618
Как хранятся строковые литералы.....	619
Операции со строковыми литералами.....	621
Строковые литералы и символьные константы.....	622

13.2 Строковые переменные.....	623
Инициализация строковой переменной.....	625
Массивы символов в сравнении с символьными указателями.....	627
13.3 Чтение и запись строк.....	630
Написание строк с помощью printf и puts.....	630
Чтение строк с помощью scanf и gets.....	632
Чтение строк посимвольно.....	635
13.4 Доступ к символам в строке.....	638
13.5 Использование библиотеки строк Си.....	641
Функция strcpy (копирование строки).....	643
Функция strlen (длина строки).....	646
Функция strcat (конкатенация строк).....	647
Функция strcmp (сравнение строк).....	649
ПРОГРАММА Печать списка напоминаний за один месяц	651
13.6 Строковые идиомы.....	658
Поиск конца строки.....	659
Копирование строки.....	663
Массивы строк.....	667
Аргументы командной строки.....	671
ПРОГРАММА Проверка имён планет.....	674
Вопросы и ответы.....	677
Упражнения.....	683
Проекты по программированию.....	691

14	Препроцессор.....	698
14.1	Как работает препроцессор.....	699
14.2.	Директивы по предварительной обработке.....	704
14.3	Определение макроса.....	706
	Простые макросы.....	707
	Параметризованные макросы.....	711
	Оператор #.....	717
	Оператор ##.....	719
	Общие свойства макросов.....	721
	Скобки в определениях макросов.....	723
	Создание более длинных макросов.....	726
	Предопределённые макросы.....	728
	Дополнительные предопределённые макросы в C99.....	731
	Пустые аргументы макроса.....	733
	Макросы с переменным числом аргументов.....	736
	Идентификатор __func__.....	738
14.4.	Условная компиляция.....	739
	Директивы #if и #endif.....	739
	Оператор defined.....	742
	Директивы #ifdef и #ifndef.....	743
	Директивы #elif и #else.....	744
	Использование условной компиляции.....	746
14.5	Прочие директивы.....	749
	Директива #error.....	749

Директива #line.....	751
Директива #pragma.....	754
Оператор _Pragma.....	755
Вопросы и ответы.....	757
Упражнения.....	765
15 Написание больших программ.....	774
15.1 Исходные файлы.....	775
15.2 Файлы заголовков.....	778
Совместное использование макроопределений и типов.....	783
Прототипы функций совместного использования.....	785
Совместное использование объявлений переменных.....	789
Вложенные включения.....	792
Защита файлов заголовков.....	793
15.3. Разделение программы на файлы.....	797
Форматирование текста.....	798
15.4 Создание многофайловой программы.....	813
Makefiles.....	814
Ошибки при компоновке.....	818
Пересборка программы.....	820
Определение макросов вне программы.....	825
Вопросы и ответы.....	826
Упражнения.....	830
Проекты по программированию.....	834
16 Структуры, объединения и перечисления.....	835

16.1	Переменные структуры.....	836
	Объявление переменных структуры.....	837
	Инициализация структурных переменных.....	841
	Назначенные инициализаторы.....	843
	Операции над структурами.....	844
	Тип структуры.....	847
	Объявление тега структуры.....	849
	Определение типа структуры.....	852
	Структуры как аргументы и возвращаемые значения.....	853
	Составные литералы.....	856
16.3	Вложенные массивы и структуры.....	858
	Вложенная структура.....	859
	Массивы структур.....	861
	Инициализация массива структур.....	862
	ПРОГРАММА Обслуживание базы данных деталей.....	865
16.4	Объединения.....	881
	Использование объединений для экономии места.....	886
	Использование объединений для построения смешанных структур данных.....	890
	Добавление "поля тег" в объединение.....	892
16.5	Перечисления.....	895
	Теги перечисления и имена типов.....	897
	Перечисления как целые числа.....	899
	Использование перечислений для объявления " полей тегов"	901

Вопросы и ответы.....	902
Упражнения.....	910
Проекты по программированию.....	924
17 Расширенное использование указателей.....	926
17.1 Динамическое распределение памяти.....	927
Функции выделения памяти.....	928
Нулевой (пустой) указатель.....	929
17.2 Динамически выделяемые строки.....	931
Использование malloc для выделения памяти для строки	932
Использование динамического выделения памяти в строковых функциях.....	934
Массивы динамически выделяемых строк.....	936
ПРОГРАММА печати списка напоминаний за один месяц (Пересмотренная).....	937
17.3 Динамически выделяемые массивы.....	941
Использование malloc для выделения памяти для массива	942
Функция calloc.....	944
Функция realloc.....	945
17.4 Освобождение памяти.....	947
Функция free.....	949
Проблема "висячего указателя".....	950
17.5 Связанные списки.....	952
Объявление типа узла.....	953

Создание узла.....	955
Оператор ->.....	957
Вставка узла в начало связанного списка.....	958
Поиск в связанном списке.....	963
Удаление узла из связанного списка.....	966
Упорядоченный список.....	971
ПРОГРАММА Ведение базы данных деталей (пересмотренная).....	971
17.6 Указатели на указатели.....	982
17.7 Указатели на функции.....	985
Указатели на функции в качестве аргументов.....	985
Функция qsort.....	987
Другое использование указателей на функции.....	992
ПРОГРАММА табуляция тригонометрических функций	995
17.8 Ограниченные указатели (C99).....	1000
19.9 Элементы гибкого массива (C99).....	1004
Вопросы и ответы.....	1008
Упражнения.....	1018
Проекты по программированию.....	1027
18 Объявления (декларации).....	1029
18.1 Синтаксис объявления.....	1030
18.2 Классы хранения.....	1034
Свойства переменных.....	1035
Класс хранения auto (автоматический).....	1037

Статический класс хранения (static).....	1038
Класс хранения extern.....	1042
Класс хранения register.....	1044
Класс хранения функций.....	1046
18.3 Квалификаторы типов.....	1051
18.4 Деклараторы.....	1054
Расшифровка сложных объявлений.....	1057
Использование определений типов для упрощения объявлений.....	1062
18.5 Инициализаторы.....	1063
Неинициализированная переменная.....	1066
18.6 Функции inline (C99).....	1068
Определения inline.....	1070
Ограничения на встроенные функции.....	1073
Использование встроенных функций с GCC.....	1073
Вопросы и ответы.....	1075
Упражнения.....	1084
19 Разработка программы.....	1090
19.1 Модули.....	1093
Согласованность и связанность.....	1098
Типы модулей.....	1099
19.2 Соккрытие информации.....	1100
Модуль стёка.....	1101
19.3 Абстрактные типы данных.....	1108
Инкапсуляция.....	1110

Неполные типы.....	1111
19.4 Абстрактный тип данных Stack.....	1113
Определение интерфейса для ADT Stack.....	1113
Реализация стёкового ADT с использованием массива фиксированной длины.....	1117
Изменение типа элемента в Stack ADT.....	1120
Реализация стёка ADT с использованием динамического массива.....	1123
Реализация стёкового ADT с использованием связанного списка.....	1128
19.5. Проблемы проектирования абстрактных типов данных.....	1133
Соглашения об именах.....	1133
Обработка ошибок.....	1134
Общие ADT.....	1135
ADT в новых языках.....	1137
Q&A (Вопросы и ответы).....	1137
Упражнения.....	1141
Проекты по программированию.....	1144
20. Низкоуровневое программирование.....	1146
29.1 Побитовые операторы.....	1147
Операторы побитового сдвига.....	1147
Побитовое дополнение, И, ИЛИ, исключающее ИЛИ..	1149
Использование побитовых операторов для доступа к битами.....	1152

Использование побитовых операторов для доступа к битовым полям.....	1155
ПРОГРАММА Шифрование XOR.....	1157
20.2 Битовые поля в структурах.....	1160
Как хранятся битовые поля.....	1163
20.3 Другие низкоуровневые техники.....	1166
Определение машинно-зависимых типов.....	1166
Использование объединений для предоставления нескольких представлений данных.....	1167
Использование указателей в качестве адресов.....	1171
Квалификатор типа volatile.....	1172
Вопросы и ответы.....	1174
Упражнения.....	1175
Проекты по программированию.....	1182
21 Стандартная библиотека.....	1183
21.1 Использование библиотеки.....	1183
Ограничения на имена, используемые в библиотеке....	1185
Функции, скрытые макросами.....	1187
21.2 Обзор библиотеки C 89.....	1189
21.3 Изменения библиотеки C99.....	1193
21.4 Заголовок <stddef. h>: Общие определения.....	1197
21.5 заголовок <stdbool.h> (C99): логический тип и значения.....	1200
Вопросы и ответы.....	1200
Упражнения.....	1201

21

Проекты по программированию.....	1205
22 Ввод/вывод.....	1205
22.1 Потоки.....	1207
Указатели файлов.....	1208
Стандартные потоки и перенаправление.....	1209
Текстовые файлы против двоичных файлов.....	1211
22.2 Файловые операции.....	1214
Открытие файла.....	1215
Режимы (Modes).....	1217
Закрытие файла.....	1219
Присоединение файла к открытому потоку.....	1221
Получение имён файлов из командной строки.....	1222
ПРОГРАММА Проверка возможности открытия файла	1224
Временные файлы.....	1225
Буферизация файлов.....	1228
Разные файловые операции.....	1232
22.3 Форматированный ввод/вывод.....	1233
Функции...printf.....	1234
... printf спецификации преобразования.....	1235
C99 Изменения в спецификациях преобразования..printf	1243
Примеры спецификаций преобразования... printf.....	1245
Функции...scanf.....	1250
Строки формата...scanf.....	1251

Спецификации преобразования ...scanf.....	1253
C99 Изменения в спецификациях преобразования ... scanf	1259
Обнаружение условий окончания файла и ошибок.....	1262
22.4 Ввод/вывод символов.....	1267
Функции вывода.....	1267
Функции ввода.....	1269
ПРОГРАММА Копирование файла.....	1272
22.5 Ввод/вывод строк.....	1274
Функции вывода.....	1274
Функции ввода.....	1275
22.6 Ввод / вывод блока.....	1277
22.7 Позиционирование файлов.....	1280
ПРОГРАММА Изменение файла записей деталей.....	1285
22.8 Строковый ввод/вывод.....	1287
Функции вывода.....	1288
Функции ввода.....	1290
Вопросы и ответы.....	1291
Упражнения.....	1302
Проекты по программированию.....	1309
23 Библиотека поддержки чисел и символьных данных.....	1320
24 Обработка ошибок.....	1320
24.1 Заголовок <assert.h>: Диагностика.....	1322
24.2 Заголовок <errno.h>: ошибки.....	1325
Функции perror и strerror.....	1328

24.3 Заголовок <signal.h>: обработка сигналов.....	1330
Сигнальные макросы.....	1331
Функция signal.....	1332
Предопределённые обработчики сигналов.....	1334
Функция raise.....	1337
Тестовые сигналы.....	1338
24.4 Заголовок <setjmp.h>: нелокальные переходы.....	1340
ПРОГРАММА Тестирование setjmp / longjmp.....	1342
Вопросы и ответы.....	1344
Упражнения.....	1349
25 Интернациональные возможности.....	1351
26 Разные библиотечные функции.....	1352
26.1 Заголовок <stdarg.h>: переменное число аргументов	1352
Вызов функции со списком переменного числа аргументов	1357
Функции v...printf.....	1358
Функции v...scanf.....	1361
26.2 Заголовок <stdlib.h>: Общие утилиты.....	1362
Числовые функции преобразования.....	1363
ПРОГРАММА Тестирование функций числового преобразования.....	1367
Функции генерации псевдослучайных последовательностей.....	1372
ПРОГРАММА Тестирование функций генерации псевдослучайной последовательности.....	1374

Связь с окружающей средой.....	1376
Утилиты поиска и сортировки.....	1380
ПРОГРАММА Определение длины воздушного пути в милях.....	1382
Целочисленные арифметические функции.....	1385
26.3 Заголовок <time.h>: Дата и время.....	1386
Функции управления временем.....	1388
Функции преобразования времени.....	1394
ISO 8601.....	1400
ПРОГРАММА Отображение даты и времени.....	1402
Вопросы и ответы.....	1407
Упражнения.....	1412
Проекты по программированию.....	1417
27 Дополнительная поддержка C99 для математики.....	1419
Приложение А Операторы C.....	1419
Приложение Е Кодированный набор символов ASCII.....	1422

K.N King
C programming
A Modern Approach
second edition

Предисловие переводчика

Эту книгу, характеризуя её как превосходную, рекомендуют авторы книги "Искусство программирования на С. Фундаментальные алгоритмы, структуры данных и примеры приложений", Ричард Хесфилд и др., как источник необходимых знаний для прочтения своей книги. Книга, программы из её текста и ответы к некоторым упражнениям размещены на моей странице [ВК](https://vk.com/id624006320) (<https://vk.com/id624006320>), там же перевод книги "Программирование на Си" Т. Габриэля.

"Ответы" в формате pdf можно открывать в LibreOffice Writer для удобного копирования.

К.Н. Кинг в предисловии пишет : "Полномасштабный курс по Си должен последовательно охватывать Главы 1-20, при этом темы из глав 21-27 добавляются по мере необходимости. (Глава 22, которая включает в себя описание ввода / вывода файлов, является самой важной главой этой группы.)"

Мною переведены главы 1-22, 24, 26.

Пропущены главы 23, 25, 27 и приложения; эти главы относятся к Стандартной библиотеке и интернационализации.

Стандартная библиотека Си хорошо представлена в интернете, а интернационализацию и локализацию можно посмотреть у Т. Габриэля.

О замеченных ошибках прошу сообщать по адресу ai-gamazin@ya.ru.

ПРЕДИСЛОВИЕ

*В программировании превращение
очевидного в полезное — это точное определение слова
"разочарование".*

*Эпиграммы в начале
каждой главы взяты из " Epigrams on Programming " Алана
Дж. Перлиса (ACM SIGPMN Notices (сентябрь. 1982): 7-13).*

За годы, прошедшие с первого издания

"Си Программирование: современный подход" был
обнародован целый ряд новых языков на основе языка Си, на
первом месте среди них в первую очередь —Java и Си#, а
родственные языки, такие как Си++ и Perl, достигли большой
известности.

Однако, Си остаётся столь же популярным, как всегда, усиленно трудясь в фоновом режиме, спокойно приводя в действие большую часть программного обеспечения в мире. Он остаётся "лингва франка" (универсальным языком) компьютерной вселенной, как это было в 1996. Более того, первая редакция с её ссылками на DOS и 16-разрядные процессоры стала устаревшей. Второе издание полностью обновлено и улучшено во многих отношениях.

Что нового во второй редакции.

Вот список новых функций и улучшений во втором издании:

■ *Полный охват как стандарта C89, так и C99.*

Большая разница между первым и вторым изданиями связана со стандартом C99. Моя цель заключалась в том, чтобы охватить все существенные различия между C89 и C99, включая все функции языка и библиотечные функции, добавленные в C99.

■ *Включение справочника по всем библиотечным функциям C89 и C99.*

Приложение D в первой редакции описывало все стандартные функции библиотеки C89. В этом издании приложение охватывает все библиотечные функции C89 и C99.

■ *Обстоятельное обсуждение GCC.*

За годы, прошедшие с первой версии, распространилось использование GCC (первоначально компилятор GNU C, теперь коллекция компиляторов GNU). GCC имеет ряд существенных преимуществ. Включая высокое качество, низкую стоимость (то есть, бесплатно) и переносимость между различными аппаратными и программными платформами. В знак признания её (коллекции) растущего значения я включил в этот выпуск больше информации о GCC. Включая обсуждение того, как её использовать, а также общие сообщения об ошибках и предупреждениях GCC.

■ *Новое освещение абстрактных типов данных.*

В первом издании значительная часть Главы 19 была посвящена C ++. Сегодня этот материал кажется менее актуальным, так как студенты, возможно, уже изучили C ++. Java или C #, прежде чем читать эту книгу. В этом выпуске обзор C ++ был заменен обсуждением того, как настраивать абстрактные типы данных в Си.

■ *Расширенное освещение интернациональных компонентов.*

Глава 25, посвящённая международным функциям Си, теперь намного длиннее и детальнее. Информация о наборе символов Unicode / UCS и его кодировке: является основным моментом расширенного освещения.

■ ***Обновлено отражение современных процессоров и операционных систем.***

Когда писал первую редакцию, 16-битовая архитектура и операционная система DOS были все ещё актуальны для многих читателей, но не сегодня.

Я обновил обсуждение, чтобы больше сосредоточиться на 32-разрядных и 64-разрядных архитектурах.

Рост Linux и других версий UNIX продиктовал более пристальное внимание к этому семейству операционных систем, хотя также упоминаются аспекты Windows и операционной системы Mac OS, которые влияют на программистов на Си.

■ ***Больше упражнений и проектов по программированию.***

Первое издание этой книги содержало 311 упражнений.

В этом выпуске почти 500 (498, если быть точным), разделены на две группы: упражнения и проекты по программированию.

■ ***Решения для отдельных упражнений и проектов по программированию.***

Самая частая просьба, которую я получил от читателей первого издания, заключалась в том, чтобы дать ответы на упражнения.

В ответ на эту просьбу я предоставил ответы примерно на одну треть упражнений и программных проектов на веб-сайте <http://www.knking.com/books/c2/answers/index.htm>.

Эта функция особенно полезна для читателей, которые не участвуют в курсах колледжа и нуждаются в способе проверить свою работу. Упражнения и проекты, для которых предоставляются ответы, помечены значком. Значок ("W" означает "ответ доступен в Интернете").

■ ***Защищенный паролем веб-сайт инструктора.***

Для этого издания, я создал новый ресурсный сайт инструктора (доступный через <http://www.knking.com/books/c2>), содержащий решения остальных упражнений и проектов, а также презентации PowerPoint для большинства разделов. Факультет может связаться со мной по адресу sbook@knking.com для получения пароля. Пожалуйста, используйте свой адрес электронной почты в кампусе и включите ссылку на веб-сайт вашего отдела, чтобы я мог подтвердить вашу личность.

Я также воспользовался возможностью, чтобы улучшить формулировки и объяснения по всей книге. Изменения обширны и кропотливы: каждое предложение проверено и, если необходимо, переписано.

Хотя в этом издании многое изменилось, я попытался максимально сохранить исходную нумерацию глав и разделов. Только одна глава (последняя) является совершенно новой, но во многих главах есть дополнительные разделы. В некоторых случаях существующие разделы были перенумерованы. Одно приложение (синтаксис Си) было удалено, но добавлено новое приложение, которое сравнивает C99 с C89.

Цели

Цели этого издания остаются теми же, что и в первом издании:

■ ***Быть ясным, читабельным и, возможно, даже развлекательным.***

Многие книги на Си слишком кратки для среднего читателя. Другие плохо написаны или просто скучны. Я попытался дать четкие, полные объяснения, заправленные достаточным юмором, чтобы заинтересовать читателя.

■ ***Быть доступным широкому кругу читателей.***

Я предполагаю, что у читателя есть, по крайней мере, небольшой предыдущий опыт программирования, но я не предполагаю знание определённого языка. Я пытался свести

жаргон к минимуму и определить термины, которые я использую. Я также попытался отделить продвинутый материал от более элементарных тем, чтобы новичок не был обескуражен.

■ ***Быть авторитетным, но без педантичности.***

Чтобы избежать произвольного выбора того, что включать и что не включать, я попытался охватить все возможности языка Си и библиотеки. В то же время я старался не обременять читателя ненужными подробностями.

■ ***Быть предназначенным для лёгкого обучения.***

Мой опыт преподавания Си подчёркивает важность постепенного изложения особенностей Си. Я использую спиральный подход, в котором кратко вводятся сложные темы, затем один или несколько раз пересматриваются в книге в подробностях, добавляемых каждый раз. Следование преднамеренное, каждая глава выстраивается постепенно на том, что изучено прежде. Для большинства студентов это, пожалуй, лучший подход: он избегает крайностей скуки, с одной стороны, или "информационной перегрузки" с другой.

■ ***Мотивировать языковые функции.***

Вместо того, чтобы просто описать каждую функцию языка и дать несколько простых примеров того, как эта функция используется, я попытался мотивировать каждую функцию и обсудить, как она используется в практических ситуациях.

■ ***Подчеркивать стиль.***

Для каждого программиста Си важно разработать последовательный стиль. Вместо того, чтобы диктовать, каким должен быть этот стиль, я обычно описываю несколько возможностей и позволяю читателю выбрать наиболее привлекательный. Знание альтернативных стилей - большая помощь при чтении чужих программ (за которым программисты часто проводят много времени).

■ *Избегать зависимости от конкретной машины, компилятора или операционной системы.*

Поскольку Си доступен на таком широком спектре платформ, я старался избегать зависимости от какой-либо конкретной машины, компилятора или операционной системы. Все программы переносимы на самые разные платформы.

■ *Для пояснения ключевых понятий используются иллюстрации.*

Я попытался вставить столько рисунков, сколько мог, поскольку, по-моему, они имеют решающее значение для понимания многих аспектов Си. В частности, я старался "анимировать" алгоритмы, по возможности показывая моментальные снимки данных в разных точках вычисления.

Что нового в современном подходе?

Одна из моих самых важных целей заключалась в том, чтобы применить "современный подход" к Си. Вот некоторые из способов, с помощью которых я пытался достичь этой цели:

■ *Поместить Си в перспективу.*

Вместо того, чтобы рассматривать Си как единственный язык программирования, который стоит знать, я рассматриваю его как один из многих полезных языков. Я обсуждаю, для каких приложений Си лучше всего подходит; я также показываю, как извлечь выгоду из сильных сторон Си, минимизируя при этом его слабые стороны.

■ *Подчеркнуть стандартные версии Си.*

Я обращаю минимальное внимание на версии языка до стандарта C89. Есть только несколько ссылок на К & Р Си (версия языка 1978 года, описанная в первом издании книги Брайана Кернигана и Денниса Ритчи "Язык программирования С"). В Приложении С перечислены основные различия между C89 и К & Р Си.

■ *Развенчание мифов.*

Сегодняшние компиляторы часто расходятся с общепринятыми предположениями о Си. Я не колеблясь развенчиваю некоторые мифы о Си или бросаю вызов

убеждениям, которые уже давно являются частью фольклора Си (например, убеждение в том, что арифметика указателей всегда быстрее чем индексирование массива). Я пересмотрел старые соглашения Си, сохранив те, которые по-прежнему полезны.

■ ***Уделено особое внимание разработке программного обеспечения.***

Я рассматриваю Си как инструмент для зрелой разработки программного обеспечения, подчёркивая, как использовать его для решения проблем, возникающих во время программирования в целом. Я подчёркиваю, что программы должны быть читабельными, поддерживаемыми, надёжными и переносимыми, и я уделяю особое внимание сокрытию информации.

■ ***Отложены "на потом" низкоуровневые функции.***

Эти функции первоначально созданные на Си, хотя и удобны для системного программирования, сейчас не столь актуальны, поскольку Си используется для самых разных приложений. Вместо того, чтобы вводить их в ранних главах, как это делают многие книги по Си, я откладываю их до Главы 20.

■ ***Отменена "ручная оптимизация".***

Многие книги учат читателя писать хитрый код, чтобы получить небольшую экономию в эффективности программы. С сегодняшним изобилием оптимизирующих компиляторов

Си эти методы часто больше не нужны; В действительности, они могут привести к программам, которые менее эффективны.

Вопросы и ответы (Q&A)

Каждая глава заканчивается "Вопросами и ответами" (Q&A)-серией вопросов и ответов, связанных с материалами, рассмотренными в этой главе. Темы, затронутые в этих разделах, включают:

■ *Часто задаваемые вопросы.*

Я пытался отвечать на вопросы, которые часто возникают на моих собственных курсах, в других книгах и в группах новостей, связанных с Си.

■ *Дополнительное обсуждение и разъяснение сложных вопросов.*

Хотя читатели, имеющие опыт работы на разных языках, могут быть удовлетворены короткими пояснениями и несколькими примерами, читателям с меньшим опытом нужно больше.

■ *Побочные проблемы, которые не входят в основной поток.*

Некоторые вопросы поднимают технические аспекты, которые не будут представлять интереса для всех читателей.

■ ***Материал слишком продвинутый или слишком эзотерический, чтобы заинтересовать среднего читателя.***

Вопросы такого рода помечены звёздочкой (*). Любопытные читатели, обладающие достаточным опытом программирования, могут пожелать сразу же разобраться в этих вопросах; другие должны обязательно пропустить их при первом чтении. *Предупреждение:* Эти вопросы часто относятся к темам, затронутым в более поздних главах.

■ ***Общие различия между компиляторами Си.***

Я обсуждаю некоторые часто используемые (но нестандартные) функции, предоставляемые конкретными компиляторами.

Другие особенности

В дополнение к разделам вопросов и ответов я включил ряд полезных функций, многие из которых отмечены простыми, но отличительными значками. Си славится своими ловушками; документирование их всех - задача практически

безнадёжная. Я попытался выявить подводные камни, которые являются наиболее распространёнными и / или наиболее важными.

Идиомы - образцы кода, часто встречающиеся в программах на Си, отмечены для быстрого ознакомления.

Рекомендации по переносимости дают советы по написанию программ, которые не зависят от конкретной машины, компилятора или операционной системы.

Врезки охватывают темы, которые не являются строго частью Си, но о которых должен знать каждый знающий программист на Си. (См. ниже пример врезки "Исходный код".)

Приложения содержат ценную справочную информацию.

Программы

Выбор иллюстративных программ - непростая задача. Если программы слишком короткие и искусственные, у читателей не будет никакого представления о том, как функции используются в реальном мире. С другой стороны, если программа слишком реалистична, её смысл можно легко потерять в лесу деталей. Я выбрал средний курс, используя небольшие простые примеры, чтобы дать ясные понятия, когда они будут впервые представлены, а затем постепенно достраивать до завершения программ. Я не включал

программы большой длины; мой опыт показывает, что у инструкторов нет времени для их изложения, а у студентов нет терпения их читать. Я не игнорирую проблемы, возникающие при создании больших программ, однако в Главе 15 ("Написание больших программ") и в Главе 19 ("Дизайн программы") они подробно рассматриваются.

Я сопротивлялся стремлению переписать программы, чтобы воспользоваться преимуществами функций C99, поскольку не каждый читатель может иметь доступ к компилятору C99 или использовать C99. Тем не менее, я использовал заголовок `<stdbool.h>` C99 в нескольких программах, потому что он удобно определяет макросы с именами *bool*, *true* и *false*. Если ваш компилятор не поддерживает заголовок `<stdbool.h>`, вам нужно будет предоставить свои собственные определения для этих имён.

Программы в этом издании претерпели очень незначительные изменения. Функция *main* теперь имеет форму `int main (void) {...}` в большинстве случаев. Это изменение отражает рекомендуемую практику и совместимо с C99, для которого требуется явный тип возврата для каждой функции.

=====

Исходный код

Исходный код для всех программ доступен на knking.com/books/c2. Обновления, исправления и новости о книге также можно найти на этом сайте.

=====

Аудитория

Эта книга разработана в качестве основного текста для курса Си на уровне бакалавриата. Предыдущий опыт программирования на языке высокого уровня или ассемблере является полезным, но не обязательным для читателя с компьютерной грамотностью ("начинающий адепт", как сказал один из моих бывших редакторов).

Поскольку книга является самодостаточной и удобной для использования, а также для обучения, она является превосходным сопроводительным текстом для курса по структурам данных, дизайну компилятора, операционным системам, компьютерной графике, встроенным системам или другим курсам, использующим Си для проектной работы. Благодаря разделам вопросов и ответов, а также акценту на практических проблемах, книга также будет интересна читателям, которые обучаются в классе или изучают Си самостоятельно.

Организация

Книга состоит из четырёх частей:

■ **Основные компоненты Си.** Главы 1-10 охватывают достаточно Си, чтобы позволить читателю писать однофайловые программы, используя массивы и функции.

■ **Расширенные возможности Си.** Главы 11-20 основаны на материале в предыдущих главах. Темы становятся немного сложнее в этих главах, которые обеспечивают углублённое освещение указателей, строк, препроцессора, структур, объединений, перечислений и низкоуровневых функций Си. Кроме того, две главы (15 и 19) предлагают руководство по разработке программ.

■ **Стандартная библиотека Си.** В главах 21-27 основное внимание уделено библиотеке Си - большому набору функций, которые поставляются с каждым компилятором. Эти главы, скорее всего, будут использоваться в качестве справочного материала, хотя частью подходят для лекций.

■ **Справка.** Приложение А даёт полный список операторов языка Си. В приложении В описаны основные различия между C99 и C89, а в Приложении С - различия между C89 и К & R Си. Приложение D представляет собой алфавитный список всех функций в стандартных библиотеках C89 и C99 с подробным описанием каждого из них. Приложение E содержит набор символов ASCII. Аннотированная библиография указывает читателю на другие источники информации.

Полномасштабный курс по Си должен последовательно охватывать Главы 1-20, при этом темы из глав 21-27 добавляются по мере необходимости. (Глава 22, которая включает в себя описание ввода / вывода файлов, является самой важной главой этой группы.) Более короткий курс может пропустить следующие темы без потери непрерывности: Раздел 8.3 (массивы переменной длины), Раздел 9.6 (рекурсия), Раздел 12.4 (указатели и многомерные массивы), Раздел 12.5 (указатели и массивы переменной длины), Раздел 14.5 (прочие директивы), Раздел 17.7 (указатели на функции), Раздел 17.8 (ограниченные указатели), Раздел 17.9 (члены гибкого массива) Раздел 18.6 (встроенные функции), Глава 19 (проектирование программы), Раздел 20.2 (битовые поля в структурах) и Раздел 20.3 (другие низкоуровневые методы).

Упражнения и проекты по программированию

Наличие целого ряда хороших задач явно необходимо для учебника. Это издание книги содержит как упражнения (более короткие задачи, не требующие написания полной программы), так и проекты по программированию (задачи, требующие написания или изменения всей программы).

В некоторых упражнениях есть неочевидные ответы (некоторые люди нескромно называют их "каверзными вопросами" — какая наглость!). Поскольку в программах на Си часто содержится множество примеров такого кода, я считаю, что необходимо обеспечить определённую практику. Однако я буду играть честно, маркируя эти упражнения звёздочкой (*). Будьте осторожны с помеченным упражнением: либо обратите пристальное внимание и задумайтесь, либо пропустите его полностью.

Ошибки, недостатки (?)

Я приложил немало усилий, чтобы обеспечить безошибочность этой книги. Однако, неизбежно, любая книга такого размера содержит несколько ошибок. Если вы заметили какую-нибудь из них, свяжитесь со мной. Я также был бы рад услышать, какие функции вы нашли особенно полезными, без каких из них вы могли бы обойтись и что вы хотели бы добавить.

Выражение признательности

Во-первых, я хотел бы поблагодарить моих редакторов в Нортон, Фреда МакФарлэнда и Аарона Джевсикаса. Фред получил второе издание в стадии реализации и Аарон вмешался с оживлённой эффективностью, чтобы довести его до конца. Я также хотел бы благодарить объединённого главного редактора Кима И, редактора Мэри Келли, начальника производства Роя Тедофф и редакционного помощника Карли Фрейзер.

Я в огромном долгу перед следующими коллегами, которые рассмотрели некоторые или все рукописи для второго издания:

Маркус Буссманн, Университет Торонто

Джим Кларк, Университет Торонто

Карен Рид, Университет Торонто

Питер Сибак, модератор `comp.lang.c.moderated`

Джим и Питер заслуживают особого упоминания за их подробные обзоры, которые спасли меня от ряда неловких промахов. Рецензентами для первого издания в алфавитном порядке были: Сьюзен Андерсон-Фрид, Мануэль Э. Бермудес, Лиза Дж. Браун, Стивен Катер, Патрик Харрисон, Брайан Харви, Генри Лейтнер, Даррелл Лонг, Артур Б. Маккабе, Кэролин Роснер и Патрик Терри.

Я получил много полезных комментариев от читателей первого издания; Я благодарю всех, кто нашёл время, чтобы написать. Студенты и коллеги из Университета штата

Джорджия также предоставили ценную информацию. Эд Буллвинкель и его жена Нэнси были любезны прочесть большую часть рукописи. Я особенно благодарен моему заведующему кафедрой, Йи Пану, который очень поддерживал этот проект.

Моя жена, Сьюзан Коул, была опорой, как всегда. Наши кошки, Деннис, Пунс и Текс также сыграли важную роль в завершении книги. Рounce и Тех с удовольствием вносили некоторые побочные эффекты, чтобы помочь мне не спать, пока я работал до поздней ночи.

Наконец, я хотел бы признать покойного Алана Дж. Перлиса, чьи эпиграммы появляются в начале каждой главы. У меня была привилегия учиться немного у Алана в Йельском университете в середине 70-ых. Я думаю, ему было бы интересно узнать его эпиграммы в книге на Си.

Copyright © 2008 W. W. Norton & Company, Inc. All rights reserved.

1 Введение в Си

Когда кто-то говорит: "Мне нужен язык программирования, в котором необходимо только сказать, что мне нужно сделать", — дайте ему леденец."

Что такое Си? Простой ответ - широко используемый язык программирования, разработанный в начале 1970-х годов в Bell Laboratories - мало передаёт особый аромат Си.

Прежде чем мы углубимся в детали языка, давайте посмотрим, откуда появился Си, для чего он был создан и как он изменился за эти годы (Раздел 1.1). Мы также обсудим сильные и слабые стороны Си и посмотрим, как извлечь максимальную пользу из этого языка (Раздел 1.2).

1.1 История Си

Давайте взглянем на историю Си от его истоков до его совершеннолетия как стандартизированного языка, его влияние на последние языки.

Происхождение

Си - побочный продукт операционной системы UNIX, разработанный в Bell Laboratories Кеном Томпсоном, Деннисом Ритчи и другими. Томпсон единолично написал оригинальную версию UNIX, которая работала на компьютере DEC PDP-7, раннем миникомпьютере с только 8К основной памяти (не стоит забывать, что это был 1969 год!). Как и другие операционные системы того времени UNIX была написана на ассемблере. Программы, написанные на языке ассемблера, как правило мучительны в отладке и их трудно улучшать; UNIX не была исключением. Томпсон решил, что необходим язык более высокого уровня для дальнейшего развития UNIX, поэтому он разработал небольшой язык под названием В.

Томпсон основал В на языке системного программирования BCPL, разработанном в середине 1960-х.

BCPL, в свою очередь, отслеживает свою родословную до Algol 60, одного из самых ранних (и наиболее эффективных) языков программирования. Ритчи вскоре присоединился к проекту UNIX и начал программировать на В. В 1970 году

Bell Labs приобрели PDP-11 для проекта UNIX. Как только В был запущен на PDP-11, Томпсон переписал часть UNIX на В. К 1971 году стало очевидно, что В не подходит для PDP-11, поэтому Ричи начал разрабатывать расширенную версию В. Поначалу он назвал свой язык NB ("New B"), а затем, по мере того, как он стал всё больше отличаться от В, Ричи изменил название на Си. Язык стал достаточно стабильным к 1973 году и UNIX был переписан на Си. Переключение на Си обеспечило важное преимущество: переносимость. Записывая компиляторы Си для других компьютеров в Bell Labs, команда могла также запустить UNIX на этих машинах.

Стандартизация

Си продолжал развиваться в течение 1975 года, особенно между 1977 и 1979 годами.

Именно в этот период появилась первая книга о Си. "Язык программирования Си ", написанный Брайаном Керниганом и Деннисом Ричи и опубликованная в 1978 году, быстро стала библией программистов на Си. В отсутствие официального стандарта для Си эта книга, известная как K & R или "Белая книга" для приверженцев, служила фактически стандартом. В 1975 году было относительно немного программистов на Си, и большинство из них были пользователями UNIX. Однако в 1980 году Си расширился за пределы узких проблем мира

UNIX. Компиляторы Си стали доступны на различных машинах, работающих под разными операционными системами. В частности, Си начал утверждать себя на быстрорастущей платформе IBM PC. С ростом популярности Си возникли проблемы.

Программисты, написавшие новые компиляторы языка Си, полагались на K&R как на эталон. К сожалению, K&R был нечетким в отношении некоторых языковых функций, поэтому компиляторы часто обрабатывали эти функции по-разному. Кроме того, K&R не удалось провести чёткое различие между функциями, принадлежащими Си и входящими в состав UNIX. Хуже того, после публикации K&R Си продолжал меняться с добавлением новых и удалением нескольких старых функций. Вскоре стала очевидной необходимость в подробном, точном и обновлённом описании языка. Без такого стандарта возникли бы многочисленные диалекты, угрожающие одной из основных сильных сторон языка Си - мобильности программ. Разработка американского стандарта для Си началась в 1983 году под эгидой Американского национального института стандартов (ANSI).

После многих пересмотров стандарт был завершён в 1988 году и официально утверждён в декабре 1989 года в качестве стандарта ANSI X3.159-1989. В 1990 году он была одобрен Международной организацией по стандартизации (ISO) в качестве международного стандарта ISO / IEC 9899: 1990. Эта

версия языка обычно называется C89 или C90, чтобы отличить её от оригинальной версии Си, часто называемой К&R Си. В приложении С приведены основные различия между C89 и К&R Си. В 1995 году в языке произошли некоторые изменения (описанные в документе, известном как Поправка 1(Amendment 1)). Более значительные изменения произошли с публикацией нового стандарта ISO / IEC 9899: 1999 в 1999 году. Язык, описанный в этом стандарте, главным образом известен как C99.

Термины "ANSI C", "ANSI / ISO C" и "ISO C", используемые для описания C89, теперь неоднозначны благодаря наличию двух стандартов.

В приложении В перечислены основные различия между C99 и C89.

Языки на основе Си

Си сильно повлиял на современные языки программирования, многие из которых значительно заимствовали у него. Из множества языков на основе Си некоторые из них особенно значимы:

- **Cu ++** включает в себя все возможности Си, но добавляет классы и другие компоненты для поддержки объектно-ориентированного программирования.

- **Java** базируется на Си ++ и поэтому наследует многие Си-

функции.

■ **Cu #** - это более свежий язык, производный от Си ++ и Java.

■ **Perl** изначально был довольно простым скриптовым языком, со временем он вырос и стал использовать многие функции Си.

Учитывая популярность этих новых языков, логично спросить, стоит ли изучать Си. Мне кажется, что стоит, по нескольким причинам. Во-первых, изучение Си может дать вам более глубокое понимание возможностей Си ++, Java, Си #, Perl и других языков на основанных на Си.

Программистам, которые в первую очередь изучают один из этих языков, часто, не удаётся освоить основные характеристики, которые были унаследованы от Си. Во-вторых, существует множество старых программ на Си; вы можете обнаружить, что вам необходимо читать и поддерживать этот код. В-третьих, Си по-прежнему широко используется для разработки нового программного обеспечения, особенно в ситуациях, когда ограничена память или вычислительная мощность или где требуется простота Си.

Если вы ещё не использовали один из новых языков на основе языка Си, вы обнаружите, что эта книга является отличной подготовкой для изучения этих языков.

Он подчёркивает абстрагирование данных, сокрытие информации и другие принципы, которые играют большую роль в объектно-ориентированном программировании. Си ++

включает в себя все возможности Си, поэтому вы сможете использовать все, что узнаете из этой книги, если позже займётесь Си ++. Многие из особенностей Си могут быть найдены и в других языках на основе Си.

1.2 Сильные и слабые стороны Си.

Как и любой другой язык программирования, Си имеет сильные и слабые стороны. Обе они связаны с оригинальным использованием языка (написанием операционных систем и другого системного программного обеспечения) и его основополагающей философией:

Си - это язык низкого уровня. Чтобы служить подходящим языком для системного программирования, Си предоставляет доступ к концептам машинного уровня (например, байтам и адресам), которые другие языки программирования пытаются скрыть. Си также обеспечивает операции, которые тесно связаны с встроенными инструкциями компьютера, так что программы могут быть быстрыми. Поскольку прикладные программы полагаются на них для ввода / вывода, управления хранилищами и множеством других услуг, операционная система не может быть медленной.

Си - малогабаритный язык.

Си предоставляет более ограниченный набор функций, чем многие языки. (Справочное руководство во втором издании K&R охватывает весь язык на 49 страницах.) Чтобы количество функций было небольшим. Си сильно зависит от "библиотеки" стандартных функций ("функция" похожа на то, что другие языки программирования могут называть "процедурой", "подпрограммой" или "методом").

Си - разрешающий язык. Си предполагает, что вы знаете, что вы делаете, поэтому он позволяет вам более широкую степень свободы, чем многие языки. Более того, в отличие от других языков, Си не требует детальной проверки ошибок.

Сильные стороны

Сильные стороны Си помогают объяснить, почему язык стал настолько популярным:

Эффективность.

Эффективность была одним из преимуществ Си с самого начала. Поскольку Си предназначался для приложений, где традиционно использовался язык ассемблера, крайне важно, чтобы программы на Си могли работать быстро и в ограниченном объёме памяти.

Переносимость.

Несмотря на то, что переносимость программы не была основной целью Си, она стала одной из сильных сторон языка. Когда программа должна запускаться на компьютерах, от ПК до суперкомпьютеров, она часто пишется на Си. Одна из причин переносимости программ на Си заключается в том, что благодаря ранней ассоциации Си с UNIX и более поздним стандартам ANSI / ISO язык не имеет расщепления на несовместимые диалекты. Другая причина в том, что компиляторы Си небольшие и написаны просто, что помогло сделать их широко доступными. Наконец, сам Си имеет функции, которые поддерживают переносимость (хотя нет ничего, что могло бы помешать программистам писать непереносимые программы).

Мощность.

Большая коллекция типов данных и операторов Си делает его мощным языком. В Си часто можно выполнить довольно многое всего лишь с несколькими строками кода.

Гибкость.

Хотя Си первоначально был разработан для системного программирования, у него нет никаких прирождённых ограничений, которые ограничивают его этой ареной. Си теперь используется для приложений всех видов, от встроенных систем до коммерческой обработки данных. Более того, Си накладывает очень мало ограничений на

использование его функций; операции, которые были бы незаконными в других языках, часто разрешаются в Си. Например, Си позволяет добавить символ к целочисленному значению (или, если на то пошло, числу с плавающей точкой). Эта гибкость может сделать программирование проще, хотя она может позволить проскользнуть некоторым ошибкам.

Стандартная библиотека.

Одной из сильных сторон Си является его стандартная библиотека, которая содержит сотни функций для ввода / вывода, обработки строк, распределения памяти и других полезных операций.

Интеграция с UNIX.

Си особенно эффективен в сочетании с UNIX (включая популярный вариант, известный как Linux).

На самом деле некоторые инструменты UNIX предполагают, что пользователь знает Си.

Недостатки

Слабые стороны Си возникают из того же источника, что и многие его сильные стороны: близость Си к машине. Вот некоторые из наиболее известных проблем Си:

Си-программы могут быть подвержены ошибкам.

Гибкость Си делает его языком, подверженным ошибкам. Ошибки программирования, которые были бы обнаружены во многих других языках, не обнаруживаются компилятором Си. В этом отношении Си очень похож на язык ассемблера, где большинство ошибок не обнаруживается до запуска программы. Хуже того, Си содержит ряд ловушек для неосторожного. В последующих главах мы увидим, как дополнительная точка с запятой может создать бесконечный цикл, а отсутствующий символ & может вызвать сбой программы.

Си программы могут быть трудны для понимания.

Хотя Си по большинству показателей является малогабаритным языком, он имеет ряд компонентов, которые не встречаются в остальных языках программирования (и, следовательно, часто неправильно понимаются). Эти компоненты можно комбинировать самыми разными способами. Многие из которых, хотя и очевидны для автора программы, могут быть трудными для понимания другими. ещё одна проблема - это краткий характер программ на Си. Си был разработан в то время, когда интерактивное общение с компьютерами было в лучшем случае утомительным. В результате Си был целенаправленно сведён к минимальным затратам времени, необходимого для ввода и редактирования программ. Гибкость Си может также быть отрицательным фактором; программисты, которые слишком умны, перемудрив, могут сделать программы почти непонятными.

Программы на Си могут быть трудно модифицируемыми.

Большие программы, написанные на Си может быть трудно модифицировать, если они не были разработаны с учётом сопровождения. Современные языки программирования обычно предоставляют такие компоненты, как классы и пакеты, которые поддерживают разделение большой программы на более управляемые куски. Си, к сожалению, не хватает таких компонентов.

=====

Запутанный Си

Даже самые горячие поклонники Си признают, что код Си может быть трудно читаемым. Ежегодный Международный конкурс запутанного кода Си на самом деле поощряет соперников писать самые, насколько это возможно, запутывающие программы Си. Программы-победители действительно чрезвычайно сложны, как показывает "Лучшая небольшая программа" за 1990г. ;

```
v,i,j,k,l,s,a[99];
```

```
main()
```

```
{
```

```

    for(scanf("%d",&s);*a-s;v=a[j*=v]-
a[i],k=i<s,j+=(v=j<s&&(!k&&!!printf(2+"\n\n%c"-(l<=<!j),"
#Q"[l^v?(l^j)&1:2])&&++l||a[i]<s&&v&&v-i+j&&v+i-j))&&!(l
%=s),v||(i==j?a[i+=k]=0:++a[i])>=s*k&&++a[--i])
;
}

```

Эта программа, написанная Дороном Осовлански и Барухом Ниссенбаумом, на самом деле печатает все решения задачи о восьми ферзях (проблема размещения восьми ферзей на шахматной доске таким образом, чтобы ни один ферзь не напал на любого другого ферзя). Она работает для любого числа ферзей от четырёх до 99. Для ознакомления с большим числом программ-победителей посетите веб-сайт конкурсов www.ioccc.org.

=====

Эффективное использование Си

Эффективное использование Си требует использования сильных сторон Си, избегая при этом его слабостей. Вот несколько советов:

Узнайте, как избежать подводных камней Си. Советы по избеганию ловушек разбросаны по всей этой книге. Более обширный список подводных камней см. "Ловушки и подводные камни Си" Эндрю Кенига (Andrew Koenig's C

Traps and Pitfalls, Reading, Mass: Addison-Wesley, 1989).

Современные компиляторы обнаружат распространённые ловушки и выдадут предупреждения, но ни один компилятор не увидит их все.

Используйте программные инструменты, чтобы сделать программы более надёжными.

Программисты Си являются продуктивными разработчиками (и пользователями) инструментов. Один из самых известных инструментов Си называется lint (статический анализатор кода). Традиционно поставляемый с UNIX lint, может подвергнуть программу более обширному анализу ошибок, чем большинство компиляторов Си. Если вам доступен lint (или подобная программа), рекомендуется использовать его. ещё один полезный инструмент - отладчик. Многие ошибки не могут быть обнаружены компилятором из-за природы Си, вместо этого они отображаются в форме ошибок времени выполнения или некорректного вывода. Следовательно, использование хорошего отладчика является практически обязательным для программистов на Си.

Пользуйтесь существующими библиотеками кода.

Одно из преимуществ Си заключается в том, что его используют большое множество людей и ими написан код, который вы можете применять в своих собственных

программах. Си-код часто объединяется в библиотеки (коллекции функций); получение подходящей библиотеки - хороший способ уменьшить количество ошибок и сэкономить значительные усилия при программировании. Библиотеки для обычных задач, включая разработку пользовательских интерфейсов, графику, связь, управление базой данных и создание сетей, легко доступны.

Некоторые библиотеки являются общественным достоянием, иные из них с открытым исходным кодом, а другие продаются как коммерческие продукты.

Примите разумный набор правил кодирования.

Соглашение о кодировании - это правила стиля, возможно, не обязательные в языке, но которые программист решил принять. Хорошо подобранные соглашения помогают сделать программы более однородными, удобными для чтения и более легкими для изменения. Соглашения очень важны при использовании любого языка программирования, но особенно в случае с Си. Как было отмечено выше, очень гибкая природа Си позволяет программистам писать код, который не читается. Примеры программирования, приведённые в этой книге, следуют одному набору соглашений, но существуют и другие, в равной степени допустимые соглашения. (Мы, время от времени, будем обсуждать некоторые из

альтернатив.) Какой набор вы используете, менее важен, чем принятие некоторых соглашений и следование им.

Избегайте "трюков" и чрезмерно сложного кода.

Си поощряет программировать трюки. Обычно в Си есть несколько способов выполнить задание: программисты часто склонны выбирать метод, который наиболее краток.

Не увлекайтесь; самое короткое решение часто сложнее всего понять. В этой книге я покажу стиль, который достаточно краток, но по-прежнему понятен.

Придерживайтесь стандарта. Большинство компиляторов языка Си предоставляют функции языка и библиотечные функции, которые не являются частью стандартов C89 или C99. Для переносимости лучше не использовать нестандартные функции и библиотеки, если они абсолютно не нужны.

Q&A (Вопросы и ответы)

Вопрос: Что такое Раздел Q&A?

Ответ: Рад, что вы спросили. Раздел вопросов и ответов, который появляется в конце каждой главы, служит нескольким целям. Основная цель Q&A — дать ответы на

вопросы, которые часто задают ученики, изучающие Си. Читатели могут участвовать в диалоге (более или менее) с автором, в значительной степени так же, как если бы они посещали один из моих классов Си. Другая цель Q&A - предоставить дополнительную информацию о темах, затронутых в этой главе. Читатели этой книги, вероятно, будут обладать самыми разными уровнями подготовки. Некоторые из них будут иметь опыт работы на других языках программирования, в то время как другие будут учиться программировать в первый раз.

Читатели, имеющие опыт в других языках могут быть удовлетворены с кратким пояснением и парой примеров, но читателям с меньшим опытом может понадобиться большее.

Итог: если вы найдёте освещение темы отрывочным, просмотрите Q&A для получения более подробной информации. В некоторых случаях Q&A обсуждает общие различия между компиляторами Си. Например, мы рассмотрим некоторые часто используемые (но нестандартные) функции, предоставляемые конкретными компиляторами.

Вопрос: *Что делает lint?*

Ответ: *lint* (статический анализатор кода) проверяет программу на наличие множества потенциальных ошибок, включая, но не ограничиваясь, подозрительные комбинации типов, неиспользуемые переменные, недостижимый код и

непереносимый код. Он создаёт список диагностических сообщений, которые программист затем просеивает.

Преимущество использования *lint* заключается в том, что он может обнаруживать ошибки, пропущенные компилятором. Не забывайте пользоваться *lint-ом*; об этом слишком легко забыть. Однако, *lint* может создавать сообщения сотнями, из которых только часть относится к фактическим ошибкам.

Вопрос: откуда *lint* ("хлопковый пух") получил своё название?

Ответ: в отличие от имён многих других инструментов Unix, *lint* это не аббревиатура; своё название он получил от того, как он подбирает кусочки "fluff" ("пуха") из программы.

Вопрос: Как мне получить копию *lint*?

Ответ: *lint* - стандартная утилита UNIX; Если вы полагаетесь на другую операционную систему, то, скорее всего, у вас нет *lint*. К счастью, версии *lint* доступны от сторонних разработчиков.

Расширенная версия *lint* известная как *splint* (Secure Programming Lint) включена во многие дистрибутивы Linux и может быть бесплатно загружена с сайта www.splint.org.

Вопрос: Есть ли способ заставить компилятор выполнить более тщательную проверку ошибок, не используя *lint*?

Ответ: Да. Большинство компиляторов проведут более тщательную проверку программы, если их попросить. В дополнение к проверке на наличие ошибок (бесспорные нарушения правил Си) большинство компиляторов также выдают предупреждающие сообщения, указывающие на возможные проблемы. Некоторые компиляторы имеют более одного "уровня предупреждения"; выбор более высокого уровня заставляет компилятор проверять большее число возможных проблем, чем выбор более низкого уровня. Если ваш компилятор поддерживает уровни предупреждений, рекомендуется выбрать самый высокий уровень, заставляя компилятор выполнять наиболее полную работу, которую он способен. Параметры проверки ошибок для компилятора GCC, распространяемого с Linux, обсуждаются в разделе вопросов и ответов в конце Главы 2.

***Вопрос:** Я заинтересован в том, чтобы моя программа была максимально надёжной. Существуют ли другие инструменты, кроме lint и debuggers?

Ответ: Да. Другие распространённые инструменты включают в себя *"bounds-checkers"* и *"leak-finders"*. Си не требует, чтобы индексы массивов проверялись, а *bounds-checkers* добавляет эту возможность.

leak-finders помогает найти "утечки памяти": блоки памяти,

которые динамически распределены, но никогда не освобождаются.

* Помеченные вопросы охватывают слишком продвинутые или слишком эзотерические материалы, чтобы заинтересовать среднего читателя, и часто ссылаются на темы рассматриваемые в последующих главах. Любопытные читатели, обладающие достаточным опытом программирования, могут сразу же вникнуть в эти вопросы; другие должны безусловно пропускать их при первом чтении.

2 Основы Си

*То, что для одного
человека константа, для другого — переменная.*

В этой главе представлены некоторые основные понятия, включая директивы препроцессора, функции, переменные и операторы, которые нам понадобятся для написания даже самых простых программ. В последующих главах эти вопросы будут рассмотрены более подробно.

Для начала в Разделе 2.1 представлена небольшая программа на языке Си и описывается, как её компилировать и связывать.

В Разделе 2.2 обсуждаются способы обобщения программы, а в Разделе 2.3 показано, как добавить пояснительные замечания, называемые комментариями.

В Разделе 2.4 представлены переменные, в которых хранятся данные, возможно изменяющиеся во время выполнения программы, а в Разделе 2.5 показано, как использовать функцию *scanf* для чтения данных. Константы - данные, которые не изменяются во время выполнения программы, могут быть заданы именами, как показано в Разделе 2.6.

Наконец, в Разделе 2.7 объясняются правила Си для создания имён (идентификаторов), а в Разделе 2.8 приведены правила для составления программы.

2.1 Создание простой программы

В отличие от программ, написанных на иных языках, программы на Си требуют небольшого "шаблона" - полная программа может быть короткой, в несколько строк.

ПРОГРАММА Печать каламбура.

Первая программа в классической книге Кернигана и Ричи "Язык программирования Си" чрезвычайно коротка; она ничего не делает, только пишет сообщение: "Привет, мир".

В отличие от других авторов Си, я не буду использовать эту программу в качестве моего первого примера.

Однако я буду придерживаться другой традиции Си - шутиливой игре слов.

Вот она: *"Си, или не Си: вот в чем вопрос"*.

Следующая программа, которую мы назовём `pun.c` выводит это сообщение при каждом запуске.

```
//pun.c
```

```
#include <stdio.h>
```

```
int main(void) {
```

```
    printf("Си или не Си: вот в чём вопрос \n");
```

```
    return 0;
```

```
}
```

В Разделе 2.2 подробно описывается форма этой программы. На данный момент, я просто сделаю несколько кратких замечаний. Строка

```
#include <stdio.h>
```

необходима для "включения" информации стандартной библиотеки I/O (ввод/вывод) Си.

Исполняемый код программы находится внутри *main*, которая представляет "главную" программу. Первая строка внутри *main* - это команда для отображения желаемого сообщения. Функция *printf* из стандартной библиотеки I/O, которая может выдавать хорошо отформатированный вывод. Код `\n` указывает *printf* перейти к следующей строке после печати сообщения. Строка *return 0* указывает на то, что программа "возвращает" значение 0 в операционную систему после её завершения.

Компиляция и связывание (компоновка)

Несмотря на свою краткость, подготовка `run.c` для запуска более путана, чем можно было бы ожидать. Во-первых, нам нужно создать файл с именем `run.c`, содержащий программу (это делается в любом текстовом редакторе). Имя файла не имеет значения, но расширение `.c` часто требуют компиляторы. В дальнейшем мы должны преобразовать программу в форму, которую может выполнить машина. Для программы на Си, обычно нужно выполнить три шага:

I. Предпроцесс. Программа сначала передаётся препроцессору, подчиняющемуся командам, с которых начинается программа (называются директивами).

Препроцессор немного похож на редактор; он может делать добавления и модификации программы.

II. Компиляция. Модифицированная программа теперь переходит к компилятору, который преобразует его в машинные инструкции (объектный код). Однако программа ещё не готова к запуску.

III. Связывание (компоновка).

На заключительном этапе компоновщик объединяет объектный код, созданный компилятором, с любым дополнительным кодом, необходимым для получения полной исполняемой программы. Этот дополнительный код включает библиотечные функции (такие как *printf*), которые используются в программе.

К счастью, этот процесс часто автоматизирован, поэтому вы не найдёте его слишком обременительным. Фактически, препроцессор обычно интегрируется с компилятором, поэтому вы, вероятно, даже не заметите его работу. Команды, необходимые для компиляции и компоновки, различаются в зависимости от компилятора и операционной системы.

В UNIX компилятор Си обычно называется **cc**.

Чтобы скомпилировать и связать программу `rip.c`, введите следующую команду в окне терминала или командной строки:

% *cc rip.c*

(Символ **%** - это приглашение UNIX, а не то, что вам нужно ввести.) Связывание происходит автоматически при использовании **cc**; Никакой отдельной команды связывания не требуется. После компиляции и компоновки программы **cc**

оставляет исполняемую программу в файле с именем *a.out* по умолчанию.

cc имеет много опций; одна из них (опция *-o*) позволяет нам выбрать имя файла, содержащего исполняемую программу. Например, если мы хотим, чтобы исполняемая версия *rip.c* называлась *rip*, мы должны выполнить следующую команду:
% *cc -o rip rip.c*

=====

Компилятор GCC

Один из самых популярных компиляторов - компилятор GCC, поставляемый с Linux, но доступный и для многих других платформ. Использование этого компилятора аналогично использованию традиционного компилятора UNIX *cc*.

Например, чтобы скомпилировать программу *rip.c*, мы использовали бы следующую команду:

% *gcc -o rip rip.c*

Раздел вопросов и ответов (Q&A) в конце главы содержит дополнительную информацию о GCC.

=====

Интегрированные среды разработки

До сих пор мы предполагали использовать компилятор командной строки, который вызывается путём ввода команды

в специальном окне, предоставляемом операционной системой. Альтернативным вариантом является использование интегрированной среды разработки (IDE), программного пакета, который позволяет редактировать, компилировать, связывать, выполнять и даже отлаживать программу, не покидая среду. Компоненты интегрированной среды разработки разработаны для совместной работы.

Например, когда компилятор обнаруживает ошибку в программе, он может вызвать редактор, чтобы выделить строку, содержащую ошибку. В средах IDE существует множество вариаций, поэтому я не буду обсуждать их далее в этой книге. Однако я бы порекомендовал проверить, какие IDE доступны для вашей платформы.

2.2 Общая форма простой программы

Давайте подробнее рассмотрим *rip.c* и посмотрим, как мы можем её немного обобщить. Простые программы на Си имеют вид:

директивы

```
int main (void){  
    инструкции
```

}

Обратите внимание, как фигурные скобки показывают, где начинается и заканчивается *main*.

Си использует скобки {} во многом таким же образом, как некоторые другие языки используют такие слова, как *begin* и *end*. Это иллюстрирует общий подход Си: он в значительной степени опирается на аббревиатуры и специальные символы, и это одна из причин того, что программы на языке Си являются лаконичными (или, менее доброжелательно - зашифрованными).

Даже самые простые программы на Си основаны на трёх основных ключевых компонентах: директивах (команды редактирования, которые модифицируют программу до компиляции), функциях (называемых блоками исполняемого кода, например *main*), и инструкциях (*statements**) командах, которые должны выполняться при запуске программы).

Теперь мы более подробно рассмотрим эти компоненты.

(*Дословно *statement* означает высказывание, утверждение однако в применении к компьютерной тематике это не совсем удачно. Часто при переводе используется термин оператор, но этот термин имеет собственное значение (примеры: +,-,* и т.д.). Поэтому при переводе термина *statement* за редкими исключениями было использовано понятие инструкция — наиболее близкое по смыслу. - Из книги У. Савитч Язык C++.)

Директивы

Перед компиляцией программы на Си она сначала редактируется препроцессором. Команды, предназначенные для препроцессора, называются директивами. В главах 14 и 15 подробно обсуждаются директивы. Пока что нас интересует только директива *#include*. Программа *run.c* начинается с строки

```
#include <stdio.h>
```

Эта директива утверждает, что информация из *<stdio.h>* должна быть включена в программу до её компиляции.

<stdio.h> содержит информацию о стандартной библиотеке ввода-вывода Си. Си имеет несколько заголовочных файлов, таких как *<stdio.h>*; каждый содержит информацию о некоторой части стандартной библиотеки.

Причина, по которой мы включаем *<stdio.h>*, заключается в том, что Си, в отличие от некоторых языков программирования, не имеет встроенных команд "read" и "write". Вместо их, возможность выполнять ввод и вывод предоставляется функциями из стандартной библиотеки.

Директивы всегда начинаются с символа #, что отличает их от других элементов в программе на Си. По умолчанию директива занимает всю строку; в конце директивы нет точки с запятой или другого специального маркера.

Функции

Функции подобны "процедурам" или "подпрограммам" в других языках программирования - они являются строительными блоками, из которых построены программы. На самом деле, программа на Си - это не более чем набор функций. Функции делятся на две категории: написанные программистом и те, которые предусмотрены как часть реализации Си. Я буду ссылаться на последние как библиотечные функции, так как они принадлежат к "библиотеке" функций, которые поставляются с компилятором.

Термин "функция" приходит из математики, где функция является правилом для вычисления значения при задании одного или нескольких аргументов:

$$f(x) = x + 1$$

$$g(u, z) = y^2 - z^2$$

В Си более свободно используется термин "функция". В языке Си функция - это просто ряд инструкций, которые были сгруппированы вместе и получили имя. Одни функции вычисляют значение, другие - нет.

Функция, которая вычисляет значение, использует инструкцию *return*, чтобы указать, какое значение она "возвращает". Например, вернуть значение функции, которая добавляет 1 к своему аргументу, может инструкция

```
return x + 1;
```

А вернуть значение функции, которая вычисляет разность квадратов своих аргументов, может инструкция

```
return y * y - z * z;
```

Хотя программа на Си может состоять из многих функций, только функция *main* является обязательной. Особенность *main*:

она вызывается автоматически при выполнении программы. До Главы 9, в которой мы узнаем, как писать другие функции, функция *main* будет единственной функцией в наших программах.

Имя *main* является крайне важным; оно не может быть *begin* или *start*, или даже *MAIN*.

Если *main* является функцией, возвращает ли она значение? Да: она возвращает код состояния, который предоставляется операционной системе, когда программа завершается.

Давайте взглянем на программу *rip.c*:

```
#include <stdio.h>
```

```
int main(void){
```

```
    printf("Си или не Си: вот в чём вопрос \n");
```

```
    return 0;  
}
```

Слово *int* непосредственно перед *main* указывает, что функция *main* возвращает целочисленное значение. Слово *void* в круглых скобках указывает, что *main* не имеет аргументов.

На данный момент мы всегда будем иметь *main*, возвращающую значение 0, что означает нормальное завершение программы. Если в конце функции *main* нет инструкции *return*, программа все равно будет завершена.

Тем не менее, многие компиляторы будут выдавать предупреждающее сообщение (потому что функция должна была вернуть целое число, но не смогла).

Инструкции

Инструкция - это команда, выполняемая при запуске программы.

Мы рассмотрим инструкции позже в книге, в основном в главах 5 и 6.

Программа *rip.c* использует только два вида инструкций. Одной из них является инструкция *return*; другая - вызов функции. Запрашивание функции для выполнения назначенной ей задачи называется вызовом функции.

Например, программа `run.c` вызывает функцию `printf` для отображения на экране строки :

```
printf ("Cu или не Cu: вот в чём вопрос \n");
```

Си требует, чтобы каждая инструкция заканчивалась точкой с запятой.

(Как и в любом хорошем правиле, есть одно исключение: составная инструкция, с которым мы встретимся позже.)

Точка с запятой показывает компилятору, где заканчивается инструкция; поскольку инструкции могут занимать несколько строк, не всегда очевидно, где они заканчиваются. Директивы, с другой стороны, как правило, располагаются по одной на строке, и они не заканчиваются точкой с запятой.

Печать строк

`printf` - это мощная функция, которую мы рассмотрим в Главе 3. До сих пор мы использовали `printf` для отображения строкового литерала - последовательности символов, заключённых в двойные кавычки. Когда `printf` отображает строковый литерал, она не показывает кавычки.

Когда `printf` заканчивает печать, то она не переходит автоматически на следующую выходную строку. Чтобы заставить `printf` перейти на одну строку, мы должны включить `\n` (**символ новой строки**) в строку для печати.

Запись символа новой строки завершает текущую строку вывода; последующий вывод переходит на следующую строку.

Чтобы проиллюстрировать этот момент, рассмотрим эффект замены выражения

```
printf("Cu или не Cu: вот в чём вопрос \n");
```

Двумя вызовами *printf*:

```
printf("Cu или не Cu:");
```

```
printf(" вот в чём вопрос \n");
```

Первый вызов *printf* пишет *Cu или не Cu:*. Второй вызов пишет *вот в чём вопрос* и переход на следующую строку.

Чистый эффект такой же, как и у исходного *printf* -

пользователь не может отличить их друг от друга. Символ новой строки может появляться более одного раза в строчном литерале. Чтобы отобразить сообщение:

Краткость - это душа остроумия.

- - Шекспир

Мы могли бы написать

```
printf ("Краткость - это душа остроумия.\n -- Шекспир\n");
```

2.3 Комментарии

Нашей программе *rip.c* по-прежнему не хватает чего-то важного, а именно - документирования. Каждая программа должна содержать идентифицирующую информацию: имя программы, дату, имя автора, цель программы и т. д.

В языке Си эта информация помещается в **комментарии**. Символ */** обозначает начало комментария, а символ **/* обозначает конец:

```
/* Это комментарий */
```

Комментарии могут появляться практически в любом месте программы, либо на отдельных строках, либо в тех же строках, что и другой текст программы. Вот как может выглядеть *rip.c* с комментариями, добавленными в начале:

```
/* Имя: rip.c */
```

```
/* Цель: печатает шуточный каламбур. */
```

```
/*Автор: К. Н. Кинг */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Cu или не Cu: вот в чём вопрос \n");
```

```
return 0;  
}
```

Комментарии могут распространяться более чем на одну строку, как только компилятор увидит символ */*он читает (и игнорирует) все, что следует, пока не встретится с символом */*.

Если необходимо, можно объединить ряд коротких комментариев в один длинный комментарий:

```
/* Имя: rip.c
```

```
    Цель: печатает шуточный каламбур.
```

```
    Автор: К. Н. Кинг */
```

Однако такой комментарий может быть трудно читаемым, потому что нелегко увидеть где он заканчивается.

Вставка **/* в отдельную строку помогает:

```
/* Имя: rip.c
```

```
    Цель: печатает шуточный каламбур.
```

```
    Автор: К. Н. Кинг
```

```
*/
```

Более того, мы можем сформировать "коробку" вокруг комментария, чтобы он выделялся:

```

/*****

*Имя: rip.c                               *
*Цель: печатает шуточный каламбур.        *
*Автор: К. Н. Кинг                         *
*****/

```

Программисты часто упрощают комментарии в коробках, опуская три стороны:

```

/*

* Имя: rip.c
* Цель: печатает шуточный каламбур.
* Автор: К. Н. Кинг
*/

```

Короткий комментарий может идти в той же строке с кодом программы:

```
int main (void) /* Начало программы main */
```

Такой комментарий иногда называют "крылатым комментарием".

Упущение завершения комментария может заставить компилятор проигнорировать часть вашей программы.

Рассмотрим следующий пример:

```
printf ("Мой "); /* Забыли закрыть комментарий...
```

```
printf ( "ком ");  
printf ("заимел"); /*поэтому он заканчивается здесь ->*/  
printf ( " блох ");
```

Поскольку мы пренебрегли закрытием первого комментария, компилятор игнорирует две средние инструкции, а пример печатает *Мой блох*.

C99 предоставляет второй вид комментариев, который начинается с *//* (два смежных слэша):

```
// Это комментарий
```

Комментарий этого стиля заканчивается автоматически в конце строки.

Чтобы создать комментарий, длина которого превышает одну строку, мы можем либо использовать старый стиль комментария (*/*...*/*), либо поставить *//* в начале каждой строки комментария:

```
// Имя: rip.c  
// Цель: печатает шуточный каламбур.  
// Автор: К. Н. Кинг
```

Новый стиль комментариев имеет несколько важных преимуществ. Во-первых, поскольку комментарий автоматически заканчивается в конце строки, нет никаких шансов, что незавершённый комментарий случайно будет поглощать часть программы. Во-вторых, многострочные

комментарии выделяются лучше, благодаря символам `//`, которые требуются в начале каждой строки.

2.4 Переменные и присваивание

Немногие программы так же просты, как в Разделе 2.1.

Большинство программ должны выполнить серию вычислений перед порождением вывода, и, таким образом, им необходим способ временного хранения данных во время выполнения программы.

В языке Си, как и в большинстве языков программирования, эти места хранения называются переменными.

Типы

Каждая переменная должна иметь *тип*, который определяет, какие данные будут храниться. Си имеет большое разнообразие типов. На данный момент мы ограничимся только двумя: *int* и *float*.

Выбор правильного типа имеет решающее значение, поскольку тип влияет на то, как хранится переменная и какие операции могут выполняться над переменной.

Тип числовой переменной определяет самые большие и самые маленькие числа, которые может хранить переменная; Он также определяет, разрешены ли цифры после десятичной точки. Переменная типа *int* (сокращение от *integer* - целое)

может хранить целое число, например 0, 1, 392 или -2553. Тем не менее, диапазон возможных значений ограничен. Самое большое значение типа *int* обычно 2 147 483 647, но может быть и 32 767. Переменная типа *float* (сокращение от floating-point для чисел с плавающей точкой) может хранить гораздо большие числа, чем переменная типа *int*. Кроме того, переменная с плавающей точкой может хранить числа с цифрами после запятой, например 379,125.

Переменные типа *float* имеют недостатки. Арифметические вычисления с типом *float* могут быть медленнее, чем вычисления для чисел типа *int*. Наиболее важно то, что значение переменной с плавающей точкой часто является просто приближением числа, которое хранилось в нем. Если мы храним 0,1 в переменной с плавающей точкой, мы можем позже обнаружить, что переменная имеет такое значение, как 0.09999999999999987, благодаря ошибке округления.

Объявления (Декларации)

Переменные должны быть объявлены - описаны для компилятора - до того, как они могут быть использованы. Чтобы объявить переменную, сначала укажем тип переменной, а затем её имя. (Имена переменных выбираются программистом с соблюдением правил, описанных в Разделе 2.7.)

Например, мы можем объявить переменные *height* и *profit*

следующим образом:

```
int height;
```

```
float profit;
```

Первое объявление гласит, что *height* является переменной типа *int*, что означает, что *height* может хранить целочисленное значение. Во втором объявлении говорится, что *profit* является переменной типа *float*. Если несколько переменных имеют один и тот же тип, их объявления могут быть объединены:

```
int height, length, width, volume;
```

```
float profit, loss;
```

Обратите внимание, что каждое полное объявление заканчивается точкой с запятой. Наш первый шаблон для *main* не включал объявления. Когда *main* содержит объявления, они должны предшествовать инструкциям:

```
int main (void) {
```

```
    объявления
```

```
    инструкции
```

```
}
```

С точки зрения стиля, хорошо оставить пустую строку между декларациями (объявлениями) и инструкциями. В C99 декларации не обязаны предшествовать всем инструкциям. Например, *main* может содержать объявление, а затем

инструкцию. А затем другое объявление. Для совместимости со старыми компиляторами программы в этой книге не используют это правило.

Однако. В Си ++ и Java-программах принято объявлять переменные перед первым употреблением, так что можно ожидать, что эта практика станет популярной и в программах C99.

Присваивание

Переменной можно передать значение посредством присваивания. Например, инструкции

```
height = 8;
```

```
length = 12;
```

```
width = 10;
```

присваивают значения переменным *height*, *length* и *width*.

Числа 8, 12 и 10 называются константами. Перед тем, как переменной присвоить значение или использовать любым другим способом она должна быть сначала объявлена. Таким образом, мы могли бы написать

```
int height;
```

```
height =8;
```

но не

```
height =8; /*** WRONG -НЕПРАВИЛЬНО ***/
```

```
int height;
```

Константа, присвоенная переменной *float*, обычно содержит десятичную точку. Например, если *profit* является переменной *float*, мы можем написать *profit = 2150.46*; лучше всего добавить букву *f* (для "*float*") к константе, которая содержит десятичную точку, если число присвоено переменной *float*:
profit = 2150.45f;

Невключение *f* может вызвать предупреждение от компилятора. Обычно переменной *int* присваивается значение типа *int*, а переменной *float* обычно присваивается значение типа *float*. Смешивание типов (такое, как присваивание значения *int* переменной *float*, или присваивание значения *float* переменной *int*) возможно, но не всегда безопасно, как мы увидим в Разделе 4.2. Как только переменной присвоено значение, она может быть использована для вычисления значения другой переменной:

```
height = 8;
```

```
length = 12;
```

```
width = 10;
```

```
volume = height * length * width; /* volume теперь 960 */
```

В языке Си знак *** представляет оператор умножения, поэтому этот оператор умножает значения, хранящиеся в *height*, *length* и *width*, а затем присваивает результат переменной *volume*. В целом, правая часть оператора присваивания (*rvalue*) может быть формулой (или выражением, в терминологии Си), включающей константы, переменные и операторы.

Печать значения переменной

Мы можем использовать *printf* для отображения текущего значения переменной. Например, чтобы написать сообщение *Высота: h*, где *h* - текущее значение переменной *height*, мы использовали бы следующий вызов *printf*:

```
printf("Высота: %d\n", height);
```

%d - это указатель места вставки, которое в ходе печати должно быть заполнено значением *height*.

Обратите внимание на размещение *\n* сразу после *%d*, так что *printf* перейдёт к следующей строке после печати значения *height*, *%d* работает только для переменных *int*; для печати переменной *float* мы будем использовать *%f*. По умолчанию *%f* отображает число с шестью цифрами после десятичной точки. Чтобы отобразить *p* цифр после десятичной точкой *%f*, мы можем поместить *.p* между *%* и *f*. Например, чтобы распечатать строку

Прибыль: \$ 2150,48

мы вызываем *printf* следующим образом:

```
printf("Прибыль: $%.2f \n", profit);
```

Не ограничивается число переменных, которые могут быть напечатаны одним вызовом *printf*.

Чтобы отобразить значения двух переменных *height* и *length*, мы могли бы использовать следующий вызов *printf*:

```
printf("Высота: %d Длина: %d\n", height, length);
```

ПРОГРАММА Вычисление габаритного веса коробки

Транспортные компании не очень любят большие, но очень лёгкие коробки, поскольку они занимают ценное место на грузовике или в самолёте.

На самом деле компании часто взимают дополнительную плату за такую коробку, обосновывая плату за её объем, а не за вес. В Соединённых Штатах обычно используется метод, который состоит в том, чтобы разделить объем на 166 (допустимое число кубических дюймов на фунт). Если это число - "размерный" или "объёмный" вес коробки - превышает её фактический вес, стоимость доставки основывается на размерном весе.

(Делитель 166 предназначен для международных перевозок, а размерный вес внутренней поставки вместо этого обычно вычисляется с использованием 194.) Предположим, что вы были наняты транспортной компанией, чтобы написать программу, вычисляющую размерный вес коробки. Поскольку вы новичок в Си, вы начинаете с того, что пишете программу,

которая вычисляет размерный вес конкретного ящика, который составляет 12 "x 10" x 8 ". (" - здесь означает дюйм)

Деление представлено в Си знаком /.

Поэтому очевидным образом вычисление размерного веса будет

$$weight = volume / 166;$$

Где *weight* и *volume* являются целочисленными переменными, представляющими вес и объем коробки. К сожалению, эта формула не совсем то, что нам нужно. В Си, когда одно целое делится на другое, ответ "усекается": все цифры после десятичной точки теряются. Объем коробки

12 "x 10" x 8 "будет 960 кубических дюймов. Деление на 166 даёт ответ 5 вместо 5.783, поэтому мы фактически округляем в меньшую сторону до следующего меньшего фунта, транспортная компания ожидает, что мы округлим в большую сторону. Одним из решений является добавление 165 к объёму до деления на 166:

$$weight = (volume + 165) / 166$$

Объем 166 дал бы вес 331/166, или 1, в то время как объем 167 дал бы 332/166, или 2.

Вычисление веса таким образом даёт нам следующую программу (для "лёгких" коробок).

```
//weight.c
```

```
/* Вычисляет размерный вес коробки 12 "x 10" x 8 "*/
```

```
#include <stdio.h>

int main (void)
{
    int height, length, width, volume, weight;
    height = 5;
    length = 12;
    width = 10;
    volume = height * length * width;
    weight = (volume + 165) / 166;
    printf ("Габаритные размеры: %dx%dx%d\n", length,
                                                    width,height);
    printf ("Объём (кубические дюймы): %d\n", volume);
    printf ("Размерный вес (фунты): %d\n", weight);
    return 0;
}
```

Вывод программы:

Габаритные размеры: 12x10x8

Объём (кубические дюймы): 960

Размерный вес (фунты): 6

Инициализация

Некоторые переменные автоматически устанавливаются в ноль, когда программа начинает выполняться, но в большинстве случаев это не так. Переменная, которая не имеет значения по умолчанию и которой ещё не присвоено значение в программе, считается неинициализированной.

Попытка получить доступ к значению неинициализированной переменной (например, путём отображения переменной с помощью *printf* или использования её в выражении) может привести к непредсказуемому результату, например 2568, - 30891 или некоторому столь же странному числу. С некоторыми компиляторами может произойти худшее - даже сбой программы.

Конечно, мы всегда можем дать переменной начальное значение с помощью присваивания. Но есть более простой способ: поместить начальное значение переменной в её объявление. Например, мы можем объявить переменную *height* и инициализировать её за один шаг:

```
int height = 8;
```

На жаргоне Си значение 8 называется инициализатором. В одном и том же объявлении может быть инициализировано любое количество переменных:

93

```
int height = 8, length = 12, width = 10;
```

Обратите внимание, что для каждой переменной требуется собственный инициализатор. В следующем примере инициализатор 10 используется только для переменной `width`, а не для `height` или `length` (которые остаются неинициализированными): `int height, length, width = 10;`

Печать выражений

printf не ограничивается отображения чисел, хранящихся в переменных; она может отображать значение любого числового выражения. Воспользовавшись этим свойством можно упростить программу и уменьшить количество переменных.

Например, инструкции

```
volume = height * length * width;
```

```
printf("%d\n", volume) ;
```

могут быть заменены

```
printf("%d\n", height * length * width);
```

возможности функции *printf* для печати выражений иллюстрирует один из общих принципов Си: везде, где требуется значение, достаточно любого выражения того же типа.

2.5 Чтение ввода

Поскольку программа *weight.c* вычисляет размерный вес только одной коробки, а это не особенно полезно. Чтобы улучшить программу, нам нужно разрешить пользователю вводить размеры. Для получения входных данных, мы будем использовать функцию *scanf*, коллегу *printf* из библиотеки Си. *f* в *scanf*, как и *f* в *printf*, означает "отформатировать"; И *scanf*, и *printf* требуют использования **строки формата** для указания внешнего вида входных или выходных данных. *scanf* должен знать, в какой форме будут вводить входные данные, так же, как *printf* должен знать, как отображать выходные данные. Для чтения значения *int*, мы будем использовать *scanf* следующим образом:

```
scanf("%d", &i); /*считывает целое число;  
сохраняет в i */
```

Строка `"%d"` сообщает *scanf*, что нужно читать ввод, представляющий целое число; *i* - это переменная типа *int*, в которую *scanf* сохранит входные данные. Символ `&` трудно объяснить на этом этапе; на данный момент, я просто отмечу, что это обычно (но не всегда) требуется при использовании *scanf*. Чтение значения с плавающей точкой требует немного другого вызова *scanf*:

```
scanf ("%f", &x); /* читает значение float; хранит в  
x */
```

`%f` работает только с переменными типа *float*, поэтому я предполагаю, что переменная *x* является переменной с плавающей точкой. Строка `"%f"` указывает *scanf* входное значение в формате *float* (число может содержать десятичную точку, но не обязательно).

ПРОГРАММА Вычисление габаритного веса коробки (Пересмотренная)

Вот улучшенная версия программы измерения размеров, в которой пользователь задаёт размеры.

Обратите внимание, что каждому вызову *scanf* непосредственно предшествует вызов *printf*. Таким образом, пользователь будет знать, когда и какие данные вводить.

```
//dweight2.c
```

/ Вычисляет размерный вес коробки из ввода,
предоставленного пользователем */*

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int height, length, width, volume, weight;
```

```
    printf("Введите высоту коробки: ");
```

```
    scanf("%d", &height);
```

```
    printf("Введите длину коробки: ");
```

```
    scanf("%d", &length);
```

```
    printf("Введите ширину коробки: : ");
```

```
    scanf("%d", &width);
```

```
    volume = height* length * width;
```

```
    weight = (volume + 165) / 166;
```

```
    printf("Объём (кубические дюймы): %d\n", volume);
```

```
    printf("Размерный вес (фунты): %d\n", weight);
```

```
    return 0;
```

```
}
```

Выход программы имеет следующий вид (данные, введённые пользователем подчёркнуты)

Введите высоту коробки: 8

Введите длину коробки: 12

Введите ширину коробки: 10

Объём (кубические дюймы) 960

Размерный вес (фунты) 6

Сообщение, которое запрашивает у пользователя ввод данных (приглашение), обычно не должно заканчиваться символом новой строки, так как мы хотим, чтобы пользователь вводил ввод в той же строке, что и сама подсказка. Когда пользователь нажимает клавишу Enter, курсор автоматически переходит к следующей строке - программе не требуется отображать символ новой строки для завершения текущей строки. Программа *dweight.c* имеет одну проблему: она работает неправильно, если пользователь вводит нечисловые данные. В Разделе 3.2 этот вопрос рассматривается более подробно.

2.6 Определение имён для констант

Когда программа содержит константы, часто полезно давать им имена. Программы *dweight.c* и *dweight2.c* полагаются на константу 166, смысл которой может быть не совсем ясен для

тех, кто читает программу позже. Используя компонент, известный как макроопределение, мы можем дать имя этой константе:

```
#define INCHES_PER_POUND 166
```

#define - это директива препроцессора, как и *#include*, так что нет точки с запятой в конце строки. Когда программа компилируется, препроцессор заменяет каждый макрос значением, который он представляет. (*Макрос (от англ. macros, мн.ч. от macro) — программный объект, который во время вычисления заменяется на новый объект, создаваемый определением макроса на основе его аргументов, затем выражается обычным образом.*)

Например,

```
weight = (volume + INCHES_PER_POUND - 1) /  
INCHES_PER_POUND;
```

станет

```
weight = (volume + 166 - 1) / 166;
```

давая такой же эффект, как если бы мы написали вторую инструкцию первой.

Значение макроса может быть выражением:

```
#define RECIPROCAL_OF_PI (1.0f / 3.14159f)
```

Если он содержит операторы, выражение должно быть заключено в круглые скобки. Заметьте, что мы использовали

только прописные буквы в именах макросов. Это - условие, которому следуют большинство программистов на языке Си, а не требование языка. (Тем не менее, программисты на Си делали это в течение десятилетий, вы, наверное, не хотели бы быть первым отступником.)

ПРОГРАММА Преобразование градусов Фаренгейта в градусы Цельсия

Следующая программа предлагает пользователю ввести температуру по Фаренгейту; затем она печатает эквивалентную температуру по Цельсию. Выход программы будет иметь следующий вид (как обычно, данные, введённые пользователем, подчёркнуты)

Введите температуру по Фаренгейту: 212

Эквивалент по шкале Цельсия: 100,0

В программе предусмотрены температуры, которые не являются целыми числами; Поэтому температура в градусах Цельсия отображается как 100.0 вместо 100. Давайте посмотрим сначала на всю программу, а затем посмотрим, как она комплектуется.

//celsius.c

/ Преобразует температуру по Фаренгейту в градусы Цельсия */*

```
#include <stdio.h>
```

```
#define FREEZING_RT 32.0f
```

```
#define SCALE_FACTOR (5.0f/9.0f)
```

```
int main(void)
```

```
{
```

```
    float fahrenheit, celsius;
```

```
    printf("Введите температуру по Фаренгейту: ");
```

```
    scanf ("%f", &fahrenheit);
```

```
    celsiue = (fahrenheit - FREEZING_PT) *SCALE_FACTOR;
```

```
    printf("Эквивалент по Цельсию: %.1f\n",celsius);
```

```
    return 0;
```

```
}
```

инструкция

*celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;*

Преобразует температуру Фаренгейта в градусы Цельсия.

поскольку

FREEZING_PT означает 32. 0f и *SCALE_FACTOR* означает (5.0f / 9.0f), компилятор видит это выражение как

*celsius = (fahrenheit - 32.0f) * (5.0f / 9.0f);*

Определение *SCALE_FACTOR* как (5.0f / 9.0f) вместо (5/9)

важно, потому что Си усекает результат, при делении двух

целых чисел. Значение (5/9) будет равно 0, что определённо не то, что мы хотим. Вызов *printf* пишет температуру по Цельсию:

```
printf ("Эквивалент по Цельсию:%.1f \n", celsius);
```

Обратите внимание на использование *%.1f* для отображения цельсия с одной цифрой после десятичной точки.

2.7 Идентификаторы

Когда мы пишем программу, нам придётся выбирать имена для переменных, функций, макросов и других объектов. Эти имена называются **идентификаторами**.

В языке Си идентификатор содержит буквы, цифры и символы подчёркивания, но должен начинаться с буквы или символа подчёркивания. (В C99 идентификаторы могут также содержать определённые "универсальные имена символов").

Вот несколько примеров допустимых идентификаторов:

```
times10  get_next_char  _done
```

Следующие не являются допустимыми идентификаторами:

```
10times  get-next-char
```

Символ 10times начинается с цифры, а не буквы или символа подчёркивания.

get-next-char содержит знаки минуса, а не подчёркивания. Си чувствителен к регистру: он различает прописные и строчные буквы в идентификаторах.

Например, следующие идентификаторы все разные:

```
job  joB  jOb  jOB  Job  JoB  JOB  JOB
```

Эти восемь идентификаторов могли бы использоваться одновременно, причём каждый для совершенно другой цели. (Разговор об обфускации!)

(Обфуска́ция- делать неочевидным, запутанным, сбивать с толку) или запутывание кода - приведение исходного текста или исполняемого кода программы к виду, сохраняющему её функциональность, но затрудняющему анализ, понимание алгоритмов работы и модификацию.)

Разумные программисты стараются сделать так, чтобы идентификаторы выглядели иначе, если они каким-то образом не связаны. Поскольку дело имеет значение в языке Си, многие программисты следуют соглашению об использовании только строчных букв в идентификаторах (кроме макросов), с подчёркиванием, вставленным при необходимости для удобочитаемости:

`symbol_table` `current_page` `name_and_address`

Другие программисты избегают подчёркивания, вместо этого используют прописную букву, чтобы начинать каждое слово в идентификаторе:

`symbolTable` `currentPage` `nameAndAddress`

(Первая буква иногда заглавная.) Хотя предшествующий стиль распространён в традиционном Си, последний стиль становится все более популярным благодаря его широкому использованию в Java и Си# (и, в меньшей степени, Си++). Существуют другие условные соглашения; просто

обязательно капитализируйте идентификатор одинаково каждый раз, когда он появляется в программе.

Си не накладывает ограничений на максимальную длину идентификатора, поэтому не бойтесь использовать длинные описательные имена. Такое имя, как *current_page*, гораздо проще понять, чем имя, например, *ср*.

Ключевые слова

Ключевые слова в таблице 2.1 имеют особое значение для компиляторов Си и поэтому не могут использоваться в качестве идентификаторов.

Обратите внимание, что в C99 было добавлено пять ключевых слов (**жирный шрифт**)

auto	enum	restrict	unsigned
break	extern	return	void
case	float	short	volatile
char	for	signed	while
const	goto	sizeof	_Bool
continue	if	static	_Complex
default	inline	struct	_Imaginary
do	int	switch	
double	long	typedef	

else	register	union	
------	----------	-------	--

Таблица 2.1 Ключевые слова

Из-за чувствительности Си к регистру, ключевые слова должны появляться в программах точно так, как показано в таблице 2.1. со всеми буквами в нижнем регистре. (Ключевые слова C99 `_Bool`, `_Complex` и `_Imaginary` являются исключениями из этого правила.) Имена функций в стандартной библиотеке (например, `printf`), также содержат только строчные буквы. Избегайте участи несчастного программиста, который вводит всю программу в верхнем регистре только чтобы узнать, что компилятор не может распознать ключевые слова и вызовы библиотечных функций.

Следите за другими ограничениями для идентификаторов. Некоторые компиляторы обрабатывают определённые идентификаторы (например, *asm*) в качестве дополнительных ключевых слов. Запрещается использовать идентификаторы, относящиеся к стандартной библиотеке. Случайное использование одного из этих имён может привести к ошибке во время компиляции или компоновки. Также не рекомендуется использовать идентификаторы, начинающиеся с подчёркивания.

2.8 Схема программы на Си

Мы можем думать о программе на языке Си как о серии токенов: группе символов, которые не могут быть разделены без изменения их значения. Идентификаторы и ключевые слова являются токенами. Токены - операторы, такие как + и -, знаки препинания, такие как запятая и точка с запятой и строковые литералы.

Например инструкция

```
printf    (    "Height : %d\n"    ,    height    )    ;
1         2         3             4      5      6      7
```

содержит семь токенов: токены 1 и 5 — идентификаторы, 3-строковый литерал, 2, 4, 6 и 7- знаки пунктуации.

В большинстве случаев число пробелов между токенами в программе не является критическим. Как крайность, токены могут примыкать друг к другу без пробелов между ними, кроме случаев, когда это приведёт к объединению двух токенов в третий токен. Например, мы могли бы удалить большую часть пробелов в программе celsius.c в Разделе 2.6. При условии, что мы оставляем пробелы между токенами такими как `int` и `main` и между `float` и `fahrenheit`:

```
/* Converts a Fahrenheit temperature to Celsius */
```

```
#include <stdio.h>
```

```
#define FREEZING_PT 32.0f
```

```
#define SCALE_FACTOR (5.0f/9.0f)

int main(void){ float fahrenheit,celsiue;printf(
"Введите температуру по Фаренгейту: ");scanf("%f",
&fahrenheit);celsius=(fahreueit-
FREEZING_PT)*SCALE_FACTOR;
printf("Эквивалент по Цельсию: %.1f\n",celsius);return 0;}
```

Фактически, если бы страница была более широкой, мы могли бы поместить всю функцию *main* в одну строку. Однако мы не можем поставить всю программу в одну строку, потому что для каждой директивы препроцессора требуется отдельная строка. Сжатие программ в такой манере — плохая идея.

Фактически, добавление пробелов и пустых строк в программу может облегчить чтение и понимание. К счастью, Си позволяет нам вставлять любое количество пробельных символов - пробелов, табуляций и символов новой строки - между токенами. Это правило даёт несколько важных следствий для шаблона программы:

■ *Инструкции можно разделить на любое количество строк.* Например, следующая инструкция настолько длинная, что трудно было бы сжать её в одну строку:

```
printf ( "Dimensional weight (pounds) : %d\n", (volume +
INCHES_PER_POUND - 1) / INCHES_PER_POUND);
```

■ *Пробелы между токенами* улучшают их восприятие. По этой причине я обычно ставлю пробел до и после каждого оператора:

```
volume = height * length * width;
```

Я также помещаю пробел после каждой запятой. Некоторые программисты идут ещё дальше, ставя пробелы вокруг круглых скобок и других знаков препинания.

■ *Отступы* могут облегчить определение вложенности. Например, нам приходится делать отступы объявлений и инструкций, чтобы было ясно, что они вложены внутрь *main*.

■ *Пустые строки* могут делить программу на логические единицы, что облегчает читателю распознавание структуры программы. Программу без пустых строк так же трудно читать, как книгу без глав.

Программа *selsius.c* раздела 2.6 иллюстрирует некоторые из этих рекомендаций. Давайте подробнее рассмотрим функцию *main* в этой программе:

```
int main (void)
{
    float fahrenheit, celsius;
```

```
printf ("Введите температуру по Фаренгейту:");  
scanf ("%f", &fahrenheit);  
  
celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;  
  
printf ("Эквивалент по Цельсию: %.1f\n", celsius);  
  
return 0;  
}
```

Во-первых, смотрите, как пробелы вокруг `=`, `-` и `*` делают эти операторы выделяющимися. Во-вторых, обратите внимание, как отступы объявлений и инструкций делают очевидным их принадлежность *main*. Наконец, обратите внимание, как пустые строки делят *main* на пять частей:

- (1) объявление переменных *fahrenheit* и *celsius*;
- (2) получение температуры по Фаренгейту;
- (3) вычисление значения *celsius*;
- (4) печать температуры по Цельсию;
- (5) возвращение в операционную систему.

При обсуждении макета программы, обратите внимание на то, как я поместил токен `{` под *main ()* и сопоставил ему токен `}` на отдельной строке, выравнивая с `{`.

Размещение `}` на отдельной строке, позволяет вставлять или удалять инструкции в конце функции; выравнивание его с `{` позволяет легко определить конец *main*.

Последние замечания:

Хотя между токенами могут быть добавлены дополнительные пробелы, невозможно добавить пробел в токен без изменения значения программы или возникновения ошибки.

Запись:

```
flo at fahrenheit, celsius;    /*** WRONG ***/
```

или:

```
fl
```

```
oat fahrenheit, celsius; /***WRONG ***/
```

выдаёт ошибку при компиляции программы.

Допускается помещение пробельного символа внутри строкового литерала, хотя это изменяет значение строки.

Однако вставка символа новой строки в строку (иными словами, разбиение строки на две строки) недопустимо:

```
printf("To C, or not to C:
```

```
that is the question.\n"); /*** WRONG ***/
```

Продолжение строки из одной линии в другую требует специального средства, о котором мы поговорим в следующей главе.

Вопросы и ответы

Вопрос: Что означает GCC?

Ответ: GCC изначально обозначал "GNU C compiler". Теперь он обозначает "GNU Compiler Collection", потому что текущая версия GCC компилирует программы, написанные на разных языках, включая Ada, C, C ++, Fortran, Java и Objective-C.

Вопрос: Понятно, а что же означает GNU?

Ответ: GNU расшифровывается как "GNU's Not UNIX!" (И, кстати, произносится как guh-NEW). GNU - это проект Фонда свободного программного обеспечения, организация, созданная Ричардом М. Столлманом в знак протеста против ограничений лицензированного программного обеспечения UNIX. Согласно его веб-сайту, Фонд свободного программного обеспечения полагает, что пользователи должны свободно "запускать, копировать, распространять, изучать, изменять и улучшать " программное обеспечение. GNU Project переписал много традиционного программного обеспечения UNIX с нуля и сделал его общедоступным бесплатно, GCC и другое программное обеспечение GNU имеют решающее значение для Linux. Linux сам по себе является лишь "ядром" операционной системы (частью, которая управляет планированием программ и базовыми услугами ввода-вывода), программное обеспечение GNU необходимо для полнофункциональной

операционной системы. Для получения дополнительной информации о проекте GNU посетите веб-сайт www.gnu.org.

Вопрос: Однако, чем важен GCC?

Ответ: GCC значим по многим причинам, не в последнюю очередь благодаря тому факту, что он свободен и способен компилировать несколько языков. Он работает под многими операционными системами и генерирует код для многих разрозненных ЦП, включая все широко используемые. GCC является основным компилятором для многих операционных систем на базе UNIX, включая Linux, BSD и Mac OS X, и широко используется для разработки коммерческого программного обеспечения. Для получения дополнительной информации о GCC посетите веб-сайт gcc.gnu.org.

Вопрос: Насколько хорош GCC при обнаружении ошибок в программах?

Ответ: GCC имеет различные параметры командной строки, которые контролируют, насколько тщательно он проверяет программы. Когда эти опции используются, GCC неплохо справляется с поиском возможных проблем в программе. Вот некоторые из наиболее популярных опций:

-Wall Заставляет компилятор выдавать предупреждающие сообщения, когда он обнаруживает возможные ошибки. -W может сопровождаться кодами для конкретных предупреждений; -Wall означает "all-W options")

Должен использоваться вместе с -O для максимального эффекта.

-W Выдаёт дополнительные предупреждающие сообщения за исключением сообщений, созданных -Wall.

-pedantic Выдаёт все предупреждения, требуемые стандартом Си, из-за которых программы, использующие нестандартные функции, будут отклонены.

-ansi Отключает функции GCC, которые не являются стандартными, и включает несколько стандартных функций, которые обычно отключены.

-std = c89

-std = c99 Определяет версию Си, которую компилятор должен использовать для проверки программы. Эти параметры часто используются в комбинации:

`% gcc -O -Wall -W -pedantic -std=c99 -o pun pun.c`

Вопрос: Почему Си настолько краток? Кажется, что программы будут более читабельными, если бы Си использовал begin и end вместо скобок {и}, integer вместо int и так далее.

Ответ: Легенда гласит, что краткость программ на Си обусловлена окружающей средой, которая существовала в Bell Labs на момент разработки языка.

Первый компилятор Си работал на DEC PDP-11 (ранний миникомпьютер); программисты использовали телетайп - по

существу, пишущую машинку, подключённую к компьютеру - для ввода и распечаток программ. Поскольку телетайпы были очень медленными (они могли печатать только 10 символов в секунду), минимизировать количество символов в программе было явно выгодно.

Вопрос: В некоторых книгах по Си функция *main* заканчивается на *exit (0)* вместо *return 0*. Это одно и то же?

Ответ: Когда они появляются внутри *main*, эти инструкции действительно эквивалентны: обе завершают программу, возвращая значение 0 в операционную систему. Какую из них использовать - это в основном вопрос вкуса.

Вопрос: Что произойдёт, если программа дойдёт до конца функции *main* без выполнения инструкции *return*?

Ответ: Инструкция *return* не является обязательной; если она отсутствует, программа все равно будет завершена. В C89 значение, возвращаемое операционной системе, не определено. В C99, если *main* объявлена как возвращающая *int* (как в наших примерах), программа возвращает 0 в операционную систему; в противном случае программа возвращает неопределённое значение.

Вопрос: Компилятор полностью удаляет комментарий или заменяет его пробелом?

Ответ: Некоторые старые компиляторы Си удалили все символы в каждом комментарии, что позволяет писать

$a/**/b = 0$, и компилятор интерпретирует его как $ab = 0$;

Однако согласно стандарту Си компилятор должен заменить каждый комментарий одним пробелом, так что этот трюк не работает. Вместо этого мы получим следующую (недопустимую) инструкцию:

$a\ b = 0$;

Вопрос: Как узнать, есть ли у моей программы незавершённый комментарий?

Ответ: Если вам повезёт, программа не будет компилироваться, потому что комментарий сделал программу недопустимой.

Если программа компилируется, существует несколько методов, которые вы можете использовать. Пошаговое выполнение программы с помощью отладчика выявит, не пропускаются ли какие-либо строки.

Некоторые IDE отображают комментарии другим цветом, чтобы отличать их от окружающего кода. Если вы используете такую среду, вы можете легко найти незавершённые комментарии, так как текст программы будет иметь другой цвет, который случайно включён в комментарий. Программа, такая как `lint`, также может помочь.

Вопрос: Можно ли вставить один комментарий в другой?

Ответ: Комментарии в старом стиле (`/* */`) не могут быть вложенными. Например, следующий код недопустим:

```
/*
```

```
/**WRONG**/
```

```
*/
```

Символ `*/` во второй строке совпадает с символом `/*` на первой строке, поэтому компилятор будет помечать символ `*/` на третьей строке как ошибку. Запрет `Си` на вложенные комментарии иногда может быть проблемой. Предположим, мы написали длинную программу, содержащую множество коротких комментариев. Чтобы отключить порцию программы, временно (скажем, на время тестирования), наш первый импульс будет заключаться в том, чтобы "закомментировать" проблемные строки с помощью `/*` и `*/`. К сожалению, этот метод не будет работать, если строки содержат комментарии старого стиля. Комментарии `C99` (те, которые начинаются с `//`, могут быть вложены внутри старых комментариев и это - ещё одно преимущество использования такого комментария. В любом случае, есть лучший способ отключить части программы, как мы увидим позже.

Вопрос: Откуда такое имя - `float`?

Ответ: `float` сокращение от "floating-point", метод хранения чисел, в котором "плавает" десятичная точка.

Значение `float` обычно хранится двумя частями: дробная часть (или мантисса) и показатель степени. Например, число 12.0 может быть сохранено как $1,5 \times 2^3$, где 1,5 - это мантисса, а 3 - показатель степени. В некоторых языках программирования этот тип называется *real*, а не `float`.

Вопрос: Почему константы с плавающей точкой должны заканчиваться буквой `f`?

Ответ: Полное объяснение см. В Главе 7. Вот короткий ответ: константа, содержащая десятичную точку, но не заканчивающуюся на `f` имеет тип `double` (сокращение от "double precision"), значения `double` хранятся более точно, чем значения `float`.

Более того, значения *double* могут быть больше значений *float*, поэтому нам нужно добавить букву *f* при присваивании переменной *float*. Без *f* может быть сгенерировано предупреждение о возможности сохранения числа в переменной *float*, которое превышает ёмкость переменной.

Вопрос: Действительно ли верно, что нет предела по длине идентификатора?

Ответ: Да и нет. В стандарте C89 говорится, что идентификаторы могут быть сколь угодно длинными. Однако компиляторам требуется запомнить только первые 31 символ (63 символа в C99). Таким образом, если два имени начинаются с одних и тех же 31 символа, компилятор может не различать их. ещё более усложняют ситуацию специальные правила для идентификаторов с внешней связью; в эту категорию попадают имена большинства функций. Так как эти имена должны быть доступны компоновщику, а поскольку некоторые старые компоновщики могут обрабатывать только короткие имена, в C89 значимы только первые шесть символов. Более того, регистр букв может не иметь значения. В результате ABCDEFG и abcdefh могут рассматриваться как одно и то же имя. (В C99 первые 31 символ значимы, и учитывается регистр букв.) Большинство компиляторов и компоновщиков более щедры, чем стандарт, поэтому эти правила не являются проблемой на практике. Не беспокойтесь об определении слишком длинных идентификаторов - беспокойтесь о том, чтобы сделать их слишком короткими.

Вопрос: Сколько пробелов я должен использовать для отступов?

Ответ: Это сложный вопрос. Оставьте слишком мало места, и глаз не сможет обнаружить отступы. Оставьте слишком много, и строки будут сползать на экране (или на странице). Многие программисты Си делают отступ инструкций в восемь пробелов (один табулятор), что, вероятно, очень много. Исследования показали, что оптимальное количество отступов составляет три пробела, но многие программисты чувствуют себя некомфортно с числами, которые не являются степенью двойки. Хотя я обычно предпочитаю делать отступы на три или четыре пробела, я буду использовать такие пробелы в этой книге, чтобы мои программы находились в пределах полей.

Упражнения

К Разделу 2.1

1. Создайте и запустите знаменитый "Привет, мир " Кернигана и Ритчи. ":

```
#include <stdio.h>
int main (void){
    printf ("Привет, мир \n");
```

```
}
```

Получили ли вы предупреждение от компилятора? Если да, то что нужно, чтобы оно ушло? Рассмотрим следующую программу:

```
#include <stdio.h>
int main (void){
    printf ("Закон Паркинсона: \nРабота занимает");
    printf (" время \n");
    printf ("отпущенное на неё. \n");
    return 0;
}
```

- (a) Определите директивы и инструкции в этой программе.
- (б) Какой результат производит программа?

3. Сократите программу dweight.c с помощью

(1) замены на присваивания с инициализаторами height, length, и width и (2) удалив переменную weight вместо этого вычисляем $(\text{volume} + 165) / 166$ в пределах последнего printf.

4. Напишите программу, объявляющую несколько переменных int и float - без их инициализации - а затем выводящую их значения.

5. Какие из перечисленных ниже идентификаторов не являются допустимыми в Си?

- (a) 100_bottles
- (b) _100_bottles
- (c) one__hundred__bottles

(d) bottles_by_the_hundred_

(*bottles* - бутылки)

Почему плохая идея иметь более одного смежного подчёркивания в идентификаторе

(Как в `current__balance`, например)?

Какие слова из перечисленных ниже ключевые в Си?

(a) `for`

(b) `if`

(c) `main`

(d) `printf`

(f) `while`

8. Сколько токенов есть в следующем утверждении?

`answer=(3*q-p*p)/3;`

9. Вставьте пробелы между токенами в упражнении 8, чтобы сделать инструкцию более лёгкой для чтения.

10. В программе `dweight.c` (Раздел 2.4), какие пробелы необходимы?

Проекты по программированию

Напишите программу, которая использует `printf`, чтобы отобразить на экране следующий рисунок:

```

        *
      *
    *
  *  *
 *  *
 *
```

2. Напишите программу, которая вычисляет объём сферы радиусом 10 метров, используя формулу $v = \frac{4}{3}\pi r^3$. Пишите дробь $\frac{4}{3}$ как 4.0f/3.0f. (Попробуйте записать её как $\frac{4}{3}$. Что будет?)

Подсказка: Си не имеет оператора возведения в степень, поэтому вам нужно умножить r на себя дважды, чтобы вычислить r^3 .

3. Измените программу проекта программирования 2 так, чтобы она предлагала пользователю ввести радиус сферы.

4. Напишите программу, которая попросит пользователя ввести сумму в долларах и центах и затем отобразит сумму с добавленным 5% - ным налогом:

121

Укажите сумму: 100,00

С добавлением налога: 105,00 \$

5. Напишите программу, которая попросит пользователя ввести значение для x , а затем отобразит значение

следующего многочлена: $3x^5 + 2x^4 - 5x^3 - x^2 + 7x - 6$

Подсказка: Си не имеет оператора возведения в степень, поэтому вам нужно умножить x на себя повторно, чтобы вычислить степени x . (Например, $x * x * x$ равно x в кубе.)

6. Измените программу проекта программирования 5 так, чтобы полином вычислялся по следующей формуле:

$$(((3x+2)x-5)x-1)x+7)x-6$$

Обратите внимание, что модифицированная программа выполняет меньшее количество умножений. Этот метод для вычисления многочленов известен как правило Горнера.

7. Напишите программу, которая попросит пользователя ввести сумму в долларах США, а затем покажет, как заплатить эту сумму, используя наименьшее количество 20, 10, 5 и 1 долларовых банкнот:

Введите сумму в долларах: 93

\$20 банкнот: 4

\$10 банкнот: 1

\$5 банкнот: 0

\$1 банкнот: 3

Подсказка. Разделите сумму на 20, чтобы определить количество необходимых 20 долларовых банкнот, а затем уменьшите сумму на общую стоимость 20 долларовых купюр. Повторите для других размеров купюр. Обязательно используйте целочисленные значения, а не числа с плавающей точкой.

8. Напишите программу, которая рассчитывает остаток ссуды по кредиту после первых, вторых и третьих ежемесячных платежей:

Введите сумму кредита: 20000,00

Введите процентную ставку 6.0

Введите ежемесячный платёж: 386.66

Остаток после: первого платежа: \$19713.34

Остаток после второго платежа: \$19425.25

Остаток после третьего платежа: \$19135.71

Отобразите каждый баланс с двумя цифрами после десятичной точки.

Подсказка: каждый месяц баланс уменьшается на сумму платежа, но увеличивается на остаток, умноженный на ежемесячную процентную ставку. Чтобы найти месячную процентную ставку, конвертируйте процентную ставку, введённую пользователем, в проценты и разделите её на 12.

3 Форматированный ввод/вывод

Простота только мешает поиску недостижимого.

`scanf` и `printf`, которые поддерживают форматированное чтение и запись, являются двумя наиболее часто используемыми функциями в Си. Как показано в этой главе, обе являются мощными, но сложными для правильного использования. Раздел 3.1 описывает `printf`, а Раздел 3.2 посвящён `scanf`. Ни в одном из разделов не приводятся подробных сведений, которых придётся подождать до Главы 22.

3.1 Функция `printf`

Функция `printf` предназначена для отображения содержимого строки, известной как строка формата, со значениями, которые могут быть вставлены в заданные точки строки. При вызове `printf` должна быть снабжена строкой формата, за которой следуют любые значения, которые должны быть вставлены в строку во время печати:

```
printf (string, expr1, expr2,...);
```

Отображаемые значения могут быть константами, переменными или более сложными выражениями. Нет ограничения на количество значений, которое может быть напечатано одним вызовом `printf`.

Строка формата может содержать как обычные символы, так и спецификации преобразования, начинающиеся с символа `%`. Спецификация преобразования является местодержателем для значения, которое подставляется во время печати. Информация, следующая за символом `%`, указывает, как значение преобразуется из внутренней формы (двоичной) в

печатную форму (символы) - вот откуда берётся термин "спецификация преобразования".

Например, спецификация преобразования `%d` указывает, что `printf` преобразует значение `int` из двоичного в строку десятичных цифр, в то время как `%f` делает то же самое для значения с плавающей точкой.

Обычные символы в строке формата печатаются точно так, как они отображаются в строке; спецификации преобразования заменяются значениями, которые должны быть напечатаны. Рассмотрим следующий пример:

```
int i, j;  
float x, y;  
i = 10;  
j = 20;  
x = 43.2892f;  
y = 5527.0f;  
printf("i = %d, j = %d, x = %f, y = %f\n", i, j, x, y);
```

Этот вызов `printf` выдаёт следующий результат:

i = 10, j = 20, x = 43.289200, y = 5527.000000

Обычные символы форматной строки просто копируются в выходную строку. Четыре спецификации преобразования заменяются значениями переменных *i*, *j*, *x* и *y* в указанном порядке.

Компиляторы Си не обязаны проверять, совпадает ли число спецификаций преобразования в строке формата с числом выходных элементов. Следующий вызов `printf` имеет больше спецификаций преобразования, чем значения, которые нужно напечатать:

```
printf ("%d%d\n", i); /* ** WRONG ** */
```

printf правильно напечатает значение *i*, а затем напечатает второе (бессмысленное) целочисленное значение. При вызове со слишком малым количеством спецификаций преобразования возникают схожие проблемы:

```
printf ("%d\n", i, j); /* ** WRONG ** */
```

В этом случае *printf* печатает значение *i*, но не показывает значение *j*. Более того, компиляторы не обязаны проверять, что спецификация преобразования подходит для типа печатаемого элемента. Если программист использует неправильную спецификацию, программа просто выдаст бессмысленный результат. Рассмотрим следующий вызов *printf*, в котором переменная *int i* и переменная *float x* находятся в неправильном порядке:

```
printf ("%f%d \n", i, x); /* ** НЕПРАВИЛЬНО ** */
```

Так как *printf* должен подчиняться форматной строке, он будет выводить значение *float*, а затем значение *int*. К сожалению, оба будут бессмысленными.

Спецификации преобразования

Спецификации преобразования дают программисту большой контроль над появлением вывода. С другой стороны, они могут быть сложными и трудными для чтения. На самом деле, описание спецификаций преобразования в деталях является слишком сложной задачей для решения этой проблемы в начале книги. Вместо этого мы просто кратко рассмотрим некоторые из их более важных возможностей.

В Главе 2 мы увидели, что спецификация преобразования может включать в себя информацию о форматировании. В частности, мы использовали `%1f`, чтобы отобразить значение с плавающей точкой с одной цифрой после десятичной точки. В общем случае спецификация преобразования может иметь вид `%m.rX` или `%-m.rX`, где `m` и `r` - целые константы, а `X` - буква. Оба значения `m` и `r` являются необязательными; если `r` опущено, точка, которая отделяет `m` и `r`, также отбрасывается. В спецификации преобразования `% 10.2f` `m` равно 10, `r` равно 2 и `X` равно `f`. В спецификации `%10f`, `m` равно 10 и `r` (вместе с точкой) отсутствует, а в спецификации `%.2f`, `r` равно 2 и `m` отсутствует. Минимальная ширина поля `m` определяет минимальное количество печатаемых символов. Если значение, которое нужно напечатать, требует меньше, чем `m` символов, значение выравнивается по правому краю в пределах поля. (Иначе говоря, дополнительные пробелы предшествуют значению.) Например, спецификация `%4d` отображает число 123 как `•123`, (В этой главе я буду

использовать **●**, чтобы представить знак пробела.) Если значению, которое нужно напечатать требуется больше, чем `m` символов, ширина поля автоматически расширяется до необходимого размера. Таким образом, спецификация `%4d` отобразит число 12345 как 12345 - никакие цифры не будут потеряны. Помещение знака минус перед `m` вызывает выравнивание по левому краю; спецификация `%-4d` отображает 123 как 123 **●**.

Значение точности.`r`, не так легко описывается, так как это зависит от выбора спецификатора преобразования `X`, `X` указывает, какое преобразование следует применить к значению до его печати. Наиболее распространёнными спецификациями преобразования для чисел являются:

■ `d` - отображает целое число в десятичной форме (основание 10).`r` указывает минимальное количество цифр для отображения (дополнительные нули добавляются к началу числа, если это необходимо); если `r` опущено, предполагается, что оно имеет значение 1. (Другими словами `%d` совпадает с `%.1d`.)

■ `e` - отображает число с плавающей точкой в экспоненциальном формате (научная нотация).`r` показывает, сколько цифр должно появиться после десятичной точки (по умолчанию 6). Если `r` равно 0, десятичная точка не отображается.

■ `f` - отображает число с плавающей точкой в "фиксированном десятичном формате" без показателя степени.`r` имеет то же значение, что и для спецификатора `e`.

■ `g` - отображает число с плавающей точкой либо в экспоненциальном формате, либо в фиксированном десятичном формате в зависимости от размера числа.

р указывает максимальное количество значащих цифр (не цифр после десятичной точки), которые должны отображаться. В отличие от преобразования **f**, преобразование **g** не будет показывать конечные нули. Кроме того, если значение, которое должно быть напечатано, не имеет цифр после десятичной точки, **g** не отображает десятичную точку.

Спецификатор **g** особенно полезен для отображения чисел, размер которых нельзя предсказать, когда программа пишется, или которые имеют тенденцию широко варьироваться по размеру. При использовании для печати умеренно большого или умеренно малого числа спецификатор **g** использует фиксированный десятичный формат. Но когда используется для печати очень большого или очень малого числа, спецификатор **g** переключается в экспоненциальный формат, так что число будет требовать меньше символов.

Есть много других спецификаторов кроме %d, %e, %f и %g. Я буду постепенно вводить многие из них в последующих главах.

Для получения полного списка, и для полного объяснения других возможностей преобразования спецификаций, обратитесь к разделу 22.3.

ПРОГРАММА Использование printf для форматирования чисел

Следующая программа иллюстрирует использование printf для печати целых чисел и чисел с плавающей точкой в различных форматах.

```
//tprintf.c
/* Печать значения int и float в различных форматах */

#include <stdio.h>

int main (void){
    int i;
    float x;
    i=40;
    x=539.21f;
    printf("|%d|%5d|%-5d|%5.3d|\n",i, i, i, i);
    printf("|%10.3f|%10.3e|%-10g|\n",x, x, x);
    return 0;
}
```

Символы | в строках формата printf предназначены только для того, чтобы показать, сколько пространства занимает каждое число при печати:

В отличие от % или \, символ | не имеет особого значения для printf. Результатом работы этой программы является:

```
|40| 40|40 | 040|
| 539.210| 5.392e+02|539.21 |
```

Давайте подробнее рассмотрим спецификации преобразования, используемые в этой программе:

■ % d - отображает i в десятичной форме, используя минимальный объем пространства.

- `% 5d` - отображает `i` в десятичной форме, используя минимум пять символов. Поскольку `i` требует только два символа, добавлено три пробела.
- `% -5d` - отображает `i` в десятичной форме, используя минимум пять символов, так как значение `i` не требует наличия пяти символов, пробелы добавляются позже (т.е. `i` выравнивается по левому краю в поле длиной пять).
- `% 5.3d` - отображает `i` в десятичной форме, используя поле шириной минимум пять символов и, как минимум, три цифры. Так как `i` имеет только две цифры, добавлен дополнительный ноль, чтобы гарантировать три цифры. Результирующее число имеет длину всего три символа, поэтому добавлено два пробела, всего пять символов (`i` выровнено по правому краю).
- `%10.3f` - отображает `x` в фиксированной десятичной форме, используя 10 символов в целом, с тремя цифрами после десятичной точки. Так как `x` требует только семь символов (три перед десятичной точкой, три после десятичной точки и одна для десятичной точки), три пробела предшествуют `x`.
- `%10.3e` - отображает `x` в экспоненциальной форме, используя 10 символов в целом, с тремя цифрами после точки, `x` требует всего девять символов (включая показатель степени), поэтому один пробел предшествует `x`.
- `% -10g` - отображает `x` либо в фиксированной десятичной форме, либо в экспоненциальной форме, используя в общей сложности 10 символов. В этом случае `printf` решит отобразить `x` в фиксированной десятичной форме. Наличие знака минус вызывает левое выравнивание, поэтому за `x` следует четыре пробела.

Эскейп-последовательности

Код `\n`, который мы часто используем в строках формата, называется `escape`-последовательность. Эскейп последовательности позволяют строкам содержать символы, которые в противном случае вызывали бы проблемы для компилятора, включая непечатаемые (управляющие) символы и символы, которые имеют особое значение для компилятора (например, `"`). Далее мы предоставим полный список управляющих последовательностей, а теперь вот пример:

Оповещение (звонок) `\a`

Возврат `\b`

Новая строка `\n`

Горизонтальная табуляция `\t`

Когда они появляются в строках формата `printf`, эти `escape`-последовательности представляют действия, выполняемые при печати. Печать `\a` вызывает звуковой сигнал на большинстве машин. Печать `\b` перемещает курсор на одну позицию назад. Печать `\n` перемещает курсор в начало следующей строки. Печать `\t` перемещает курсор на следующий табулятор.

Строка может содержать любое количество управляющих последовательностей. Рассмотрим следующий пример `printf`, в котором строка формата содержит шесть управляющих последовательностей:

```
printf ("Item\tUnit\tPurchase\n\tPrice\tDate\n");
```

Выполнение этой инструкции печатает двухстрочный заголовок:

Item	Unit	Purchase
	Price	Date

ещё одна распространённая escape-последовательность - это `\`, которая представляет символ `"`. Поскольку символ обозначает начало и конец строки, он не может появляться внутри строки без использования этой escape-последовательности. Вот пример:

```
printf ("\Здравствуйте!\n ");
```

Эта инструкция выдаёт следующий результат:
"Здравствуйте!"

Кстати, вы не можете просто вставить один символ `\` в строку; компилятор предположит, что это начало escape-последовательности. Чтобы напечатать один символ `\`, добавьте два символа в строке:

```
printf ("\\"); /* Печатает один символ \ */
```

3.2 Функция `scanf`

Подобно тому, как `printf` выводит выходные данные в указанном формате, `scanf` читает входные данные в соответствии с определенным форматом.

Строка формата `scanf`, как и строка формата `printf`, может содержать как обычные символы, так и спецификации преобразования. Преобразования, разрешённые в `scanf`, по сути, такие же, как и в `printf`.

133

Во многих случаях строка формата `scanf` будет содержать только спецификации преобразования, как в следующем примере:

```
int i, j;
```

```
float x, y;
```

```
scanf ( "%d%d%f%f", &i, &j, &x, &y);
```

Предположим, что пользователь вводит следующую строку ввода:

```
1 -20.3 -4.0e3
```

`scanf` будет читать строку, преобразовывая её символы в числа, которые они представляют, и затем присваивает 1, -20, 0,3. и 4000,0 переменным `i`, `j`, `x` и `y` соответственно. "Плотно упакованные" форматные строки, такие как `"%d%d%f%f"`, являются обычными для вызовов `scanf`.

В строках формата `printf` менее вероятно наличие таких сомкнутых спецификаций преобразования.

`scanf`, как и `printf`, содержит несколько ловушек для неосторожного. При использовании `scanf` программист должен проверить, что число спецификаций преобразования соответствует количеству входных переменных и что каждое преобразование подходит для соответствующей переменной - как в случае с `printf`, компилятор не предусматривает проверки возможного несоответствия. Другая ловушка это символ `&`, который обычно предшествует каждой переменной в вызове `scanf`. `&` обычно (но не всегда) требуется и программист должен помнить о его использовании.

Забытая поставить символ `&` перед переменной в вызове `scanf`, вы получите непредсказуемые и, возможно, пагубные

результаты. Аварийное завершение программы - обычный результат. По меньшей мере, значение, считанное с входа, не будет сохранено в переменной; вместо этого переменная сохранит своё старое значение (которое может быть бессмысленным, если переменной не было присвоено начальное значение). Опускание `&` является чрезвычайно распространённой ошибкой - будьте осторожны!

Некоторые компиляторы могут обнаружить эту ошибку и выдать предупреждающее сообщение, такое как "формальный аргумент не является указателем" (термин "указатель" определён в Главе 11, символ `&` используется для создания указателя на переменную). Если вы получаете предупреждение, проверьте отсутствие `&`.

Вызов `scanf` является мощным, но не прощающим ошибок способом чтения данных. Многие профессиональные программисты Си избегают `scanf`, вместо этого считывая все данные в форме символов и далее конвертируя их в числовую форму. Мы будем использовать `scanf` совсем немного, особенно в ранних главах этой книги, потому что она обеспечивает простой способ читать числа. Помните, однако, что многие наши программы не будут вести себя корректно, если пользователь введёт неожиданный данные. Как мы увидим позже, возможен программный тест, успешно ли `scanf` читает запрошенные данные (и попытаться восстановить, если это не сделано). Такие тесты нецелесообразны для программ в этой книге - они добавили бы слишком много инструкций и затенили бы суть примеров.

Как работает scanf

scanf действительно может сделать гораздо больше, чем я пока показал. По существу, это функция "сопоставления шаблону" ("сопоставления с образцом") пытается сопоставить группы входных символов спецификациям преобразования. Как и функция printf, scanf управляется строкой формата. Когда она вызывается, scanf начинает обработку информации в строке, начиная слева.

Для каждой спецификации преобразования в строке формата, scanf пытается найти элемент соответствующего типа во входных данных, в случае необходимости пропуская пробелы. Затем scanf читает элемент, останавливаясь, когда он встречается символ, который не может принадлежать элементу. (например, для %d -> 12-) Если элемент был успешно прочитан, scanf продолжает обрабатывать оставшуюся часть строки формата.

Если какой-либо элемент не читается успешно, scanf немедленно возвращает его без просмотра остальной части строки формата (или остальных входных данных). При поиске начала числа scanf игнорирует символы пробела (пробел, горизонтальная и вертикальная табуляции, разрыв страницы и символ перевода строки). В результате числа могут быть помещены в одну строку или разбросаны по нескольким строкам. Рассмотрим следующий вызов scanf:

```
scanf("%d%d%f%f",&i,&j,&x,&y);
```

Предположим, что пользователь вводит три строки ввода:

(Ввод пользователя уходит в буфер и scanf считывает его из буфера.)

1

-20 .3

-4.0e3

scanf видит один непрерывный поток символов:

●●1 ☼ -20●●●.3 ☼ ●●●-4.0e3 ☼

(Я использую ● для представления символа пробела и ☼ для представления символа новой строки.)

Так как она пропускает символы пробела, когда ищет начало каждого числа, scanf сможет успешно прочитать числа. На следующей диаграмме символ *s* под символом указывает, что он был пропущен, а *r* указывает, что он был прочитан как часть входного элемента:

●●1 ☼ -20●●●.3 ☼ ●●●-4.0e3 ☼

SSrSrrrSSSrrSSSSrrrrrrr

scanf "заглядывает" в последний символ "новая строка", фактически не считывая его. (*оставляя в буфере.*) Эта новая строка будет первым символом, прочитанным следующим вызовом scanf. Какие правила выполняет scanf для распознавания целого числа или числа с плавающей точкой?

Когда требуется прочитать целое число, scanf сначала ищет цифру, знак "плюс" или знак "минус"; затем она читает цифры до тех пор, пока не достигнет не цифры.

Когда требуется прочитать число с плавающей точкой, scanf ищет знак плюс или минус (необязательно), за которым следует ряд цифр (возможно, с десятичной точкой), за которыми следует буква e (или E) (необязательно). За буквой

е (или E), необязательный знак (+ или -) и одна или несколько цифр.

Преобразования %e, %f и %g являются взаимозаменяемыми при использовании со scanf; все три следуют тем же правилам для распознавания числа с плавающей точкой. Когда scanf встречается символ, который не может быть частью текущего элемента, символ "помещается обратно" (в буфер), чтобы прочитать его снова во время сканирования следующего элемента ввода или во время следующего вызова scanf.

Рассмотрим следующее (предположительно патологическое) расположение из наших четырёх чисел:

1-20.3-4.0e3 ☼. Давайте использовать тот же вызов scanf, что и раньше:

```
scanf ( "%d%d%f%f",&i,&j,&x,&y);
```

Вот как scanf обрабатывает новый вход:

■ Спецификация преобразования: %d. Первый непробельный входной символ равен 1; Так как целые числа могут начинаться с 1, scanf затем читает следующий символ -.

Распознавая, что - не может появляться внутри целого числа, scanf сохраняет 1 в i и помещает символ - (минус) обратно (в буфер.).

■ Спецификация преобразования: %d. Затем scanf читает символы -, 2, 0 и. (точку). Так как целое число не может содержать десятичную точку, scanf сохраняет -20 в j и помещает символ. (точку) обратно (в буфер.).

■ Спецификация преобразования: %f. scanf читает символы., 3 и -. Так как число с плавающей точкой не может содержать знак минус после цифры, scanf сохранит 0.3 в x и помещает символ - (минус) обратно (в буфер.)

■ Спецификация преобразования: %f. Наконец, scanf считывает символы -, 4,., 0, e, 3. и ⌘ (новая строка). Поскольку число с плавающей точкой не может содержать символ новой строки, scanf сохранит $-4,0 \times 10^3$ в у и поместит символ новой строки обратно (в буфер.).

В этом примере scanf смогла сопоставить каждую спецификацию преобразования в строке формата с элементом ввода. Поскольку символ новой строки не был прочитан, он будет оставлен (в буфере) для следующего вызова scanf.

Обычные символы в форматной строке

Концепцию сопоставления с образцом можно продвинуть ещё на один шаг, написав форматные строки, которые содержат обычные символы, в дополнение к спецификациям преобразования. Меры, которое предпринимает scanf при обработке обычного символа в строке формата, зависит от того, является ли он символом пробела или нет.

Пробельные символы.

Когда scanf встречает один или несколько последовательных символов пробела в строке формата, он многократно читает символы пробела из входа до тех пор, пока не достигнет непробельного (который является "возвращаемым"). Число символов пробела в строке формата не имеет значения; один

символ пробела в строке формата будет соответствовать любому количеству символов пробела на входе. (Кстати, помещение символа пробела в строку формата не заставляет ввод содержать символы пробела. Символ пробела в строке формата соответствует любому количеству символов пробела во вводе, включая ни одного.)

Другие символы.

Когда в строке формата встречается непробельный символ, `scanf` сравнивает его со следующим входным символом. Если оба символа совпадают, `scanf` отбрасывает входной символ и продолжает обработку строки формата. Если символы не совпадают, `scanf` помещает негодный символ обратно во вход (буфер), а затем прерывает дальнейшую обработку строки формата или считывание символов из ввода. Например, предположим, что строка формата `"%d/%d"`, если ввод `5/96` `scanf` пропускает первый пробел при поиске целого числа, сопоставляет `%d` с 5, сопоставляет `/` с `/`, пропускает пробел и ищет другое целое число и сопоставляет `%d` с 96.

С другой стороны, если вход равен `5 / 96` `scanf` пропускает один пробел, сопоставляет `%d` и 5. Затем пытается сопоставить `/` в строке формата с пробелом во вводе. Нет соответствия, поэтому `scanf` помещает пробел обратно (в буфер); символы `5/96` остаются для считывания при следующем вызове `scanf`. Чтобы разрешить пробелы после первого числа, следует использовать строку формата

"%d /%d" вместо прежней.

Перепутывание printf со scanf

Хотя вызовы scanf и printf могут казаться похожими, между двумя функциями существуют значительные различия; Игнорирование этих различий может быть опасным для здоровья вашей программы. Одна распространённая ошибка - помещать & перед переменными в вызове printf:

```
printf("%d%d",&i,&j); /*** WRONG ***/
```

(Современный компилятор отобразит предупреждение, что в строке формата ожидается min int, а второй аргумент имеет min int.).* К счастью, эту ошибку довольно легко обнаружить: printf отобразит пару странного вида чисел вместо значений i и j. Поскольку scanf обычно пропускает символы пробела при поиске элементов данных, часто нет необходимости в том, чтобы строка формата включала символы, отличные от спецификаций преобразования.

Неправильное предположение, что строки формата scanf должны соответствовать строкам формата printf - другая распространённая ошибка - может привести к неожиданному поведению scanf. Давайте посмотрим, что произойдёт, когда будет выполнен следующий вызов функции scanf:

```
scanf ("%d, %d", &i, &j);
```

scanf будет сначала искать целое число во входном файле, которое он сохраняет в переменной *i*, scanf попытается сопоставить запятую со следующим входным символом. Если следующим входным символом является пробел, а не запятая, scanf завершится без чтения значения *j*.

Хотя строки формата `printf` часто заканчиваются символом `\n`, перенос символа новой строки в конец строки формата `scanf` обычно является плохой идеей. Для `scanf` символ новой строки в строке формата эквивалентен пробелу; оба вызывают `scanf` для продвижения к следующему не пробельному символу.

Например, если строка формата `"%d\n"`, `scanf` пропустит пробел, прочитает целое число, а затем переходит к следующему символу, отличному от пробела.

Такая строка формата может привести к зависанию интерактивной программы до тех пор, пока пользователь не введёт непустой символ.

ПРОГРАММА Сложение дробей

Чтобы проиллюстрировать способность `scanf` сопоставлять шаблоны, рассмотрим проблему чтения дроби, введённой пользователем. Дроби обычно пишутся в виде :

числитель / знаменатель.

Вместо того, чтобы пользователь вводил числитель и знаменатель дроби в виде отдельных целых чисел, `scanf` даёт возможность прочитать всю дробь. Следующая программа, которая складывает две дроби, иллюстрирует эту технику.

```
//addfrac.c
```

```
/* Складывает две дроби */
```

```
#include <stdio.h>
```

```
int main(void){  
    int num1, denom1, num2, denom2, result_num,  
        result_denom;  
    printf("Введите первую дробь: ");  
    scanf("%d/%d",&num1, &denom1);  
    printf("Введите вторую дробь: ");  
    scanf("%d/%d", &num2, &denom2);  
    result_num = num1 * denom2 + num2 * denom1;  
    result_denom = denom1 * denom2;  
    printf(" Сумма составляет %d/%d\n",result_num,  
        result_denom);  
    return 0;  
}
```

Сеанс с этой программой может иметь следующий вид:

Введите первую дробь: 5/6

Введите вторую дробь: 3/4

Сумма составляет 38/24

Обратите внимание, что результирующая дробь не сокращена.

Вопросы и ответы

Вопрос: Я видел преобразование %i, используется для чтения

и записи целых чисел. В чем разница между %i и %d?

Ответ: В строке формата printf между ними нет разницы. Однако в строке формата scanf %d может соответствовать целому числу, записанному только в десятичной (основание 10) форме, в то время как %i может соответствовать целому числу, выраженному в восьмеричной (основание 8), десятичной или шестнадцатеричной (основание 16) форме. Если входное число имеет префикс 0 (как в 056), %i рассматривает его как восьмеричное число: если оно имеет префикс 0x или 0X (как в 0x56), %i рассматривает его как шестнадцатеричное число. Использование %i вместо %d для чтения числа может иметь неожиданные результаты, если пользователь случайно поставит 0 в начале числа. Из-за этой ловушки я рекомендую придерживаться %d.

Вопрос: Если printf рассматривает % как начало спецификации преобразования, как я могу напечатать символ %?

Ответ: Если printf встречает два последовательных символа %, в строке формата, он печатает один символ %. Например, инструкция

```
printf ("Чистая прибыль:%d %% \n", profit);
```

Может распечатать

Чистая прибыль: 10%

Вопрос: Предполагается, что `\t` заставит `printf` перейти к следующей позиции табуляции. Как узнать, насколько далеки позиции табуляции?

Ответ: Вы не узнаете. Эффект печати `\t` определяется не в Си; это зависит от того, что делает ваша операционная система при запросе печати символа табуляции. Табуляция обычно состоит из восьми символов, но Си не даёт никакой гарантии.

Вопрос: Что делает `scanf`, если его попросят прочесть число, но пользователь вводит нечисловой ввод?

Ответ: Давайте рассмотрим следующий пример:

```
printf ("Введите число:");  
scanf ("%d", &i);
```

Предположим, что пользователь вводит допустимое число, за которым следуют нечисловые символы:

Введите число: 23foo

В этом случае `scanf` считывает 2 и 3, сохраняя 23 в `i`.

Остальные символы (foo) остаются (в буфере) для чтения следующим вызовом `scanf` (или другой функцией ввода).

С другой стороны, предположим, что ввод неправилен с самого начала:

Введите число: foo

В этом случае в `i` не сохраняется никакого значения, а foo остаётся (в буфере) для следующего вызова `scanf`.

Что мы можем сделать с этим печальным положением дел?

Позже мы увидим, как проверить, удалось ли выполнить вызов `scanf`. Если вызов завершился неудачно, мы можем заставить программу либо прекратить работу, либо попытаться вернуться, скорее всего, отменив проблемный

ввод и попросив пользователя повторить попытку. (Способы отказа от плохого ввода обсуждаются в разделе вопросов и ответов в конце Главы 22.)

Вопрос: Я не понимаю, как `scanf` может "вернуть" символы и прочитать их позже.

Ответ: Как оказалось, программы не читают данные пользователя при вводе. Вместо этого данные сохраняются в скрытом буфере, к которому имеет доступ `scanf`. `scanf` легко помещает символы обратно в буфер для последующего чтения. В Главе 22 более подробно рассматривается буферизация ввода.

Вопрос: Что делает `scanf`, если пользователь ставит знаки препинания (например, между цифрами)?

Ответ: Давайте рассмотрим простой пример. Предположим, что мы пытаемся прочитать пару целых чисел, используя `scanf`:

```
printf ("Введите два числа: ");  
scanf ("%d%d",&d,&d);
```

Если пользователь вводит 4,28 `scanf` прочитает 4 и сохранит его в `i`. Когда ищет начало второго числа, `scanf` встречает запятую. Поскольку числа не могут начинаться с запятой, `scanf` немедленно её возвращает.

Запятая и второе число остаются для следующего вызова `scanf`.

Конечно, мы можем легко решить проблему, добавив запятую в строку формата, если мы уверены, что числа всегда будут разделяться запятой:

```
printf ("Введите два числа, разделённые запятой:");  
scanf ("%d,%d",&i,&j);
```

Упражнения

1. Какой вывод будут производить следующие вызовы printf?

- (a) printf ("%6d,%4d", 86, 1040);
- (b) printf ("%12.5e", 30.253);
- (c) printf ("%4f", 83.162);
- (d) printf ("% - 6.2g",.0000009979);

W2. Напишите вызовы printf, которые отображают переменную с плавающей точкой x в следующих форматах:

- a) экспоненциальная запись; выровнено по левому краю в поле размером 8; одна цифра после десятичной точки.
- b) экспоненциальная запись; выровнено по правому краю в поле размером 10; шесть знаков после десятичной точки.
- c) фиксированная десятичная нотация; выровнено по левому краю в поле размером 8; три цифры после десятичной точки.
- d) фиксированная десятичная нотация; выровнено по правому краю в поле размером 6; нет цифр после десятичной точки.

3. Для каждой из следующих пар форматных строк scanf укажите, являются ли две строки эквивалентными. Если нет, покажите, как их можно отличить.

- (a) "%d" в сравнении с " %d"
- (b) "%d-%d-%d" в сравнении с "%d -%d -%d"
- (c) "%f" в сравнении с "%f "
- (d) "%f,%f" в сравнении с "%f, %f"

* 4. Предположим, что мы вызываем scanf следующим образом:

```
scanf ("%d%f%d", &i, &x, &j);
```

Если пользователь вводит 10.3 5 6, какими будут значения *i*, *x*, и *j* после вызова! (Предположим, что *i* и *j* являются `int` переменными, а *x* является переменной с плавающей точкой.)

W*5. Предположим, что мы вызываем `scanf` следующим образом:

```
scanf ("%f%d%f", &x,&i,&y);
```

Если пользователь вводит 12.3 45.6 789, какими будут значения *x*, *i* и *y* после вызова? (Предположим, что *x* и *y* являются переменными типа `float`, а *i* является `int`-переменной.)

6. Покажите, как модифицировать программу `addfrac.c` из раздела 3.2, чтобы пользователю разрешалось вводить дроби, содержащие пробелы до и после каждого символа `/`.

* Помеченные упражнения сложны - правильный ответ обычно не является очевидным. Внимательно прочитайте вопрос, просмотрите соответствующий Раздел в случае необходимости и будьте осторожны!

Проекты по программированию

1. Напишите программу, которая принимает дату от пользователя в форме `mm/dd/yyyy` и затем отображает её в форме `yyyymmdd`:

Введите дату (мм/дд/гггг): 17/02/2011

Вы указали дату 20110217

2. Напишите программу, которая форматирует информацию о продукте, введенную пользователем. Сеанс с программой должен выглядеть так:

Введите номер товара: 553

Введите цену за единицу: 13,5

Введите дату (дд/дд/гггг): 24/10/2010

Товар	Цена	Дата
	за единицу	покупки
583	\$13,50	24.10.2010 г.

Номер товара и дата должны быть выровнены по левому краю; цена за единицу должна быть выровнена по правому краю. Сумма в долларах США может составлять до 9999,99 доллара США. Подсказка: используйте табуляцию для выравнивания столбцов.

3. Книги идентифицируются по международному стандартному номеру книги (ISBN). Номера ISBN, присвоенные после 1 января 2007 года, содержат 13 цифр, расположенных в пяти группах, например 978-0-393-97950-3. (Старые номера ISBN используют 10 цифр). Первая группа (префикс GSI) в настоящее время составляет 978 или 979. Идентификатор группы указывает язык или страну происхождения (например, 0 и 1 используются в англоязычных странах). Код издателя идентифицирует издателя (393 - код W. W. Norton). Номер позиции присваивается издателем для идентификации конкретной книги (97950 - код для этой книги). ISBN заканчивается контрольной цифрой, которая используется для проверки

точности предыдущих цифр. Напишите программу, которая разбивает на группы ISBN, введенный пользователем:

Введите номер ISBN: 978-0-393-97950-3

Префикс GSI: 978

Идентификатор группы: 0

Код издателя: 393

Номер по каталогу: 97950

Контрольная цифра: 3

Примечание. Количество цифр в каждой группе может варьироваться. Не следует предполагать, что группы имеют длины, показанные в этом примере.

Проверьте свою программу с реальными значениями ISBN (обычно на задней обложке книги и на странице авторских прав).

4. Напишите программу, которая предложит пользователю ввести телефонный номер в форме (xxx) xxx-xxxx, а затем отобразит номер в форме xxx.xxx.xxxx :

Введите номер телефона [(xxx) xxx-xxxx]: (404) 817-6900

Вы ввели 404.817.6900

5. Напишите программу, которая попросит пользователя ввести цифры от 1 до 16 (в любом порядке), потом отобразит числа, расположив их по схеме 4 на 4, а затем суммы строк, столбцов и диагоналей:

Введите числа от 1 до 16 в любом порядке:

16 3 2 13 5 10 11 8 9 6 7 12 4 15 14 1

16 3 2 13

5 10 11 8

9 6 7 12

4 15 14 1

Суммы строк: 34 34 34 34

Суммы столбцов: 34 34 34 34

Диагональные суммы: 34 34

Если суммы строк, столбцов и диагоналей одинаковы (как в этом примере), считается, что числа образуют магию.

Магический квадрат, показанный здесь, появляется на гравюре 1514 года художника и математикам Альбрехта Дюрера (обратите внимание, что средние цифры в последней строке указывают дату гравюры).

6. Измените программу `addfrac.c` раздела 3.2 так, чтобы пользователь вводил обе дроби одновременно, разделяя их знаком "плюс":

Введите две дроби, разделённые знаком плюс: $5/6 + 3/4$

Сумма составляет $38/24$

4 Выражения

Нельзя научиться вычислениям с помощью ручного калькулятора, но можно забыть арифметику.

Одной из отличительных характеристик Си является его акцент на выражениях - формулах, которые показывают, как вычислять значение, - а не инструкциях. Простейшими выражениями являются переменные и константы. Переменная представляет собой значение, которое вычисляется при запуске программы; константа представляет собой значение, которое не изменяется. Более сложные выражения применяют операторы к операндам (которые сами являются

выражениями). В выражении $a + (b * c)$ оператор $+$ применяется к операндам a и $(b * c)$, каждый из которых является полноправным выражением.

Операторы являются базовыми инструментами для построения выражений и Си имеет необычно богатый набор операторов. Для начала, Си предоставляет элементарные операторы, которые встречаются в большинстве языков программирования:

Арифметические операторы, включая сложение, вычитание, умножение и деление.

Операторы сравнения для выполнения сравнений, таких как " i больше 0".

Логические операторы для создания условий, таких как " i больше 0, и i меньше 10."

Но Си на этом не останавливается; он продолжает предоставлять десятки других операторов. На самом деле существует так много операторов, что нам нужно будет постепенно их вводить в первых двадцати главах этой книги. Освоение столь большого количества операторов может быть сложной задачей, но оно необходимо, чтобы стать профессионалом в Си. В этой главе мы рассмотрим некоторые из наиболее фундаментальных операторов Си:

арифметические операторы (Раздел 4.1), операторы присваивания (Раздел 4.2), операторы инкремента и декремента (Раздел 4.3). В Разделе 4.1 также объясняются приоритет и ассоциативность операторов, которые важны для выражений, содержащих более одного оператора. В Разделе 4.4 описывается, как вычисляются выражения Си. Наконец, в Разделе 4.5 представлено *выражение как инструкция*,

необычная конструкция, которая позволяет любому выражению служить в качестве инструкции.

4.1 Арифметические операторы

Арифметические операторы это операторы, которые выполняют сложение, вычитание, умножение и деление, являются рабочими лошадками многих языков программирования, включая Си.

Таблица 4.1 показывает арифметические операторы Си.

Унарные	Бинарные	Бинарные
	Аддитивные	Мультипликативные
+ унарный плюс	+ сложение	* умножение
- унарный минус	- вычитание	/ деление
		% остаток

Таблица 4.1 Арифметические операторы Си

Аддитивные и мультипликативные операторы называются бинарными, потому что они требуют два операнда. Унарные операторы требуют один операнд:

`i = + 1;` /* + Используется как унарный оператор */

`j = -1;` /* - используется как унарный оператор */

Унарный `+` оператор ничего не делает: фактически, он даже не существовал в К & Р Си. Он используется в основном для того, чтобы подчеркнуть, что числовая константа положительна. Бинарные операторы, вероятно, выглядят знакомыми.

Единственное, что может быть незнакомым, это оператор остатка `%`. Значение $i \% j$ является остатком, когда i делится на j . Например, значение $10 \% 3$ равно 1, а значение $12 \% 4$ равно 0. Бинарные операторы в таблице 4.1 - за исключением `%` - допускают либо целочисленные, либо плавающие операнды, при этом допускается смешение.

Когда смешанные операнды `int` и `float` смешиваются, результат имеет тип `float`. Таким образом, $9 + 2,5f$ имеет значение 11,5, а $6,7f / 2$ имеет значение 3,35.

Операторы `/` и `%` требуют особого внимания:

- Оператор `/` может приводить к неожиданным результатам. Когда оба его операнда являются целыми числами, оператор `/` усекает результат, отбрасывая дробную часть. Таким образом, значение $1/2$ равно 0, а не 0,5. Оператор `%` требует целочисленные операнды; если любой операнд не является целым числом, программа не будет компилироваться.
- Использование нуля в качестве правого операнда в операциях `/` или `%` приводит к неопределённому поведению.
- Описывать результат, когда операция `/` используется с отрицательными операндами, сложно. В стандарте C89 говорится, что если любой из операндов отрицательный, результат деления может быть округлён вверх или вниз. (Например, значение $-9 / 7$ может быть либо -1, либо -2). Если i или j отрицательны, то знак $i \% j$ в C89 зависит от реализации. (Например, значение $-9 \% 7$ может быть равно -2 или 5). В C99, с другой стороны, результат деления всегда

усекается к нулю (так что $-9/7$ имеет значение -1), а значение $i \% j$ имеет тот же знак, что и i (следовательно, значение $-9 \% 7$ равно -2).

=====

Определяемое реализацией поведение

Термин "Определяемое реализацией" будет возникать достаточно часто, поэтому стоит обсудить этот момент. Стандарт с намеренно оставляет части языка неопределёнными, при том понимании, что "Реализация" - программное обеспечение, необходимое для компиляции, компоновки и выполнения программ на конкретной платформе, будет заполнено деталями. В результате поведение программы может несколько отличаться от одной реализации к другой. Поведение операторов $/$ и $\%$ для отрицательных операндов в C89 является примером определяемого реализацией поведения. Оставление неопределёнными частей языка может показаться странным или даже опасным, но оно отражает философию Си. Одной из целей языка является эффективность, которая часто означает соответствие поведения оборудованию. Некоторые ЦП дают -1 , когда -9 делится на 7 , в то время как другие дают -2 ; стандартная версия C89 отражает этот факт жизни. Лучше избегать написания программ, зависящих от поведения, определяемого реализацией. Если это невозможно, по крайней мере, внимательно прочитайте руководство - стандарт Си требует, чтобы документально фиксировалось поведение, определяемое реализацией.

=====

Приоритет операторов и ассоциативность

Когда выражение содержит более одного оператора, его интерпретация может быть не сразу ясна. Например, $i + j * k$ означает "сложить i и j , а затем умножить результат на k ", или это означает "умножить j и k , а затем добавить i "? Одно из решений этой проблемы - добавить круглые скобки, написать Либо $(i + j) * k$, либо $i + (j * k)$. Как правило, Си допускает использование скобок для группировки во всех выражениях. Что, если мы не используем круглые скобки?

$i + j * k$ как $(i + j) * k$ или $i + (j * k)$?

Как и многие другие языки, Си использует правила приоритета операторов для разрешения этой потенциальной неоднозначности. Арифметические операторы имеют следующий относительный приоритет:

Высший:	+	-	(унарные)
	*	/	%

Самый низкий:	+	-	(бинарные)
---------------	---	---	------------

Операторы, перечисленные в одной строке (такие как $+$ и $-$), имеют одинаковый приоритет. Когда в одном выражении появляются два или более оператора, мы можем определить, как компилятор будет интерпретировать выражение, неоднократно помещая круглые скобки вокруг подвыражений, начиная с операторов высокого приоритета и двигаясь к операторам с низким приоритетом. Результат иллюстрируется следующими примерами:

$i + j * k$ эквивалентно $i + (j * k)$

$-i * -j$ эквивалентно $(-i) * (-j)$

$+ i + j / k$ эквивалентно $(+ i) + (j / k)$

Одних правил приоритета оператора недостаточно, если

выражение содержит два или более операторов с одинаковым уровнем приоритета. В этой ситуации вступает в действие ассоциативность операторов.

Говорят, что оператор лево-ассоциативен, если он группирует слева направо.

Бинарные арифметические операторы (*, /, %, +, -) все лево-ассоциативные, поэтому

$i - j - k$ эквивалентно $(i - j) - k$

$i * j / k$ эквивалентно $(i * j) / k$

Оператор является право-ассоциативным, если он группирует справа налево.

Унарные арифметические операторы (+ и -) являются право-ассоциативными.

Поэтому $- + i$ эквивалентен $-(+ i)$

Правила приоритета и ассоциативности важны для многих языков, но особенно для Си.

Тем не менее, Си имеет так много операторов (почти пятьдесят!), что немногие программисты потрудились запомнить правила приоритета и ассоциативности.

Вместо этого они обращаются к таблице операторов, когда возникают сомнения или просто используют много круглых скобок.

ПРОГРАММА Вычислить контрольную цифру UPC

В течение ряда лет производители товаров, продаваемых в американских и канадских магазинах, устанавливали штрих-

код на каждый продукт. Этот код, известный как Универсальный код продукта (UPC). Идентифицирует производителя и продукт.

Каждый штрих-код представляет собой двенадцатизначное число, которое обычно печатается под штрих-кодом.

Например, следующий штрих-код на пакете французского хлеба "Пепперони Пицца" Стоуффера:



Цифры

0 13800 15173 5

Появляются под штрих-кодом.

Первая цифра идентифицирует тип товара (0 или 7 для большинства предметов, 2 для предметов, которые должны быть взвешены, 3 для товаров и медикаментов, связанных со здоровьем, и 5 для купонов). Первая группа из пяти цифр идентифицирует производителя 03800 - код для Nestlé Отдел замороженных продуктов США).

Вторая группа из пяти цифр идентифицирует продукт (включая размер пакета).

Конечная цифра - это "контрольная цифра", единственная цель которой - помочь выявить ошибку в предыдущих цифрах. Если UPC сканируется неправильно, первые 11 цифр, вероятно, не будут соответствовать последней цифре, а сканер

магазина отклонит весь код. Вот один из методов вычисления контрольной цифры:

Сложите первую, третью, пятую, седьмую, девятую и одиннадцатую цифры.

Сложите вторую, четвертую, шестую, восьмую и десятую цифры.

Умножьте первую сумму на 3 и добавьте её ко второй сумме. Вычтите 1 из общего числа. Вычислите остаток, от деления полученного числа на 10. Вычтите остаток из 9. Используя пример Стоффера, получим $0 + 3 + 0 + 1 + 1 + 3 = 8$ для первой суммы и $1 + 8 + 0 + 5 + 7 = 21$ для второй суммы.

Умножение первой суммы на 3 и добавление второй даёт 45.

Вычитание 1 даёт 44. Остаток после деления на 10 равен 4.

Когда остаток вычитается из 9, результат равен 5. Вот несколько других UPC, если хотите попробовать свои силы при вычислении контрольной цифры (набег на кухонный шкаф для ответа не разрешён):

Jif Creamy Peanut Butter (18 oz.) 0 51500 24128 ?

Ocean Spray Jellied Cranberry Sauce (8 oz.): 0 31200 01005 ?

Ответы отображаются в нижней части страницы. (*

Пропущенные контрольные цифры - 8 (Jif) и 6 (Ocean spray).)

Давайте напишем программу, которая вычисляет контрольную цифру для произвольного UPC. Мы попросим пользователя ввести первые 11 цифр UPC, после чего отобразим соответствующую контрольную цифру. Чтобы избежать путаницы, мы попросим пользователя ввести число в трёх частях: одна цифра слева, первая группа из пяти цифр и вторая группа из пяти цифр. Вот как будет выглядеть сессия с программой:

Введите первую (одиначную) цифру: 0

Введите первую группу из пяти цифр: 13800

Введите вторую группу из пяти цифр: 15173

Контрольная цифра: 5

Вместо того, чтобы читать каждую группу цифр как пятизначное число, мы будем читать её как пять чисел из одной цифры. Чтение чисел как отдельных цифр более удобно; кроме того, нам не придётся беспокоиться о том, что одно из пятизначных чисел слишком велико для хранения в переменной типа `int`. (Некоторые старые компиляторы ограничивают максимальное значение `int`-переменной до 32767.) Для чтения одиночных цифр мы будем использовать `scanf` с спецификацией преобразования `% 1d`, которая соответствует одноразрядному целому числу.

`/* Вычисляет контрольную цифру универсального
продуктового кода */`

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int d, i1, i2, i3, i4, i5, j1, j2, j3, j4, j5, first_sum, second_sum,  
                                              total;
```

```
    printf("Введите первую (одиначную) цифру: ");
```

```
    scanf("%1d", &d);
```

```
    printf("Введите первую группу из пяти цифр: ");
```

```
    scanf("%1d%1d%1d%1d%1d", &i1, &i2, &i3, &i4, &i5);
```

```
    printf("Введите вторую группу из пяти цифр: ");
```

```
    first_sum = d + i2 + i4 + j1 + j3 + j5;
```

```
second_sum = i1 + i3 + i5 + j2 + j4;  
total = 3 * first_aum + second_sum;  
printf("Контрольная цифра: %d\n", 9 - ((total - 1) % 10));  
return 0;  
}
```

Обратите внимание, что выражение $9 - ((total - 1) \% 10)$ могло быть написано $9 - (total - 1) \% 10$, но дополнительные круглые скобки облегчают понимание.

4.2 Операторы присваивания

Как только значение выражения будет вычислено, нам часто нужно будет сохранить его в переменной для последующего использования.

Для этой цели используется оператор Си = (простое присваивание). Для обновления значения, уже сохранённого в переменной, Си предоставляет ассортимент составных операторов присваивания.

Простое присваивание

Эффект присваивания $v = e$ заключается в вычислении выражения e и копировании его значения в v . Как видно из следующих примеров, e может быть константой, переменной или более сложным выражением:

```
i = 5; /* i теперь равно 5 */
j = i; /* j теперь равно 5 */
k = 10 * i + j; /* k теперь равно 55 */
```

Если **v** и **e** не имеют одного и того же типа, то значение **e** преобразуется в тип **v** при присвоении:

```
int i;
float f;
i = 72,99f; /* i теперь 72 */
f = 136; /* f теперь составляет 136,0 */
```

Мы вернёмся к теме преобразования типов позже. Во многих языках программирования присваивание - это инструкция; В Си, однако присваивание является оператором. Как и +.

Другими словами, действие присваивания приводит к результату, точно так же, как сложение двух чисел даёт результат. Значение присвоения $v = e$ - это значение **v** после присвоения.

Таким образом, значение $i = 72.99f$ равно 72 (а не 72.99).

=====

Побочные эффекты

Обычно мы не ожидаем, что операторы модифицируют свои операнды, поскольку операторов в математике нет. Запись $i + j$ не изменяет ни i , ни j ; она просто вычисляет результат сложения i и j . Большинство операторов Си не модифицируют свои операнды, но некоторые делают это. Мы говорим, что эти операторы имеют побочные эффекты, так как они делают больше, чем просто вычисляют значение. Оператор простого присваивания - это первый оператор, который мы видели, который имеет побочные эффекты; Он изменяет свой левый

операнд. Вычисление выражения $i = 0$ приводит к результату 0 и - как побочному эффекту - присваиванию 0-ля i .

=====

Поскольку присваивание является оператором, несколько присваиваний могут быть соединены вместе: $i = j = k = 0$; Оператор = является правым ассоциативным, поэтому это присваивание эквивалентно $i = (j = (k = 0))$; Присвойте 0 сначала k , затем j и, наконец, i . Просмотрите неожиданные результаты в цепных присвоениях в результате преобразования типов:

```
int i;
```

```
float f;
```

```
f = i = 33.3f;
```

i присваивается значение 33, то f присваивается 33.0 (а не 33.3, как вы могли бы подумать). В общем случае форма присваивания $v = e$ допускается везде, где допустимо значение типа v . В следующем примере выражение $j = i$ копирует i в j ; новое значение j затем добавляется к 1, производя новое значение k :

```
i = 1;
```

```
k = 1 + (j = i);
```

```
printf ("%d %d %d \n", i, j, k); /* печатает "1 1 2" */
```

Использование этого оператора присваивания обычно не является хорошей идеей. Во-первых, "встроенные присваивания" могут затруднять чтение программ, а также могут быть источником тонких ошибок, как мы увидим в Разделе 4.4.

lvalue (л-значения)

Большинство операторов Си позволяют их операндам быть переменными, константами или выражениями, содержащими другие операторы. Однако оператору присваивания требуется lvalue в качестве левого операнда.

lvalue (произносится как "L-value") представляет объект, хранящийся в памяти компьютера, а не константу или результат вычисления. Переменные являются lvalue; такие выражения, как 10 или $2 * i$, нет. На данный момент переменные являются единственными lvalue, о которых мы знаем; другие виды lvalues появятся в последующих главах. Поскольку оператор присваивания требует lvalue в качестве своего левого операнда, нельзя помещать любое другое выражение в левую часть выражения присваивания:

```
12 = i; /**НЕПРАВИЛЬНО***/
```

```
i + j = 0; /**НЕПРАВИЛЬНО ***/
```

```
-i = j; /**НЕПРАВИЛЬНО***/
```

Компилятор обнаружит ошибки такого характера, и вы получите сообщение об ошибке, такое как "invalid lvalue in assignment" ("недопустимые л-значения в присвоении").

Составное присваивание

Присвоения, которые используют старое значение переменной для вычисления своего нового значения, являются общепринятыми в программах на Си. Следующий оператор, например, прибавляет 2 к значению, хранящемуся в

i:

$i = i + 2;$

Комбинированные операторы присваивания Си позволяют нам сократить эту инструкцию и подобные ей.

Используя оператор $+=$, мы просто пишем:

$i += 2;$ /* тоже, что $i = i + 2;$ */

Оператор $+=$ добавляет значение правого операнда к переменной слева. Существует девять других составных операторов присваивания, включая следующие:

$- =$ $* =$ $/ =$ $\% =$

(Мы рассмотрим оставшиеся составные операторы присвоения в следующей главе.)

Все составные операторы присваивания работают примерно таким же образом:

$v += e$ добавляет v к e , сохраняя результат в v

$v -= e$ вычитает e из v , сохраняя результат в v

$v *= e$ умножает v на e , сохраняя результат в v

$v /= e$ делит v на e , сохраняя результат в v

$v \% = e$ вычисляет остаток, когда v делится на e , сохраняя результат в v

Заметьте, что я был осторожен, не сказав, что $v += e$ эквивалентно $v = v + e$.

Одна из проблем - это приоритет операторов:

$i *= j + k$ не совпадает с $i = i * j + k$. (Приоритет $+$ выше чем $* =$) Встречаются также редкие случаи, когда $v += e$ отличается от $v = v + e$, поскольку сама v имеет побочный эффект.

Подобные замечания применимы к другим составным операторам присваивания.

Операторы составного присваивания имеют те же свойства, что и оператор $=$. В частности, они право-ассоциативные,

поэтому утверждение

$$i += j += k;$$

означает

$$i += (j += k);$$

При использовании составных операторов присваивания будьте внимательны, чтобы не переставить два символа, которые составляют оператор. Перестановка символов может привести к выражению, приемлемому для компилятора, но не имеющему предполагаемого значения.

Например, если вы намеревались написать $i += j$, но набрали $i =+ j$, программа будет компилироваться. К сожалению, последнее выражение эквивалентно $i = (+ j)$, которое просто копирует значение j в i .

4.3 Операторы инкремента и декремента

Две из наиболее распространённых операций над переменной - "инкремент" (добавление 1) и "декремент" (вычитание 1).

Мы можем, конечно, выполнить эти задачи, написав

$$i = i + 1;$$
$$j = j - 1;$$

Составные операторы присваивания позволяют нам немного сгустить эти утверждения:

$$i += 1;$$
$$j -= 1;$$

Но Си позволяет ещё более сократить инкремент и декремент,

используя операторы ++ (инкремент) и -- (декремент). На первый взгляд, операторы инкремента и декремента являются простыми:

++ добавляет 1 к своему операнду, тогда как -- вычитает 1. К сожалению, эта простота вводит в заблуждение - операторы инкремента и декремента могут быть каверзными в использовании. Одно осложнение состоит в том, что ++ и -- могут использоваться в качестве префиксных операторов (например, ++ i и --i) или постфиксных операторов (i ++ и i--). Правильность программы может зависеть от выбора надлежащей версии. Другое осложнение состоит в том, что, как и операторы присваивания, ++ и -- имеют побочные эффекты: они изменяют значения своих операндов. Вычисление выражения ++ i ("префиксный инкремент") даёт i + 1 и - как побочный эффект - увеличивает i:

```
i = 1;
printf("i равно %d\n", ++i); /* печатает "i равно 2" */
printf("i равно %d\n", i); /* печатает "i равно 2" */
```

Вычисление выражения i ++ ("постфиксный инкремент") даёт результат 1, а затем увеличивает i:

```
i = 1;
printf("i равно %d\n", i++); /* печатает "i равно 1" */
printf("i равно %d\n", i); /* печатает "i равно 2" */
```

Первый вызов printf показывает исходное значение *i*, прежде чем она (переменная) будет увеличена.

Второй вызов printf показывает новое значение. Как показано в этих примерах, ++ *i* означает "немедленное приращение *i*", в то время как *i* ++ означает "использовать старое значение *i* на данный момент, но прирастить *i* позже".

На сколько позже? В стандарте Си не указано точное время, но можно с уверенностью предположить, что *i* будет увеличиваться до выполнения следующей инструкции.

Оператор -- имеет аналогичные свойства:

```
i = 1;
printf ("i равно %d \n", -- i); /* Выводит "i равно 0" */
```

```
printf ("i равно %d \n", i); /* Печатает "i равно 0" */
```

```
i = 1;
```

```
printf ("i равно %d \n", i--); /* Печатает "i равно 1" */
```

```
printf ("i равно %d \n", i); /* Печатает "i равно 0" */
```

Когда ++ или -- используется более одного раза в одном выражении, результат часто бывает трудно понять.

Рассмотрим следующие инструкции:

```
i = 1;
```

```
j = 2;
```

```
k = ++ i + j ++;
```

Каковы значения *i*, *j* и *k* после выполнения этих инструкций?

Так как *i* увеличивается до того, как используется его значение, но *j* увеличивается с шагом после его использования, последний оператор эквивалентен

```
i = i + 1;
```

```
k = i + j;
```

```
j = j + 1;
```

Поэтому конечные значения i , j и k равны 2, 3 и 4 соответственно. В отличие от этого, выполнение инструкций

```
i = 1;
j = 2;
k = i ++ + j ++;
```

будет давать i , j и k значения 2, 3 и 3 соответственно. Для записи постфиксные версии $++$ и $--$ имеют более высокий приоритет, чем унарные плюс и минус и являются лево-ассоциативными. Префиксные версии имеют тот же приоритет, что и унарные плюс и минус и являются право-ассоциативными.

4.4 Вычисление выражений

В таблице 4.2 приведены операторы, которые мы видели на настоящий момент. (Приложение А имеет подобную таблицу, которая отображает все операторы.) Первый столбец показывает приоритет каждого оператора относительно других операторов в таблице (самый высокий приоритет - 1, самый низкий - 5). В последнем столбце показана ассоциативность каждого оператора.

Приоритет	Имя операции(ий)	Символ(ы)	Ассоциативность
1	инкремент (постфикс)	++	левая

	декремент (постфикс)	--	
2	инкремент (префикс)	++	правая
	декремент (префикс)	—	
	унарный плюс	+	
	унарный минус	-	
3	мультипликат ивные	* / %	левая
4	аддитивные	+ -	левая
5	присваивания	= *= /= %= += -=	правая

Таблица 4.2 Частичный список операторов Си

Таблица 4.2 (или её более обширный родственник в Приложении А) имеет множество применений. Давайте

рассмотрим одно из них. Предположим, что мы просматриваем сложное выражение, такое как

$a = b += c++ - d + --e / -f$, когда читаем чью-то программу. Это выражение было бы легче понять, если бы были скобки, чтобы показать, как выражение строится из подвыражений.

С помощью таблицы 4.2 добавление круглых скобок к выражению выполняется просто: после изучения выражения для поиска оператора с наивысшим приоритетом мы помещаем скобки вокруг оператора и его операндов, указывая, что с этого момента он должен рассматриваться как один операнд. Затем мы повторяем процесс, пока выражение не будет полностью заключено в круглые скобки. В нашем примере оператор с наивысшим приоритетом - это ++, используемый здесь как постфиксный оператор, поэтому мы помещаем скобки вокруг ++ и его операнда:

$a = b += (c++) - d + --e / -f$

Теперь мы определим в выражении префиксный оператор -- и оператор унарный минус - (оба имеют приоритет 2):

$a = b += (c++) - d + (--e) / (-f)$

Обратите внимание, что у другого знака минус есть операнд слева от него, поэтому он должен быть оператором вычитания, а не унарным отрицательным оператором. Далее мы обратим внимание на оператор / (приоритет 3):

$a = b += (c++) - d + ((--e) / (-f))$

Выражение содержит два оператора с приоритетом 4, вычитание и сложение. Всякий раз, когда два оператора с одинаковым приоритетом смежны с операндом, мы должны быть осторожны с ассоциативностью. В нашем примере, - и + смежны с **d**, поэтому применяются правила ассоциативности. Операторы - и + группируются слева направо, поэтому

скобками сначала охватывается вычитание, а затем сложение:

$$a = b += (((c++) - d) + ((--e) / (-f)))$$

Единственными оставшимися операторами являются = и +=.

Оба оператора смежны с **b**, поэтому мы должны учитывать ассоциативность. Операторы присваивания группируются справа налево, поэтому скобки идут сначала вокруг

выражения +=, а затем вокруг выражения =

$$(a = (b += (((c++) - d) + ((--e) / (-f)))))$$

Выражение теперь полностью заключено в круглые скобки.

Порядок вычисления подвыражений

Правила приоритета и ассоциативности операторов позволяют нам разбивать любое выражение на подвыражения - чтобы однозначно определить, как будут расставлены скобки, если выражение было полностью заключено в круглые скобки. Как ни парадоксально, эти правила не всегда позволяют нам определять значение выражения, которое может зависеть от порядка, в котором вычисляются его подвыражения. Си не определяет порядок, в котором вычисляются подвыражения (за исключением подвыражений, включающих логические операторы И, ИЛИ, условный и запятая). Таким образом, в выражении $(a + b) * (c - d)$, мы не знаем, будет ли $(a + b)$ вычисляться до $(c - d)$. Большинство выражений имеют одинаковое значение независимо от порядка, в котором вычисляются их подвыражения.

Однако это может быть не так, когда подвыражение изменяет один из своих операндов. Рассмотрим следующий пример:

$a = 5;$

$c = (b = a + 2) - (a = 1);$

Эффект выполнения второй инструкции не определён; Стандарт Си не говорит, что произойдёт. Для большинства компиляторов значение c будет равно 6 или 2.

Если сначала вычисляется подвыражение $(b = a + 2)$, b присваивается значение 7, а c присваивается 6. Но если сначала вычисляется $(a = 1)$, b присваивается 3, а c присваивается 2.

Избегайте написания выражений, которые обращаются к значению переменной, а также изменяют переменную в другом месте выражения. Выражение $(b = a + 2) - (a = 1)$ обращается к значению a (чтобы вычислить $a + 2$), а также изменяет значение a (присвоив a 1). Некоторые компиляторы могут выдавать предупреждающее сообщение, такое как "операция над 'a' может быть не определена", когда они сталкиваются с таким выражением.

Чтобы предотвратить проблемы, рекомендуется избегать использования операторов присваивания в подвыражениях; вместо этого используйте ряд отдельных присваиваний. Например, приведённые выше инструкции можно было бы переписать как:

$a = 5;$

$b = a + 2;$

$a = 1;$

$c = b - a;$

Значение c всегда будет равно 6 после выполнения этих инструкций.

Помимо операторов присваивания, единственными операторами, которые изменяют свои операнды, являются инкремент и декремент. При использовании этих операторов будьте внимательны, чтобы ваши выражения не зависели от конкретного порядка вычислений. В следующем примере, j может быть присвоено одно из двух значений:

```
i = 2;
j = i * i ++;
```

Естественно предположить, что j присваивается значение 4. Тем не менее, эффект выполнения оператора не определён, и j также может быть присвоено 6. Вот сценарий:

(1) Выбирается второй операнд (исходное значение i), затем i увеличивается.

(2) Выбирается первый операнд (новое значение i).

(3) Новые и старые значения i умножаются, что даёт 6.

(у меня gcc даёт 6, а clang с предупреждением "неупорядоченная модификация и доступ к i" даёт 4.)

"Выбор" переменной означает извлечение значения переменной из памяти. Более позднее изменение этой переменной не повлияет на полученное значение, которое обычно хранится в специальном месте (известном как регистр) внутри CPU.

=====

Неопределённое поведение

Согласно стандарту Си, инструкции, такие как

$c = (b = a + 2) - (a = 1)$; and $j = i * i++$; вызывают неопределённое поведение, которое отличается от поведения, определённого реализацией (см. Раздел 4.1). Когда программа вторгается в область неопределённого поведения, ситуация становится непредсказуемой. Программа может вести себя по-разному при компиляции разными компиляторами. Но это не единственное, что может произойти. В первую очередь, программа может не компилироваться, если она компилируется, то может не запускаться, если запустится, то может аварийно завершиться, может вести себя хаотично, или производить бессмысленные результаты. Иными словами, неопределённого поведения следует избегать, как чумы.

=====

4.5 Инструкции-выражения

Си имеет необычное правило: любое выражение может использоваться как инструкция. То есть любое выражение, независимо от его типа или того, что оно вычисляет, можно превратить в инструкцию, добавив точку с запятой.

Например, мы можем превратить выражение $++i$ в инструкцию: $++i$;

Когда выполняется эта инструкция, i сначала увеличивается, затем выбирается новое значение i (как если бы оно использовалось во вменяющем выражении). Однако, поскольку $++i$ не является частью более крупного выражения, его значение отбрасывается и выполняется следующая инструкция. (Разумеется, изменение i не подлежит отмене.) Поскольку его значение отбрасывается, мало смысла

использовать выражение в качестве инструкции, если выражение не имеет побочного эффекта. Давайте рассмотрим три примера. В первом примере 1 сохраняется в i , затем новое значение i выбирается, но не используется:

$i = 1;$

во втором примере значение i выбирается, но не используется; однако i в дальнейшем уменьшается:

$i--;$

в третьем примере значение выражения $i * j - 1$ вычисляется и затем отбрасывается: $i * j - 1$; Так как i и j не изменяются, эта инструкция не имеет никакого эффекта и, следовательно, не имеет никакой цели. Соскальзывание пальца может легко создать инструкцию - выражение "do-nothing" ("ничего не делающую"). Например, вместо ввода $i = i$ мы могли бы случайно ввести $i + i$ (этот вид ошибки более распространён, чем можно ожидать, поскольку символы $=$ и $+$ обычно занимают одну и ту же клавишу). Некоторые компиляторы могут обнаруживать бессмысленные выражения; вы получите предупреждение типа "statement with no effect" ("инструкция без выполнения").

Вопросы и ответы

Вопрос: Я замечаю, что Си не имеет оператора возведения в степень. Как я могу возвести число в степень?

Ответ: Возведение целого числа в маленькую положительную целую степень лучше всего производить путём повторного умножения ($i * i * i$ - куб). Чтобы возвести число в нецелую степень, вызовите функцию `pow`.

Вопрос: Я хочу применить оператор % к операнду с плавающей точкой, но моя программа не компилируется. Что я могу сделать?

Ответ: Оператор % требует целочисленные операнды. Попробуйте вместо этого функцию fmod.

Вопрос: Почему правила использования операторов / и % с отрицательными операндами сложны?

Ответ: Правила не так сложны, как могут показаться. И в C89 и в C99 цель состоит в том, чтобы значение

$(a / b) * b + a \% b$ всегда было равно **a**. (И действительно, оба стандарта гарантируют, что это так, при условии, что значение a / b является "представимым").

Проблема состоит в том, что для a / b и $a \% b$ есть два способа, удовлетворить этому равенству, если либо a , либо b отрицательно, как показано в C89, где либо $-9 / 7$ равно -1 , и $-9 \% 7$ равно -2 , либо $-9 / 7$ равно -2 , а $-9 \% 7$ равно 5 . В первом случае $(-9 / 7) * 7 + -9 \% 7$ имеет значение

$-1 \times 7 + -2 = -9$ и во втором случае $(-9 / 7) * 7 + -9 \% 7$ имеет значение $-2 \times 7 + 5 = -9$.

К тому времени, когда C99 обкатывался, большинство процессоров были сконструированы для усечения результата деления в сторону нуля, поэтому это было записано в стандарт как единственный допустимый результат.

Вопрос: Если Си имеет lvalues, имеет ли он также rvalues?

Ответ: Да, действительно. lvalue - это выражение, которое

может появляться в левой части присваивания; `rvalue` - это выражение, которое может появляться на правой стороне.

Таким образом, `rvalue` может быть переменной, константой или более сложным выражением. В этой книге, как и в стандарте Си, мы будем использовать термин "выражение" вместо "`rvalue`".

Вопрос: Вы сказали, что `v += e` не эквивалентно `v = v + e`, если `v` имеет побочный эффект. Вы можете объяснить?

Ответ: Вычисление `v += e` заставляет `v` вычислиться только один раз; вычисление `v = v + e` заставляет `v` вычислиться дважды. Любой побочный эффект, вызванный вычислением `v`, будет иметь место дважды в последнем случае. В следующем примере `i` увеличивается один раз:

```
a[i++] += 2;
```

Если мы используем `=` вместо `+=`, вот как будет выглядеть выражение:

```
a[i++] = a[i++] + 2;
```

Значение `i` изменяется, а также используется в другом месте инструкции, поэтому эффект выполнения инструкции не определён. Вполне вероятно, что `i` будет увеличен в два раза, но мы не можем с уверенностью сказать, что произойдёт.

Вопрос: Почему Си предоставляет операторы `++` и `--`?

Являются ли они быстрее, чем другие способы увеличения и уменьшения, или они просто более удобны?

Ответ: Си унаследовал `++` и `--` от более раннего языка В, созданного Кеном Томпсоном.

Томпсон, по-видимому, создал эти операторы, потому что его компилятор В мог сгенерировать более компактную трансляцию для `++ i`, чем для `i = i + 1`. Эти операторы стали

укоренившейся частью Си (в самом деле многие из самых известных идиом Си полагаются на них). С современными компиляторами использование ++ и -- не делает скомпилированную программу меньшей или более быстрой; Продолжающаяся популярность этих операторов объясняется главным образом их краткостью и удобством.

Вопрос: Работают ли ++ и -- с переменными типа float?

Ответ: Да. Операции инкремента и декремента могут быть применены к числам с плавающей точкой также как к целым числам.

На практике, однако, довольно редко можно встретить инкрементирование или декрементирование значения переменной float.

Вопрос: При использовании постфиксной версии ++ или --, когда именно выполняется инкремент или декремент?

Ответ: Отличный вопрос.

К сожалению, ответить на него также сложно. В стандарте Си вводится понятие "точка следования" и говорится, что "обновление сохранённого значения операнда должно происходить между предыдущей и следующей точкой следования". (*"Точка следования определяет любую точку выполнения компьютерной программы, при которой гарантируется, что все побочные эффекты предыдущих вычислений будут выполнены, и никаких побочных эффектов от последующих вычислений ещё не было выполнено."*)

Существуют различные виды точек следования в Си:

конец инструкции - выражения является одним из примеров. К концу инструкции-выражения должны быть выполнены все инкременты и декременты внутри инструкции; следующая инструкция не может начать выполнение до тех пор, пока это условие не будет выполнено.

Некоторые операторы, с которыми мы встретимся в последующих главах - логические `&&`, логические `||`, условные операторы (тернарные) и запятые, также задают точки следования.

Если аргумент является выражением, содержащим оператор `++` или `--`, то прежде чем может произойти вызов, должен осуществиться инкремент или декремент.

(ещё точки следования:

- контролирующее выражения инструкции выбора (*if* или *switch*) ;
- управляющее выражение *while* или *do* ;
- каждое из (опциональных) выражений инструкции *for* ;
- (опциональное) выражение в инструкции *return*.)
- после действий, связанных с каждым форматным спецификатором преобразования функции ввода / вывода.)

Вопрос: Что вы подразумеваете, когда вы говорите, что значение выражения отбрасывается?

Ответ: По определению, выражение представляет значение.

Если *i* имеет значение 5, например, то вычисление *i + 1* даёт значение 6. Давайте обратим *i + 1* в инструкцию, поставив точку с запятой после *i+1*; При выполнении этой инструкции, вычисляется значение *i + 1*. Так как нам не удалось сохранить

это значение или, по крайней мере, использовать его каким-либо образом, оно потеряно.

Вопрос: Но как насчёт таких утверждений, как $i = 1$;
Я не вижу, что здесь отбрасывается.

Ответ: Не забывайте, что `=` является оператором в Си и производит значение точно так же, как любой другой оператор. Присваивание $i = 1$; присваивает 1 i . Значение всего выражения равно 1, которое отбрасывается. Отказ от значения выражения не является большой потерей, поскольку причиной написания инструкции в первую очередь было изменение i .

Упражнения

1. Покажите результат, полученный каждым из следующих фрагментов программы. Предположим, что i , j и k являются `int` переменными.

(a) $i = 5$; $j = 3$;

`printf("%d %d", i / j, i % j);`

(b) $i = 2$; $j = 3$;

`printf ("%d ", (i + 10) % j);`

(c) $i=7$; $j=8$; $k =9$;

`printf("%d", (i + 10) % k / j);`

(d) $i=1$; $j=2$; $k=3$;


```
printf("%d", (i + 5) % (j + 2) / k);
```

* 2. Если i и j - положительные целые числа, то $(-i) / j$ всегда имеют то же значение, что и $-(i / j)$? Обосновать ответ.

3. Каково значение каждого из следующих выражений в C89! (Укажите все возможные значения, если выражение может иметь более одного значения.)

- (a) $8 / 5$
- (b) $-8 / 5$
- (c) $8 / -5$
- (d) $-8 / -5$

4. Повторите упражнение 3 для C99.

5. Каково значение каждого из следующих выражений в C89? (Укажите все возможные значения, если выражение `if` может иметь более одного значения.)

- (a) $8 \% 5$
- (b) $-8 \% 5$
- (c) $8 \% -5$
- (d) $-8 \% -5$

6. Повторите упражнение 5 для C99.

7. Алгоритм вычисления контрольной цифры UPC завершается следующими шагами:
Вычтите 1 из общего числа.

Вычислите остаток деления скорректированной суммы на 10.

Вычтите остаток от 9.

Заманчиво попытаться упростить алгоритм, используя вместо этого следующие шаги:

Вычислите остаток, деления суммы на 10.

Вычтите остаток из 10.

Почему эта техника не работает?

8. Будет ли программа `urs.c` работать, если выражение

$9 - ((total - 1) \% 10)$ будет заменено $(10 - (total \% 10)) \% 10$?

9. Покажите результат, полученный каждым из следующих фрагментов программы. Предположим, что `i`, `j` и `k` являются `int` переменными.

(a) `i = 7; j = 8; i *= j + 1;`

`printf("%d %d", i, j);`

(b) `i = j = k = 1; i += j += k;`

`printf("%d %d %d", i, j, k);`

(c) `i = 1; j = 2; k = 3; i -= j -= k;`

`printf("%d %d %d", i, j, k);`

(d) `i = 2; j = 1; k = 0; i *= j *= k;`

`printf("%d %d %d", i, j, k);`

10. Покажите результат, полученный каждым из следующих фрагментов программы. Предположим, что `i` и `j` являются `int` переменными.

(a) `i = 6; j = i += i;`

183

```
printf("%d %d", i, j);
```

(b) `i = 5; j = (i -= 2) + 1;`

```
printf("%d %d", i, j);
```

(c) `i = 7; j=6 + (i = 2.5);`

```
printf("%d %d", i, j);
```

(d) `i = 2; j = 8; j = ( i = 6 ) + ( j =3 );`

```
printf("%d %d", i, j);
```

11. Покажите результат, полученный каждым из следующих фрагментов программы. Предположим, что `i`, `j` и `k` являются `int` переменными.

(a) `i = 1;`

```
printf("%d ", i++ -1);
```

```
printf("%d", i);
```

(b) `i = 10; j = 5;`

```
printf("%d ", i++ - ++j);
```

```
printf("%d %d", i, j);
```

(c) `i = 7; j = 8;`

```
printf("%d", i++ - --j);
```

```
printf("%d %d", i, j);
```

(d) `i = 3; j = 4; k = 5;`

```
printf("%d", i++ - j++ + --k);
```

```
printf("%d %d %d", i, j, k);
```

12. Покажите результат, полученный каждым из следующих фрагментов программы. Предположим, что *i* и *j* являются `int` переменными.

(a) `i = 5; j = ++i * 3 - 2;`

`printf("%d %d", i, j);`

(b) `i = 5; j = 3 - 2 * i++;`

`printf("%d %d", i, j);`

(c) `i = 7; j = 3 * i-- + 2;`

`printf("%d %d", i, j);`

(d) `i = 7; j = 3 + --i * 2;`

`printf("%d %d", i, j);`

13. Только одно из выражений `++i` и `i++` в точности совпадает с `(i += 1)`; какое? Обосновать ответ.

14. Расставьте круглые скобки, чтобы показать, как компилятор Си интерпретирует каждое из следующих выражений.

(a) `a * b - c * d + e`

(b) `a / b % c / d`

(c) `-a - b + c - +d`

(d) `a * --b / c - d`

15. Дайте значения *i* и *j* после выполнения каждой из следующих инструкций - выражений. (Предположим, что изначально *i* имеет значение 1, а *j* имеет значение 2.)

(a) `i += j;`

- (b) `i--;`
- (c) `i * j / i;`
- (d) `i % ++j;`

Проекты по программированию

1. Напишите программу, которая попросит пользователя ввести двузначное число, а затем напечатает число с обратным порядком цифр (реверс).

Сеанс с программой должен иметь следующий вид:

Введите двузначное число: 28

Реверс: 82

Прочитайте число, используя `%d`, затем разбейте его на две цифры.

Подсказка: если `n` - целое число, то `n % 10` является последней цифрой в `n`, а `n / 10` — это `n` с удалением последней цифры.

2. Расширьте программу в Проекте по программированию 1, чтобы обрабатывать трёхзначные числа.

3. Перепишите программу Проекта по программированию 2 так, чтобы она печатала обращение трёхзначного числа, не используя арифметику для разбиения числа на цифры.

Подсказка: см. Программу `urs.c` в Разделе 4.1.

4. Напишите программу, которая читает целое число, введённое пользователем, и отображает его в восьмеричной системе (основание 8):

Введите число от 0 до 32767: 1953

В восьмеричном системе ваше число: 03641

Вывод должен отображаться с использованием пяти цифр,

даже если число цифр меньше.

Подсказка: Целая часть переводится из десятичной системы счисления в другую систему счисления с помощью последовательного деления целой части числа на основание системы счисления до получения целого остатка, меньшего основания системы счисления. Пример: перевести число 273_{10} в восьмиричную систему счисления.

Решение: $273 / 8 = 34$ и остаток 1, $34 / 8 = 4$ и остаток 2, 4 меньше 8, поэтому вычисления завершены. Запись из остатков будет иметь следующий вид: 421

(printf может отображать числа по основанию 8, как мы увидим в Главе 7, так что на самом деле проще написать эту программу).

5. Перепишите урс. с в Разделе 4.1, так чтобы пользователь вводил 11 цифр за один раз вместо ввода одной цифры, затем пяти цифр, а затем ещё пяти цифр.

Введите первые 11 цифр UPC: 01380015173

Контрольная цифра: 5

6. В европейских странах вместо 12-значного универсального кода продукта (UPC), применяемого в Северной Америке, используется 13-значный код, известный как европейский номер товара (EAN). Каждый EAN заканчивается контрольной цифрой, как и UPC. Методика расчета контрольной цифры также аналогична.

Сложите вторую, четвертую, шестую, восьмую, десятую и двенадцатую цифры.

Сложите первую, третью, пятую, седьмую, девятую и одиннадцатую цифры.

Умножьте первую сумму на 3 и сложите её со второй суммой.

Вычтите 1 из общей суммы.

Найдите остаток, от деления скорректированной суммы на 10. Вычтите остаток из 9.

Например, рассмотрим Giilllioglu Turkish Delight Pistachio & Cocomut, который имеет EAN 8691484260008.

Первая сумма $6 + 1 + 8 + 2 + 0 + 0 = 17$,

а вторая сумма равна $8 + 9 + 4 + 4 + 6 + 0 = 31$.

Умножение первой суммы на 3 и сложение со второй даёт 82.

Вычитание 1 даёт 81. Остаток при делении на 10 равен 1.

Когда остаток вычитается из 9, результат равен 8, что соответствует последней цифре исходного кода. Ваша задача - изменить программу ирс.с Раздела 4.1 так, чтобы она вычисляла контрольную цифру для EAN. Пользователь будет вводить первые 12 цифр EAN как одно число:

Введите первые 12 цифр EAN: 869148426000

Контрольная цифра: 8

5 Инструкции выбора

Работу программистов следует оценивать не по их изобретательности и логике, а по полноте анализа каждой ситуации.

Хотя Си имеет много операторов, у него относительно мало инструкций. Мы столкнулись только с двумя: инструкция return и выражение- инструкция. Большинство оставшихся инструкций Си делятся на три категории, в зависимости от того, как они влияют на порядок выполнения инструкций:

- Инструкции выбора. Инструкции `if` и `switch` позволяют программе выбирать конкретный путь выполнения из набора альтернатив.
- Итерационные инструкции. Инструкции `while`, `do` и `for` поддерживают итерацию (цикл).
- Инструкции перехода. Инструкции `break`, `continue` и `goto` вызывают безусловный переход в другое место в программе. (Инструкция `return` также относится к этой категории.)

Из других инструкций в Си остаются только составные инструкции, которые объединяют в себе несколько инструкций в одну инструкцию, и нулевая инструкция, которая не выполняет никаких действий.

В этой главе обсуждаются инструкции выбора и составные инструкции (глава 6 охватывает инструкции итерации, инструкции перехода и нулевую инструкцию). Прежде чем мы сможем написать инструкции `if`, нам понадобятся логические выражения: условия, которые могут проверять инструкции `if`. Раздел 5.1 объясняет, как логические выражения строятся из реляционных операторов (`<`, `<=`, `>` и `>=`), операторов равенства (`==` и `!=`) и логических операторов (`&&`, `||` и `!`). Раздел 5.2 охватывает инструкцию `if` и составные инструкции, а также условные операторы (`? :`) (*он же тернарный оператор (от латинского слова *ternarius* — тройной) — это оператор, состоящий из двух символов `?` и `:`, который формирует условную тернарную операцию, возвращающую свой второй или третий операнд в зависимости от выполнения или невыполнения определённого условия, или*

другими словами от результата логического выражения.) В Разделе 5.3 описывается инструкция switch.

5.1 Логические выражения

Некоторые из инструкций Си, включая инструкцию if, должны проверять значение выражения, чтобы увидеть, истинно оно или ложно. Например, для инструкции if может потребоваться проверить выражение $i < j$; значение "истина" будет указывать, что i меньше, чем j .

Во многих языках программирования такое выражение, как $i < j$, имело бы специальный "булев" или "логический" тип. Такой тип имел бы только два значения: false (ложь) и true (истина). Однако в Си такое сравнение, как $i < j$ даёт целое число: либо 0 (false), либо 1 (true). Имея это в виду, давайте посмотрим на операторы, которые используются для построения логических выражений.

Реляционные операторы

Операторы отношения Си (Таблица 5.1) соответствуют математическим операторам $<$, $>$, $\leq$ и $\geq$, за исключением того, что они производят 0 (ложь) или 1 (истина) при использовании в выражениях.

Например, значение $10 < 11$ равно 1; Значение $11 < 10$ равно 0

Символ	Что означает
<	Меньше чем
>	Больше чем
<=	Меньше или равно
>=	Больше или равно

Операторы отношения могут использоваться для сравнения целых чисел и чисел с плавающей точкой, с допустимыми операндами смешанных типов. Таким образом, $1 < 2,5$ имеет значение 1, а $5,6 < 4$ имеет значение 0. Приоритет реляционных операторов ниже, чем у арифметических операторов: например, $i + j < k - 1$ означает $(i + j) < (k - 1)$. Операторы отношения являются лево-ассоциативными. Выражение $i < j < k$ является допустимым в Си, но не имеет значения, которое можно было бы ожидать. Поскольку оператор $<$ лево-ассоциативен, это выражение эквивалентно $(i < j) < k$. Другими словами, выражение сначала проверяет, меньше ли i , чем j ; 1 или 0, произведённые этим сравнением, затем сравнивают с k . Выражение не проверяет, находится ли j между i и k . (Мы увидим далее в этом разделе, что правильным выражением будет $i < j \ \&\& \ j < k$.)

Операторы равенства

Хотя операторы отношения обозначаются теми же символами, что и во многих других языках программирования, операторы равенства имеют уникальный внешний вид (Таблица 5.2).

Оператор "равно"- это два смежных символа ==, а не один, так как один символ = представляет оператор присваивания. Оператор "не равно"также состоит из двух символов:! И =.

Символ	Что означает
==	равно
!=	Не равно

Таблица 5.2 Операторы равенства

Как и операторы отношения, операторы равенства являются лево-ассоциативными и дают в качестве результата 0 (ложь) или 1 (истина). Тем не менее, операторы равенства имеют меньший приоритет, чем операторы отношения. Например, выражение

$$i < j == j < k$$

эквивалентно

$$(i < j) == (j < k)$$

которое истинно, если $i < j$ и $j < k$ оба истинны или оба ложны. Знающие программисты иногда используют тот факт, что операторы отношения и равенства возвращают целочисленные значения. Например, значение выражения $(i > j) + (i == j)$ равно либо 0, либо 1, либо 2, в зависимости от того i меньше чем, больше чем или равно j , соответственно. Подобное коварное кодирование, как правило, не является хорошей идеей; это делает программы трудными для понимания.

Логические операторы

Более сложные логические выражения могут быть построены из более простых, используя логические операторы: `and`, `or` и `not` (Таблица 5.3). Оператор `!` является унарным, а `&&` и `||` являются двоичными.

Символ	Что означает
<code>!</code>	Логическое отрицание
<code>&&</code>	Логическое и (<code>and</code>)
<code> </code>	Логическое или (<code>or</code>)

Таблица 5.3 Логические операторы

Логические операторы дают в качестве результата 0 или 1. Часто операнды будут иметь значения 0 или 1, но это не является обязательным требованием; логические операторы рассматривают ненулевой операнд как истинное значение, а любой нулевой операнд - как ложное значение. Логические операторы ведут себя следующим образом:

`!expr` имеет значение 1, если `expr` имеет значение 0.

`expr1 && expr2` имеет значение 1, если значения `expr1` и `expr2` оба отличны от нуля.

`expr1 || expr2` имеет значение 1, если либо `expr1`, либо `expr2` (или оба) имеет ненулевое значение. Во всех остальных случаях эти операторы производят значение 0. Оба оператора `&&` и `||` выполняют "короткое замыкание" в вычислении своих

операндов, то есть эти операторы сначала вычисляют левый операнд, а затем правый операнд. Если значение выражения может быть выведено из значения только левого операнда, тогда правый операнд не вычисляется. Рассмотрим следующее выражение:

$(i \neq 0) \&\& (j / i > 0)$

Чтобы найти значение этого выражения, мы должны сначала вычислить $(i \neq 0)$. Если i не равно 0, тогда нам нужно вычислить $(j / i > 0)$ чтобы определить, истинно или ложно целое выражение.

Однако, если i равно 0, тогда все выражение должно быть ложным, поэтому нет необходимости вычислять $(j / i > 0)$. Преимущество коротко-замкнутого вычисления очевидно - без него вычисление выражения вызвало бы деление на ноль. Будьте осторожны с побочными эффектами в логических выражениях. Благодаря характеру короткого замыкания операторов $\&\&$ и $\|$, побочные эффекты в операндах могут не всегда встречаться. Рассмотрим следующее выражение:

$i > 0 \&\& ++j > 0$

Хотя j , по-видимому, увеличивается как побочный эффект вычисления выражения, это не всегда так. Если $i > 0$ ложно, то $++j > 0$ не вычисляется, поэтому j не увеличивается. Проблема может быть решена путём изменения условия на $++j > 0 \&\& i > 0$ или, что ещё лучше, путём увеличения j отдельно.

Оператор $!$ имеет тот же приоритет, что и унарные операторы "плюс" и "минус". приоритет $\&\&$ и $\|$ ниже, чем у операторов отношения и равенства; например, $i < j \&\& k == m$ означает $(i < j) \&\& (k == m)$. Оператор $!$ является право-ассоциативным; $\&\&$ и $\|$ лево-ассоциативны.

5.2 Инструкция if

Инструкция if позволяет программе выбирать между двумя альтернативами, проверяя значение выражения. В своей простейшей форме инструкция if имеет форму

if (*выражение*) инструкция

Обратите внимание, что круглые скобки вокруг *выражение* являются обязательными; они являются частью инструкции if, а не частью выражения. Также обратите внимание, что слово then не появляется после круглых скобок, как это было бы в некоторых языках программирования.

Когда выполняется инструкция if, вычисляется выражение в круглых скобках; если значение выражения отлично от нуля - что Си интерпретирует как true (истина) — выполняется инструкция после круглых скобок. Вот пример:

```
if (line_num == MAX_LINES)
    line_num = 0;
```

Инструкция `line_num = 0;` выполняется, если условие `line_num == MAX_LINES` истинно (имеет ненулевое значение). Не путайте `==` (равенство) с `=` (присваивание). Инструкция `if (i == 0)` проверяет, равна ли *i* нулю. А инструкция `if (i = 0)...` присваивает 0 *i*, а затем проверяет, не является ли результат ненулевым. В этом случае тест всегда терпит неудачу.

Перепутывание `==` с `=` является, пожалуй, самой распространённой ошибкой программирования на Си, вероятно, потому что `=` означает "равно" в математике (и в некоторых языках программирования). Некоторые компиляторы выдают предупреждение, если замечают `=`, где обычно появляется `==`.

Выражение в инструкции `if` проверяет, попадает ли переменная в диапазон значений. Например, чтобы проверить, будет ли $0 \leq i \leq n$, напомним

идиома `if ( 0 <= i && i < n )....`

Чтобы проверить противоположное условие (*i* выходит за пределы диапазона), мы должны написать

идиома `if ( i < 0 || i >= n )`

Обратите внимание на использование оператора `||` вместо оператора `&&`.

Составные инструкции

В нашем шаблоне инструкции `if` обратите внимание на то, что *инструкция* является единичной, а не множественной:

If (выражение)
инструкция

Как быть, если нам нужно, чтобы инструкция `if` управляла двумя или несколькими инструкциями? Вот здесь появляется составная инструкция. Составная инструкция имеет форму

{инструкции}

Помещая скобки вокруг группы инструкций, мы можем заставить компилятор трактовать её как единую инструкцию.

Вот пример составной инструкции:

```
{line_num = 0; page_num ++;}
```

Для большей наглядности я обычно помещаю составную инструкцию в несколько строк, по одной в каждой строке:

```
{  
    line_num = 0;  
    page_num ++;  
}
```

Обратите внимание, что каждая внутренняя инструкция по-прежнему заканчивается точкой с запятой; а составная инструкция - нет.

Вот как будет выглядеть составная инструкция при использовании внутри инструкции `if`:

```
if (line_num == MAX_LINES){  
    line_num = 0;
```



```
    page_num ++;  
}
```

Составные инструкции также часто встречаются в циклах и других местах, где синтаксис Си требует одну инструкцию, а нам нужно больше одной.

Компонент else

У инструкции if может быть компонент else:

If (выражение) инструкция else инструкция

Инструкция, следующая за словом else, выполняется, если выражение в круглых скобках имеет значение 0. Вот пример инструкции if с компонентом else:

```
if ( i > j )  
    max = i;  
else  
    max = j;
```

Обратите внимание, что обе "внутренние" инструкции заканчиваются точкой с запятой. Если инструкция if содержит компонент else, возникает проблема с макетом: где должно размещаться else?

Многие программисты Си выравнивают его с `if` в начале инструкции, как в предыдущем примере. Внутренние инструкции обычно имеют отступы, но если они короткие, их можно поместить в ту же строку, что и `if`:

```
if (i > j) max=i;  
else max=j;
```

Нет никаких ограничений на то, какие инструкции могут появляться внутри инструкции `if`. На самом деле, нет ничего необычного в том, что инструкции `if` вкладываются внутрь других инструкций `if`. Рассмотрим следующую инструкцию `if`, которая находит наибольшее из чисел, хранящихся в `i`, `j` и `k`, и сохраняет это значение в `max`:

```
if ( i > j )  
    if ( i > k )  
        max = i;  
    else  
        max = k;  
else  
    if ( j > k )  
        max = j;  
    else  
        max = k;
```

Инструкции `if` могут быть вложены на любую глубину. Обратите внимание, как выравнивание каждого `else` с сопоставлением с `if` упрощает просмотр вложенности. Если вы все ещё находите вложение непонятным, не стесняйтесь добавлять фигурные скобки:

```
if ( i > j ){  
    if ( i > k )  
        max = i;  
    else  
        max = k;  
  
} else {  
    if ( j > k )  
        max = j;  
    else  
        max = k;  
  
}
```

Добавление фигурных скобок к инструкциям, даже когда они не нужны, подобно использованию круглых скобок в выражениях: обе методики помогают сделать программу более читаемой, в то же время избегая возможности того, что компилятор не поймёт программу так, как мы думали.

Некоторые программисты используют как можно больше скобок внутри инструкций if (и итераций). Программист, который принимает это соглашение, будет включать пару фигурных скобок для каждого компонента if и каждого компонента else:

```
if ( i > j ){  
    if ( i > k ){  
        max = i;  
    } else {
```

```
    max = k;  
    }  
} else {  
    if ( j > k ){  
        max = j;  
  
    } else {  
        max=k;  
    }  
}
```

Использование фигурных скобок, даже когда они не требуются, имеет два преимущества.

Во-первых, программу становится легче модифицировать, потому что можно легко добавить больше инструкций к любому компоненту if или else. Во-вторых, это помогает избежать ошибок, которые могут возникнуть из-за забывания использовать фигурные скобки при добавлении инструкций в предложение if или else.

Каскадированные инструкции if

Нам часто нужно будет проверить ряд условий, останавливаясь, как только одно из них станет истинным. "Каскадная" инструкция if часто является лучшим способом написать такую серию тестов. Например, следующая

201

каскадная инструкция `if` проверяет, является ли *n* меньше 0, равно 0 или больше 0:

```
if ( n < 0)
    printf("n меньше 0\n");
else
    if (n == 0)
        printf("n равно 0\n");
    else
        printf("n больше 0\n");
```

Хотя вторая инструкция `if` вкладывается внутрь первой, программисты Си обычно не делают её с отступом. Вместо этого они выравнивают каждый `else` с первоначальным `if`:

```
if ( n < 0)
    printf("n меньше 0\n");
else if (n == 0)
    printf("n равно 0\n");
else
    printf("n is больше 0\n");
```

Такое расположение даёт каскадной `if` отличительный вид:

if (выражение)

инструкция

else if (выражение)

инструкция

else if (выражение)

инструкция

else

инструкция

Конечно, последние две строки (*else инструкция*) не всегда присутствуют. Этот способ создания отступов для каскадной инструкции *if* позволяет избежать проблемы чрезмерного отступа, когда число тестов велико.

Более того, он убеждает читателя, что это не что иное, как серия тестов. Помните, что каскадная инструкция *if* не является каким-то новым типом инструкции; это просто обычная инструкция *if*, которая имеет другую инструкцию *if* со своим компонентом *else* (и что в инструкции *if* есть ещё одна инструкция *if* со своим компонентом *else* и т.д.).

ПРОГРАММА Расчёт комиссии брокера

Когда акции продаются или приобретаются через брокера, комиссия брокера часто вычисляется с использованием скользящей шкалы, которая зависит от стоимости акции. Предположим, что брокер взимает суммы, указанные в следующей таблице:

Размер транзакции	Комиссионная ставка
Менее \$2,500	\$30 + 1.7%
\$2500 - \$6250	\$56 + 0.66%
\$6250—\$20000	\$76 + 0.34%
\$20000—\$50000	\$100+ 0.22%
\$50000—\$500000	\$155+ 0.11%
Свыше \$500000	\$255+ 0.09%

Минимальная оплата составляет 39 долларов США. Наша следующая программа просит пользователя ввести сумму сделки, а затем отображает сумму комиссии:

Введите стоимость сделки: 30000

Комиссия: \$166.00

Сердцем программы является каскадная инструкция if, определяющая, в какой диапазон попадает сделка с ценными бумагами.

//broker.c

/ Вычисляет комиссию брокера */*

```
#include <stdio.h>
```

```
int main(void){
```

```
    float commission, value;
```

```
printf("Введите стоимость сделки: ");
scanf("%f", &value);
if (value < 2500.00f)
    commission = 30.00f + 0.17f * value;
else if (value < 6250.00f)
    commission = 56.005 + .0066f * value;
else if (value < 20000.00f)
    commission = 76.00f + .0034f * value;
else if (value < 50000.00f)
    commission = 100.00f + .0022f * value;
else if (value < 500000.00f)
    commission = 155.00f + .0011f * value;
else
    commission = 255.00f + .0009f * value;
if (commission < 39.00f)
    commission = 39.00f;
printf("Комиссия: $%.2f\n", commission);
return 0;
}
```

Вместо этого каскадная инструкция if могла бы быть написана таким образом:

```
if (value < 2500.00f)
```


205

```
commission = 30.00f +.017f * value;  
else if (value >2 2500.00f && value <6250.005)  
commission = 56.00f +.0065* value;  
else if (value >= 6250.00 && value <20000.00f)  
commission = 76.00f +.0034f * value;  
.....
```

В программе есть избыточные условия, без которых она будет работать. Например, первая if компонента проверяет, меньше ли значение, чем 2500, и, если это так, вычисляет комиссию. Когда мы достигнем второго if-теста (value > = 2500.00f && value < 6250.00f), мы знаем, что значение не может быть меньше 2500 и поэтому должно быть больше или равно 2500. Условие value > = 2 500.00f всегда будет истинным, поэтому проверять его не имеет смысла.

Проблема "висячего else"

Когда инструкции if вложены, мы должны следить за пресловутой проблемой "висячего else". Рассмотрим следующий пример:

```
if ( y!= 0 )  
    if ( x!= 0 )  
        result = x / y;  
else  
    printf("Ошибка:x равен 0\n");
```

К какой инструкции `if` принадлежит компонент `else`? Отступы указывают, что он принадлежит к внешней `if`-инструкции. Тем не менее, Си следует правилу, что компонент `else` принадлежит к ближайшей инструкции `if`, которая ещё не была сопряжена с `else`. В этом примере компонент `else` фактически принадлежит внутренней инструкции `if`, поэтому правильно отформатированная версия будет выглядеть так:

```
if ( y != 0 )
    if ( x != 0 )
        result = x / y;
    else
        printf("Ошибка: x равен 0\n");
```

Чтобы сделать компонент `else` частью внешней инструкции `if`, мы можем заключить внутреннюю `if` в фигурные скобки:

```
if ( y != 0 ) {
    if ( x != 0 )
        result = x / y;
}
else
    printf("Ошибка: y равен 0\n");
```

Этот пример иллюстрирует значение фигурных скобок; если бы мы использовали их в первоначальной инструкции `if`, мы бы вообще не попали в эту ситуацию.

Условные выражения

Инструкция Си `if` позволяет программе выполнять одно из двух действий в зависимости от значения условия. Си также предоставляет оператор, который позволяет выражению выбирать одно из двух значений в зависимости от значения условия. Условный оператор состоит из двух символов (`?` и `:`), которые должны использоваться вместе следующим образом:

`expr1 ? expr2 : expr3`

`expr1`, `expr2` и `expr3` могут быть выражениями любого типа. Результирующее выражение называется условным выражением. Условный оператор уникален среди операторов Си, так как ему требуется три операнда вместо одного или двух. По этой причине его часто называют тернарным оператором. Условное выражение `expr1 ? expr2 : expr3` следует читать "if `expr1` then `expr2` else `expr3`". ("если `expr1`, тогда `expr2`, иначе `expr3`") Выражение вычисляется поэтапно: `expr1` вычисляется первым, если его значение не равно нулю, тогда вычисляется `expr2`, а его значение является значением всего условного выражения. Если значение `expr1` равно нулю, тогда значение `expr3` является значением условного оператора. Следующий пример иллюстрирует условный оператор:

```
int i, j, k;
```

```
i=1; j=2;
```

```
k = i > j ? i : j; /* к сейчас 2 */
```

```
k = (i >= 0 ? i : 0) + j; /* к сейчас 3 */
```

Условное выражение $i > j ? i : j$ в первом присваивании k возвращает значение i или j , в зависимости от того, что из них больше. Так как i имеет значение 1, а j имеет значение 2, сравнение $i > j$ не выполняется, а значение условия равно 2, которое присваивается k . Во втором задании k сравнение $i > 0$ выполняется успешно; условное выражение $(i > 0 ? i : 0)$ имеет значение 1, которое затем добавляется к j для получения 3. Скобки, кстати, необходимы; приоритет условного оператора меньше, чем у других операторов, которые мы обсуждали до сих пор, за исключением операторов присваивания. Условные выражения имеют тенденцию делать программы короче, но их труднее понять, поэтому лучше их избегать. Есть, однако, несколько мест, в которых они заманчивы; одним из них является инструкция `return`. Вместо написания

```
if ( i > j )
    return i;
else
    return j;
```

многие программисты пишут

```
return i > j ? i : j;
```

Вызовы `printf` иногда могут извлекаться из условных выражений. Вместо

```
if (i > j)
    printf ("%d\n", i);
else
```

```
printf("%d\n", j);
```

мы могли бы просто написать

```
printf("%d\n", i > j ? i : j);
```

Условные выражения также распространены в некоторых типах макроопределений.

Логические значения в C89

В течение многих лет на языке Си не было правильного логического типа, не было его и в стандарте C89.

Это упущение является незначительным раздражителем, поскольку многие программы нуждаются в переменных, которые могут хранить как ложные, так и истинные значения.

Один из способов обойти это ограничение C89 - объявить переменную типа `int`, а затем присвоить ей либо 0, либо 1:

```
int flag;
```

```
flag = 0;
```

```
.....
```

```
flag = 1;
```

Хотя эта схема работает, она не вносит большой вклад в читаемость программы. Не очевидно, что `flag` должны быть присвоены только логические значения и что 0 и 1 представляют `false` и `true`. Чтобы программы были более

понятными, программисты C89 часто определяют макросы с именами, такими как TRUE и FALSE:

```
#define TRUE 1
```

```
#define FALSE 0
```

Присваивания для flag теперь имеют более естественный вид:

```
flag = FALSE;
```

```
flag = TRUE;
```

Чтобы проверить, является ли flag истиной (true), мы можем записать

```
if (flag == TRUE)....
```

или просто

```
if (flag)....
```

Последняя форма лучше, не только потому, что она более кратка, но также и потому, что она все равно будет работать правильно, если flag имеет значение, отличное от 0 или 1.

Чтобы проверить, является ли flag ложным (false), мы можем написать

```
if (flag == FALSE)....
```

или

```
if (!flag)....
```

Продвигая эту идею на один шаг дальше, мы можем даже определить макрос, который можно использовать как тип:

```
#define BOOL int
```

BOOL может заменить int при объявлении булевых переменных:

`BOOL flag;`

Теперь стало ясно, что `flag` - не обычная целочисленная переменная, а представляет собой булево условие. (Конечно, компилятор обрабатывает `flag` как переменную типа `int`). В последующих главах мы найдём более эффективные способы настройки булевого типа в C89 с помощью определений типов и перечислений.

Логические значения в C99

Многолетнее отсутствие булевого типа было исправлено в C99, который предоставляет тип `_Bool`. В этой версии Си, булева переменная может быть объявлена записью

`_Bool flag;`

`_Bool` является целочисленным типом (точнее, беззнаковым целочисленным (`unsigned`) типом), поэтому переменная `_Bool` на самом деле является всего лишь замаскированной целочисленной переменной. В отличие от обычной целочисленной переменной переменной `_Bool` могут быть присвоены только 0 или 1. В общем случае попытка сохранить ненулевое значение в переменной `_Bool` вызовет присваивание переменной 1:

```
flag = 5; /* flag присвоена 1 */
```

Допустимо (хотя и не желательно) выполнять арифметические действия над переменными `_Bool`; также допустимо печатать переменную `_Bool` (отображаются 0 или 1). И, конечно, переменная `_Bool` может быть проверена в инструкции `if`:

```
if (flag) /* проверяет является ли flag 1 */
```

Помимо определения типа `_Bool`, C99 также предоставляет новый заголовок `<stdbool.h>`, который упрощает работу с булевыми значениями. Этот заголовок содержит макрос `bool`, который означает `_Bool`. Если `<stdbool.h>` включён, мы можем написать

```
bool flag; /* то же, что _Bool flag; */
```

Заголовок `<stdbool.h>` также поставляет макросы с именами `true` и `false`, которые обозначают 1 и 0 соответственно, что позволяет писать

```
flag = false;
```

```
flag = true;
```

Поскольку заголовок `<stdbool.h>` так удобен, я буду использовать его в последующих программах, когда нужны булевы переменные.

5.3 Инструкция `switch`

В повседневном программировании нам часто приходится сравнивать выражение с рядом значений, чтобы увидеть, какому из них в данный момент оно соответствует. В Разделе 5.2 мы видели, что для этой цели можно использовать каскадную инструкцию `if`. Например, следующая каскадная инструкция `if` печатает английское слово, которое соответствует числовой оценке:

```
if (grade == 4)
    printf("Excellent");
```


213

```
else if (grade == 3)
    printf ("Good");
else if (grade == 2)
    printf ("Average");
else if (grade == 1)
    printf ("Poor");
else if (grade == 0)
    printf ( "Failing");
else
    printf("Illegal grade");
```

В качестве альтернативы этому типу каскадной инструкции if, Си предоставляет инструкцию switch. Следующая switch эквивалентна нашей каскадной if:

```
switch (grade) {
case 4: printf("Excellent");
    break;
case 3: printf("Good");
    break;
case 2: printf("Average");
    break;
case 1: printf("Poor");
    break;
case 0: printf("Failing");
    break;
```

```
default: printf("Illegal grade");
    break;
}
```

Когда выполняется эта инструкция, значение переменной `grade` проверяется на соответствие 4, 3, 2, 1 и 0. Если оно соответствует 4, например, выводится сообщение `Excellent` и инструкция `break` переносит управление на инструкцию, следующую за `switch`. Если значение `grade` не соответствует ни одному из перечисленных вариантов, применяется случай по умолчанию (`default`) и выводится сообщение `illegal grade`.

Инструкцию `switch` часто проще читать, чем каскадную инструкцию `if`. Кроме того, инструкции `switch` часто бывают быстрее, чем инструкции `if`, особенно если их несколько. В наиболее распространённой форме инструкция `switch` имеет форму

```
switch (выражение) {
    case выражение-константа : инструкция
    .....
    case выражение-константа : инструкция
    default : инструкция
```

Инструкция `switch` довольно сложна; давайте рассмотрим её компоненты по одному:

- **Управляющее выражение.** За словом `switch` следует целочисленное выражение в круглых скобках. Символы рассматриваются как целые числа в Си и, следовательно, могут быть протестированы в инструкциях `switch`. Однако

числа с плавающей точкой и строки не предназначены для использования в этих целях.

case метка. Каждый (компонент) case начинается с метки в форме

case константное-выражение :

Константное выражение очень похоже на обычное выражение, за исключением того, что оно не может содержать переменные или вызовы функций.

Таким образом, 5 является константным выражением и $5 + 10$ является константным выражением, но $n + 10$ не является константным выражением (если *n* не является макросом, который представляет константу). Константное выражение в метке case должно вычисляться до целого числа (символы также допустимы).

Инструкции. После каждой метки case появляется любое количество инструкций. Вокруг инструкций не требуется никаких скобок. (Наслаждайтесь этим - это одно из немногих мест в Си, где скобки не требуются.) Последняя инструкция в каждой группе обычно — break.

Повторяющиеся метки case не допускаются. Порядок cases не имеет значения; В частности, случай по умолчанию не обязательно должен быть последним. Только одно константное выражение может следовать за словом case; Однако несколько *меток case* могут предшествовать одной и той же группе операторов:

```
switch (grade) {
```

```
    case 4:
```

```
    case 3:
```

```
    case 2:
```

```
case 1: printf("Passing");
        break;

case 0: printf("Failing");
        break;

default: printf("Illegal grade");
        break;

}
```

Чтобы сэкономить место, программисты иногда помещают несколько *меток case* в одну строку:

```
switch (grade) {
    case 4: case 3: case 2: case 1:
        printf("Passing");
        break;
    case 0: printf("Failing");
        break;
    default: printf("Illegal grade");
        break;
}
```

```
}
```

К сожалению, нет способа написать метку case, которая устанавливает диапазон значений, как есть в некоторых языках программирования. У инструкции switch нет необходимости иметь случай по умолчанию (default). Если значение по умолчанию отсутствует и значение управляющего выражения не совпадает ни с одной из меток case, управление просто переходит к следующей инструкции после switch.

Роль инструкции break

Теперь давайте подробнее рассмотрим загадочную инструкцию break. Как мы видели, выполнение инструкции break приводит к тому, что программа "вываливается" из инструкции switch; Выполнение продолжается в следующей инструкции после switch. Причина, по которой нам нужно break, связана с тем, что инструкция switch действительно является формой "вычисляемого скачка". Когда определяется значение управляющего выражения, управление переходит к сопоставлению меток case значению выражения switch.

Метка case является не более чем маркером, указывающим положение в switch. Когда последняя инструкция в case будет выполнена, управление "проваливается" к первой инструкции в следующем case; метка case для следующего case игнорируется. Без break (или какой-либо другой инструкции перехода) управление будет переходить от одного case к следующему. Рассмотрим следующий оператор switch:

```
switch (grade){
```

```

case 4: printf("Exe11ent");
case 3: printf("Good");
case 2: printf ("Average");
case 1: printf("Poor");
case 0: printf("Failing");
default: printf("Illegal grade");
}

```

Если значение grade равно 3, выводится сообщение:

GoodAveragePoorFailingIllegal grade

Забывать использовать break - распространённая ошибка. Хотя исключение break иногда делается намеренно, чтобы несколько cases могли совместно использовать код, это обычно просто недосмотр.

Поскольку преднамеренное сваливание с одного case на другое встречается редко, неплохо указать любой умышленный пропуск break:

```

switch (grade) {
case 4: case 3: case 2: case 1:
    num_passing++;
    /* ПРОВАЛИВАЕТСЯ */
case 0:  total_grades++;
    break;
}

```

Без комментария кто-то может позже исправить "ошибку", добавив нежелательную инструкцию break. Хотя последнее

case в инструкции switch вообще не нуждается в инструкции break, обычная практика заключается в том, чтобы, в любом случае, поместить её там для защиты от проблемы "недостающая break", если case должен быть добавлен позже.

ПРОГРАММА Программирование печати даты в юридической форме

Контракты и другие юридические документы часто датируются следующим образом:

С сегодняшнего дня. 20__

Давайте напишем программу, которая отображает даты в этой форме.

Dated this_____ day of_____, 20_____.

Мы попросим пользователя ввести дату в форме месяц / день / год, после чего мы отобразим дату в "узаконенной" форме:

Enter date (mm/dd/yy): 7/19/14

Dated this 19th day of July, 2014.

Мы можем заставить printf выполнять большую часть форматирования.

Однако у нас остались две проблемы:

Как добавить "th" (или "st", "ad" или "rd") ко дню и как напечатать месяц словом, а не числом. К счастью, инструкция switch идеальна для обеих ситуаций; у нас будет одна switch печатать суффикс ко дню, а другая - печатать название месяца.

/* Печатает дату в юридической форме */

```
#include <stdio.h>

int main(void)
{
    int month, day, year;
    printf("Enter date (mm/dd/yy):");
    scanf("%d/%d/%d",&month, &day, &year);
    printf("Dated this %d", day);
    switch (day) {
        case 1: case 21: case 31:
            printf("st"); break;
        case 2: case 22:
            printf("nd"); break;
        case 3: case 23:
            printf("rd"); break;
        default: printf("th"); break;
    }
    printf(" day of ");
    switch (month) {
        case 1: printf("January"); break;
        case 2: printf("February"); break;
        case 3: printf("March"); break;
        case 4: printf("April"); break;
        case 5: printf("May"); break;
```



```
case 6: printf("June");break;
case 7 printf("July"); break;
case 8 printf("August"); break;
case 9 printf("September"); break;
case 10: printf("October"); break;
case 11: printf("November"); break;
case 12: printf("December"); break;
}
printf(", 20%.2d.\n", year);
return 0 ;
}
```

Обратите внимание на использование %. 2d, чтобы отобразить последние две цифры года. Если бы мы использовали % d вместо этого, одноцифровые годы отображались бы неправильно (2005 будет напечатан как 205).

Вопросы и ответы

Вопрос: Мой компилятор не выдаёт предупреждение, когда я использую = вместо ==. Есть ли какой-нибудь способ заставить компилятор заметить проблему?

Ответ: Вот трюк, который используют некоторые программисты: вместо написания if (i == 0)...

Они обычно пишут

If (0 == i)...

Предположим, что оператор `==` случайно записан как `=` :
`if (0 = i)...`

Компилятор выдаст сообщение об ошибке, так как невозможно присвоить значение нулю. Я не использую этот трюк, потому что я считаю, что программы выглядят неестественно. Кроме того, его можно использовать только в том случае, если один из операндов в тестовом состоянии не является алгоритмом. К счастью, многие компиляторы способны проверять подозрительные использования оператора `i = 1` в условиях `if`.

Например, компилятор GCC проведёт эту проверку, если используется опция `-Wparentheses` или если выбрано `-Wall` (все предупреждения). GCC позволяет программисту подавлять предупреждение в конкретном случае, заключая условие `if` во вторую пару скобок:

`if ((i = j))....`

Вопрос: Книжки по Си, похоже, используют несколько разных стилей отступа и размещения скобок для составления инструкций. Какой стиль лучше?

Ответ: Согласно "Словарю нового хакера" (Cambridge, Mass: MIT Press, 1996), существует четыре общих стиля размещения отступов и скобок:

Стиль K & R, используемый в в книге Кернигана и Ритчи "Язык программирования Си", является тем, который я выбрал для программ в этой книге. В стиле K & R левая фигурная скобка появляется в конце строки:

```
if (line_num == MAX_LINES) {
    line_num = 0;
```

```
    page_num++;  
}
```

■ Стил К & R сохраняет компактность программ, не оставляя левую фигурную скобку на отдельной строке. Недостаток: левую фигурную скобку может быть трудно найти. (Я не считаю это проблемой, так как отступ внутренних инструкций делает ясным, где должна быть левая фигурная скобка.) Стил К & R, кстати, наиболее часто используется в Java.

■ Стил Allman, названный в честь Эрика Аллмана (автора sendmail и других UNIX утилит), ставит левую фигурную скобку на отдельной строке:

```
if (line_num == MAX__LINES)  
{  
    line_num = 0;  
    page_num++;  
}
```

Этот стиль позволяет легко проверить, что фигурные скобки образуют соответствующие пары.

■ Стил Whitesmiths, популяризированный компилятором Whitesmiths Си, диктует, что фигурные скобки должны иметь отступ:

```
if (line_num == MAX_LINES)  
{  
    line_num = 0;
```

```
page_num++;  
}
```

■ Стилль GNU, используемый в программном обеспечении, разработанном Проектом GNU, располагает фигурные скобки с отступом, затем с дальнейшим отступом внутренние инструкции:

```
if (line_num == MAX_LINES)  
{  
    line_num = 0;  
    page_num++;  
}
```

Какой стиль вы используете, в основном дело вкуса: нет никаких доказательств того, что один стиль явно лучше других. В любом случае выбор правильного стиля менее важен, чем последовательное его применение.

Вопрос: Если *i* - переменная типа int, а *f* - переменная типа float, каков тип условного выражения (*i* > 0 ? *i*: *f*)?

Ответ: Когда значения int и float смешиваются в условном выражении, как они здесь, выражение имеет тип float. Если *i* > 0 истинно, значением выражения будет значение *i* после преобразования в тип float.

Вопрос: Почему C99 не нашёл лучшего имени для булевского типа?

Ответ: `_Bool` не очень элегантное имя, не так ли? Более распространённые имена, такие как `bool` или `boolean`, не были выбраны, поскольку существующие программы на Си могли уже определять эти имена, и старый код перестал бы компилироваться.

Вопрос: Хорошо, так, почему бы имени `_Bool` также не повреждать более старые программы?

Ответ: Стандарт C89 указывает, что имена, начинающиеся с символа подчёркивания, за которым следует заглавная буква, зарезервированы для будущего использования и не должны использоваться программистами.

Вопрос: Шаблон, указанный для инструкции `switch`, описан как "наиболее распространённая форма". Существуют ли другие формы?

Ответ: Инструкция `switch` является немного более общей, чем описано в этой главе, хотя приведённое здесь описание является достаточно общим практически для всех программ. Например, инструкция `switch` может содержать метки, которым не предшествует слово `case`, что приводит к забавным (?) ловушкам. Предположим, что мы случайно неправильно набрали слово `default`:

```
switch (...) {  
    ...  
    defualt:...
```

```
}
```

Компилятор может не обнаружить ошибку, так как он предполагает, что default: является обычной меткой.

Вопрос: Я видел несколько методов выполнения отступов в инструкции switch. Какой способ лучше?

Ответ: Существует как минимум два распространённых метода.

Один заключается в том, чтобы выставлять инструкции в каждом case после метки:

```
switch (coin) {  
    case 1: printf("Cent");  
        break;  
    case 5: printf("Nickel");  
        break;  
    case 10: printf("Dime");  
        break;  
    case 25: printf("Quarter");  
        break;  
}
```

Если каждый case состоит из одного действия (вызов printf, в этом примере), инструкция break может даже идти в той же строке, что и действие:

```
switch
```

```
(coin) {  
    case 1: printf("Cent"); break;  
    case 5: printf("Nickel"); break;  
    case 10: printf("Dime"); break;  
    case 25: printf("Quarter"); break;  
}
```

Другой метод заключается в том, чтобы выставить инструкции под *case метка*, делая отступы инструкций, чтобы выделить *case метка*:

```
switch (coin) {  
    case 1:  
        printf("Cent");  
        break;  
    case 5:  
        printf("Nickel");  
        break;  
    case 10:  
        printf("Dime");  
        break;  
    case 25:  
        printf ("Quarter");  
        break;  
}
```

В одной из вариаций этой схемы каждая *case метка* выравнивается по слову *switch*.

Первый метод хорош, когда инструкции в каждом *case* коротки и их относительно мало. Вторым способом лучше подходит для больших инструкций *switch*, в которых значения в каждом *case* являются сложными и / или многочисленными.

Упражнения

1. Следующие фрагменты программы иллюстрируют операторы отношения и равенства. Покажите результат, полученный каждым из них, считая, что *i*, *j* и *k* являются *int* переменными.

(a) *i* = 2; *j* = 3;

k = *i* * *j* == 6;

(b) *i*=5; *j*=10; *k*=1;

printf("%d", *k* > *i* < *j*);

(c) *i*=3; *j*=2; *k*=1;

printf("%d", *i* < *j* == *j* < *k*);

(d) *i*=3; *j*=4; *k*=5;

printf("%d", *i* % *j* + *i* < *k*);

2. Следующие фрагменты программы иллюстрируют логические операторы. Покажите результат, полученный каждым из них, считая, что *i*, *j* и *k* являются *int* переменными.

- (a) `i=10; j=5;`
`printf ( "%d", i < j );`
- (b) `i = 2; j = 1;`
`printf ( "%d", !!i + !j );`
- (c) `i=5; j=0; k=-5;`
`printf ( "%d", i && j || k );`
- (d) `i=1; j=2; k=3;`
`printf ( "%d", i < j || k );`

*3. Следующие фрагменты программы иллюстрируют поведение коротких замыканий в логических выражениях. Покажите результат, полученный каждым из них, считая, что `i`, `j` и `k` являются `int` переменными.

* 4. Напишите одно выражение, значение которого равно -1, 0 или +1, в зависимости от того, меньше ли `i`, равно или больше `j`, соответственно.

* 5. Является ли следующая инструкция `if` допустимой?

```
if ( n >= 1 <= 10 )
printf ("n находится между 1 и 10 \ n");
```

Если да, что она делает, когда `n` равно 0?

* 6. Является ли следующая инструкция `if` допустимой?

```
if ( n == 1-10 )
printf ("n находится между 1 и 10 \ n");
```

Если да, что она делает, когда `n` равно 5?

7. Что печатает следующая инструкция, если *i* имеет значение 17 ? Что она печатает, если *i* имеет значение -17 ?

```
printf ("%d \n", 1 >= 0 ? i: -i);
```

8. Следующая инструкция `if` неоправданно сложная.

Упростите её как можно больше. (Подсказка: всю инструкцию можно заменить одним присваиванием.)

```
if (age >= 13)
    if (age <= 19)
        teenager = true;
    else
        teenager = false;
else if (age < 13)
    teenager = false;
```

9. Эквивалентны ли следующие инструкции `if`? Если нет, то почему?

if (score >= 90) printf ("A");	if (score < 60) printf("F");
else if (score >= 80) printf ("B");	else if (score < 70) printf ("D") ;
else if (score >= 70) printf ("C");	else if (score < 80) printf ("C") ;

else if (score >= 60) printf ("D");	else if (score < 90) printf ("B") ;
else printf ("F");	else printf ("A");

10. Какой результат даёт следующий фрагмент программы?
(Предположим, что *i* - целочисленная переменная.)

```
i = 1;  
switch (i % 3){  
    case 0:  printf("zero");  
    case 1:  printf("one");  
    case 2:  printf("two");  
}
```

11. В следующей таблице показаны телефонные коды в штате Джорджия, а также самый большой город в каждой области:

Код города	Крупный город
229	Albany
404	Atlanta
470	Atlanta
478	Macon
678	Atlanta
706	Columbus
762	Columbus

770	Atlanta
912	Savannah

Напишите инструкцию switch, управляющим выражением которой является переменная `area_code`. Если значение `area_code` указано в таблице, инструкция switch напечатает соответствующее ему имя города. В противном случае инструкция switch отобразит сообщение "Area code not: recognated". Используйте методы, описанные в Разделе 5.3, чтобы сделать инструкцию switch как можно более простой.

Проекты по программированию

1. Напишите программу, которая вычисляет количество цифр, которое содержит число:

Введите число: 374

Число 374 имеет 3 цифры

Вы можете предположить, что число имеет не более четырёх цифр.

Подсказка: используйте инструкции `if` для проверки числа.

Например, если число находится между 0 и 9, оно имеет одну цифру. Если число от 10 до 99, оно имеет две цифры.

233

2. Напишите программу, которая запрашивает у пользователя 24-часовое время, затем отображает время в 12-часовой форме:

Введите 24-часовой формат времени: 21:11
12-часовой эквивалент:: 21:11PM

Будьте внимательны, чтобы не отображать 12:00 как 0:00.

3. Измените программу broker.c Раздела 5.2, внося в нее следующие изменения:

(a) Попросите пользователя ввести количество акций и цену за акцию. Вместо стоимости сделки.

(b) Добавьте инструкции, которые вычисляют комиссию, взимаемую конкурирующим брокером (\$33 плюс 35 центов на акцию для менее 2000 акций, \$33 плюс 2 цента на акцию за 2000 и более акций). Покажите комиссию конкурента, а также комиссию, взимаемую первоначальным брокером.

4. Вот упрощённая версия шкалы Бофорта, которая используется для оценки силы ветра:

Скорость (узлы) Описание

Меньше чем 1	штиль
1-3	легкий бриз
4-27	бриз
28-47	сильный ветер
48-63	шторм
Выше 63	ураган

Напишите программу, которая попросит пользователя ввести скорость ветра (в узлах), а затем отобразит соответствующее описание.

5. В одном штате отдельные жители облагаются следующим подоходным налогом:

Доход Сумма налога

Не более \$750 1% от дохода

\$750—\$2250 \$7.50 плюс 2% от суммы более \$750

\$2250-\$3750 \$37.50 плюс 3% от суммы более \$2250

\$3750—\$5250 \$82.50 плюс 4% от суммы более \$3750

\$5250—\$7000 \$142.50 плюс 5% от суммы более \$5250

Свыше \$7000 \$230.00 плюс 6% от суммы более \$7000

Напишите программу, которая попросит пользователя ввести сумму налогооблагаемого дохода, а затем отобразит сумму налога.

6. Измените программу урс.с Раздела 4.1 программа так, чтобы она проверяла, действителен ли UPC. После того, как пользователь вводит UPC, программа выведет на экран или VALID или NOT VALID.

7. Напишите программу, которая найдет наибольшее и наименьшее из четырёх целых чисел, введённых пользователем:

Введите четыре целых числа: 21 43 10 35

Наибольшее: 43

Наименьшее: 10

Используйте как можно меньше инструкций if. Подсказка: четырёх инструкций if достаточно.

8. В следующей таблице показаны ежедневные рейсы из одного города в другой:

Время вылета	Время прибытия
8:00 am.	10: 16 am.
9:43 am.	11:52 am.
11:19 a.m.	1:31 pm.
12:47 pm.	3:00 pm.
2:00 pm.	4:08 pm.
3:45 p.m.	5:55 pm.
7:00 pm.	9:20 pm.
9:45 p.m.	11:58 p.m.

Напишите программу, которая просит пользователя ввести время (выраженное в часах и минутах, используя 24-х часовой формат). Затем программа отображает время отправления и прибытия для полёта, время вылета которого ближе всего к времени, введённому пользователем:

Введите 24-часовой формат времени: 13:15

Ближайшее время вылета - 12: 47 вечера, прибытие в 3:00.

Подсказка. Преобразуйте входные данные во время, выраженное в минутах с полуночи, и сравните его со временем отправления, также выраженным в минутах с полуночи. Например, 13:15 составляет $13 \times 60 + 15 = 795$

минут с полуночи, что ближе к 12:47 после полудня. (767 минут с полуночи), чем в любое другое время отправления.

9. Напишите программу, в которой пользователю предлагается ввести две даты, а затем указать, какая дата наступает раньше в календаре:

Введите первую дату (мм / дд / гг): 3/6/08

Введите вторую дату (мм / дд / гг): 5/17/07

5/17/07 раньше, чем 3/6/08

10. Используя инструкцию switch, напишите программу, которая преобразует числовую оценку в буквенную оценку:

Введите числовую оценку: 84

Буквенная оценка: В

Используйте следующую шкалу оценок:

A = 90-100, B = 80-89, C = 70-79,

D = 60-69, F = 0-59.

Распечатайте сообщение об ошибке, если оценка превышает 100 или меньше 0. Совет. Разбейте оценку на две цифры, затем используйте инструкцию switch для проверки десяти чисел.

11. Напишите программу, которая запрашивает у пользователя ввести двузначное число, затем печатает словесное название числа:

Введите двузначное число: 45

Вы ввели число сорок пять.

Подсказка: разделите число на две цифры. Используйте одну

инструкцию switch для печати слова для первой цифры ("двадцать", "тридцать" и т. Д.). Используйте вторую инструкцию switch, чтобы напечатать слово для второй цифры.

Не забывайте, что числа между 11 и 19 требуют особого обращения.

6 Циклы

Не стоит писать программу без цикла и структурированной переменной.

В Главе 5 описывались инструкции выбора языка Си - if и switch. В этой главе описываются итерационные инструкции Си, которые позволяют нам организовывать циклы. Цикл - это инструкция, задачей которой является многократное выполнение некоторой другой инструкции (тела цикла). В Си каждый цикл имеет управляющее выражение. Каждый раз, когда выполняется тело цикла (итерация цикла), управляющее выражение вычисляется, если выражение истинно - имеет значение, отличное от нуля - цикл продолжает выполняться. Си предоставляет три инструкции итерации: while, do и for, которые рассматриваются в разделах 6.1. 6.2 и 6.3 соответственно. Инструкция while используется для циклов, контрольное выражение которых проверяется перед

выполнением тела цикла. Инструкция `do` используется, если выражение проверяется после выполнения тела цикла. Инструкция `for` удобна для циклов, которые увеличивают или уменьшают переменную -счётчик. В Разделе 6.3 также вводится оператор запятая, который используется в основном для инструкции `for`. Последние два раздела этой главы посвящены компонентам языка Си, которые используются в сочетании с циклами. В Разделе 6.4 описаны инструкции `break`, `continue` и `goto`. `break` выходит из цикла и передаёт управление следующей инструкции после цикла, `continue` пропускает остальную часть итерации цикла и `goto` переходит в любую инструкцию внутри функции. В Разделе 6.5 описывается инструкция `Null`, который можно использовать для создания циклов с пустыми телами.

6.1 Инструкции `while`.

Из всех способов организации циклов в Си инструкция `while` является самой простой и самой фундаментальной.

Инструкции `while` имеет форму:

`while (выражение) инструкция`

Выражение внутри круглых скобок является управляющим выражением; инструкция после круглых скобок является телом цикла. Вот пример:

```
while ( i < n ) /* управляющее выражение */  
    i = i * 2; /* Тело цикла */
```

Обратите внимание, что круглые скобки являются обязательными и что между правой круглой скобкой и телом цикла ничего нет. (Для некоторых языков требуется слово `do`.) Когда выполняется инструкция `while`, управляющее выражение вычисляется первым. Если его значение отлично от нуля (`true`), выполняется тело цикла, а выражение проверяется снова. Процесс продолжается таким образом: сначала проверяется управляющее выражение, а затем выполняется тело цикла - до тех пор, пока управляющее выражение в конечном итоге не будет иметь значение ноль. Следующий пример использует инструкцию `while` для вычисления наименьшей степени 2, которая больше или равна числу n :

```
i = 1;
```

```
while ( i < n )
```

```
    i = i * 2;
```

Предположим, что n имеет значение 10. Следующая трассировка показывает, что происходит, когда выполняется инструкция `while`:

$i = 1$; i теперь равно 1.

Является ли $i < n$? Да, продолжить.

$i = i * 2$; i теперь равно 2.

Является ли $i < n$? Да; продолжить.

$i = i * 2$; i теперь равно 4.

Является ли $i < n$? Да; продолжить.

$i = i * 2$; i теперь равно 8.

Является ли $i < n$? Да; продолжить.

$i = i * 2$; i теперь равно 16.

Является ли $i < n$? Нет; выход из цикла.

Обратите внимание, что цикл продолжается до тех пор, пока управляющее выражение ($i < n$) истинно. Когда выражение становится ложным, цикл завершается, а i , как ожидается, больше или равно n . Хотя тело цикла должно иметь одну инструкцию, это всего лишь формальность. Если нам нужно больше одной инструкции, мы можем просто использовать фигурные скобки для создания одной составной инструкции:

```
while (i > 0) {  
    printf("Т минус %d и подсчет\n", i);  
    i--;  
}
```

Некоторые программисты всегда используют фигурные скобки, даже если они не являются строго необходимыми:

```
while (i < n) { /* Скобки допускаются, но не обязательны */  
    i = i * 2;  
}
```

В качестве второго примера проследим выполнение следующих инструкций, которые отображают ряд сообщений "обратный отсчёт":

```
i = 10;  
while ( i > 0 ){  
    printf( "Т минус %d и подсчет\n", i);  
    i--;  
}
```

Перед выполнением инструкции `while` переменной *i* присваивается значение 10. Так как 10 больше 0, тело цикла выполняется, вызывая сообщение "Т минус 10 и подсчёт" для печати, и *i* - декрементируется. Затем снова проверяется условие $i > 0$. Так как 9 больше 0, тело цикла выполняется ещё раз. Этот процесс продолжается до тех пор, пока не будет напечатано сообщение Т минус 1 и подсчёт, а *i* не станет 0. Тест $i > 0$ даёт ложь, что ведёт к завершению цикла. Пример обратного отсчёта приводит нас к тому, чтобы сделать несколько замечаний относительно инструкции `while`:

- Управляющее выражение ложно, и цикл `while` завершается. Таким образом, когда цикл, управляемый выражением $i > 0$, завершается, *i* должна быть меньше или равен 0. (В противном случае мы бы все равно выполняли цикл!)

- Тело цикла `while` может вообще не выполняться. Поскольку управляющее выражение проверяется перед выполнением тела цикла, возможно, что тело не выполнится ни разу. Если *i* имеет отрицательное или нулевое значение при первом вводе цикла обратного отсчёта, цикл ничего не делает.

- Инструкция `while` часто может быть написана множеством способов. Например, мы могли бы сделать цикл обратного отсчёта более кратким, уменьшая *i* внутри вызова `printf`:

```
while (i > 0)
```

```
    printf ( "Т минус %d и подсчет\n", i-- );
```

Бесконечные циклы

Инструкция `while` не будет завершаться, если управляющее выражение всегда имеет ненулевое значение.

На самом деле программисты C иногда сознательно создают бесконечный цикл, используя в качестве управляющего выражения ненулевую константу:

`while (1)....`

Инструкция `while` этой формы будет выполняться вечно, если в её теле не содержится инструкция, которая передаёт управление из цикла (`break`, `goto`, `return`) или вызывает функцию, которая заставляет программу завершать работу

ПРОГРАММА Печать таблицы квадратов

Давайте напишем программу, которая печатает таблицу квадратов. Программа сначала предложит пользователю ввести число n . Затем она выводит n строк вывода, каждая из которых содержит числа от 1 до i вместе со своим квадратом:

Эта программа печатает таблицу квадратов.

Введите количество записей в таблице: 5

1	1
2	4
3	9
4	16

5 25

Давайте сохраним в программе нужное количество квадратов в переменной n . Нам понадобится цикл, который многократно печатает число i и его квадрат, начиная с i , равного 1. Цикл повторяется, пока i меньше или равно n .

Мы должны обязательно добавить 1 к i каждый раз по циклу. Мы напишем цикл как выражение `while`. (Честно говоря, у нас нет большого выбора, так как инструкция `while` является единственным видом цикла, который мы рассмотрели до сих пор.)

Вот законченная программа:

```
//square.c
```

```
/* Печать таблицы квадратов с использованием инструкции  
while */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i, n;
```

```
    printf("Эта программа печатает таблицу квадратов..\n");
```

```
    printf("Введите количество записей в таблице:: ");
```

```
    scanf("%d", &n);
```

```
    i=1;
```

```
    while ( i <= n ){
```

```
        printf("%10d%10d\n", i, i * i);
```

```
        i++;
```

```
    }
```

```
    return 0;
```

```
}
```

Обратите внимание, что `square.c` отображает числа в аккуратно выровненных столбцах. Трюк заключается в том, чтобы использовать спецификацию преобразования, такую как `%10d`, а не просто `%d`, пользуясь тем фактом, что `printf` выравнивает числа по правому краю, когда задана ширина поля.

ПРОГРАММА Суммирование ряда чисел

В качестве второго примера инструкции `while` напишем программу, суммирующую ряд целых чисел, введенных пользователем. Вот что увидит пользователь:

```
//sum.c
```

```
#include <stdio.h>
```

```
int main(void){
```

```
    int n, sum = 0;
```

```
    printf("Эта программа суммирует ряд целых чисел.\n");
```

```
    printf("Введите целые числа (0 для завершения):\n");
```

```
    scanf("%d", &n);
```

```
    while (n != 0) {
```

```
        sum += n;
```

```
        scanf("%d", &n);
```

```
    }
```

```
    printf("Сумма равна: %d\n",sum);
```



```
    return 0;  
}
```

Обратите внимание, что условие $n \neq 0$ проверяется сразу после считывания числа, что позволяет завершить цикл как можно скорее. Также обратите внимание, что есть два идентичных вызова `scanf`, которых часто трудно избежать при использовании циклов `while`.

6.2 Инструкция *do*

Инструкция `do` тесно связана с инструкцией `while`; на самом деле, инструкция `do` является по существу просто инструкцией `while`, контрольное выражение которого проверяется после каждого выполнения тела цикла.

Инструкция `do` имеет следующий вид:

do инструкция *while* (выражение);

Как и в случае с инструкцией `while`, тело инструкции `do` должно быть одной инструкцией (возможно, составной, конечно), а управляющее выражение должно быть заключено в круглые скобки. Когда выполняется инструкция `do`, тело цикла выполняется первым, затем вычисляется управляющее выражение. Если значение выражения отлично от нуля, тело цикла выполняется снова, а затем выражение вычисляется ещё раз. Выполнение инструкции `do` прекращается, когда управляющее выражение имеет значение 0 после выполнения тела цикла. Давайте перепишем пример 6.1 "Обратный отсчёт", используя инструкцию `do`:

```
i = 10;
do {
    (printf ("Т минус %d и подсчёт\n", i);
    --i;
} while (i > 0);
```

Когда выполняется инструкция `do`, тело цикла сначала выполняется, вызывая сообщение "Т минус 10 и подсчёт" для печати и декрементирования *i*. Затем проверяется условие $i > 0$. Так как 9 больше 0, тело цикла выполняется во второй раз. Этот процесс продолжается до тех пор, пока не будет напечатано сообщение Т минус 1 и подсчёт, а *i* станет 0. Тест $i > 0$ выдаст ложь, что приводит к завершению цикла. Как показано в этом примере, инструкция `do` часто неотличима от инструкции `while`. Различие между ними состоит в том, что тело инструкции `do` выполняется всегда хотя бы один раз; тело инструкции `while` будет полностью пропущено, если управляющее выражение равно 0 изначально.

Кстати, это хорошая идея использовать фигурные скобки во всех инструкциях `do`, независимо от того, нужны ли они или нет, потому что инструкция `do` без скобок легко может быть ошибочно принята за инструкцию `while`:

```
do
    printf("Т минус %d и подсчёт\n",i--);
while (i > 0);
```

Невнимательный читатель может подумать, что слово `while` было началом инструкции `while`.

ПРОГРАММА Вычисление числа цифр в целых числах

Хотя инструкция `while` появляется в программах на языке Си гораздо чаще, чем инструкция `do`, последняя удобна для циклов, которые должны выполняться хотя бы один раз. Чтобы проиллюстрировать этот момент, давайте напишем программу, которая вычисляет количество цифр в целых числах, введенных пользователем:

Введите неотрицательное целое число: 60

Число имеет 2 цифры.

Наша стратегия состоит в том, чтобы делить ввод пользователя на 10 последовательно, пока он не станет 0; Количество выполненных делений - это количество цифр. Ясно, что нам понадобится какой-то цикл, так как мы не знаем, сколько нужно делений, чтобы достичь значения 0. Но должны ли мы использовать инструкцию `while` или инструкцию `do`? Инструкция `do` выглядит более привлекательной, потому что каждое целое число - даже 0 - имеет хотя бы одну цифру. Вот программа:

```
//numdigits.c
```

```
/* Вычисление числа цифр целого числа*/
```

```
#include <stdio.h>
```

```
int main(void){

    int digits = 0, n;

    printf("Введите неотрицательное целое: ");
    scanf("%d", &n);

    do {
        n /= 10;
        digits++;
    } while (n > 0);

    printf("Число состоит из %d цифр(ы).\n", digits);

    return 0;
}
```

Чтобы понять, почему инструкция `do` является правильным выбором, давайте посмотрим, что произойдёт, если мы заменим цикл `do` похожим циклом `while`:

```
while (n > 0) {
    (n /= 10;
    digits++;
}
```

Если ***n*** равно 0, этот цикл не будет выполнен вообще, и программа напечатает.

Число состоит из 0 цифр(ы)

6.3 Инструкция for

Теперь мы переходим к последнему из циклов Си: выражение `for`. Не обескураживайтесь кажущейся сложностью инструкции `for`; на самом деле это лучший способ написания многих циклов.

Инструкция `for` идеально подходит для циклов с переменной "счётчиком", но она достаточно универсальна, чтобы использовать её и для других видов циклов. Инструкция `for` имеет форму

`for (expr1; expr2; expr3) инструкция`

где `expr1`, `expr2` и `expr3` являются выражениями.

Вот пример:

```
for (i = 10; i > 0; i--)  
    printf("Т минус %d и подсчёт\n", i);
```

Когда выполняется эта инструкция `for`, переменная *i* инициализируется значением 10, затем *i* проверяется больше ли она 0. Поскольку так оно и есть, выводится сообщение -Т минус 10 и подсчёт, - затем *i* декрементируется. Условие *i* > 0 затем проверяется снова. Тело цикла будет выполняться 10 раз, при этом *i* варьируется от 10 до 1. Инструкция `for` тесно связана с инструкцией `while`. На самом деле, за исключением нескольких редких случаев, цикл `for` всегда можно заменить эквивалентным циклом `while`:

```
expr1;  
while ( expr2){  
    инструкция  
    expr3;  
}
```

Как видно из этого шаблона, `expr1` - это шаг инициализации, который выполняется только один раз, перед началом цикла, `expr2` управляет завершением цикла (цикл продолжает выполняться, пока значение `expr2` не равно нулю), а `expr3` является операцией выполняемой в конце каждой итерации цикла. Применяя этот шаблон к нашему предыдущему примеру для цикла, мы приходим к следующему:

```
i = 10;  
while (i > 0) {  
    printf("Т минус %d и подсчет\n",i);  
    i--;  
}
```

Изучение эквивалентной инструкции `while` может помочь нам разобраться в тонкости инструкции `for`. Например, предположим, что мы заменяем `i--` на `--i` в нашем примере цикла `for`:

```
for (i = 10; i > 0; --i)  
    printf("Т минус %d и подсчет\n", i);
```

Как это изменение влияет на цикл? Глядя на эквивалентный цикл `while`, мы видим, что оно не имеет никакого эффекта:

```
i = 10 ;  
while (i > 0){  
    printf ("Т минус %d и подсчет\n", i);  
    --i;  
}
```

Первое и третье выражения в инструкции `for` выполняются как инструкции и их значения бесполезны - они полезны только для их побочных эффектов. Следовательно, эти два выражения обычно являются присваиваниями или выражениями увеличения / уменьшения.

Идиомы инструкции `for`

Инструкция `for` обычно является лучшим выбором для циклов, которые увеличивают значение переменной-счётчика или уменьшают значение переменной-счётчика. Инструкция `for`, которая инкрементирует или декрементирует *n* раз переменную-счётчик обычно будет иметь одну из следующих форм:

■ **Счёт от 0 до n-1:**

идиома `for ( i = 0; i < n; i++ )...`

■ **Счёт от 1 до n:**

идиома `for ( i = 1; i <= n; i++ )...`

■ **Счёт от $n-1$ до 0:**

идиома `for ( i = n- 1; i >= 0; i-- )...`

■ **Счёт от n до 1:**

идиома `for ( i = n; i > 0; i-- )...`

Имитация этих шаблонов поможет избежать некоторых ошибок, перечисленных далее, которые часто делают начинающие Си-программисты:

■ Использование `<` вместо `>` (или наоборот) в управляющих выражениях. Обратите внимание, что циклы "прямого счета" используют оператор `<` или `<=`, а циклы "обратного счета" полагаются на `>` или `>=`.

■ Использование `==` в управляющем выражении вместо `<`, `<=`, `>` или `>=`. Управляющее выражение должно быть истинным в начале цикла, а затем становиться ложным, чтобы цикл мог завершиться. Тест, такой как `i == n`, не имеет смысла, потому что это не верно изначально.

■ Ошибки "занижения или завышения на единицу", такие как запись управляющего выражения как `i <= n` вместо `i < n`.

Опускание выражений в инструкции `for`

Инструкция `for` более гибкая, чем мы видели до сих пор. Некоторым циклам `for` могут не понадобиться все три выражения, которые обычно управляют циклом, поэтому Си позволяет нам опустить любое или даже все выражения. Если первое выражение опущено, инициализация не выполняется до выполнения цикла:

`i = 10;`


```
for (; i > 0; --i)
    printf("Т минус %d и подсчет\n", i);
```

В этом примере инициализация *i* выполняется с помощью отдельного присваивания, поэтому мы опустили первое выражение в инструкции `for`. (Обратите внимание, что точка с запятой между первым и вторым выражениями остаётся. Две точки с запятой должны присутствовать всегда, даже если мы опустили некоторые выражения.) Если мы опускаем третье выражение в выражении `for`, тело цикла отвечает за то, что значение второго выражения, в конечном счёте станет ложным. Наш пример инструкции `for` можно было бы написать так:

```
for ( i = 10; i > 0; )
    printf( "Т минус %d и подсчёт\n", i-- );
```

Чтобы компенсировать отсутствие третьего выражения, мы организовали для *i* уменьшение в теле цикла.

Когда первое и третье выражения опущены, результирующий цикл представляет собой не что иное, как скрытую инструкцию `while`. Например, цикл

```
for (; i > 0 ;)
    printf( "Т минус %d и подсчет\n", i-- );
```

такой же как

```
while ( i > 0)
    printf( "Т минус %d и подсчет\n", i-- );
```

Версия `while` является более ясной и поэтому предпочтительной.

Если второе выражение отсутствует, оно по умолчанию имеет значение истина, поэтому инструкция `for` не прекращает работу (если только она не будет остановлена каким-либо другим образом).

Например, некоторые программисты используют следующую инструкцию `for` для создания бесконечного цикла:

идиома `for (;;) ...`

Инструкции `for` в C99

В C99 первое выражение в инструкции `for` может быть заменено декларацией. Это свойство позволяет программисту объявлять переменную для использования циклом:

```
for ( int i = 0; i < n; i++ )
```

...

Переменную *i* не нужно было объявлять перед этой инструкцией. (На самом деле, если объявление *i* уже существует, эта инструкция создаёт новую версию *i*, которая будет использоваться исключительно внутри цикла.)

Переменная, объявленная в инструкции `for`, не может быть доступна за пределами тела цикла (мы говорим что она не видима вне цикла):

```
for (int i = 0; i < n; i++){
    printf("%d", i); /*Правильно; i видна внутри цикла */
}
printf("%d", i); /*** НЕПРАВИЛЬНО ***/
```

Объявление собственной управляющей переменной внутри инструкции `for`, как правило, является хорошей идеей: это удобно, и это может сделать программы более понятными. Однако, если программе необходимо получить доступ к переменной после завершения цикла, необходимо использовать более старую форму инструкции `for`. Кстати, инструкция `for` может объявлять более одной переменной, при условии, что все переменные имеют один и тот же тип:

```
for ( int i = 0, j = 0; i < n; i++ )
```

Оператор запятая

Иногда нам может понадобиться написать инструкцию `for` с двумя (или более) выражениями инициализации или с одним, каждый раз увеличивающим количество переменных в цикле. Мы можем сделать это, используя выражение-запятая в качестве первого или третьего выражения в выражении `for`. Выражение запятая имеет вид

```
expr1, expr2
```

`expr1` и `expr2` -любые два выражения

Выражение-запятая вычисляется в два этапа: во-первых вычисляется `expr1` и его значение отбрасывается.

Во-вторых, вычисляется `expr2`; его значение является значением всего выражения. Вычисление `expr1` всегда должно иметь побочный эффект; если это не так, то `expr1` не имеет никакой цели. Например, предположим, что *i* и *j* имеют

значения 1 и 5 соответственно. Когда вычисляется выражение-запятая $++i$, $i + j$ сначала инкрементируется i , затем вычисляется $i + j$, поэтому значение выражения равно 7. (И, конечно, i теперь имеет значение 2.) Приоритет оператора запятая меньше, чем у всех других операторов, между прочим, поэтому нет необходимости заключать в круглые скобки $++i$ и $i + j$. Иногда нам нужно связать целую серию выражений-запятых, так же как мы иногда связываем вместе присваивания. Оператор запятая является левым ассоциативным, поэтому компилятор интерпретирует

$i = 1, j = 2, k = i + j$ как $((i = 1), (j = 2), (k = (i + j)))$

Поскольку левый операнд в выражении-запятой вычисляется перед правым операндом, присваивания $i = 1$, $j = 2$ и $k = i + j$ выполняются слева направо. Оператор запятая предоставляется для ситуаций, где Си требует одно выражение, а мы хотели бы иметь два или более выражений. Другими словами, запятая позволяет нам "склеить" два выражения вместе, чтобы сформировать одно выражение. (Обратите внимание на сходство с составной инструкцией, которая позволяет рассматривать группу инструкций как одну инструкцию.) Необходимость склеивания выражений вместе возникает не так часто. Оператор запятая, как мы увидим в следующей главе, может использовать некоторые макроопределения. Инструкция `for` является единственным местом, где скорее всего встретится оператор запятая.

Например, предположим, что мы хотим инициализировать две переменные при записи выражения `for`. Вместо написания

```
sum = 0;
for ( i = 1; i <= n; i++ )
    sum += i;
```

МЫ МОЖЕМ НАПИСАТЬ

```
for ( sum = 0, i = 1; i <= n; i++)  
    sum += i;
```

Выражение `sum = 0, i = 1` сначала присваивает 0 `sum`, а затем присваивает 1 `i`. С дополнительными запятыми в инструкции `for` можно инициализировать более двух переменных.

ПРОГРАММА Печать таблицы квадратов (повторение)

Программа `square.c` (Глава 6) может быть улучшена путём преобразования цикла `while` в цикл `for`:

```
//square2.c
```

```
/* Выводит таблицу квадратов с помощью инструкции for */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i, n;
```

```
    printf("Эта программа печатает таблицу квадратов.\n");
```

```
    printf("Введите число записей в таблице: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++)
```

```
printf("%10d%10d\n", i, i * i);

return 0;
}
```

Мы можем использовать эту программу, чтобы проиллюстрировать важный момент в инструкции for: Си не налагает никаких ограничений на три выражения, которые управляют его поведением. Хотя эти выражения обычно инициализируют, тестируют и обновляют одну и ту же переменную, нет никакого требования, чтобы они были связаны каким-либо образом. Рассмотрим следующую версию одной и той же программы:

```
square3.c
/* Выводит таблицу квадратов с использованием метода odd
*/
#include <stdio.h>

int main(void)
{
    int i, n, odd, square;

    printf("Эта программа печатает таблицу квадратов.\n");
    printf("Введите число записей в таблице: ");
    scanf("%d", &n);

    i = 1;
    odd = 3;
    for (square = 1; i <= n; odd += 2) {
        printf("%10d%10d\n", i, square);
        ++i;
```

```
        square += odd;
    }

    return 0;
}
```

Инструкция `for` в этой программе инициализирует одну переменную (`square`), проверяет другую (`i`) и увеличивает третью (`odd`). i - это число, которое должно быть возведено в квадрат, `square` - квадрат i , а `odd` - нечётное число, которое должно быть добавлено к текущему квадрату, чтобы получить следующий квадрат (позволяя программе вычислять последовательные квадраты без выполнения каких-либо умножений) $((n+1)^2 = n^2 + 2n + 1)$. Огромная гибкость инструкции `for` иногда может быть полезна и мы увидим её большую помощь при работе со связанными списками. Инструкцию `for` можно легко использовать, но не переборщите. Цикл `for` в `square3.c` будет намного понятнее, если мы переставим его части так, чтобы цикл чётко контролировался `i`.

6.4 Выход из цикла

Мы видели, как писать циклы, у которых есть точка выхода перед телом цикла (с использованием инструкций `while` и `for`) или после него (с помощью инструкций `do`). Иногда, однако, нам нужен цикл с точкой выхода в середине. Нам может даже потребоваться, чтобы цикл имел более одной точки выхода. Инструкция `break` позволяет написать любой вид цикла. После того, как мы рассмотрим инструкцию `break`, мы

рассмотрим пару связанных инструкций: `continue` и `goto`.

Инструкция `continue` позволяет пропустить часть итерации цикла без выхода из цикла.

Инструкция `goto` позволяет программе переходить от одной инструкции к другой. Благодаря наличию таких инструкций, как `break` и `continue`, инструкция `goto` редко используется.

Инструкция `break`

Мы уже обсуждали, как инструкция `break` может передавать управление из инструкции `switch`. Инструкцию `break` можно также использовать для выхода из цикла `while`, `do` или `for`.

Предположим, что мы пишем программу, которая проверяет, является ли число n простым. Наш план состоит в том, чтобы написать инструкцию `for`, которая делит n на числа от 2 до $n-1$. Мы должны выйти из цикла, как только будет найден любой делитель; нет необходимости пробовать остальные возможности.

После завершения цикла мы можем использовать инструкцию `if`, чтобы определить, было ли преждевременное завершение (следовательно, n не простое) или нормальное (n простое):

```
for (d = 2; d < n; d++)
```

```
    if (n % d == 0)
```

```
        break;
```


261

```
if (d < n)
    printf("%d делится на %d\n",n,d);
else
    printf("%d -простое число \n", n);
```

Инструкция `break` особенно полезна для написания циклов, в которых точка выхода находится в середине тела, а не в начале или конце. Циклы, которые читают ввод пользователя, завершается при вводе определённого значения, часто попадают в эту категорию:

```
for (;;)
    (printf("Введите число (введите 0 для остановки):");
    scanf("%d", &n);
    if (n == 0)
        break;
    printf("%d cubed is %d\n", n, n * n * n);
```

Инструкция `break` передаёт управление из самой внутренней вложенной инструкции `while`, `do`, `for` или `switch`. Таким образом, когда эти инструкции являются вложенными, инструкция `break` может выйти только на один уровень

вложенности. Рассмотрим случай инструкции switch, вложенной внутрь инструкции while:

```
while (...){  
    (switch (...) {  
  
        ...  
  
        break;  
  
        ...  
    })  
}
```

Инструкция break передаёт управление из инструкции switch, но не из цикла while. Я вернусь к этому вопросу позже.

Инструкция continue

Инструкции continue здесь не место, потому что она не выводит из цикла. Тем не менее, она подобна break, таким образом, её включение в этот Раздел не является полностью произвольным, break передаёт управление в точку сразу после конца цикла, а continue передаёт управление в точку перед концом тела цикла. При break управление покидает цикл; с continue, управление остаётся внутри цикла. Существует ещё одна разница между break и continue: break может использоваться в инструкциях switch и циклах (while, do и

263

for), в то время как continue ограничена циклами. В следующем примере, читается ряд чисел и вычисляется их сумма и иллюстрируется простое использование continue. Цикл завершается, когда прочитано 10 ненулевых чисел. Всякий раз, когда читается число 0, выполняется инструкция continue, пропуская остальную часть тела цикла (инструкции sum += i; и n++;), но оставаясь внутри цикла.

```
n=0;
sum = 0;
while (n < 10){
    (scanf("%d", &i);
    if (i == 0)
        continue;
    sum += i;
    n++;
    /* continue прыгает сюда */
}
```

Без continue, мы могли бы написать пример следующим образом:

```
n = 0;
sum = 0;
while (n < 10) {
```

```
scanf("%d",&i);  
if (i!= 0){  
    sum += i;  
    n++;  
}  
}
```

Инструкция goto

break и continue являются инструкциями перехода, которые передают управление из одной точки программы в другую. Обе они ограничены: цель break - это точка, находящаяся непосредственно за концом замкнутого цикла, в то время как цель continue - это точка непосредственно перед концом цикла. С другой стороны, инструкция goto способна перескакивать на любую инструкцию в функции при условии, что у инструкции есть **метка**.

(C99 накладывает дополнительное ограничение на инструкции goto: её нельзя использовать для обхода объявления массива переменной длины.) Метка - это всего лишь идентификатор, размещённый в начале инструкции (которая будет выполняться после скачка):

метка: инструкция

Инструкция goto имеет форму

goto метка;

Выполнение инструкции goto L; передаёт управление в инструкцию, следующую за меткой L, которая должна находиться в той же самой функции, что и сама инструкция goto. (Переход в другую функцию невозможен. Метка может находиться как до так после инструкции goto. Например:

x=1;

loop1:

x++;

if(x <= 100) goto loop1;

).

goto может иметь несколько меток.(Например:

if (size >15)

goto a;

goto b;

a: len =30;

b: len =5;

)

Если бы у Си не было инструкции break, мы могли бы использовать инструкцию goto для преждевременного выхода из цикла:

for (d = 2; d < n; d++)

if (n%d== 0)

goto done;

266

done:

if ($d < n$)

printf("%d делится на %d\n", n, d);

else

printf("%d is prime\n", n);

Инструкция goto также полезна для выхода из вложенных циклов.

ПРОГРАММА Составление баланса чековой книжки

Многие простые интерактивные программы основаны на меню: они представляют пользователю список команд, которые можно выбрать.

Как только пользователь выбрал команду, программа выполняет требуемое действие, а затем предлагает пользователю другую команду. Этот процесс продолжается до тех пор, пока пользователь не выберет команду "exit" или "quit". Сердцем такой программы, очевидно, будет цикл. Внутри цикла будут инструкции, которые запрашивают у пользователя команду, читают команду и затем решают, какое действие выполнять:

```
for (;;) {  
    приглашение пользователю ввести команду ;  
    читать команду;  
    выполнить команду;  
}
```

Выполнение команды потребует инструкции switch (или каскадной инструкции if):

```
for (;;) {  
    приглашение пользователю ввести команду ;  
    считать команду;  
    switch (команда) {  
        case команда1: исполнить operation1; break;  
        case команда2: исполнить operation2; break;  
        .  
        .  
        .  
        case командаn: исполнить operationn; break;  
    }  
}
```

Чтобы проиллюстрировать эту схему, давайте разработаем программу, которая поддерживает баланс чековой книжки.

Программа предложит пользователю меню выбора: очистить баланс счета, зачислить деньги на счёт, дебетовать деньги со счета, отобразить текущий баланс и выйти из программы.

Выбор представлен целыми числами 0, 1, 2, 3 и 4 соответственно. Вот как будет выглядеть сессия с программой:

*** Составление баланса чековой книжки АСМЕ ***

Команды: 0 = очистить, 1 = кредит, 2 = дебит,

3 = баланс, 4 = выход.

Введите команду: 1

Введите сумму кредита: 1042.56

Введите команду: 2

Введите сумму дебета: 133.79

Введите команду: 1

Введите сумму кредита: 1754.32

Введите команду: 2

Введите сумму дебета: 1400

Введите команду: 2

Введите сумму дебета: 68

Введите команду: 2

Введите сумму дебета: 50

Введите команду: 3

Текущий баланс: \$1145.09

Введите команду: 4

Когда пользователь вводит команду 4 (выход), программа должна выйти из инструкции switch и окружающего цикла. Инструкция break не поможет, и мы бы предпочли не использовать инструкцию goto. Вместо этого мы используем в программе инструкцию return, которая заставит функцию main возвратиться в операционную систему.

```
//checking.c
```

```
/* Balances a checkbook */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int cmd;
```

```
    float balance = 0.0f, credit, debit;
```

```
    printf("*** ACME checkbook-balancing program ***\n");
```

```
    printf("Commands: 0=clear, 1=credit, 2=debit, ");
```

```
    printf("3=balance, 4=exit\n\n");
```

```
for (;;) {  
    printf("Enter command: ");  
    scanf("%d", &cmd);  
    switch (cmd) {  
        case 0:  
            balance = 0.0f;  
            break;  
        case 1:  
            printf("Enter amount of credit: ");  
            scanf("%f", &credit);  
            balance += credit;  
            break;  
        case 2:  
            printf("Enter amount of debit: ");  
            scanf("%f", &debit);  
            balance -= debit;  
            break;  
        case 3:  
            printf("Current balance: $%.2f\n", balance);
```

```
break;
case 4:
    return 0;
default:
    printf("Commands: 0=clear, 1=credit, 2=debit, ");
    printf("3=balance, 4=exit\n\n");
    break;
}
}
}
```

Обратите внимание, что за инструкцией `return` не следует инструкция `break`. Невозможно выполнить `break` сразу после `return`, и многие компиляторы выдадут предупреждающее сообщение.

6.5 Инструкция Null

Инструкция может быть пустой (`null`) - лишённой символов, кроме точки с запятой в конце. Вот пример:

```
i = 0; ; j = 1;
```

Эта строка содержит три инструкции: присваивание *i*, инструкцию `null` и присваивание *j*. Инструкция `null` в первую очередь хороша для написания циклов, тела которых

пусты. В качестве примера напомним цикл поиска простых чисел в Разделе 6.4:

```
for (d = 2; d < n; d++)
```

```
    if( n % d == 0 )
```

```
        break;
```

Если мы переместим условие $n \% d == 0$ в управляющее выражение цикла, тело цикла станет пустым:

```
for ( d = 2 ; d < n && n % d !=0; d++ )
```

```
    ;    /* пустое тело цикла */
```

Каждый раз по циклу сначала проверяется условие $d < n$; если это ложь, цикл завершается.

В противном случае проверяется условие $n \% d != 0$, и если оно ложно, цикл завершается.

(В последнем варианте $n \% d == 0$ должно быть истинным, другими словами, мы нашли делитель ***n***). Обратите внимание, что мы поместили null- инструкцию в строку отдельно, вместо того, чтобы писать

```
for ( d=2; d < n && n % d !=0; d++ );
```

Программисты Си обычно по умолчанию ставят null-инструкцию на отдельной строке. В противном случае, кто-то, читающий программу, может запутаться в том, была ли инструкция после for для его тела:

```
for ( d=2; d < n && n % d != 0; d++);
```

```
    if (d < n)
```

```
printf("%d is divisible by %d\n",n, d);
```

Преобразование обычного цикла в цикл с пустым телом мало что даёт: новый цикл часто более краток, но обычно не более эффективен. Однако в некоторых случаях цикл с пустым телом явно превосходит альтернативы. Например, эти циклы удобны для чтения символьных данных. Случайно поставленная точка с запятой после круглых скобок в `if`, `while`, или `for` создаёт инструкцию `null`, тем самым преждевременно завершая `if`, `while` или `for`.

В инструкции `if`, поставив точку с запятой после круглых скобок, получим инструкцию `if`, которая, очевидно, выполняет те же действия независимо от значения её управляющего выражения:

```
if (d == 0);    /*** WRONG ***/  
    printf("Error: Division by zero\n");
```

Вызов `printf` не находится внутри инструкции `if`, поэтому он выполняется независимо от того, равна ли `d` нулю.

В инструкции `while`, помещая точку с запятой после круглых скобок, можно создать бесконечный цикл:

```
i = 10;  
  
while ( i > 0 );    /*** WRONG ***/  
{  
    printf("Т минус %d и подсчёт\n", i);  
    --i;
```

```
}
```

Другой вариант состоит в том, что цикл завершается. Но инструкция, которая должна быть телом цикла, выполняется только один раз, после завершения цикла:

```
i = 11;  
while (--i > 0); /*** WRONG ***/  
printf("Т минус %d и подсчёт\n", i);
```

В этом примере выводится сообщение
Т минус 0 и подсчёт

В выражении for, при поставленной точке с запятой после круглых скобок, инструкция, которая должна быть телом цикла, выполнится только один раз:

```
for (i = 10; i > 0; i-- ) ; /*** WRONG ***/  
printf("Т минус %d и подсчёт\n", i);
```

В этом примере также выводится сообщение
Т минус 0 и подсчёт

Вопросы и ответы

Вопрос: В Разделе 6.1 показан следующий цикл:

```
while (i > 0)
```

```
    printf("Т минус %d и подсчет\n", i);
```

Почему бы не сократить цикл ещё больше, удалив тест
" > 0"?

```
while (i)
```

```
    printf("Т минус %d и подсчет\n", i);
```

Эта версия остановится, когда *i* достигнет 0, поэтому она должна быть фактически как оригинал.

Ответ: Новая версия, безусловно, более кратка, и многие программисты Си будут писать цикл именно таким образом. Хотя, у него есть недостатки.

Во-первых, новый цикл не так прост для чтения, как оригинал. Ясно, что цикл завершится, когда *i* достигнет 0, но не очевидно, считаем ли мы вверх или вниз. В исходном цикле эта информация может быть выведена из управляющего выражения, *i* > 0.

Во-вторых, новый цикл будет иметь отличие от исходного, если *i* имеет отрицательное значение, когда цикл начнёт выполняться. Исходный цикл немедленно завершится, а новый цикл нет.

Вопрос: В Разделе 6.3 говорится, что, за исключением редких случаев, циклы `for` могут быть преобразованы в циклы `while` с

использованием стандартного шаблона. Можете ли вы привести пример такого случая?

Ответ: Когда тело цикла for содержит инструкцию continue, шаблон while, показанный в Разделе 6.3, больше недействителен. Рассмотрим следующий пример из раздела 6.4:

```
n = 0;
sum = 0;
while (n < 10){
    (scanf("%d", &i);
    if ( i == 0 )
        continue;
    sum += i;
    n++;
}
```

На первый взгляд кажется, что мы можем преобразовать цикл while в цикл for:

```
sum = 0;
for (n = 0; n < 10; n++)
    (scanf("%d", &i);
    if (i == 0)
        continue;
sum += i;
```


К сожалению, этот цикл не эквивалентен оригиналу. Когда i равно 0, исходный цикл не увеличивает n , а новый цикл увеличивает.

Вопрос: Какая форма бесконечного цикла предпочтительнее: while (1) или for (;;)?

Ответ: Программисты Си традиционно предпочитали for (;;) по соображениям эффективности, старые компиляторы часто заставляли программы каждый раз проверять условие 1 в цикле while. Однако в современных компиляторах не должно быть никакой разницы в производительности.

Вопрос: Я слышал, что программисты никогда не должны использовать инструкцию continue. Это правда?

Ответ: Верно, что инструкции continue встречаются редко. Тем не менее, continue бывает иногда удобна. Предположим, что мы пишем цикл, который считывает некоторые входные данные, проверяет их правильность и, если это так, обрабатывает вход каким-либо способом. Если есть несколько тестов на достоверность, или они сложны, continue может быть полезным.

Цикл выглядел бы примерно так:

```
for ( ;; ) {
    чтение данных
    if (данные не проходят первый тест)
        continue;
    if ( данные не проходят второй тест)
        continue;
    .
```

```

.
.
if ( данные не проходят последний тест)
    continue;
    обработать данные
}

```

Вопрос: Что плохого в инструкции goto?

Ответ: Инструкция goto по сути не является "злом", просто у нас обычно есть лучшие альтернативы. Программы, которые используют много инструкций goto, могут быстро выродиться в "спагетти-код", при этом управление беспечно перескакивает туда-сюда. Спагетти-код трудно понять и его трудно модифицировать, инструкции goto затрудняют чтение программ, потому что они могут прыгать либо вперёд либо назад (напротив, break и continue переходят только вперёд).

Программа, которая содержит инструкции goto, часто требует, чтобы внимание читателя прыгало вперёд и назад, пытаясь уследить за потоком управления, инструкции goto могут сделать программы трудно модифицируемыми, поскольку они позволяют фрагменту кода выполнять несколько предназначений. Например, инструкция, которой предшествует метка, может быть достижима либо "проваливаясь" из предыдущей инструкции, либо выполнив одну из нескольких инструкций goto.

Вопрос: Имеет ли инструкция null какие-либо применения, кроме указания на то, что тело цикла пусто?

Ответ: Очень мало. Так как инструкция `null` может появляться везде, где разрешена инструкция, существует много потенциальных применений для инструкции `null`. На практике, однако, есть только одно использование инструкции `null`, и оно достаточно редкое. Предположим, что нам нужно поместить метку в конце составного выражения. Метка не может стоять одна; она всегда должна сопровождаться инструкцией. Помещение выражения `null` после метки решает проблему:

```
...
{
    goto end_of_stmt;
    ...
    end_of_stmt: ;
}
```

Вопрос: Существуют ли какие-либо другие способы обозначить пустое тело цикла, помимо того, что инструкция `null` будет стоять на отдельной строке?

Ответ: Некоторые программисты используют инструкцию `continue` в холостую:

```
for ( d = 2; d < n && n % d != 0; d++ )
    continue;
```

Другие используют пустую составную инструкцию:

```
for ( d = 2; d < n && n % d != 0; d++ )
    {}
```

Упражнения

К разделу 6.1

1. Какой результат даёт следующий фрагмент программы?

```
i= 1;  
while ( i <= 128 ){  
    printf ( " %d ", i);  
    i *= 2;  
}
```

К разделу 6.2

2. Какой результат даёт следующий фрагмент программы?

```
i= 9384;  
do {  
    printf ( "%d", i );  
    i /= 10;  
  
} while ( i > 0 );
```

К разделу 6.3

*3. Какой результат даёт следующий фрагмент программы?

281

```
For ( i = 5, j = i - 1; i > 0, j > 0; --i, j = i - 1)
printf("%d ", i);
```

W 4. Какое из следующих утверждений не эквивалентно двум другим (при условии, что тела циклов одинаковы)?

- (a) for (i = 0; i < 10; i++) ...
- (b) for (i = 0; i < 10; ++i) ...
- (c) for (i = 0; i++ < 10;)...

5. Какое из следующих утверждений не эквивалентно двум другим (при условии, что тела циклов одинаковы)?

- (a) while (i < 10) {...}
- (b) for (; i < 10;) {...}
- (e) do {...} while (i < 10);

6. Переведите фрагмент программы из упражнения 1 в одиночную инструкцию for.

7. Переведите фрагмент программы из упражнения 2 в одну инструкцию for.

*8. Какой вывод производит следующая инструкция for?

```
for ( i = 10; i >= 1; i /= 2 )  
    printf ("%d ", i++);
```

9. Переведите инструкцию for упражнения упражнения 3 в эквивалентную инструкцию while. Вам понадобится одна инструкция в дополнение к самому циклу while.

К разделу 6.4

W 10. Покажите, как заменить инструкцию continue эквивалентной инструкцией goto.

11. Какой выход будет создан следующим фрагментом программы?

```
sum = 0;  
for (i = 0; i < 10; i++) {  
    if (i % 2)  
        continue;  
    sum += 1;  
}  
printf (" %d\n", sum);
```

W12. Следующий цикл "поиск простых чисел" появился в Разделе 6.4 в качестве примера:

```
for (d = 2; d < n; d++)  
    if (n % d == 0)
```

```
break;
```

Этот цикл не очень эффективен. Нет необходимости делить n на все числа между 2 и $n - 1$, чтобы определить, является ли оно простым. Фактически нам нужно проверять только делители до квадратного корня из n . Измените цикл, чтобы воспользоваться этим фактом. Подсказка: не пытайтесь вычислить квадратный корень из n ; вместо этого сравните $d * d$ с n .

К разделу 6.5

* 13. Перепишите следующий цикл так, чтобы его тело было пустым:

```
for (n = 0; m > 0; n++)  
    m /= 2;
```

W *14. Найдите ошибку в следующем фрагменте программы и исправьте её.

```
if (n % 2 == 0);  
    printf ("n is even\n");
```

Проекты по программированию

1. Напишите программу, которая найдёт наибольшее из серии чисел, введённых пользователем. Программа должна предлагать пользователю вводить числа по одному. Когда

284

пользователь вводит 0 или отрицательное число, программа должна отображать наибольшее введенное неотрицательное число:

Enter a number: 60

Enter a number: 38.3

Enter a number: 4.89

Enter a number: 100.62

Enter a number: 75.2295

Enter a number: 0

The largest number entered was 100.62

Обратите внимание, что числа не обязательно целые.

W2. Напишите программу, которая попросит пользователя ввести два целых числа, затем вычисляет и отображает их наибольший общий делитель (НОД):

Введите два целых числа: 12 28

Наибольший общий делитель: 4

Подсказка: классический алгоритм вычисления НОД, известный как алгоритм Евклида, выглядит следующим образом:

Пусть m и n - переменные, содержащие два числа. Если n равно 0, стоп: m является НОД. В противном случае

вычислите остаток, при делении m на n . Скопируйте n в m и скопируйте остаток в n . Затем повторите процесс, начиная с проверки, является ли n 0.

3. Напишите программу, которая попросит пользователя ввести дробь, а затем приводит её к несократимому виду :

Введите дробь: 6/12

В несократимом виде: 1/2

Подсказка: Чтобы привести дробь к несократимому виду, сначала вычислите НОД числителя и знаменателя. Затем разделите числитель и знаменатель на НОД.

W4. Добавьте цикл в программу broker.c раздела 5.2, чтобы пользователь мог ввести более одной сделки и программа будет рассчитывать комиссионные на каждой сделке.

Программа должна завершаться, когда пользователь вводит 0 в качестве стоимости сделки:

Введите стоимость сделки: 30000

Комиссия: \$166,00

Введите стоимость сделки: 20000

Комиссия: \$144,00

Введите стоимость сделки: 0

5. Проект по программированию 1 в Главе 4 предлагал написать программу, которая отображает двузначное число с обращёнными цифрами. Обобщите программу так, чтобы число могло иметь одну, две, три или более цифры.

Подсказка: используйте цикл `do`, который неоднократно делит число на 10, останавливаясь, когда частое станет равно 0.

W6. Напишите программу, которая предложит пользователю ввести число `n`, а затем распечатает все чётные квадраты между 1 и `n`. Например, если пользователь вводит значение 100, программа должна распечатать следующее:

4
16
36
64
100

7. Перестройте программу `square3.c` так, чтобы цикл `for` инициализировал `i`, тестировал `i` и увеличивал `i`. Не переписывайте программу; в частности, не используйте никаких умножений.

W8. Напишите программу, которая печатает календарь на один месяц. Пользователь указывает количество дней в месяце и в день недели, с которого начинается месяц:

Введите количество дней в месяце: 31

Введите начальный день недели (1 = Воскресенье, 7 = Суббота): 3

1 2 3 4 5
6 7 8 9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26

27 28 29 30 31

Подсказка. Эта программа не так сложна, как кажется. Наиболее важная часть - это инструкция `for`, который использует переменную `i` для подсчёта от 1 до `n`, где `n` - количество дней в месяце и печатает каждое значение `i`. Внутри цикла инструкция `if` проверяет, является ли `i` последним днём недели; если это так, то печатает символ новой строки.

9. Проект по программированию 8 в Главе 2 предлагал написать программу, которая вычисляет остаток по кредиту после первого, второго и третьего ежемесячных платежей. Измените программу, чтобы она также предлагала пользователю ввести количество платежей, а затем отображала остаток после каждого из этих платежей.

10. Проект по программированию 9 в Главе 5 предлагал написать программу, которая определяет, какая из двух дат наступает раньше в календаре. Обобщите программу, чтобы пользователь мог ввести любое количество дат. Пользователь будет вводить 0/0/0, чтобы указать, что больше не будет введённых дат:

Введите дату (мм / дд / гг): 3/6/08

Введите дату (мм / дд / гг): 5/17/07

Введите дату (мм / дд / гг): 6/3 / 07

Введите дату (мм / дд / гг): 0/0/0

5/17/07 - самая ранняя дата

11. Величина математической константы **e** может быть выражена в виде бесконечного ряда:

$$e = 1 + 1/1! + 1/2! + 1/3! + \dots$$

Напишите программу, которая аппроксимирует **e**, вычисляя значение

$$1 + 1/1! + 1/2! + 1/3! + \dots + 1/n!$$

Где n - целое число, введённое пользователем.

12. Измените проект по программированию 11 так, чтобы программа продолжала добавлять члены до тех пор, пока текущий член не станет меньше ϵ , где ϵ - это небольшое число (с плавающей запятой), введённое пользователем.

7. Основные типы

Не заблуждайтесь: компьютеры обрабатывают числа, а не символы. Мы измеряем своё понимание (и контроль) степенью арифметизации деятельности.

До сих пор мы использовали только два базовых (встроенных) типа Си: `int` и `float`. (Мы также видели тип `_Bool`, который является базовым типом в C99.) В этой главе описываются остальные основные типы и обсуждаются важные вопросы о типах в целом.

В Разделе 7.1 показан полный диапазон целочисленных типов, которые включают длинные целые числа (`long integers`), короткие целые числа (`short integers`) и целые числа без знака (`unsigned integers`).

В Разделе 7.2 представлены двойные (`double`) и длинные двойные (`long double`) типы, которые обеспечивают больший диапазон значений и большую точность, чем `float`.

Раздел 7.3 охватывает тип `char`, который нам понадобятся для работы с символьными данными.

В Разделе 7.4 рассматривается сложная тема преобразования значения одного типа в эквивалентное значение другого.

В Разделе 7.5 показано, как использовать `typedef` для определения новых имён типов.

Наконец, в Разделе 7.6 описывается оператор `sizeof`, который измеряет объем памяти, необходимый для типа.

7.1. Целочисленные типы

Си поддерживает два принципиально различных типа числовых типов: целочисленные типы и типы чисел с плавающей точкой (*запятой по-русски*), значения целочисленного типа - целые числа, в то время как значения типа чисел с плавающей точкой могут также иметь дробную часть. Целочисленные типы, в свою очередь, делятся на две категории: знаковые и беззнаковые.

Знаковые и беззнаковые целые числа

Самый левый бит целого числа со знаком (называемого знаковым битом) равен 0, если число положительное, и 1, если оно отрицательное.

Таким образом, наибольшее 16-битное целое число имеет двоичное представление

0111111111111111

которое имеет значение 32 767 ($2^{15}-1$).

Самое большое 32-битное целое число

01111111111111111111111111111111

которое имеет значение 2 147 483 647 ($2^{31}-1$).

Целое число, без знакового бита (самый левый бит считается частью числа), считается беззнаковым. Наибольшее 16-

разрядное целое число без знака равно $65535 (2^{16} - 1)$, а наибольшее 32-разрядное целое число без знака - $4\ 294\ 967\ 295 (2^{32} - 1)$.

По умолчанию, целочисленные переменные являются знаковыми в Си - самый левый бит зарезервирован для знака. Чтобы сообщить компилятору, что переменная не имеет знакового бита, мы объявляем её беззнаковой (`unsigned`). Беззнаковые числа в первую очередь полезны для системного программирования и низкоуровневых машинно-зависимых приложений. Мы обсудим типичные приложения для чисел без знака в Главе 20: до тех пор мы, как правило, будем избегать их.

Целочисленные типы имеют разные размеры. Тип `int` обычно имеет 32 бита, но может быть 16 бит для более старых процессоров. Поскольку некоторым программам требуются числа, которые слишком велики для хранения в форме `int`, Си также предоставляет длинные целые числа (`long integers`). Иногда нам может понадобиться сэкономить память, поручив компилятору хранить число в меньшем пространстве, чем обычно; такое число называется коротким целым числом (`short integer`). Чтобы построить целочисленный тип, который точно соответствует нашим потребностям, мы можем указать, что переменная является длинной или короткой, знаковой или беззнаковой (`long` или `short`, `signed` или `unsigned`). Мы даже можем комбинировать спецификаторы (например, `long unsigned int`). Однако, только следующие шесть комбинаций фактически представляют разные типы:

`short int`

`unsigned short int`

`int`

unsigned int

long int

unsigned long int

Другие комбинации являются синонимами для одного из этих шести типов (например, long signed int является тем же самым, что и long int, поскольку целые числа всегда знаковые, если не указано иное.) Кстати, порядок спецификаторов не имеет значения; unsigned short int является тем же, что и short unsigned int. Си, позволяет нам сокращать имена целочисленных типов, отбрасывая слово int. Например, unsigned short int может быть сокращён до unsigned short, а long int может быть сокращён до long. Опускание int - распространённая практика среди программистов на Си, а некоторые новые языки основанные на Си (включая Java) фактически требуют, чтобы программист писал short или long, а не short int или long int. По этим причинам я часто опускаю слово int, когда оно не является строго необходимым.

Диапазон значений, представленных каждым из шести целых типов, варьируется от одной машины к другой.

Однако существует несколько правил, которым должны подчиняться все компиляторы.

Во-первых, стандарт Си требует, чтобы short int, int и long int каждый покрывал определённый минимальный диапазон значений (подробнее см. Раздел 23.2).

Во-вторых, стандарт требует, чтобы int не был короче, чем short int, а long int не был короче, чем int. Однако возможно, что short int представляет тот же диапазон значений, что и int; кроме того, int может иметь тот же диапазон, что и long int. В таблице 7.1 показан обычный диапазон значений для целочисленных типов на 16-разрядной машине; Обратите внимание, что short int и int имеют одинаковые диапазоны.

Тип	Наименьшее значение	Наибольшее значение
short int	-32 768	32 767
unsigned short int	0	65 535
int	-32 768	32 767
unsigned int	0	65 535
long int	-2 147 483 648	2 147 483 647
unsigned long int	0	4 294 967 295

Таблица 7.1. Целочисленные типы на 16-разрядной машине

В таблице 7.2 показаны обычные диапазоны на 32-битной машине, здесь int и long int имеют одинаковые диапазоны.

Тип	Наименьшее значение	Наибольшее значение
short int	-32 768	32 767
unsigned short int	0	65 535
int	-2 147 483 648	2 147 483 647
unsigned int	0	4 294 967 295
long int	-2 147 483 648	2 147 483 647

unsigned long int	0	4 294 967 295
-------------------	---	---------------

Таблица 7.2. Целочисленные типы на 32-разрядной машине

В последние годы 64-битные процессоры стали более распространёнными. В таблице 7.3 показаны типичные диапазоны для целочисленных типов на 64-битной машине (особенно в UNIX).

Тип	Наименьшее значение	Наибольшее значение
short int	-32 768	32 767
unsigned short int	0	65 535
int	-2 147 483 648	2 147 483 647
unsigned int	0	4 294 967 295
long int	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
unsigned long int	0	18 446 744 073 709 551 615

Таблица 7.3. Целочисленные типы на 64-разрядной машине

ещё раз, позвольте мне подчеркнуть, что диапазоны, показанные в таблицах 7.1, 7.2 и 7.3, не предусмотрены стандартом Си и могут варьироваться от одного компилятора к другому. Один из способов определить диапазоны целочисленных типов для конкретной реализации - проверить

заголовок `<limits.h>` (23.2), который является частью стандартной библиотеки. Этот заголовок определяет макросы, которые представляют наименьшие и наибольшие значения каждого целочисленного типа.

Целочисленные типы в C99

C99 предоставляет два дополнительных стандартных целочисленных типа: `long long int` и `unsigned long long int`. Эти типы были добавлены из-за растущей потребности в очень больших целых числах и способности новых процессоров поддерживать 64-разрядную арифметику. Оба длинных типа должны иметь ширину не менее 64 бит, поэтому диапазон длинных значений `int` обычно составляет

-2^{63} (-9 223 372 036 854 775 808)

до $2^{63} - 1$ (9 223 372 036 854 775 807),

а диапазон значений `unsigned long long int` обычно составляет от 0 до $2^{64} - 1$

(18 446 744 073 709 551 615).

Типы `short int`, `int`, `long int` и `long long int` (наряду с типом `signed char`) называются стандартными знаковыми целочисленными типами в C99. `unsigned short int`, `unsigned int`, `unsigned long int` и `unsigned long long int` типы (наряду с типами `unsigned char` и `_Bool`) называются стандартными

беззнаковыми целочисленными типами. В дополнение к стандартным целочисленным типам стандарт C99 допускает определяемые реализацией расширенные целочисленные типы, как знаковые (signed), так и беззнаковые unsigned. Например, компилятор может предоставить знаковые и беззнаковые 128-битные целочисленные типы.

Целочисленные константы

Обратим внимание на константы - числа, которые появляются в тексте программы, а не числа, которые читаются, записываются или вычисляются. Си позволяет целочисленным константам быть записанными в десятичной (основание 10), восьмеричной (основание 8) или шестнадцатеричной (основание 16).

Восьмеричные и шестнадцатеричные числа

Восьмеричное число записывается с использованием только цифр от 0 до 7. Каждая позиция в восьмеричном числе представляет степень 8 (так же, как каждая позиция в десятичном числе представляет степень 10). Таким образом, восьмеричное число 237 представляет десятичное число $2 \times 8^2 + 3 \times 8^1 + 7 \times 8^0 = 128 + 24 + 7 = 159$.

Шестнадцатеричное число записывается с использованием цифр от 0 до 9 плюс буквы от A до F, которые обозначают от 10 до 15, соответственно. Каждая позиция в

шестнадцатеричном числе представляет степень 16;
 Шестнадцатеричное число 1AF имеет десятичное значение
 $1 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 = 256 + 160 + 15 = 431$.

■ Десятичные константы содержат цифры от 0 до 9, но не должны начинаться с нуля:

15 255 32767

■ Восьмеричные константы содержат только цифры от 0 до 7 и должны начинаться с нуля:

017 0377 077777

■ Шестнадцатеричные константы содержат цифры от 0 до 9 и буквы между a и f и всегда начинаются с 0x:

0xf 0xff 0x7f f f

Буквы в шестнадцатеричной константе могут быть в верхнем или нижнем регистре: 0xff 0xfF 0xFf 0xFF 0Xff 0XfF 0XFf 0XFF

Имейте в виду, что восьмеричные и шестнадцатеричные числа представляют собой не более чем альтернативный способ написания чисел; они не влияют на то, как числа фактически хранятся.

(Целые числа всегда хранятся в двоичной форме, независимо от того, какую нотацию мы использовали для их выражения.)

Мы можем в любой момент переключиться с одной нотации на другую и даже смешать их сумма: $10 + 015 + 0x20$ имеет значение 55 (десятичное). Восьмеричная (octal) и шестнадцатеричная (hex) нотации наиболее удобны для написания программ низкого уровня; мы почти не будем использовать эти обозначения до Главы 20.

Тип десятичной целочисленной константы - это обычно `int`. Однако, если значение константы слишком велико для хранения в качестве `int`, константа вместо этого имеет тип `long int`.

В маловероятном случае, когда константа слишком велика для хранения в качестве `long int`, компилятор попытается использовать `unsigned long int` в качестве последнего средства.

Правила определения типа восьмеричной или шестнадцатеричной константы немного отличаются: компилятор будет проходить через типы `int`, `unsigned int`, `long int` и `unsigned long int` пока не найдёт тип, способный представлять константу. Чтобы заставить компилятор обрабатывать константу как длинное целое число (`long integer`), просто следуйте ей с буквой `L` (или `l`): `15L` `0377L`. `0x7fffL`. Чтобы указать, что константа без знака, поставьте после неё букву `U` (или `u`): `15U` `0377U` `0x7fffU` и `U` можно использовать в комбинации, чтобы показать, что константа является `long` и `unsigned`: `0xffffffffUL`. (Порядок `L` и `U` не имеет значения, как и их регистр.)

Целочисленные константы в C99

В C99 целочисленные константы, которые заканчиваются на

LL или ll (регистр двух букв должен совпадать), имеют тип long long int. Добавление буквы U (или u) до или после LL или ll обозначает константу типа unsigned long long int. Общие правила C99 для определения типа целочисленной константы немного отличаются от правил в C89. Тип десятичной константы без суффикса (U, u, L, l, LL или ll) является "наименьшим" из типов int, long int или long long int, которые могут представлять значение этой константы. Для восьмеричной или шестнадцатеричной константы, однако, списком возможных типов являются int, unsigned int, long int, unsigned long int, long long int и unsigned long long int, в указанном порядке.

Любой суффикс в конце константы изменяет список возможных типов. Например, константа, которая заканчивается символом U (или u), должна иметь один из типов unsigned int, unsigned long int или unsigned long long int. Десятичная константа, которая заканчивается на L (или l), должна иметь один из типов long int или long long int. Также существует условие, чтобы константа имела расширенный целочисленный тип, если она слишком велика для представления с использованием одного из стандартных целочисленных типов.

Переполнение целых чисел

Когда арифметические операции выполняются над целыми числами, вполне возможно, что результат будет слишком большим для представления. Например, когда арифметическая операция выполняется с двумя значениями типа `int`, результат должен быть представлен как `int`. Если результат не может быть представлен как `int` (потому что он требует слишком много бит), мы говорим, что произошло переполнение. Поведение при целочисленном переполнении происходит в зависимости от того, были ли операнды знаковыми или беззнаковыми. Когда переполнение происходит во время операции над целыми со знаком, поведение программы не определено. Напомним из раздела 4.4, что последствия неопределённого поведения могут различаться. Скорее всего, результат операции будет просто неправильным, но программа может аварийно завершиться или демонстрировать другое нежелательное поведение. Когда переполнение происходит во время операции над целыми числами без знака, тем не менее, результат определяется: мы получим правильный ответ по модулю 2^n , где n - количество битов, используемых для хранения результата.

Например, если мы добавим 1 к 16-разрядному числу без знака 65 535, результат будет гарантированно равен 0.

Чтение и запись целых чисел

Предположим, что программа не работает, потому что одна из её переменных `int` переполнена. Наша первая мысль - изменить тип переменной с `int` на `long int`. Однако это ещё не всё - нам нужно посмотреть, как это изменение повлияет на остальную часть программы. В частности, нужно проверить, используется ли переменная в вызове `printf` или `scanf`. Если это так, строка формата в вызове должна быть изменена, так как преобразование `%d` работает только для типа `int`. Чтение и запись целых чисел `unsigned`, `short` и `long` требует иных спецификаций преобразования:

■ Когда вы читаете или пишете беззнаковое целое, используйте букву `u`, `o` или `x` вместо `d` в спецификации преобразования. Если присутствует спецификатор `u`, число читается (или записывается) в десятичной системе счисления; `o` обозначает восьмеричную нотацию, а `x` обозначает шестнадцатеричную запись.

`unsigned int u;`

`scanf("%u", &u); /* читает u в базе 10 */`

`printf("%u", u); /* записывает u в базе 10 */`

`scanf("%o", &u); /* читает u в базе 8 */`

`printf("%o", u); /* записывает u в базе 8 */`

302

```
scanf("%x", &u); /* читает u в базе 16 */  
printf("%x", u); /* записывает u в базе 16 */
```

■ Когда вы читаете или пишете целое число short, ставьте букву h перед d, o, u или x:

```
short s;  
scanf("%hd",&s);  
printf("%hd", s);
```

■ Когда вы читаете или пишете длинное целое, перед буквами d, o, u или x ставьте букву l (l - 'л', а не единица - 1):

```
long l;  
scanf("%ld",&l);  
printf("%ld", l);
```

■ Когда вы читаете или пишете long long integer (только C99), поместите буквы ll перед d, o, u или x:

```
long long ll;  
scanf("%lld",&ll);  
printf("%lld",ll);
```

ПРОГРАММА суммирования ряда чисел (повторение)

В Разделе 6.1. Мы написали программу, суммирующую ряд целых чисел, введенных пользователем. Одна из проблем этой программы заключается в том, что сумма (или одно из входных чисел) может превышать наибольшее допустимое значение для переменной типа `int`.

Вот что может произойти, если программа запущена на машине, где целые числа имеют длину 16 бит:

Эта программа суммирует ряд целых чисел.

Введите целые числа (от 0 до): 10000 20000 30000

Сумма: -5536

Сумма составила 60 000, что не соответствует переменной типа `int`, поэтому произошло переполнение. Когда происходит переполнение с числами со знаком, результат не определен. В этом случае мы получили явно бессмысленное число. Чтобы улучшить программу, давайте переключимся на переменные типа `long`.

//sum2.c

/ Суммирует ряд чисел (используя переменные типа long) */*

304

```
#include <stdio.h>
```

```
int main(void){
```

```
    long n, sum = 0;
```

```
    printf("Эта программа суммирует ряд целых чисел.\n");
```

```
    printf("Введите целые числа (от 0 до завершения): ");
```

```
    scanf("%ld", &n);
```

```
    while (n != 0) {
```

```
        sum += n;
```

```
        scanf("%ld", &n);
```

```
    }
```

```
    printf("The sum is: %ld\n", sum);
```

```
    return 0;
```

```
}
```

Изменение было довольно простым: мы объявили ***n*** и *sum* переменными *long* вместо переменных *int*, затем мы изменили спецификации преобразования в *scanf* и *printf* на *%ld* вместо *%d*.

7.2 Типы чисел с плавающей точкой

Целочисленные типы не подходят для всех приложений. Иногда нам нужны переменные, которые могут хранить числа с цифрами после десятичной точки или числа, которые являются чрезвычайно большими или маленькими. Такие числа хранятся в формате с плавающей точкой (называемым так, потому что десятичная точка "плавает"). Си предоставляет три типа чисел с плавающей точкой, соответствующих различным форматам с плавающей точкой:

float	одиночная точность чисел с плавающей точкой
double	двойная точность чисел с плавающей точкой
long double	расширенная точность чисел с плавающей точкой

float подходит, когда высокая точность не является критичной (например, расчёт температуры до одной десятой), double обеспечивает большую точность - почти для большинства программ. long double, который обеспечивает максимальную точность, используется редко.

В стандарте Си не указано, насколько точны типы float, double и long double, поскольку разные компьютеры могут хранить числа с плавающей точкой разным способом. Большинство современных компьютеров соответствуют спецификациям стандарта IEEE 754 (также известного как IEC 60559), поэтому мы будем использовать его в качестве примера.

Стандарт чисел с плавающей точкой IEEE

Стандарт IEEE 754, разработанный Институтом инженеров по электротехнике и электронике, обеспечивает два основных формата чисел с плавающей точкой: одинарная точность (32 бита) и двойная точность (64 бита). Числа хранятся в форме научной нотации, причём каждое число состоит из трёх частей: знака, экспоненты и дробной части.

Количество бит, зарезервированных для экспоненты (*показателя степени*), определяет, насколько большими (или маленькими) могут быть числа, в то время как количество бит в дробной части определяет точность. В формате с одинарной точностью экспонента имеет длину 8 бит, а дробная часть занимает 23 бита. В результате число одинарной точности имеет максимальное значение приблизительно $3,40 \times 10^{38}$ с точностью около 6 десятичных цифр. Стандарт IEEE также описывает два других формата, одинарную расширенную точность и двойную расширенную точность.

Стандарт не определяет количество бит в этих форматах, хотя он требует, чтобы одинарный расширенный тип занимал не менее 43 бит, а удвоенный расширенный тип - не менее 79 бит. Более подробную информацию о стандарте IEEE и арифметике чисел с плавающей точкой можно найти в статье "Что каждый компьютерный специалист должен знать об арифметике чисел с плавающей точкой" Дэвида Голдберга (ACM Computing Surveys, том 23. № 1 (март 1991 г.): 5-48).

В таблице 7.4 приведены характеристики типа чисел с плавающей точкой при реализации в соответствии со стандартом IEEE. (В таблице показаны наименьшие положительные нормированные значения. Субнормальные числа могут быть меньше.) Тип long double не отображается в таблице, так как его длина варьируется от одной машины к другой, причём 80 бит и 128 бит являются наиболее распространёнными размерами.

Тип	Наименьшее положительно е значение	Наибольшее значение	Точность
float	1.17549×10^{-38}	3.40282×10^{38}	6 цифр
double	2.22507×10^{-308}	1.79769×10^{308}	15 цифр

Таблица 7.4 характеристики типа чисел с плавающей точкой
(Стандарт IEEE)

На компьютерах, не соответствующих стандарту IEEE, таблица 7.4 не будет действительной. Фактически, на некоторых машинах float может иметь тот же набор значений, что и double, или double может иметь те же значения, что и long double. Макросы, определяющие характеристики типов чисел с плавающей точкой, можно найти в заголовке <float.h>. В C99 типы чисел с плавающей точкой делятся на две категории.

Типы float, double и long double относятся к одной категории,

называемой типами действительных чисел с плавающей точкой (real floating types). К типам чисел с плавающей точкой относятся также комплексные типы (float_Complex, double_Complex и long double_Complex), которые являются новыми в C99.

Константы из чисел с плавающей точкой

Константы из чисел с плавающей точкой могут быть записаны различными способами. Например, следующие константы - это все допустимые способы записи числа

57.0 57.5 57.0e0 57E0 5.7e1 5.7e + 1
.57e2 570.e-1

Константа с плавающей точкой должна содержать десятичную точку и / или экспоненту (показатель степени); показатель показывает степень 10, по которой число должно масштабироваться. Если присутствует экспонента, ей должна предшествовать буква E (или e). Необязательный знак + или - может появиться после E (или e). По умолчанию константы с плавающей точкой хранятся как числа с двойной точностью. Другими словами, когда компилятор Си обнаруживает константу 57.0 в программе, он организует сохранение числа в памяти в том же формате, что и переменная double. Это правило обычно не вызывает проблем, поскольку значения double автоматически преобразуются в float при необходимости.

Иногда бывает необходимо заставить компилятор сохранить константу с плавающей точкой в формате `float` или `long double`.

Чтобы указать, что желательна только одинарная точность, поместите букву `F` (или `f`) в конце константы (например, `57.0F`). Чтобы указать, что константа должна храниться в формате `long double`, поместите букву `L` (или `l`) в конце (`57.0L`). В C99 предусмотрена возможность записи констант с плавающей точкой в шестнадцатеричном формате. Такая константа начинается с `0x` или `0X` (как шестнадцатеричная целая константа). Эта возможность редко используется.

Чтение и запись чисел с плавающей точкой

Как мы уже обсуждали, спецификации преобразования `%e`, `%f` и `%g` используются для чтения и записи чисел с плавающей точкой одинарной точности. Значения типов `double` и `long double` требуют несколько иных преобразований:

■ При чтении значения типа `double` поставьте букву `l` перед `e`, `f` или `g`:

```
double d;
```

```
scanf("%lf", &d);
```

Примечание. Используйте `l` только в строке формата `scanf`, а не в строке `printf`.

В строке формата `printf` преобразования `e`, `f` и `g` могут использоваться для записи значений типа `float` или `double`. (C99 узаконивает использование `%le`, `%lf` и `%lg` в вызовах `printf`, хотя `l` не имеет никакого эффекта.)

■ При чтении или записи значения типа `long double` добавьте букву `L` перед `e`, `f` или `g`:

```
long double ld;
```

```
scanf("%Lf", &ld);
```

```
printf("%Lf",ld);
```

7.3 Типы символов

Единственным оставшимся базовым типом является `char`, тип символа. Значения типа `char` могут варьироваться от одного компьютера к другому, потому что разные машины могут иметь разные базовые наборы символов.

=====

Наборы символов

Самый популярный сегодня набор символов - ASCII (Американский стандартный код для обмена информацией), 7-битный код, способный представлять 128 символов. В ASCII цифры от 0 до 9 представлены кодами 011000-04111001, а прописные буквы от A до Z — 1000001-1011010. ASCII часто распространяется на 256-символьный код, известный как Latin-1, который обеспечивает символы, необходимые для западноевропейских и многих африканских языков.

=====

Переменной типа `char` может быть присвоен любой одиночный символ:

```
ch = 'a'; /* a в нижнем регистре */
```

```
ch = 'A'; /* A в верхнем регистре */
```

```
ch = '0'; /*ноль*/
```

```
ch = ' '; /* пробел*/
```

Обратите внимание, что символьные константы заключены в одинарные кавычки, а не в двойные кавычки.

Операции над символами

Работа с символами в Си проста, из-за одного факта: Си рассматривает символы как маленькие целые числа. В конце концов, символы кодируются в двоичном виде. И не нужно большой фантазии, чтобы рассматривать эти двоичные коды как целые числа. В ASCII, например, коды символов варьируются от 00000000 до 11111111, которые мы можем рассматривать как целые числа от 0 до 127. Символ 'a' имеет значение 97, 'A' имеет значение 65, '0' имеет значение 48, а ' ' имеет значение 32. Связь между символами и целыми числами в Си настолько сильна, что символьные константы на самом деле имеют тип `int`, а не тип `char` (интересный факт, но не из тех, которые часто будут иметь для нас значение). Когда символ появляется в вычислении, Си просто использует его целочисленное значение. Рассмотрите следующие примеры, которые предполагают набор символов ASCII:

```
char ch;
```

```
int i;
```

```
i = 'a';           /* i сейчас 97 */
```

```
ch = 65;           /* ch сейчас 'A' */
```

```
ch = ch + 1;  /* сейчас 'B' */
```

```
ch++;        /* ch сейчас 'C' */
```

Символы можно сравнивать, так как цифры.

Следующая инструкция `if` проверяет, содержит ли `ch` строчную букву; если это так, он преобразует `ch` в верхний регистр.

```
if ('a' <= ch && ch <= 'z')
```

```
    ch = ch - 'a' + 'A';
```

Сравнения, такие как `'a' <= ch`, выполняются с использованием целочисленных значений соответствующих символов. Эти значения зависят от используемого набора символов. Поэтому программы, которые используют `<`, `<=`, `>` и `>=` для сравнения символов, могут не быть переносимыми. Тот факт, что символы имеют те же свойства, что и числа, имеет некоторые преимущества. Например, мы можем легко написать инструкцию `for`, управляющая переменная которого проходит через все заглавные буквы:

```
for (ch = 'A'; ch <= 'Z'; ch ++)...
```

С другой стороны, обработка символов как чисел может привести к различным ошибкам программирования, которые не будут уловлены компилятором, и позволяет нам писать бессмысленные выражения, такие как `'a' * 'b' / 'c'`.

Это может также затруднить переносимость, поскольку наши программы могут основываться на предположениях относительно базового набора символов. (Наш цикл `for`,

например, предполагает, что буквы от A до Z имеют последовательные коды).

Знаковые и беззнаковые символы

Поскольку Си допускает использование символов в качестве целых чисел, неудивительно, что тип `char`, как целочисленные типы, существует как в знаковых, так и в беззнаковых версиях. Знаковые символы обычно имеют значения от -128 до 127, в то время как символы без знака имеют значения от 0 до 255. В стандарте Си не указывается, является ли обычный символ знаковым или беззнаковым; некоторые компиляторы рассматривают его как знаковый тип, тогда как другие рассматривают его как тип без знака. (Некоторые даже позволяют программисту выбирать через опцию компилятора, должен ли быть символ знаковым или беззнаковым).

Большую часть времени нам совершенно неважно, знаковый ли тип `char` или нет.

Время от времени, тем не менее, мы это делаем, особенно если мы используем символьную переменную для хранения небольшого целого числа. По этой причине Си позволяет использовать слова `signed` и `unsigned` для модификации `char`:

```
signed char sch;
```

unsigned char uch;

Не предполагайте, что символ char по умолчанию signed или unsigned.

Если это имеет значение, используйте символ signed или unsigned char вместо char.

В свете тесной взаимосвязи между символами и целыми числами C89 использует термин **интегральные типы** для обозначения как целочисленных типов, так и типов символов. Перечислимые типы (*Встречаются в литературе наименования и перечисляемый тип, и перечисление*) также являются целочисленными типами. В C99 не используется термин "интегральные типы". Вместо этого он расширяет значение "целочисленные типы", чтобы включить типы символов и перечислимые типы. Тип C99 _Bool считается беззнаковым целым типом.

Арифметические типы

Целочисленные типы и типы с плавающей точкой в совокупности известны как арифметические типы. Вот краткое изложение арифметических типов в C89. Разделённые на категории и подкатегории:

■ Целочисленные типы

- char
- Знаковые целые типы (signed char, short int, int, long int)
- Беззнаковые целочисленные типы (unsigned char, unsigned short int, unsigned int, unsigned long int)
- Перечислимые типы

■ Типы с плавающей точкой (float, double, long double)

C99 имеет более сложную иерархию для арифметических типов:

■ Целочисленные типы

- char
- Типы целого числа со знаком, и стандартный (signed char, short int, int, long int, long long int) и расширенный
- Типы целого без знака и стандартный (unsigned char, unsigned short int, unsigned int, unsigned long int, unsigned long long int, _Bool) и расширенный
- Перечислимые типы

■ Типы с плавающей точкой

- Реальные типы с плавающей точкой (float, double, long double)
- Комплексные типы (float_Complex, double_Complex, long double_Complex)

Управляющие последовательности (Escape-последовательности)

Символьная константа обычно представляет собой один символ, заключенный в одинарные кавычки, как мы уже видели в предыдущих примерах. Однако некоторые специальные символы, включая символ новой строки, не могут быть записаны таким образом, потому что они невидимы (непечатные) или потому, что они не могут быть введены с клавиатуры. И поэтому, чтобы программы могли иметь дело с каждым символом в базовом наборе символов, Си обеспечивает специальную нотацию, *escape-последовательности* (управляющей последовательности).

Существует два вида управляющих последовательностей: *escape-символы* и *числовые escape-последовательности*. Мы видели неполный список *escape-символов* в Разделе 3.1; в табл. 7.5 приведён полный набор.

Имя	Управляющая последовательность
Звуковой сигнал (предупреждение)	\a
Возврат на один символ	\b
Перевод страницы	\f
Переход на новую строку	\n
Возврат каретки	\r
Горизонтальная табуляция	\t
Вертикальная табуляция	\v
Обратная косая черта	\\

Знак вопроса	\?
Одинарная кавычка	\'
Двойная кавычка	\"

Escape-символы \a, \b, \f, \r, \t и \v представляют общие управляющие символы ASCII.

Escape-символ \n представляет собой символ перевода строки ASCII.

Escape-символ \\ позволяет символьной константе или строке содержать символ \.

Escape-символ \' позволяет символьной константе содержать символ ', а escape-символ \" позволяет строке содержать символ ". Escape-символ \? редко используется.

Escape-символы удобны, но у них есть проблема:

Список escape-символов не включает все непечатаемые символы ASCII, только самые общие. Escape-символы также бесполезны для представления символов за пределами базовых 128 символов ASCII. Числовые escape-последовательности, которые могут представлять любой символ, являются решением этой проблемы. Чтобы написать числовой escape-символ для конкретного символа, сначала найдите восьмеричное или шестнадцатеричное значение символа в таблице, например, той, которая указана в Приложении Е. Например, ASCII escape-символ (десятичное

значение: 27) имеет восьмеричное значение 33 и шестнадцатеричное 1В. Любой из этих кодов может использоваться для записи ескаре-последовательности:

■ Восьмеричная ескаре-последовательность состоит из символа \, за которым следует восьмеричное число, состоящее максимум из трёх цифр. (Это число должно быть представлено в виде символа без знака, поэтому его максимальное значение обычно равно восьмеричному 377). Например, ескаре-символ может быть записан как \33 или \033. Восьмеричные числа в управляющих последовательностях - в отличие от восьмеричных констант - не обязаны начинаться с 0.

■ Шестнадцатеричная управляющая последовательность состоит из \x, за которым следует шестнадцатеричное число. Хотя Си не накладывает ограничений на количество цифр в шестнадцатеричном числе, оно должно быть представлено в виде символа без знака (следовательно, оно не может превышать FF, если длина символов равна восьми битам). Используя это обозначение, ескаре-символ будет записан \x1b или \x1В. Символ x должен быть в нижнем регистре, но шестнадцатеричные цифры (например, **b**) могут быть в верхнем или в нижнем регистре.

При использовании в качестве символьной константы ескаре-последовательность должна быть заключена в одинарные

кавычки. Например, константа, представляющая escape-символ, будет записана '\33' (или '\x1b'). Escape-последовательности имеют тенденцию становиться немного загадочными, поэтому часто полезно давать им имена с помощью #define:

```
#define ESC '\33' /* ASCII escape-символ*/
```

Escape-последовательности также могут быть встроены в строки, как мы видели в Разделе 3.1. Escape-последовательности не являются единственными специальными обозначениями для представления символов. Последовательности триграфов обеспечивают способ представления символов #, [, \,], ^, {, |, } и ~. Которые могут быть недоступны на клавиатурах в некоторых странах. C99 добавляет универсальные имена символов, которые напоминают escape-последовательности. Однако, в отличие от управляющих последовательностей, универсальные имена символов допускаются в идентификаторах.

Функции для работы с символами

Ранее в этом разделе мы видели, как писать `if` инструкцию, которая преобразует строчную букву в верхний регистр:

```
if ('a' <= ch && ch <= 'z')  
    ch = ch — 'a' + 'A';
```

Однако это не лучший способ.

Более быстрый и более переносимый способ конвертировать регистр - это вызвать библиотечную функцию `Си toupper`:

```
ch = toupper (ch) ; /* преобразует в верхний регистр */
```

При вызове, `toupper` проверяет, является ли его аргумент (`ch` в данном случае) буквой нижнего регистра. Если это так, он возвращает соответствующую заглавную букву. В противном случае `toupper` возвращает значение аргумента. В нашем примере мы использовали оператор присваивания для сохранения возвращаемого значения `toupper` обратно в переменную `ch`, хотя мы могли бы так же легко сделать что-то ещё с ним - сохранить его в другой переменной, скажем, или протестировать его с помощью инструкции `if` :

```
if (toupper(ch) == 'A')...
```

Программы, вызывающие `toupper`, должны иметь следующую директиву `#include`:

```
#include <ctype.h>
```

Функция `tolower` не является единственной полезной функцией обработки символов в библиотеке Си. Раздел 23.5 описывает их все и даёт примеры их использования.

Чтение и запись символов с помощью `scanf` и `printf`

Спецификация преобразования `%c` позволяет `scanf` и `printf` читать и записывать одиночные символы:

```
char ch;
```

```
scanf ("%c", &ch); /* читает одиночный символ */
```

```
printf ("%c", ch); /* записывает одиночный символ */
```

`scanf` не пропускает символы пробела перед чтением символа. Если следующим непрочитанным символом является пробел, тогда переменная `ch` в предыдущем примере будет содержать пробел после возврата `scanf`. Чтобы заставить `scanf` пропускать пробел перед чтением символа, поместите пробел в строку формата перед `%c`:

```
scanf (" %c", &ch); /* Пропускает пробел, затем читает ch */
```

Вспомните из раздела 3.2, что пробел в строке формата `scanf` означает "пропустить нуль или больше символов пробела".

Поскольку `scanf` обычно не пропускает пробелы, легко определить конец строки ввода:

проверьте, является ли прочитанный символ символом новой строки. Например, следующий цикл будет читать и игнорировать все оставшиеся символы (*после '\n'*) в текущей строке ввода:

```
do {  
    scanf("%c", &ch);  
}while (ch != '\n');
```

Когда `scanf` вызывается в следующий раз, он будет читать первый символ на следующей строке ввода.

Чтение и запись символов с помощью `getchar` и `putchar`

Си предоставляет другие способы для чтения и записи отдельных символов. В частности, мы можем использовать функции `getchar` и `putchar` вместо вызова `scanf` и `printf`.

`putchar` пишет один символ:

```
putchar (ch);
```

Каждый раз, когда `getchar` вызывается, она читает один символ, который возвращает. Чтобы сохранить этот символ, мы должны использовать присваивание для его хранения в переменной:

```
ch = getchar(); /* Считывает символ и сохраняет его в ch */
```

`getchar` фактически возвращает значение типа `int`, а не значение `char` (причина будет рассмотрена в следующих главах). В результате для переменной весьма обычно иметь тип `int` а не `char`, если она будет использоваться для сохранения символа, считанного `getchar`.

Подобно `scanf`, `getchar` не пропускает символы пробела при чтении. Использование `getchar` и `putchar` (а не `scanf` и `printf`) экономит время при выполнении программы.

`getchar` и `putchar` работают быстро по двум причинам. Во-первых, они намного проще, чем `scanf` и `printf`, которые предназначены для чтения и записи различных типов данных в различных форматах. Во-вторых, `getchar` и `putchar` обычно реализуются как макросы для дополнительной скорости, у `getchar` есть ещё одно преимущество перед `scanf`: поскольку она возвращает символ, который она читает `getchar` годится для различных Си-идиом, включая циклы, которые ищут символ или пропускают все вхождения символа. Рассмотрим цикл `scanf`, который мы использовали, чтобы пропустить оставшуюся часть строки ввода:

```
do {  
    scanf( "%c", &ch );  
} while ( ch != '\n' );
```

Переписывание этого цикла с помощью `getchar` даёт нам следующее:

```
do {  
    ch = getchar();
```



```
} while (ch != '\n');
```

Перемещение вызова `getchar` в управляющее выражение позволяет нам уплотнить цикл:

```
while ((ch = getchar()) != '\n')
    ;
```

Этот цикл читает символ, сохраняет его в переменной `ch`, а затем проверяет, не совпадает ли `ch` с символом новой строки. Если проверка завершается успешно, тело цикла (которое является пустым) выполняется, затем цикл выполняется ещё раз, в результате чего считывается новый символ.

На самом деле, нам даже не нужна переменная `ch`: мы можем просто сравнить возвращаемое значение `getchar` с символом новой строки:

идиома: `while (getchar() != '\n') /* пропускает остаток строки */`

```

;

```

Получившийся цикл - это хорошо знакомая Си-идиома, загадочная, но заслуживающая изучения.

`getchar` полезна в циклах, пропускающих символы, а также в циклах, которые выполняют поиск символов. Рассмотрим следующую инструкцию, который использует `getchar` для пропуска неопределённого числа пустых символов:

идиома: `while ((ch = getchar()) == ' ') /* пропуск пробела */`

;

Когда цикл завершается, `ch` будет содержать первый не пустой символ, с которым сталкивался `getchar`. Будьте осторожны, если вы смешиваете `getchar` и `scanf` в одной и той же программе, `scanf` имеет тенденцию оставлять символы, которые он "подглядывает", но не читает, в том числе символ новой строки. Подумайте, что произойдёт, если мы попробуем сначала прочесть число, а затем символ:

```
printf("Enter an integer: ");
```

```
scanf("%d", &i);
```

```
printf("Enter a command: ");
```

```
command = getchar();
```

Вызов `scanf` оставит после себя любые символы, которые не были использованы во время чтения `i`, включая (но не ограничиваясь этим) символ новой строки. `getchar` выведет первый оставшийся символ, что будет не тем, что мы имели в виду.

ПРОГРАММА Длина сообщения

Чтобы проиллюстрировать, как читаются символы, давайте напишем программу, которая вычисляет длину сообщения.

После того, как пользователь вводит сообщение, программа отображает длину:

Enter a message: Brevity is the soul of wit.

Your message was 27 character(s) long.

Длина включает пробелы и знаки препинания, но не символ новой строки в конце сообщения. Нам понадобится цикл, тело которого читает символ и увеличивает счётчик. Цикл прекращается, как только появляется символ новой строки. Мы могли бы использовать либо `scanf`, либо `getchar` для чтения символов; большинство Си-программистов выбрали бы `getchar`. Используя простой цикл `while`, мы можем получить следующую программу.

```
//length.c
```

```
/* Определяет длину сообщения. */
```

```
#include <stdio.h>
```

```
int main(void){
```

```
    char ch;
```

```
    int len = 0;
```

```
    printf("Enter a message: ");
```

```
    ch = getchar();
```

```
    while (ch != '\n') {
```

```
        len++;
```

```
        ch = getchar();
```

```
    }  
    printf("Your message was %d character(s) long.\n", len);  
    return 0;  
}
```

Вспоминая наше обсуждение идиом с использованием циклов `while` и `getchar`, мы понимаем, что программу можно сократить:

//length2.c

/ Определяет длину сообщения. */*

```
#include <stdio.h>
```

```
int main(void){  
    int len = 0;  
    printf("Enter a message: ");  
    while (getchar() != '\n')  
        len++;  
    printf("Your message was %d character(s) long.\n", len);  
    return 0;  
}
```

7.4 Преобразование типов

Компьютеры имеют тенденцию быть более ограничивающими, чем Си, когда речь заходит об арифметике. Для того, чтобы вычислитель выполнял арифметическую операцию, операнды обычно должны иметь одинаковый размер (такое же количество бит) и сохраняться таким же образом. Компьютер может складывать два 16-разрядных целых числа, но не 16-битное целое число и 32-разрядное целое число, или 32-разрядное целое число и 32-разрядное число с плавающей точкой. Си, с другой стороны, позволяет смешивать основные типы в выражениях. Мы можем комбинировать целые числа, числа с плавающей запятой и даже символы в одном выражении. Компилятор Си может затем генерировать инструкции, которые преобразуют некоторые операнды в другие типы, чтобы аппаратное обеспечение могло вычислить выражение. Например, если мы складываем 16-битный `short` и 32-битный `int`, компилятор организует преобразование `short` в 32 бита. Если мы сложим `int` и `float`, компилятор организует преобразование `int` в формат с плавающей запятой. Это преобразование немного сложнее, поскольку значения `int` и `float` хранятся по-разному. Поскольку компилятор обрабатывает эти преобразования автоматически, без участия программиста, они называются неявными преобразованиями. Си также позволяет программисту

выполнять явные преобразования, используя оператор `cast` (*оператор () – внутри пишем тип, в который нужно конвертировать*). Сначала я буду обсуждать неявные преобразования, откладывая явные преобразования до тех пор, пока они не появятся позже в этом разделе. К сожалению, правила для выполнения неявных преобразований являются несколько сложноватыми, в первую очередь потому, что в языке Си так много разных арифметических типов. Неявные преобразования выполняются в следующих ситуациях:

- Когда операнды в арифметическом или логическом выражении имеют разный тип. (Си выполняет так называемые обычные арифметические преобразования.)
- Когда тип выражения в правой части присваивания не соответствует типу переменной на левой стороне.
- Когда тип аргумента в вызове функции не соответствует типу соответствующего параметра.
- Когда тип выражения в инструкции `return` не соответствует возвращаемому типу функции.

Мы обсудим первые два случая и оставим остальные для Главы 9.

Обычные арифметические преобразования

Обычные арифметические преобразования применяются к операндам большинства бинарных операторов, включая

арифметические, реляционные и оператор равенства. Например, допустим, что f имеет тип `float`, а i имеет тип `int`. Обычные арифметические преобразования будут применены к операндам в выражении $f + i$, потому что их типы не совпадают. Очевидно, что безопаснее преобразовать i в тип `float` (тип соответствующий f) вместо преобразования f в тип `int` (совпадающий с типом i). Целое число всегда может быть преобразовано во `float`; худшее, что может случиться, - незначительная потеря точности. С другой стороны, преобразование числа с плавающей запятой в `int` будет стоить нам дробной части числа. Хуже того, мы получили бы абсолютно бессмысленный результат, если бы исходное число было больше наибольшего возможного целого или меньше наименьшего целого. Стратегия обычных арифметических преобразований заключается в том, чтобы преобразовать операнды в "самый узкий" тип, который будет безопасно размещать оба значения. (Грубо говоря, один тип является более узким, чем другой, если для его хранения требуется меньше байтов.) Типы операндов часто могут быть согласованы путём преобразования операнда более узкого типа в тип другого операнда (это действие называется расширением). Среди наиболее распространённых расширений являются интегральные расширения, которые преобразуют символ или короткое целое число в тип `int` (или в `unsigned int` в некоторых случаях).

Мы можем разделить правила для выполнения обычных арифметических преобразований на два случая:

■ Тип любого из операндов является типом с плавающей точкой. Используйте следующую диаграмму для расширения операнда, тип которого уже:

long double

↑

double

↑

float

То есть, если один операнд имеет тип long double, то другой операнд конвертируется в тип long double. В противном случае, если один операнд имеет тип double, другой операнд (если его тип уже) конвертируется в тип double. В противном случае, если один операнд имеет тип float, другой операнд (если его тип уже) конвертируется в тип float. Обратите внимание, что эти правила охватывают смеси типов целого и с плавающей запятой: если один операнд имеет тип long int, например, а другой имеет тип double, то операнд long int преобразуется в double.

■ Ни один из типов операндов не является типом с плавающей запятой. Сначала выполните интегральное расширение на обоих операндах (гарантируя, что ни один операнд не будет символом или коротким целым числом).

333

Затем используйте следующую диаграмму для расширения операнда, тип которого уже:

unsigned long int

↑

long int

↑

unsigned int

↑

int

Есть один частный случай, но это происходит только тогда, когда long int и unsigned int имеют одинаковую длину (скажем, 32 бита). В этих обстоятельствах, если один операнд имеет тип long int, а другой имеет тип unsigned int, оба преобразуются в unsigned long int.

Когда знаковый операнд объединяется с беззнаковым операндом, знаковый операнд преобразуется в беззнаковое значение. Преобразование включает в себя сложение или вычитание числа кратного $n + 1$, где n - наибольшее представимое значение беззнакового типа. Это правило может стать причиной непонятных ошибок программирования. Предположим, что int переменная *i* имеет значение -10 и unsigned int переменная *u* имеет значение 10. Если мы сравним *i* и *u*, используя оператор $<$, мы можем ожидать в результате 1 (true). Однако перед сравнением *i* преобразуется в unsigned int. Поскольку отрицательное число не может быть

представлено как целое без знака, преобразованное значение не будет равно -10. Вместо этого добавляется значение 4 294 967 296 (предполагается, что 4 294 967 295 является наибольшим значением unsigned int), что даёт преобразованное значение 4 294 967 286. Таким образом, сравнение $i < u$ выдаст 0. Некоторые компиляторы выдают предупреждающее сообщение, такое как "сравнение знаковых и беззнаковых чисел", когда программа пытается сравнить знаковое число с беззнаковым числом. Из-за таких ловушек лучше использовать целые числа без знака и, особенно, никогда не смешивать их со знаковыми целыми.

Следующий пример показывает обычные арифметические преобразования в действии:

```
char c;  
short int s;  
int i;  
unsigned int u;  
long int l;  
unsigned long int ul;  
float f;  
double d;  
long double ld;  
i = i + c; /* c конвертируется в int */  
i = i + s; /* s конвертируется в int */  
u = u + i; /* i конвертируется в unsigned int */  
l = l + u; /* u конвертируется в long int */
```

```
ul = ul + 1; /* l конвертируется в unsigned long int */  
f = f + ul; /* ul конвертируется в float */  
d = d + f; /* f конвертируется в double */  
ld = ld + d; /* d конвертируется в long double */
```

Преобразование при присваивании

Обычные арифметические преобразования не применяются при присваивании. Вместо этого Си следует простому правилу, что выражение в правой части присваивания преобразуется в тип переменной с левой стороны. Если тип переменной не меньше "ширины" выражения, это будет работать без проблем. Например:

```
char c;  
int i;  
float f;  
double d;  
  
i = c;    /* c конвертируется в int */  
f = i;    /* i конвертируется в float */  
d = f;    /* f конвертируется в double */
```

Другие случаи проблематичны. Присваивание числа с плавающей точкой целочисленной переменной приводит к отбрасыванию дробной части числа:

```
int i;  
  
i = 842.97; /* i сейчас 842 */
```

```
i = -842.97; /* i сейчас -842 */
```

Более того, присваивание значения переменной более узкого типа даст результат с меньшим значением (или хуже), если значение выходит за пределы диапазона типа переменной:

```
c = 10000; /*** WRONG ***/
```

```
i = 1.0e20; /*** WRONG ***/
```

```
f = 1.0e100; /*** WRONG ***/
```

"Сужающее" присваивание может выявить предупреждение от компилятора или от инструментов, таких как `lint`.

Правильно будет добавить суффикс *f* к константе с плавающей точкой, если она будет присвоена переменной `float`, как мы делали начиная с Главы 2:

```
f = 3.14159f;
```

Без суффикса константа 3.14159 будет иметь тип `double`, возможно, вызывая предупреждение.

Неявные преобразования в C99

Правила неявных преобразований в C99 несколько отличаются от правил в C89, в первую очередь потому, что C99 имеет дополнительные типы (типы `_Bool`, `long long`, расширенные целочисленные типы и `complex` типы).

В целях определения правил преобразования C99 даёт каждому целочисленному типу " ранг преобразования целых чисел". Ниже приведены ранги от самого высокого до самого низкого:

1. long long int, unsigned long long int
2. long int, unsigned long int
3. int, unsigned int
4. short int, unsigned short int
5. char, signed char, unsigned char
6. _Bool

Для простоты я игнорирую расширенные целочисленные типы и типы перечислений. Вместо интегральных продвижений C89, C99 имеет "целочисленные продвижения", которые включают преобразование любого типа, чей ранг меньше, чем int и unsigned int до int (при условии, что все значения типа могут быть представлены с помощью int) или же с помощью unsigned int. Как и в C89, правила C99 для выполнения обычных арифметических преобразований можно разделить на два случая:

■ Тип любого из операндов является типом с плавающей точкой. Пока ни один операнд не имеет комплексного типа, правила те же, что и раньше. (Правила преобразования для комплексных типов будут рассмотрены в Разделе 27.3.)

■ Тип любого из операндов не является типом с плавающей точкой. Сначала выполняется целочисленное продвижение по обоим операндам. Если типы двух операндов сейчас одинаковы, процесс завершается. В противном случае используются следующие правила, останавливаясь на первом:

- Если оба операнда имеют знаковые типы или оба имеют

типы без знака, преобразуется операнд, тип которого имеет меньший ранг целочисленного преобразования, к типу операнда с большим рангом.

- Если беззнаковый операнд имеет ранг, больший или равный рангу типа знакового операнда, преобразуется знаковый операнд в тип беззнакового операнда.
- Если тип знакового операнда может представлять все значения типа беззнакового операнда, преобразуется беззнаковый операнд в тип указанного операнда.
- В противном случае преобразовываются оба операнда в тип без знака, соответствующий типу знакового операнда.

Кстати, все арифметические типы могут быть преобразованы в тип `_Bool`. Результат преобразования равен 0, если исходное значение равно 0; В противном случае результат равен 1.

Операции приведения типов

Хотя неявные преобразования Си удобны, иногда нам нужен более высокий уровень контроля над преобразованием типов. По этой причине Си предоставляет операции приведения типов. Выражение приведения имеет форму:

(type-name) выражение

type-name определяет тип, к которому должно быть преобразовано *выражение*. В следующем примере показано, как использовать выражение приведения для вычисления дробной части значения с плавающей запятой:

```
float f, frac_part;
frac_part = f - (int) f;
```

Выражение преобразования типов `(int) f` представляет результат преобразования значения *f* в тип `int`. Затем потребуются обычные арифметические преобразования Си, чтобы `(int) f` был преобразован обратно в тип `float` до того, как может быть выполнено вычитание. Разница между *f* и `(int) f` является дробной частью *f*, которая была отброшена во время преобразования. Выражение преобразования типов позволяют нам документировать преобразования типов, которые имели бы место в любом случае:

```
i = (int)f; /* f конвертируется в int */
```

Они также позволяют нам отменить правила компилятора и заставить его делать нужные нам преобразования.

Рассмотрим следующий пример:

```
float quotient;
int dividend, divisor;
quotient = dividend / divisor;
```

Как написано выше, результат деления - целое число - будет преобразовано в форму с плавающей точкой перед сохранением в переменной `quotient`. Вероятно, мы бы хотели, чтобы `divend` и `divisor` были конвертированы во `float` перед делением, чтобы получить более точный ответ. Выражение приведения типов делает такой трюк:

```
quotient = (float) dividend / divisor;
```

divisor не нуждается в приведении, так как приведение dividend ко float заставляет компилятор также конвертировать divisor во float. Кстати, Си считает (*type-name*) унарным оператором. Унарные операторы имеют более высокий приоритет, чем бинарные операторы.

Поэтому компилятор интерпретирует

```
(float) dividend / divisor
```

как

```
((float) dividend) / divisor
```

Если вы считаете, что это сбивает с толку, обратите внимание, что есть и другие способы добиться такого же эффекта:

```
quotient = dividend / (float) divisor;
```

или

```
quotient = (float) dividend / (float) divisor;
```

Иногда необходимо использовать приведение типов, чтобы избежать переполнения. Рассмотрим следующий пример:

```
long i;
```

```
int j = 1000;
```

```
i = j * j; /* может произойти переполнение */
```

На первый взгляд эта инструкция выглядит отлично.

Значение $j * j$ равно 1 000 000, а i равно long, поэтому она может легко сохранять значения этого размера, верно?

Проблема в том, что при умножении двух значений int

результат будет иметь тип `int`. Но `j * j` слишком велико, чтобы представлять `int` на некоторых машинах, вызывая переполнение. К счастью, использование преобразования типов позволяет избежать проблемы:

```
i = (long) j * j;
```

Поскольку оператор преобразования имеет приоритет над `*`, первая `j` конвертируется в тип `long`, заставляя также преобразовывать вторую `j`. Заметим, что инструкция

```
i = (long)(j * j); /*** WRONG ***/
```

не будет работать, так как к моменту преобразования типов уже было бы переполнение.

7.5 Определения типов

В Разделе 5.2 мы использовали директиву `#define` для создания макроса, который можно было бы использовать в качестве булевого типа:

```
#define BOOL int
```

Однако есть лучший способ настроить булев тип, используя функцию, известную как определение типа:

```
typedef int Bool;
```

Обратите внимание, что имя определяемого типа идёт последним. Заметьте также, что я сделал заглавной первую букву слова `Bool`. Заглавные первые буквы имени типа не требуются, это просто соглашение, которым пользуются

некоторые программисты на языке Си. Используя `typedef` для определения `Bool`, компилятор добавляет `Bool` в список имён типов, которые он распознает.

Теперь `Bool` можно использовать так же, как и встроенные имена типов - в объявлении переменной, в выражениях преобразования типов и в других местах. Например, мы можем использовать `Bool` для объявления переменных:

```
Bool flag; /* то же, что int flag; */
```

Компилятор рассматривает `Bool` как синоним `int`; таким образом, `flag` - не что иное, как обычная переменная типа `int`.

Преимущества определений типов

Определения типов могут сделать программу более понятной (при условии, что программист тщательно выбирает значимые имена типов). Например, предположим, что переменные `cash_in` и `cash_out` будут использоваться для хранения суммы в долларах. Объявление доллара как

```
Typedef float Dollars;
```

А затем написание

```
Dollars cash_in, cash_out;
```

будет более информативно, чем просто запись

```
float cash_in, cash_out;
```

Определения типов также могут облегчить изменение программы. Если мы позже решим, что Dollars действительно должны быть определены как double, все, что нам нужно сделать, это изменить определение типа:

```
typedef double Dollars;
```

Объявления переменных Dollars изменять не нужно. Без определения типа нам нужно было бы найти все переменные с плавающей точкой, в которых хранятся долларовые суммы (не совсем простая задача), и изменять их объявления.

Определения типов и переносимость

Определения типов являются важным инструментом для написания переносимых программ. Одна из проблем при перемещении программы с одного компьютера на другой заключается в том, что на разных машинах типы могут иметь разные диапазоны. Если *i* - это переменная типа int, то присваивание типа *i* = 100 000 подходит для машины с 32-разрядными целыми числами, но она не будет работать на машине с 16-разрядными целыми числами. Для большей переносимости рассмотрим возможность использования typedef для определения новых имён целочисленных типов. Предположим, что мы пишем программу, которая нуждается в переменных, способных хранить количества продуктов в диапазоне 0-50 000. Для этой цели мы могли бы использовать

переменные `long` (так как они гарантированно способны хранить числа, по крайней мере, до 2 147 483 647), но мы предпочитаем использовать переменные типа `int`, поскольку арифметика для значений `int` может быть быстрее, чем операции с значениями `long`; кроме того, переменные типа `int` могут занимать меньше места. Вместо использования типа `int` для объявления переменных количества мы можем определить наш собственный тип "quantity":

```
typedef int Quantity;
```

и использовать этот тип для объявления переменных:

```
Quantity q;
```

К сожалению, эта методика не решает всех наших проблем, так как изменение определения `Quantity` (на *тип long*) может повлиять на способ использования переменных `Quantity`. По крайней мере, вызовы `printf` и `scanf`, использующие переменные `Quantity`, должны быть изменены, с заменой спецификаций `%d` на `%ld`. Библиотека Си сама использует `typedef` для создания имён для типов, которые могут отличаться от одной реализации Си к другой ; у этих типов часто есть имена, которые заканчиваются на `_t`, такие как `ptrdiff_t`, `size_t` и `wchar_t`. Точные определения этих типов будут разными, но вот некоторые типичные примеры:

```
typedef long int ptrdiff_t;
```

```
typedef unsigned long int size_t;
```

```
typedef int wchar_t;
```

В C99 заголовок `<stdint.h>` использует `typedef` для определения имён целочисленных типов с определенным количеством бит. Например, `int32_t` является знаковым целочисленным типом с ровно 32 битами. Использование этих типов - эффективный способ сделать программы более переносимыми.

Оператор `sizeof`

Оператор `sizeof` позволяет программе определять объем памяти, необходимый для хранения значений определённого типа. Значение выражения `sizeof (имя-типа);`

Представляет собой целое число без знака, представляющее число байтов, необходимых для хранения значения, принадлежащего *имя-типа*. `sizeof (char)` всегда равен 1, но размеры других типов могут отличаться. На 32-битной машине `sizeof (int)` обычно равен 4. Обратите внимание, что `sizeof` является довольно необычным оператором, так как сам компилятор обычно может определять значение выражения `sizeof`. Оператор `sizeof` также может применяться к константам, переменным и выражениям в целом. Если *i* и *j* являются переменными типа `int`, то `sizeof (i)` равно 4 на 32-разрядной машине, как и `sizeof (i + j)`. При применении к выражению - в отличие от типа - `sizeof` не требует скобок; мы могли бы написать `sizeof i` вместо `sizeof (i)`. Однако скобки могут потребоваться в любом случае из-за приоритета

оператора. Компилятор будет интерпретировать `sizeof i + j` как `(sizeof i) + j`, потому что `sizeof` - унарный оператор - имеет приоритет над бинарным оператором `+`. Чтобы избежать проблем, я всегда использую круглые скобки в выражениях `sizeof`. Печать значения `sizeof` требует осторожности, потому что тип выражения `sizeof`, является определяемым реализацией типом `size_t`. В C89 лучше преобразовать значение выражения к известному типу перед его печатью, `size_t` гарантированно будет беззнаковым целочисленным типом, поэтому безопаснее привести выражение `sizeof` к `unsigned long` (самому большому из беззнаковых типов C89) и затем распечатать его с помощью преобразования `%lu`:

```
printf("size of int: %lu\n", (unsigned long) sizeof(int));
```

В C99 тип `size_t` может быть больше, чем `unsigned long`. Однако функция `printf` в C99 может отображать значения `size_t` напрямую, без необходимости приведения. Трюк заключается в использовании буквы `z` в спецификации преобразования, за которой следует один из обычных целочисленных кодов (обычно `u`):

```
printf("size of int: %zu\n", sizeof(int) ); /* C99 only */
```

Вопросы и ответы

Вопрос: В Разделе 7.1 говорится, что %o и %x используются для записи целых чисел без знака в восьмеричной и шестнадцатеричной нотациях.

Как написать обычные (знаковые) целые числа в восьмеричном или шестнадцатеричном виде?

Ответ: Вы можете использовать %o и %x для печати знакового целого, если его значение не является отрицательным. Эти преобразования заставляют printf обрабатывать целое число со знаком, как если бы оно было беззнаковым: другими словами, printf будет считать, что знаковый бит часть величины числа. Пока знаковый бит равен 0, проблем нет.

Если бит знака равен 1, printf напечатает неожиданно большое число.

Вопрос: Но что, если число отрицательное?

Как я могу записать его в восьмеричном или шестнадцатеричном виде?

Ответ: Прямого способа напечатать отрицательное число в восьмеричном или шестнадцатеричном формате нет. К счастью, это необходимо сделать довольно редко. Вы можете, конечно, проверить, является ли это число отрицательным и вывести знак минус самостоятельно:

```
if (i < 0)
```

```
printf (" -%x", -i ) ;
```

```
else
```

```
printf (" %x", i ) ;
```

Вопрос: Почему константы с плавающей точкой хранятся в форме double, а не в форме float?

Ответ: По историческим причинам. Си отдаёт предпочтение типу double; float рассматривается как гражданин второго сорта. Рассмотрим, например, обсуждение float в книге Кернигана и Ритчи "Язык программирования Си": "Основная причина использования float заключается в экономии памяти в больших массивах или, реже, в экономии времени на машинах, где арифметика с двойной точностью особенно дорога".

Си первоначально требовал, чтобы вся арифметика с плавающей точкой выполнялась с двойной точностью. (C89 и C99 не имеют такого требования.)

Вопрос: Что представляют собой шестнадцатеричные константы с плавающей точкой и для чего они хороши?

Ответ: Шестнадцатеричная константа с плавающей точкой начинается с 0x или 0X и должна содержать показатель степени, которому предшествует буква P (или p). Показатель может иметь знак, а константа может оканчиваться на f, F, l или L. Показатель выражается в десятичном виде, но представляет степень 2, а не степень 10. Например, 0x1.Vp3 представляет собой Число $1,6875 \times 2^3 = 13,5$.

Шестнадцатеричная цифра V соответствует битовой последовательности 1011. V находится справа от точки,

поэтому каждый 1 бит представляет собой отрицательную степень 2. Суммируя эти числа $2 (2^{-1} + 2^{-3} + 2^{-4})$, получаем .6875. Шестнадцатеричные константы с плавающей точкой в первую очередь полезны для указания констант, требующих высокой точности (включая математические константы, такие как e и π). Шестнадцатеричные числа имеют точное двоичное представление, в то время как константа, написанная десятичным числом, может быть подвержена небольшой ошибке округления при преобразовании в двоичное. Шестнадцатеричные числа также полезны для определения констант с экстремальными значениями, такими как значения макросов в заголовке `<float.h>`. Эти константы легко записать в шестнадцатеричном виде, но трудно записать в десятичной системе.

Вопрос: Почему мы используем `%lf` для чтения значения `double`, но `%f` для его печати?

Ответ: Это сложный вопрос. Во-первых, обратите внимание, что `scanf` и `printf` являются необычными функциями, поскольку они не ограничены фиксированным числом аргументов. Мы говорим, что `scanf` и `printf` имеют списки аргументов переменной длины. Когда вызывается функция со списками аргументов переменной длины, компилятор организует автоматическое преобразование аргументов типа `float` в тип `double`. В результате, `printf` не может различать аргументы `float` и `double`.

Это объясняет, почему `%f` работает для `float` и `double` аргументов в вызовах `printf`.

`scanf`, с другой стороны, передаёт указатель на переменную. `%f` сообщает `scanf` сохранить значение `float` по адресу, переданному ему, в то время как `%lf` сообщает `scanf`

сохранить значение `double` по этому адресу. Здесь принципиальное различие между `float` и `double`. Если задана неправильная спецификация преобразования, `scanf`, вероятно, сохранит неправильное количество байтов (не говоря уже о том, что битовый шаблон для `float` не то же самое, что для `double`).

Вопрос: Как правильно произносится "char"?

Ответ: Нет общепринятого произношения. Некоторые люди произносят чар так же, как первый слог "характер". Другие говорят "чар", как про жареного цыплёнка;

Вопрос: Когда имеет значение, является ли символьная переменная знаковой или беззнаковой?

Ответ: Если мы сохраняем в переменной только 7-битные символы, это не имеет значения, поскольку знаковый бит будет равен нулю. Однако, если мы планируем хранить 8-битные символы, мы, вероятно, хотим, чтобы переменная имела тип `unsigned char`. Рассмотрим следующий пример:

```
ch = '\xdb';
```

Если `ch` объявлен как тип `char`, компилятор может выбрать рассмотрение его как символ со знаком (многие компиляторы делают это). Пока `ch` используется только как символ, проблем не будет. Но если `ch` будет использоваться когда-либо в контексте, который требует, чтобы компилятор конвертировал своё значение в целое число, у нас, скорее всего, будут проблемы: результирующее целое будет отрицательным, поскольку знаковый бит `ch` равен 1.

Вот ещё одна ситуация. В некоторых видах программ принято использовать переменные типа `char` для хранения однобайтовых целых чисел. Если мы пишем такую

программу, нам придётся решить, должна ли каждая переменная быть символом `char` или `unsigned char`, так же, как мы должны решить, должны ли обычные целочисленные переменные иметь тип `int` или `unsigned int`.

Вопрос: Я не понимаю, как символ новой строки может быть символом перевода строки ASCII. Когда пользователь вводит входные данные и нажимает клавишу Enter, разве программа не считывает это как символ возврата каретки или возврат каретки плюс перевод строки?

Ответ: Нет. В рамках наследия UNIX в Си всегда считается, что конец строки помечен одним символом перевода строки. (В текстовых файлах UNIX в конце каждой строки отображается одиночный символ перевода строки, но без возврата каретки). Библиотека Си заботится о переводе нажатия клавиши пользователя на символ перевода строки. Когда программа считывает данные из файла, библиотека I/O (ввода / вывода) преобразует файловый маркер конца строки (независимо от того, чем он может быть) в один символ перевода строки. Те же преобразования происходят - в обратном порядке - когда вывод данных записывается на экран или в файл. (Подробности см. В Разделе 22.1.) Хотя эти переводы могут показаться запутанными, они служат важной цели: изолировать программы от деталей, которые могут отличаться от одной операционной системы к другой.

Вопрос: Каково предназначение `\?` escape-последовательности?

Ответ: escape-последовательность `\?` связана с триграфными последовательностями, которые начинаются с `??`. Если вы должны поместить `??` в строку, то существует

вероятность, что компилятор ошибётся в начале триграфа. Замена второго? на \? решает проблему.

Вопрос: Если `getchar` работает быстрее, почему мы хотим использовать `scanf` для чтения отдельных символов?

Ответ: Хотя она не так быстра, как `getchar`, функция `scanf` более гибкая. Как мы видели ранее, строка формата `"%c"` заставляет `scanf` читать следующий входной символ; `" %c"` заставляет её читать следующий символ, не являющийся пробелом. Кроме того, `scanf` хорошо читает символы, которые смешиваются с другими видами данных. Предположим, что наши входные данные состоят из целого числа, затем одного нечислового символа, затем другого целого числа. Используя строку формата `"%d%c%d"`, мы можем применить `scanf` для чтения всех трёх элементов.

Вопрос: При каких обстоятельствах *продвижение по шкале целочисленных типов* превращает `char` или `short` в `unsigned int`?

Ответ: *Продвижение по шкале целочисленных типов* даёт `unsigned int`, если тип `int` не достаточно велик, чтобы включать все возможные значения исходного типа. Так как символы обычно имеют длину восемь бит, они почти всегда преобразуются в `int`, который гарантированно имеет длину не менее 16 бит.

Знаковые короткие целые числа (`signed short`) также всегда могут быть преобразованы в `int`. Беззнаковые короткие целые (`unsigned short`) являются проблематичными. Если короткие целые числа имеют одинаковую длину с обычными целыми числами (как на 16-разрядной машине), то беззнаковые короткие целые числа (`unsigned short`) должны быть преобразованы в `unsigned int`, поскольку наибольшее целое

число без знака (65535 на 16-разрядной машине) больше самого большого `int` (32 767).

Вопрос: Что именно происходит, если я присваиваю значение переменной, размер которой недостаточно велик для его хранения?

Ответ: Грубо говоря, если значение имеет целочисленный тип и переменная имеет тип без знака, лишние биты выбрасываются; Если у переменной знаковый тип, результат определяется реализацией.

Присваивание числа с плавающей точкой переменной целочисленной или с плавающей точкой слишком малой, чтобы удерживать его, приводит к неопределённому поведению: может случиться что угодно, включая завершение программы.

*** Вопрос:** Почему Си беспокоится о предоставлении определений типов?

Не является ли определение макроса `BOOL` столь же хорошим, как определение типа `Bool` с помощью `typedef`?

Ответ: Существует два важных различия между определениями типов и макроопределениями. Во-первых, определения типов более мощные, чем макроопределения. В частности, типы массива и указателя не могут быть определены как макросы. Предположим, что мы пытаемся использовать макрос для определения типа "указатель на целое":

```
#define PTR_TO_INT int *
```

Объявление

```
PTR_TO_INT p, q, r;
```

станет

```
int * p, q, r;
```

после препроцессора. К сожалению, только `r` является указателем; `q` и `r` - обычные целочисленные переменные. Определения типов не имеют этой проблемы. Во-вторых, имена `typedef` подчиняются тем же правилам области видимости, что и переменные; имя `typedef`, определяемое внутри тела функции, не будет распознано вне функции. Имена макросов, напротив, заменяются препроцессором везде, где они появляются.

*** Вопрос:** Вы сказали, что компиляторы "обычно могут определять значение выражения `sizeof`". Может ли компилятор всегда определять значение выражения `sizeof`?

Ответ: В C89, да. Однако в C99 есть одно исключение. Компилятор не может определить размер массива переменной длины, потому что количество элементов в массиве может измениться во время выполнения программы.

Упражнения

1. Задайте десятичное значение для каждой из следующих целых констант.

(a) 077

(b) 0x77

(c) 0XABC

2. Какие выражения из нижеследующих не являются константами в Си? Классифицируйте каждую допустимую константу как целочисленную или с плавающей точкой.

a) 010E2

355

(b) 32.1E+5

(c) 0790

(d) 100_000

(e) 3.978e—2

W 3. Что из нижеследующего не являются допустимыми типами в Си?

(a) short unsigned int

(b) short float

(c) long double

(d) unsigned long

W 4. Если c является переменной типа char, какое из следующих утверждений является недопустимым?

(a) i += c; /* i имеет тип int */

(b) c = 2 * c - 1;

(c) putchar(c);

(d) print(c);

5. Что из следующего не является допустимым способом написать число 65?

(Предположим, что набор символов - ASCII.)

(a) 'A'

(b) 0b1000001

(c) 0101

(d) 0x41

6. Для каждого из следующих элементов данных укажите, какой из типов `char`, `short`, `int` или `long` является самым маленьким, гарантированно достаточным для хранения элемента.

(a) Дни в месяце

(b) Дни в году

(c) Минут в дне

(d) Секунд в дне

7. Для каждого из следующих `escape`-последовательностей символов дайте эквивалентный восьмеричный `escape`-символ. (Предположим, что набор символов - ASCII.) Вы можете ознакомиться с Приложением E., в котором перечислены числовые коды символов ASCII.

(a) `\b`

(b) `\n`

(c) `\r`

(d) `\t`

8. Повторите упражнение 7, но приведите эквивалентные шестнадцатеричные `escape`-символы.

9. Предположим, что `i` и `j` являются переменными типа `int`, Каков тип выражения `i / j + 'a'`?

W 10. Предположим, что `i` - переменная типа `int`, `j` -

переменная типа long, а k - переменная типа unsigned int;

Какой тип выражения

$i + (\text{int}) j * k$?

11. Предположим, что i - переменная типа int, f - переменная типа float, а d - переменная типа double. Какой тип выражения $i * f / d$?

W 12. Предположим, что i - переменная типа int, f - переменная типа float, а d - переменная типа double.

Объясните, какие преобразования выполняются во время выполнения следующей инструкции:

$d = i + f$;

13. Предположим, что программа содержит следующие объявления:

char c = '\1';

short s = 2;

int i = -3;

long m = 5;

float f = 6.5f;

double d = 7.5;

Укажите значение и тип каждого выражения, указанного ниже.

- | | | |
|-------------|-------------|----------------------|
| (a) $c * i$ | (c) f / c | (e) $f - d$ |
| (b) $s + m$ | (d) d / s | (f) $(\text{int}) f$ |

W 14. Всегда ли следующая инструкция будет корректно вычислять дробную часть f (считая, что f и frac_part являются

переменными с плавающей точкой)?

```
frac_part = f - (int) f;
```

Если нет, в чем проблема?

К разделу 7.5

15. Используйте `typedef` для создания типов с именами `int8`, `int16` и `int32`. Определите типы, чтобы они отображали 8-разрядные, 16-разрядные и 32-разрядные целые числа на вашей машине.

Проекты по программированию

W1. Программа `square2.c` в Разделе 6.3 даёт сбой (обычно печатая странные ответы), если $i * i$ превышает максимальное значение `int`. Запустите программу и определите наименьшее значение n , которое вызывает сбой. Попробуйте изменить тип i на `short` и снова запустить программу.

(Не забудьте обновить спецификации преобразования в вызове `printf`!) Затем попробуйте `long`. Из этих экспериментов какой вывод вы можете сделать о количестве битов, используемых для хранения целочисленных типов на вашей машине?

W2. Измените программу `square2.c` в Разделе 6.3 так, чтобы она останавливалась после каждых 24 квадратов и отображала следующее сообщение:

Нажмите Enter для продолжения...

После отображения сообщения программа должна использовать `getchar` для чтения символа. `getchar` не позволит программе продолжать работу до тех пор, пока пользователь не нажмёт клавишу Enter.

3. Измените программу `sum2.c` раздела 7.1 для суммирования ряда значений `double`.

4. Напишите программу, которая переводит алфавитный телефонный номер в числовую форму:

Введите номер телефона: CALLATT
2255288

(Если у вас нет телефона поблизости, вот буквы на клавишах:
2 = ABC, 3 = DEF, 4 = GHI, 5 = JKL, 6 = MNO, 7 = PRS, 8 = TUV, 9 = WXY.)

Если исходный номер телефона содержит неалфавитные символы (например, цифры или знаки препинания), оставьте их без изменений:

Введите номер телефона: 1-800-COL-LECT
1-800-265-5328

Вы можете предположить, что любые буквы, введенные пользователем, являются прописными.

W5. В Кроссворде SCRABBLE игроки формируют слова, используя маленькие плитки. Каждая из которых содержит букву и номинал. Номинал варьируется от одной буквы к другой, в зависимости от редкости букв.

(Вот номиналы: 1: AEILNORSTU, 2: DG, 3: BCMP, 4: FHVWY, 5: K, 8: JX, 10: QZ. Напишите программу, вычисляющую значение слова, суммируя значения его букв:

Введите слово: pitfall

Значение scrabble: 12

Ваша программа должна допускать любое сочетание строчных и прописных букв в слове.

Подсказка: используйте библиотечную функцию toupper.

W 6. Напишите программу, которая печатает значения sizeof (int), sizeof (short), sizeof (long), sizeof (float), sizeof (double) и sizeof (long double).

7. Измените проект по программированию 6 из Главы 3, чтобы пользователь мог складывать, вычитать, умножать или делить две дроби (вводя +, -, * или / между дробями).

8. Измените проект по программированию 8 из Главы 5, чтобы пользователь вводил время, используя 12-часовой формат времени. Ввод будет иметь вид часы: минуты, за которыми следуют либо А, Р, АМ, либо РМ (строчные или прописные). Разрешён (но не обязательно) пробел между числом времени и индикатором АМ / РМ. Примеры допустимого ввода:

1: 15P

1:15PM

361

1:15pm

1:15 P

1:15 PM

1:15 p

1:15 pm

Вы можете предположить, что вход имеет одну из этих форм;
Нет необходимости проверять наличие ошибок.

9. Напишите программу, запрашивающую у пользователя ввести время в 12-часовом формате, затем отображает время в 24-часовой форме:

Введите 12-часовое время: 9:11 PM

Эквивалентное 24-часовое время: 21:11

См. Проект по программированию 8 для описания входного формата.

10. Напишите программу, которая подсчитывает число гласных (a, e, i, o и u) в предложении:

Введите предложение: And that's the way it is.

Ваше предложение содержит 6 гласных.

11. Напишите программу, которая принимает имя и фамилию, введенные пользователем, и отображает фамилию, запятую и первый инициал, за которым следует точка:

Введите имя и фамилию: Lloyd Foedick

Fosdick, L.

Ввод пользователя может содержать дополнительные пробелы перед именем, между именем и фамилией и после фамилии.

12. Напишите программу, которая вычисляет выражение:

Введите выражение: $1 + 2,5 * 3$

Значение выражения: 10.5

Операнды в выражении - это числа с плавающей точкой; Операторами являются +, -, * и /. Выражение вычисляется слева направо (ни один оператор не имеет приоритета над любым другим оператором).

13. Напишите программу, которая вычисляет среднюю длину слова для предложения:

Введите предложение: It was deja vu all over again.

Средняя длина слова: 3,4

Для простоты ваша программа должна рассматривать знак препинания как часть слова, к которому она присоединена. Отображение средней длины слова до одного десятичного знака.

14. Напишите программу, которая использует метод Ньютона для вычисления квадратного корня из положительного числа с плавающей точкой:

Введите положительное число: 3

Квадратный корень: 1.73205

Пусть x - число, введенное пользователем. Для метода Ньютона требуется предположить исходное значение y для квадратного корня из x (мы будем использовать $y = 1$).

363

Последовательные приближения можно найти, вычисляя среднее значение y и x/y . Следующая таблица показывает, как будет найден квадратный корень из 3:

x	y	x/y	Среднее от y и x/y
3	1	3	2
3	2	1.5	1.75
3	1.75	1.71429	1.73214
3	1.73214	1.73196	1.73205
3	1.73205	1.73205	1.73205

Обратите внимание, что значения y постепенно приближаются к истинному квадратному корню из x . Для большей точности ваша программа должна использовать переменные типа `double`, а не `float`. Прекратить выполнение программы, когда абсолютное значение разницы между старым значением y и новым значением y меньше, чем произведение `0.00001` и y .

Подсказка: вызовите функцию `fabs`, чтобы найти абсолютное значение `double`. (Чтобы использовать `fabs`, вам нужно включить заголовок `<math.h>` в начале вашей программы).

15. Напишите программу, которая вычисляет факториал положительного целого числа:

Введите положительное целое число: 6

Факториал 6: 720

- (a) Используйте переменную `short` для хранения значения факториала. Какое наибольшее значение `n`, для которого программа корректно печатает факториал `n`?
- (b) Повторите часть (a), используя вместо этого переменную типа `int`.
- (c) Повторите часть (a), используя вместо этого переменную `long`.
- (d) Повторите часть (a), используя вместо этого переменную `long long` (если ваш компилятор поддерживает тип `long long`).
- (e) Повторите часть (a), используя вместо этого переменную `float`.
- (f) Повторите часть (a), используя вместо этого переменную `double`.
- (g) Повторите часть (a), используя вместо этого переменную `long double`.

В случаях (e) - (g) программа покажет близкое приближение факториала, не обязательно точное значение.

8 Массивы

Если программа манипулирует большим количеством данных, она делает это лишь несколькими способами.

До сих пор единственными переменными, которые мы видели, были скалярные: они могут содержать один элемент данных. Си также поддерживает агрегатные (составные) переменные, которые могут хранить коллекции значений. В Си есть два вида агрегатов: массивы и структуры. В этой главе показано, как объявлять и использовать массивы, как одномерные (Раздел 8.1), так и многомерные (Раздел 8.2). В Разделе 8.3 рассматриваются массивы переменной длины в C99. Основное внимание в этой главе уделяется одномерным массивам, которые играют гораздо большую роль в Си, чем многомерные массивы. Более поздние главы (в частности, глава 12) предоставляют дополнительную информацию о массивах; глава 16 охватывает структуры.

8.1 Одномерные массивы

Массив - это структура данных, содержащая несколько значений данных, все из которых имеют один и тот же тип. Эти значения, известные как элементы, могут быть индивидуально выбраны по их позиции внутри массива. Простейший вид массива имеет только одно измерение. Элементы одномерного массива концептуально расположены один за другим в одной строке (или столбце, если вы предпочитаете). Вот как мы можем визуализировать одномерный массив с именем ***a***:

a

[illegible]

Чтобы объявить массив, мы должны указать тип элементов массива и их количество. Например, чтобы объявить, что массив ***a*** содержит 10 элементов типа ***int***, мы должны написать

```
int a[10];
```

Элементы массива могут быть любого типа; длина массива может быть задана любым (целым) константным выражением. Так как длину массива, возможно, нужно будет регулировать, когда программа будет позже модифицироваться, использование макроса для определения длины массива - отличная практика:

```
#define N 10
```

...

```
int a [N];
```

Индексация массива

Чтобы получить доступ к определённом элементу массива, мы записываем имя массива, за которым следует целочисленное значение в квадратных скобках (это называется индексированием массива). Элементы массива всегда нумеруются начиная с 0, поэтому элементы массива длины ***n*** индексируются от 0 до ***n*** - 1. Например, если ***a*** - массив с 10 элементами, они обозначаются ***a***[0], ***a***[1],... ***a***[9], как показано на следующем рисунке:

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

Выражения вида $a[i]$ являются lvalues, поэтому их можно использовать так же, как обычные переменные:

```
a[0] = 1;
printf("%d\n", a[5]);
++a[i];
```

В общем случае, если массив содержит элементы типа T , то каждый элемент массива обрабатывается так, как если бы он был переменной типа T . В этом примере элементы $a[0]$, $a[5]$ и $a[i]$ ведут себя как `int` переменные. Массивы и циклы идут рука об руку.

Многие программы содержат циклы, задача которых выполнять некоторую операцию над каждым элементом массива. Вот несколько примеров типичных операций над массивом a длины N :

```
идиома   for (i = 0; i < N; i++); /* установка  $a$  в нуль */
           a[i] = 0;
```

```
идиома   for (i = 0; i < N; i++)
           scanf("%d", &a[i]); /* считывает данные в  $a$  */
```

```
идиома   for (i = 0; i < N; i++)
           sum += a[i]; /* суммирует элементы из  $a$  */
```

Обратите внимание, что мы должны использовать символ `&`: при вызове `scanf` для чтения элемента массива, как и в случае с обычной переменной.

Си не требует проверки границ индексов; если индекс выходит за пределы диапазона, поведение программы не определено. Одна из причин выхода индекса за пределы: забываем, что массив с n элементами индексируется от 0 до $n-1$, а не от 1 до n . (Как сказал один из моих профессоров: "В этом бизнесе вы всегда смещаете на единицу". Разумеется, он был прав.) Следующий пример иллюстрирует странный эффект, который может быть вызван этой распространённой ошибкой:

```
int a[10], i;  
for ( i = 1; i <= 10; i++ )  
    a[i] = 0;
```

С некоторыми компиляторами эта невинно выглядящая инструкция вызывает бесконечный цикл! Когда i достигает 10, программа сохраняет 0 в `a[10]`. Но `a[10]` не существует, поэтому 0 переходит в память расположенную сразу после `a[9]`. Если переменная i , окажется, следующей за `a[9]` в памяти - что может иметь место - тогда i , то будет сброшена в 0, заставляя цикл запускаться.

Индексом массива может быть любое целочисленное выражение:

```
a[i+j*10] = 0;
```

Выражение может иметь побочные эффекты:

```
i = 0;
while (i < N)
    a[i++] = 0;
```

Давайте проследим этот код. После того, как *i* установлен в 0, инструкция `while` проверяет, является ли *i* меньшим *N*. Если это так, то значение 0 присваивается `a[0]`, *i* увеличивается и цикл повторяется. Обратите внимание, что `[++ i]` было бы неправильным, потому что 0 был бы присвоен `a[1]` во время первой итерации цикла. Будьте осторожны, если индекс массива имеет побочный эффект. Например, следующий цикл, который должен копировать элементы массива *b* в массив *a*, может работать некорректно:

```
i = 0;
while (i < N)
    a[i] = b[i++];
```

Выражение `a[i] = b[i++]` получает значение *i*, а также ещё изменяет *i* - в выражении, которое, как мы видели в Разделе 4.4—вызывает неопределённое поведение. Конечно, мы можем легко избежать проблемы, удалив инкремент из индекса:

```
for ( i= 0; i < N; i++ )
    a[i]=b[i];
```

ПРОГРАММА Запись ряда чисел в обратном порядке

Наша первая программа с массивом запрашивает у пользователя ввод ряда чисел, а затем записывает числа в обратном порядке:

Введите 10 чисел: 34 32 49 102 7 94 23 11 50 31

В обратном порядке: 31 50 11 23 94 7 102 49 32 34

```
//reverse.c
```

```
/* реверсирует ряд чисел */
```

```
#include <stdio.h>
```

```
#define N 10
```

```
int main(void){
```

```
    int a[N], i;
```

```
    printf("Введите %d чисел: ", N);
```

```
    for (i = 0; i < N; i++)
```

```
        scanf("%d", &a[i]);
```

```
    printf("В обратном порядке:");
```

```
    for (i = N - 1; i >= 0; i--)
```

```
        printf(" %d", a[i]);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

Эта программа показывает, насколько полезные макросы могут быть в сочетании с массивами. Макрос *N* используется четыре раза в программе: в объявлении *a*, в `printf`, отображающем приглашение, и в обоих циклах `for`. Если мы позже решим изменить размер массива, нам нужно только отредактировать определение *N* и перекомпилировать программу. Ничто другое не нужно будет изменять; даже приглашение все равно будет правильным.

Инициализация массива

Массив, как и любая другая переменная, может быть задан начальным значением в момент его объявления. Однако правила несколько сложноваты, поэтому мы рассмотрим некоторые из них сейчас и оставим другие на потом.

Наиболее распространённой формой инициализатора массива является список константных выражений, заключённых в фигурные скобки и разделённых запятыми:

```
int a [10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

Если инициализатор короче массива, оставшимся элементам массива присваивается значение 0:

```
int a[10] = {1, 2, 3, 4, 5, 6}; /* инициализированные значения a  
{1, 2, 3, 4, 5, 6, 0, 0, 0, 0} */
```

Используя эту возможность, мы можем легко инициализировать массив нулями:

```
int a [10] = {0}; /* инициализированные значения a равны  
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0} */
```

Недопустимо, чтобы инициализатор был полностью пуст, поэтому мы поместили 0 в фигурные скобки. Также недопустимо, чтобы инициализатор был длиннее, чем инициализируемый массив. Если присутствует инициализатор, длина массива может быть опущена

```
int a [] = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
```

Компилятор использует длину инициализатора для определения длины массива. Массив по-прежнему имеет фиксированное количество элементов (10, в этом примере), как если бы мы указали длину явно.

Назначенные инициализаторы

Часто бывает, что относительно небольшое количество элементов массива необходимо инициализировать явно; Другим элементам могут быть заданы значения по умолчанию. Рассмотрим следующий пример:

```
int a [15] = (0, 0, 29, 0, 0, 0, 0, 0, 0, 7, 0, 0, 0, 0, 43);
```

Мы хотим, чтобы элемент 2 массива был 29, элемент 9 - 7 и элемент 14 - 48, но остальные значения равны нулю. Для большого массива запись такого инициализатора является утомительной и подверженной ошибкам (что, если между двумя ненулевыми значениями было бы 200 нулей?). Для решения этой проблемы могут использоваться назначенные инициализаторы C99. Вот как мы могли бы повторить предыдущий пример, используя назначенный инициализатор:

```
int a [15] = {[2] = 29, [9] = 7, [14] = 48};
```


Каждое число в скобках называется индексом.

(Назначенные инициализаторы имеют вид: [индекс] = значение)

Помимо того, что они короче и легче читаются (по крайней мере для некоторых массивов), у назначенных инициализаторов есть ещё одно преимущество: порядок, в котором перечислены эти элементы, уже не имеет значения. Таким образом, наш предыдущий пример мог быть также написан следующим образом:

```
int a [15] = {[14] = 48, [9] = 7, [2] = 29};
```

Индексы должны быть целыми константными выражениями. Если инициализируемый массив имеет длину *n*, каждый индекс должен быть между 0 и *n* - 1.

Однако, если длина массива опущена, индекс может быть любым неотрицательным целым числом. В последнем случае компилятор выведет длину массива из наибольшего индекса.

В следующем примере тот факт, что 23 появляется как индекс, заставит массив иметь длину 24:

```
int b [] = {[5] = 10, [23] = 13, [11] = 36, [15] = 29};
```

Инициализатор может использовать как более старую (поэлементную) технику, так и более новую технику (назначения):

```
int c[10] = {5, 1, 9, [4] = 3, 7, 2, [8] = 6};
```

ПРОГРАММА Проверка числа на наличие в нем повторных цифр

Наша следующая программа проверяет, появляются ли какие-либо цифры в числе более одного раза. После того, как пользователь вводит число, программа печатает либо

Есть повторяющаяся цифра, либо

Нет повторяющейся цифры:

Введите номер телефона: 28212

Повторяющаяся цифра

Число 28212 имеет повторяющуюся цифру (2); число, подобное 9357, нет.

Программа использует массив булевых значений, чтобы отслеживать, какие цифры появляются в числе. Массив, названный `digit_seen`, индексируется от 0 до 9, чтобы соответствовать 10 возможным цифрам. Первоначально каждый элемент массива имеет значение `false` (инициализатор для `digit_seen` имеет значение `{false}`, которое инициализирует только первый элемент массива. Однако компилятор автоматически сделает оставшиеся элементы равными нулю, что равно `false`.)

Когда задано число n , программа проверяет цифры n по одной за раз, сохраняя каждую в числовой переменной и затем используя её как индекс в `digit_seen`. Если `digit_seen [digit]` истинно, то цифра появляется по крайней мере дважды в n . С другой стороны, если `digit_seen [digit]` - `false`, то цифры не

были замечены раньше, поэтому программа устанавливает `digit_seen [digit]` в `true` и продолжает движение.

// repdigit.c (глава 8)

/ Проверяет числа на наличие повторных цифр */*

```
#include <stdbool.h> /* C99 only */
```

```
#include <stdio.h>
```

```
int main(void){
```

```
    bool digit_seen[10] = {false};
```

```
    int digit;
```

```
    long n;
```

```
    printf("Enter a number: ");
```

```
    scanf("%ld", &n);
```

```
    while (n > 0) {
```

```
        digit = n % 10;
```

```
        if (digit_seen[digit])
```

```
            break;
```

```
        digit_seen[digit] = true;
```

```
        n /= 10;
```

```
    }
```

```
    if (n > 0)
```

```
        printf(" Есть повторяющаяся цифра\n");  
    else  
        printf("Нет повторяющейся цифры\n");  
  
    return 0;  
}
```

Эта программа использует имена `bool`, `true` и `false`, которые определены в заголовочном файле `<stdbool.h>` C99. Если ваш компилятор не поддерживает этот заголовок, вам нужно будет определить эти имена самостоятельно. Один из способов сделать это - поставить над функцией `main` следующие строки:

```
#define true 1  
#define false 0  
typedef int bool;
```

Обратите внимание, что ***n*** имеет тип `long`, что позволяет пользователю вводить числа до 2 147 483 647 (или более, на некоторых машинах).

Использование оператора `sizeof` с массивами

Оператор `sizeof` может определять размер массива (в байтах). Если `a` представляет собой массив из 10 целых чисел, то `sizeof(a)` обычно равен 40 (при условии, что каждое целое число требует четыре байта). Мы также можем использовать `sizeof` для измерения размера элемента массива, например `a[0]`. Деление размера массива на размер элемента даёт длину массива:

`sizeof(a) / sizeof(a[0])`

Некоторые программисты используют это выражение, когда необходима длина массива. Чтобы установить массив в 0, например, мы могли бы написать

```
for (i = 0; i < sizeof(a) / sizeof(a[0]); i++)  
    a[i]=0;
```

При использовании этой методики цикл не нужно модифицировать, если позднее длина массива будет изменена. Конечно, использование макроса для представления длины массива обладает определёнными преимуществами, но метод `sizeof` немного лучше, так как нет имени макроса, которое нужно запомнить (и, возможно, неправильно). Одно незначительное раздражение заключается в том, что некоторые компиляторы выдают предупреждающее сообщение для выражения `i < sizeof (a) / sizeof (a [0])`. Переменная `i`, вероятно, имеет тип `int` (знаковый тип), тогда как `sizeof` создаёт значение типа `size_t` (тип без знака). Из раздела 7.4 мы знаем, что сравнение знакового целого с беззнаковым целым является опасной практикой, хотя в этом случае это безопасно, поскольку и `i`, и `sizeof (a) / sizeof (a [0])` имеют неотрицательные значения. Чтобы избежать

предупреждения, мы можем добавить приведение, которое преобразует

`sizeof (a) / sizeof (a [0])` в целое со знаком:

```
for (i = 0; i < (int) (sizeof(a) / sizeof(a[0])); i++)  
    a[i] = 0;
```

Написание `(int) (sizeof (a) / sizeof (a [0]))` немного громоздко: определение макроса, который его представляет, часто бывает полезным:

```
#define SIZE ((int) (sizeof(a) / sizeof(a[0])))
```

```
for (i = 0; i < SIZE; i++)  
    a[i] = 0;
```

Если мы вернулись к использованию макроса, в чем же преимущество `sizeof`? Мы ответим на этот вопрос в следующей главе (трюк заключается в добавлении параметра в макрос).

ПРОГРАММА Расчёт процентов

Наша следующая программа печатает в таблице, показывающую стоимость 100 долларов США, инвестированных с различными процентными ставками в течение нескольких лет. Пользователь вводит процентную ставку и количество лет, на которые будут инвестированы деньги. В таблице покажут ценность денег с интервалом в

один год по этой процентной ставке и по следующим четырем более высоким ставкам - при условии, что проценты начисляются один раз в год. Вот как будет выглядеть сессия с программой:

Введите процентную ставку: 6

Введите число лет: 5

Годы	6%	7%	8%	9%	10%
<u>1</u>	<u>106.00</u>	<u>107.00</u>	<u>108.00</u>	<u>109.00</u>	<u>110.00</u>
<u>2</u>	<u>112.36</u>	<u>114.49</u>	<u>116.64</u>	<u>118.81</u>	<u>121.00</u>
<u>3</u>	<u>119.10</u>	<u>122.50</u>	<u>125.97</u>	<u>129.50</u>	<u>133.10</u>
<u>4</u>	<u>126.25</u>	<u>131.08</u>	<u>136.05</u>	<u>141.16</u>	<u>146.41</u>
<u>5</u>	<u>133.82</u>	<u>140.26</u>	<u>146.93</u>	<u>153.86</u>	<u>161.05</u>

Ясно, что мы можем использовать инструкцию **for** для печати первой строки. Вторая строка немного сложнее, поскольку её значения зависят от чисел в первой строке. Наше решение состоит в том, чтобы сохранить первую строку в массиве по мере её вычисления. Затем используйте значения в массиве для вычисления второй строки. Конечно, этот процесс можно повторить для третьей и последующих строк. Мы получим две инструкции `for`, одну внутри другой. Внешний цикл будет отсчитываться от *i* до количества лет, запрашиваемых пользователем. Внутренний цикл будет увеличивать процентную ставку от её самого низкого значения до его самого высокого значения.

```
//interest.c
```

```
/* Печать таблицы расчёта процентов */
```

```
#include <stdio.h>

#define NUM_RATES ((int) (sizeof(value) / sizeof(value[0])))
#define INITIAL_BALANCE 100.00

int main(void){
    int i, low_rate, num_years, year;
    double value[5];

    printf("Введите процентную ставку: ");
    scanf("%d", &low_rate);
    printf("Введите число лет: ");
    scanf("%d", &num_years);

    printf("\nГоды");
    for (i = 0; i < NUM_RATES; i++) {
        printf("%6d%%", low_rate + i);
        value[i] = INITIAL_BALANCE;
    }
    printf("\n");

    for (year = 1; year <= num_years; year++) {
        printf("%3d ", year);
```



```
    for (i = 0; i < NUM_RATES; i++) {  
        value[i] += (low_rate + i) / 100.0 * value[i];  
        printf("%7.2f", value[i]);  
    }  
    printf("\n");  
}  
  
return 0;  
}
```

Обратите внимание на использование NUM_RATES для управления двумя циклами for. Если позже мы изменим размер массива значений, циклы будут корректироваться автоматически.

8.2 Многомерные массивы

Массив может иметь любое количество измерений. Например, следующее объявление создаёт двумерный массив (матрицу в математической терминологии):

```
int m [5] [9];
```

Массив *m* имеет 5 строк и 9 столбцов.

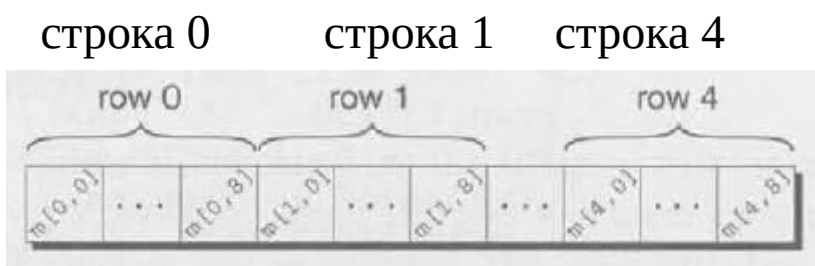
И строки, и столбцы индексируются от 0. Как показано на следующем рисунке:

	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									

Чтобы получить доступ к элементу m в строке i , столбце j , мы должны написать $m[i][j]$. Выражение $m[i]$ обозначает строку i в m и $m[i][j]$, затем выбирает элемент j в этой строке.

Сопровождайтесь соблазну писать $m[i, j]$ вместо $m[i][j]$. Си рассматривает запятую как оператор в этом контексте, поэтому $m[i, j]$ совпадает с $m[j]$.

Хотя мы визуализируем двумерные массивы как таблицы, в компьютерной памяти они фактически хранятся не так. Си хранит массивы в построчном порядке: сначала строка 0, затем строка 1 и т. Д. Например, вот как хранится массив m :



Мы обычно будем игнорировать эту деталь, но иногда она будет влиять на наш код. Как циклы `for` идут рука об руку с одномерными массивами, так и вложенные циклы `for` идеально подходят для обработки многомерных массивов. Рассмотрим, например, проблему инициализации массива для использования в качестве единичной матрицы. (В математике единичная матрица имеет 1 на главной диагонали, где индексы строк и столбцов одинаковы, и 0 - где индексы различны).

Мы должны будем посетить каждый элемент в массиве некоторым систематическим способом.

Пара вложенных циклов `for`, один из которых проходит по каждому индексу строк, и другой, который проходит по каждому индексу столбцов, идеально подходит для задания:

```
#define N 10
double ident[N][N];
int row, col;
for (row = 0; row < N; row++)
    for (col = 0; col < N; col++)
        if (row == col)
            ident [row] [col] = 1.0;
        else
            ident [row] [col] = 0. 0;
```

Многомерные массивы играют меньшую роль в Си, чем во многих других языках программирования, в первую очередь

потому, что Си предоставляет более гибкий способ хранения многомерных данных: массивы указателей.

Инициализация многомерного массива

Мы можем создать инициализатор для двумерного массива, вложив в него одномерные инициализаторы:

```
int m [5] [9] = {{1, 1, 1, 1, 1, 0, 1, 1, 1},  
                 {0, 1, 0, 1, 0, 1, 0, 1, 0},  
                 {0, 1, 0, 1, 1, 0, 0, 1, 0},  
                 {1, 1, 0, 1, 0, 0, 0, 1, 0},  
                 {1, 1, 0, 1, 0, 0, 1, 1, 1}};
```

Каждый внутренний инициализатор предоставляет значения для одной строки матрицы. Аналогичным образом строятся и инициализаторы для многомерных массивов.

Си предоставляет множество способов сокращения инициализаторов для многомерных массивов:

■ Если инициализатор недостаточно велик для заполнения многомерного массива, оставшимся элементам присваивается значение 0. Например, следующий инициализатор заполняет только первые три строки **m**; последние две строки будут содержать нули:

```
int m[5][9] = {{1, 1, 1, 1, 1, 0, 1, 1, 1},
               {0, 1, 0, 1, 0, 1, 0, 1, 0},
               {0, 1, 0, 1, 1, 0, 0, 1, 0}};
```

■ Если внутренний список недостаточно длинный для заполнения строки, остальные элементы в строке инициализируются значением 0:

```
int m [5] [9]
= {{1, 1, 1, 1, 1, 0, 1, 1, 1},
   {0, 1, 0, 1, 0, 1, 0, 1},
   {0, 1, 0, 1, 1, 0, 0, 1},
   {1, 1, 0, 1, 0, 0, 0, 1},
   {1, 1, 0, 1, 0, 0, 1, 1, 1}};
```

Мы можем даже опустить внутренние фигурные скобки:

```
int m [5] [9] = {1, 1, 1, 1, 1, 0, 1, 1, 1,
                 0, 1, 0, 1, 0, 1, 0, 1, 0,
                 0, 1, 0, 1, 1, 0, 0, 1, 0,
                 1, 1, 0, 1, 0, 0, 0, 1, 0,
                 1, 1, 0, 1, 0, 0, 1, 1, 1};
```

Как только компилятор увидел достаточно значений для заполнения одной строки, он начинает заполнять следующую.

Опускание внутренних фигурных скобок в многомерном инициализаторе массива может быть рискованным, поскольку дополнительный элемент (или, что ещё хуже, отсутствующий элемент) повлияет на остальную часть инициализатора.

Игнорирование фигурных скобок заставляет некоторые компиляторы производить предупреждающее сообщение, такое как "отсутствие скобок вокруг инициализации",

Назначенные инициализаторы C99 работают с многомерными массивами. Например, мы могли бы создать матрицу идентичности 2×2 следующим образом:

```
double ident [2] [2] = {[0] [0] = 1.0, [1] [1] = 1.0};
```

Как обычно, все элементы, для которых не задано значение, по умолчанию будут равны нулю.

Константные массивы

Любой массив, одномерный или многомерный, можно сделать "константным", начав его объявление со слова `const`:

```
const char hex_chars [] = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A',  
'B', 'C', 'D', 'E', 'F'};
```

Массив, объявленный как `const`, не должен модифицироваться программой; компилятор будет сразу обнаруживать попытку модифицировать элемент. Объявление массива как `const`, имеет несколько основных преимуществ.

Этим документируется что программа не будет изменять массив, что может быть ценной информацией для тех, кто позже читает код. Это также помогает компилятору выявлять ошибки, сообщая ему, что мы не намерены изменять массив, `const` не ограничивается массивами; он работает с любой переменной, как мы увидим позже. Однако, `const` особенно полезно в объявлениях массивов, потому что массивы могут содержать справочную информацию, которая не изменится во время выполнения программы.

ПРОГРАММА Сдача карт

Наша следующая программа иллюстрирует как двумерные массивы, так и константные массивы. В программе используется случайная комбинация из стандартной колоды игровых карт. (Если вы не успели поиграть в игры в последнее время, на каждой карте в стандартной колоде есть масти-трефы, бубны, червы или пики, и достоинство - двойка, тройка, четвёрка, пятёрка, шестёрка, семёрка, восьмёрка, девятка, десятка, валет, дама, король или туз.) Мы попросим пользователя указать, сколько карт должно быть в руке:

Введите количество карт в руке: 5

Ваша рука: 7c 2s 5d как 2h

Не сразу видно, как мы будем писать такую программу. Как мы можем случайно выбрать карты из колоды? И как мы

избежим выбора одной и той же карты дважды? Давайте рассмотрим эти проблемы отдельно.

Чтобы выбрать карты случайным образом, мы будем использовать несколько функций библиотеки Си.

Функция времени (из `<time.h>`) возвращает текущее время, закодированное одним числом. Функция `srand` (из `<stdlib.h>`) инициализирует генератор случайных чисел Си. Передача возвращаемого значения времени в `srand` предотвращает выполнение программой одной и той же карты каждый раз, когда мы запускаем её. Функция `rand` (также из `<stdlib.h>`) производит кажущееся случайным число каждый раз, когда она вызывается. Используя оператор `%`, мы можем масштабировать возвращаемое значение от `rand` так, чтобы оно попадало между 0 и 3 (для масти) или между 0 и 12 (для достоинства). Чтобы избежать двойного выбора одной карты, нам нужно будет отслеживать, какие карты уже выбраны. Для этой цели мы будем использовать массив с именем `in_hand`, который имеет четыре строки (по одному для каждой масти) и 13 столбцов (по одному для каждого достоинства).

Другими словами, каждый элемент в массиве соответствует одной из 52 карт колоды. Для начала все элементы массива будут ложными. Каждый раз, когда мы выбираем карту случайным образом, мы проверяем, является ли элемент `in_hand`, соответствующий этой карте, `true` или `false`. Если он истинный, нам придётся выбрать другую карту. Если `false`, мы сохраним `true` в элементе массива этой карты, чтобы позже напоминать, что эта карта уже выбрана.

Как только мы проверим, что карта "новая" - ранее не была выбрана, нам нужно перевести её числовое достоинство и масть в символы, а затем отобразить карту.

Чтобы перевести достоинство и масть в форму символа, мы создадим два массива символов: один для достоинства и один для масти, а затем используем числа для индексирования массивов. Эти массивы не изменятся во время выполнения программы, поэтому мы можем объявить их константами.

```
//deal.c
```

```
/* Делает раздачу карт случайной */
```

```
#include <stdbool.h> /* C99 only */
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define NUM_SUITS 4
```

```
#define NUM_RANKS 13
```

```
int main(void){
```

```
    bool in_hand[NUM_SUITS][NUM_RANKS] = {false};
```

```
    int num_cards, rank, suit;
```

```
    const char rank_code[] = {'2','3','4','5','6','7','8',
```

```
    '9','t','j','q','k','a'}; /* t-10, j - валет 11, q - дама 12, к -  
король 13, а- туз 14 */
```

```
    const char suit_code[] = {'c','d','h','s'};
```

```
/* c - трефы, d - бубны, h - червы, s - пики */
```

```
srand((unsigned) time(NULL));

printf("Введите количество карт в руке: ");
scanf("%d", &num_cards);

printf("В вашей руке:");
while (num_cards > 0) {
    suit = rand() % NUM_SUITS; /* выбирает случайную
                                масть */
    rank = rand() % NUM_RANKS; /* выбирает
                                случайный ранг */
    if (!in_hand[suit][rank]) {
        in_hand[suit][rank] = true;
        num_cards--;
        printf(" %c%c", rank_code[rank], suit_code[suit]);
    }
}
printf("\n");

return 0;
}
```

Обратите внимание на инициализатор для массива in_hand:

```
bool in_hand [NUM_SUITS] [NUM_RANKS] = {false} ;
```

Хотя `in_hand` - двумерный массив, мы можем использовать одну пару фигурных скобок (рискуя, возможно, получить предупреждение от компилятора).

Кроме того, в инициализаторе мы указали только одно значение, зная, что компилятор напишет 0 (`false`) для остальных элементов.

8.3 Массивы переменной длины (C99)

В Разделе 8.1 указано, что длина переменной массива должна определяться константным выражением.

Однако в C99 иногда можно использовать выражение, которое не является константой. Следующая модификация программы `reverse.c` (Раздел 8.1) иллюстрирует эту возможность:

```
//reverse2.c
```

```
/* Реверсирование последовательности чисел используя  
массив переменной длины — только в C99 */
```

```
#include <stdio.h>
```

```
int main(void){
```

```
    int i, n;
```

```
    printf("Сколько чисел вы хотите реверсировать? ");
```

```
    scanf("%d", &n);
```

```
int a[n]; /* только в C99 - длина массива зависит от n */

printf("Введите %d чисел: ", n);
for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

printf("В обратом порядке:");
for (i = n - 1; i >= 0; i--)
    printf(" %d", a[i]);
printf("\n");

return 0;
}
```

Массив *a* в этой программе является примером массива переменной длины (или VLA для краткости (variable-length array)). *(Несмотря на своё название, эти массивы не могут изменять своих размеров, иначе это поставило бы под угрозу целостность данных, находящихся в памяти по соседству с ними. Массив переменной длины - это массив длина (или размер) которого определяется во время выполнения, но только один раз.)*

Длина VLA вычисляется при выполнении программы, а не при компиляции. Главное преимущество VLA состоит в том, что программисту не нужно выбирать произвольную длину при объявлении массива; вместо этого сама программа может

точно рассчитать, сколько элементов необходимо. Если выбор сделает программист, вероятно, что массив будет слишком длинным (трата памяти) или слишком коротким (в результате чего программа не будет выполнена).

В программе `reverse2.c` число, введённое пользователем, определяет длину *a*; программисту не нужно выбирать фиксированную длину, в отличие от исходной версии программы. Длина VLA не обязана указываться одной переменной. Правильными являются также произвольные выражения, возможно содержащие операторы. Например:

```
int a [3*i+5];
```

```
int b[j+k];
```

VLA также могут быть многомерными:

```
int c [m] [n];
```

Основное ограничение на VLA заключается в том, что у них не может быть статической длительности хранения (мы ещё не видели никаких массивов с этим свойством.) Другое ограничение заключается в том, что VLA может не иметь инициализатора.

Массивы переменной длины чаще всего встречаются в функциях, отличных от `main`. Одно большое преимущество VLA, который принадлежит функции *f*, состоит в том, что он может иметь разную длину при каждом вызове *f*. Мы рассмотрим эту функцию в Разделе 9.3.

Вопросы и ответы

Вопрос: Почему индексы массива начинаются с 0, а не с 1?

Ответ: Начинающиеся с 0 индексы, позволяют немного упростить компилятор. Кроме того, индексирование массива будет немного быстрее.

Вопрос: Что если мне нужен массив с индексами, которые идут от 1 до 10 вместо 0 до 9?

Ответ: Вот общий приём: объявите массив, который будет содержать 11 элементов вместо 10. Индексы будут идти от 0 до 10, но вы можете просто игнорировать элемент 0.

Вопрос: Можно ли использовать символ в качестве индекса массива?

Ответ: Да, потому что Си обрабатывает символы как целые числа. Вероятно, вам придётся "масштабировать" символ, прежде чем использовать его в качестве индекса.

Предположим, что мы хотим, чтобы массив `letter_count` отслеживал число для каждой буквы в алфавите. Массиву понадобится 26 элементов, поэтому мы объявим его следующим образом:

```
int letter_count[26];
```

Однако мы не можем использовать буквы для индексирования `letter_count` напрямую, потому что их целочисленные значения не попадают в диапазон от 0 до 25. Чтобы масштабировать нижний регистр до нужного диапазона, мы можем просто

вычесть 'a'; Для масштабирования заглавной буквы мы вычтем 'A'. Например, если ch содержит строчную букву, мы пишем

```
letter_count[ch - 'a'] = 0;
```

для сброса числа, соответствующего ch. Небольшое предупреждение: этот метод не является полностью переносимым, поскольку предполагает, что буквы имеют последовательные коды. Тем не менее, он работает с большинством наборов символов, включая ASCII.

Вопрос: Кажется, что назначенный инициализатор может в конечном итоге инициализировать элемент массива более одного раза. Рассмотрим следующее объявление массива:

```
int a[]={ 4, 9, 1, 8, [0] = 5, 7 };
```

Является ли это объявление допустимым, и если да, какова длина массива?

Ответ: Да, объявление допустимо. Вот как это работает: когда компилятор обрабатывает список инициализаторов, он отслеживает, какой элемент массива должен быть инициализирован следующим. Обычно следующий элемент является элементом, который следует за последним инициализированным элементом. Однако, когда назначенный инициализатор появляется в списке, он заставляет следующий элемент быть тем, который представлен индексом, даже если этот элемент уже был инициализирован. Вот пошаговое описание того, как компилятор будет обрабатывать инициализатор для массива **a**:

4 инициализирует элемент 0, а следующий элемент, который должен быть инициализирован, - элемент 1.

9 инициализирует элемент 1; следующим элементом, который должен быть инициализирован, является элемент 2.

1 инициализирует элемент 2; следующим элементом, который должен быть инициализирован, является элемент 3.

8 инициализирует элемент 3; Следующим элементом, который должен быть инициализирован, является элемент 4.

Обозначение [0] приводит к тому, что следующим элементом становится 0, поэтому 5 инициализирует элемент 0 (заменяя ранее сохранённые 4).

Следующим элементом, который должен быть инициализирован, является элемент 1.

7 инициализирует элемент 1 (заменяя ранее сохранённые 9).

Следующий элемент, который нужно инициализировать, - это элемент 2 (что не имеет значения, поскольку мы находимся в конце списка).

Чистый эффект такой же, как если бы мы написали

```
int a[] = {5, 7, 1, 8};
```

Таким образом, длина этого массива равна четырём.

Вопрос: Компилятор выдаст мне сообщение об ошибке, если я попытаюсь скопировать один массив в другой, используя оператор присваивания. Что не так?

Ответ: Хотя это выглядит вполне правдоподобно, присваивание

```
a = b; /* a и b - массивы */
```

действительно является недопустимым.

Причина его недопустимости не очевидна; это связано со

своеобразным соотношением между массивами и указателями на Си, которое мы рассмотрим в Главе 12. Самый простой способ скопировать один массив в другой - использовать цикл, который копирует элементы один за другим:

```
for (i = 0; i < N; i++)  
    a[i] = b[i];
```

Другая возможность заключается в использовании функции `memcpy` ("memory copy") из заголовка `<string.h>`, `memcpy` - это функция низкого уровня, которая просто копирует байты из одного места в другое. Чтобы скопировать массив **b** в массив **a**, Используйте `memcpy` следующим образом:

```
memcpy(a,b, sizeof (a) ) ;
```

Многие программисты предпочитают `memcpy`, особенно для больших массивов, потому что она потенциально быстрее обычного цикла.

Вопрос: В Разделе 6.4 указано, что C99 не позволяет инструкции `goto` обходить объявление массива переменной длины. В чем причина этого ограничения?

Ответ: Память, используемая для хранения массива переменной длины обычно выделяется тогда, когда во время выполнения программы достигается объявление массива. Обход объявления с помощью инструкции `goto` может привести к тому, что программа обратится к элементам массива, которым память никогда не выделялась.

Упражнения

К разделу 8.1

W 1. Мы обсудили использование выражения

`sizeof (a) / sizeof (a [0])`

для вычисления количества элементов в массиве. Выражение `sizeof (a) / sizeof (t)`, где *t* является типом элементов *a*, также будет работать, но это считается худшим способом. Почему?

W 2. В разделе Вопросы и ответы показано, как использовать букву в качестве индекса массива. Опишите, как использовать цифру (в символьной форме) в качестве индекса.

3. Напишите объявление массива с именем `weekend`, содержащее семь значений `bool`. Включите инициализатор, который делает первые и последние значения истинными; все остальные значения должны быть ложными.

4. (C99) Повторите упражнение 3, но на этот раз используйте назначенный инициализатор. Сделайте инициализатор как можно короче.

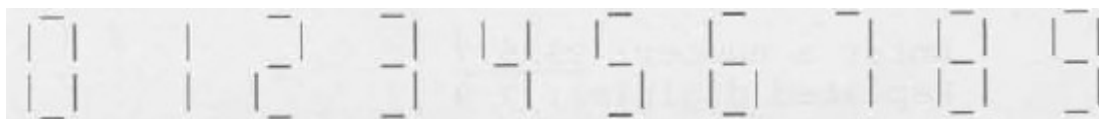
5. Числа Фибоначчи равны 0, 1, 1, 2, 3, 5, 8, 13, *t*..., где каждое число является суммой двух предыдущих чисел. Напишите

399

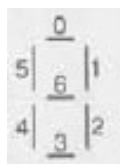
фрагмент программы, объявляющий массив с именем `fib_numbers` длиной 40 и заполняющий массив с первыми 40 числами Фибоначчи.

Подсказка: Заполните первые два числа по отдельности, затем используйте цикл для вычисления оставшихся чисел.

6. Калькуляторы, часы и другие электронные устройства часто используют семисегментные дисплеи для числового вывода. Чтобы сформировать цифру, такие устройства "включают" некоторые из семи сегментов, оставляя другие "выключенными":



Предположим, что мы хотим настроить массив, который запоминает, какие сегменты должны быть в состоянии "включено" для каждой цифры. Давайте пронумеруем сегменты следующим образом:



Вот как может выглядеть массив: каждая строка представляет одну цифру:

```
const int segments [10] [7] = {{1, 1, 1, 1, 1, 1, 0},...};
```

Я дал вам первую строку инициализатора; заполните остальные.

W 7. Используя сочетания клавиш, описанные в Разделе 8.2, сжимайте инициализатор для массива сегментов (Упражнение 6) настолько, насколько сможете.

8. Напишите объявление для двумерного массива с именем `temperature_readings`, в котором хранится один месяц часовых показаний температуры. (Для простоты предположим, что месяц имеет 30 дней.) Строки массива должны представлять дни месяца; столбцы должны представлять часы дня.

9. Используя массив Упражнения 8, напишите фрагмент программы, который вычисляет среднюю температуру за месяц (усреднённый по всем дням месяца и всем часам дня)

10. Напишите объявление для массива 8 x 8 `char` с именем `chess_board`. Включите инициализатор, который помещает в массив следующие данные (один символ на элемент массива)

```

r n b q k b n r
P P P P P P P P
. . . . .
. . . . .
. . . . .
P P P P P P P P
R N B Q K B N R
```

11. Напишите фрагмент программы, который объявляет массив символов 8 x 8 с именем `checkers board` и сохранит в массиве следующие данные (по одному символу на элемент массива):

B	R	B	R	B	R	B	R
R	B	R	B	R	B	R	B
B	R	B	R	B	R	B	R
R	B	R	B	R	B	R	B
B	R	B	R	B	R	B	R
R	B	R	B	R	B	R	B
B	R	B	R	B	R	B	R
R	B	R	B	R	B	R	B

Подсказка: элемент в строке i , столбце j , должен быть буквой B, если $i + j$ - чётное число.

Проекты по программированию

1. Измените программу `repdigit.c` Раздела 8.1 так, чтобы она показывала, какие цифры (если они были) были повторены:

Введите число: 939577

Повторяющиеся цифры: 7 9

2. Измените программу `repdigit.c` Раздела 8.1 так, чтобы она выводила таблицу, показывающую, сколько раз каждая цифра

появляется в числе:

Введите число: 41271092

Цифра: 0 1 2 3 4 5 6 7 8 9

Вхождения: 1 2 2 0 1 0 0 1 0 1

3. Измените программу `repdigit.c` Раздела 8.1, чтобы пользователь мог ввести более одного числа для проверки на повторяющиеся цифры.

Программа должна завершаться, когда пользователь вводит число, меньшее или равное 0.

4. Измените программу `reverse.c` Раздела 8.1 на использование выражения `(int) (sizeof (a) / sizeof (a [0]))` (или макроса с этим значением) для длины массива.

W 5. Измените программу `interest.c` Раздела 3.1 так, чтобы она представляла проценты ежемесячно, а не ежегодно. Форма вывода не должна изменяться; баланс должен показываться с ежегодными интервалами.

6. Прототипом интернет-новичка является парень по имени B1FF, у которого есть уникальный способ писать сообщения. Вот типичное коммюнике B1FF:

H3Y DUD3, C 15 RILLY COOL!!!!!!!!!!!!

Напишите "B1FF фильтр", который читает сообщение, введённое пользователем, и переводит его на жаргон B1FF:

Введите сообщение: Hey dude, C is rilly cool

В жаргоне B1FF: HSY DUD3, C 15 RILLY COOL !!!!!!!!!!!!!!!

(прим.пер: "Эй, чувак, Си реально круто", rilly - опечатка really. Часто совершаемая при случае идиотами, её можно использовать нарочно в насмешке или сарказме.)

Ваша программа должна преобразовать сообщение в прописные буквы, заменить цифры на определённые буквы (A -> 4, B -> 8, E -> 3, I -> 1, O -> 0, S -> 5) и затем добавьте 10 или около того восклицательных знаков.

Подсказка. Сохраните исходное сообщение в массиве символов, затем возвращайте из массива, переводя и печатая символы по одному.

7. Напишите программу, которая читает массив целых чисел 5 x 5 и затем печатает суммы строк и суммы столбцов:

Введите строку 1: 8 3 9 0 10

Введите строку 2: 3 5 17 1 1

Введите строку 3: 2 8 6 23 1

Введите строку 4: 15 7 3 2 9

Введите строку 5: 6 14 2 6 0

Всего по строкам: 30 27 4036 28

Всего по колонкам: 34 37 37 32 21

8. Измените Проект по программированию 7 так, чтобы он запрашивал пять оценок по экзаменам для каждого из пяти студентов, затем вычисляет общий балл и средний балл для каждого учащегося, а также средний балл, высший балл и низший балл для каждого экзамена.

9. Напишите программу, которая генерирует "случайное блуждание" по массиву 10 x 10. Массив будет содержать символы (все изначально "."). Программа должна беспорядочно "ходить" от элемента к элементу, всегда идя на один элемент вверх, вниз, налево или направо. Элементы, посещённые программой, будут помечены буквами от A до Z в порядке следования. Вот пример желаемого результата:

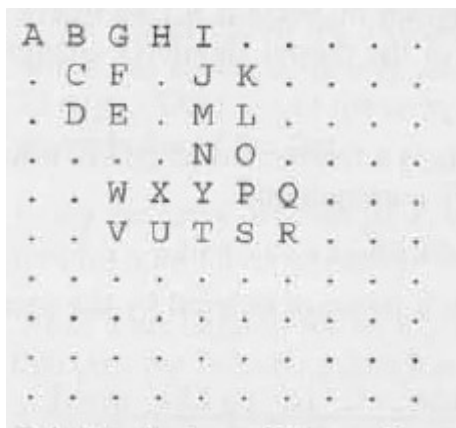
```

A . . . . . . . . .
B C D . . . . . . .
. F E . . . . . . .
H G . . . . . . . .
I . . . . . . . . .
J . . . . . . . Z .
K . . R S T U V Y .
L M P Q . . . W X .
. N O . . . . . . .
. . . . . . . . .

```

Подсказка: используйте функции `srand` и `rand` (см. `deal.c`) для генерации случайных чисел. После генерирования числа посмотрите его остаток при делении на 4. Возможны четыре возможных значения для остатка: 0, 1, 2 и 3 - указание направления следующего хода. Перед выполнением перемещения убедитесь, что (а) оно не выйдет за пределы массива, и (б) оно не приведёт нас к элементу, которому уже присвоена буква. Если какое-либо из условий нарушено, попробуйте двигаться в другом направлении. Если все четыре

направления заблокированы, программа должна завершиться. Вот пример преждевременного завершения:



Y блокируется со всех четырёх сторон, поэтому нет места для Z.

10. Измените Проект по программированию 8 из Главы 5 так, чтобы времена отправления сохранялись в массиве, а времена прилёта сохранялись во втором массиве. (Время представляет собой целое число, представляющее число минут с полуночи.) Программа будет использовать цикл для поиска в массиве времени отправления, ближайшего ко времени, введённому пользователем.

11. Измените Проект по программированию из Главы 7 так, чтобы программа маркировала свой вывод:

Введите номер телефона: 1-800-COL-LECT

В числовой форме: 1-800-265-5328

Программе необходимо будет сохранить номер телефона (в оригинальной форме или в его числовой форме) в виде массива символов, пока он не будет напечатан.

Вы можете предположить, что номер телефона не более 15 символов.

12. Измените Проект программирования 5 из Главы 7, чтобы значения SCRABBLE букв были сохранены в массиве.

Массив будет содержать 26 элементов, соответствующих 26 буквам алфавита.

Например, элемент 0 массива будет хранить 1 (потому что значение SCRABBLE для буквы А равно 1), элемент 1 массива будет хранить 3 (потому что значение SCRABBLE в букве В равно 3) и так далее. По мере считывания каждого символа входного слова программа будет использовать массив, чтобы определить значение SCRABBLE этого символа. Используйте инициализатор массива для настройки массива.

13. Измените Проект программирования 11 из Главы 7, чтобы программа маркировала свой вывод:

Введите имя и фамилию: Lloyd Fosdick

Вы ввели имя: Fosdick, L.

Программе необходимо будет сохранить фамилию (но не имя) в массиве символов, пока она не будет напечатана. Вы можете предположить, что фамилия не более 20 символов.

14. Напишите программу, которая изменяет порядок слов предложения на обратный:

Введите предложение: you can cage a swallow can't you?

Обратный порядок слов: you can ' t swallow a cage can you?

Подсказка: используйте цикл, чтобы читать символы один за другим и хранить их в одномерном массиве символов. Пусть цикл останавливается на точку, вопросительный знак или восклицательный знак ("завершающий символ"), который сохраняется в отдельной переменной char.

Затем используйте второй цикл для поиска назад по массиву для начала последнего слова. Распечатайте последнее слово, затем выполните поиск в обратном направлении для следующего за последним словом. Повторяйте до тех пор, пока не будет достигнуто начало массива. Наконец, напечатайте завершающий символ.

15. Одним из старейших известных методов шифрования является шифр Цезаря, приписываемый Юлию Цезарю. Он предполагает замену каждой буквы в сообщении другой буквой, которая расположена на фиксированное число позиций дальше в алфавите. (Если замена будет проходить за буквой Z, шифр "заворачивается" к началу алфавита. Например, если каждая буква будет заменена буквой, расположенной на две позиции после неё, тогда Y будет заменено на A, а Z будет заменена на B.) Напишите

программу, которая шифрует сообщение, используя шифр Цезаря. Пользователь будет вводить сообщение, которое будет зашифровано, и величину сдвига (количество позиций, на которое должны быть сдвинуты буквы):

Введите сообщение, которое нужно зашифровать: Go ahead, make my day.

Введите величину сдвига (1-25): 3

Зашифрованное сообщение: Jr dkhdg, pdnh pb gdb.

Вы можете предположить, что сообщение не превышает 80 символов. Символы, отличные от букв, следует оставлять без изменений. В зашифрованном виде буквы нижнего регистра сохраняются в нижнем регистре, а прописные буквы - в верхнем регистре. Подсказка: чтобы справиться с проблемой заворачивания, используйте выражение

$((ch - 'A') + n) \% 26 + 'A'$,

чтобы вычислить зашифрованную версию прописной буквы, где *ch* хранит букву и *n* — сдвиг. (Вам потребуется аналогичное выражение для строчных букв.)

16. Напишите программу, которая проверяет, являются ли два слова анаграммами (перестановками одних и тех же букв):

Введите первое слово: smartest

Введите второе слово: mattress

Слово является анаграммой.

Введите первое слово: dumbest

Введите второе слово: `stumble`

Слово не является анаграммой.

Напишите цикл, который читает первое слово, посимвольно. Используя массив из 26 целых чисел, чтобы отслеживать, сколько раз встречается каждая буква. (Например, после того, как слово `smartest` было прочитано, массив должен содержать значения `10001000000010000122000000`, отражающие факт, что `smartest` содержит одно `a`, одно `e`, одно `m`, одно `r`, два `s` и два `t`). Используйте ещё один цикл, чтобы прочитать второе слово, но на этот раз используя уменьшение соответствующего элемента массива, при чтении каждой буквы. Оба цикла должны игнорировать любые символы, которые не являются буквами, и оба должны обрабатывать прописные буквы так же, как строчные буквы. После того, как второе слово прочитано, используйте третий цикл, чтобы проверить, все ли элементы в массиве равны нулю. Если это так, слова являются анаграммами. Подсказка: вы можете использовать функции из `<ctype.h>`, такие как `isalpha` и `tolower`.

17. Напишите программу, которая печатает магический квадрат $n \times n$ (квадратное расположение чисел $1, 2, \dots, n^2$, в которых суммы строк, столбцов и диагоналей одинаковы). Пользователь укажет значение n :

Эта программа создаёт магический квадрат заданного размера.

410

Размер должен быть нечётным числом от 1 до 99.

Введите размер магического квадрата: 5

17 24 1 8 15

23 5 7 14 16

4 6 13 20 22

10 12 19 21 3

11 18 25 2 9

Сохраните магический квадрат в двумерном массиве. Начните с размещения числа 1 в середине строки 0. Поместите каждое из оставшихся чисел 2, 3, ..., n^2 , перемещая вверх одну строку и одну колонку. Любая попытка выйти за пределы массива должна "обернуться" на противоположной стороне массива. Например, вместо того, чтобы хранить следующее число в строке -1, мы будем хранить его в строке $n - 1$ (последняя строка). Вместо того, чтобы хранить следующее число в столбце n , мы будем хранить его в столбце 0. Если конкретный элемент массива уже занят, поместите его непосредственно под ранее сохраненным номером. Если ваш компилятор поддерживает массивы переменной длины, объявите массив, состоящий из n строк и n столбцов. Если нет, объявите массив, состоящий из 99 строк и 99 столбцов.

9 Функции

Если в вашей процедуре 10 параметров, вероятно, вы что-то упускаете.

Мы видели в Главе 2, что функция - это просто ряд инструкций, которые были сгруппированы вместе и получили имя. Хотя термин "функция" родом из математики, функции Си не всегда напоминают математические функции.

В Си функция не обязательно имеет аргументы и не обязательно вычисляет значение. (В некоторых языках программирования "функция" возвращает значение, а "процедура" нет. В Си нет такого различия.)

Функции являются строительными блоками программ на Си. Каждая функция по существу представляет собой небольшую программу с собственными декларациями и инструкциями. Используя функции, мы можем разделить программу на

небольшие кусочки, которые для нас и других - проще понимать и модифицировать. Функции берут на себя часть рутинной работы программирования, позволяя избежать дублирования кода, который используется более одного раза.

Кроме того, функции можно использовать повторно: мы можем взять функцию, которая изначально была частью одной программы и использовать её в других программах. На сегодняшний день наши программы состояли только из функции `main`. В этой главе мы увидим, как писать функции, отличные от `main`, и мы узнаем больше о самой `main`. В Разделе 9.1 показано, как определять и вызывать функции. В Разделе 9.2 обсуждаются объявления функций и то, как они отличаются от определения функции. Далее в Разделе 9.3 рассматривается, как аргументы передаются функциям. В оставшейся части главы рассматривается инструкция `return` (Раздел 9.4), связанная с этим проблема завершения программы (Раздел 9.5) и рекурсия (Раздел 9.6).

9.1 Определение и вызов функции

Прежде чем перейти к формальным правилам определения функции, рассмотрим три простые программы, определяющие функции.

ПРОГРАММА Вычисление средних значений

Допустим, нам нужно вычислить среднее двух значений `double`. Библиотека Си не имеет функции "average" ("средних значений"), но мы легко можем определить нашу собственную. Вот как это будет выглядеть:

```
double average(double a, double b){  
    return (a + b) / 2;  
}
```

Слово `double` в начале - это возвращаемый тип `average`: тип данных, который функция возвращает каждый раз, когда она вызывается.

Идентификаторы ***a*** и ***b*** (параметры функции) представляют два числа, которые будут переданы при вызове `average`. Каждый параметр должен иметь тип (как и у каждой переменной есть тип); в этом примере ***a*** и ***b*** имеют тип `double`. (Может показаться странным, но слово `double` должно появляться дважды, один раз для ***a*** и один для ***b***.) Параметр функции по сути является переменной, начальное значение которой будет указано позже, когда функция будет вызвана.

Каждая функция имеет исполняемую часть, называемую телом, которая заключена в фигурные скобки. Тело `average` состоит из единственной инструкции `return`. Выполнение этой инструкции заставляет функцию "вернуться" в то место, откуда она была вызвана, значение $(a + b) / 2$ будет значением, возвращаемым функцией.

Чтобы вызвать функцию, напомним имя функции, за которой следует список аргументов. Например, `average(x, y)` является

вызовом функции `average`. Аргументы используются для предоставления информации функции; в этом случае `average` необходимо знать, какие два числа должны усредняться.

Эффект вызова `average(x, y)` заключается в том, чтобы скопировать значения `x` и `y` в параметры ***a*** и ***b***, а затем выполнить тело `average`. Аргумент не обязательно должен быть переменной; подойдёт любое выражение совместимого типа, что позволит нам написать `average(5.1, 8.9)` или `average(x / 2, y / 3)`. Мы помещаем вызов `average` в то место, где нам нужно использовать возвращаемое значение, например, мы могли бы написать

```
printf("Среднее: %g \n", average(x, y));
```

чтобы вычислить среднее значение `x` и `y`, а затем распечатать его.

Это инструкция выполняет следующее:

1. Функция `average` вызывается с аргументами `x` и `y`.
2. `x` и `y` копируются в ***a*** и ***b***.
3. `average` выполняет свою инструкцию `return`, возвращая среднее ***a*** и ***b***.
4. `printf` печатает значение, которое возвращает `average`.
(Возвращаемое значение `average` становится одним из аргументов `printf`.)

Обратите внимание, что возвращаемое значение `average` нигде не сохраняется: программа печатает его, а затем отбрасывает. Если бы нам понадобилось возвращаемое значение позже в программе. Мы могли бы зафиксировать его в переменной:

```
avg = average(x, y);
```

Эта инструкция вызывает `average`, а затем сохраняет её возвращаемое значение в переменной `avg`. Теперь давайте использовать функцию `average` в полной программе.

Следующая программа считывает три числа и вычисляет их средние значения по одной паре за раз:

Введите три числа: 3.5 9.6 10.2

Среднее значение 3.5 и 9.6: 6.55

Среднее значение 9.6 и 10.2: 9.9

Среднее значение 3.5 и 10.2: 5.85

Помимо всего прочего, эта программа показывает, что функцию можно вызывать сколь угодно раз.

```
//average.c
```

```
/* Вычисляет попарные средние трёх чисел */
```

```
#include <stdio.h>
```

```
double average(double a, double b){  
    return (a + b) / 2;  
}
```

```
int main(void){  
    double x, y, z;  
  
    printf("Введите три числа: ");  
    scanf("%lf%lf%lf", &x, &y, &z);
```

```
printf(" Среднее значение %g и %g: %g\n", x, y,  
average(x, y));  
printf(" Среднее значение %g и %g: %g\n", y, z,  
average(y, z));  
printf(" Среднее значение %g и %g: %g\n", x, z,  
average(x, z));  
  
return 0;  
}
```

Обратите внимание, что я ставлю определение `average` до `main`. Мы увидим в Разделе 9.2, что размещение `average` после `main` создаёт проблему.

ПРОГРАММА Печать обратного отсчёта

Не каждая функция возвращает значение. Например, функция, задача которой - производить вывод может ничего не возвращать. Чтобы указать, что функция не имеет возвращаемого значения, мы указываем, что возвращаемый ею тип является `void`. (`void` - тип без значений.) Рассмотрим следующую функцию, которая выводит сообщение - `T minus n and counting` - где *n* предоставляется при вызове функции:

```
void print_count(int n){  
    printf("T minus %d and counting\n", n);  
}
```

417

`print_count` имеет один параметр, ***n***, типа `int`. Она ничего не возвращает, поэтому я указал тип `void` как возвращаемый тип и опустил инструкцию `return`. Так как `print_count` не возвращает значение, мы не можем вызвать её так же, как мы вызываем `average`. Вместо этого вызов `print_count` должен появляться в инструкции отдельно(сам по себе):

```
print_count (i);
```

Ниже приведена программа, вызывающая `print_count` 10 раз внутри цикла:

```
//countdown.c
```

```
/* Prints a countdown */
```

```
#include <stdio.h>
```

```
void print_count(int n){  
    printf("T minus %d and counting\n", n);  
}
```

```
int main(void){  
    int i;  
  
    for (i = 10; i > 0; --i)  
        print_count(i);  
}
```

```
    return 0;  
}
```

Первоначально значение *i* имеет значение 10. Когда `print_count` вызывается в первый раз, *i* копируется в *n*, так что *n* также принимает значение 10. В результате первый вызов `print_count` напечатает

T minus 10 and counting.

`print_count` затем возвращается к точке, из которой она была вызвана, что является телом инструкции `for`. Инструкция `for` возобновляет свою работу там, где она остановилась, уменьшает *i* до 9 и проверяет, будет ли *i* больше или равно 0. Это так, поэтому `print_count` вызывается снова, на этот раз печатает

T minus 9 and counting

Каждый раз, когда вызывается `print_count`, *i* изменяется, поэтому `print_count` будет печатать 10 различных сообщений.

ПРОГРАММА Печать каламбура (повторение)

Некоторые функции вообще не имеют параметров.

Рассмотрим `print_run`, которая печатает дурацкий каламбур каждый раз, когда она вызывается:

419

```
void print_pun (void) {  
    printf("To C, or not to C: that is the question.\n");  
}
```

Слово `void` в круглых скобках указывает, что `print_pun` не имеет аргументов (на этот раз мы используем `void` в качестве заполнителя, что означает "здесь ничего не требуется").

Чтобы вызвать функцию без аргументов, мы пишем имя функции, за которым следуют скобки :

```
print_pun ();
```

Скобки должны присутствовать, даже если аргументов нет.

Вот крошечная программа, которая проверяет функцию `print_pun`:

```
//pun2.c
```

```
/* Prints a bad pun */
```

```
#include <stdio.h>
```

```
void print_pun(void){  
    printf("To C, or not to C: that is the question.\n");  
}
```

```
int main(void){
```

```
    print_pun();  
    return 0;  
}
```

Выполнение этой программы начинается с первой инструкции в `main`, которая является вызовом `print_pun`. Когда `print_pun` начинает выполняться, она, в свою очередь, вызывает `printf` для отображения строки. Когда `printf` вернётся, `print_pun` вернётся к `main`.

Определения функций

Теперь, когда мы видели несколько примеров, давайте рассмотрим общую форму определения функции:

```
возвращаемый тип имя_функции (параметры){  
    объявления  
    инструкции  
}
```

Возвращаемый тип функции - это тип значения, возвращаемого функцией. Тип возвращаемого значения определяется следующими правилами:

■ Функции не могут возвращать массивы, но нет других ограничений для возвращаемого типа.

■ Спецификация возвращаемого типа `void`, указывает, что функция не возвращает значение.

Если возвращаемый тип пропускается в C89, предполагается, что функция возвращает значение типа `int`. В C99 запрещено пропускать возвращаемый тип функции.

Некоторые программисты, используя своеобразный стиль, помещают возвращаемый тип выше имени функции:

```
double
average( double a, double b){
    return (a + b) / 2;
}
```

Помещение возвращаемого типа в отдельной строке особенно полезно, если тип возвращаемого значения длинный, например, `unsigned long int`. После имени функции выводится список параметров. Каждому параметру предшествует спецификация его типа; параметры разделяются запятыми. Если у функции нет параметров, в круглых скобках должно появиться слово `void`.

Примечание. Для каждого параметра необходимо указывать отдельный тип, даже если несколько параметров имеют один и тот же тип:

```
double average (double a, b) {    /*** WRONG ***/
    return (a + b) / 2;
}
```

Тело функции может включать как объявления, так и инструкции. Например, функцию `average` можно записать так:

```
double average(double a, double b){  
    double sum; /* объявление */  
    sum = a + b; /* инструкция */  
    return sum / 2; /* инструкция */  
}
```

Переменные, объявленные в теле функции, принадлежат исключительно этой функции; они не могут быть исследованы или изменены другими функциями.

В C89 объявления переменных должны наступать раньше всех инструкций в теле функции. В C99 объявления переменных и инструкции могут быть смешанными, если каждая переменная будет объявлена перед первой инструкцией, которая использует эту переменную.

(Некоторые компиляторы pre-C99 также позволяют смешивать объявления и инструкции.) Тело функции, возвращаемый тип которой является `void` (которую я буду называть "void function"-"пустая функция"), может быть пустым:

```
void print_pun(void){  
}
```

Оставлять тело пустым может иметь смысл во время разработки программы; мы можем оставить место для функции, не затрачивая времени на её написание, а затем вернёмся позже и напишем тело.

Вызовы функций

Вызов функции состоит из имени функции, за которым следует список аргументов, заключенный в круглые скобки:

`average (x, y)`

`print_count (i)`

`print_pun ()`

Если скобки отсутствуют, функция не будет вызвана:

`print_pun; /*** WRONG ***/`

Результатом является допустимая (хотя и бессмысленная) инструкция-выражение, которое выглядит правильно, но не имеет никакого эффекта.

Некоторые компиляторы выдают предупреждение: такие, как "инструкция без эффекта".

За вызовом пустой (void) функции всегда следует точка с запятой, чтобы превратить его (вызов) в инструкцию:

`print_count (i);`

`print_pun();`

Вызов не пустой функции, с другой стороны, даёт значение которое может быть сохранено в переменной, протестировано, распечатано или использовано каким-либо другим способом:

```
avg = average(x, y) ;  
if (average(x, y) > 0)  
    printf("Среднее положительно\n");  
printf("Среднее равно %g\n", average(x, y));
```

Значение, возвращаемое не пустой функцией всегда может быть отброшено, если оно не требуется:

```
average (x, y); /* отбрасывает возвращаемое значение */
```

Этот вызов `average` является примером инструкции-выражения:

инструкция вычисляет значение выражения, но затем сбрасывает результат.

Игнорировать возвращаемое значение `average` - довольно странная вещь, но для некоторых функций, это имеет смысл.

Функция `printf`, например, возвращает количество символов, которое она печатает. После следующего вызова `num_chars` будет иметь значение 9:

```
num_chars = printf("Hi, Mom!\n");
```

Поскольку мы, вероятно, не заинтересованы в числе напечатанных символов, мы обычно отбрасываем возвращаемое значение printf :

```
printf("Hi, Mom!\n"); /* discards return value */
```

Для того, чтобы прояснить, что мы намеренно отбрасываем возвращаемое значение функции, Си позволяет нам ставить (void) перед вызовом:

```
(void) printf("Hi, Mom!\n");
```

То, что мы делаем, является операцией приведения типа (преобразование) возвращаемого значения printf к типу void.

(В Си, "приведение к типу void" является вежливым способом сказать "отбросить"). Использование (void) делает ясным, что вы сознательно отказались от возвращаемого значения, а не просто забыли, что оно существует.

К сожалению, есть очень много функций в библиотеке Си, значения которых обычно игнорируются; использование (void) при вызове их всех может стать утомительным, поэтому, я не делаю так в этой книге.

ПРОГРАММА Проверка: является ли число простым

Чтобы увидеть, как функции могут сделать программу проще для понимания, давайте напишем программу, проверяющую является ли число простым.

Программа предложит пользователю ввести число, а затем ответит сообщением о том, является ли число простым:

Введите число: 34

Составное

Вместо того, чтобы поместить детали проверки чисел на простоту в `main`, мы определим отдельную функцию которая возвращает истину, если её параметр простое число, и ложь, если это не так. Когда дано число *n*, функция `is_prime` будет делить *n* на каждое из чисел от 2 до квадратного корня из *n*; если остаток равен 0, то *n* не является простым числом.

```
//prime.c
```

```
/* Проверка является ли число простым */
```

```
#include <stdbool.h> /* только для C99 */
```

```
#include <stdio.h>
```

```
bool is_prime(int n){
```

```
    int divisor;
```

```
    if (n <= 1)
```

```
        return false;
```

```
    for (divisor = 2; divisor * divisor <= n; divisor++)
```

```
        if (n % divisor == 0)
```

427

```
        return false;
    return true;
}

int main(void){
    int n;

    printf("Введите число: ");
    scanf("%d", &n);
    if (is_prime(n))
        printf("Простое\n");
    else
        printf("Составное\n");

    return 0;
}
```

Заметьте, что `main` содержит переменную, с именем ***n*** даже при том, что параметр `is_prime` также назван ***n***. В общем случае, функция может объявить переменную с таким же именем переменной, что и в другой функции. Эти две переменные представляют собой разные места в памяти, так

что присваивание нового значения одной переменной не меняет другую. (Это свойство также распространяется на параметры.) В Разделе 10.1 этот вопрос обсуждается более подробно. Как демонстрирует `is_prime`, функция может иметь более одной инструкции `return`. Тем не менее, мы можем выполнить только одну из этих инструкций во время данного вызова функции, потому что выполнение инструкции `return` заставляет функцию возвращаться туда, откуда она была вызвана. Подробнее об инструкции `return` в Разделе 9.4.

9.2. Объявление функций

В программах раздела 9.1 определение каждой функции всегда располагалось над местом, из которого она вызывалась.

На самом деле, Си не требует, чтобы определение функции предшествовало её вызовам.

Предположим, что мы переставили программу `average.c`, поставив определение `average` после определения `main`:

```
#include <stdio.h>
int main(void){
    double x, y, z;
    printf("Enter three numbers: ");
    scanf("%lf%lf%lf", &x, &y, &z);
```



```
printf("Average of %g and %g: %g\n", x, y, average(x, y));  
printf("Average of %g and %g: %g\n", y, z, average(y, z));  
printf("Average of %g and %g: %g\n", x, z, average(x, z));  
  
return 0;  
}
```

```
double average(double a, double b){  
    return (a + b) / 2;  
}
```

Когда компилятор встречается первый вызов `average` в `main`, он не имеет информации о `average`: он не знает, сколько параметров имеет `average`, какие типы этих параметров, или какое значение возвращает `average`. Вместо того, чтобы выдавать сообщение об ошибке, компилятор предполагает, что `average` возвращает значение `int` (вспомните из раздела 9.1, что возвращаемый тип функции - `int` по умолчанию).

Мы говорим, что компилятор создал неявное объявление функции. Компилятор не может проверить, что мы передаём правильное количество аргументов и что аргументы имеют правильный тип. Вместо этого он выполняет продвижение аргументов по умолчанию и надеется на лучшее. Когда позже в программе он встречается определение `average`, компилятор замечает, что возвращаемый тип функции действительно `double`, а не `int`, и мы получаем сообщение об ошибке. Один из

способов избежать проблемы вызова перед определением состоит в том, чтобы расположить программу таким образом, чтобы определение каждой функции предшествовало всем её вызовам. К сожалению, такая схема не всегда существует, и даже если она есть, это может сделать программу более сложной для понимания, поставив определения её функций в неестественном порядке. К счастью, Си предлагает лучшее решение: объявить каждую функцию перед её вызовом. Объявление функции предоставляет компилятору краткий обзор функции, чьё полное определение появится позже. Объявление функции похоже на первую строку определения функции с добавленной точкой с запятой:

return type function-name (parameters);

возвращаемый-тип имя-функции (параметры);

Излишне говорить, что объявление функции должно быть согласовано с определением функции.

Вот как будет выглядеть наша программа с добавлением объявления ***average***:

```
#include <stdio.h>
```

```
double average(double a, double b); /* ОБЪЯВЛЕНИЕ */
```

```
int main(void){
```

```
    double x, y, z;
```

```
printf("Enter three numbers: ");
scanf("%lf%lf%lf", &x, &y, &z);
printf("Average of %g and %g: %g\n", x, y, average(x, y));
printf("Average of %g and %g: %g\n", y, z, average(y, z));
printf("Average of %g and %g: %g\n", x, z, average(x, z));

return 0;
}

double average(double a, double b) { /* ОПРЕДЕЛЕНИЕ */

    return (a + b) / 2;

}
```

Объявления функций, которые мы обсуждали, известны как прототипы функций, чтобы отличить их от старого стиля объявления функций, в котором скобки остаются пустыми. Прототип предоставляет полное описание того, как вызывать функцию: сколько нужно аргументов, какие должны быть их типы и какой тип результата будет возвращён. Кстати, прототипу функции не нужно указывать имена параметров функции, при условии что есть их типы:

`double average(double, double);`

Как правило, лучше не опускать имена параметров, так как они помогают документировать назначение каждого параметра и напоминают программисту порядок, в котором аргументы должны появляться при вызове функции.

Однако существуют законные причины для пропуска имён параметров, и некоторые программисты предпочитают это делать.

В C99 было принято правило, что либо объявление, либо определение функции должно присутствовать до любого вызова функции.

Вызов функции, для которой компилятор ещё не видел объявления или определения, является ошибкой.

9.3 Аргументы

Рассмотрим разницу между параметром и аргументом.

Параметры появляются в определениях функций; они являются фиктивными именами, представляющими значения, которые должны быть предоставлены при вызове функции. Аргументы - это выражения, которые появляются в вызовах функций.

Когда различие между аргументом и параметром не важно, я иногда буду использовать термин аргумент для обозначения того и другого.

В Си аргументы передаются по значению: при вызове функции каждый аргумент вычисляется и его значение присваивается соответствующему параметру. Поскольку параметр содержит копию значения аргумента, любые изменения, внесённые в параметр во время выполнения функции, не влияют на аргумент.

Фактически каждый параметр ведёт себя как переменная, которая была инициализирована значением соответствующего аргумента. Тот факт, что аргументы передаются по значению, имеет как преимущества, так и недостатки. Поскольку параметр можно модифицировать, не затрагивая соответствующий аргумент, мы можем использовать параметры как переменные внутри функции, тем самым уменьшая количество необходимых реальных переменных. Рассмотрим следующую функцию, которая возводит число x в степень n :

```
int power (int x, int n){  
    int i, result = 1;  
    for ( i= 1; i <= n; i++ )  
        result = result * x;  
    return result;  
}
```

Поскольку *n* является копией исходного показателя, мы можем изменять её внутри функции, устраняя необходимость *i*:

```
int power (int x, int n){  
    int result = 1;  
    while (n-- > 0)  
        result = result * x;  
    return result;  
}
```

К сожалению, требование Си о том, чтобы аргументы передавались по значению, затрудняет запись некоторых видов функций.

Например, предположим, что нам нужна функция, которая будет разлагать значение `double` на целую часть и дробную часть. Поскольку функция не может вернуть два числа, мы можем попытаться передать пару переменных в функцию и изменить их:

```
void decompose (double x, long int_part, double frac_part){  
    int_part = (long) x; /* отбрасывает дробную часть x */  
    frac_part = x - int_part;  
}
```

Предположим, что функцию мы вызываем следующим образом:

```
decompose(3.14155, i, d);
```

В начале вызова, 3.14159 копируется в x , значение i копируется в `int_part`, а значение d копируется во `frac_part`. Инструкции внутри `decompose` затем присваивают 3 `int_part` и 14159 `frac_part` и функция возвращается. К сожалению, i и d не были затронуты присваиваниями `int_part` и `frac_part`, поэтому они имеют те же значения после вызова, что и до вызова. С небольшими дополнительными усилиями `decompose` можно заставить работать, как мы увидим в Разделе 11.4. Однако сначала нам нужно будет познакомиться с большим числом функций Си.

Преобразования аргументов

Си допускает вызовы функций, в которых типы аргументов не соответствуют типам параметров. Правила, регулирующие преобразование аргументов, зависят от того, видел ли компилятор прототип функции (или полное определение функции) перед вызовом или нет:

- Компилятор столкнулся с прототипом до вызова.

Значение каждого аргумента неявно преобразуется в тип соответствующего параметра как при присваивании.

Например, если аргумент `int` передан функции, ожидавшей `double`, аргумент преобразуется в `double` автоматически.

- Компилятор не сталкивался с прототипом до вызова.

Компилятор выполняет продвижение аргументов по

умолчанию:

- (1) аргументы `float` конвертируются в `double`.
 - (2) Выполняются продвижение целых типов, принуждая аргументы `char` и `short` конвертироваться в `int`. (В C99 проводится продвижение целых типов.)
-

Опираясь на продвижение типов аргументов по умолчанию, опасно. Рассмотрим следующую программу:

```
#include <stdio.h>

int main(void){
    double x = 3.0;
    printf("Square: %d\n", square (x) );
    return 0;
}

int square (int n) {
    return n * n;
}
```

В то время, когда вызывается `square`, компилятор ещё не видел прототипа, поэтому он не знает, что `square` ожидает аргумент типа `int`. Вместо этого компилятор выполняет продвижение типов аргументов по умолчанию для `x`, но без результата. Поскольку он ожидает аргумент типа `int`, но вместо этого ему даётся значение `double`, результат вызова `square` не определён.

Проблема может быть решена путём приведения аргумента square к соответствующему типу:

```
printf("Square: %d\n", square( (int) x ) );
```

Конечно, гораздо лучшим решением является предоставление прототипа square перед его вызовом. В C99 вызов square без предварительного объявления или определения функции является ошибкой.

Массивы как аргументы

Массивы часто используются в качестве аргументов. Если параметром функции является одномерный массив, длина массива может быть (и обычно) не указана.

(Г.Шилдт "Полный справочник по Си - Передача одномерного массива в функцию:

В языке Си нельзя передать весь массив как аргумент функции. Однако можно передать указатель на массив, т.е. имя массива без индекса. Например, в представленной программе в func1() передаётся указатель на массив i:

```
int main(void){  
    int i[10];
```

```
func1(i);
```

```
/*... */
```

```
}
```

Если в функцию передаётся указатель на одномерный массив, то в самой функции его можно объявить одним из трёх вариантов: как указатель, как массив определённого размера и как массив без определённого размера. Например, чтобы функция func1() получила доступ к значениям, хранящимся в массиве i, она может быть объявлена как

```
void func1(int *x) {/* указатель */
```

```
/*... */
```

```
}
```

или как

```
void func1(int x[10]){ /* массив определённого размера */
```

```
/*... */
```

```
}
```

и наконец как

```
void func1(int x[]) { /* массив без определённого размера */  
    /*... */  
}
```

Эти три объявления тождественны, потому что каждое из них сообщает компилятору одно и то же: в функцию будет передан указатель на переменную целого типа. В первом объявлении используется указатель, во втором — стандартное объявление массива. В последнем примере изменённая форма объявления массива сообщает компилятору, что в функцию будет передан массив неопределённой длины. Как видно, длина массива не имеет для функции никакого значения, потому что в Си проверка границ массива не выполняется. Эту функцию можно объявить даже так:

```
void func1(int x[32]){  
    /*... */  
}
```

И при этом программа будет выполнена правильно, потому что компилятор не создаёт массив из 32 элементов, а только подготавливает функцию к приёму указателя.)

Аргументом может быть любой одномерный массив, элементы которого имеют правильный тип. Есть только одна проблема: как узнать, какова длина массива?

К сожалению Си не предоставляет какого-либо простого способа для функции определить длину массива, передаваемого в неё. Вместо этого нам нужно указать длину - если функция нуждается в ней - в качестве дополнительного аргумента.

Хотя мы можем использовать оператор `sizeof` для определения длины переменной - массива, он не даёт правильного ответа для параметра-массива:

```
int f(int a[]){  
    int len = sizeof(a) / sizeof(a[0]);  
  
    /**WRONG: sizeof(a)- даёт размер указателя, нет  
информации о числе элементов в a ***/  
  
    ....  
}
```

Раздел 12.3 объясняет, почему.

Следующая функция иллюстрирует использование одномерного массива как аргумента. Когда задан массив **a** значений `int`, `sum_array` возвращает сумму элементов в **a**. Так как `sum_array` нужно знать длину **a**, мы должны предоставить её в качестве второго аргумента.

441

```
int sum_array (int a[], int n){  
    int i, sum = 0;  
    for (i = 0; i < n; i++)  
        sum += a[i];  
    return sum;  
}
```

Прототип для sum_array имеет следующий вид:

```
int sum_array ( int a[], int n );
```

Как обычно, мы можем опустить имена параметров, если мы хотим:

```
int sum_array(int [], int) ;
```

Когда вызывается sum_array, первым аргументом будет имя массива, а вторым будет его длина.

Например:

```
#define LEN 100
```

```
int main(void){  
    int b[LEN], total;  
    ...  
    total = sum_array (b, LEN);  
    ...  
}
```

Заметьте, что мы не помещаем скобки после имени массива при передаче его функции:

```
total = sum_array(b[], LEN); /*** WRONG ***/
```

Важный момент по поводу аргументов-массивов. Функция не имеет возможности проверить, что мы передали правильную длину массива. Мы можем использовать этот факт, указав функции, что массив меньше, чем есть на самом деле.

Предположим, что мы сохранили только 50 чисел в массиве `b`, хотя он может содержать и 100. Мы можем суммировать только первые 50 элементов, написав

```
total = sum_array(b, 50); /* sums first 50 elements */
```

`sum_array` будет игнорировать остальные 50 элементов. (Действительно, она не будет знать даже, что они существуют!)

Будьте осторожны, чтобы не сказать функции, что аргумент-массив больше, чем он есть на самом деле:

```
total = sum_array(b, 150); /*** WRONG ***/
```

В этом примере `sum_array` пройдёт мимо конца массива, что приведёт к неопределённому поведению.

ещё одна важная вещь, которую нужно знать, - это то, что функция позволяет изменять элементы параметра-массива, и это изменение отражается в соответствующем аргументе.

443

Например, следующая функция изменяет массив, сохраняя ноль в каждом из его элементов:

```
void store_zeros (int a[],int n){  
    int i;  
    for (i = 0; i < n; i++)  
        a[i] = 0;  
}
```

Вызов

```
store_zeros (b, 100);
```

будет хранить ноль в первых 100 элементах массива **b**. Эта способность изменять элементы аргумента-массива может казаться противоречащей тому факту, что Си передает аргументы по значению. На самом деле, нет никаких противоречий, но я не смогу объяснить почему, до раздела 12.3. Если параметр является многомерным массивом, при объявлении параметра может быть опущена только длина первого измерения. Например, если мы пересматриваем функцию `sum_array` так, что **a** является двумерным массивом, мы должны указать количество столбцов в **a**, хотя нам не нужно указывать количество строк:

```
#define COL 10
```

```
int sum_two_dimensional_array(int a[][COL], int n){  
    int i, j, sum = 0;  
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < COL; j++)  
            sum += a[i][j];  
  
    return sum;  
  
}
```

Неспособность передавать многомерные массивы с произвольным числом столбцов может быть неприятной. К счастью, мы часто можем обойти эту трудность, используя массивы указателей.

Параметры массива переменной длины C99 обеспечивают ещё лучшее решение проблемы.

Параметры - массивы переменной длины

C99 добавляет несколько новых поворотов в аргументы-массивы. Первое относится к массивам переменной длины (VLAs), это является особенностью C99, что позволяет указывать длину массива с помощью выражения, не являющегося константой. Как выясняется, массивы переменной длины также могут быть параметрами.

Рассмотрим функцию `sum_array`, рассмотренную ранее в этом разделе. Вот определение `sum_array`, где тело опущено:

```
int sum_array(int a[], int n){  
  
    ...  
  
}
```


На данный момент, прямой связи между ***n*** и длиной массива ***a*** нет. Хотя тело функции рассматривает ***n*** как длину, действительная длина массива может на самом деле быть больше ***n*** (или меньше и, в этом случае, функция не будет работать правильно). Используя параметр-массив переменной длины, мы можем явно указать, что длина ***a*** равна ***n***:

```
int sum_array (int n, int a[n] ){
    ...
}
```

Значение первого параметра (***n***) определяет длину второго параметра (***a***). Обратите внимание, что порядок параметров был изменён; порядок важен при использовании параметров-массивов переменной длины. Следующая версия `sum_array` является недопустимой:

```
int sum_array(int a[n], int n){ /*** WRONG ***/
    ...
}
```

Компилятор выдаст сообщение об ошибке в `int a[n]`, потому что он ещё не видел ***n***.

Есть несколько способов написать прототип для нашей новой версии `sum_array`. Одна из возможностей заключается в том, чтобы сделать его похожим на определение функции:

```
int sum_array(int n, int a[n] ); /* Version 1 */
```

Другая возможность заключается в замене длины массива звёздочкой (*):

```
int sum_array(int n, int a[*] ); /* Version 2a */
```

Причина использования нотации * заключается в том, что имена параметров являются необязательными в объявлениях функций.

Если имя первого параметра опущено, невозможно указать, что длина массива равна *n*, но * предоставляет ключ, указывающий, что длина массива связана с параметрами, которые ранее были в списке:

```
int sum_array(int, int [*]; /* Version 2b */
```

Также допустимо оставлять скобки пустыми, как это обычно делается при объявлении параметра массива:

```
int sum_array(int n, int a[] ); /* Version 3a */
```

```
int sum_array(int, int [] ); /* Version 3b: */
```

Если оставить скобки пустыми, это не лучший выбор, поскольку он не раскрывает отношения между *n* и *a*.

В общем случае длина параметра-массива переменной длины может быть любым выражением. Например, предположим, что мы должны написать функцию, которая объединяет два массива *a* и *b* путём копирования элементов *a*, а затем элементов *b*, в третий массив *c*:

```
int concatenate( int m, int n, int a[m], int b[n], int c[m+n]){
    ...
}
```

Длина **c** является суммой длин **a** и **b**. Выражение, используемое для указания длины **c**, включает в себя два других параметра, но в общем оно может ссылаться на переменные вне функции или даже вызывать другие функции.

Параметры-массивы переменной длины с одним измерением - как и во всех наших примерах до сих пор - имеют ограниченную полезность. Они делают объявление или определение функции более наглядным, указывая желаемую длину аргумента массива. Однако дополнительная проверка ошибок не выполняется; все ещё возможно, чтобы аргумент-массив был слишком длинным или слишком коротким.

Оказывается, параметры-массивы переменной длины наиболее полезны для многомерных массивов. Ранее в этом разделе мы попытались написать функцию, суммирующую элементы в двумерном массиве. Наша исходная функция ограничивалась массивами с фиксированным числом столбцов. Если мы используем параметр-массив переменной длины, мы можем обобщить функцию на любое количество столбцов:

```
int sum_two_dimensional_array( int n, int m, int a[n] [m]){  
    int i, j, sum = 0;  
    for (i = 0; i < n; i++)  
        for (j = 0; j < m; j++)  
            sum += a[i] [j];  
    return sum;  
}
```

Прототипы для этой функции включают следующее:

```
int sum_two_dimensional_array(int n, int m, int a[n][m]);
```

```
int sum_two_dimensional_array(int n, int m, int a [*] [*]);
```

```
int sum_two_dimensional_array(int n, int m, int a[] [m]);
```

```
int sum_two_dimensional_array(int n, int m, int a[] [*]);
```

Использование static в объявлениях параметров-массивов

C99 позволяет использовать ключевое слово `static` в объявлении параметров-массивов (само ключевое слово существовало до C99. В Разделе 18.2 обсуждаются его традиционные применения.)

В следующем примере ввод `static 3` перед числом 3 указывает, что длина объекта ***a*** гарантируется как минимум 3:

```
int sum_array(int a[static 3], int n){
```

```
    ...
```

```
}
```

Использование `static` в этом случае не влияет на поведение программы. Наличие `static` - всего лишь "подсказка", которая может позволить компилятору Си генерировать быстрые инструкции для обращения к массиву.

(Если компилятор знает, что у массива всегда будет

определённая минимальная длина, он может организовать "предварительную выборку" этих элементов из памяти при вызове функции, прежде чем элементы действительно понадобятся инструкциям внутри функции.)

Последнее замечание о `static`: если параметр-массив имеет более одного измерения, `static` может использоваться только в первом измерении (например, при указании числа строк в двумерном массиве).

Сложные (составные) литералы

Вернёмся к исходной функции `sum_array` в последний раз. Когда вызывается `sum_array`, первым аргументом обычно является имя массива (того, чьи элементы должны быть просуммированы). Например, мы могли бы вызвать `sum_array` следующим образом:

```
int b[]={3,0,3,4,1};  
total = sum_array(b, 5);
```

Единственная проблема с такой компоновкой заключается в том, что ***b*** должен быть объявлен как переменная и затем инициализирован до вызова. Если ***b*** не требуется для каких-либо других целей, то создание его исключительно с целью вызова `sum_array`, может быть слегка раздражающим.

В C99 мы можем избежать этой досады, используя составной литерал: безымянный массив, который создаётся "на лету", просто указывая, какие элементы он содержит. Следующий вызов `sum_array` имеет составной литерал (выделен жирным шрифтом) в качестве первого аргумента:

```
total = sum_array(( int [ ]){ 3, 0, 3, 4, 1 },5);
```

В этом примере составной литерал создаёт массив, содержащий пять целых чисел 3, 0, 3, 4 и 1. Мы не указали длину массива, поэтому он определяется числом элементов в литерале. У нас также есть возможность явно указать длину:

```
(int [4] ) {1, 9, 2, 1}
```

эквивалентно

```
(int []){1, 9, 2, 1}.
```

В общем случае составной литерал состоит из имени типа с круглыми скобками, за которым следует набор значений, заключённых в фигурные скобки.

Составной литерал похож на приведение, применённое к инициализатору. На самом деле составные литералы и инициализаторы подчиняются тем же правилам. Составной литерал может содержать индексы, как и назначенный инициализатор, и он может не обеспечить полную инициализацию (в этом случае любые неинициализированные элементы по умолчанию равны нулю). Например, литерал `(int [10]) {8, 6}` имеет 10 элементов; первые два имеют значения 8 и 6, а остальные элементы имеют значение 0. Сложные литералы, созданные внутри функции, могут содержать произвольные выражения, а не только константы. Например, мы могли бы написать

```
total = sum_array((int []){2 * i, i + j, j * k},3);
```

где *i*, *j* и *k* - переменные.

Этот аспект составных литералов значительно повышает их полезность. Составной литерал является *lvalue*, поэтому значения его элементов могут быть изменены. Если необходимо, составной литерал можно сделать "только для чтения", добавив слово *const* к его типу, как в `(const int []) {5, 4}`.

Инструкция `return`

Функция `non-void` должна использовать инструкцию `return`, чтобы указать, какое значение оно вернёт. Инструкции `return` имеет форму

return выражение ;

Выражение часто является константой или переменной:

```
return 0;
```

```
return status;
```

Возможны более сложные выражения. Например, не редкость видеть условный оператор, используемый в выражении возврата:

```
return n >= 0 ? n : 0 ;
```

Когда эта инструкция выполняется, выражение $n \geq 0 ? n : 0$ вычисляется первым. Инструкция возвращает значение n , если оно не отрицательное; в противном случае оно возвращает 0. Если тип выражения в инструкции `return` не соответствует возвращаемому типу функции, выражение будет неявно преобразовано в тип возвращаемого значения. Например, если функция объявлена как возвращающая `int`, но инструкция `return` содержит выражение `double`, значение выражения преобразуется в `int`.

Инструкции `return` могут появляться в функциях, у которых тип возвращаемого значения `void`, при условии, что выражение не задано:

```
return; /* Возврат void функции */
```

Помещение выражения в такую инструкцию `return` даст вам ошибку времени компиляции. В следующем примере инструкция `return` возвращает функцию сразу же после получения отрицательного аргумента:

```
void print_int (int i){
    if (i < 0)
        return;
    printf("%d",i);
}
```

Если i меньше 0, `print_int` выполнит возврат без вызова `printf`. Инструкция `return` может появиться в конце функции `void`:

```
void print_pun (void){
    printf("To C, or not to C: that is the question.\n");
}
```



```
    return; /* OK, but not needed */  
}
```

Однако использование `return` не нужно, так как функция выполнит возврат автоматически после того, как будет выполнена её последняя инструкция.

Если не пустая (`non-void`) функция достигает конца своего тела и не выполняет инструкцию `return`, поведение программы не определено, если она пытается использовать значение, возвращаемое функцией. Некоторые компиляторы выдают предупреждение, такое как "управление достигает конца `non-void` функции", если они обнаруживают возможность того, что непустая функция "отвалится" в конце её тела.

Завершение программы

Поскольку `main` является функцией, она должна иметь тип возвращаемого значения. Как правило, возвращаемый тип `main` - `int`, поэтому программы, которые мы видели до сих пор, определили `main` следующим образом:

```
int main(void){  
    ...  
}
```

Старые программы Си часто опускали тип возвращаемого значения `main`, воспользовавшись тем фактом, что он традиционно по умолчанию `int`:

```
main(){  
  
    ...  
  
}
```

Опускание возвращаемого типа функции не является допустимым в C99, поэтому лучше избегать этой практики. Опускание слова `void` в списке параметров `main` остаётся допустимым, но — в рамках стиля - лучше всего чётко указать на то, что `main` не имеет параметров. (Мы увидим позже, что у `main` иногда есть два параметра, обычно называемых `argc` и `argv`.)

Значение, возвращаемое `main`, является кодом состояния, который в некоторых операционных системах может быть проверен, когда программа завершается. `main` должна возвращать 0, если программа завершается нормально; для указания ненормального завершения, `main` должна вернуть значение, отличное от 0.

(На самом деле, нет правила, которое запрещало бы нам использовать возвращаемое значение для других целей.)

Хорошая практика состоит в том, чтобы удостовериться, что каждая программа на языке Си возвращает код состояния, даже если нет планов использовать его, поскольку кто-то, запустивший программу позже, может решить проверить её.

Функции `exit`

Выполнение инструкции `return` в `main` является одним из способов завершения программы. Другим является вызов функции `exit`, которая принадлежит `<stdlib.h>`. Аргумент, переданный в `exit`, имеет то же значение, что и возвращаемое значение `main`: оба указывают статус программы при завершении. Чтобы указать нормальное завершение, мы должны передать 0:

```
exit(0); /* normal termination */
```

Поскольку 0 является немного непонятным, Си позволяет нам вместо этого передавать `EXIT_SUCCESS` (эффект тот же):

```
exit (EXIT_SUCCESS); /* normal termination */
```

Передача `EXIT_FAILURE` указывает на аномальное завершение:

```
exit(EXIT_FAILURE); /* abnormal termination */
```

`EXIT_SUCCESS` и `EXIT_FAILURE` являются макросами, определёнными в `<stdlib.h>`. Значения `EXIT_SUCCESS` и `EXIT_FAILURE` определяются реализацией; типичными значениями являются 0 и 1 соответственно. В качестве методов завершения программы `return` и `exit` тесно связаны.

Фактически, инструкция

`return` выражение;

в `main` эквивалентно

`exit (выражение);`

Разница между `return` и `exit` заключается в том, что `exit` вызывает завершение программы независимо от того, какая функция вызывает её.

Инструкции `return` вызывает завершение программы только тогда, когда он появляется в функции `main`. Некоторые программисты используют `exit` исключительно для облегчения размещения всех точек выхода в программе.

9.6. Рекурсия

Функция рекурсивна, если она вызывает сама себя. Например, следующая функция рекурсивно вычисляет $n!$, используя формулу $n! = n \times (n - 1)!$

```
int fact (int n){  
    if (n <= 1)  
        return 1;  
    else  
        return n * fact (n -1);  
}
```

Некоторые языки программирования в значительной степени зависят от рекурсии, в то время как другие даже не допускают её. Си находится где-то посередине: он разрешает рекурсию, но большинство Си-программистов не используют её так часто. Чтобы увидеть, как работает рекурсия, проследим

457

выполнение инструкции

`i = fact(3);`

Вот что происходит:

`fact (3)` находит, что 3 не меньше или равно 1, поэтому она вызывает

`fact (2)`, которая обнаруживает, что 2 не меньше или равно 1, поэтому она вызывает

`fact (1)`, которая обнаруживает, что 1 меньше или равно 1, поэтому она возвращает 1 и вызывает

`fact (2)`, чтобы вернуть $2 \times 1 = 2$

и вызывает

`fact (3)`, чтобы вернуть $3 \times 2 = 6$.

Обратите внимание на то, как неполные вызовы `fact` "накапливаются" до тех пор, пока `fact` не передаётся окончательно 1. В этой точке старые вызовы `fact` начинают "разматываться" по очереди, пока исходный вызов - `fact (3)` не возвращает ответ 6.

Вот ещё один пример рекурсии: функция, которая вычисляет y^n , используя формулу $y^n = y \times y^{n-1}$.

```
int power( int x, int n ){  
    if (n == 0)  
        return 1;  
    else  
        return x * power (x, n-1);  
}
```

}

Вызов `power (5, 3)` будет выполнен следующим образом:
`power (5, 3)` обнаруживает, что 3 не равно 0, поэтому она вызывает

`power (5, 2)`, которая обнаруживает, что 2 не равно 0,
поэтому она вызывает

`power (5, 1)`, которая обнаруживает что 1 не равно 0,
поэтому она вызывает

`power (5, 0)`, которая находит 0 равным 0,
поэтому она возвращает 1, вызывая

`power (5, 1)` которая возвращает $5 \times 1 = 5$, вызывая

`power (5, 2)`, чтобы вернуть $5 \times 5 = 25$, вызывая

`power (5, 3)`, чтобы вернуть $5 \times 25 = 125$.

Кстати, мы можем немного сжать степенную функцию,
помещая условное выражение в инструкции `return`:

```
int power(int x, int n){
```

```
    return n == 0 ? 1 : x * power (x, n - 1);
```

```
}
```

Алгоритм быстрой сортировки

(Рекурсию целесообразно применять в задачах, которые можно разбить на множество меньших подобных задач.)

На практике рекурсия часто возникает естественным образом в результате методики проектирования алгоритмов, известной как "разделяй и властвуй", в которой большая проблема делится на более мелкие части, которые затем решаются одним и тем же алгоритмом.

Классический пример стратегии "разделяй и властвуй" можно найти в популярном алгоритме сортировки, известном как быстрая сортировка (Quicksort).

Алгоритм быстрой сортировки выглядит следующим образом (для простоты предположим, что сортируемый массив индексируется от 1 до n):

1. Выберите элемент массива e ("элемент разбиения"), затем перестройте массив так, чтобы элементы 1, ..., $i-1$ были меньше или равны e , элемент i содержит e и элементы $i + 1$... n больше или равны e .
2. Рекурсивная сортировка элементов 1,..., $i - 1$ с помощью Quicksort.
3. Рекурсивная сортировка $i + 1$,..., n с помощью Quicksort рекурсивно.

После шага 1 элемент e находится в своём правильном расположении. Так как элементы слева от e все меньше или равны ему, они будут в своих местах, как только они будут отсортированы в шаге 2; Аналогичные рассуждения применимы к элементам справа от e . Шаг 1 алгоритма Quicksort, очевидно, критичен. Существуют различные

методы разбиения массива, некоторые из них намного лучше, чем другие.

Мы будем использовать технику, которую легко понять, но не особенно эффективную.

Вначале я опишу алгоритм разделения неформально; позже мы переведем его в код на языке Си.

Алгоритм опирается на два "маркера" с именами `low` и `high`, которые отслеживают позиции внутри массива.

Первоначально, `low` указывает на первый элемент массива и `high` указывает на последний элемент.

Мы начнем с копирования первого элемента (элемента разбиения) во временное место в другом месте, оставив в массиве "дыру".

Затем мы перемещаем `high` по массиву справа налево, пока он не укажет на элемент, который меньше элемента разбиения.

Затем мы копируем элемент в дырку, на которую указывает `low`, что создаёт новую дырку (на которую указывает `high`)

Теперь мы перемещаем `low` слева направо, ищем элемент, который больше, чем элемент разбиения.

Когда он будет найден, мы скопируем его в дырку, на которую указывает `high`.

Процесс повторяется, с `low` и `high` поочередно, пока они не встретятся где-то посередине массива.

В это время оба будут указывать на дырку; всё, что нам нужно сделать, это скопировать элемент разбиения в дырку.

На следующих диаграммах показано, как Quicksort сортирует массив целых чисел:

461

Начнём с массива, содержащего семь элементов, low указывает на первый элемент high указывает на последний.

12	3	6	18	7	15	10
↑						↑
low						high

Первый элемент, 12, является элементом разбиения. Копирование его в другое место оставляет дыру в начале массива.

	3	6	18	7	15	10
						12
↑						↑
low						high

Теперь сравним элемент, на который указывает high с 12. Так как 10 меньше 12, он находится на неправильной стороне массива, поэтому мы перемещаем его в дырку и сдвигаем low вправо.

10	3	6	18	7	15	
						12
	↑					↑
	low					high

462

low указывает на число 3, которое меньше 12 и, следовательно, его не нужно перемещать. Вместо этого мы сместим low вправо.

12

10	3	6	18	7	15	
----	---	---	----	---	----	--

↑
low

↑
high

Так как 6 также меньше 12, мы снова сдвигаем low.

12

10	3	6	18	7	15	
----	---	---	----	---	----	--

↑ low ↑ high

low теперь указывает на 18, которое больше 12 и, следовательно, вне позиции. После перемещения 18 в дырку сдвигаем high влево.

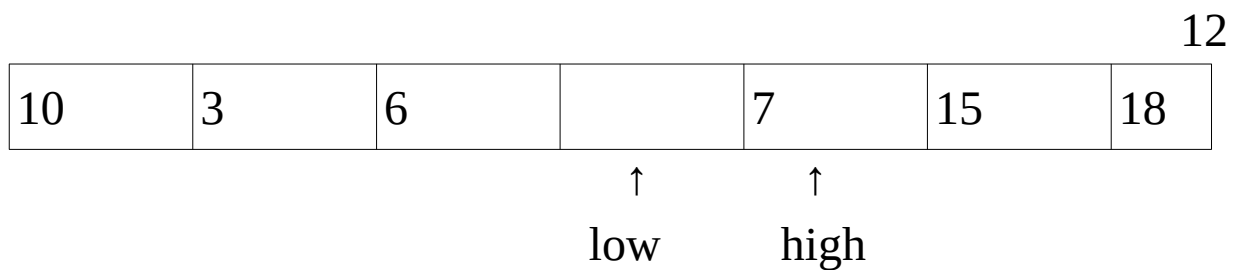
12

10	3	6		7	15	18
----	---	---	--	---	----	----

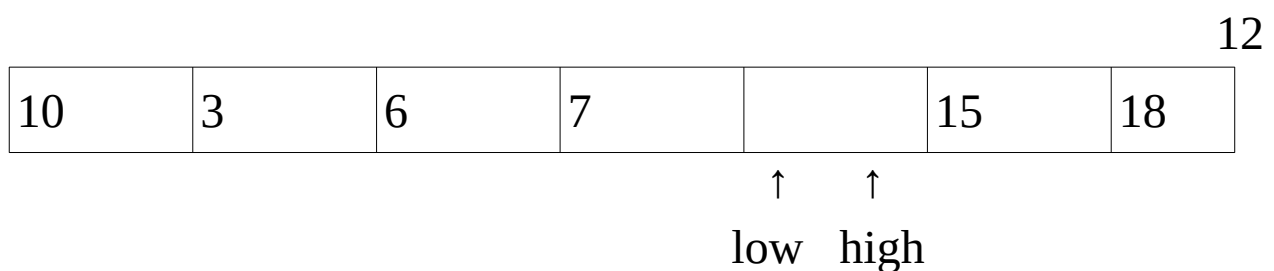
↑ low ↑ high

463

high указывает на 15, что больше 12 и, следовательно, не нужно перемещать. Мы сдвигаем high влево и продолжаем.



high указывает на 7, что находится вне позиции. После перемещения 7 в дырку, мы сдвигаем low вправо.



low и high теперь равны, поэтому мы переместим элемент разбиения в дырку.

10	3	6	7	12	15	18
----	---	---	---	----	----	----

На данный момент мы достигли нашей цели: все элементы слева от элемента разбиения меньше или равны 12, а все элементы справа больше или равны 12.

Теперь, когда массив был секционирован, мы можем рекурсивно использовать Quicksort для сортировки первых четырёх элементов массива (10, 3, 6 и 7) и двух последних (15 и 18).

ПРОГРАММА Quicksort

Разработаем рекурсивную функцию quicksort, которая использует алгоритм Quicksort для сортировки массива целых чисел.

Чтобы протестировать функцию, мы будем читать в строке 10 чисел, вызывать quicksort для сортировки массива, а затем печатаем элементы массива:

Введите 10 чисел для сортировки: 9 16 47 82 4 66 12 3 25 51

В отсортированном порядке: 3 4 9 12 16 25 47 51 66 82

Так как код для разбиения массива на части длинный. Я положу его в отдельную функцию split.

```
//qsort.c
```

```
/* Сортировка массива целых чисел с помощью алгоритма
быстрой сортировки */
```

465

```
#include <stdio.h>
```

```
#define N 10
```

```
void quicksort(int a[], int low, int high);
```

```
int split(int a[], int low, int high);
```

```
int main(void){
```

```
    int a[N], i;
```

```
    printf("Введите числа для сортировки в количестве: %d ",  
           N);
```

```
    for (i = 0; i < N; i++)  
        scanf("%d", &a[i]);
```

```
    quicksort(a, 0, N - 1);
```

```
    printf("В отсортированном порядке: ");
```

```
    for (i = 0; i < N; i++)  
        printf("%d ", a[i]);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

```
void quicksort(int a[], int low, int high){
    int middle;

    if (low >= high) return;
        middle = split(a, low, high);
    quicksort(a, low, middle - 1);
    quicksort(a, middle + 1, high);
}

int split(int a[], int low, int high){
    int part_element = a[low];

    for (;;) {
        while (low < high && part_element <= a[high])
            high--;
        if (low >= high) break;
        a[low++] = a[high];

        while (low < high && a[low] <= part_element)
            low++;
        if (low >= high) break;
        a[high--] = a[low];
    }
}
```

```
a[high] = part_element;  
return high;  
}
```

Хотя эта версия Quicksort работает, она не лучшая.

Существует множество способов улучшить производительность программы, в том числе:

■ Улучшение алгоритма разбиения.

Наш метод не является наиболее эффективным. Вместо выбора первого элемента массива в качестве элемента разбиения лучше брать среднее первого элемента, среднего элемента и последнего элемента.

■ Сам процесс секционирования также можно ускорить. В частности, можно избежать тестов $low < high$ в двух циклах `while`.

■ Использование другого метода для сортировки небольших массивов. Вместо того, чтобы использовать Quicksort рекурсивно вплоть до массивов с одним элементом, лучше использовать более простой метод для небольших массивов (с менее чем 25 элементами).

■ Сделать Quicksort нерекурсивным. Хотя Quicksort по своей природе является рекурсивным алгоритмом, и его проще всего понять в рекурсивной форме, на самом деле он более эффективен, если рекурсия удалена. Для получения дополнительной информации об улучшении Quicksort обратитесь к книге по разработке алгоритмов, такой как Роберт Сежвик, Фундаментальные алгоритмы на Си, третья редакция, Части 1-5, изд-во Диасофт, 2003

Вопросы и ответы

Вопрос: В некоторых книгах по Си используются термины, отличные от терминов параметр и аргумент. Существует ли стандартная терминология?

Ответ: Как и во многих других аспектах Си, нет общего соглашения по терминологии, хотя в стандартах C89 и C99 используются термины параметр и аргумент. Следующая таблица поможет вам перевести:

Эта книга:	Другие книги:
параметр	формальный аргумент, формальный параметр
аргумент	фактический аргумент фактический параметр

Имейте в виду, что, когда не будет путаницы, я иногда намеренно стираю различие между этими двумя терминами, используя аргумент, чтобы означать любой из них.

Вопрос: Я видел программы, в которых типы параметров указываются в отдельных объявлениях после списка параметров, как в следующем примере:

```
double average(a, b)
double a, b;
{
    return (a + b) / 2;
}
```

Является ли эта практика допустимой?

Ответ: Этот метод определения функций исходит от К & Р. Си поэтому вы можете столкнуться с ним в старых книгах и программах. C89 и C99 поддерживают этот стиль, поэтому старые программы будут компилироваться. Однако я бы не стал использовать его в новых программах, по нескольким причинам.

Во-первых, функции, которые определены более старым способом, не подвержены одинаковой степени проверки ошибок. Когда функция определена более старым способом - и прототип отсутствует, компилятор не будет проверять, что функция вызывается с нужным числом аргументов, и не будет проверять, что аргументы имеют соответствующие типы.

Вместо этого он будет выполнять продвижение аргументов по умолчанию.

Во-вторых, стандарт Си говорит, что старый стиль "устарел", что означает, что его использование не рекомендуется, и что он может быть удалён из Си в конце концов.

Вопрос: В некоторых языках программирования процедуры и функции могут быть вложены друг в друга. Разрешает ли Си определениям функций быть вложенными?

Ответ: Нет, Си не позволяет определять одну функцию в теле другой. Помимо всего прочего, это ограничение упрощает компилятор.

Вопрос: Почему компилятор допускает использование имён функций, которые не сопровождаются круглыми скобками?

Ответ: В следующей главе мы увидим, что компилятор рассматривает имя функции, не сопровождаемое круглыми скобками, как указатель на функцию. Указатели на функции имеют правомерное применение, поэтому компилятор не может автоматически предполагать, что имя функции без скобок является ошибкой. Инструкция

```
print_run;
```

Является допустимой, поскольку компилятор рассматривает `print_run` как указатель и, следовательно, выражение, что

делает его допустимым (хотя и бессмысленным) выражением-инструкцией.

Вопрос: В вызове функции $f(a, b)$, каким образом компилятор распознаёт, является ли запятая знаком пунктуации или она является оператором?

Ответ: Оказывается, аргументы в вызове функции не могут быть произвольными выражениями. Вместо этого они должны быть "выражениями присваивания", которые не могут содержать запятых, используемых в качестве операторов, если они не заключены в круглые скобки.

Другими словами, в вызове $f(a, b)$ запятая является знаком пунктуации; в вызове $f((a, b))$ это оператор.

Вопрос: Должны ли имена параметров в прототипе функции соответствовать именам, указанным позднее в определении функции?

Ответ: Некоторые программисты используют этот факт, давая длинные имена параметрам в прототипе, а затем используя более короткие имена в фактическом определении. Или говорящий на французском языке программист может использовать английские имена в прототипах, а затем переключиться на более знакомые французские имена в определениях функций.

Вопрос: Я до сих пор не понимаю, почему мы возимся с прототипами функций. Если мы просто зададим определения всех функций перед main, не будет ли этого достаточно?

Ответ: Нет. Во-первых, вы предполагаете, что только main вызывает другие функции, что нереально. На практике некоторые из функций будут вызывать друг друга.

Если мы поместим все определения функции перед main, нам придётся внимательно следить за их порядком.

Вызов функции, которая ещё не определена, может привести к большим проблемам. Но это не все. Предположим, что две функции вызывают друг друга (что не является надуманным, как может показаться). Независимо от того, какую функцию мы определим первой, она в конечном итоге вызовет функцию, которая ещё не определена. Но и это ещё не всё! Как только программы достигнут определённого размера, больше будет невозможно поместить все функции в один файл. Когда мы достигнем этого момента, нам понадобятся прототипы, чтобы сообщить компилятору о функциях в других файлах.

Вопрос: Я видел объявления функций, которые опускают всю информацию о параметрах:

```
double average();
```

Является ли эта практика допустимой?

Ответ: Да. Это объявление сообщает компилятору, что `average` возвращает значение `double`, но не предоставляет информацию о количестве и типах его параметров.

(Оставленные пустыми скобки не обязательно означают, что у `average` нет параметров.) В К & Р Си этот вид объявления функции является единственным допустимым; форма, которую мы использовали - прототип функции, в который включена информация о параметрах - была введена в С89. Устаревшее объявление функции теперь устарело, хотя и разрешено.

Вопрос: Почему программист намеренно пропускает имена параметров в прототипе функции? Разве не проще просто оставить имена?

Ответ: Опускание имён параметров в прототипах обычно делается для защитных целей. Если макрос имеет то же самое имя что и у параметра, имя параметра будет заменено во время предварительной обработки, тем самым повредив прототип, в котором оно появляется. Это вряд ли будет проблемой в небольшой программе, написанной одним человеком, но она может возникнуть в больших приложениях, написанных разными людьми.

Вопрос: Можно ли поместить объявление функции внутри тела другой функции?

Ответ: Да. Вот пример:

```
int main (void){  
    double average(double a, double b);  
    ...  
}
```

Это объявление `average` действительно только для тела `main`; Если другие функции должны вызывать `average`, каждая из них должна будет объявить её. Преимущество этой практики в том, что читателю понятнее, какие функции вызывают другие функции. (В этом примере мы видим, что `main` будет вызывать `average`.) С другой стороны, это может быть неудобным, если несколько функций должны вызывать одну и ту же функцию. Хуже того, попытка добавить и удалить объявления во время сопровождения программы может стать настоящим наказанием. По этим причинам я всегда помещаю объявления функций вне функций.

Вопрос: Если несколько функций имеют один и тот же тип возврата, можно ли их объявления объединить?

Например, поскольку и `print_pun`, и `print_count` имеют тип `void` в качестве возвращаемого типа, является ли следующее объявление допустимым?

```
void print_pun(void), print_count(int n);
```

Ответ: Да. Фактически Си даже позволяет нам комбинировать объявления функций с объявлениями

переменных:

```
double x, y, average (double a, double b);
```

Однако объединение объявлений таким способом обычно не является хорошей идеей; это может легко вызвать путаницу.

Вопрос: Что произойдёт, если задать длину для одномерного массива-параметра?

Ответ: Компилятор игнорирует её. Рассмотрим следующий пример:

```
double inner_product(double v[3], double w[3] );
```

Помимо документирования, что аргументы `inner_product` должны быть массивами длиной 3, указание длины не даёт нам ничего. Компилятор не будет проверять, что аргументы действительно имеют длину 3, поэтому нет дополнительной безопасности.

На самом деле, такая практика вводит в заблуждение, поскольку предполагает, что `inner_product` может быть переданы только массивы длиной 3, тогда как на самом деле мы можем передавать массивы произвольной длины.

Вопрос: Почему первое измерение в параметре-массиве может быть оставлено неопределённым, но не в других измерениях?

Ответ: Во-первых, нам нужно обсудить, как в Си передаются массивы. Как описано в Разделе 12.3, когда массив передаётся функции, функция получает указатель на

первый элемент массива. Далее нам нужно знать, как работает индексный оператор. Предположим, что $\mathbf{a}$ - это одномерный массив, переданный функции. Когда мы пишем

$$\mathbf{a}[\mathbf{i}] = 0$$

компилятор генерирует инструкции, которые вычисляют адрес $[\mathbf{i}]$, умножая $\mathbf{i}$ на размер элемента массива и добавляя результат к адресу, который представляет $\mathbf{a}$ (указатель, переданный функции). Это вычисление не зависит от длины $\mathbf{a}$, что объясняет, почему мы можем опустить его при определении функции.

А как насчёт многомерных массивов? Вспомните, что Си хранит массивы в порядке строк, причём сначала хранятся элементы в строке 0, затем элементы в строке 1 и т. д. Предположим, что $\mathbf{a}$ является двумерным параметром-массивом, и мы пишем $\mathbf{a}[\mathbf{i}][\mathbf{j}] = 0$;

Компилятор генерирует инструкции, чтобы сделать следующее:

- (1) умножить $\mathbf{i}$ на размер одной строки $\mathbf{a}$;
 - (2) добавить этот результат к адресу, который представляет $\mathbf{a}$;
 - (3) умножить $\mathbf{j}$ на размер элемента массива; а также
 - (4) добавить этот результат к адресу, вычисленному на шаге 2.
- Для генерации этих инструкций компилятор должен знать размер строки в массиве, который определяется количеством столбцов.

В итоге: программист должен объявить количество столбцов в $\mathbf{a}$.

Вопрос: Почему некоторые программисты ставят круглые скобки вокруг выражения в инструкции return?

Ответ: В примерах первого издания "Язык программирования Си" Кернигана и Ритчи всегда есть скобки в инструкциях return, хотя они и не требуются. Программисты (и авторы последующих книг) подхватили эту привычку у К & Р.

Я не использую эти скобки, так как они не нужны и не дают ничего для читаемости. (Керниган и Ритчи, по-видимому, согласны с этим, и во втором выпуске "Язык программирования Си" в инструкции return отсутствуют скобки.

Вопрос: Что произойдёт, если non-void функция попытается выполнить инструкцию return, у которой нет выражения?

Ответ: Это зависит от версии Си. В C89 выполнение инструкции return без выражения в непустой функции вызывает неопределённое поведение (но только в том случае, если программа пытается использовать значение, возвращаемое функцией). В C99 такая инструкция является недопустимой и должна обнаруживаться компилятором как ошибка.

Вопрос: Как я могу проверить возвращаемое значение main, чтобы увидеть, нормально ли завершилась программа?

Ответ: Это зависит от вашей операционной системы.

Многие операционные системы позволяют тестировать это значение в "пакетном файле" или "сценарии оболочки", который содержит команды для запуска нескольких программ. Например, строка

if errorlevel 1 command

В пакетном файле Windows выполнит команду, если последняя программа завершена с кодом состояния, большим или равным 1. В UNIX каждая оболочка имеет свой собственный метод тестирования кода состояния.

В оболочке Bourne переменная \$? содержит состояние последней запущенной программы. Оболочка C shell - имеет похожую переменную, но её имя равно \$status.

Вопрос: Почему мой компилятор выдаёт предупреждение "control reaches end of non-void function" (контроль достигает конца не void функции), когда компилирует main?

Ответ: Компилятор заметил, что main, несмотря на наличие в качестве возвращаемого типа int, не имеет инструкции return. Помещение инструкции

return 0;

в конец main убогит компилятор. Кстати, это хорошая практика, даже если ваш компилятор не возражает против отсутствия инструкции return. Когда программа компилируется с использованием компилятора C99, это

предупреждение не должно появляться. В C99 все в порядке, чтобы "отпасть" в конце `main`, не возвращая значение; в стандарте говорится, что `main` автоматически возвращает 0 в этой ситуации.

Вопрос: Что касается предыдущего вопроса: почему бы просто не определить тип возвращаемого значения `main` как `void`?

Ответ: Хотя эта практика довольно распространена, она запрещена в соответствии со стандартом C89. Даже если бы она была допустима, было бы не очень хорошо, так как предполагается, что никто не будет проверять статус программы после завершения. C99 открывает дверь для легализации этой практики, позволяя `main` быть объявленным "в некотором другом определённом реализацией способом" (с типом возврата, отличным от `int`, или параметрами, отличными от тех, которые указаны стандартом). Однако любое такое использование не является переносимым, поэтому лучше объявить возвращаемый тип `main` равным `int`.

Вопрос: Допустимо ли для функции `f1` вызывать функцию `f2`, которая затем вызывает `f1`?

Ответ: Да. Это просто прямая форма рекурсии, в которой один вызов `f1` приводит к другому. (Но убедитесь, что либо `f1`, либо `f2` в конце концов завершается!)

Упражнения

К разделу 9.1

1. Следующая функция, вычисляющая площадь треугольника, содержит две ошибки. Найдите ошибки и покажите, как их устранить. (Подсказка: в формуле нет ошибок.)

```
double triangle_area(double base, height){  
    double product;  
    product = base * height;  
    return product / 2;  
}
```

2. Напишите функцию `check (x, y, n)`, которая возвращает 1, если x и y лежат между 0 и $n-1$ включительно. В противном случае функция должна вернуть 0. Предположите, что x , y и n имеют тип `int`.

3. Напишите функцию `gcd (m, n)`, которая вычисляет наибольший общий делитель целых чисел m и n . (Проект по программированию 2 в Главе 6 описывает алгоритм Евклида для вычисления НОД (GCD).)

4. Напишите функцию `day_of_year (month, day, year)`, которая возвращает день года (целое число от 1 до 366), определяемое тремя аргументами.

5. Напишите функцию `num_digits (n)`, которая возвращает число цифр в `n` (положительное целое число). Подсказка: Чтобы определить количество цифр в числе `n`, разделите его на 10 несколько раз. Когда частное достигает 0, количество делений укажет, сколько цифр было изначально в `n`.

6. Напишите функцию `digit (n, k)`, которая возвращает `k`-ю цифру (справа) в `n` (положительное целое число). Например, `digit (829, 1)` возвращает 9, `digit (829, 2)` возвращает 2, а `digit (829, 3)` возвращает 8. Если `k` больше числа цифр в `n`, функция возвращает 0.

7. Пусть функция `f` имеет следующее определение:

```
int f (int a, int b) {...}
```

Какое из следующих утверждений является допустимым? (Предположим, что `i` имеет тип `int`, а `x` имеет тип `double`.)

(a) `i = f (83, 12);`

(b) `x = f (83,12);`

(c) `i = f (3,15, 5,28);`

(d) `x = f (3,15, 9,28);`

(e) `f (83, 12);`

К разделу 9.2

8. Что из следующего будет правильным прототипом для функции, которая ничего не возвращает и имеет один

482

параметр double?

(a) void f(double x);

(b) void f(double);

(c) void f (x);

(d) f (double x);

К разделу 9.3

* 9. Каким будет результат следующей программы?

```
#include <stdio.h>
```

```
void swap(int a, int b);
```

```
int main (void)
```

```
{
```

```
    int i = 1, j = 2;
```

```
    swap(i, j);
```

```
    printf("i = %d, j = %d\n", i, j);
```

```
    return 0;
```

```
}
```

```
void swap(int a, int b){
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

W10. Напишите функции, которые возвращают следующие значения. (Предположим, что *a* и *n* являются параметрами, где *a* - массив значений *int*, а *n* - длина массива.)

- (a) Самый большой элемент в *a*.
- (b) Среднее значение всех элементов в *a*.
- (c) Число положительных элементов в *a*.

11. Напишите следующую функцию:

```
float compute_GPA(char grades[], int n);
```

Массив *grades* будет содержать буквенные оценки (A, E, C, D или F, в верхнем или нижнем регистре);

n - длина массива. Функция должна возвращать среднее значение оценок (предположим, что A = 4, B = 3, C = 2, D = 1 и F = 0).

12. Напишите следующую функцию:

```
double inner_product (double a [], double b [], int n);
```

Функция должна возвращать $a[0] * b[0] + a[1] * b[1] + \dots + a[n-1] * b[n-1]$.

13. Напишите следующую функцию, которая оценивает положение шахмат:

```
int evaluate_pozition (char board [8] [8] );
```

board представляет собой конфигурацию фигур на шахматной доске, где буквы K, Q, R, B, N, P представляют белые фигуры, а буквы k, q, r, b, n и p представляют чёрные фигуры, evaluate_position должна суммировать значения белых фигур (Q = 9, R = 5, B = 3, N = 3, P = 1). Она также должна суммировать значения чёрных фигур (аналогичным образом). Функция возвращает разность между двумя числами. Это значение будет положительным, если у белых есть материальное преимущество в отрицательное, если преимущество есть у чёрных.

К разделу 9.4

14. Следующая функция должна возвращать true, если любой элемент массива a имеет значение 0 и false, если все элементы отличны от нуля. К сожалению, она содержит ошибку.

Найдите ошибку и покажите, как её исправить:

```
bool has_zero(int a[], int n)
```

```
{
```

```
int i;
```

```
for (i = 0; i < n; i++)
```

```
    if (a[i] == n)
```

```
        return true;
```

```
    else
```

```
        return false;
```

```
}
```


485

W 15. Следующая (довольно запутанная) функция находит число находящееся посередине. Перепишите функцию так, чтобы она имела только одну инструкцию return.

```
double median (double x, double y, double z){  
    if (x <= y)  
        if (y <= z) return y;  
        else if (x <= z) return z;  
        else return x;  
    if (z <= y) return y;  
    if (x <= z) return x;  
    return z;  
}
```

К разделу 9.6

16. Сожмите функцию fact так же, как мы сжимали функцию power.

W 17. Перепишите функцию fact так, чтобы она больше не была рекурсивной.

18. Напишите рекурсивную версию функции gcd (см. упражнение 3).

Вот стратегия использования gcd (m, n): если n равно 0, верните m. В противном случае, вызовите gcd рекурсивно,

486

передав n в качестве первого аргумента и $m \% n$ в качестве второго.

19. Рассмотрим следующую "загадочную" функцию:

```
void pb (int n)
{
    if (n!= 0){
        Pb(n/2);
        putchar('0' + n % 2);
    }
}
```

Проследите выполнение функции вручную. Затем напишите программу, которая вызывает функцию, передав ей число, введенное пользователем. Что делает функция?

Проекты по программированию

1. Напишите программу, которая попросит пользователя ввести ряд целых чисел (которые она хранит в массиве), затем сортирует целые числа, вызывая функцию `selection_sort`. Когда задан массив с n элементами, `selection_sort` должна

сделать следующее:

1. Просмотреть массив, чтобы найти наибольший элемент, затем переместить его в последнюю позицию в массиве.
2. Рекурсивно вызывать первые $n-1$ элементов массива.

2. Измените Проект по программированию 5 из Главы 5, чтобы он использовал функцию для вычисления суммы подоходного налога. Когда передаётся сумма налогооблагаемого дохода, функция возвращает налог.

3. Измените Проект по программированию 9 из Главы 8, чтобы он включал в себя следующие функции:

```
void generate_random_walk (char walk [10] [10]);  
void print_array (char walk[0] [10]);
```

main сначала вызывает generate_random_walk, инициализирующую массив, который должен содержать символ ".", а затем заменяет некоторые из этих символов буквами от A до Z, как описано в исходном проекте. main затем вызывает print_array для отображения массива на экране.

4. Измените Проект по программированию 16 из Главы 8, чтобы он включал в себя следующие функции:

```
void read_word(int counts[26]);  
  
bool equal_array(int counts1[26], int counts2[26]);
```

main вызовет `read_word` дважды, для каждого из двух слов, введенных пользователем. Когда он читает слово, `read_word` будет использовать буквы в слове для обновления массива `counts`, как описано в исходном проекте.

(main объявит два массива, по одному для каждого слова. Эти массивы используются для отслеживания того, сколько раз каждая буква встречается в словах.)

main затем вызовет `equal_array`, передав ему два массива, `equal_array` вернёт `true`, если элементы в двух массивах идентичны (это означает, что слова являются анаграммами) и `false` в противном случае.

5. Измените Проект по программированию 17 из Главы 8, чтобы он включал в себя следующие функции:

```
void create_magic_square(int n, int magic_square[n] [n]);
```

```
void print_magic_square(int n, int magic_square[n] [n] );
```

После получения числа `n` от пользователя main вызовет `create_magic_square`, передав ему массив `n x n`, объявленный внутри main.

`create_magic_square` заполняет массив числами $1, 2, \dots, n^2$, как описано в исходном проекте.

main затем вызовет `print_magic_square`, который отобразит массив в формате, описанном в исходном проекте.

Примечание: Если ваш компилятор не поддерживает массивы переменной длины, объявите массив в main равным `99 x 99` вместо `n x n` и используйте следующие прототипы:

```
void create_magic_square(int n, int magic_square [99] [99]);
```

```
void print_magic_square(int n, int magic_square[99] [99]);
```

6. Напишите функцию, вычисляющую значение следующего многочлена:

$$3x^5 + 2x^4 - 5x^3 - x^2 + 7x - 6$$

Напишите программу, которая попросит пользователя ввести значение для x , вызовет функцию для вычисления значения многочлена и затем отобразит значение, возвращённое функцией.

7. Функция `power` раздела 9.6 может быть выполнена быстрее, если вычислить x^n по-другому. Сначала заметим, что если n - степень 2, то x^n может быть вычислено возведением в квадрат. Например, x^4 - это квадрат x^2 , поэтому x^4 можно вычислить, используя только два умножения вместо трёх.

Между прочим, этот метод можно использовать, даже когда n не является степенью 2.

Если n чётное, мы используем формулу $x^n = (x^{n/2})^2$. Если n нечётно, то $x^n = x * x^{(n-1)}$.

Напишите рекурсивную функцию, которая вычисляет x^n .

(Рекурсия заканчивается, когда $n = 0$, и в этом случае функция возвращает 1.)

Чтобы проверить свою функцию, напишите программу, которая просит пользователя ввести значения для x и n , вызывает `power` для вычисления x^n и затем отображает значение, возвращаемое функцией.

8. Напишите программу, которая имитирует игру в крэпс, в которую играют две кости. В первом броске игрок выигрывает, если сумма костей равна 7 или 11. Игрок проигрывает, если сумма равна 2, 3 или 12.

Любой другой бросок называется "точкой", и игра продолжается.

На каждом последующем броске игрок выигрывает, если он или она снова бросает точку.

Игрок проигрывает, бросая 7. Любой другой бросок игнорируется, и игра продолжается.

В конце каждой игры программа спросит пользователя, будет ли он играть снова.

Когда пользователь вводит ответ, отличный от у или Y, программа отображает количество выигрышей и потерь, а затем завершает работу.

Ваш бросок: 8

Ваша точка равняется 8

Ваш бросок: 3

Ваш бросок: 10

Ваш бросок: 8

Вы победили!

Играть снова? Y

Ваш бросок: 6

Ваша точка равняется 6

Ваш бросок: 5

Ваш бросок: 12

Ваш бросок: 3

491

Ваш бросок: 7

Вы проиграли

Играть снова? у

Ваш бросок: 11

Вы победили!

Играть снова? n

Победы: 2 проигрыши: 1

Напишите свою программу в виде трёх функций:

`main`, `roll_dice`, и `play_game`.

Вот прототипы для последних двух функций:

```
int roll_dice(void);
```

```
bool play_game (void);
```

`roll_dice` должна генерировать два случайных числа, каждое между 1 и 6, и возвращать их сумму.

`play_game` должна сыграть в одну игру в крэпс (вызывая `roll_dice`, чтобы определить результат каждого броска костей); Она вернёт `true`, если игрок выиграет, и `false`, если игрок проиграет.

`play_game` также несёт ответственность за отображение сообщений, показывающих результаты бросков костей игрока, `main` многократно вызывает `play_game`, отслеживает количество побед и поражений и отображает сообщения "вы выиграли" и "вы проиграли". Подсказка: используйте функцию `rand` для генерации случайных чисел.

См. Программу `deal.c` в Разделе 8.2 для примера того, как вызвать `rand` и соответствующую функцию `stand`.

10 Организация программы

Как сказал бы Уилл Роджерс: "В природе нет такой вещи, как свободная переменная".

Рассмотрев функции в Главе 9, мы готовы противостоять нескольким проблемам, возникающим, когда программа содержит более одной функции. Глава начинается с обсуждения различий между локальными переменными (Раздел 10.1) и внешними переменными (Раздел 10.2). Затем в Разделе 10.3 рассматриваются блоки (составные инструкции, содержащие объявления). В Разделе 10.4 рассматриваются правила области действия (видимости), применяемые к локальным именам, внешним именам и именам, объявленным в блоках. Наконец, в Разделе 10.5 предлагается способ организации прототипов функций, определения функций, деклараций переменных и других частей программы на Си.

10.1 Локальные переменные

Переменная, объявленная в теле функции, называется локальной для функции. В следующей функции `sum` является локальной переменной:

```
int sum_digits (int n){  
    int sum = 0; /* local variable */  
  
    while (n > 0) {  
        sum += n % 10;  
        n /= 10;  
    }  
    return sum;  
}
```

По умолчанию локальные переменные имеют следующие свойства:

■ **Автоматическую длительность хранения.** Длительность хранения переменной - это часть выполнения программы, в течение которой существует хранилище для переменной. Хранилище для локальной переменной выделяется "автоматически", когда вызывается заключающая её функция

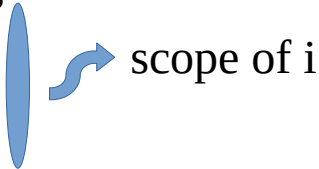
и освобождается при возврате функции, поэтому считается, что переменная имеет автоматическую длительность хранения. Локальная переменная не сохраняет своего значения после возврата заключающей её функции. Когда функция вызывается снова, нет гарантии, что переменная будет все ещё иметь старое значение.

■ **Блочная область видимости.** Область видимости переменной - это часть текста программы, в которой можно ссылаться на переменную.

Локальная переменная имеет блочную область видимости: она видна от её точки объявления до конца охватывающего тела функции. Поскольку область видимости локальной переменной не выходит за пределы функции, к которой она принадлежит, другие функции могут использовать одно и то же имя для других целей.

В Разделе 18.2 эти и другие связанные понятия рассматриваются более подробно. Поскольку C99 не требует объявления переменных в начале функции, локальная переменная может иметь очень маленькую область видимости. В следующем примере область *i* не начинается до строки, в которой она объявлена, которая может находиться ближе к концу тела функции:

```
void f(void){  
    ...  
    int i;  
    ...  
}
```



Статические локальные переменные

Помещение слова `static` в объявление локальной переменной приводит к тому, что вместо автоматической длительности хранения у него будет статическая длительность хранения.

Переменная со статической длительностью хранения имеет постоянное место хранения, поэтому она сохраняет своё значение на протяжении всего выполнения программы.

Рассмотрим следующую функцию:

```
void f(void){  
    static int i; /* static local variable */  
    ...  
}
```

Так как локальная переменная *i* была объявлена статической, она занимает одно и то же место в памяти во время выполнения программы. После возврата *f*, *i* не будет терять своё значение. Статическая локальная переменная все ещё имеет блочную область видимости, поэтому она не видна другим функциям.

В двух словах, статическая локальная переменная - это место, где можно скрыть данные от других функций, но сохранить их для будущих вызовов той же функции.

Параметры

Параметры имеют подобные свойства - автоматическую длительность хранения и блочную область видимости - как локальные переменные. Фактически, единственное реальное различие между параметрами и локальными переменными состоит в том, что каждый параметр инициализируется автоматически при вызове функции (путём присваивания значения соответствующего аргумента).

10.2 Внешние переменные

Передача аргументов является одним из способов передачи информации в функцию. Функции также могут взаимодействовать через внешние переменные - переменные, объявленные вне тела любой функции. Свойства внешних переменных (или глобальных переменных, как их иногда называют) отличаются от свойств локальных переменных:

- Статическая длительность хранения. Внешние переменные имеют статическую длительность хранения, как и локальные переменные, которые были объявлены статическими. Значение, хранящееся во внешней переменной, останется там неограниченно.
- Файловая область действия. Внешняя переменная имеет файловую область видимости: она видна от точки объявления до конца охватывающего файла. В результате внешняя

переменная может быть доступна (и потенциально модифицируема) всеми функциями, которые следуют за её объявлением.

Пример: использование внешних переменных для реализации стёка

Чтобы проиллюстрировать использование внешних переменных, давайте рассмотрим структуру данных, известную как стёк. (стёки - это абстрактная концепция, а не функция Си, они могут быть реализованы в большинстве языков программирования.)

стёк, как и массив, может хранить несколько элементов данных одного типа. Однако операции в стёке ограничены: Мы можем либо вставить элемент в стёк (добавить его в один конец - "верх стёка"), либо вытащить его из стёка (удалить его с одного конца). Запрещается проверять или изменять элемент, который находится не в верхней части стёка.

Одним из способов реализации стёка в Си является сохранение его элементов в массиве, который мы будем называть `contents`. Отдельная целочисленная переменная с именем `top` отмечает положение вершины стёка. Когда стёк пуст, `top` имеет значение 0.

Чтобы протолкнуть элемент в стёк, мы просто сохраняем элемент в `contents` в позиции, обозначенной `top`, а затем инкрементируем `top`. Выталкивание элемента требует декрементирования `top`, затем использование его в качестве индекса в `contents` для извлечения элемента, который выталкивается.

На основе этих соображений представлен фрагмент программы (не полная программа), который объявляет переменные `contents` и `top` для стёка и даёт набор функций, которые представляют операции в стёке. Все пять функций нуждаются в доступе к переменной `top`, а две функции нуждаются в доступе к `contents`, поэтому мы сделаем `contents` и `top` внешними.

```
#include <stdbool.h>          /* C99 only */
#define STACK_SIZE 100       /* external variables */
int contents [STACK_SIZE];
int top = 0;

void make_empty(void){
    top = 0;
}

bool is_empty(void){
    return top == 0;
}

bool is_full(void){
    return top == STACK_SIZE;
}

void push (int i){
    if (is_full())
```

```
        stack_overflow();
    else
        contents[top++] = i;
}

int pop (void){
    if (is_empty())
        stack_underflow();
    else
        return contents[--top];
}
```

Плюсы и минусы внешних переменных

Внешние переменные удобны, когда много функций должны совместно использовать переменную или когда несколько функций совместно используют большое количество переменных. Однако в большинстве случаев функциям лучше взаимодействовать через параметры, а не через общие переменные. Вот почему:

■ Если мы изменяем внешнюю переменную во время обслуживания программы (например, изменяем её тип), нам нужно будет проверить каждую функцию в том же файле, чтобы увидеть, как это изменение влияет на неё.

■ Если внешней переменной присваивается некорректное значение, может быть сложным определение виновной функции. Это похоже на попытку раскрыть убийство, совершенное на переполненной вечеринке - нет простого способа сузить список подозреваемых.

■ Функции, которые полагаются на внешние переменные, трудно использовать в других программах. Функция, зависящая от внешних переменных, не является самодостаточной; для повторного использования функции нам придётся тащить за ней все внешние переменные, которые ей нужны.

Многие программисты на Си слишком полагаются на внешние переменные.

Одно распространённое злоупотребление: использование одной и той же внешней переменной для разных целей в разных функциях. Предположим, что нескольким функциям нужна переменная с именем *i* для управления инструкцией `for`.

Вместо объявления *i* в каждой функции, которая её использует, некоторые программисты объявляют её в верхней части программы, тем самым делая видимой переменную для всех функций. Эта практика плоха не только по причинам, перечисленным ранее, но и потому, что она вводит в заблуждение; некто, читающий эту программу позже, может подумать, что использование переменной связано, а на самом деле это не так. При использовании внешних переменных, убедитесь, что они имеют осмысленные имена. (Локальным переменным не всегда нужны осмысленные имена: часто трудно придумать лучшее имя, чем *i* для управляющей переменной в цикле `for`.) Если вы вдруг обнаружите, что

501

используете имена типа *i* и temp для внешних переменных, это подсказка, что, возможно, они действительно должны быть локальными переменными. Ввод внешних переменных, когда они должны быть локальными, может привести к некоторым довольно разочаровывающим ошибкам. Рассмотрим следующий пример, который должен отображать звёздочки: 10 x 10:

```
int i;
void print_one_row (void){
    for (i = 1; i <= 10; i++)
        printf("*");
}
void print_all_rows (void){
    for (i = 1; i <= 10; i++){
        print_one_row ();
        printf ( "\n");
    }
}
```

Вместо печати 10 строк print_all_rows печатает только одну строку. Когда print_one_row возвращается (заканчивает работу) после вызова в первый раз, *i* будет иметь значение 11. Инструкция for в print_all_rows проверяет *i* меньше или равно 10. Поскольку это не так, цикл завершается и функция возвращается(заканчивает работу).

ПРОГРАММА Угадывание числа

Чтобы получить больше опыта с внешними переменными, мы напишем простую игровую программу. Программа генерирует случайное число от 1 до 100, которое пользователь пытается угадать делая как можно меньше попыток. Вот что увидит пользователь при запуске программы:

Угадайте секретное число между 1 и 100.

Выбрано новое число.

Введите догадку: 55

Слишком мало; попробовать ещё раз.

Введите догадку: 65

Слишком велико; попробовать ещё раз.

Введите догадку:60

Слишком велико; попробовать ещё раз.

Введите догадку: 58

Вы победили в 4 предположениях!

Играть снова? (Y/N) Y

Выбрано новое число:

Введите догадку: 78

Слишком велико; попробовать ещё раз.

Введите догадку:34

Вы победили в 2 предположениях!

503

Играть снова? (Y/N) n

Эта программа должна будет выполнять несколько различных задач: инициализацию генератора случайных чисел, выбор секретного числа и взаимодействие с пользователем до тех пор, пока не будет выбрано правильное число. Если мы напишем отдельную функцию для обработки каждой задачи, мы можем в итоге получить следующую программу.

```
// guess.c
```

```
/* Предлагает пользователю угадать скрытое число */
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define MAX_NUMBER 100
```

```
/* внешняя переменная*/
```

```
int secret_number;
```

```
/* прототипы */
```

504

```
void initialize_number_generator(void);
void choose_new_secret_number(void);
void read_guesses(void);

int main(void){
    char command;

    printf("Угадайте секретное число между 1 и %d.\n\n",
    MAX_NUMBER);
    initialize_number_generator();
    do {
        choose_new_secret_number();
        printf("Выбрано новое число.\n");
        read_guesses();
        printf("Играть снова? (Y/N) ");
        scanf(" %c", &command);
        printf("\n");
    } while (command == 'y' || command == 'Y');
    return 0;
}
```

```

/
*****

* initialize_number_generator: Инициализирует *
*
*     генератор случайных чисел *
*
*     используя время суток *

*****

/

void initialize_number_generator(void){
    srand((unsigned) time(NULL));
}

/
*****

*****

* choose_new_secret_number: Произвольно выбирает число *
* между 1 и MAX_NUMBER и *
* сохраняет его в secret_number. *

*****

*****/

```

506

```
void choose_new_secret_number(void){
```

```
    secret_number = rand() % MAX_NUMBER + 1;
```

```
}
```

```
/
```

```
*****
```

```
* read_guesses: Неоднократно читает угадывание *
```

```
* пользователя и сообщает пользователю, слишком *
```

```
* мала, слишком велика или правильна каждая догадка *
```

```
* Когда предположение верное, выводится общее *
```

```
* количество догадок и возвратов *
```

```
*****
```

```
/
```

```
void read_guesses(void){
```

```
    int guess, num_guesses = 0;
```

```
    for (;;) {
```

```
        num_guesses++;
```

```
        printf("Введите догадку: ");
```

```
        scanf("%d", &guess);
```

```
    if (guess == secret_number) {  
        printf("Вы победили в %d предположениях!\n\n",  
            num_guesses);  
        return;  
    } else if (guess < secret_number)  
        printf("Слишком мало; попробовать ещё раз\n");  
    else  
        printf("Слишком велико; попробовать ещё раз"  
            "\n");  
}  
}
```

Для генерации случайных чисел программа `guess.c` опирается на функции `time`, `srand` и `rand`, которые мы впервые использовали в `deal.c` (Раздел 8.2).

На этот раз мы масштабируем возвращаемое значение `rand` так, чтобы оно попадало между 1 и `MAX_NUMBER`. Хотя `guess.c` работает точно, она полагается на внешнюю переменную. Мы сделали `secret_number` внешней таким образом, чтобы к ней могли обращаться как `select_new_secret_number`, так и `read_guesses`; если мы немного изменим `select_new_secret_number` и `read_guesses`, мы сможем переместить `secret_number` в функцию `main`. Мы

508

изменим параметр `choose_new_secret_number` так, чтобы он возвращал новое число, и мы перепишем `read_guesses`, чтобы `secret_number` мог быть передан в качестве аргумента. Вот наша новая программа. С изменениями жирным шрифтом:

```
//guess2.c
```

```
/* Asks user to guess a hidden number */
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define MAX_NUMBER 100
```

```
/* prototypes */
```

```
void initialize_number_generator(void);
```

```
int new_secret_number(void);
```

```
void read_guesses(int secret_number);
```

```
int main(void){
```

```
    char command;
```

```
    int secret_number;
```



```

printf("Guess the secret number between 1 and %d.\n\n",
MAX_NUMBER);
initialize_number_generator();
do {
    secret_number = new_secret_number();
    printf("A new number has been chosen.\n");
    read_guesses(secret_number);
    printf("Play again? (Y/N) ");
    scanf(" %c", &command);
    printf("\n");
} while (command == 'y' || command == 'Y');

return 0;
}

/
*****
*****

* initialize_number_generator: Initializes the random *
*          number generator using          *

```

* the time of day. *

*****/

```
void initialize_number_generator(void){
    srand((unsigned) time(NULL));
}
```

/

*new_secret_number: возвращает случайно выбранное *

*число *

* из диапазона 1 - MAX_NUMBER. *

/

```
int new_secret_number(void){
    return rand() % MAX_NUMBER + 1;
}
```

/

* read_guesses: многократно читает догадки пользователя и *

*сообщает пользователю, является ли каждое *

*предположение слишком маленьким, слишком большим *

*или правильным. Если предположение верно, выводится *

*общее число предположений и возвращается. *

*****/

void read_guesses(int secret_number){

int guess, num_guesses = 0;

for (;;) {

num_guesses++;

printf("Введи догадку: ");

scanf("%d", &guess);

if (guess == secret_number) {

 printf("Ты победил в %d догадках!\n\n",
 num_guesses);

return;

```

    } else if (guess < secret_number)
        printf("Слишком мало; попробуй снова.\n");
    else
        printf("Слишком велико; попробуй снова.\n");
}
}

```

10.3 Блоки

В Разделе 5.2 мы встретили составные инструкции вида
 { инструкции }

Оказывается, что Си позволяет составным инструкциям содержать объявления:

Блок {объявления инструкции }

Я буду использовать термин **блок**, чтобы описать такую составную инструкцию. Вот пример блока:

```

if (i > j) {
    /* обмен значениями i и j */
    int temp = i;
    i = j;
    j = temp;
}

```

По умолчанию продолжительность хранения переменной, объявленной в блоке, является автоматической:

Память для переменной выделяется, когда блок начинает выполняться и освобождается, когда блок заканчивает выполнение. Переменная имеет область видимости блок; на неё нельзя ссылаться вне блока. Переменная, принадлежащая блоку, может быть объявлена статичной, чтобы придать ей статическое время хранения.

Тело функции является блоком. Блоки также полезны внутри тела функции, когда нам нужны переменные для временного использования. В нашем последнем примере мы временно нуждались в переменной, чтобы мы могли менять значения i и j . Помещение временных переменных в блоки имеет два преимущества:

(1) Это позволяет избежать захламления объявлений в начале тела функции с помощью переменных, которые используются только кратко.

(2) Это уменьшает конфликты имён. В нашем примере имя `temp` может использоваться в другом месте той же функции для других целей - переменная `temp` строго локальна в блоке, в котором она объявлена.

C99 позволяет объявлять переменные в любом месте блока, так же, как позволяет объявлять переменные в любом месте функции

10.4 Область действия (видимости)

В программе на Си, один и тот же идентификатор может иметь несколько различных значений. Правила области видимости Си позволяют программисту (и компилятору), определить, какое значение актуально в данный момент в программе.

Вот самое важное правило области видимости: когда внутри блока объявляется идентификатор, который уже виден (поскольку он имеет файловую область видимости или потому что он объявлен в охватывающем блоке), новое объявление временно "скрывает" старое. И идентификатор приобретает новое значение. В конце блока идентификатор восстанавливает своё прежнее значение.

Рассмотрим (несколько экстремальный) пример (см. рисунок ниже), в котором идентификатор *i* имеет четыре разных значения:

- В объявлении 1 *i* имеет статическую длительность хранения и файловую область видимости.
 - В объявлении 2 *i* является параметром с блоковой областью видимости.
 - В объявлении 3 *i* - это автоматическая переменная с блоковой областью видимости.
 - В объявлении 4 *i* также является автоматической переменной с блоковой областью видимости.
- i* используется пять раз.

```

int (i) ;           /* Declaration 1 */

void f(int (i) )    /* Declaration 2 */
{
    i = 1;
}

void g(void)
{
    int (i) = 2;     /* Declaration 3 */
    if (i > 0) {
        int (i) ;    /* Declaration 4 */
        i = 3;
    }
    i = 4;
}

void h(void)
{
    i = 5;
}

```

Правила области видимости Си позволяют нам определять значение ***i*** в каждом случае:

- ***i* = 1** присваивание относится к параметру в объявлении 2, не переменной в объявлении 1, так как объявление 2 скрывает объявление 1.

- Тест $i > 0$ ссылается на переменную в объявлении 3, поскольку объявление 3 скрывает объявление 1, а объявление 2 расположено вне области видимости.
- Присваивание $i = 3$ относится к переменной в объявлении 4, которое скрывает объявление 3.
- Присваивание $i = 4$ относится к переменной в объявлении 3. Оно не может относиться к объявлению 4, которое находится вне области видимости.
- Присваивание $i = 5$ ссылается на переменную в объявлении 1.

10.5. Организация программы на Си

Теперь, когда мы увидели основные элементы, составляющие программу на Си, пришло время разработать стратегию их организации. Пока же мы предполагаем, что программа всегда находится в одном файле.

В Главе 15 показано, как организовать программу, которая разделена на несколько файлов. До сих пор мы видели, что программа может содержать следующее:

Директивы препроцессорной обработки, такие как `#include` и `#define`

Определения типов

Объявления внешних переменных

Прототипы функций

Определения функции

Си накладывает только несколько правил на порядок этих элементов: директива препроцессорной обработки не вступает в силу до строки, в которой она появляется. Имя типа не может использоваться до тех пор, пока оно не будет определено. Нельзя использовать переменную, пока она не будет объявлена. Хотя Си не так придирчиво относится к функциям, я настоятельно рекомендую, чтобы каждая функция определялась или объявлялась до её первого вызова. (C99 делает это необходимым условием.) Существует несколько способов организовать программу, чтобы эти правила выполнялись. Вот один из возможных вариантов:

Директивы `#include`

Директивы `#define`

Определения типов

Объявления внешних переменных

Прототипы для функций, отличных от `main`

Определение `main`

Определения других функций

Имеет смысл сначала поместить директивы `#include`, так как они вводят информацию, которая, вероятно, понадобится в нескольких местах внутри программы, директивы `#define` создают макросы, которые обычно используются во всей программе. Помещение определений типов перед объявлениями внешних переменных логично, так как объявления этих переменных могут ссылаться на только что определённые имена типов. Объявление внешних переменных затем делает их доступными для всех последующих функций. Объявление всех функций, кроме `main`, позволяет избежать проблем, возникающих при вызове функции до того, как компилятор увидел её прототип. Эта практика также позволяет расположить определения функции в любом

порядке: в алфавитном порядке по имени функции или, например, со связанными функциями, сгруппированными вместе. Определение `main` перед другими функциями облегчает читателю поиск начальной точки программы.

Последнее предложение: перед каждым определением функции помещается комментарий, который даёт имя функции, объясняет её назначение, обсуждает значение каждого параметра, описывает её возвращаемое значение (если таковое имеется) и перечисляет любые побочные эффекты, которые она имеет (например, изменение внешних переменных).

ПРОГРАММА Классификация покера

Чтобы показать, как может быть организована программа на Си, давайте попробуем программу, которая немного сложнее, чем наши предыдущие примеры.

Программа читает и классифицирует покерную комбинацию.

Каждая карта в руке будет иметь как масть (трефы-clubs, бубны-diamonds, червы-hearts или пики-spade), так и ранг (два, три, четыре, пять, шесть, семь, восемь, девять, десять, валет-jack, дама-queen, король-king или туз- ace). Мы не используем джокеров, и будем считать, что наиболее высокий ранг у туза. Программа прочитает карточную комбинацию из пяти карт, затем классифицирует карточную комбинацию в одну из следующих категорий (перечисленных в порядке от лучшей к худшей):

стрейт-флаш (стрит и флеш)(пять последовательных карт одной масти)

четыре карты одного достоинства (четыре карты одного ранга)

фул-хаус ; "полный дом"; три карты одного достоинства и две другого

флеш (пять карт одной масти)

тройка карт одинакового достоинства

две пары пар (по две карты одного ранга)

старшая карта (любая другая комбинация)

Если карточная комбинация попадает в две или более категорий, программа выберет наиболее подходящую. Для удобства ввода мы сократим до буквенных значений ранги и масти следующим образом (буквы могут быть либо строчными, либо прописными):

Ранги: 2 3 4 5 6 7 8 9 t j q k a

Масти: c d h s

Если пользователь вводит недопустимую карту или пытается дважды ввести одну и ту же карту, программа проигнорирует карту, выдаёт сообщение об ошибке и затем запросит другую карту.

Ввод числа 0 вместо карты приведёт к завершению программы. Сессия с программой будет иметь следующий вид:

520

Введите карту: 2b

Введите карту: 5b

Введите карту: 4b

Введите карту: 3b

Введите карту: 6b

Стрит-флеш

Введите карту: 8с

Введите карту: ас

Введите карту: 8с

Дубликат карты; игнорируются.

Введите карту: 7с

Введите карту: объявление

Введите карту: 3h

Пара

Введите карту: 6 с

Введите карту: d2

Неправильная карта; игнорируются.

Введите карту: 2d

Введите карту: 9с

Введите карту: 4h

Введите карту: ts

Высшая карта

Введите карту:0

Из этого описания программы мы видим, что она имеет три задачи:

Прочитать покерную комбинацию из пяти карт.

Проанализировать покерную комбинацию на предмет пар,

стрейт и тому подобное.

Распечатать классификацию покерной комбинации.

Мы разделим программу на три функции: `read_cards`, `analyze_hand` и `print_result`, которые выполняют эти три задачи, `main` ничего не делает, но вызывает эти функции внутри бесконечного цикла.

Функциям требуется передать достаточно большой объем информации, поэтому нам придется с ними общаться через внешние переменные.

`read_cards` будет хранить информацию о комбинации карт в нескольких внешних переменных.

Затем `analyze_hand` исследует эти переменные, сохраняя их результаты в других внешних переменных для передачи в `print_result`.

Основываясь на этих предварительных соображениях, мы можем выполнить набросок плана программы.

```
/* Директивы #include здесь */
/* Директивы # define здесь */
/* Объявления внешних переменных идут сюда */
/* Прототипы */
void read_cards(void);
void analyze_hand (void);
void print_result (void);
/*****
* main: вызывает read_cards, analyze_hand и print_result *
* неоднократно *
```

```

*****/

```

```

int main (void){
    for (;;)
        read_cards ();
        analyze_hand();
        print_result ();
}

```

```

/*****

```

```

*read_cards: считывает карты во внешние переменные; *
* проверяет наличие плохих карт и дубликатов карт.    *

```

```

*****/

```

```

void read_cards (void){

    ...

}

```

```

/*****

```

```

analyze_hand: определяет, содержит ли комбинация карт
"стрит"( подобранные подряд по достоинству, "порядок"),
флеш (в покере - все или несколько карт одной масти)
, четыре карты одного достоинства (четыре короля и т. п.)
и / или тройка карт одинакового достоинства; определяет
количество пар, сохраняет результаты во внешних
переменных.

```

```

*****/

```

```
void analyze_hand (void){
```

```
    ...
```

```
}
```

```
/******
```

print_result: уведомляет пользователя о результате, используя набор внешних переменных: от analyze_hand.

```
*****/
```

```
void print_result (void){
```

```
    ...
```

```
}
```

Самый актуальный вопрос, который остаётся, - как изобразить комбинацию карт. Давайте посмотрим, какие операции read_cards и analyze_hand выполняют на комбинациях. Во время анализа комбинации analyze_hand нужно будет узнать, сколько карт каждого ранга и каждой масти. Это позволяет предположить, что мы используем два массива num_in_rank и num_in_suit. Значение num_in_rank [r] будет количеством карт ранга r, а значение num_in_suit [s] будет числом карт масти s. (Мы закодируем ранги числами от 0 до 12 и масти числами от 0 до 3). Нам также понадобится третий массив card_existe, чтобы read_cards мог обнаруживать дубликаты карт.

Каждый раз, когда `read_cards` считывает карту с рангом *r* и мастью *s*, она проверяет, истинно ли значение `card_exists [r] [s]`. Если это так, карта была введена ранее; если нет, `read_cards` присваивает значение `true` для `card_exists [r] [s]`. Как функция `read_cards`, так и функция `analyze_hand` должны иметь доступ к массивам `num_in_rank` и `num_in_suit`, поэтому я сделаю их внешними переменными.

Массив `card_exists` используется только с помощью `read_cards`, поэтому он может быть локальным для этой функции. Как правило, переменные должны быть внешними только в случае необходимости. Решив основные структуры данных, мы можем теперь закончить программу:

```
//poker.c
/* Classifies a poker hand */

#include <stdbool.h> /* C99 only */
#include <stdio.h>
#include <stdlib.h>

#define NUM_RANKS 13
#define NUM_SUITS 4
#define NUM_CARDS 5

/* внешние переменные */
int num_in_rank[NUM_RANKS];
int num_in_suit[NUM_SUITS];
```



```
bool straight, flush, four, three;
int pairs; /* can be 0, 1, or 2 */
```

```
/* прототипы */
```

```
void read_cards(void);
void analyze_hand(void);
void print_result(void);
```

```
/
```

```
*****
```

```
main: Вызывает read_cards, analyze_hand, и print_result
неоднократно. *
```

```
*****
```

```
/
```

```
int main(void) {
    for (;;) {
        read_cards();
        analyze_hand();
        print_result();
    }
}
```

```
/
```

```
*****
```

read_cards: считывает карты во внешние переменные num_in_rank и num_in_suit;

выявляет неправильно введенные карты и дубликаты.

/

```
void read_cards(void){
    bool card_exists[NUM_RANKS][NUM_SUITS];
    char ch, rank_ch, suit_ch;
    int rank, suit;
    bool bad_card;
    int cards_read = 0;

    for (rank = 0; rank < NUM_RANKS; rank++) {
        num_in_rank[rank] = 0;
        for (suit = 0; suit < NUM_SUITS; suit++)
            card_exists[rank][suit] = false;
    }

    for (suit = 0; suit < NUM_SUITS; suit++)
        num_in_suit[suit] = 0;

    while (cards_read < NUM_CARDS) {
        bad_card = false;
```

```
printf("Enter a card: ");

rank_ch = getchar();
switch (rank_ch) {
case '0':    exit(EXIT_SUCCESS);
case '2':    rank = 0; break;
case '3':    rank = 1; break;
case '4':    rank = 2; break;
case '5':    rank = 3; break;
case '6':    rank = 4; break;
case '7':    rank = 5; break;
case '8':    rank = 6; break;
case '9':    rank = 7; break;
case 't': case 'T': rank = 8; break;
case 'j': case 'J': rank = 9; break;
case 'q': case 'Q': rank = 10; break;
case 'k': case 'K': rank = 11; break;
case 'a': case 'A': rank = 12; break;
default:     bad_card = true;
}

suit_ch = getchar();
switch (suit_ch) {
case 'c': case 'C': suit = 0; break;
```

```

case 'd': case 'D': suit = 1; break;
case 'h': case 'H': suit = 2; break;
case 's': case 'S': suit = 3; break;
default:    bad_card = true;
}

```

```

while ((ch = getchar()) != '\n')
    if (ch != ' ') bad_card = true;

```

```

if (bad_card)
    printf("Bad card; ignored.\n");
else if (card_exists[rank][suit])
    printf("Duplicate card; ignored.\n");
else {
    num_in_rank[rank]++;
    num_in_suit[suit]++;
    card_exists[rank][suit] = true;
    cards_read++;
}

```

```

}
}

```

```

/

```

```

*****
*****

```

```

* analyze_hand: Determines whether the hand contains a *
*   straight, a flush, four-of-a-kind, *
*   and/or three-of-a-kind; determines the *
*   number of pairs; stores the results into *
*   the external variables straight, flush, *
*   four, three, and pairs. *

```

```

*****

```

```

*****/

```

```

void analyze_hand(void){
    int num_consec = 0;
    int rank, suit;

    straight = false;
    flush = false;
    four = false;
    three = false;
    pairs = 0;

    /* check for flush */
    for (suit = 0; suit < NUM_SUITS; suit++)
        if (num_in_suit[suit] == NUM_CARDS)
            flush = true;

    /* check for straight */

```

```

rank = 0;
while (num_in_rank[rank] == 0) rank++;
    for (; rank < NUM_RANKS && num_in_rank[rank] >
        0; rank++)
        num_consec++;
if (num_consec == NUM_CARDS) {
    straight = true;
    return;
}

/* check for 4-of-a-kind, 3-of-a-kind, and pairs */
for (rank = 0; rank < NUM_RANKS; rank++) {
    if (num_in_rank[rank] == 4) four = true;
    if (num_in_rank[rank] == 3) three = true;
    if (num_in_rank[rank] == 2) pairs++;
}
}

/
*****
*****

* print_result: Prints the classification of the hand, *
*   based on the values of the external   *
*   variables straight, flush, four, three, *
*   and pairs.           *
```

```
*****
*****/
```

```
void print_result(void){
    if (straight && flush) printf("Straight flush");
    else if (four)    printf("Four of a kind");
    else if (three && pairs == 1) printf("Full house");
    else if (flush)   printf("Flush");
    else if (straight) printf("Straight");
    else if (three)   printf("Three of a kind");
    else if (pairs == 2) printf("Two pairs");
    else if (pairs == 1) printf("Pair");
    else              printf("High card");

    printf("\n\n");
}
```

Обратите внимание на использование функции `exit` в `read_cards` (в случае "0" первой инструкции `switch`). `exit` удобна для этой программы из-за её способности прекратить выполнение из любой точки программы.

Вопросы и ответы

Вопрос: Какое влияние оказывают локальные переменные со статическим временем хранения на рекурсивные функции?

Ответ: Когда функция вызывается рекурсивно, для каждого вызова создаются новые копии её автоматических переменных. Однако это не происходит для статических переменных. Вместо этого все вызовы функции совместно используют одни и те же статические переменные.

Вопрос: В следующем примере *j* инициализируется тем же значением, что и *i*, но есть две переменные с именем *i*:

```
int i = 1;
void f(void){
    int j= i;
    int i=2;
    ...
}
```

Является ли этот код допустимым? Если да, то каково исходное значение *j*, 1 или 2?

Ответ: Код действительно допустим. Область видимости локальной переменной не начинается до её объявления. Следовательно, объявление *j* относится к внешней переменной с именем *i*. Начальное значение *j* будет равно 1.

Упражнения

К разделу 10.4

W1. Следующая схема программы показывает только определения функций и объявления переменных.

```
int a;  
void f(int b){  
    int c;  
}
```

```
void g(void){  
    int d;  
    {  
        int e;  
    }  
}
```

```
int main(void){  
    int f;  
}
```

Для каждой из перечисленных областей перечислите все имена переменных и параметров, видимые в этой области:

- (a) Функция f
- (b) Функция g

- (с) Блок, в котором объявляется e
- (d) Функция main

2. Следующая схема программы показывает только определения функций и объявления переменных.

```
int b, c ;  
void f (void){  
    int b, d;  
}  
  
void g(int a){  
    int c;  
    {  
        int a,d;  
    }  
}  
  
int main(void) {  
    int c, d;  
}
```

Для каждой из следующих областей, перечислите все имена переменных и параметров, видимые в этой области. Если

имеется более одной переменной или параметра с тем же именем, укажите, какая из них видима.

- (a) Функция *f*
- (b) Функция *g*
- (c) Блок, в котором объявлены *a* и *d*
- (d) Функция *main*

* 3. Предположим, что программа имеет только одну функцию (*main*). Сколько различных переменных с именем *i* может содержаться в этой программе?

Проекты по программированию

1. Измените пример стёка в Разделе 10.2, чтобы он хранил символы вместо целых чисел. Далее добавьте функцию *main*, которая просит пользователя ввести ряд круглых скобок и / или фигурных скобок, а затем указывает, правильно ли они вложены:

Введите круглые скобки и / или фигурные скобки: `() {} {()}}`

Круглые скобки и фигурные скобки вложены правильно.

Подсказка: по мере того, как программа читает символы, следует добавлять в стёк каждую левую круглую или фигурную скобку. При считывании правой круглой или фигурной скобки, выталкивается из стёка и проверяется, является ли извлекаемый элемент, соответствующей круглой или фигурной скобкой. (Если нет, круглые / фигурные скобки не вложены должным образом.) Когда программа считывает символ новой строки проверьте, является ли стёк пустым;

Если это так, круглые / фигурные скобки соответствуют друг другу, если стёк не пуст (или когда `stack_underflow` когда-либо вызывался), круглые / фигурные скобки не совпадают. Если вызывается `stack_overflow`, программа распечатывает сообщение - Переполнение стёка - и немедленно завершает работу.

2. Измените программу `roker.c` в Разделе 10.5, переместив массивы `num_in_rank` и `num_in_suit` в `main`, которые передадут их как аргументы `read_cards` и `analyze_hand`.

W3. Удалите массивы `num_in_rank`, `num_in_suit` и `card_exists` из программы `roker.c` раздела 10.5. Попросите программу хранить карты в массиве размером 5 x 2. Каждая строка массива будет представлять карту. Например, если массив называется `hand`, тогда `hand [0] [0]` сохранит ранг первой карты, а `hand [0] [1]` сохранит масть первой карты.

4. Измените программу `roker.c` в Разделе 10.5, узнав дополнительную категорию "королевский флэш"

("royal flush"- туз, король, дама, валет, десять одной и той же масти). Королевский флэш по рангу выше всех остальных.

W5. Измените программу `roker.c` в Разделе 10.5, разрешив "**туз-низший**" стрейт (туз, два, три, четыре, пять).(Стрейт, стрит (в покере; комбинация карт, идущих по старшинству непосредственно одна за другой, не обязательно одной масти)

6. Некоторые калькуляторы (в частности, из Hewlett-Packard) используют систему написания математических выражений, известных как Reverse Polish Notation -RPN (Обратная польская нотация), в этих обозначениях операторы помещаются после их операндов, а не между ними. Например, $1 + 2$ записывается как $1\ 2\ +$ в RPN, а $1 + 2 * 3$ будет записано $1\ 2\ 3\ * \ +$. Выражения RPN могут быть легко вычислены с использованием стёка. Алгоритм включает чтение операторов и операндов в выражении слева направо, выполняя следующие действия:

Когда встречается операнд, задвиньте его в стёк,
Когда встречается оператор, вытащите его операнды из стёка, выполните операцию над этими операндами, а затем задвиньте результат в стёк.

Напишите программу, которая вычисляет выражения RPN. Операнды будут одноцифровыми целыми числами. Операторы $+$, $-$, $*$, $/$, и $=$. Оператор $=$ вызывает отображение верхнего элемента стёка, после чего стёк очищается, и пользователю предлагается ввести другое выражение. Процесс продолжается до тех пор, пока пользователь не вводит символ, который не является оператором или операндом:

Введите выражение RPN: $1\ 2\ 3\ * \ + \ =$
Значение выражения: 7

Введите выражение RPN: $5\ 8\ * \ 4\ 9\ - \ / \ =$
Значение выражения: -8

Введите выражение RPN: q

Если будет переполнение стёка, программа отобразит сообщение - *Выражение слишком сложное* - и завершится. Если стёк выйдет за нижнюю границу (из-за такого выражения, как `1 2 + +`), программа отобразит сообщение - *Недостаточно операндов в выражении* - и завершится. Советы: включить код стёка из раздела 10.2 в вашу программу. Используйте `scanf ("% c", & ch)` для чтения операторов и операндов.

7. Напишите программу, которая запрашивает у пользователя число, а затем отображает число, используя символы для имитации эффекта семи-сегментного дисплея:

Введите число: 4919014



Символы, отличные от цифр, следует игнорировать. Запишите программу так, чтобы максимальное количество цифр контролировалось макросом с именем `MAX_DIGITS`. Который имеет значение 10.

Если число содержит цифр больше, чем это число, дополнительные цифры игнорируются.

Советы. Используйте два внешних массива. Один из них - массив `segments` (см. Упражнение 6 в Главе 8), в котором хранятся данные, представляющие соответствие между цифрами и сегментами. Другой массив, `digits`, будет массивом символов с 3 строками (так как каждая сегментированная цифра имеет три символа) и `MAX_DIGITS * 4` столбца (цифры имеют три символа в ширину, но между цифрами для

чтения требуется пробел).

Напишите свою программу из четырёх функций:

main, clear_digits_array, process_digit и print_digits_array.

Вот прототипы последних трёх функций:

```
void clear_digits_array(void);
```

```
void process_digit (int digit, int position);
```

```
void print_digits_array(void);
```

clear_digits_array будет хранить пустые символы во всех элементах массива digits, process_digit сохранит семисегментное представление цифры в заданное положение в массиве digits (позиции от 0 до MAX_DIGITS - 1).

print_digits_array будет отображать строки массива digits, каждую в одной строке, производя вывод, такой как показан в примере.

11 Указатели

Одиннадцатая заповедь гласит: "Вычисляй" или "не вычисляй" — я уже не помню.

Указатели являются одним из наиболее важных и наиболее часто неправильно понимаемых инструментов Си. Из-за их важности мы посвятим указателям три главы. В этой главе мы сосредоточимся на основах; Главы 12 и 17 охватывают более сложные применения указателей. Мы начнём с обсуждения адресов памяти и их связи с переменными типа "указатель" (Раздел 11.1). Раздел 11.2 вводит операторы взятия адреса и разыменования. Раздел 11.3 охватывает присваивание указателя. В Разделе 11.4 объясняется, как передавать указатели функции, в то время как в Разделе 11.5 обсуждаются возвращение указателей из функций.

11.1 Переменные-указатели

Первым шагом в понимании указателей является визуализация того, что они представляют на уровне машины. В большинстве современных компьютеров основная память разделяется на байты, причём каждый байт способен хранить восемь бит информации:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Каждый байт имеет уникальный адрес, чтобы отличить его от других байтов в памяти. Если в памяти имеется n байтов, мы можем рассматривать адреса как числа, которые варьируются

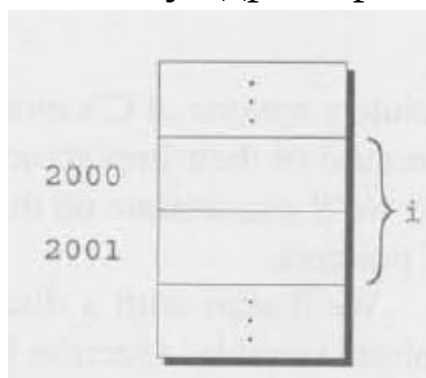
541

от 0 до $n - 1$ (см. рисунок). Исполняемая программа состоит из кода (машинных инструкций, соответствующих инструкциям в исходной программе Си) и данных (переменных в исходной программе). Каждая переменная в программе занимает один или несколько байтов памяти:

адрес содержимое

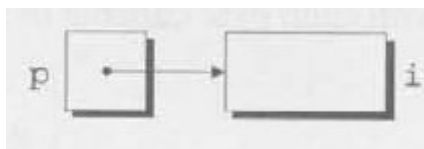
0	01010011
1	01110101
2	01110011
3	01100001
4	01101110
	⋮
n-1	01000011

Адрес первого байта называется адресом переменной. На следующем рисунке переменная i занимает байты по адресам 2000 и 2001, поэтому адрес i равен 2000:



Здесь вступают в дело указатели. Хотя адреса представлены числами, их диапазон значений может отличаться от значений

целых чисел, поэтому мы не можем хранить их в обычных целочисленных переменных. Однако мы можем хранить их в специальных переменных-указателях. Когда мы храним адрес переменной ***i*** в переменной-указателе ***p***, мы говорим, что ***p*** "указывает на" ***i***. Другими словами, указатель - это не что иное, как адрес, а переменная-указатель - это просто переменная, которая может хранить адрес. Вместо того, чтобы показывать адреса как числа в наших примерах, я использую более простые обозначения. Чтобы указать, что переменная-указатель ***p*** хранит адрес переменной ***i***, я покажу содержимое ***p*** как стрелку, направленную к ***i***:



Объявление переменных-указателей

Переменная-указатель объявляется так же, как обычная переменная. Единственное различие заключается в том, что перед именем переменной-указателя должна быть звездочка: `int *p;` (Годится любое, на выбор, обозначение `int *p;` `int * p;` `int* p;` `int*p;`)

В этом объявлении указано, что ***p*** - это переменная-указатель, способная указывать на объект типа `int`. Я использую термин объект вместо переменная, поскольку, как мы увидим в главе 17 ***p*** может указывать на область памяти, которая не принадлежит переменной.

(Имейте в виду, что "объект" будет иметь другое значение, когда мы обсудим проектирование программы в Главе 19.) Переменные указателя могут появляться в объявлениях вместе с другими переменными:

```
int i, j, a[10], b[20], *p, *q;
```

В этом примере *i* и *j* являются обычными целыми переменными, *a* и *b* являются массивами целых чисел, а *p* и *q* являются указателями на целые объекты. Си требует, чтобы каждая переменная-указатель указывала только на объекты определённого типа (ссылочный тип):

```
int *p; /* Указывает только на целые числа */  
double *q; /* Указывает только на числа типа double */  
char *r; /* Указывает только на символы */
```

Нет никаких ограничений на то каким должен быть ссылочный тип. Фактически, переменная-указатель может указывать даже на другой указатель.

11.2 Операторы взятия адреса и разыменования

Си предоставляет пару операторов, специально предназначенных для использования с указателями. Чтобы найти адрес переменной, мы используем оператор **&** (взятие адреса). Если *x* - переменная, то **&x** - адрес *x* в памяти. Чтобы получить доступ к объекту, на который указывает указатель, мы используем оператор ***** (разыменования). Если *p* является указателем, то ***p** представляет объект, на который в данный момент указывает *p*. (У звёздочки три значения:

умножение $a*b$, разыменование (оно же косвенное обращение) $*p$ и определение указателя

`int *p;)`

Оператор адреса (взятия адреса)

Объявление переменной-указателя выделяет место для указателя, но не указывает на объект:

`int *p; /* конкретно ни на что не указывает */`

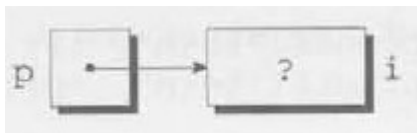
Крайне важно инициализировать ***p***, прежде чем мы её используем. Один из способов инициализации переменной-указателя - назначить ей адрес некоторой переменной или, в более общем смысле lvalue - с помощью оператора `&`:

`int i, *p;`

....

`p = &i;`

присваивая адрес ***i*** переменной ***p***, эта инструкция делает ***p*** указателем на ***i***:



Также можно инициализировать переменную-указатель во время её объявления:

`int i ;`

```
int *p = &i;
```

Оператор разыменования

Когда переменная-указатель указывает на объект, мы можем использовать оператор `*` (разыменования) для доступа к тому, что хранится в объекте.

Если ***p*** указывает на ***i***, например, мы можем напечатать значение ***i*** следующим образом:

```
printf("%d\n",*p);
```

`printf` отобразит значение ***i***, а не адрес ***i***. Склонный к математике читатель может думать о `*` как об инверсии `&`.

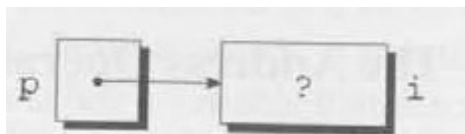
Применение `&` к переменной создаёт указатель на переменную; применение `*` к указателю возвращает нас к исходной переменной:

```
j = *&i; /* тоже что и j = i; */
```

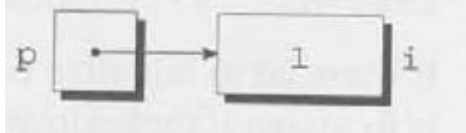
Пока ***p*** указывает на ***i***, `*p` является псевдонимом для ***i***. Мало того, что `*p` имеет то же значение, что и ***i***, но изменение значения `*p` также изменяет значение ***i***. (`*p` является lvalue, поэтому присваивание ему является допустимым.)

Следующий пример иллюстрирует эквивалентность `*p` и ***i***; Диаграммы показывают значения ***p*** и ***i*** в разных точках вычисления.

`p=&i;`



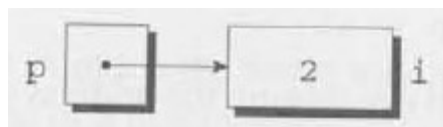
`i=1;`



`printf("%d\n", i); /* печатает 1 */`

`printf("%d\n", *p); /* печатает 1 */`

`*p=2;`



`printf("%d\n", i); /* печатает 2 */`

`printf("%d\n", *p); /* печатает 2 */`

Никогда не применяйте оператор разыменования к неинициализированной переменной-указателю. Если переменная-указатель ***p*** не была инициализирована, попытка использовать значение ***p*** в любом случае вызывает неопределённое поведение. В следующем примере вызов `printf` может напечатать мусор, привести к сбою программы или получить какой-то другой эффект:

`int *p;`

`printf("%d", *p); /***WRONG ***/`

Присваивание значения **p* особенно опасно. Если *p* содержит допустимый адрес памяти, следующее присваивание будет пытаться изменить данные, хранящиеся по этому адресу:

```
int *p;
*p = 1; /*** WRONG ***/
```

Если объект модифицированный этим присваиванием принадлежит программе, она может повести себя хаотично; если он принадлежит операционной системе, программа, скорее всего, потерпит крах. Ваш компилятор может выдать предупреждение о том, что *p* не инициализирован, поэтому обратите пристальное внимание на любые предупреждающие сообщения, которые вы получаете.

Присваивание указателя

Си позволяет использовать оператор присваивания для копирования указателей при условии, что они имеют один и тот же тип. Предположим, что *i*, *j*, *p* и *q* объявлены следующим образом:

```
int i, j, *p, *q;
```

инструкция

```
p = &i;
```

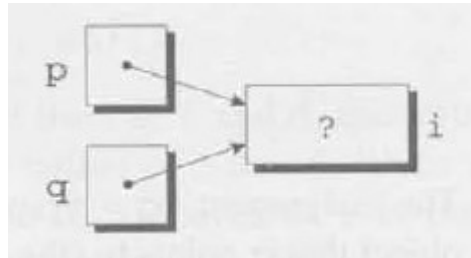
является примером присваивания указателя; адрес *i* копируется в *p*.

Вот ещё один пример присваивания указателя:

```
q = p;
```

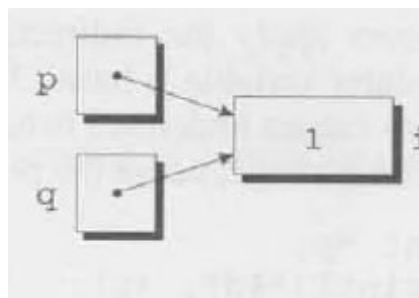
548

Эта инструкция копирует содержимое p (адрес i) в q , фактически делая q указывающим на то же место, что и p :

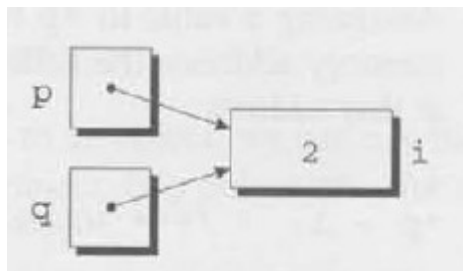


И p , и q теперь указывают на i , поэтому мы можем изменить i , присвоив новое значение либо $*p$, либо $*q$:

$*p = 1;$



$*q = 2;$



Любое количество переменных-указателей может указывать на один и тот же объект. Будьте осторожны, чтобы не путать

549

$q = p;$

c

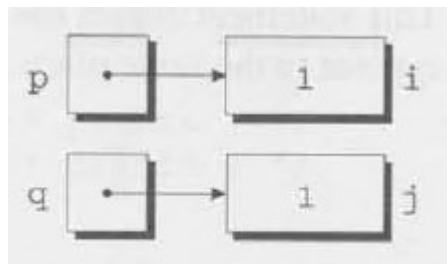
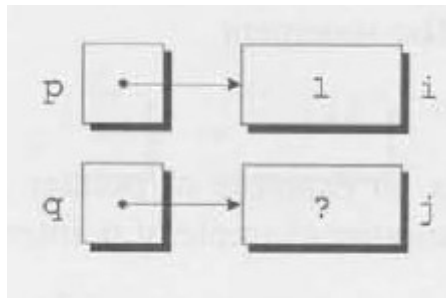
$*q = *p;$

Первая инструкция - присваивание указателя; вторая - нет, как показано в следующем примере:

$p = \&i;$

$q = \&j;$

$i = 1;$



$*q = *p;$

Присваивание $*q = *p$ копирует значение, на которое указывает p (значение i) в объект, на который указывает q (переменная j).

(при этом $q \neq p$)

11.4 Указатели в качестве аргументов

До сих пор нам удавалось избежать довольно важного вопроса: для чего нужны указатели? На этот вопрос нет единого ответа, так как указатели имеют несколько различных применений в Си. В этом разделе мы увидим, как указатель на переменную может быть полезен в качестве аргумента функции. Мы найдём другие применения указателей в Разделе 11.5 и в главах 12 и 17. В Разделе 9.3 мы видели, что переменная, предоставленная в качестве аргумента в вызове функции, защищена от изменения, потому что Си передаёт аргументы по значению. Это свойство Си может быть неприятным, если мы хотим, чтобы функция могла изменять переменную. В Разделе 9.3 мы попытались и не смогли написать функцию `decompose`, которая могла бы изменить два своих аргумента.

Указатели предлагают решение этой проблемы: вместо того, чтобы передать переменную `x` как параметр функции, мы предоставим `&x` указатель на `x`. Мы объявим соответствующий параметр `p` как указатель. Когда функция вызывается, `p` будет иметь значение `&x`, поэтому `*p` (объект, на который `p` указывает) будет псевдонимом для `x`.

Каждое появление `*p` в теле функции будет косвенной ссылкой на `x`, позволяя функции как читать `x`, так и изменять её. Чтобы увидеть этот метод в действии, давайте изменим функцию `decompose`, объявив параметры `int_part` и `frac_part` - указателями. Определение `decompose` теперь будет выглядеть так:

```
void decompose(double x, long *int_part, double *frac_part){
```

551

```
*int_part = (long) x;  
*frac_part = x - *int_part;  
}
```

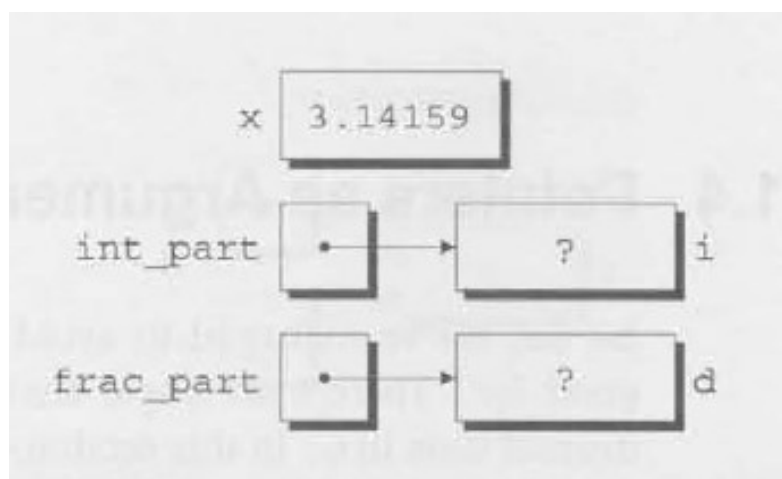
Прототипом для `decompose` может быть либо
`void decompose(double x, long *int_part, double *frac_part);`
или

`void decompose (double, long *, double *);`

Мы будем вызывать `decompose` следующим образом:

```
decompose(3.14159, &i, &d);
```

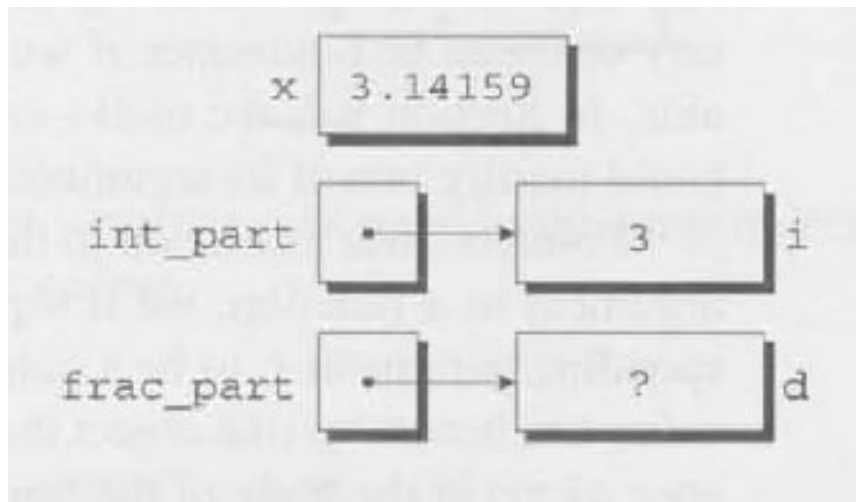
Из-за оператора `&` перед ***i*** и ***d*** аргументы для `decompose` являются указателями на ***i*** и ***d***, а не значениями ***i*** и ***d***. Когда вызывается `decompose`, значение 3.14159 копируется в ***x***, указатель на ***i*** сохраняется в `int_part`, а указатель на ***d*** сохраняется в `frac_part`:



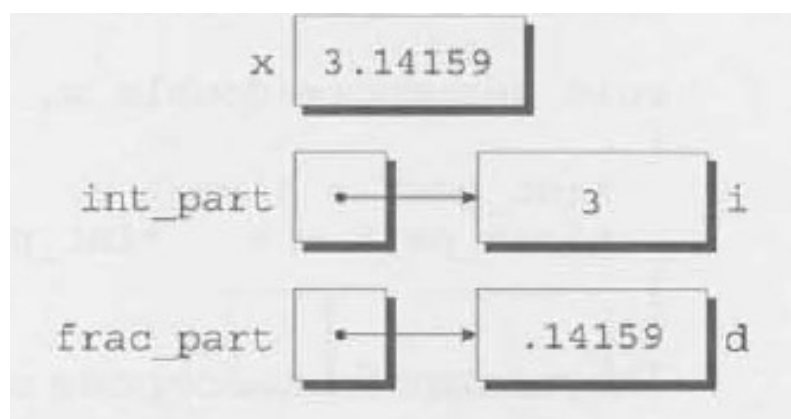
Первое присваивание в теле `decompose` преобразует значение ***x*** в тип `long` и сохраняет его в объекте, на который указывает

552

int_part. Поскольку int_part указывает на *i*, присваивание помещает значение 3 в *i*:



Второе присваивание извлекает значение, на которое указывает int_part (значение *i*), которое равно 3. Это значение преобразуется в тип double и вычитается из *x*, давая .14159, которое затем сохраняется в объекте, на который указывает frac_part:



Когда `decompose` возвращается, ***i*** и ***d*** будут иметь значения 3 и .1459, что и требовалось.

В использовании указателей в качестве аргументов для функций на самом деле нет ничего нового; мы делали это в вызовах `scanf` начиная с Главы 2. Рассмотрим следующий пример:

```
int i;  
....  
scanf("%d", &i);
```

Мы должны поставить оператор `&` перед ***i***, так что `scanf` получает указатель на ***i***; Этот указатель сообщает `scanf`, где нужно поместить значение, которое она читает. Без `&`, `scanf` был бы предоставлен значением ***i***. Хотя аргументы `scanf` должны быть указателями, не всегда верно, что каждому аргументу нужен оператор `&`. В следующем примере `scanf` передаётся переменная-указатель:

```
int i, *p;  
...  
p = &i;  
scanf("%d", p);
```

Поскольку ***p*** содержит адрес ***i***, `scanf` будет читать целое число и хранить его в ***i***. Использование оператора `&` в вызове будет неправильным:

```
scanf("%d", &p); /*** WRONG ***/
```

scanf будет читать целое число и хранить его в *p* вместо *i*. Неспособность передать функции указатель, когда он ожидается, может иметь катастрофические результаты. Предположим, что мы вызываем `decompose` без оператора `&` перед *i* и *d*:

```
decompose(3.14159, i, d);
```

`decompose` ожидает, что вторым и третьим аргументом будут указатели, но вместо этого им присваиваются значения *i* и *d*. `decompose` не имеет возможности проверить, поэтому она будет использовать значения *i* и *d*, как если бы они были указателями. Когда `decompose` сохранит значения в `*int_part` и `*frac_part`, она будет пытаться изменить неизвестные ячейки памяти вместо изменения *i* и *d*. Если мы предоставили прототип для `decompose` (что, конечно, мы всегда должны делать), компилятор сообщит нам, что мы пытаемся передать аргументы неправильного типа. В случае `scanf`, однако, передача указателей часто игнорируется компилятором, что делает `scanf` особенно уязвимой функцией.

ПРОГРАММА Поиск наибольших и наименьших элементов в массиве

Чтобы проиллюстрировать, как передаются указатели функции, давайте рассмотрим функцию с именем `max_min`, которая находит самые большие и самые меньшие элементы в

массиве. Когда мы вызовем `max_min`, мы передадим ей указатели на две переменные; `max_min` затем сохранит свои ответы в этих переменных.

`max_min` имеет следующий прототип:

```
void max_min(int a[],int n, int *max, int *min);
```

Вызов `max_min` может иметь следующий вид:

```
max_min(b,N, &big, &small);
```

b - массив целых чисел; ***N*** - количество элементов в ***b***, ***big*** и ***small*** - обычные целочисленные переменные. Когда `max_min` устанавливает наибольший элемент в ***b***, она сохраняет значение в ***big***, присваивая его `* max`. (Поскольку `max` указывает на ***big***, присваивание `* max` будет изменять значение ***big***.)

`max_min` хранит наименьший элемент ***b*** в ***small***, присваивая его `* min`. Чтобы проверить `max_min`, мы напишем программу, которая считывает 10 чисел в массив, передает массив в `max_min` и печатает результаты:

Введите 10 цифр: 34 82 49 102 7 94 23 11 50 31

Наибольшее: 102

Наименьшее: 7

Вот полная программа:

```
// maxmin.c
```

```
/* Нахождение наибольшего и наименьшего элементов  
массива */
```

556

```
#include <stdio.h>
```

```
#define N 10
```

```
void max_min(int a[], int n, int *max, int *min);
```

```
int main(void){
```

```
    int b[N], i, big, small;
```

```
    printf("Введите %d чисел: ", N);
```

```
    for (i = 0; i < N; i++)
```

```
        scanf("%d", &b[i]);
```

```
    max_min(b, N, &big, &small);
```

```
    printf("Наибольшее: %d\n", big);
```

```
    printf("Наименьшее: %d\n", small);
```

```
    return 0;
```

```
}
```

```
void max_min(int a[], int n, int *max, int *min){
```

```
    int i;
```



```
*max = *min = a[0];  
for (i = 1; i < n; i++) {  
    if (a[i] > *max)  
        *max = a[i];  
    else if (a[i] < *min)  
        *min = a[i];  
}  
}
```

Использование const для защиты аргументов

Когда мы вызываем функцию и передаём ей указатель на переменную, мы обычно предполагаем, что функция изменит эту переменную (в противном случае, почему функция должна иметь указатель?) Например, если мы видим инструкцию типа `f(&x);`

в программе мы, вероятно, ожидаем, что *f* изменит значение *x*. Возможно, однако, что *f* просто нужно проверить значение *x*, а не изменять его. Причиной использования указателя может быть эффективность: передача значения переменной может напрасно тратить время и занимаемую память, если переменная требует большого объёма памяти. (Раздел 12.3 более подробно описывает этот вопрос.)

Мы можем использовать слово `const` для подтверждения того, что функция не изменит объект, адрес которого передаётся

функции. `const` идёт в объявлении параметра непосредственно перед определением его типа:

```
void f(const int *p){  
    *p = 0; /*** WRONG ***/  
}
```

Это использование `const` указывает, что *p* является указателем на "числовую целую константу". Попытка изменить `*p` - это ошибка, которую обнаружит компилятор.

11.5 Указатели как возвращаемые значения

Мы можем не только передавать указатели на функции, но и писать функции, которые возвращают указатели. Такие функции встречаются довольно часто; Мы увидим некоторые в Главе 13. Следующая функция при задании указателей на два целых числа возвращает указатель на то, какое целое больше:

```
int *max(int *a, int *b){  
    if (*a > *b)  
        return a;  
    else  
        return b;  
}
```

Когда мы вызовем `max`, мы передадим указатели на две переменные `int` и сохраним результат в переменной-указателе:

```
int *p, i, j;
p = max(&i, &j);
```

Во время вызова `max`, `*a` является псевдонимом для `i`, а `*b` является псевдонимом для `j`. Если `i` имеет большее значение, чем `j`, `max` возвращает адрес `i`; В противном случае он возвращает адрес `j`. После вызова `p` указывает на `i` или `j`.

Хотя функция `max` возвращает один из указателей, переданных ей в качестве аргумента, это не единственная возможность. Функция также может возвращать указатель на внешнюю переменную или на локальную переменную, объявленную как статическую.

Никогда не возвращают указатель на автоматическую локальную переменную:

```
int *f(void){
    int i;
    ...
    return &i;
}
```

Переменная `i` не существует, как только `f` возвращается, поэтому указатель на неё будет недопустимым. Некоторые компиляторы выдают предупреждение, такое как "функция возвращает адрес локальной переменной" в этой ситуации. Указатели могут указывать на элементы массива, а не только на обычные переменные. Если `a` - массив, то `&a[i]` является указателем на `i`-тый элемент массива `a`. Когда функция имеет

аргумент-массив, иногда полезно, чтобы функция возвращала указатель на один из элементов массива. Например, следующая функция возвращает указатель на средний элемент массива ***a***, считая, что ***a*** имеет ***n*** элементов:

```
int *find_middle(int a[], int n){  
    return &a[n/2];  
}
```

В Главе 12 подробно рассматривается взаимосвязь между указателями и массивами.

Вопросы и ответы

*** Вопрос:** Всегда ли указатель совпадает с адресом?

Ответ: Обычно, но не всегда. Рассмотрим компьютер, основная память которого разделена на слова, а не на байты. Слово может содержать 36 бит, 60 бит или некоторое количество бит. Если предположить 36-битные слова, память будет иметь следующий вид:

Address	Contents
0	001010011001010011001010011001010011
1	001110101001110101001110101001110101
2	001110011001110011001110011001110011
3	001100001001100001001100001001100001
4	001101110001101110001101110001101110
	⋮
n-1	001000011001000011001000011001000011

Когда память разделяется на слова, каждое слово имеет адрес. Целое число обычно занимает одно слово, поэтому указатель на целое может быть просто адресом. Однако слово может содержать более одного символа. Например, 36-разрядное слово может содержать шесть 6-битовых символов:

010011	110101	110011	100001	101110	000011
--------	--------	--------	--------	--------	--------

или четыре 9-ти битовых символов:

001010011	001110101	001110011	001100001
-----------	-----------	-----------	-----------

По этой причине указатель на символ может быть сохранен в другой форме, чем другие указатели. Указатель на символ может состоять из адреса (слова, в котором хранится символ), плюс небольшое целое число (позиция символа внутри слова).

На некоторых компьютерах указатели могут быть "смещениями", а не полными адресами. Например, процессоры семейства Intel x86 (используемые на многих персональных компьютерах) могут выполнять программы в нескольких режимах. Самый старый из них, который восходит к процессору 8086 1978 года, называется реальным режимом.

В этом режиме адреса иногда представлены одним 16-разрядным числом (смещением), а иногда и двумя 16-разрядными числами (пара сегмент: смещение). Смещение не является истинным адресом памяти, а ЦПУ должен объединять его со значением сегмента, хранящимся в специальном регистре. Для поддержки реального режима более старые компиляторы Си часто предоставляют два типа указателей: ближние указатели (16-разрядные смещения) и дальние указатели (32-разрядный пары сегмент: смещение). Эти компиляторы обычно резервируют слова почти так же, как нестандартные ключевые слова, которые можно использовать для объявления переменных-указателей.

***Вопрос:** Если указатель может указывать на данные в программе, возможно ли иметь указатель на программный код?

Ответ: Да. Мы рассмотрим указатели на функции в Разделе 17.7.

Вопрос: Мне кажется, что существует несогласованность между объявлением

```
int *p = &i;
```

и инструкцией

```
p = &i;
```

Почему символ `*` не предшествует ***p*** в инструкции, как в объявлении?

Ответ: Источником путаницы является тот факт, что символ `*` может иметь разные значения в Си, в зависимости от контекста, в котором он используется. В объявлении `int *p = & i;`

символ `*` не является оператором разыменования. Вместо этого он помогает указать тип ***p***, информируя компилятор о том, что ***p*** является указателем на `int`. Однако, когда он появляется в инструкции, символ `*` выполняет разыменование (при использовании в качестве унарного оператора).

Инструкция `*p = & i; /*** WRONG ***/`

была бы неправильной, потому что она присваивает адрес `i` объекту, на который указывает ***p***, а не самому ***p***.

Вопрос: Есть ли способ распечатать адрес переменной?

Ответ: Любой указатель, включая адрес переменной, можно отобразить, вызывая функцию `printf` и используя `%p` в качестве спецификации преобразования. Подробности см. В Разделе 22.3.

Вопрос: Следующее объявление вводит в заблуждение:

```
void f (const int *p);
```

Означает ли это, что *f* не может изменить *p*?

Ответ: Нет. В нем говорится, что *f* не может изменить целое число, на которое указывает *p*; Это не мешает *f* изменять *p*.

```
void f(const int *p){
    int j;
    *p= a; /***WRONG ***/
    p = &j; /* legal */
}
```

Так как аргументы передаются по значению (*в том числе и указатели*) то присваивание *p* нового значения заставляет его указать на какое-то другое место, но это не будет иметь никакого эффекта вне функции.

Вопрос: При объявлении параметра типа указателя допустимо ли поставить слово `const` перед именем параметра, как в следующем примере?

```
void f (int * const p);
```

Ответ: Да, хотя эффект не совпадает с тем, когда `const` предшествует типу *p* (`const int * p`). В Разделе 11.4 мы видели, что размещение `const` до типа *p* защищает объект, на который указывает *p*. Помещение `const` после типа *p* защищает *p*:

```
void f(int * const p){
    int j;
    *p = 0; /*legal*/
    p = &j; /*** WRONG ***/)
```



```
}
```

Эта возможность используется не очень часто. Поскольку **p** - это просто копия другого указателя (аргумент при вызове функции), редко встречаются причины защищать его.

ещё большая редкость заключается в необходимости защитить как **p**, так и объект, на который он указывает, что можно сделать, поставив `const` как до, так и после типа **p**:

```
void f(const int * const p){
    int j;
    *p = 0; /***WRONG ***/
    p = aj; /***WRONG ***/
}
```

Упражнения

К разделу 11.2

1. Если **i** - переменная и **p** указывает на **i**, какие из следующих выражений псевдонимы для **i**?

- | | | | |
|--------|---------|--------|---------|
| (a) *p | (c) *&p | (e) *i | (g) *&i |
| (b) &p | (d) &*p | (f) &i | (h) &*i |

К разделу 11.3

W2. Если **i** является переменной `int`, а **p** и **q** являются

указателями на `int`, какое из следующих присваиваний является допустимым?

- | | | |
|-------------------------------|-------------------------------|---------------------------|
| (a) <code>p = i;</code> | (d) <code>p = &q;</code> | (g) <code>p = *q;</code> |
| (b) <code>*p = &q;</code> | (e) <code>p = *&q;</code> | (h) <code>*p = q;</code> |
| (c) <code>&p = q;</code> | (f) <code>p = q;</code> | (i) <code>*p = *q;</code> |

К разделу 11.4.

3. Следующая функция предположительно вычисляет сумму и среднее чисел в массиве `a`, имеющем длину `n`.

`avg` и `sum` указывают на переменные, которые функция должна изменять. К сожалению, функция содержит несколько ошибок; найти и исправить их.

```
void avg_sum(double a[], int n, double *avg, double *sum)
{
    int i sum = 0.0;
    for (i = 0; i < n; i++)
        sum += a[i];
    avg = sum / n;)
}
```

W4. Напишите следующую функцию:

```
void swap (int *p, int *q);
```

Когда передаются адреса двух переменных, `swap` должен обменивать значения переменных:

```
swap (& i, & j); /* Меняет значения i и j */
```

567

5. Напишите следующую функцию:

```
void split_time(long total_sec, int *hr, int *min, int *sec);
```

total_sec - это время, представленное как количество секунд с полуночи.

hr, min и sec, указатели на переменные, в которых функция будет хранить эквивалентное время в часах (0-23), минутах (0-59) и секундах (0-59) соответственно.

W6. Напишите следующую функцию:

```
void find_two_largest(int a[], int n, int *largest, int  
*second_largest);
```

Когда передаётся массив a длины n, функция будет искать в a свои самый большой и второй за ним по величине элемент, сохраняя их в переменных, на которые указывают largest и second_largest, соответственно.

7. Напишите следующую функцию:

```
void split_date(int day_of_year, int year, int *month, int *day);
```

day_of_year - целое число от 1 до 366, указывающее определённый день года, обозначенного year.

month и day указывают на переменные, в которых функция будет хранить эквивалентный месяц (1-12) и день в течение этого месяца (1-31).

К разделу 11.5

7. Напишите следующую функцию:

```
int *find_largest(int a[], int n);
```

При передаче массива *a* длины *n*, функция вернёт указатель на самый большой элемент массива.

Проекты по программированию.

1. Измените проект 7 проекта из Главы 2, чтобы он включал следующую функцию:

```
void pay_amount(int dollars, int *twenties, int *tens, int *fives, int *ones);
```

Функция определяет наименьшее количество купюр в размере \$ 20, \$ 10, \$ 5 и \$ 1, необходимых для оплаты суммы, представленной параметром *dollars*.

Параметр *twenties* указывает на переменную, в которой функция будет хранить необходимое количество 20 долларовых купюр. Параметры *tens*, *fives* и *ones* аналогичны.

2. Измените Проект по программированию 8 из Главы 5, чтобы он включал следующую функцию:

```
void find_closest_flight(int desired_time, int *departure_time, int *arrival_time);
```

Эта функция находит рейс, время отправления которого ближе всего к желаемому времени (выраженное в минутах с полуночи).

Она сохранит время отправления и прибытия этого рейса (также выраженное в минутах от полуночи) в переменных, на которые указывают *departure_time* и *arrival_time*, соответственно.

3. Измените Проект по программированию 3 из Главы 6, чтобы он включал следующую функцию:

```
void reduce(int numerator, int denominator, int  
*reduced_numerator, int *reduced_denominator);
```

numerator и denominator - числитель и знаменатель дроби.
reduced_numerator и reduced_denominator - указатели на переменные, в которых функция будет хранить числитель и знаменатель дроби, как только она будет приведена к несократимому виду.

4. Измените программу poker.c в Разделе 10.5, перемещая все внешние переменные в main и изменяя функции, чтобы они обменивались данными путём передачи аргументов.

Функция analyze_hand должна изменять переменные straight, flush, four, three и pairs, поэтому им нужно будет передать указатели на эти переменные.

12 Указатели и массивы

Оптимизация препятствует эволюции.

В Главе 11 были введены указатели и показано, как они используются в качестве аргументов функции и в качестве значений, возвращаемых функциями. В этой главе рассматривается другое приложение указателей. Когда

указатели указывают на элементы массива. Си позволяет нам выполнять арифметическое сложение и вычитание на указателях, что приводит к альтернативному способу обработки массивов, в которых указатели заменяют индексы массива. Связь между указателями и массивами в Си является близкой и это мы скоро увидим. Мы будем использовать эту связь в последующих главах, включая главу 13 (Строки) и главу 17 (Расширенное использование указателей).

Понимание связи между указателями и массивами имеет решающее значение для овладения языком Си: оно даст вам понять, как был разработан Си и поможет понять существующие программы. Однако имейте в виду, что одна из основных причин использования указателей для обработки массивов - эффективность - уже не так важна, как когда-то, благодаря улучшенным компиляторам. В Разделе 12.1 обсуждается арифметика указателей и показано, как можно сравнивать указатели с использованием операторов равенства и отношения. В Разделе 12.2 показано, что мы можем использовать арифметику указателей для обработки элементов массива. В Разделе 12.3 раскрывается ключевой факт о массивах - имя массива может служить указателем на первый элемент массива - и использование его, чтобы показать, как действительно работают аргументы-массивы. В Разделе 12.4 показано, как темы первых трёх разделов применяются к многомерным массивам. Раздел 12.5 завершает работу главы, исследуя взаимосвязь между

указателями и массивами переменной длины, функциональным свойством C99.

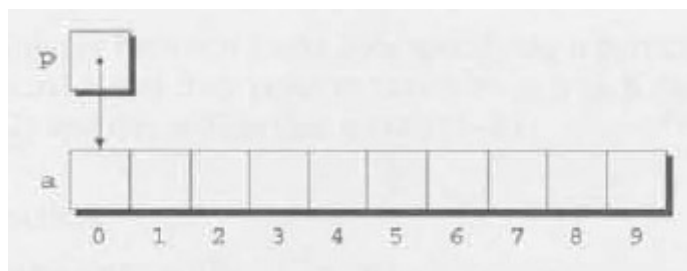
12.1 Арифметика указателей

В Разделе 11.5 мы видели, что указатели могут указывать на элементы массива. Например, предположим, что **a** и **p** были объявлены следующим образом:

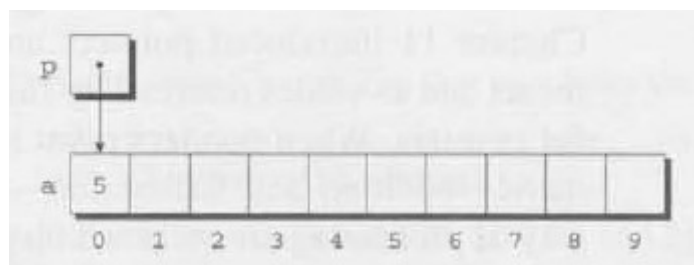
```
int a [10], * p;
```

Мы можем сделать **p** указывающим на **a[0]**, написав **p = & a [0]**;

Графически, вот что мы только что сделали:



Теперь мы можем получить доступ к **a[0]** через **p**; например, мы можем сохранить значение 5 в **a[0]**, написав *** p = 5**. Вот наша картина сейчас:



Создание указателя ***p***, указывающего на элемент массива ***a***, не особенно интересно.

Однако, выполняя арифметику указателя (или адресную арифметику) с ***p***, мы можем получить доступ к другим элементам ***a***. Си поддерживает три (и только три) формы арифметики указателя:

Добавление целого числа к указателю

Вычитание целого числа из указателя

Вычитание одного указателя из другого

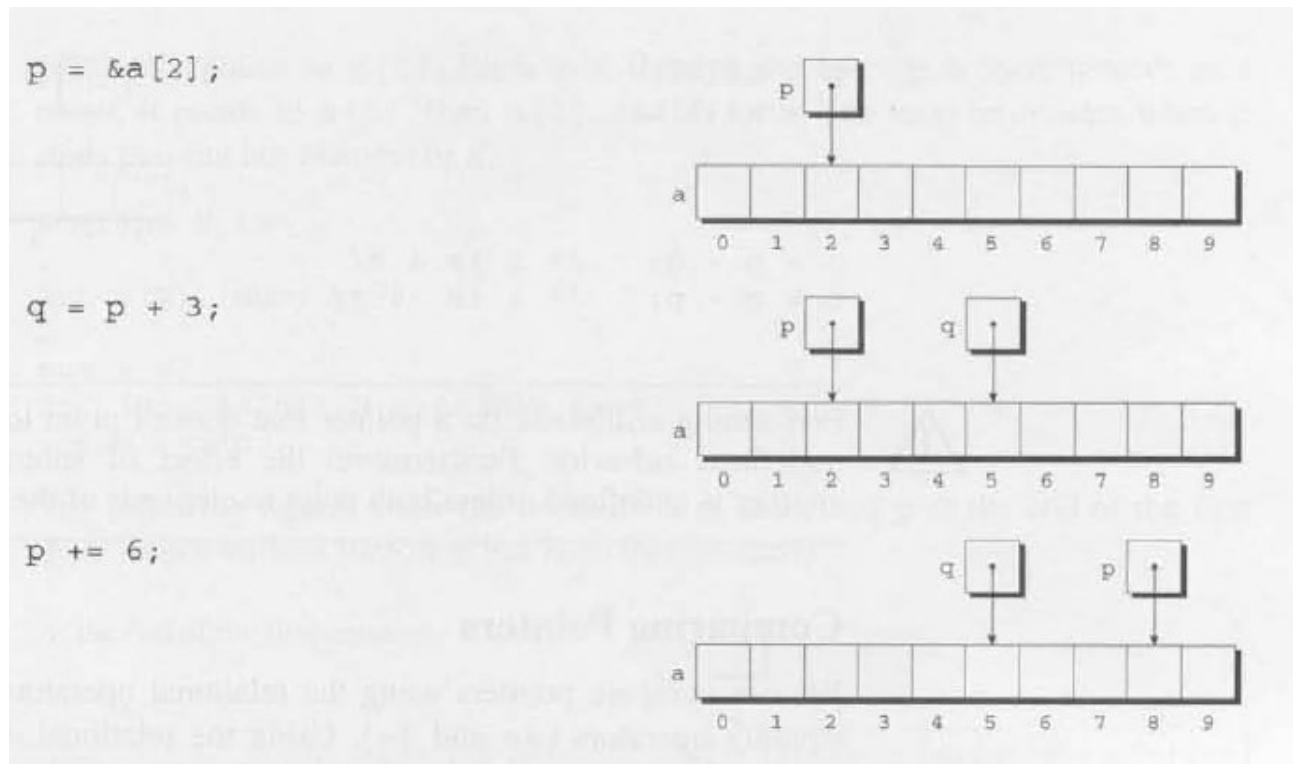
Давайте внимательно рассмотрим каждую из этих операций. Наши примеры предполагают, что действуют следующие объявления:

```
int a [10], * p, * q, i;
```

Прибавление целого числа к указателю

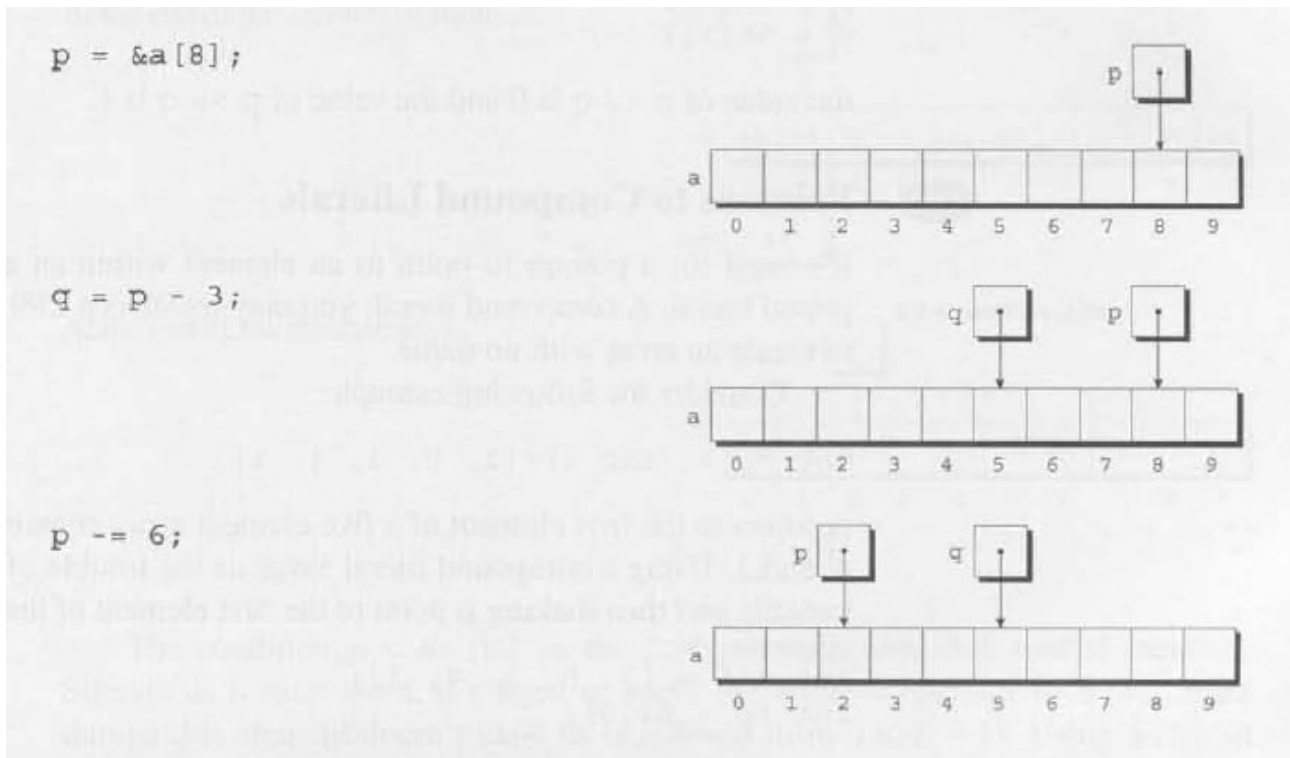
Прибавление целого числа ***j*** к указателю ***p*** даёт указатель на элемент ***j***, расположенный после того, на который указывает ***p***. Точнее, если ***p*** указывает на элемент массива ***a[i]***, то ***p + j*** указывает на ***[i + j]*** (при условии, конечно, что существует ***[i + j]***). Следующий пример иллюстрирует прибавление целого

числа к указателю; диаграммы показывают значения p и q в разных точках вычисления.



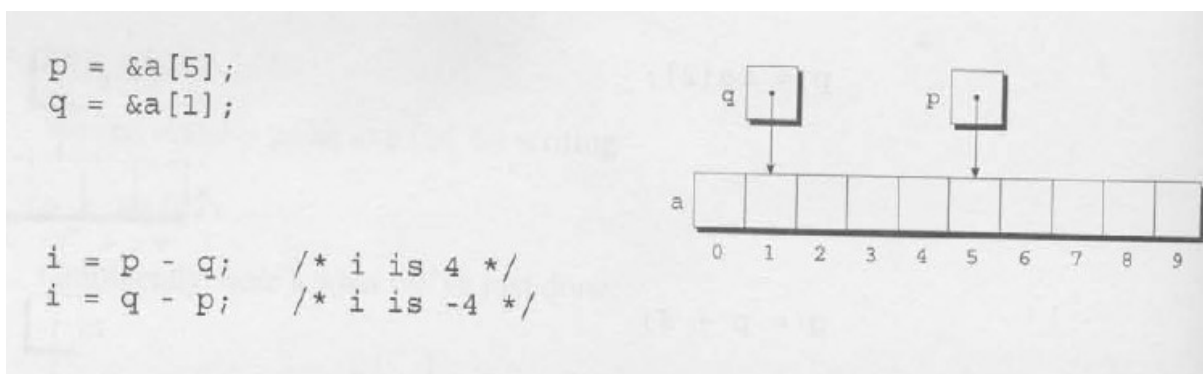
Вычитание целого числа из указателя

Если p указывает на элемент массива $a[i]$, то $p-j$ указывает на $a[i-j]$.



Вычитание одного указателя из другого

Когда один указатель вычитается из другого, результатом является расстояние (измеренное в элементах массива) между указателями. Таким образом, если ***p*** указывает на `a[i]`, а ***q*** указывает на `a[j]`, то `p - q` равно `i - j`. Например:



Выполнение арифметических действий над указателем, не указывающим на элемент массива, приводит к неопределённому поведению.

Кроме того, эффект вычитания одного указателя из другого не определён, если оба не указывают на элементы одного массива.

Сравнение указателей

Мы можем сравнивать указатели, используя реляционные операторы (<, <=, >, >=) и операторы равенства (== и !=).

Использование реляционных операторов для сравнения двух указателей имеет смысл только тогда, когда оба указывают на элементы одного и того же массива.

Результат сравнения зависит от относительного положения двух элементов в массиве. Например, после присвоений

```
p = &a[5];
```

```
q=&a[1];
```

значение $p \leq q$ есть 0, а значение $p \geq q$ есть 1.

Указатели на составные литералы

Указатель может указывать на элемент в массиве, созданном составным литералом.

Составной литерал, как вы помните, является функциональным средством C99, которое можно использовать для создания массива без имени. Рассмотрим следующий пример:

```
int *p = (int []){3, 0, 3, 4, 1};
```

p указывает на первый элемент пятиэлементного массива содержащего целые числа 3, 0, 3, 4 и 1. Использование составного литерала избавляет нас от необходимости сначала объявлять переменную массива, а затем делать ***p*** указывающим на первый элемент этого массива:

```
int a[] = {3,0,3,4,1};
```

```
int *p= &a[0];
```

12.2 Использование указателей для обработки массива

Арифметика указателей позволяет нам пробегать по элементам массива, многократно увеличивая переменную указателя. Следующий фрагмент программы суммирования элементов массива ***a***, иллюстрирует эту технику.

В этом примере переменная указателя ***p*** изначально указывает на ***a[0]***.

После каждого цикла ***p*** увеличивается; в результате он указывает на ***a[1]***, затем на ***a[2]*** и так далее. Цикл завершается, когда ***p*** проходит мимо последнего элемента ***a***.

```
#define N 10
```

```
...
```

```
int a[N], sum, *p;
```

```
...
```

```
sum = 0;
```

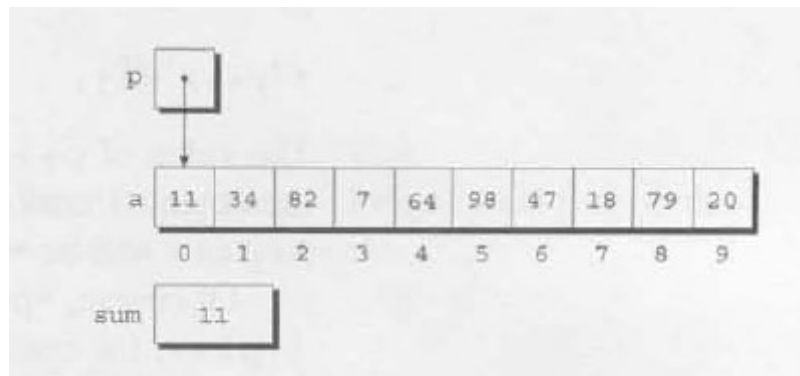
```
for (p=&a[0]; p < &a[N]; p++)
```

```
    sum += *p;
```

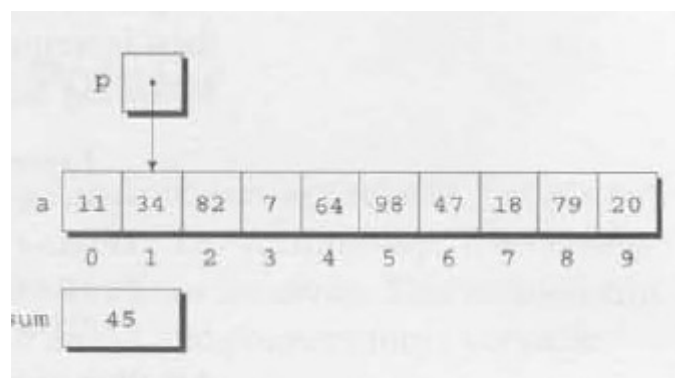
578

Следующие рисунки показывают содержимое **a**, **sum** и **p** в конце первых трёх итераций цикла (до увеличения **p**).

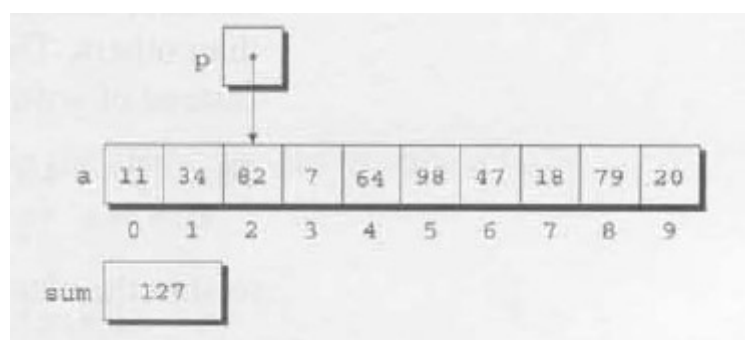
В конце первой итерации:



В конце второй итерации:



В конце третьей итерации:



Условие $p < \& a[N]$ в инструкции `for` заслуживает особого упоминания. Как ни странно, допустимо применять оператор адреса к $a[N]$, даже если этот элемент не существует (a индексируется от 0 до $N - 1$). Использование $a[N]$ таким образом совершенно безопасно, так как цикл не пытается исследовать его значение. Тело цикла будет выполнено с p , равным $\&a[0]$, $\&a[1]$..., $\&a[N-1]$, но когда p равно $\&a[N]$, цикл завершается. Мы могли бы так же легко написать цикл без указателей, конечно, используя вместо этого индексацию.

Аргумент, наиболее часто приводимый в поддержку арифметики указателей, заключается в том, что она может сэкономить время выполнения. Однако это зависит от реализации - некоторые компиляторы Си фактически производят лучший код для циклов, которые полагаются на индексацию.

Комбинация операторов * и ++

Программисты Си часто объединяют операторы * (косвенная адресация) и ++ в операторах, которые обрабатывают элементы массива. Рассмотрим простой случай сохранения значения в элементе массива и последующего перехода к следующему элементу. Используя индексирование массива, мы можем написать

```
a[i++] = j;
```

Если ***p*** указывает на элемент массива, соответствующая инструкция будет

****p++ = j;***

Поскольку постфиксная версия ++ имеет приоритет над *, компилятор видит это как

****(p++) = j;***

Значение ***p++*** есть ***p***. (Так как мы используем постфиксную версию ++, ***p*** не будет увеличиваться до тех пор, пока выражение не будет вычислено.) Таким образом, значение

**** (p++)*** будет ****p*** - объектом, на который указывает ***p***.

Конечно, **** p++*** не является единственной допустимой комбинацией * и ++. Например, мы могли бы написать ***(*p) ++***, которое возвращает значение объекта, на который указывает ***p***, а затем увеличивает этот объект (***p*** сам не изменяется). Если вы почувствуете, что это сбивает с толку, следующая таблица может помочь:

Выражение	Смысловое содержание
<i>* p++</i> или <i>*(p++)</i>	значение выражения <i>*p</i> перед инкрементом; инкремент <i>p</i> позже
<i>(*p) ++</i>	значение выражения <i>*p</i> перед инкрементом; инкремент <i>* p</i> позже
<i>* ++p</i> или <i>*(++p)</i>	инкремент <i>p</i> первым; значение выражения <i>*p</i> после инкремента

`++ *p` или `++ (*p)`

инкремент `*p` первым; значение выражения `*p` после инкремента (*преинкремент и разыменование имеют одинаковый приоритет и правую ассоциативность*)

Все четыре комбинации появляются в программах, хотя некоторые из них гораздо более распространены, чем другие. Чаще всего мы увидим `* p ++`, что удобно в циклах. Вместо написания

```
for (p = &a[0]; p < &a[N]; p++)
```

```
    sum += *p);
```

для суммирования элементов массива мы можем записать

```
p = &a[0];
```

```
while (p < &a[N])
```

```
    sum += *p++;
```

Операторы `*` и `-` комбинируются так же, как `*` и `++`. Для иллюстрации комбинирования `*` и `--`, вернёмся к примеру стёка раздела 10.2. Исходная версия стёка основывалась на целочисленной переменной `top`, чтобы отслеживать позицию "top-of-stack" (вершины стёка) в массиве `contents`. Давайте заменим вершину на переменную - указатель, которая сначала указывает на элемент 0 массива `contents`:

```
int *top_ptr = &contents[0];
```

Вот новые функции push и pop (обновление других функций стёка предлагается в виде упражнения):

```
void push(int i){  
    if (is_full())  
        stack_overflow();  
    else  
        *top_ptr++ = i;  
}
```

```
int pop_void) {  
    if (is_empty())  
        stack_underflow();  
    else  
        return *--top_ptr;  
}
```

Обратите внимание, что я написал * --top_ptr, а не * top_ptr--, так как я хочу, чтобы pop уменьшал top_ptr, прежде чем извлекать значение, на которое он указывает.

12.3 Использование имени массива в качестве указателя

Арифметика указателей - это один из способов, с помощью которого связаны массивы и указатели, но это не единственная связь между ними. Вот ещё одно ключевое отношение:

*Имя массива может использоваться как указатель на первый элемент массива. Эта связь упрощает арифметику указателя и делает массивы и указатели более универсальными. Например, предположим, что **a** объявлено следующим образом:*

```
int a [10];
```

Используя **a** как указатель на первый элемент в массиве, мы можем изменить **a[0]**:

```
* a = 7;                                /* сохранит 7 в [0] */
```

Мы можем изменить **a[1]** с помощью указателя **a + 1**:

```
* (a + 1) = 12;                          /* сохранит 12 в a[1] */
```

В общем случае **a + i** совпадает с **&a[i]** (оба представляют собой указатель на элемент **i** из **a**), а *** (a + i)** эквивалентен

$a[i]$ (оба представляют собой элемент i).

Другими словами индекс массива можно рассматривать как разновидность арифметики указателей. Тот факт, что имя массива может служить указателем, упрощает запись циклов, которые проходят через массив. Рассмотрим следующий цикл из раздела 12.2:

```
for (p = &a[0]; p < &a[N]; p++)
    sum += *p;
```

Чтобы упростить цикл, мы можем заменить $\&a[0]$ на a и $\&a[N]$ на $a + N$:

идиома:

```
for (p = a; p < a + N; p++)
    sum += *p;
```

Хотя имя массива может использоваться как указатель, невозможно присвоить ему новое значение. Попытка сделать его указывающим на другое место - это ошибка:

```
while (*a != 0)
    a++; /*** WRONG ***/
```

Это не так уж и страшно; мы всегда можем скопировать a в переменную-указатель, а затем изменить переменную-указатель:

```
p = a;
```

```
while (*p != 0)
```

```
    p++;
```

ПРОГРАММА Реверсирование последовательности чисел (Пересмотренная)

Программа reverse.c в Разделе 8.1 считывает 10 чисел, затем печатает их в обратном порядке. Когда программа считывает числа, она сохраняет их в массиве. Когда все числа считаны, программа проходит через массив в обратном порядке и печатает числа. Первоначальная программа использовала индексы для доступа к элементам массива. Вот новая версия, в которой я заменил индексы на арифметику указателей.

```
//reverse3.c
```

```
/* реверсирует ряд чисел */
```

```
#include <stdio.h>
```

```
#define N 10
```

```
int main(void){  
    int a[N], *p;
```

```
printf("Введите %d числа: ", N);  
for (p = a; p < a+N; p++)  
    scanf("%d", p);  
  
printf("В обратном порядке:");  
for (p = a + N - 1; p >= a; p--)  
    printf(" %d", *p);  
printf("\n");  
  
return 0;  
}
```

В исходной программе целочисленная переменная *i* отслеживала текущую позицию в массиве. новая версия заменяет *i* переменной *p*. Числа сохраняются в массиве; мы просто используем другой метод, чтобы отслеживать, где мы находимся в массиве.

Обратите внимание, что вторым аргументом для `scanf` является *p*, а не `&p`. Поскольку *p* указывает на элемент массива, это приемлемый аргумент для `scanf`; `&p`, напротив, будет указателем на указатель на элемент массива.

Массивы как аргументы (пересмотрено)

При передаче в функцию имя массива всегда обрабатывается как указатель. Рассмотрим следующую функцию, которая возвращает наибольший элемент в массиве целых чисел:

```
int find_largest(int a[], int n){  
    int i, max;  
    max = a[0];  
    for (i = 1; i < n; i++)  
        if (a[i] > max)  
            max = a[i];  
    return max;  
}
```

Предположим, что мы вызываем `find_largest` следующим образом:

```
largest = find_largest (b, N);
```

При этом вызове указатель на первый элемент ***b*** присваивается элементу ***a***; сам массив не копируется. Тот факт, что аргумент-массив рассматривается как указатель, имеет некоторые важные последствия:

■ Когда функции передаётся обычная переменная, её значение копируется; любые изменения соответствующего параметра не влияют на переменную. Напротив, массив, используемый в качестве аргумента, не защищён от изменений, поскольку копия самого массива не создаётся. (*Защищён указатель, ибо его копия передаётся в функцию.*)

Например, следующая функция (которую мы впервые увидели в Разделе 9.3) изменяет массив, сохраняя ноль в каждом из его элементов:

```
void store_zeros(int a[], int n){  
    int i;  
    for (i = 0; i < n; i++)  
        a[i] = 0;  
}
```

Чтобы указать, что параметр-массив не должен изменяться, мы можем включить слово `const` в его объявление:

```
int find_largest(const int a[], int n){  
    ...  
}
```

Если присутствует `const`, компилятор проверит, что в теле `find_largest` не появляется никакого присвоения элементу ***a***.

■ Время, необходимое для передачи массива функции, не зависит от размера массива. Нет никаких издержек при передаче большого массива, так как копия массива не создаётся.

■ При необходимости параметр массив может быть объявлен как указатель. Например, `find_largest`: можно определить следующим образом:

```
int find_largest(int *a, int n){  
    ...  
}
```

Объявление ***a*** указателем эквивалентно объявлению его массивом; компилятор обрабатывает объявления, как если бы они были идентичны.

Хотя объявление параметра массива равнозначно объявлению его указателем, то же самое не относится к переменной.

Декларация

```
int a [10];
```

заставляет компилятор выделить пространство для 10 целых чисел. Напротив, объявление

```
int *a;
```

заставляет компилятор выделять пространство для переменной-указателя.

В последнем случае ***a*** не является массивом; попытка использовать его как массив может иметь катастрофические результаты. Например, присваивание

```
* a = 0; /*** WRONG ***/
```

сохранит 0, там куда указывает ***a***. Поскольку мы не знаем, куда указывает ***a***, эффект программы не определён.

■ Функции с параметром-массивом может быть передан "срез" массива - ряд последовательных элементов.

Предположим, что мы хотим с помощью `find_largest` найти самый большой элемент в некоторой части массива ***b***, скажем, среди элементов

`b [5]..... b [14]`.

Когда мы вызовем `find_largest`, мы передадим ему адрес `b[5]` и число 10, указывающее, что мы хотим, чтобы `find_largest` исследовал 10 элементов массива, начиная с `b [5]`:

```
large= find_largest(&b[5], 10);
```

Использование указателя в качестве имени массива

Если мы сможем использовать имя массива в качестве указателя, то позволит ли нам Си индексировать указатель так, как если бы он был именем массива?

591

К этому времени вы, вероятно, ожидаете, что ответ будет "да", и вы будете правы. Вот пример:

```
#define N 10  
  
int a[N], i, sum = 0, *p = a;  
  
for (i = 0; i < N; i++)  
    sum += p[i];
```

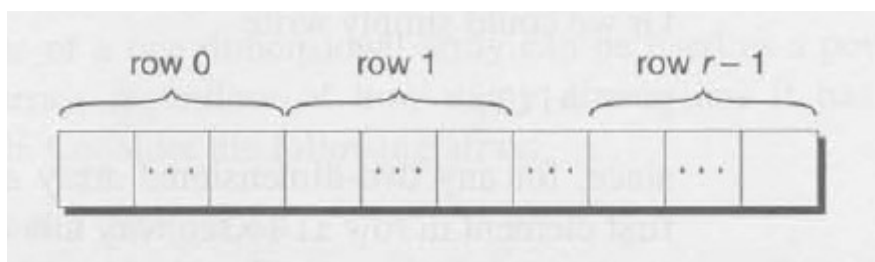
Компилятор рассматривает `p[i]` как `* (p + i)`, что является совершенно допустимым использованием арифметики указателей. Хотя способность индексировать указатель может показаться просто любопытной, в Разделе 17.3 мы увидим, что это действительно полезно.

12.4 Указатели и многомерные массивы

Подобно тому, как указатели могут указывать на элементы одномерных массивов, они также могут указывать на элементы многомерных массивов. В этом разделе мы рассмотрим общие методы использования указателей для обработки элементов многомерных массивов. Для простоты я буду рассматривать двумерные массивы, но все, что мы будем делать, в равной степени относится к многомерным массивам.

Обработка элементов многомерного массива

Мы видели в Разделе 8.2, что Си размещает двумерный массив данных в памяти по строкам; другими словами, элементы строки 0 идут первыми, а затем элементы строки 1 и так далее. Массив с r строками будет иметь следующий вид:



Мы можем воспользоваться этим макетом при работе с указателями. Если мы сделаем указатель p , указывающим на первый элемент в двумерном массиве (элемент в строке 0, столбец 0). Мы можем посетить каждый элемент в массиве, постоянно увеличивая p . В качестве примера рассмотрим задачу инициализации всех элементов двумерного массива значением нуля. Предположим, что массив был объявлен следующим образом:

```
int a[NUM_ROWS][NUM_COLS];
```

Очевидным методом было бы использовать вложенные циклы for:

```
int row, col;
```

...

```
for (row = 0; row < NUM_ROWS; row++)  
    for (col = 0; col < NUM_COLS; col++)  
        a[row] [col] = 0;
```

Но если мы рассматриваем ***a*** как одномерный массив целых чисел (в соответствии с тем как он хранится), мы можем заменить пару циклов одним циклом:

```
int *p;  
for (p= &a[0] [0]; p <= &a[NUM_ROWS - 1] [NUM_COLS - 1];  
p++)  
    *p = 0;
```

Цикл начинается с ***p***, указывающего на ***a*** [0] [0].

Последовательные приращения ***p*** указывают на ***a*** [0] [1],

a[0] [2], ***a***[0] [3] и так далее. Когда ***p*** достигает

a[0] [NUM_COLS - 1] (последний элемент в строке 0), новое его приращение делает ***p*** указателем на ***a***[1][0], первый элемент в строке 1. Процесс продолжается до тех пор, пока ***p*** не пройдет мимо ***a*** [NUM_ROWS - 1] [NUM_COLS - 1], последнего элемента в массиве. Хотя обработка двумерного массива как одномерного может показаться обманом, она работает с большинством компиляторов Си. Другое дело - будет ли это неплохой идеей.

Такие методы, как этот, определённо мешают читаемости программы, но, по крайней мере, с некоторыми старыми компиляторами, производят компенсирующее увеличение эффективности. Однако многие современные компиляторы практически не дают никакого преимущества в скорости.

Обработка строк многомерного массива

Как насчёт обработки элементов только в одной строке двумерного массива? Опять же, у нас есть возможность использовать переменную указателя ***p***. Чтобы посетить элементы строки ***i***, мы инициализируем ***p*** так, чтобы указать на элемент 0 строки ***i*** в массиве ***a***:

```
p = &a[i] [0];
```

Или можно просто записать

```
p = a[i];
```

так как для любого двумерного массива ***a*** выражение ***a[i]*** является указателем на первый элемент в строке ***i***. Чтобы понять, почему это работает, вспомните магическую формулу, которая связывает индексирование массива с арифметикой указателей: для любого массива ***a*** выражение ***a[i]*** эквивалентно **** (a + i)***. Таким образом, ***&a[i][0]*** совпадает с ***&(* (a [i] + 0))***, что эквивалентно ***&*a[i]*** и совпадает с ***a[i]***, так как операторы ***&*** и ******* аннулируются. *(Я бы объяснил это несколько иначе: рассмотрим ***a[i][j]*** как одномерный массив по имени ***a[i]***, тогда ***a[i]*** и будет указателем на 1-й элемент этого одномерного массива)*

Мы будем использовать это упрощение в следующем цикле, который устанавливает в 0 строку *i* массива *a*:

```
int a[NUM_ROWS] [NUM_COLS], *p, i;  
for (p = a[i]; p < a[i] + NUM_COLS; p++)  
    *p = 0;
```

Поскольку *a[i]* является указателем на строку *i* массива *a*, мы можем передать *a[i]* в функцию, которая ожидает одномерный массив в качестве аргумента. Другими словами, функция, предназначенная для работы с одномерными массивами, также будет работать со строкой, принадлежащей двумерному массиву. В результате такие функции, как *find_largest* и *store_zeros*, являются более универсальными, чем можно было бы ожидать. Рассмотрим *find_largest*, которую мы изначально разработали, чтобы найти самый большой элемент одномерного массива. Мы можем так же легко использовать *find_largest* для определения самого большого элемента в строке *i* двумерного массива:

```
largest = find_largest (a[i], NUM_COLS);
```

Обработка столбцов многомерного массива

Обработка элементов в столбце двумерного массива не так проста, поскольку массивы хранятся по строкам, а не по столбцам. Вот цикл, который устанавливает в 0 столбец *i* массива *a*:

```
int a[NUM_ROWS] [NUM_COLS], (*p) [NUM_COLS], i;
for (p = &a[0]; p < &a[NUM_ROWS]; p++)
    (*p) [i] = 0;
```

Я объявил *p* указателем на массив длины NUM_COLS, элементы которого являются целыми числами. Круглые скобки вокруг *p в (*p) [NUM_COLS] обязательны; без них компилятор будет рассматривать *p* как массив указателей вместо указателя на массив. Выражение *p++* продвигает *p* к началу следующей строки. В выражении (*p) [i], *p представляет собой целую строку *a*, (массив из элементов строки) поэтому (*p) [i] выбирает элемент в столбце *i* этой строки. Скобки в (*p) [i] имеют важное значение, поскольку компилятор интерпретирует *p [i] как * (p [i]).

Использование имени многомерного массива в качестве указателя

Имя любого массива, независимо от того, сколько измерений он имеет, может использоваться в качестве указателя так же, как имя одномерного массива. Однако требуется некоторая осторожность.

Рассмотрим следующий массив:

```
int a[NUM_ROWS] [NUM_COLS];
```

a не является указателем на `a [0] [0]`; вместо этого это указатель на одномерный массив `a [0]`. Это имеет смысл, если мы посмотрим на это с точки зрения Си, который рассматривает ***a*** не как двумерный массив, а как одномерный массив, элементами которого являются одномерные массивы. При использовании в качестве указателя ***a*** имеет тип `int (*) [NUM_COLS]` (указатель на целочисленный массив длины `NUM_COLS`). Знание того, что ***a*** указывает на `a[0]`, полезно для упрощения циклов, которые обрабатывают элементы двумерного массива, Например, вместо написания

```
for (p = &a[0]; p < &a[NUM_ROWS]; p++)  
    (*p)[i] = 0;
```

чтобы установить в 0 столбец ***i*** массива ***a***, мы можем записать

```
for (p = a; p < a + NUM_ROWS; p++)  
    (*p)[i] = 0;
```

Другая ситуация, в которой эти знания пригодятся, - это когда мы хотим "обмануть" функцию, заставив её думать, что многомерный массив действительно одномерен.

Например, рассмотрим, как можно использовать `find_largest` для поиска самого большого элемента в ***a***. В качестве первого аргумента `find_largest` попробуем передать ***a*** (адрес массива); в качестве второго передадим `NUM_ROWS * NUM_COLS` (общее количество элементов в массиве:

```
largest = find_largest(a, NUM_ROWS * NUM_COLS);
```

```
/**WRONG ***/
```

К сожалению, компилятор будет возражать против этого утверждения, потому что тип ***a*** есть `int (*) [NUM_COLS]`, но `find_largest` ожидает аргумент типа `int *`. Правильный вызов

```
largest = find_largest (a [0 ], NUM_ROWS * NUM_COLS);
```

`a [0]` указывает на элемент ***0*** в строке ***0*** и имеет тип `int *` (после преобразования компилятором), поэтому последний вызов будет работать правильно.

Указатели и массивы переменной длины (C99)

599

Указателям разрешено указывать на элементы массивов переменной длины (VLA), особенность C99.

Обычная указательная переменная будет использоваться для указания элемента одномерного VLA:

```
void f(int n){  
    int a[n], *p;  
    p = a;  
    ...  
}
```

Когда VLA имеет более одного измерения, тип указателя зависит от длины каждого измерения, кроме первого. Давайте посмотрим на двумерный случай:

```
void f(int m, int n){  
    int a[m][n], (*p) [n];  
    p = a;  
}
```

Поскольку тип ***p*** зависит от ***n***, которое не является постоянным, ***p***, как говорят, имеет изменяемый тип переменной. Обратите внимание, что допустимость такого присваивания, как `p = a`, не всегда может быть определена

компилятором. Например, следующий код компилируется, но корректен только в том случае, если *m* и *n* равны:

```
int a[m] [n], (*p) [m] ;
```

```
p = a;
```

Если $m \neq n$, любое последующее использование *p* вызовет неопределённое поведение. Изменяемые типы подвержены определенным ограничениям, как и массивы переменной длины. Наиболее важным ограничением является то, что объявление изменяемого типа должно находиться внутри тела функции или в прототипе функции. Арифметика указателей работает с VLAs так же, как и с обычными массивами.

Возвращаясь к примеру раздела 12.4, который очищает один столбец двумерного массива *a*, давайте объявим *a* как VLA на этот раз:

```
int a [m] [n];
```

Указатель, способный указывать на строку *a*, будет объявлен следующим образом: `int (*p) [n]`.

Цикл, который очищает столбец *i*, почти идентичен тому, который мы использовали в Разделе 12.4:

```
for (p=a; p<a+m; p++)
```

```
    (*p)[i]= 0;
```

Вопросы и ответы

Вопрос: Я не понимаю арифметику указателей. Если указатель является адресом, означает ли это, что такое выражение, как $p + j$, добавляет j к адресу, хранящемуся в p ?

Ответ: Нет. Целые числа, используемые в арифметике указателя, масштабируются в зависимости от типа указателя. Если p имеет тип int^* , например, тогда $p + j$ обычно добавляет $4 \times j$ к p , предполагая, что значения int хранятся в 4 байтах. Но если p имеет тип double^* , то $p + j$, вероятно, добавит $8 \times j$ к p , так как значения double обычно имеют длину 8 байт.

Вопрос: При написании цикла для обработки массива лучше использовать индексацию массива или арифметику указателей?

Ответ: На этот вопрос нет простого ответа, так как он зависит от используемой машины и самого компилятора. В первые дни C на PDP-11 арифметика указателя давала более быструю программу. На современных машинах, использующих современные компиляторы, индексация массива часто так же хороша, а иногда даже лучше. Итог: изучите оба способа, а затем используйте то, что более естественно для той программы, которую вы пишете.

Вопрос: Я где-то читал, что $i[a]$ то же, что и $a[i]$. Это правда?

Ответ: Да, это, как ни странно. Компилятор обрабатывает $i[a]$ как $*(i+a)$, что совпадает с $*(a+i)$. (Сложение указателей, как и обычное сложение, является коммутативным.) Но $*(a+i)$ эквивалентно $a[i]$, что и требовалось доказать, но, пожалуйста, не используйте $i[a]$ в программах, если вы не планируете участвовать в следующем конкурсе запутанных программ C.

Вопрос: Почему $*a$ совпадает с $a[]$ в объявлении параметра?

Ответ: Оба выражения указывают, что аргумент должен быть указателем. В обоих случаях возможны одни и те же операции над a (в частности, арифметика указателя и индексация массива).

И в обоих случаях самому a может быть присвоено новое значение внутри функции (хотя Си позволяет использовать имя переменной массива только как "постоянный указатель", такого ограничения на имя параметра-массива нет.)

Вопрос: Что является лучшим стилем объявления параметра массива $*a$ или $a[]$?

Ответ: Это очень сложный вопрос. С одной стороны, `a[]` является очевидным выбором, поскольку `*a` неоднозначен (нужен ли функции массив объектов или указатель на один объект?). С другой стороны, многие программисты утверждают, что объявление параметра как `*a` является более точным, так как это напоминает нам, что передаётся только указатель, а не копия массива.

Другие выбирают между `*a` и `a []` в зависимости от того, использует ли функция арифметику указателя или индексацию для доступа к элементам массива. (Этот подход, который я буду использовать.)

На практике `*a` встречается чаще, чем `[]`, поэтому вам лучше привыкнуть к нему. Чтобы там ни было, Деннис Ричи теперь ссылается на все обозначения как на "живое ископаемое", которое "служит, чтобы запутать ученика, и насторожить читателя."

Вопрос: Мы видели, что массивы и указатели тесно связаны в С. Можно ли сказать, что они взаимозаменяемы?

Ответ: Нет. Это правда, что параметры-массивы взаимозаменяемы с параметрами-указателями, но переменные массивы не совпадают с переменными указателями.

Формально, имя массива не является указателем; скорее, компилятор Си преобразует его в указатель, когда это необходимо. Чтобы увидеть эту разницу более ясно,

рассмотрим, что происходит, когда мы применяем оператор `sizeof` к массиву ***a***.

Значением `sizeof (a)` является общее количество байт в массиве - размер каждого элемента, умноженный на количество элементов. Но если ***p***-переменная-указатель, то `sizeof (p)` - это количество байтов, необходимое для хранения значения указателя.

Вопрос: Вы сказали, что обработка двумерного массива как одномерного работает с "большинством" компиляторов Си. Не работает ли она со всеми компиляторами?

Ответ: Нет. Некоторые современные компиляторы "с проверкой границ" отслеживают не только тип указателя, но - когда он указывает на массив - также длину массива.

Предположим, например, что ***p*** присваивается указатель на `a[0][0]`.

Фактически ***p*** указывает на первый элемент одномерного массива `a[0]`. Если мы несколько раз увеличиваем ***p***, попытавшись посетить все элементы ***a***, мы выйдем за границы, когда ***p*** пройдёт мимо последнего элемента `a[0]`.

Компилятор, который выполняет проверку границ, может вставить код, чтобы проверить, что ***p*** используется только для доступа к элементам в массиве, на который указывает `a[0]`; попытка инкремента ***p*** в конце этого массива будет определяться как ошибка.

Вопрос: Если ***a*** является двумерным массивом, почему мы можем передать `a[0]`, но не сам ***a***, на `find_largest`? Не указывают ли оба ***a*** и `a[0]` на одно и то же место (начало

массива)?

Ответ: Они, по сути, оба указывают на элемент `a[0][0]`. Проблема в том, что `a` имеет неправильный тип. При использовании в качестве аргумента это указатель на массив, но `find_largest` ожидает указатель на целое число. Однако `a[0]` имеет тип `int *`, поэтому это приемлемый аргумент для `find_largest`. Эта забота о типах на самом деле хороша; если бы Си не был таким придирчивым, мы могли бы сделать все виды ужасных ошибок с указателями, не имея замечаний компилятора.

Упражнения

К разделу 12.1

1. Предположим, что действуют следующие объявления:

```
int a[] = {5, 15, 34, 54, 14, 2, 52, 72};
```

```
int *p = &a[1], *q = &a[5];
```

a) каково значение `*(p+3)`?

b) каково значение `*(q-3)`?

c) каково значение `q - p`?

d) является ли условие `p < q` истинным или ложным?

e) является ли условие `*p < *q` истинным или ложным?

W*2. Предположим, что `high`, `low` и `middle` - все переменные указатели одного типа, `low` и `high` указывают на элементы

606

массива. Почему следующее утверждение недопустимо и как его можно исправить?

```
middle = (low + high) / 2 ;
```

К разделу 12.2

3. Каким будет содержимое массива **a** после выполнения следующих инструкций?

```
#define N 10
```

```
int a[] = {1,2, 3, 4, 5, 6, 7, 8, 9,10);
```

```
int *p = &a[0], *q = &a[N-1], temp;
```

```
while (p < q){
```

```
    temp = *p;
```

```
    *p++ = *q;
```

```
    *q--= temp;
```

```
}
```

W4. Перепишите `make_empty`, `is_empty`, и `is_full` функции раздела 10.2 используя переменную указатель `top_ptr` вместо целочисленной переменной `top`.

К разделу 12.3

5. Предположим, что ***a*** - одномерный массив, а ***p*** - переменная указатель. Предположим, что присваивание ***p* = *a*** только что было выполнено. какие из следующих выражений являются недопустимыми из-за несовпадения типов? Из оставшихся выражений, которые истинны (имеют ненулевое значение)?

(a) ***p* == *a*[0]**

(b) ***p* == &*a*[0]**

(c) ****p* == *a*[0]**

(d) ***p*[0] == *a*[0]**

W6. Перепишите следующую функцию, чтобы использовать арифметику указателей вместо индексации массива. (Другими словами, исключите переменную ***i*** и все виды использования оператора **[]**.) Сделайте как можно меньше изменений.

```
int sum_array(const int a[], int n)
{
    int i,sum;
    sum = 0;
    for (i = 0; i < n; i++)
        sum += a[i];
    return sum;
}
```

7. Напишите следующую функцию:

```
bool search(const int a[], int n, int key);
```

a - массив для поиска, ***n*** - количество элементов в массиве, а ***key*** - ключ поиска.

Поиск должен возвращать true, если ключ соответствует некоторому элементу ***a***, и false, если нет. Для посещения элементов массива используйте арифметику указателя, а не индексацию.

8. Перепишите следующую функцию, чтобы использовать арифметику указателей вместо индексации массива. (Другими словами, исключите переменную ***i*** и все виды использования оператора `[]`.) Сделайте как можно меньше изменений.

```
void store_zeros(int a[], int n)
{
    int i;
    for (i=0; i < n; i++)
        a[i] = 0;
}
```

9. Напишите следующую функцию:

```
double inner_product(const double *a, const double *b, int n)
```

a и **b** указывают на массивы длины **n**. Функция должна вернуть $a[0] * b[0] + a[1] * b[1] + \dots + a[n-1] * b[n-1]$. Для посещения элементов массива используйте арифметику указателя, а не индексацию.

10. Измените функцию `find_middle` в Разделе 11.5, чтобы она использовала арифметику указателя для вычисления возвращаемого значения.

11. Измените функцию `find_largeet` так, чтобы она использовала арифметику указателей, а не индексацию, для посещения элементов массива.

12. Напишите следующую функцию:

```
void find_two_largest(const int *a, int n, int *largest, int
*second_largest);
```

a указывает на массив длины **n**. Функция ищет в массиве самые большие и вторые по величине элементы, сохраняя их в переменных, на которые указывают наибольшие и вторые по величине, соответственно.

Используйте арифметику указателей, а не индексацию, чтобы посещать элементы массива.

К разделу 12.4

W13. Раздел 8.2 имел фрагмент программы, в котором два вложенных цикла `for` инициализировали массив `idem`; для использования в качестве единичной матрицы.

Перепишите этот код, используя один указатель для пошагового продвижения по массиву по одному элементу за раз. Подсказка: так как мы не будем использовать

переменные индекса `row` и `col`, будет не легко сказать где хранить 1. Вместо этого мы можем использовать тот факт, что первый элемент массива должен быть 1, следующие `N` элементов должны быть 0, следующий элемент должен быть 1, и так далее. Используйте переменную, чтобы отслеживать, сколько последовательных нулей было сохранено; когда количество достигнет `N`, сохраните 1.

14. Предположим, что в следующем массиве содержатся показания температуры за неделю, причём каждая строка содержит показания за один день:

```
int temperatures[7][24];
```

Напишите инструкцию, использующую функцию поиска (см. упражнение 7) для поиска значения 32 во всем массиве температур.

611

W15. Напишите цикл, который печатает все показания температуры, хранящиеся в строке *i* массива `temperatures` (см. упражнение 14). Используйте указатель, чтобы посетить каждый элемент строки.

16. Напишите цикл, который печатает самую высокую температуру в массиве `temperatures` (см. упражнение 14) для каждого дня недели. Тело цикла должно вызывать функцию `find_largest`, передавая ей по одной строке массива за раз.

17. Перепишите следующую функцию, чтобы использовать арифметику указателей вместо индексов массива. (Другими словами, исключите переменные *i* и *j* и все виды использования оператора `[]`.) Используйте один цикл вместо вложенных циклов.

```
int sum_two_dimeneional_array(const int a[] [LEN], int n)
```

```
int i, j, sum = 0;
{
    for (i = 0; i < n; i++)
        for (j = 0; j < LEN; j++)
            sum += a[i] [j];
    return sum;
```

}

18. Напишите функцию `evaluate_position`, описанную в упражнении 13 Главы 9 используйте арифметику указателя - не индексацию - для посещения элементов массива. Используйте один цикл вместо вложенных циклов.

Проекты по программированию

W1. (a) Напишите программу, которая считывает сообщение, а затем печатает его разворот:

Введите сообщение: Don't get mad, get even.

Разворот будет :.neve teg,dam teg t'noD

Подсказка: прочитайте сообщение по одному символу за раз (используя `getchar`) и сохраните символы в массиве.

Прекратите чтение, когда массив заполнен или прочитан символ `'\n'`.

(b) Пересмотрите программу, чтобы использовать указатель вместо целого числа для отслеживания текущей позиции в массиве.

2. (a) Напишите программу, которая читает сообщение, затем проверяет, является ли оно палиндромом (буквы в сообщении те же слева направо, что и справа налево):

613

Введите сообщение: He lived as a devil, eh?

Палиндром

Введите сообщение: Madam, I am Adam.

Палиндром

Игнорируйте все символы, которые не являются буквами.

Используйте целочисленные переменные для отслеживания позиций в массиве.

(b) Пересмотрите программу, чтобы использовать указатели вместо целых чисел для отслеживания позиций в массиве

W3. Упростите Проект по программированию 1 (b), воспользовавшись тем, что в качестве указателя можно использовать имя массива.

4. Упростите Проект по программированию 2 (b), воспользовавшись тем, что в качестве указателя можно использовать имя массива.

5. Измените проект программирования 14 из Главы 8, чтобы он использовал указатель вместо целого числа для отслеживания текущей позиции в массиве, содержащем предложение.

6. Изменить `qsort.c` программа раздела 9.6, так что `low`, `high` и `middle` являются указателями на элементы массива, а не целые числа.

Функция `split` должна возвращать указатель, а не целое число.

7. Измените `maximin.c` программу раздела 11.4, так что функция `max_min` использует указатель вместо целого числа для отслеживания текущей позиции в массиве.

13 Строки

Трудно извлечь смысл из строк, но это единственная коммуникационная "монета", на которую мы можем рассчитывать.

Хотя мы использовали переменные типа `char` и массивы значений типа `char` в предыдущих главах, нам все ещё не хватает удобного способа обработки серии символов (строк, в терминологии Си).

Мы исправим этот дефект в этой главе, которая охватывает как строковые константы (или литералы, как они называются

в стандарте Си) и строковые переменные, которые могут изменяться во время выполнения программы. Раздел 13.1 объясняет правила, которые управляют строковыми литералами, включая правила встраивания escape-последовательностей в строковые литералы и разбиения длинных строковых литералов.

Раздел 13.2 затем показывает, как объявить строковые переменные, которые являются просто массивами символов, в которых специальный символ—нулевой символ—отмечает конец строки.

Раздел 13.3 описывает способы чтения и записи строк.

Раздел 13.4 показывает, как написать функции, которые обрабатывают строки, и Раздел 13.5 описывает некоторые функции обработки строк в библиотеке Си.

В Разделе 13.6 представлены идиомы, которые часто используются при работе со строками.

Наконец, в Разделе 13.7 описывается, как настроить массивы, элементы которых являются указателями на строки различной длины.

Этот раздел также объясняет, как Си использует такой массив для предоставления информации программам командной строки.

13.1 Строковый литерал

Строковый литерал - это последовательность символов, заключённая в двойные кавычки:

"Когда окажешься на перепутье, смело иди вперёд!"

Мы впервые столкнулись со строковыми литералами в Главе 2; они часто появляются как строки формата в вызовах `printf` и `scanf`.

Escape-последовательности в строковых литералах

Строковые литералы могут содержать те же escape -последовательности, что и символьные константы. Некоторое время мы использовали Escape - символы в строках формата `printf` и `scanf`.

Например, мы видели, что каждый символ `\n` в строке

"Candy\nIs dandy\nBut liquor\nIs quicker.\n--Ogden Nash\n"

перемещает курсор на следующую строку:

Candy

Is dandy

But liquor

Is quicker

--Ogden Nash

("Конфеты это прекрасно, но ликёр надёжнее."--Огден Нэш)

Хотя восьмеричные и шестнадцатеричные escape-последовательности также являются допустимыми в строковых литералах, они не так распространены, как символьные escape-последовательности. Будьте осторожны при использовании восьмеричных и шестнадцатеричных escape-последовательностей в строковых литералах.

Восьмеричная escape-последовательность заканчивается после трёх цифр или перед первым не восьмеричным символом. Например, строка "\1234" содержит два символа (\123 и 4), а строка "\189" содержит три символа (\1,8 и 9).

Шестнадцатеричная escape-последовательность, с другой стороны, не ограничивается тремя цифрами: она продолжается до первого символа, отличного от шестнадцатеричного. Рассмотрим, что произойдёт, если строка содержит escape-последовательность \xfc, представляющую символ ! в наборе символов Latin1-обычного расширения ASCII. Строка "Z\xfc rich" ("Zürich") имеет шесть символов (Z, \xfc, r, i, c и h), но строка "\xfcber" (неудачная попытка в "über") имеет только два (\xfcbe и r). Большинство компиляторов будут возражать против последней строки, так как шестнадцатеричные escape-

последовательности обычно ограничиваются диапазоном \ x0-\ xff.

Продолжение строкового литерала

Если мы обнаружим, что строковый литерал слишком длинный, чтобы удобно поместиться на одной строке, Си позволяет нам продолжить его на следующей строке, при условии, что мы заканчиваем первую строку символом обратной косой черты (\). Никакие другие символы не могут следовать за \ на той же строке, кроме (невидимого) символа новой строки в конце:

```
printf("when you come to a fork in the road, take it. \n\n--Yogi Berra");
```

В общем случае символ \ может использоваться для соединения двух или более строк программы в одну строку (процесс, который стандарт С называет "сплайсингом" ("соединением")).

Мы увидим больше примеров соединений в Разделе 14.3.

Метод \ имеет один недостаток:

строка должна продолжаться с начала следующей строки, тем самым разрушая структуру отступов программы. Есть лучший способ справиться с длинными строковыми

литералами, благодаря следующему правилу: когда два или более строковых литерала смежны (разделены только пробельным символом), компилятор объединит их в одну строку.

Это правило позволяет разделить строковый литерал на две или более строк:

```
printf("Когда окажешься на перепутье, смело иди вперед! "  
      "--Yogi Berra");
```

Как хранятся строковые литералы

Мы часто использовали строковые литералы в вызовах `printf` и `scanf`. Но когда мы вызываем `printf` и передаём строковый литерал в качестве аргумента, что мы на самом деле передаём?

Чтобы ответить на этот вопрос, нам нужно знать, как хранятся строковые литералы. В сущности, Си рассматривает строковые литералы как массивы символов. Когда компилятор Си встречает строковый литерал длины *n* в программе, он выделяет *n + 1* байт памяти для строки.

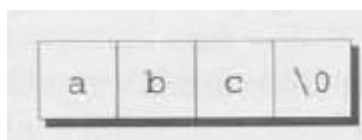
Эта область памяти будет содержать символы строки, а также один дополнительный символ-нулевой символ (`null`) - для обозначения конца строки.

620

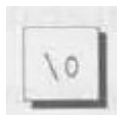
Нулевой символ - это байт, биты которого равны нулю, поэтому он представлен escape-последовательностью `\0`.

Не путайте нулевой символ (`\0`) с символом нуля (`'0'`). Нулевой символ имеет код 0; символ нуля имеет другой код (48 в ASCII).

Например, строковый литерал `"abc"` хранится в виде массива из четырёх символов (a, b, c и `\0`):



Строковые литералы могут быть пустыми; строка `""` хранится как один символ `null`:



Поскольку строковый литерал хранится в виде массива, компилятор рассматривает его как указатель типа `char *`.

Например, и `printf`, и `scanf` ожидают значения типа `char *` в качестве первого аргумента. Рассмотрим следующий пример:

```
printf("abc");
```

При вызове `printf` передаётся адрес `"abc"` (указатель на то, где в памяти хранится буква **a**).

Операции со строковыми литералами

В общем, мы можем использовать строковый литерал везде, где Си позволяет использовать указатель `char*`. Например, строковый литерал может отображаться справа от оператора присваивания:

```
char *p;
```

```
p = "abc";
```

Это присваивание не копирует символы "abc"; оно просто делает ***p*** указывающим на первый символ строки. Си позволяет указателям быть индексированными, поэтому мы можем индексировать строковые литералы:

```
char ch;
```

```
ch = "abc"[1];
```

Новым значением ***ch*** будет буква ***b***. Другими возможными индексами являются ***0*** (для выбора буквы ***a***), ***2*** (буква ***c***), и ***3*** (нулевой символ).

Это свойство строковых литералов используется не так часто, но иногда оно удобно.

Рассмотрим следующую функцию, которая преобразует число от 0 до 15 в символ, представляющий эквивалентную шестнадцатеричную цифру:

```
char digit_to_hex_char(int digit){  
    return "0123456789ABCDEF"[digit];  
}
```

Попытка изменить строковый литерал вызывает неопределённое поведение:

```
char *p = "abc";  
(*p == 'a')  
*p = 'd'; /*** WRONG ***/
```

Программа, которая пытается изменить строковый литерал, может аварийно завершить работу или вести себя хаотично.

Строковые литералы и символьные константы

Строковый литерал, содержащий один символ, не является символьной константой. Строковый литерал **"a"** представлен указателем на ячейку памяти, содержащую символ **a** (за которым следует символ null).

Символьная константа **'a'** представлена целым числом (числовой код символа). Никогда не используйте символ

когда требуется строка (или наоборот). Вызов `printf("\n");` является допустимым, поскольку `printf` ожидает указатель в качестве первого аргумента. Однако следующий вызов не является допустимым :

```
printf('\n'); /*** WRONG ***/
```

13.2 Строковые переменные

Некоторые языки программирования предоставляют специальный тип `string` для объявления строковых переменных. Си принимает другой подход: любой одномерный массив символов может быть использован для хранения строки с пониманием того, что строка заканчивается нулевым символом. Этот подход прост, но имеет существенные трудности. Иногда трудно определить, используется ли массив символов в качестве строки. Если мы пишем наши собственные функции обработки строк, мы должны быть осторожны в том, чтобы они правильно работали с нулевым символом. Кроме того, нет более быстрого способа определить длину строки, чем посимвольный поиск нулевого символа. Допустим, нам нужна переменная, способная хранить строку длиной до 80 символов. Поскольку в конце строки потребуется символ `null`, мы объявим переменную массивом из 81 символа:

```
#define STR_LEN 80  
char str[STR_LEN+1];
```

Мы определили STR_LEN как 80, а не 81, тем самым подчёркивая тот факт, что str может хранить строки не более 80 символов, а затем добавил 1 к STR_LEN в объявлении str. Это обычная практика среди программистов. При объявлении массива символов, который будет использоваться для хранения строки, всегда делайте массив на один символ длиннее строки из-за соглашения Си о том, что каждая строка заканчивается нулевым символом. Неспособность оставить место для символа null может привести к непредсказуемым результатам при выполнении программы, так как функции в библиотеке Си предполагают, что строки завершаются символом null.

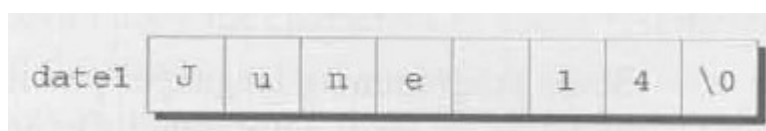
Объявление массива символов длиной STR_LEN + 1 не означает, что он всегда будет содержать строку символов STR_LEN. Длина строки зависит от позиции завершающего символа null, а не от длины массива, в котором хранится строка. Массив символов STR_LEN + 1 может содержать строки различной длины, начиная от пустой строки и заканчивая строками STR_LEN.

Инициализация строковой переменной

Строковая переменная может быть инициализирована одновременно с объявлением:

```
char date1 [8] = "June 14";
```

Компилятор поместит символы из "June 14" в массив date1, а затем добавит нулевой символ, чтобы date1 можно было использовать в качестве строки. Вот как будет выглядеть date1:



Хотя "June 14" кажется строковым литералом, это не так. Вместо этого Си рассматривает его как сокращённое наименование инициализатора массива.

Фактически, мы могли бы написать

```
char date1 [8]={'J', 'u', 'n', 'e', ' ', '1', '4', '\0'};
```

Я думаю, вы согласитесь, что оригинальную версию легче читать. Что делать, если инициализатор слишком короткий для заполнения строковой переменной?

В этом случае, компилятор добавляет дополнительные символы null.

Таким образом, после декларации

626

```
char date2[9] = "June 14";
```

date2 будет иметь следующий вид:

date2	J	u	n	e		1	4	\0	\0
-------	---	---	---	---	--	---	---	----	----

Такое поведение в целом согласуется с обработкой инициализаторов массивов в Си. Если инициализатор массива короче самого массива, остальные элементы инициализируются нулём. Инициализируя оставшиеся элементы массива символом в \0, компилятор следует тому же правилу. Что делать, если инициализатор длиннее строковой переменной?

Это недопустимо для строк, так же как и для других массивов. Однако Си позволяет инициализатору (не считая нулевого символа) иметь точно такую же длину, что и переменная:

```
char date3[7]= "June 14";
```

Там нет места для символа null, поэтому компилятор не пытается сохранить его:

date3	J	u	n	e		1	4
-------	---	---	---	---	--	---	---

Если вы планируете инициализировать массив символов, содержащий строку, убедитесь, что длина массива больше длины инициализатора.

В противном случае компилятор незаметно опустит символ `null`, что сделает массив непригодным для использования в качестве строки. Объявление строковой переменной может опустить её длину, в этом случае компилятор вычисляет её:

```
char date4[] = "June 14";
```

Компилятор выделяет восемь символов для `date4`, чтобы сохранить символы в "June 14" плюс нулевой символ. (Тот факт, что длина `date4` не указана, не означает, что длина массива может быть изменена позже. После компиляции программы длина `date4` фиксируется в восемь.) Пропуск длины строковой переменной особенно полезен, если инициализатор длинный, так как вычисление длины вручную подвержено ошибкам.

Массивы символов в сравнении с символьными указателями

Давайте сравним объявление

```
char date [] = "June 14";
```

которое объявляет `date` массивом с похожим видом

```
char * date= "June 14";
```

который объявляет `date` указателем. Благодаря тесной связи между массивами и указателями, мы можем использовать любую версию `date` в качестве строки.

В частности, любая функция, ожидающая передачи массива символов или символьного указателя, будет принимать в качестве аргумента любую из версий `date`.

Однако мы должны быть осторожны и не допустить ошибки, полагая, что эти два варианта `date` взаимозаменяемы.

Между ними существуют значительные различия:

- В версии массива символы, хранящиеся в `date`, могут быть изменены, как и элементы любого массива.

В версии указателя `date` указывает на строковый литерал, и мы видели в Разделе 13.1, что строковые литералы не должны изменяться.

- В версии массива `date` - это имя массива. В версии указателя `date` - это переменная, которая может указывать на другие строки во время выполнения программы.

Если нам нужна строка, которая может быть изменена, мы должны установить массив символов, в котором будет храниться строка; объявление переменной-указателя здесь неуместно.

Объявление:

```
char *p;
```


заставляет компилятор выделить достаточно памяти для переменной указателя; к сожалению, он не выделяет место для строки. (От чего же? Да мы не указали, длину строки!) Прежде чем мы сможем использовать **p** в качестве строки, он должен указывать на массив символов. Можно создать **p**, указывающем на строковую переменную:

```
char str[STR_LEN+1], *p;
```

```
p = str;
```

p теперь указывает на первый символ str, поэтому мы можем использовать **p** как строку.

Другая возможность - сделать **p**, указывающим на динамически выделяемую строку (*Об этом разговор позднее*).

Использование неинициализированной переменной указателя в качестве строки является серьёзной ошибкой. Рассмотрим следующий пример, в котором предпринимается попытка построения строки "abc":

```
char *p;
```

```
p[0] = 'a'; /**** WRONG ****/
```

```
p[1] = 'b'; /****WRONG ****/
```

```
p[2] = 'c'; /**** WRONG ****/
```

```
p[3] = '\0'; /**** WRONG ****/
```

Поскольку **p** не был инициализирован, мы не знаем, куда он указывает. Использование указателя для записи символов a, b, c и \0 в память приводит к неопределённому поведению.

13.3 Чтение и запись строк

Написать строку легко с помощью функций printf или puts.

Чтение строки немного сложнее, в первую очередь из-за возможности того, что входная строка может быть длиннее строковой переменной, в которую она сохраняется.

Чтобы прочесть строку за один шаг, мы можем использовать scanf или gets. В качестве альтернативы, мы можем читать строки по одному символу за раз.

Написание строк с помощью printf и puts

Спецификация преобразования %s позволяет printf записать строку.

Рассмотрим следующий пример:

```
char str[] = "Are we having fun yet?"; // "Нам уже весело?"
```

631

```
printf("%s\n", str);
```

Выход будет:

```
"Are we having fun yet?"
```

printf записывает символы в строку по одному, пока не встретит символ null. (Если отсутствует символ null, printf продолжает после конца строки, пока - в конце концов - она находит нулевой символ где-то в памяти.)

Для печати только части строки можно использовать спецификацию преобразования ***%.ps***, где ***p***-количество отображаемых символов.

Инструкция

```
printf("%.6s\n", str);
```

будет печатать

```
Are we
```

Строка, как и число, может быть напечатана в поле.

Преобразование ***%ms*** отобразит строку в поле размера ***m***. (Строка, содержащая более ***m*** символов, будет напечатана полностью, а не усечена.)

Если строка содержит менее ***m*** символов, она будет выровнена по правому краю поля. Чтобы, вместо этого, выполнить левое выравнивание, мы можем поставить знак минус перед ***m***. Значения ***m*** и ***p*** можно использовать в комбинации:

Спецификация преобразования формы `%m.ps` вызывает отображение первых ***p*** символов строки в поле размера ***m***.

`printf` - не единственная функция, которая может писать строки. Библиотека Си также предоставляет функцию `puts`, которая используется следующим образом:

```
puts(str);
```

`puts` имеет только один аргумент (строку для печати). После записи строки `puts` всегда записывает дополнительный символ новой строки, таким образом переходя к началу следующей строки вывода.

Чтение строк с помощью `scanf` и `gets`

Спецификация преобразования `%s` позволяет `scanf` считывать строку в массив символов:

```
scanf("%s", str);
```

Нет необходимости ставить оператор **&** перед `str` в вызове `scanf`; как и любое имя массива, `str` обрабатывается как указатель при передаче в функцию.

При вызове `scanf` она пропускает пробелы, затем считывает символы и сохраняет их в `str`, пока не встретит пробел.

`scanf` всегда сохраняет нулевой символ в конце строки.

Строка, прочитанная с помощью `scanf`, никогда не будет содержать пробелов.

Следовательно, `scanf` обычно не читает полную строку ввода; символ новой строки, а также символ пробела или табуляции приведёт к остановке чтения `scanf`.

Чтобы прочитать всю строку ввода за раз, мы можем использовать `gets`. Как и `scanf`, функция `gets` считывает входные символы в массив, а затем сохраняет нулевой символ.

В остальном, однако, `gets` несколько отличается от `scanf`:

- `gets` не пропускает пробел перед началом чтения строки (`scanf` пропускает).

- `gets` читает, пока не находит символ новой строки (`scanf` останавливается на любом пробельном символе).

Кстати, `gets` отбрасывает символ новой строки вместо того, чтобы хранить его в массиве; положенное ему место занимает `null` символ.

Чтобы увидеть разницу между `scanf` и `gets`, рассмотрим следующий фрагмент программы:

```
char sentence[SENT_LEN+1];  
printf("Enter a sentence:\n");  
scanf("%s", sentence);
```

Предположим, что после подсказки

Введите предложение:

пользователь вводит в строку

To C, or not to C: that is the question.

scanf сохранил строку "To" в sentence. Следующий вызов scanf возобновит чтение строки через пробел после слова To. Предположим теперь, что мы заменим scanf на gets:

```
gets (sentence);
```

Когда пользователь вводит то же самое, что и раньше, gets сохранит строку

" To C, or not to C: that is the question."

в sentence.

Когда они считывают символы в массив, scanf и gets не могут обнаружить, когда массив будет полон. Следовательно, они

могут хранить символы после конца массива, вызывая неопределённое поведение.

scanf можно сделать более безопасным, используя спецификацию преобразования *%ns* вместо *%s*, где *n*-целое число, указывающее максимальное число символов для хранения.

gets, к сожалению, по своей сути небезопасен; *fgets*-гораздо лучшая альтернатива.

Чтение строк посимвольно

Поскольку *scanf* и *gets* являются рискованными и недостаточно гибкими для многих приложений, программисты Си часто пишут свои собственные функции ввода. Читая строки по одному символу за раз, эти функции обеспечивают большую степень контроля, чем стандартные входные функции.

Если мы решим разработать свою собственную функцию ввода, нам нужно будет рассмотреть следующие вопросы:

- Должна ли функция пропускать пробелы перед началом сохранения строки?
- Какой символ заставляет функцию прекращать чтение: символ новой строки, пробел или другой символ?

Этот символ хранится в строке или отбрасывается?

- Что делать функции, если входная строка слишком длинная для хранения: отбросить лишние символы или оставить их для следующей операции ввода?

Предположим, нам нужна функция, которая не пропускает пробельные символы, прекращает чтение на первом символе новой строки (который не сохранится в строке) и отбрасывает дополнительные символы.

Функция может иметь следующий прототип:

```
int read_line(char str[], int n);
```

str представляет массив, в котором мы будем хранить входные данные, а *n*-максимальное количество символов для чтения.

Если строка ввода содержит более *n* символов, *read_line* отбросит дополнительные символы.

read_line вернёт количество символов, которые она фактически хранит в *str* (число от 0 до *n*).

Возможно, нам не всегда нужно возвращаемое значение *read_line*, но это не повредит, если оно будет доступно, *read_line* состоит в основном из цикла, который вызывает *getchar* для чтения символа, а затем сохраняет символ в *str*, при условии, что осталось место.

Цикл завершается при чтении символа новой строки. (Строго говоря, у нас также должно быть завершение цикла, если `getchar` не сможет прочесть символ, но пока мы проигнорируем это усложнение.)

Вот полное определение `read_line`:

```
int read_line(char str[], int n){
    int ch, i = 0;
    while ((ch = getchar()) != '\n')
        if (i < n)
            str[i++] = ch;
    str[i] = '\0'; /* terminates string */
    return i; /* number of characters stored */
}
```

Обратите внимание, что `ch` имеет тип `int`, а не `char`, так как `getchar` возвращает символ, который она читает как значение `int`. Перед возвратом `read_line` помещает нулевой символ в конец строки.

Стандартные функции, такие как `scanf` и `gets`, автоматически помещают нулевой символ в конец входной строки; однако, если мы пишем нашу собственную входную функцию, мы должны взять на себя эту ответственность.

13.4 Доступ к символам в строке

Поскольку строки хранятся в виде массивов, мы можем использовать индексацию для доступа к символам в строке.

Например, чтобы обработать каждый символ в строке *s*, мы можем создать цикл, который увеличивает счётчик *i* и выбирает символы через выражение *s[i]*.

Предположим, что нам нужна функция, которая подсчитывает количество пробелов в строке. Используя индексацию массива, мы можем написать функцию следующим образом:

```
int count_spaces(const char s[]){  
    int count = 0, i;  
    for (i = 0; s[i] != '\0'; i++)  
        if (s[i] == ' ')  
            count++;  
    return count;  
}
```

Я включил `const` в объявление *s*, чтобы указать, что `count_spaces` не изменяет массив, который представляет *s*.

Если *s* не является строкой, функции потребуется второй аргумент, указывающий длину массива. Однако, поскольку *s* является строкой, `count_spaces` может определить, где она заканчивается путём тестирования нулевого символа. Многие программисты на Си не пишут `count_spaces` так, как мы. Вместо этого они будут использовать указатель для отслеживания текущей позиции в строке.

Как мы видели в Разделе 12.2, этот метод всегда доступен для обработки массивов, но особенно удобен для работы со строками. Давайте перепишем функцию `count_spaces`, используя арифметику указателей вместо индексов массива.

Мы устраним переменную *i* и будем использовать *s* для отслеживания нашей позиции в строке. Посредством приращения *s* повторно, `count_spaces` может проходить через каждый символ в строке. Вот наша новая версия функции:

```
int count_spaces (const char * s){
    int count = 0;
    for(; *s != '\0'; s++)
        if (*s == ' ')
            count++;
    return count;
}
```

Обратите внимание, что `const` не запрещает `count_spaces` изменять `s`; он предназначен для того, чтобы функция не изменяла то, на что указывает `s`.

И поскольку `s` является копией указателя, который передаётся в `count_spaces`, увеличение `s` не влияет на исходный указатель.

Пример `count_spaces` вызывает некоторые вопросы о том, как писать строковые функции:

■ Что лучше использовать операции массива или операции указателя для доступа к символу в строке? Мы вправе использовать то, что удобнее; мы можем даже смешать их. Во второй версии `count_spaces` обработка `s` как указателя упрощает функцию немного, устраняя необходимость в переменной `i`.

Традиционно программисты Си склоняются к использованию операций указателя для обработки строк.

■ Должен ли строковый параметр объявляться как массив или как указатель?

Две версии `count_spaces` иллюстрируют параметры:

первая версия объявляет `s` как массив;

второй объявляет `s` указателем.

Фактически, нет никакой разницы между двумя объявлениями —напомним из раздела 12.3, что компилятор обрабатывает параметр массив, как если бы он был объявлен как указатель.

■ Влияет ли форма параметра (`s[]` или `*s`) на то, что может быть предоставлено в качестве аргумента?

Нет. При вызове `count_spaces` аргумент может быть именем массива, переменной указателем или строковым литералом - `count_spaces` не может определить разницу.

13.5 Использование библиотеки строк Си

Некоторые языки программирования предоставляют операторы, которые могут копировать строки, сравнивать строки, объединять строки, выбирать подстроки и тому подобное.

Операторы Си, напротив, по существу бесполезны для работы со строками. Строки рассматриваются как массивы в Си, поэтому они ограничены теми же способами, что и массивы - в частности, их нельзя копировать или сравнивать с помощью операторов.

Прямые попытки скопировать или сравнить строки завершатся неудачей.

Например, предположим, что `str1` и `str2` были объявлены следующим образом:

```
char str1[10], str2[10];
```

Копирование строки в массив символов с помощью оператора = невозможно:

```
str1 = "abc"; /*** WRONG ***/
```

```
str2 = str1; /***WRONG ***/
```

Мы видели в Разделе 12.3, что использование имени массива в качестве левого операнда = является недопустимым. Однако инициализация массива символов с помощью = допустима:

```
char str1[10] = " abc";
```

В контексте объявления = не является оператором присваивания. Попытка сравнить строки с помощью оператора отношения или равенства является допустимой, но не приведёт к желаемому результату:

```
if (str1 == str2)... /*** WRONG ***/
```

Этот оператор сравнивает str1 и str2 как указатели; он не сравнивает содержимое двух массивов. Так как str1 и str2 имеют разные адреса, выражение str1 == str2 должно иметь значение 0.

К счастью, не все потеряно: Библиотека Си предоставляет богатый набор функций для выполнения операций над строками. Прототипы этих функций находятся в файле

заголовков `<string.h>`, поэтому программы, которым нужны строковые операции, должны содержать следующую строку:

```
#include <string.h>
```

Большинству функций объявленных в `<string.h>` требуется по крайней мере одна строка в качестве аргумента. Строковые параметры объявляются с типом `char*`, что позволяет аргументу быть массивом символов, переменной типа `char *` или строковым литералом—все они подходят в качестве строк. Однако следите за строковыми параметрами, которые не объявлены `const`. Такой параметр может быть изменён при вызове функции, поэтому соответствующий аргумент не должен быть строковым литералом.

В `<string.h>` есть много функций; я расскажу о некоторых из самых основных. В последующих примерах предполагается, что `str1` и `str2` являются символьными массивами, используемыми в качестве строк.

Функция `strcpy` (копирование строки)

Функция `strcpy` имеет следующий прототип в `<string.h>`:

```
char *strcpy(char *s1, const char *s2);
```

strcpy копирует строку *s2* в строку *s1*. (Чтобы быть точным, мы должны сказать: "strcpy копирует строку, на которую указывает *s2*, в массив, на который указывает *s1*.")

То есть strcpy копирует символы из *s2* в *s1* до (и включая) первого нулевого символа в *s2*.

strcpy возвращает *s1* (указатель на строку назначения).

Строка, на которую указывает *s2*, не изменяется, поэтому она объявлена const.

Существование strcpy компенсирует тот факт, что мы не можем использовать оператор присваивания для копирования строк. Например, предположим, что мы хотим сохранить строку "abcd" в str2.

Мы не можем использовать присваивание

```
str2 = "abcd"; /*** WRONG ***/
```

поскольку str2 является именем массива и не может отображаться в левой части присваивания. Вместо этого мы можем вызвать strcpy:

```
strcpy(str2, "abcd"); /* str2 теперь содержит "abcd" */
```

Точно так же мы не можем напрямую присвоить str2 для str1, но мы можем вызвать strcpy:

```
strcpy(str1, str2); /* str1 теперь содержит "abcd" */
```


В большинстве случаев мы отбрасываем значение, возвращаемое `strcpy`.

Однако иногда может быть полезно вызвать ***strcpy*** как часть большего выражения, чтобы использовать его возвращаемое значение. Например, мы могли бы объединить ряд вызовов `strcpy`:

```
strcpy(str1, strcpy(str2, "abcd")); /* как str1, так и str2 теперь  
содержат "abcd" */
```

В вызове `strcpy(str1, str2)` `strcpy` не имеет возможности проверить, что строка, на которую указывает `str2`, действительно поместится в массив, на который указывает `str1`. Предположим, что `str1` указывает на массив длины *n*. Если строка, на которую указывает `str2`, содержит не более *n* - 1 символов, то копирование будет выполнено успешно.

Но если `str2` указывает на более длинную строку, возникает неопределённое поведение. (Так как `strcpy` всегда копирует до первого символа `null`, копирование будет продолжено после конца массива, на который указывает `str1`.)

Вызов функции `strncpy` является более безопасным, хотя и медленным, способом копирования строки.

`strncpy` похожа на `strcpy`, но имеет третий аргумент, который ограничивает количество копируемых символов.

Чтобы скопировать `str2` в `str1`, мы могли бы использовать следующий вызов `strncpy`:

```
strncpy(str1, str2, sizeof(str1));
```

До тех пор, пока `str1` достаточно велик, чтобы содержать строку, хранящуюся в `str2` (включая нулевой символ), копия будет сделана правильно, хотя сама `strncpy` не без опасности.

Во-первых, она оставит строку в `str1` без завершающего нулевого символа, если длина строки, хранящейся в `str2`, больше или равна размеру массива `str1`. Вот более безопасный способ использовать `strncpy`:

```
strncpy(str1, str2, sizeof(str1)-1);
```

```
str1[sizeof(str1) - 1] = '\0';
```

Вторая инструкция гарантирует, что ***str1*** всегда завершается нулём, даже если `strncpy` не удаётся скопировать нулевой символ из `str2`.

Функция `strlen` (длина строки)

Функция `strlen` имеет следующий прототип:

```
size_t strlen(const char *s);
```

size_t, определённый в библиотеке C, является именем ***typedef***, представляющим один из целочисленных типов Си без знака.

Если мы не имеем дело с очень длинными строками, эта формальность не должна нас беспокоить - мы можем просто рассматривать возвращаемое значение `strlen` как целое число, `strlen` возвращает длину строки `s`: количество символов в `s` до,

647

но не включая, первый нулевой символ. Вот несколько примеров:

```
int len;  
  
len = strlen("abc") ; /* len is now 3 */  
  
len = strlen(""); /* len is now 0 */  
  
strcpy(str1, "abc");  
  
len = strlen(str1); /* len is now 3 */
```

Последний пример иллюстрирует важный момент. Если в качестве аргумента задан массив, `strlen` не измеряет длину самого массива, а возвращает длину строки, хранящейся в массиве.

Функция `strcat` (конкатенация строк)

Функция `strcat` имеет следующий прототип:

```
char *strcat: (char *s1, const char *s2) ;
```

`strcat` добавляет содержимое строки *s2* в конец строки *s1*; она возвращает *s1* (указатель на полученную строку). Вот несколько примеров `strcat` в действии:

```
strcpy(str1, "abc");
```

648

```
strcat(str1, "def"); /*str1 now contains "abcdef" */
```

```
strcpy(str1, "abc");
```

```
strcpy(str2, "def");
```

```
strcat(str1, str2); /* str1 now contains "abcdef" */
```

Как и в случае **strcpy**, значение, возвращаемое **strcat**, обычно отбрасывается. В следующем примере показано, как можно использовать возвращаемое значение:

```
strcpy(str1, "abc");
```

```
strcpy(str2, "def");
```

```
strcat(str1, strcat(str2, "ghi")):
```

```
/*
```

```
str1 now contains "abcdefghi";
```

```
str2 contains "defghi"
```

```
*/
```

Эффект вызова `strcat (str1, str2)` не определён, если массив, на который указывает `str1`, недостаточно длинный для размещения дополнительных символов из `str2`.

Рассмотрим следующий пример:

```
char str1[6] = "aha";
```

```
strcat(str1, "def"); /*** WRONG ***/
```

strcat попытается добавить символы *d*, *e*, *f* и *\0* в конц строки, уже сохраненной в str1. К сожалению, str1 ограничен шестью символами, что приводит к тому, что strcat выполняет запись после конца массива. Функция strncat безопаснее, но медленнее strcat. Как и strncpy, она имеет третий аргумент, ограничивающий количество символов, которые она будет копировать. Вот как может выглядеть вызов:

```
strncat(str1, str2, sizeof(str1) - strlen(str1) - 1);
```

strncat завершит str1 символом null, который не включён в третий аргумент (количество копируемых символов). В примере третий аргумент вычисляет, сколько свободного места в str1 (данное выражение sizeof (str1) - strlen(str1)), а затем вычитается *1*, чтобы убедиться, что там будет место для нуль-символа.

Функция strcmp (сравнение строк)

Функция strcmp имеет следующий прототип:

```
int strcmp(const char *s1, const char *s2);
```

`strcmp` сравнивает строки **s1** и **s2**, возвращая значение меньше, равно или больше 0, в зависимости от того, является ли **s1** меньше, равно или больше **s2**. Например, чтобы узнать, меньше ли `str1` `str2`, мы напишем:

```
if (strcmp(str1, str2) < 0) /* is str1 < str2? */
```

Чтобы проверить, является ли `str1` меньше или равна `str2`, мы напишем:

```
if (strcmp(str1, str2) <= 0) /* is str1 <= str2? */
```

Выбрав правильный оператор отношения (<, <=, >, >=) или оператор равенства (==, !=), мы можем проверить любые возможные отношения между `str1` и `str2`, `strcmp` сравнивает строки на основе их лексикографического порядка, который напоминает способ, которым слова расположены в словаре.

Точнее, `strcmp` считает, что **s1** будет меньше **s2**, если выполнено одно из следующих условий:

- Первые *i*-символов **s1** и **s2** совпадают, но (i + 1) -ый символ **s1** меньше, чем (i + 1) -ый символ **s2**. Например, "abc" меньше, чем "bcd", а "abd" меньше, чем "abe".

- Все символы **s1** соответствуют **s2**, но **s1** короче **s2**. Например, "abc" меньше "abcd".

Сравнивая символы из двух строк, `strcmp` смотрит на числовые коды, которые представляют символы. Некоторые

знания о базовом наборе символов полезны для прогнозирования того, что будет делать `strcmp`. Например, вот несколько важных свойств набора символов ASCII:

- Символы в каждой из последовательностей A-Z, a-z и 0-9 имеют последовательные коды.
- Все прописные буквы меньше, чем все строчные. (В ASCII коды от 65 до 90 представляют заглавные буквы; коды от 97 до 122 представляют строчные буквы.)
- Цифры меньше букв. (Коды от 48 до 57 представляют собой цифры.)
- Пробелы меньше, чем все печатные символы. (Символ пробела имеет значение 32 в ASCII.)

ПРОГРАММА Печать списка напоминаний за один месяц

Чтобы проиллюстрировать использование библиотеки строк Си, мы разработаем программу, которая печатает список ежедневных напоминаний за один месяц. Пользователь вводит серию напоминаний с префиксом дня месяца.

Когда пользователь введёт 0 вместо допустимого дня, программа напечатает список всех введённых напоминаний, отсортированных по дням. Вот как будет выглядеть сеанс работы с программой:

652

Enter day and reminder: 24 Susan's birthday

Enter day and reminder: 5 6:00 - Dinner with Marge and Russ

Enter day and reminder: 26 Movie - "Chinatown"

Enter day and reminder: 7 10:30 A Dental appointment

Enter day and reminder: 12 Movie - "Dazed and Confused"

Enter day and reminder: 5 Saturday class

Enter day and reminder: 12 Saturday class

Enter day and reminder: 0

Day Reminder

5 Saturday class

5 6:00 - Dinner with Marge and Russ

7 10:30 - Dental appointment

12 Saturday class

12 Movie - "Dazed and Confused."

24 Susan's birthday

26 Movie — "Chinatown"

Общая стратегия не очень сложная: у нас будет программа, которая читает серию комбинаций "день-и-напоминание", сохраняя их в порядке (отсортировано по дням), а затем отображает их.

Чтобы прочитать дни, мы будем использовать scanf; чтобы

прочитать напоминания, мы будем использовать функцию `read_line` из Раздела 13.3.

Мы будем хранить строки в двумерном массиве символов, причём каждая строка массива содержит одну строку ввода. После того, как программа прочитает день и связанное с ним напоминание, она будет просматривать массив, чтобы определить, куда поместить день, используя для сравнения `strcmp`.

Затем она будет использовать `strcpy` для перемещения всех строк ниже этой точки вниз на одну позицию. Наконец, программа скопирует день в массив и вызовет `strcat`, чтобы добавить напоминание в день. (До этого момента день и напоминание хранились отдельно).

Конечно, всегда есть несколько незначительных осложнений. Например, мы хотим, чтобы дни были выровнены по правому краю в двухсимвольном поле, чтобы их цифры совпадали. Есть много способов справиться с этой проблемой. Я выбрал, чтобы программа использовала `scanf` для чтения дня в целочисленную переменную, а затем вызвала `sprintf` для преобразования дня обратно в строковую форму, `sprintf`-это библиотечная функция, похожая на `printf`, за исключением того, что она записывает вывод в строку. Вызов

```
sprintf(day_str, "%2d", day);
```

записывает значение дня в `day_str`. Поскольку `sprintf` автоматически добавляет нулевой символ при записи, `day_str`

654

будет содержать правильную строку с завершающим нулем. ещё одно осложнение заключается в том, что пользователь не вводил более двух цифр. Для этой цели мы будем использовать следующий вызов scanf:

```
scanf("%2d", &day);
```

Число 2 между % и d сообщает scanf прекратить чтение после двух цифр, даже если на входе больше цифр.

С этими предварительными замечаниями, вот программа:

```
// remind.c
```

```
/* Prints a one-month reminder list */
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_REMIND 50 /* maximum number of reminders */
```

```
#define MSG_LEN 60 /* max length of reminder message */
```

```
int read_line(char str[], int n);
```

```
int main(void){
```

655

```
char reminders[MAX_REMIND][MSG_LEN+3];
char day_str[3], msg_str[MSG_LEN+1];
int day, i, j, num_remind = 0;

for (;;) {
    if (num_remind == MAX_REMIND) {
        printf("-- No space left --\n");
        break;
    }

    printf("Enter day and reminder: ");
    scanf("%2d", &day);
    if (day == 0)
        break;
    sprintf(day_str, "%2d", day);
    read_line(msg_str, MSG_LEN);

    for (i = 0; i < num_remind; i++)
        if (strcmp(day_str, reminders[i]) < 0)
            break;
```

656

```
    for (j = num_remind; j > i; j--)  
        strcpy(reminders[j], reminders[j-1]);
```

```
    strcpy(reminders[i], day_str);  
    strcat(reminders[i], msg_str);
```

```
    num_remind++;
```

```
}
```

```
printf("\nDay Reminder\n");  
for (i = 0; i < num_remind; i++)  
    printf(" %s\n", reminders[i]);
```

```
return 0;
```

```
}
```

```
int read_line(char str[], int n){  
    int ch, i = 0;  
    while ((ch = getchar()) != '\n')  
        if (i < n)
```

657

```
    str[i++] = ch;  
    str[i] = '\0';  
    return i;  
}
```

(Массив reminders[MAX_REMIND][MSG_LEN+3] постепенно инициализируется таким образом, что он всегда будет упорядочен по возрастанию дней. Например:

1, 4, 8, 30, 31. Когда вводится, допустим, 11-й день, "счётчик i" при просмотре массива по возрастанию дней увеличивается за каждый цикл на единицу до первого числа большего 11, т.е. до 30 и i будет равна 3. Переменная num_remind к этому времени равна 5, соответственно j=5 и запускается цикл копирования: сначала 5-я строка 31 копируется в "пустую" 6-ю, затем 4-я 30 "затирает" 5-ю, "освобождается" строка 3 (i=3) для последующей "вставки" 11-ти. Переменная num_remind увеличивается до 6.

Итог:

```
reminders[0] = 1  
reminders[1] = 4  
reminders[2] = 8  
reminders[3] = 11  
reminders[4] = 30  
reminders[5] = 31
```

*Первое присваивание при $i = 0$, $num_remind = 0$ минует все вложенные циклы `for` и непосредственно выполняет инструкции `strcpy(reminders[i], day_str);`
`strcat(reminders[i], msg_str);`*

Хотя `remind.c` полезен для демонстрации функций `strcpy`, `strcat` и `strchr`, ему не хватает чего-то в качестве практической программы напоминания. Очевидно, что требуется ряд улучшений, начиная от незначительных настроек до крупных улучшений (например, сохранение напоминаний в файле при завершении программы).

Мы обсудим несколько улучшений в проектах по программированию в конце этой главы и в последующих главах.

13.6 Строковые идиомы

Функции работы со строками являются особенно богатым источником идиом. В этом разделе мы рассмотрим некоторые из самых известных идиом, используя их для написания функций **`strlen`** и **`strcat`**.

Вам никогда не придётся писать эти функции, конечно, так как они являются частью стандартной библиотеки, но вам, возможно, придётся писать похожие функции.

Краткий стиль, который я буду использовать в этом разделе, популярен среди многих программистов на Си. Вы должны освоить этот стиль, даже если Вы не планируете использовать его в своей собственной программе, так как вы, вероятно, столкнётесь с ним в коде, написанном другими.

Последнее замечание, прежде чем мы начнём. Если вы хотите попробовать любую из версий `strlen` и `strcat` в этом разделе обязательно измените имя функции(например, изменив `strlen` на `my_strlen`).

Как поясняется в Разделе 21.1.нам не разрешается писать функцию с тем же именем, что и у стандартной библиотечной функции, даже если мы не включаем заголовочный файл, к которому принадлежит функция.

Фактически, все имена, начинающиеся с `str` и строчной буквы, зарезервированы (чтобы разрешить добавление функций в заголовочный файл `<string.h>` в будущих версиях стандарта Си.

Поиск конца строки

Многие строковые операции требуют поиска конца строки. Функция `strlen` является ярким примером.

Следующая версия `strlen` просматривает свой строковый аргумент, чтобы найти конец, используя переменную для отслеживания длины строки:

```
size_t strlen(const char *s){  
    size_t n;  
    for (n = 0; *s != '\0'; s++)  
        n++;  
    return n;  
}
```

Поскольку указатель *s* перемещается по строке слева направо, переменная *n* отслеживает, сколько символов до сих пор было обнаружено.

Когда в конечном итоге указывается нулевой символ, *n* будет содержать длину строки. Давайте посмотрим, можем ли мы сжать функцию. Сначала мы переместим инициализацию *n* в её объявление:

```
size_t strlen(const char *s){  
    size_t n = 0;  
    for (;*s != '\0'; s++)  
        n++;  
    return n;  
}
```


Далее заметим, что условие $*s \neq '\backslash 0'$ совпадает с $*s \neq 0$, так как целочисленное значение нулевого символа равно 0. Но тестирование $*s \neq 0$ совпадает с тестированием $*s$; оба варианта истинны, если $*s$ не равно 0. Эти наблюдения приводят к нашей следующей версии strlen:

```
size_t strlen(const char *s){
    size_t n = 0;
    for (; *s; s++)
        n++;
    return n;
}
```

Но, как мы видели в Разделе 12.2, можно увеличить s и тестировать $*s$ в одном выражении:

```
size_t strlen(const char *s){
    size_t n = 0;
    for (; *s++; )
        n++;
    return n;
}
```

Заменяя инструкцию for инструкцией while, мы приходим к следующей версии strlen:

662

```
size_t strlen(const char *s){  
    size_t n = 0;  
    while (*s++)  
        n++;  
    return n;  
}
```

Хотя мы немного сократили `strlen`, вполне вероятно, что мы не увеличили её скорость. Вот версия, которая работает быстрее, по крайней мере, с некоторыми компиляторами:

```
size_t strlen(const char *s){  
    const char *p = s;  
    while (*s)  
        s++;  
    return s-p;  
}
```

Эта версия ***strlen*** вычисляет длину строки, определяя положение нулевого символа, а затем вычитая из неё позицию первого символа в строке. Улучшение скорости происходит из-за того, что не нужно увеличивать ***n*** внутри цикла ***while***. Обратите внимание на появление слова ***const*** в объявлении ***p***,

между прочим; без него компилятор заметил бы, что присвоение *s p* приводит к тому, что строка, на которую указывает *s*, подвержена риску.

Инструкция

идиома *while (*s)*
 s++;

и, соответственно,

идиома *while (*s++)*
 ;

являются идиомами, означающими "поиск нулевого символа в конце строки".

Первая версия оставляет указатель на нулевой символ.

Вторая версия более кратка, но только оставляет указатель за нулевым символом.

Копирование строки

Копирование строки является ещё одной распространённой операцией. Чтобы ввести идиому "копирования строк" языка Си, мы разработаем две версии функции *strcat*.

Давайте начнём с простой, но несколько длинной версии:

664

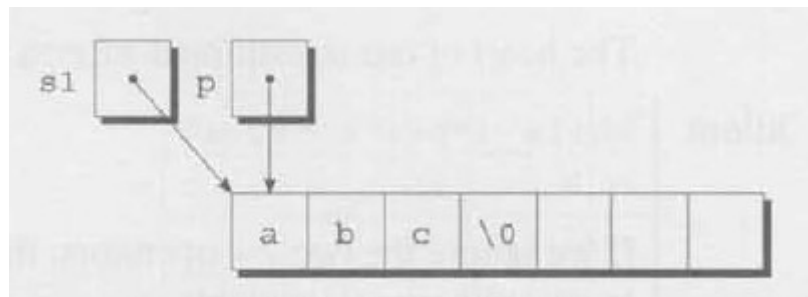
```
char *strcat(char *s1, const char *s2){
    char *p = s1;
    while (*p != '\0')
        p++;
    while (*s2 != '\0'){
        *p = *s2;
        p++;
        s2++;
    }
    *p = '\0';
    return s1;
}
```

Эта версия **strcat** использует двухэтапный алгоритм:

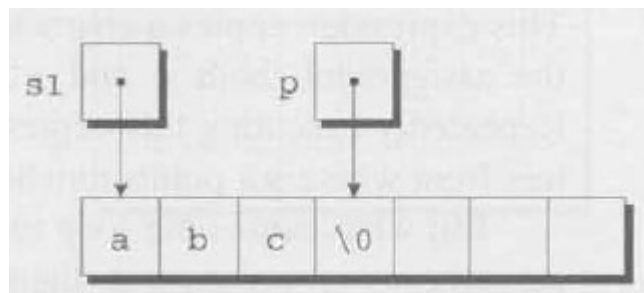
(1) найти нулевой символ в конце строки **s1** и сделать **p** указывающим на него.

(2) скопировать символы один за другим из **s2** туда, куда указывает **p**.

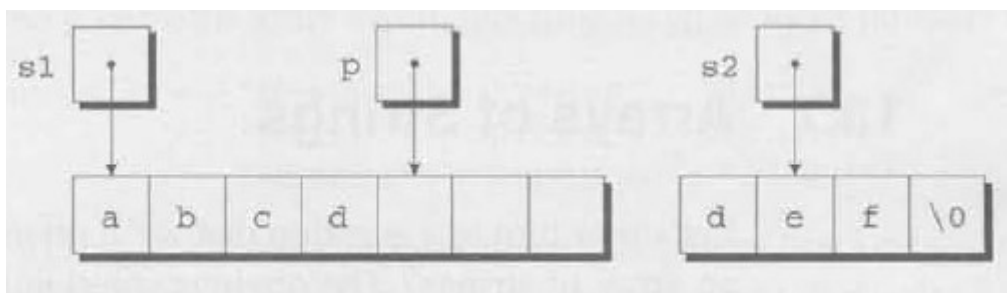
Первая инструкция `while` в функции реализует шаг (1), **p** устанавливается так, чтобы указывать на первый символ в строке **s1**. Предполагая, что **s1** указывает на строку "abc", мы имеем следующую картину:



затем значение ***p*** увеличивается, пока оно не укажет на нулевой символ. Когда цикл завершается, ***p*** должен указывать на символ ***null***:

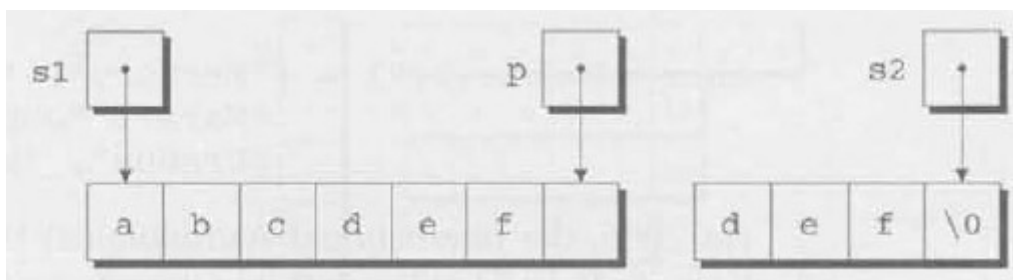


Вторая инструкция **while** реализует шаг (2). Тело цикла копирует один символ, с того места на которое указывает ***s2*** туда, куда указывает ***p***, а затем увеличивает как ***p***, так и ***s2***. Если ***s2*** первоначально указывает на строку "def", вот как выглядят строки после первой итерации цикла:



666

Цикл завершается, когда **s2** указывает на нулевой символ:



после ввода нулевого символа, туда куда указывает **p**, **strcat** возвращается. С помощью процесса, аналогичного тому, который мы использовали для **strlen**, мы можем уплотнить определение **strcat**, получив следующую версию:

```
char *strcat(char *s1, const char *s2){
    char *p = s1;
    while (*p)
        p++;
    while (*p++ = *s2++)
        ;
    return s1;
}
```

Сердцем нашей оптимизированной функции **strcat** является идиома "копирование строки":

идиома `while (* p ++ = * s2 ++)`
 `;`

Если мы проигнорируем два оператора ++, выражение внутри круглых скобок упростится до обычного присваивания:

* p = * s2

Это выражение копирует символ, на который указывает *s2* в место на которое указывает *p*.

После присваивания, как *p*, так и *s2* увеличиваются, благодаря операторам ++. Неоднократное выполнение этого выражения имеет эффект копирования ряда символов, на которые указывает *s2* в места на которые указывает *p*. Но что заставляет цикл прекратиться?

Поскольку основным оператором внутри скобок является присваивание, инструкция while проверяет значение присваивания - копируемый символ. Все символы, кроме нулевого символа, дают true, поэтому цикл не будет завершён до тех пор, пока не будет скопирован нулевой символ.

И так как цикл завершается после присваивания, нам не нужна отдельная инструкция, чтобы поместить нулевой символ в конец новой строки.

Массивы строк

Давайте перейдём к вопросу, с которым мы часто сталкиваемся: как лучше всего хранить массив строк?

Очевидным решением является создание двумерного массива

символов, а затем сохранение строк в массиве, по одному в строке. Рассмотрим следующий пример:

```
char planets[][8] = ("Mercury","Venus","Earth",  
    "Mars", "Jupiter", "Saturn",  
    "Uranus ", "Neptune", "Pluto");
```

(В 2006 году Международный астрономический Союз понизил Плутон с "планеты" до "карликовой планеты", но я оставил его в массиве планет по старой памяти.)

Обратите внимание, что нам разрешено опускать количество строк в массиве планет—так как это количество элементов в инициализаторе—но Си требует, чтобы мы указали количество столбцов.

Рисунок в верхней части следующей страницы показывает, как планеты могут выглядеть.

Не все наши строки были достаточно длинными, чтобы заполнить всю строку массива, поэтому Си дополнял их символами *null*.

В этом массиве есть немного неиспользуемого пространства, так как только три планеты имеют имена, достаточно длинные, чтобы требовать восьми символов (включая завершающий нулевой символ).

Программа remind.c (Раздел 13.5) является ярким примером такого рода трат. Она хранит напоминания в строках двумерного массива символов, с 60 символами, отведёнными для каждого напоминания. В нашем примере длина напоминаний составляла от 18 до 37 символов, поэтому объем потраченного впустую пространства был значительным.

	0	1	2	3	4	5	6	7
0	M	e	r	c	u	r	y	\0
1	V	e	n	u	s	\0	\0	\0
2	E	a	r	t	h	\0	\0	\0
3	M	a	r	s	\0	\0	\0	\0
4	J	u	p	i	t	e	r	\0
5	S	a	t	u	r	n	\0	\0
6	U	r	a	n	u	s	\0	\0
7	N	e	p	t	u	n	e	\0
8	P	l	u	t	o	\0	\0	\0

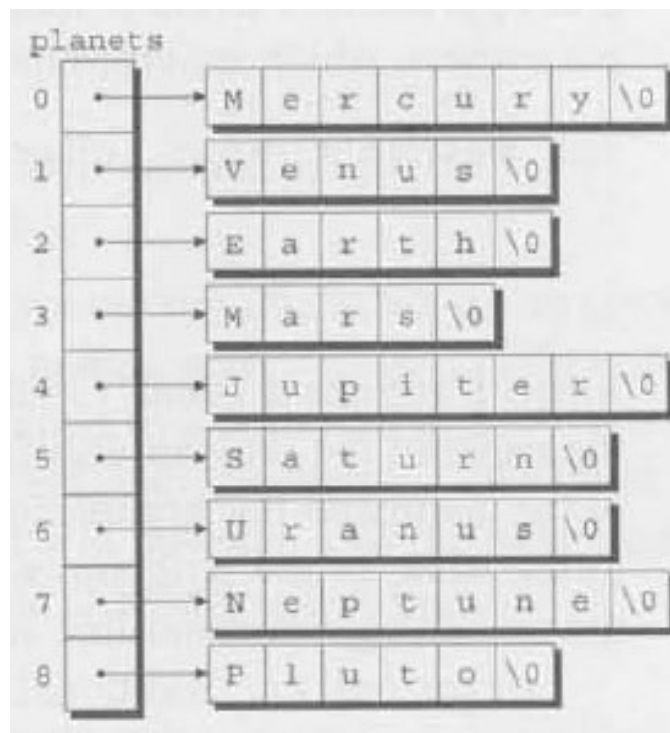
Неэффективность, которая проявляется в этих примерах, часто встречается при работе со строками, так как большинство коллекций строк будет иметь смесь длинных и коротких строк.

Нам нужен рваный массив: двумерный массив, строки которого могут иметь разную длину.

C не предоставляет "рваный тип массива", но он даёт нам инструменты для его имитации. Секрет в том, чтобы создать массив, элементы которого являются указателями на строки. Вот массив планет снова, на этот раз в виде массива указателей на строки:

```
char *planets[] = {"Mercury", "Venus", "Earth",
                  "Mars", "Jupiter", "Saturn",
                  "Uranus", "Neptune", "Pluto"};
```

Ничего особенного не изменилось, да? Мы просто убрали пару скобок и поставили звёздочку перед `planets`. Однако влияние на то, как планеты хранятся, драматично:



Каждый элемент `planets` является указателем на строку, заканчивающуюся `null`. В строках нет ненужных символов, хотя нам пришлось выделить место для указателей в массиве `planets`. Чтобы получить доступ к одному из имён `planets`, все, что нам нужно сделать, это получить индекс массива `planets`.

Из-за связи между указателями и массивами доступ к символу в имени планеты осуществляется так же, как и доступ к элементу двумерного массива.

Для поиска в массиве планет строк, начинающихся, например, с буквы `M`, можно использовать следующий цикл:

```
for (i = 0; i < 9; i++)
    if (planets[i][0] == 'M')
```

```
printf("%s begins with M\n",planets[i]);
```

Аргументы командной строки

Когда мы запускаем программу, то часто должны предоставить ей информацию — возможно имя файла или ключ, который изменяет поведение программы. Рассмотрим команду UNIX - *ls*.

Если запустить *ls*, набрав *ls* в командной строке, она будет отображать имена файлов в текущем каталоге. Но если вместо этого набрать

```
ls -l
```

то *ls* отобразит "длинный" (подробный) список файлов, показывающий размер каждого файла, владельца файла, дату и время последнего изменения файла и т. д.

Чтобы изменить поведение *ls* далее, можно указать, чтобы она показывала детали только для одного файла:

```
ls -l remind.c
```

ls отобразит подробную информацию о файле с именем *remind.c*.

Информация о командной строке доступна всем программам, а не только командам операционной системы.

Чтобы получить доступ к этим аргументам командной строки (называемым параметрами программы в стандарте Си), мы должны определить *main* как функцию с двумя параметрами, которые обычно называются *argc* и *argv*:

```
int main(int argc, char *argv[]){
    ...
}
```

`argc` ("argument count - счётчик аргументов") - количество аргументов командной строки (включая имя самой программы).

`argv` ("argument vector - вектор аргументов") представляет собой массив указателей на аргументы командной строки, которые хранятся в строковой форме.

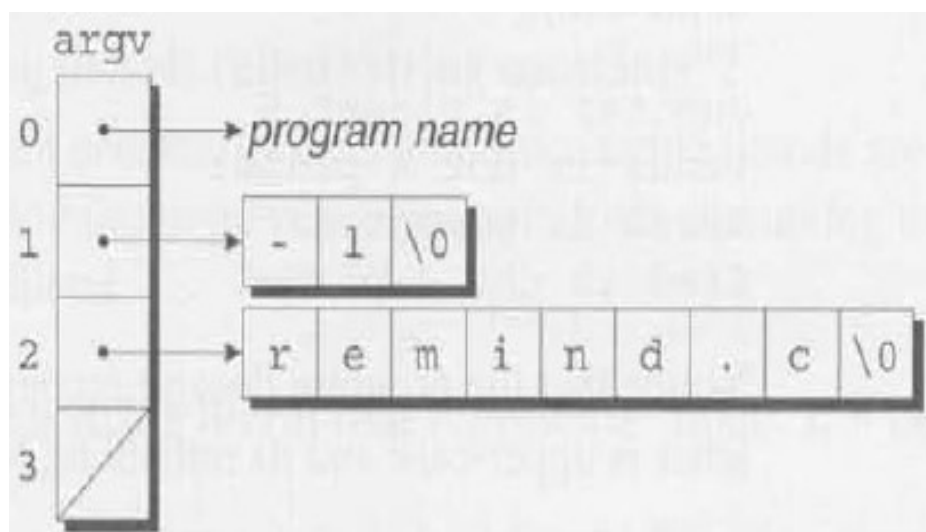
`argv [0]` указывает на имя программы, а `argv [1] ÷ argv [argc-1]` указывают на остальные аргументы командной строки.

argv имеет один дополнительный элемент `argv [argc]`, который всегда является нулевым указателем - специальным указателем, который указывает на ничего. Мы обсудим нулевые указатели в следующей главе; на данный момент все, что нам нужно знать, это то, что макрос `NULL` представляет собой нулевой указатель. Если пользователь вводит в командную строку

```
ls -l remind.c
```

то `argc` будет с 3-мя аргументами.

`argv [0]` укажет на строку, содержащую имя программы, `argv [1]` укажет на строку `"-l"`, `argv [2]` укажет на строку `"remind.c"`, а `argv [3]` будет нулевым указателем:



На этом рисунке имя программы не показано подробно, так как оно может включать путь или другую информацию, которая зависит от операционной системы. Если имя программы недоступно, `argv [0]` указывает на пустую строку.

Поскольку `argv` представляет собой массив указателей, доступ к аргументам командной строки очень прост. Как правило, программа, ожидающая аргументы командной строки, создаёт цикл, который по очереди проверяет каждый аргумент.

Одним из способов записи такого цикла является использование целочисленной переменной в качестве индекса в массиве `argv`. Например, следующий цикл выводит аргументы командной строки по одному в строке:

```
int i;
for (i = 1; i < argc; i++)
    printf("%s\n", argv[i]);
```

Другой метод заключается в настройке указателя на `argv[1]`, а затем многократно увеличивайте указатель, чтобы пройти через остальную часть массива.

Поскольку последний элемент `argv` всегда является нулевым указателем, цикл может завершиться при обнаружении нулевого указателя в массиве:

```
char **p;
for (p = &argv[1]; *p != NULL; p++)
    printf("%s\n", *p);
```

Поскольку ***p*** - это указатель на указатель на символ, мы должны использовать его аккуратно. Значение ***p***, равное `&argv[1]`, имеет смысл; `argv[1]` является указателем на символ, поэтому `&argv[1]` будет указателем на указатель. Тест `*p != NULL` нормальный, так как `*p` и `NULL` - оба указатели.

Приращение ***p*** работает правильно; ***p*** указывает на элемент массива, поэтому приращение ***p*** приведёт его к следующему элементу. Печать `*p` - в порядке вещей, так как `*p` указывает на первый символ в строке.

ПРОГРАММА Проверка имён планет

Наша следующая программа, `planet.c`, иллюстрирует, как обращаться к аргументам командной строки. Программа предназначена для проверки ряда строк, чтобы увидеть, какие

675

из них являются именами планет. Когда программа запускается, пользователь будет проверять строки в командной строке:

```
planet Jupiter venue Earth fred
```

Программа укажет, будет ли каждая строка именем планеты; если это так, программа также отобразит номер планеты (при этом планета **1** будет самой близкой к Солнцу):

```
Jupiter is planet 5
```

```
venus is not a planet
```

```
Earth is planet 3
```

```
fired is not a planet
```

Обратите внимание, что программа не распознает строку как имя планеты, если её первая буква не является верхним регистром, а остальные её буквы имеют нижний регистр

```
// planet.c
```

```
/* Checks planet names */
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define NUM_PLANETS 9
```

```
int main(int argc, char *argv[]){
```

```
    char *planets[] = {"Mercury", "Venus", "Earth",
```

676

```
"Mars", "Jupiter", "Saturn",  
"Uranus", "Neptune", "Pluto"};  
int i, j;  
  
for (i = 1; i < argc; i++) {  
    for (j = 0; j < NUM_PLANETS; j++)  
        if (strcmp(argv[i], planets[j]) == 0) {  
            printf("%s is planet %d\n", argv[i], j + 1);  
            break;  
        }  
    if (j == NUM_PLANETS)  
        printf("%s is not a planet\n", argv[i]);  
}  
return 0;  
}
```

Программа поочерёдно посещает каждый аргумент командной строки, сравнивая её со строками в массиве планет, пока не найдёт совпадение или не достигнет конца массива. Наиболее интересной частью программы является вызов **strcmp**, в которой аргументы **argv [i]** (указатель на аргумент командной строки) и **planets [j]** (указатель на имя планеты).

Вопросы и ответы

Вопрос: Какой длины может быть строковый литерал?

Ответ: Согласно стандарту C89, компиляторы должны позволять строковым литералам быть длиной не менее 509 символов. (Да, вы правильно прочитали—509. И не спрашивайте.)

C99 увеличивает минимальное значение до 4095 символов.

Вопрос: Почему строковые литералы не называются "строковыми константами"?

Ответ: Потому что они не обязательно постоянны. Поскольку доступ к строковым литералам осуществляется через указатели, ничто не мешает программе попытаться изменить символы в строковом литерале.

Вопрос: Как написать строковый литерал, представляющий "über", если "\xfcber " не работает?

Ответ: Секрет в том, чтобы написать два соседних строковых литерала и позволить компилятору объединить их в один. В этом примере написание " \xfc " ber " даст нам строковый литерал, представляющий слово "über".

Вопрос: Изменение строкового литерала кажется достаточно безопасным. Почему это вызывает неопределённое поведение?

Ответ: Некоторые компиляторы пытаются уменьшить требования к памяти, сохраняя отдельные копии одинаковых строковых литералов. Рассмотрим следующий пример:

```
типа char *p = "abc", *q = "abc";
```

Компилятор может выбрать для хранения строк "abc" только одну, что делает оба *p* и *q* указывающими на неё. Если бы мы изменили "abc" через указатель *p*, строка, на которую указывает *q*, также была бы затронута.

Излишне говорить, что это может привести к некоторым раздражающим ошибкам. Другая потенциальная проблема заключается в том, что строковые литералы могут храниться в области памяти "только для чтения"; программа, которая пытается изменить такой литерал, просто аварийно завершит работу.

Вопрос: Должен ли каждый массив символов содержать место для нулевого символа?

Ответ: Не обязательно, так как не каждый массив символов, используется в качестве строки. Включение места для символа null (и фактически помещение его в массив) необходимо только в том случае, если вы планируете передать его функции, для которой требуется строка с нулевым завершением.

Вам не нужен нулевой символ, если вы будете выполнять операции только с отдельными символами. Например, программа может иметь массив символов, который будет использоваться для перевода из одного набора символов в другой:

```
char translation_table[128];
```

Единственная операция, которую программа будет выполнять с этим массивом — индексация. (Значением `translation_table[ch]` будет переведённая версия символа `ch`.) Мы не рассматриваем `translation_table` как строку: она не обязательно должна содержать нулевой символ, и никакие строковые операции над ней выполняться не будут.

Вопрос: Если `printf` и `scanf` ожидают, что их первый аргумент имеет тип `char *`, означает ли это, что аргумент может быть строковой переменной вместо строкового литерала?

Ответ: Да, как показано в следующем примере:

```
char fmt [] = "%d\n";
```

```
int i;
```

```
printf(fmt, i);
```

Эта способность открывает двери для некоторых интригующих возможностей - например, чтение строки формата в качестве входных данных.

Вопрос: При попытке `printf` написать строку `str`, не могу ли я просто указать `str` как строку формата, как в следующем примере?

```
printf (str);
```

Ответ: Да, но это рискованно. Если `str` содержит символ `%`, вы не получите желаемого результата, так как `printf` предположит, что это начало спецификации преобразования.

Вопрос: Как в `read_line` определить что `getchar` не удалось прочитать символ?

Ответ: Если она не может прочитать символ, либо из-за ошибки, либо из-за конца файла `getchar` возвращает значение EOF, которое имеет тип `int`. Вот пересмотренная версия `read_line`, которая проверяет, является ли возвращаемое значение `getchar` EOF. Изменения выделены жирным шрифтом:

```
int read_line(char str[], int n){
    int ch, i = 0;
    while ((ch = getchar()) != '\n' && ch != EOF)
        if (i < n)
            str[i++] = ch;
    str[i] = '\0';
    return i;
```

Вопрос: почему `strcmp` возвращает число, которое меньше, равно или больше нуля? Кроме того, имеет ли значение точное возвращаемое значение?

Ответ: возвращаемое `strcmp` значение, вероятно, связано с тем, как традиционно пишется функция. Рассмотрим версию из книги Кернигана и Ричи "Язык программирования Си" :

```
int strcmp(char *s, char *t){
    int i;
    for (i = 0; s[i] == t[i]; i++)
        if (s[i] == '\0')
            return 0;
    return s[i] - t[i];
```

```
}
```

Возвращаемое значение - это разница между первыми символами "несоответствия" в строках *s* и *t*, которая будет отрицательной, если *s* указывает на строку "меньшую", чем *t* и положительной, если *s* указывает на строку "большую".

Однако нет никакой гарантии, что `strcmp` на самом деле написана таким образом, поэтому лучше не предполагать, что величина её возвращаемого значения имеет какое-либо конкретное значение.

Вопрос: Мой компилятор выдаёт предупреждение при попытке компиляции инструкции `while` в функции `strcat`:

```
while (*p++ = *s2++)
    ;
```

Что я делаю неправильно?

Ответ: Ничего. Многие компиляторы, но не все, каким-либо образом, вызывают предупреждение, если вы используете `=` где обычно ожидается `==`. Это предупреждение действует как минимум в 95% случаев, и это позволит вам сэкономить много времени в отладке, если вы его учтёте.

К сожалению, предупреждение не актуально в данном конкретном примере; мы на самом деле имеем в виду использование `=`, а не `==`.

Чтобы избавиться от предупреждения, перепишите цикл `while` следующим образом:

```
while ((*p++ = *s2++) != 0)    ;
```

Поскольку инструкция `while` обычно проверяет, является ли

* p ++ = * s2 ++ не 0, мы не изменили значение выражения. Однако предупреждение уходит, потому что инструкция теперь проверяет условие, а не присваивание. С компилятором GCC размещение пары круглых скобок вокруг присваивания - это ещё один способ избежать предупреждения:

```
while ((* p ++ = * s2 ++))
```

Вопрос: Являются ли функции strlen и strcat фактически написанными, как показано в Разделе 13.6?

Ответ: Возможно, хотя обычно поставщики компиляторов пишут эти функции—и многие другие строковые функции—на языке ассемблера вместо Си. Строковые функции должны выполняться как можно быстрее, так как они используются часто и приходится иметь дело со строками произвольной длины. Написание этих функций на языке ассемблера позволяет достичь высокой эффективности за счёт использования любых специальных инструкций по обработке строк, которые может предоставить ЦП.

Вопрос: Почему в стандарте C вместо "аргументов командной строки" используется термин "параметры программы"?

Ответ: Программы не всегда запускаются из командной строки. В типичном графическом интерфейсе пользователя, например, программы запускаются щелчком мыши. В такой среде нет командной строки, хотя могут быть и другие способы передачи информации в программу; термин "параметры программы" оставляет открытой дверь для этих альтернатив.

Вопрос: Должен ли я использовать имена `argc` и `argv` для параметров *main*?

Ответ: Нет. Использование имён `argc` и `argv` - это просто соглашение, а не требование языка.

Вопрос: Я видел, что `argv` объявлен как `**argv` вместо `*argv []`. Это допустимо ?

Ответ: Конечно, при объявлении параметра запись `*a` всегда совпадает с записью `a []`, независимо от типа элементов *a*.

Вопрос: Мы видели, как настроить массив, элементы которого являются указателями на строковые литералы. Существуют ли другие приложения для массивов указателей?

Ответ: Да. Хотя мы сосредоточились на массивах указателей на символные строки, это не единственное применение массивов указателей.

Мы могли бы так же легко иметь массив, элементы которого указывают на любой тип данных, будь то в форме массива или нет. Массивы указателей особенно полезны в сочетании с динамическим выделением памяти.

Упражнения

К Разделу 13.3.

1. Следующие вызовы функции предположительно пишут один символ новой строки, но некоторые неверны.

Определите, какие вызовы не работают, и объясните, почему.

(a) `printf ( "%c", '\n' );` (g) `putchar( ' \n' );`

(b) `printf("%c", "\n");` (h) `putchar("\n");`

(c) `printf("%s", '\n');` (i) `puts( '\n');`

(d) `printf ("%s", "\n");` (j) `puts("\n");`

(e) `printf ('\n');` (k) `puts ("");`

(f) `printf ("\n");`

W2. Предположим, что **p** было объявлено следующим образом:

```
char * p = "abc";
```

Какие из следующих вызовов функций являются допустимыми? Покажите результат, полученный каждым допустимым вызовом, и объясните, почему другие являются недопустимыми.

(a) `putchar(p);`

(b) `putchar(*p);`

(c) `puts (p);`

(d) `puts (*p);`

* 3. Предположим, что мы вызываем `scanf` следующим образом:

```
scanf ("%d %s %d", &i, s, &j);
```

Если пользователь вводит 12abc34 56def78, каковы будут значения *i*, *s* и *j* после вызова?

(Предположим, что *i* и *j* являются переменными *int*, а *s* - массив символов.)

W4. Измените функцию `read_line` следующим образом:

(a) Пропускать пробел перед началом хранения входных символов.

(b) Прекращать чтение с первого символа пробела.

Подсказка. Чтобы определить, является ли символ пробелом, вызовите функцию `isspace`.

(c) Прекратить чтение на первом символе новой строки, затем сохранить символ новой строки в строке

(d) Пропустить символы, для которых нет места для хранения.

К Разделу 13.4.

5. (a) Напишите функцию с именем `capitalize`, которая делает заглавными все буквы в аргументе. Аргумент будет строкой, содержащей произвольные символы, а не только буквы и оканчиваться нулевым символом.

Используйте индексацию массива для доступа к символам в строке. Подсказка: используйте функцию `toupper` для каждого символа в верхнем регистре.

(b) Перепишите функцию `capitalize`, на этот раз используя арифметику указателя для доступа к символам в строке.

W6. Напишите функцию с именем ***censor***, которая модифицирует строку, заменяя каждое появление `foo` на `xxx`. Например, строка `"food fool"` станет `"xxxd xxxl"`.

Сделайте функцию максимально короткой, не жертвуя ясностью.

К Разделу 13.5

7. Пусть `str` - массив символов. Какое из следующих утверждений не эквивалентно другим трем?

- (a) `*str = 0;`
- (b) `str[0] = '\0';`
- (c) `strcpy(satr, " ");`
- (d) `strcat(str, " ");`

W * 8. Каким будет значение строки `str` после выполнения следующих инструкций?

687

```
strcpy (str, "tire-bouchon");  
strcpy(&str[4], "de-or-wi");  
strcat (str, "red?");
```

9. Каким будет значение строки s1 после выполнения следующих инструкций?

```
strcpy(s1,"computer");  
strcpy(s2,"science");  
if (strcmp(s1, s2)< 0)  
    strcat(s1, s2);  
else  
    strcat(s2, s1);  
s1[strlen(s1) — 6]='\0';
```

W10. Следующая функция якобы создаёт идентичную копию строки. Что не так с функцией?

```
char *duplicate (const char *p)  
{  
    char *q;  
    strcpy(q, p);  
    return q;  
}
```

11. В разделе вопросов и ответов в конце этой главы показано, как функция `strcmp` может быть написана с использованием индексации массива. Измените функцию, используя арифметику указателя.

12. Напишите следующую функцию:

```
void get_extension(const char *file_name, char *extension);
```

`file_name` указывает на строку, содержащую имя файла.

Функция должна хранить расширение в имени файла в строке, на которую указывает расширение. Например, если имя файла "memo.txt", функция сохранит "txt" в строке, на которую указывает расширение.

Если имя файла не имеет расширения, функция должна хранить пустую строку (единственный нулевой символ) в строке, на которую указывает расширение.

Сконструируйте функцию как можно проще, используя функции `strlen` и `strcpy`.

13. Напишите следующую функцию:

```
void build_index_url(const char *domain, char *index_url);
```

`domain` указывает на строку, содержащую домен Интернета, например "knking.com".

Функция должна добавить "http: // www." В начало этой строки и "/index.html" в конец строки, сохраняя результат в строке, на которую указывает `index__url`.

(В этом примере результатом будет "http://www.knking.com/index.html".)

Вы можете предположить, что `index_url` указывает на переменную, которая достаточно длинна, чтобы удерживать

полученную строку. Держите функцию максимально простой, если она использует функции `strcat` и `strcpy`.

К Разделу 13.6

*14. Что печатает следующая программа?

```
#include <stdio.h>
int main (void)
{
    char s[] = "Hsjodi",*p;
    for (p = s; *p; p++)
        --*p;
    puts(s);
    return 0;
}
```

W * 15. Пусть `f` - следующая функция:

```
int f (char *s, char *t)
{
    char *p1, *p2;
    for (p1 = s; *p1; p1++){
        (for (p2 = t;*p2; p2++)
            if (*p1 == *p2) break;
        if (*p2 == '\0') break;
    }
    return p1 - s;
}
```

```
}
```

(a) каково значение `f("abod", "babe")`?

(b) какое значение `f("abod", "bod")` ?

(c) в общем случае, какое значение возвращает *f* при передаче двух строк *s* и *t*?

W16. Используйте методы раздела 13.6 для уплотнения функции `count_spaces` Раздела 13.4. В частности, замените инструкцию `for` циклом `while`.

17. Напишите следующую функцию:

```
bool test_extension(const char *file_name, const char *extension);
```

`file_name` указывает на строку, содержащую имя файла.

Функция должна возвращать значение **true**, если расширение файла совпадает с строкой, на которую указывает расширение, игнорируя регистр букв.

Например, вызов `test_extension("memo.txt", "TXT")` вернёт **true**. Включите в свою функцию идиому "поиск конца строки".

Подсказка: используйте функцию `toupper` для преобразования символов в верхний регистр перед их сопоставлением.

18. Напишите следующую функцию:

```
void remove_filename(char * url);
```

url указывает на строку, содержащую URL-адрес (Uniform Resource Locator), который заканчивается именем файла (например, как "http://www.knking.com/index.html"). Функция должна изменить строку, удалив имя файла и предыдущую косую черту.

(В этом примере результатом будет "http: //www.knking.com".) Включить в вашу функцию идиому "поиск конца строки". Подсказка: попросите функцию заменить последнюю косую черту в строке на символ null.

Проекты по программированию

1. Напишите программу, которая находит "наименьшее" и "наибольшее" в серии слов. После того, как пользователь введёт слова, программа определит, какие слова будут первыми и последними, если слова были перечислены в словарном порядке.

Программа должна прекратить приём ввода, когда пользователь вводит слово из четырёх букв. Предположим, что ни одно слово не длиннее 20 букв. Интерактивный сеанс с программой может выглядеть следующим образом:

Enter word 5103

Enter word zebra

Enter word rabbit

Enter word: catfish

Enter word: walrus

Enter word:c_a£

Enter word: fish

Smallest word: cat

Largest word: zebra

Подсказка: используйте две строки с именами `smallest_word` и `largest_word`, чтобы отслеживать "наименьшие" и "наибольшие" слова, введенные до сих пор.

Каждый раз, когда пользователь вводит новое слово, используйте `strcmp`, чтобы сравнить его с `smallest_word`, если новое слово "меньше," используйте `strcpy`, чтобы сохранить его в `smallest_word`. Сделайте ему подобное сравнение с `largest_word`. Используйте `strlen`, чтобы определить, когда пользователь ввел слово из четырех букв.

2. Улучшите программу `remind.c` раздела 13.5 следующими способами:

(a) Программа должна напечатать сообщение об ошибке и проигнорировать напоминание, если соответствующий день отрицательный или больше 31. Совет: используйте инструкцию `continue`.

(b) разрешить пользователю вводить день, 24-х часовое время и напоминание. Распечатанный список напоминаний должен быть отсортирован сначала по дням, затем по времени. (Оригинальная программа позволяет пользователю ввести время, но это рассматривается как часть напоминания.)

(c) программа должна распечатать список напоминаний на один год. Требовать от пользователя ввода дней в форме месяц / день.

3. Измените программу `deal.c` в Разделе 8.2 так, чтобы она печатала полные имена карточек, которые она имеет:

Enter number of cards in hand: 5

Your hand:

Seven of clubs

Two of spades

Five of diamonds

Ace of spades

Two of hearts

Подсказка: замените `rank_code` и `suit_code` массивами, содержащими указатели на строки.

w4. Напишите программу с именем `reverse.c`, которая перекликается с аргументами командной строки в обратном порядке. Запуск программы путём ввода

`reverse void and null`

должен производить следующий результат:

`null and void`

5. Напишите программу `sum.c`, которая складывает аргументы командной строки, которые считаются целыми числами.

694

Запуск программы путём ввода

sum 8 24 62

должен выдавать следующий результат:

Total: 94

Совет: используйте функцию `atoi` для преобразования каждого аргумента командной строки из строковой формы в целочисленную.

W6. Улучшите программу `planet.c` в Разделе 13.7, не обращая внимания на регистр символов при сравнении аргументов командной строки со строками в массиве `planets`.

7. Измените Проект по программированию 11 из Главы 5, чтобы он использовал массивы, содержащие указатели на строки, вместо инструкции `switch`.

Например, вместо использования инструкции `switch` для печати слова для первой цифры используйте цифру как индекс массива, содержащего строки "двадцать", "тридцать" и т. д.

8. Изменить Проект по программированию 5 из Главы 7 таким образом, чтобы он включал следующую функцию:

```
int compute_scrabble_value (const char *word);
```

Функция возвращает значение SCRABBLE строки, на которую указывает word.

9.Изменить Проект по программированию 10 из Главы 7 таким образом, чтобы он включал следующую функцию:

```
int compute_vowel_count ( const char *sentence);
```

Функция возвращает количество гласных в строке, на которую указывает параметр sentence.

10. Изменить Проект по программированию 11 из Главы 7 так что она включает в себя следующие функции:

```
void reverse_name (char *name);
```

Функция ожидает, что name указывает на строку, содержащую имя, за которым следует фамилия. Она изменяет строку таким образом, чтобы фамилия была первой, за ней следовали запятая, пробел, первый инициал и точка.

Исходная строка может содержать дополнительные пробелы перед именем, между именем и фамилией, а также после имени.

11. Изменить Проект по программированию 13 из Главы 7 таким образом, чтобы он включал следующую функцию:

```
double compute_average_word_length (const char *sentence);
```

Функция возвращает среднюю длину слов в строке, на которую указывает sentence.

12. Измените Проект по программированию 14 из Главы 8, чтобы он сохранял слова в двумерном массиве символов, когда он читает предложение, причём каждая строка массива хранит одно слово.

Предположим, что предложение содержит не более 30 слов, а слово не более 20 символов. Не забудьте сохранить нулевой символ в конце каждого слова, чтобы его можно было рассматривать как строку.

13. Измените Проект по программированию 15 из Главы 8 так, чтобы она включала следующую функцию:

```
void encrypt (char * message, int shift);
```

Функция ожидает, что сообщение укажет на строку, содержащую зашифрованное сообщение; shift представляет собой число, на которое должна быть сдвинута каждая буква в сообщении.

14. Измените Проект по программированию 16 из Главы 8, чтобы он включал следующую функцию:

```
bool are_anagrams (const char * word1, const char * word2);
```

697

Функция возвращает **true**, если строки, на которые указывают **word1** и **word2**, являются анаграммами.

15. Изменить Проект по программированию 6 из Главы 10 таким образом, чтобы он включал следующую функцию:

```
int evaluate_RPN_expression (const char *expression);
```

Функция возвращает значение выражения RPN, на которое указывает expression.

16. Изменить Проект по программированию 1 из Главы 12 таким образом, чтобы он включал следующую функцию:

```
void reverse (char *message);
```

Функция переворачивает строку, на которую указывает сообщение. Подсказка: используйте два указателя, один из которых изначально указывает на первый символ строки, а другой - на последний.

Пусть функция инвертирует эти символы, а затем перемещает указатели друг к другу, повторяя процесс до тех пор, пока указатели не встретятся.

17. Изменить Проект по программированию 2 из Главы 12 таким образом, чтобы он включал следующую функцию:

```
bool is_palindrome (const char *message);
```

Функция возвращает true, если строка, на которую указывает message, является палиндромом.

18. Напишите программу, которая принимает дату от пользователя в виде **mm/dd/yyyy**, а затем отображает её в виде **month dd.yyyy**, где **month** - название месяца:

Enter a date (mm/dd/yyyy): 2/17/2011

You entered the date February 17, 2011

Храните имена месяцев в массиве, содержащем указатели на строки.

14 Препроцессор

Мы всегда хотим сказать в своих программах что-то такое, что на всех известных языках можно сказать только плохо

В предыдущих главах я использовал директивы `#define` и `#include`, не вдаваясь в подробности о том, что они делают. Эти директивы—и другие, которые мы ещё не рассмотрели—обрабатываются препроцессором, частью программного

обеспечения, которая редактирует программы Си непосредственно перед компиляцией. Зависимость от препроцессора делает Си (наряду с C++) уникальным среди основных языков программирования. Препроцессор является мощным инструментом, но он также может быть источником труднодоступных ошибок.

Более того, препроцессор можно легко использовать не по назначению для создания программ, которые практически невозможно понять. Хотя некоторые программисты на Си сильно зависят от препроцессора, я рекомендую его, как и многие другие вещи в жизни, использовать умеренно.

Эта глава начинается с описания работы препроцессора (Раздел 14.1) и некоторых общих правил, влияющих на все директивы препроцессора (Раздел 14.2). Разделы 14.3 и 14.4 охватывают две основные возможности препроцессора: определение макросов и условную компиляцию.

(Я отложу подробное освещение включения файлов, другой важной особенности до Главы 15.) В Разделе 14.5 рассматриваются менее используемые директивы препроцессора: `#error`, `#line` и `#pragma`.

14.1 Как работает препроцессор

Поведение препроцессора управляется директивами препроцессора: командами, начинающимися с символа `#`.

Мы столкнулись с двумя из этих директив, `#define` и `#include`, в предыдущих главах. Директива `#define` определяет макрос—

имя, представляющее что-то другое, например, константу или часто используемое выражение.

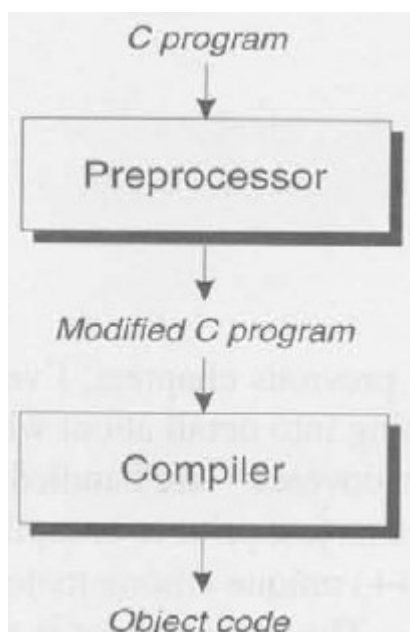
Препроцессор реагирует на директиву `#define`, сохраняя имя макроса вместе с его определением.

При последующем использовании макроса в программе препроцессор "разворачивает" макрос, заменяя его заданным значением.

Директива `#include` указывает препроцессору открыть определённый файл и "включить" его содержимое как часть компилируемого файла.

Например, строка `#include <stdio.h>` указывает препроцессору открыть файл с именем `stdio.h` и внести его содержимое в программу. (Среди прочего, `stdio.h` содержит прототипы стандартных функций ввода / вывода языка Си.)

На следующей схеме показана роль препроцессора в процессе компиляции:



Вход препроцессора - это программа на языке Си, которая может содержать директивы. Препроцессор выполняет эти

директивы, удаляя их в процессе. Выход препроцессора - это ещё одна программа на Си: отредактированная версия исходной программы, не содержащая директив.

Выход препроцессора поступает непосредственно в компилятор, который проверяет программу на наличие ошибок и переводит её в объектный код (машинные инструкции).

Чтобы узнать, что делает препроцессор, давайте применим его к программе celsius.c раздела 2.6. Вот оригинальная программа:

```
/* Converts a Fahrenheit temperature to Celsius */
```

```
#include <stdio.h>
```

```
#define FREEZING_PT 32.0f
```

```
#define SCALE_FACTOR (5.0f / 9.0f)
```

```
int main(void)
```

```
{
```

```
    float fahrenheit, celsius;
```

```
    printf("Enter Fahrenheit temperature: ");
```

```
    scanf("%f", &fahrenheit);
```

```
    celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;
```

702

```
printf("Celsius equivalent: %.1f\n", celsius);

return 0;
}
```

После предварительной обработки программа будет выглядеть следующим образом:

```
Пустая строка
Пустая строка
Строки, привлечённые из stdio.h
Пустая строка
Пустая строка
Пустая строка
Пустая строка
int main(void)
{
    float fahrenheit, celsius;
```

```
    printf("Enter Fahrenheit temperature: ");
    scanf("%f", &fahrenheit);
```

```
    celsius = (fahrenheit - 32.0f) * (5.0f / 9.0f);
```

```
    printf("Celsius equivalent is: %.1f\n", celsius);
```

```
    return 0;
```

```
}
```

Препроцессор ответил на директиву `#include`, введя содержимое `stdio.h`. Препроцессор также удалил директивы `#define` и заменил `FREEZING_PT` и `SCALE_FACTOR` везде, где они появлялись позже в файле.

Обратите внимание, что препроцессор не удаляет строки, содержащие директивы, а просто делает их пустыми. Как показывает этот пример, препроцессор выполняет немного больше, чем просто выполняет директивы.

В частности, он заменяет каждый комментарий одним пробелом. Некоторые препроцессоры идут дальше и удаляют лишние пробельные символы, включая пробелы и табуляцию в начале строк с отступами. На заре Си препроцессор был отдельной программой, которая передавала свои выходные данные в компилятор.

В настоящее время препроцессор часто является частью компилятора и часть его выхода необязательно может быть Си-кодом. (Например, включение стандартного заголовка, такого как `<stdio.h>`, может привести к тому, что его функции будут доступны для программы, не обязательно копируя содержимое заголовка в исходный код программы.)

Тем не менее, полезно думать о препроцессоре отдельно от компилятора.

Фактически, большинство компиляторов Си предоставляют способ просмотра выходных данных препроцессора.

Некоторые компиляторы генерируют вывод препроцессора, когда задана определённая опция (GCC будет делать это, когда используется опция `-E`). Другие имеют отдельную программу,

которая ведёт себя как интегрированный препроцессор. Дополнительную информацию см. в документации вашего компилятора.

Предупреждение: препроцессор имеет только ограниченное знание Си. В результате он вполне способен создавать недопустимые программы в процессе того как он выполняет директивы. Часто оригинальная программа выглядит нормально, затрудняя поиск ошибок. В сложных программах изучение выходных данных препроцессора может оказаться полезным для обнаружения такого рода ошибок.

14.2. Директивы по предварительной обработке

Большинство препроцессорных директив относятся к одной из трёх категорий:

■ Макроопределение.

Директива `#define` определяет макрос; директива `#undef` удаляет макроопределение.

■ Включение файла.

Директива `#include` приводит к включению содержимого специфицированного файла в программу.

■ Условная компиляция.

Директивы `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else` и `#endif` позволяют включать или исключать блоки текста в программе, в зависимости от условий, которые могут быть проверены

препроцессором.

Остальные директивы - `#error`, `#line` и `#pragma` - более специализированы и поэтому используются реже. Мы рассмотрим оставшуюся часть этой главы для углублённого изучения директив предварительной обработки.

Единственная директива, которую мы не будем обсуждать подробно, - `#include`, так как она будет раскрыта в Разделе 15.2. Прежде чем идти дальше, давайте рассмотрим несколько правил, применимых ко всем директивам:

■ Директивы всегда начинаются с символа `#`.

Символ `#` не обязательно должен быть в начале строки, при условии, что перед ним только пробельные символы. После `#` идёт имя директивы, сопровождаемое любой другой информацией, которую запрашивает директива.

Токены в директиве могут разделяться любым количеством пробелов и горизонтальных символов табуляции.

Например, следующая директива является допустимой:

```
# define N          100
```

■ Директивы всегда заканчиваются на первом символе новой строки, если явно не продолжены. Чтобы перейти на следующую строку, нужно закончить текущую строку символом `\`.

Например, следующая директива определяет макрос, представляющий ёмкость жесткого диска, измеренную в байтах:

```
#define DISK_CAPACITY (SIDES *  
                        TRACKS_PER_SIDE *  
                        SECTORS_PER_TRACK *  
                        BYTES_PER_SECTOR)
```

Директивы могут появляться в любом месте программы. Хотя мы обычно помещаем директивы `#define` и `#include` в начало файла, другие директивы, скорее всего, появятся позже, даже в середине определений функций.

Комментарии могут отображаться в той же строке, что и директива.

На самом деле, весьма уместно, поместить комментарий в конце определения макроса объясняя смысл макроса:

```
#define FREEZING_PT 32.05 /*freezing point of water */
```

14.3 Определение макроса

Макросы, которые мы используем с Главы 2, известны как простые макросы, потому что у них нет параметров.

Препроцессор также поддерживает параметризованные макросы. Сначала мы рассмотрим простые макросы, а затем параметризованные макросы.

После изучения их по отдельности, мы рассмотрим свойства, общие для обоих.

Простые макросы

Определение простого макроса (или объектно-подобного макроса, как его называют в стандарте Си) имеет вид:

```
#define identifier replacement-list (список замены)
```

replacement-list-это любая последовательность токенов препроцессорной обработки, которые аналогичны токенам, описанным в Разделе 2.8. Всякий раз, когда мы используем термин "токен" в этой главе, это означает "токен препроцессорной обработки."

Список замен макроса может включать идентификаторы, ключевые слова, числовые константы, символьные константы, строковые литералы, операторы и знаки препинания. При обнаружении определения макроса препроцессор отмечает, что идентификатор представляет список замен; везде, где идентификатор появляется позже в файле, препроцессор заменяет его списком замен.

Не добавляйте лишние символы в определение макроса—они станут частью списка замен. Размещение символа = в определении макроса является распространённой ошибкой:

```
#define N = 100 /*** WRONG ***/
```

```
....
```

```
int a[N]; /* превращается в int a[= 100]; */
```

В этом примере мы (неправильно) определили *N* как пару токенов (= и **100**). Другой распространённой ошибкой является завершение определения макроса точкой с запятой:

```
#define N 100; /**WRONG **/  
int a[N]; /*превращается в int a[100;]; */
```

Здесь *N* определяется как токены **100** и **;**. Компилятор обнаруживает большинство ошибок, вызванных дополнительными символами в макроопределении. К сожалению, компилятор отметит каждое использование макроса как неправильное, вместо того, чтобы идентифицировать фактического виновника - определение макроса - которое будет удалено препроцессором. Простые макросы в основном используются для определения того, что Керниган и Ричи называют "манифестными константами". Используя макросы, мы можем присвоить имена числовым, символьным и строковым значениям:

```
#define STR_LEN 80  
#define TRUE 1  
#define FALSE 0  
#define PI 3.14159  
#define CR '\r'  
#define EOS '\0'  
#define MEM_ERR "Error: not enough memory"
```


Использование `#define` для создания имён для констант имеет несколько существенных преимуществ:

■ ***Упрощает чтение программ.*** Имя макроса, если оно хорошо выбрано, позволяет читателю понять смысл константы. Альтернативой является программа, полная "магических чисел", которые могут легко мистифицировать читателя.

■ ***Упрощает модификацию программ.*** Мы можем изменить значение константы во всей программе, изменив одно макроопределение. "Жёстко закодированные" константы более трудно изменить, тем более, что они иногда появляются в слегка изменённой форме.

(Например, программа с массивом длиной 100 может иметь цикл, который идёт от 0 до 99. Если мы просто попытаемся найти вхождения 100 в программе, мы пройдем мимо 99.)

■ ***Помогает избежать несоответствий и опечаток.*** Если числовая константа, такая как 3.14159, появляется много раз в программе, она может быть случайно записана как 3.1416 или 3.14195.

Хотя простые макросы чаще всего используются для определения имён констант, у них есть другие приложения:

■ ***Внесение незначительных изменений в синтаксис Си.*** Мы можем - по существу - изменить синтаксис Си путём определения макросов, которые служат альтернативными

именами для символов Си. Например, программисты, которые предпочитают имена Pascal `begin` и `end` для фигурных скобок `{и}` в Си, могут определять следующие макросы:

```
#define BEGIN {  
#define END }
```

Мы можем пойти так далеко, что изобретём свой собственный язык. Например, мы могли бы создать "инструкцию" `LOOP`, которая устанавливает бесконечный цикл:

```
#define LOOP for (;;) 
```

Изменение синтаксиса Си обычно плохая идея, так как это может затруднить понимание программы другими пользователями.

■ **Переименование типов.** В Разделе 5.2. мы создали булевский тип, переименовав `int`:

```
#define BOOL int
```

Хотя некоторые программисты используют макросы для этой цели, ***typedef*** лучший способ определения имён типов.

■ **Управление условной компиляцией.** Макросы играют важную роль в управлении условной компиляцией, как мы увидим в Разделе 14.4. Например, наличие следующей строки в программе может означать, что она должна быть скомпилирована в "режиме отладки", с дополнительными

инструкциями, включёнными для получения отладочной информации:

```
#define DEBUG
```

Кстати, как показывает этот пример, допустим пустой список замен макроса.

Когда макросы используются как константы, программисты Си обычно задают прописными все буквы в их именах. Тем не менее, нет соглашения относительно того, как использовать заглавные буквы в макросах, используемых для других целей. Поскольку макросы (особенно параметризованные макросы) могут быть источником ошибок, некоторые программисты хотели бы обратить на них внимание, используя в своих именах все прописные буквы. Другие предпочитают имена в нижнем регистре, следуя стилю книги Кернигана и Ричи "Язык программирования Си".

Параметризованные макросы

Определение параметризованного макроса (также называемого функционально подобным макросом) имеет вид

```
#define identifier(x1, x2,... xn) replacement-list
```

где $x_1, x_2, \dots, x_n$ - идентификаторы (параметры макроса). Параметры может быть столько, сколько их требуется в списке замены.

Между именем макроса и левой скобкой не должно быть пробелов. Если пробел имеется, препроцессор будет считать, что мы определяем простой макрос; он будет обрабатывать $(x_1, x_2, \dots, x_n)$ как часть списка замещения.

Когда препроцессор сталкивается с определением параметризованного макроса, он сохраняет это определение для последующего использования. Везде, где вызов макрокоманды формы *identifier*($y_1, y_2, \dots, y_n$) появляется позже в программе (где $(y_1, y_2, \dots, y_n)$ - последовательности токенов), препроцессор заменяет его списком замен, заменяя y_1 на x_1 , y_2 на x_2 и т. д.

Например, предположим, что мы определили следующие макросы:

```
#define MAX(x,y) ((x) > (y)?(x):(y))
#define IS_EVEN(n) ((n)%2==0)
```

(Количество скобок в этих макросах может показаться чрезмерным, но есть причина, как мы увидим далее в этом разделе.)

Теперь предположим, что мы вызываем два макроса следующим образом:

```
i = MAX(j+k, m-n);
if (IS__EVEN(i)) i++;
```

Препроцессор заменит эти строки на:

```
i = ((j+k)>(m-n)?(j+k):(m—n));
if (((i)%2==0))i++;
```

Как показывает этот пример, параметризованные макросы часто служат простыми функциями.

MAX ведёт себя как функция, которая вычисляет большее из двух значений.

IS_EVEN ведёт себя как функция, которая возвращает 1, если её аргумент является чётным числом и 0 в противном случае.

Вот более сложный макрос, который ведёт себя как функция:

```
#define TOUPPER(c) ('a' <= (c) && (c) <= 'z' ? (c) - 'a' + 'A' : (c))
```

Этот макрос проверяет, находится ли символ *c* между 'a' и 'z'. Если это так, он создаёт версию с верхним регистром *c* путём вычитания "a" и добавления "A". Если нет, то *c* остаётся неизменным. (Заголовок `<ctype.h>` предоставляет аналогичную функцию с именем `toupper`, которая более переносима.)

Макрос с параметрами может иметь пустой список параметров. Вот пример:

```
#define getchar() getc(stdin)
```

Пустой список параметров действительно не нужен, но делает `getchar` похожим на функцию. (Да, это та же `getchar`, которая относится к `<stdio.h>`. В Разделе 22.4 мы увидим, что `getchar` обычно реализуется как макрос, а также как функция.)

Использование параметризованного макроса вместо настоящей функции имеет несколько преимуществ:

■ **Программа может быть немного быстрее.** Вызов функции обычно требует некоторых накладных расходов во время выполнения программы — информация контекста должна быть сохранена, скопированы аргументы и т. д. Вызов макроса, с другой стороны, не требует дополнительных издержек во время выполнения.

(Обратите внимание, что встроенные функции C99 предоставляют способ избежать этих издержек без использования макросов.)

■ Макросы "универсальны". Параметры макроса, в отличие от параметров функции, не имеют определённого типа. В результате макрос может принимать аргументы любого типа при условии, что результирующая программа — после работы препроцессора - действует.

Например, мы могли бы использовать макрос `MAX` для определения большего из двух значений типа `int`, `long`, `float`, ***double*** и т. д.

Но параметризованные макросы также имеют недостатки:

■ Скомпилированный код часто будет больше. Каждый вызов макроса вызывает вставку списка замены макроса, тем самым увеличивая размер исходной программы (и, следовательно, скомпилированного кода). Чем чаще используется макрос, тем более выражен этот эффект. Проблема усугубляется, когда макрос-вызовы вложены. Подумайте, что произойдёт, когда мы используем `MAX` для получения наибольшего из трёх чисел:

```
n = MAX(i, MAX(j,k);
```

Вот та же инструкция после препроцессорной обработки:

```
n = ((i) > (((j) > (k) ? (j) : (k))) ? (i) : (((j) > (k) ? (j) : (k))));
```

■ **Аргументы не проверяются по типу.** Когда вызывается функция, компилятор проверяет каждый аргумент, чтобы узнать, имеет ли он соответствующий тип. Если нет, либо аргумент преобразуется в правильный тип, либо компилятор выдаёт сообщение об ошибке. Макро-аргументы не проверяются препроцессором и не преобразуются.

■ **Невозможно иметь указатель на макрос.** Как мы увидим в Разделе 17.7, Си позволяет указатели на функции, что весьма полезно в некоторых ситуациях программирования. Макросы удаляются во время предварительной обработки, поэтому нет соответствующего понятия "указатель на макрос", в результате макросы не могут использоваться в подобных ситуациях.

Макрос может вычислять свои аргументы более одного раза. Функция вычисляет свои аргументы только один раз; макрос может вычислять свои аргументы два или более раз. Вычисление аргументов более одного раза может привести к непредвиденному поведению, если аргумент имеет побочные эффекты. Рассмотрим, что произойдёт, если один из аргументов MAX имеет побочный эффект:

```
n = MAX(i++, j);
```

Вот та же строка после препроцессорной обработки:

```
n = ((i++)>(j)?(i++):(j));
```

Если *i* больше *j*, тогда *i* будет (неправильно) увеличен дважды, а *n* будет присвоено непредвиденное значение.

Ошибки, вызванные вычислением аргумента макроса более одного раза, может быть трудно найти, так как вызов макроса выглядит так же, как вызов функции. Хуже того, макрос может работать должным образом большую часть времени, ошибаясь только на некоторых аргументах, которые имеют побочные эффекты. Для самозащиты, рекомендуется избегать побочных эффектов в аргументах.

Параметризованные макросы хороши не только для имитации функций. В частности, они часто используются в качестве шаблонов для сегментов кода, которые многократно повторяются. Предположим, что мы устали писать

```
printf("%d\n",i);
```

каждый раз, когда нам нужно печатать целое число *i*. Мы можем определить следующий макрос, который упрощает отображение целых чисел:


```
#define PRINT_INT (n) printf("%d\n", n)
```

Как только PRINT_INT будет определён, препроцессор превратит строку

```
PRINT_INT( i/j );
```

в

```
printf ( "%d\n", i/j);
```

Оператор

Макроопределения могут содержать два специальных оператора: # и ##.

(Оператор # помещает аргумент, перед которым он стоит, в двойные кавычки.)

Оператор # преобразует аргумент макрокоманды в строковый литерал; он может появиться только в списке замены параметризованного макроса. (Операция, выполняемая #, называется "stringization" (стрингификация), термин, который, я уверен, вы не найдёте в словаре.)

Существует несколько применений для #; давайте рассмотрим только одно. Предположим, что мы решили использовать макрос PRINT_INT во время отладки как удобный способ печати значений целых переменных и выражений. Оператор #

позволяет PRINT_INT маркировать каждое значение, которое он печатает. Вот наша новая версия PRINT_INT:

```
#define PRINT_INT(n) printf(#n " = %d\n", n)
```

Оператор `#` перед *n* инструктирует препроцессор создать строковый литерал из аргумента PRINT_INT. Таким образом, вызов

```
PRINT_INT (i / j);
```

станет

```
printf ("i / j " = % d \ n", i / j);
```

В Разделе 13.1 мы видели, что компилятор автоматически соединит соседние строковые литералы, поэтому эта инструкция эквивалентна

```
printf ("i / j = % d \ n", i / j);
```

Когда программа будет выполнена, *printf* отобразит как выражение *i / j*, так и его значение. Если *i* равно **11**, а *j* равно **2**, то для

Например, выход будет

```
i / j = 5
```

Оператор

Оператор `##` может "склеить" два токена (например идентификаторы,) вместе, чтобы сформировать один токен. (Не удивительно, что `##` известен как "склеиватель токенов".) Если один из операндов является макропараметром, склейка происходит после замены параметра соответствующим аргументом. Рассмотрим следующий макрос:

```
#define MK_ID(n) i##n
```

Когда `MK_ID` вызывается (например, `MK_ID (1)`), препроцессор сначала заменяет параметр *n* аргументом (**1** в этом случае). Затем препроцессор соединяет *i* и **1**, чтобы сделать один токен (`i1`). В следующей декларации используется `MK_ID` для создания трёх идентификаторов:

```
int MK_ID (1), MK_ID (2), MK_ID (3)
```

После препроцессорной обработки эта объявление становится

```
int i1, i2, i3;
```

Оператор `##` не является одной из наиболее часто используемых функций препроцессора; на самом деле, сложно вообразить ситуации, требующие этого. Чтобы найти реалистичное приложение `##`, давайте пересмотрим макрос

MAX, описанный ранее в этом разделе. Как мы видели, MAX не ведёт себя должным образом, если его аргументы имеют побочные эффекты. Альтернативой использованию макроса MAX является запись функции `max`. К сожалению, одной функции `max` обычно недостаточно; нам может понадобиться функция `max`, аргументы которой являются значениями `int`, аргументы которой являются значениями `float` и т. д. Все эти версии `max` были бы идентичны, кроме типов аргументов и типа возврата, поэтому было бы досадно определять каждую из них с нуля. Решение состоит в том, чтобы написать макрос, который расширяется до определения функции `max`. Макрос будет иметь один параметр, тип, представляющий тип аргументов и возвращаемое значение. Есть только одна загвоздка: если мы используем макрос для создания более одной функции `max`, программа не будет компилироваться. (Си не позволяет двум функциям иметь одно и то же имя, если обе они определены в одном файле).

Для решения этой проблемы, мы будем использовать оператор `#`, чтобы создать другое имя для каждой версии `max`. Вот как будет выглядеть макрос:

```
#define GENERIC_MAX(type) \
type type##_max(type x, type y) \
{ \
    return x > y ? x : y; \
}
```

Обратите внимание, как тип соединяется с `_max`, чтобы сформировать имя функции. Предположим, что нам

понадобится функция `max`, которая работает с значениями `float`. Вот как мы будем использовать `GENERIC_MAX` для определения функции:

`GENERIC_MAX (float)`

Препроцессор развернёт эту строку в следующий код:

```
float float_max(float x, float y) {return x > y ? x : y;}
```

Общие свойства макросов

Теперь, когда мы обсудили как простые, так и параметризованные макросы, давайте рассмотрим некоторые правила, которые применяются к тем и другим:

■ ***Список замен макроса может содержать вызовы других макросов.*** Например, мы могли бы определить макрос `TWO_PI` в терминах макроса `PI`:

```
#define PI 3.14159
```

```
#define TWO_PI (2*PI)
```

Когда препроцессор встречает `TWO_PI` позже в программе, то заменяет его на `(2 * PI)`. Затем препроцессор пересканирует список замен, чтобы увидеть, содержит ли он вызовы других макросов (в этом случае `PI`). Препроцессор будет

пересканировать список замен столько раз, сколько необходимо, чтобы устранить все имена макросов.

■ **Препроцессор заменяет только целые токены, а не части токенов.** В результате препроцессор игнорирует имена макросов, которые встроены в идентификаторы, символьные константы и строковые литералы. Например, предположим, что программа содержит следующие строки:

```
#define SIZE 256  
int BUFFER_SIZE;  
if (BUFFER_SIZE > SIZE)  
    puts("Error: SIZE exceeded");
```

Идентификатор `BUFFER_SIZE` и строка `"Error: SIZE exceeded"` не были подвержены предварительной обработке, хотя оба содержат слово `SIZE`.

■ Макроопределение обычно остаётся в силе до конца файла, в котором он появляется. Поскольку макросы обрабатываются препроцессором, они не соблюдают нормальные правила области видимости. Макроопределение, определяемое внутри тела функции, не является локальным для этой функции; оно остаётся определённым до конца файла.

■ Макрос не может быть определён дважды, если новое определение не совпадает со старым. Различия в интервалах разрешены, но токены в списке замены макроса (и параметры, если они есть) должны быть одинаковыми. Макросы могут быть "деопределены" директивой `#undef`.

Директива `#undef` имеет форму

`#undef identifier`

где `identifier` - это макрокоманда. Например, директива

`#undef N`

удаляет текущее определение макроса *N*. (Если *N* не был определён как макрос, директива `#undef` не имеет эффекта)
Один вариант использования макроса `#undef` заключается в удалении существующего определения макроса, так что ему может быть дано новое определение.

Скобки в определениях макросов

Списки замены в наших макроопределениях полны круглых скобок. Неужели их так много нужно?

Ответ - решительное да; если мы используем меньше круглых скобок, макросы будут иногда давать неожиданные и нежелательные результаты.

При определении того, где расположить скобки в макроопределении, необходимо следовать двум правилам.

Во-первых, если список замены макроса содержит оператор, всегда вставляйте список замен в круглые скобки:

`#define TWO_PI (2*3.14159)`

Во-вторых, если макрос имеет параметры, поместите круглые скобки вокруг каждого параметра каждый раз, когда он появится в списке замен:

```
#define SCALE(x) ((x)*10)
```

Без круглых скобок мы не можем гарантировать, что компилятор будет обрабатывать списки замен и аргументы как целые выражения. Компилятор может применять правила приоритета и ассоциативности операторов неожиданными способами. Чтобы проиллюстрировать важность размещения круглых скобок вокруг списка замены макроса, рассмотрите следующее макроопределение, в котором отсутствуют скобки:

```
#define TWO_PI 2*3,14159
```

```
/* требуется скобки вокруг списка замен */
```

Во время препроцессорной обработки инструкция
`conversion_factor = 360 / TWO_PI;`

становится

```
conversion_factor = 360/2 * 3.14159;
```

Деление будет выполняться до умножения, что даст результат, отличный от того, который предполагался. Недостаточно поместить круглые скобки вокруг списка замен, если у макроса есть параметры - для каждого вхождения параметра нужны скобки. Например, предположим, что `SCALE` определяется следующим образом:


```
#define SCALE (x) (x * 10)
    /* требуется скобки вокруг x */
```

Во время препроцессорной обработки инструкция

```
j = SCALE (i + 1);
```

становится

```
j = (i + 1 * 10);
```

Поскольку умножение имеет приоритет над сложением, эта инструкция эквивалентна

```
j = i + 10;
```

Конечно, мы хотели, чтобы $j = (i + 1) * 10$;

Недостаток скобок в макроопределении может вызвать некоторые из самых разочаровывающих ошибок Си. Программа, как правило, компилируется и макрос будет казаться работающим, переставая работать только в наименее удобное время.

Создание более длинных макросов

Оператор запятая может быть полезен для создания более сложных макросов, позволяя нам сделать списком замен серию выражений. Например, следующий макрос прочитает строку, а затем напечатает её:

```
#define ECHO(s) (gets(s), puts(s))
```

Вызовы `get` и `puts` являются выражениями, поэтому вполне логично комбинировать их с помощью оператора запятая. Мы можем вызвать `ECHO`, как если бы это была функция:

```
ECHO(str); /* получим (gets(str), puts(str)); */
```

Вместо использования оператора запятая в определении `ECHO` мы могли бы заключить вызовы `get` и `puts` в фигурные скобки, чтобы сформировать составную инструкцию:

```
#define ECHO(s) { gets(s); puts(s); }
```

К сожалению, этот метод не работает. Предположим, что мы используем `ECHO` в выражении `if`:

```
if (echo_flag)
    ECHO(str);
```

```
else gets(str);
```

Замена ECHO даёт следующий результат:

```
if (echo_flag)
    { gets(str); puts(str); };
else
    gets(str);
```

Компилятор рассматривает первые две строки как полную инструкцию if:

```
if (echo_flag)
    { gets(str); puts(str); };
```

Он обрабатывает точку с запятой, которая следует за инструкцией null и выдаёт сообщение об ошибке для предложения else, поскольку оно не принадлежит никакому if.

Мы могли бы решить эту проблему, не забывая не ставить точку с запятой после каждого вызова ECHO, но тогда программа выглядела бы необычно. Оператор запятая решает эту проблему для ECHO, но не для всех макросов.

Предположим, что макрос должен содержать ряд инструкций, а не только ряд выражений. Оператор запятая не помогает: он может связывать выражения, но не инструкции. Решение состоит в том, чтобы обернуть инструкции в цикле do,

условие которого является ложным (и поэтому будет выполнено только один раз):

```
do { ... } while (0)
```

Обратите внимание, что инструкция `do` не является завершённой - ей нужна точка с запятой в конце. Чтобы увидеть этот трюк (гм!, метод) в действии, давайте включим его в наш макрос `ECHO`:

```
#define ECHO(s)  \
do {            \
    gets (s);    \
    puts (s);    \
} while (0)
```

Когда используется `ECHO`, за ним следует точка с запятой, которая завершает инструкцию `do`:

```
ECHO(str);
/* получаем do { gets(str); putststr); } while (0); */
```

Предопределённые макросы

Си имеет несколько заранее заданных макросов. Каждый

макрос представляет собой целочисленную константу или строковый литерал. Как видно из таблицы 14.1, эти макросы предоставляют информацию о текущей компиляции или о самом компиляторе.

Название	Описание
<code>__LINE__</code>	Номер строки скомпилированного файла
<code>__FILE__</code>	Имя скомпилированного файла
<code>__DATE__</code>	Дата компиляции (в форме "mm dd yyyy")
<code>__TIME__</code>	Время компиляции (в форме "hh: mm: ss")
<code>__STDC__</code>	1, если компилятор соответствует стандарту Си (C89 или C99)

Макросы `__DATE__` и `__TIME__` определяют, когда была скомпилирована программа. Например, предположим, что программа начинается со следующих инструкций:

```
printf("Wacky Windows (c) 2010 Wacky Software, Inc.\n");  
printf("Compiled on %s at: %s\n", __DATE__, __TIME__);
```

Каждый раз, когда она начинает выполняться, программа печатает две строки формы:

```
Wacky Windows (c) 2010 Wacky Software, Inc.  
Compiled on Dec 23 2010 at 22:18:48
```

Эта информация может быть полезна для отличия различных версий одной и той же программы.

Мы можем использовать макросы `_LINE_` и `_FILE_`, как помощь в поисках ошибок. Рассмотрим проблему обнаружения местоположения деления на ноль. Когда программа Си заканчивается преждевременно, потому что она делит на ноль, обычно нет признаков того, какое деление вызвало проблему. Следующий макрос может помочь нам определить источник ошибки:

```
#define CHECK_ZERO(divisor) \  
if (divisor == 0) \  
    printf("*** Попытка деления на ноль в строке %d " \  
        " файла %s ***\n", __LINE__, __FILE__)
```

Макрос `CHECK_ZERO` будет вызываться до деления:

```
CHECK_ZERO(j);  
k = i / j;
```

Если `j` окажется равным нулю, будет напечатано сообщение следующего вида:

```
*** Попытка деления на ноль в строке 9 файла foo.c ***
```

Макросы обнаружения ошибок, подобные этому, весьма полезны. Фактически, библиотека Си имеет универсальный

макрос обнаружения ошибок с именем `assert`.

Макрос `__STDC__` существует и имеет значение 1, если компилятор соответствует стандарту Си (либо C89, либо C99). Благодаря тому, что препроцессор проверяет этот макрос, программа может адаптироваться к компилятору, который предшествует стандарту C89 (см. например Раздел 14.4).

Дополнительные предопределённые макросы в C99

C99 предоставляет несколько дополнительных заранее заданных макросов (таблица 14.2)

Имя	Описание
<code>__STDC_HOSTED__</code>	1, если это хостинг-реализация; 0, если она является автономной
<code>__STDC_VERSION__</code>	Поддерживается версия стандарта Си
<code>__STDC_IEC_559__*</code>	1, если поддерживается стандартная арифметика IEC60559

<code>__STDC_IEC_559__COMPLEX__</code> *	1, если поддерживается комплексная арифметика IEC60559
<code>__STDC_ISO_10646__</code> *	ууууmmL, если значения <code>wchar_t</code> соответствуют стандарту ISO10646 указанного года и месяца

* Условно определено

Чтобы понять значение `__STDC_HOSTED__`, нам нужен некоторый новый словарь. Реализация Си состоит из компилятора плюс другое программное обеспечение, необходимое, чтобы выполнить программы Си. C99 делит реализации на две категории: хост и автономную.

(Хост-реализация - это среда, которая предоставляет стандартные библиотеки)

Хост-реализация должна принимать любую программу, которая соответствует стандарту C99, тогда как автономной реализации не требуется компилировать программы, которые используют сложные типы или стандартные заголовки, помимо нескольких основных. (В частности, автономная реализация не должна поддерживать заголовков `<stdio.h>`.) Макрос `__STDC_HOSTED__` представляет собой константу 1, если компилятор является хост-реализацией; в противном случае макрос имеет значение 0.

`__STDC_VERSION__` макро предоставляет возможность

проверить, какая версия стандарта Си распознаётся компилятором. Этот макрос впервые появился в Поправке 1 к стандарту C89, где его значение было определено как целочисленная константа `long 199409L` (представляющая год и месяц поправки). Если компилятор соответствует стандарту C99, значение равно `199901L`. Для каждой последующей версии стандарта (и каждой поправки к стандарту) этот макрос будет иметь другое значение. Компилятор C99 может (или не может) определять три дополнительных макроса.

Каждое макроопределение определяется только в том случае, если компилятор удовлетворяет определённому требованию:

- `__STDC_IEC_559__` определяется (и имеет значение 1), если компилятор выполняет арифметику с плавающей точкой в соответствии со стандартом IEC 60559 (другое имя для стандарта IEEE 754).

- `__STDC_IEC_559_COMPLEX__` определяется (и имеет значение 1), если компилятор выполняет арифметику комплексных чисел в соответствии со стандартом IEC 60559.

- `__STDC_ISO_10646__` определяется как целочисленная константа формы `уууutmL` (например, `199712L`), если значения типа `wchar_t` представлены кодами в стандарте ISO / IEC 10646 (с изменениями по конкретному году и месяцу).

Пустые аргументы макроса

C99 позволяет любому или всем аргументам в макроса быть пустым. Однако такой вызов будет содержать такое же

количество запятых, как и обычный вызов. (Таким образом, легко увидеть, какие аргументы опущены.) В большинстве случаев эффект пустого аргумента ясен. Там, где в списке замен появляется соответствующее имя параметра, оно не заменяется ничем - оно просто исчезает из списка замен. Вот пример:

```
#define ADD(x y) (x+y)
```

После препроцессора инструкция

```
i = ADD(j,k);
```

становится

```
i = (j+k);
```

тогда как инструкция

```
i = ADD(,k);
```

становится

```
i = (+k);
```

Когда пустой аргумент является операндом операторов `#` или `##`, применяются специальные правила. Если пустой аргумент **"stringized"** (стрингифицируется) с помощью оператора `#`, результатом является `" "` (пустая строка):

```
#define MK_STR(x) #x
```

...

```
char empty_string[] = MK_STR();
```

После препроцессорной обработки декларация будет выглядеть следующим образом:

```
char empty_string [] = "";
```

Если один из аргументов оператора `##` пуст, он заменяется невидимым токеном "метка места". Конкатенация обычного токена с токеном метки места даёт исходный токен (метка места исчезает), если два токена метка места объединены, результатом является одна метка места. После того, как макрос завершён, токены "метка места" исчезают из программы. Рассмотрим следующий пример:

```
#define JOIN(x,y,z) x##y##z
```

```
...
```

```
int JOIN(a,b,c), JOIN(a,b), JOIN(a,,c),JOIN(,,c);
```

После препроцессорной обработки декларация будет выглядеть следующим образом:

```
int abc, ab, ac, c;
```

Пропущенные аргументы были заменены токенами метки места, которые затем исчезли при объединении с любыми непустыми аргументами. Все три аргумента макроса **JOIN** могут даже отсутствовать, что дало бы пустой результат.

Макросы с переменным числом аргументов

В C89 макрос должен иметь фиксированное количество аргументов, если оно вообще есть. C99 немного ослабляет ситуацию, позволяя макросам принимать неограниченное количество аргументов. Эта возможность уже давно доступна для функций, поэтому неудивительно, что макросы были окончательно вставлены на равную основу. Основная причина наличия макроса с переменным числом аргументов заключается в том, что он может передать эти аргументы функции, которая принимает переменную количество аргументов, таких как `printf` или `scanf`. Вот пример:

```
#define TEST(condition, ... ) ((condition)? \
    printf("Passed test: %s \n", #condition) : \
    printf (_VA_ARGS_))
```

Токен... известный как многоточие (ellipsis), идет в конце списка параметров макроса, которому предшествуют обычные параметры, если они есть. VA_ARGS - специальный идентификатор, который может отображаться только в списке замен макроса с переменным числом аргументов; он представляет все аргументы, соответствующие эллипсису. (Должен быть хотя бы один аргумент, соответствующий эллипсису, хотя этот аргумент может быть пустым.) Макрос `TEST` требует не менее двух аргументов. Первый аргумент соответствует параметру `condition`; остальные аргументы

соответствуют эллипсису. Вот пример, показывающий, как можно использовать макрос TEST:

```
TEST(voltage <= max_voltage,  
     "Voltage %d exceeds %d\n", voltage, max_voltage);
```

Препроцессор будет выдавать следующий результат (переформатированный для удобочитаемости):

```
((Voltage <= max_voltage)?  
printf("Passed test: %s\n", "voltage <= max_voltage"):  
printf("Voltage %d exceeds %d\n", voltage, max_voltage) );
```

Когда программа будет выполнена, программа отобразит сообщение

```
Passed test: voltage <= max_voltage  
(прошёл тест: напряжение <= max_voltage )
```

если напряжение не больше максимального напряжения. В противном случае будут отображаться значения напряжения и max_voltage:

```
Voltage 125 exceeds 120 (Напряжение 125 превышает 120)
```

Идентификатор `__func__`

ещё одной новой особенностью C99 является идентификатор `__func__`.

`__func__` не имеет ничего общего с препроцессором, поэтому на самом деле он не надлежит рассмотрению этой главе.

Однако, как и многие функции препроцессора, он полезен для отладки, поэтому я решил обсудить его здесь. Каждая функция имеет доступ к идентификатору `__func__`, который ведёт себя как строковая переменная, в которой хранится имя текущей исполняемой функции. Эффект такой же, как если бы каждая функция содержала следующее объявление в начале своего тела:

```
static const char _func_[] = "function-name";
```

где `function-name` - это имя функции. Существование этого идентификатора позволяет писать макросы отладки, такие как:

```
#define FUNCTION_CALLED() printf("%s called\n", __func__);  
#define FUNCTION_RETURNS() printf("%s returns\n",  
__func__);
```

Вызовы этих макросов затем могут быть помещены внутри функций для отслеживания их вызовов:

```
void f (void)
{
    FUNCTION_CALLED (); /* displays " f called" */
    ...
    FUNCTION_RETURNS(); /* diaplays "f returns" */)
}
```

Другое использование `__func__`: он может быть передан в функцию, чтобы она знала имя функции, которая её вызывала.

14.4. Условная компиляция

Препроцессор Си распознает ряд директив, которые поддерживают условную компиляцию - включение или исключение раздела текста программы в зависимости от результата теста, выполняемого препроцессором.

Директивы `#if` и `#endif`

Предположим, что мы находимся в процессе отладки программы. Мы хотим, чтобы программа печатала значения определённых переменных, поэтому мы помещаем вызовы `printf` в критические части программы. После того, как мы обнаружили ошибки, часто рекомендуется оставлять вызовы `printf` на всякий случай, если они нам понадобятся позже.

Условная компиляция позволяет нам оставить вызовы на месте, но компилятор их игнорирует. Вот как мы продолжим. Мы сначала определим макрос и придадим ему ненулевое значение:

```
#define DEBUG 1
```

Имя макроса не имеет значения. Затем мы будем окружать каждую группу вызовов *printf* с помощью пары *# if- # endif*:

```
#if DEBUG
printf("Value of i: %d\n", i);
printf("Value of j: %d\n", j);
#endif
```

Во время препроцессорной обработки директива *#if* проверяет значение *DEBUG*. Поскольку его значение не равно нулю, препроцессор оставит два вызова *printf* в программе (хотя строки *#if* и *#endif* исчезнут). Если мы изменим значение *DEBUG* на ноль и перекомпилируем программу, препроцессор удалит все четыре строки из программы. Компилятор не увидит вызовы *printf*, поэтому они не будут занимать какое-либо место в объектном коде и ничего не будут делать всякий раз, когда программа будет запущена. Мы можем оставить блоки *# if- # endif* в окончательной программе, что позволит получить диагностическую информацию позже (путём перекомпиляции с *DEBUG*,

установленным на 1), если возникнут какие-либо проблемы. В общем случае директива `#if` имеет вид:

`#if constant-expression`

Директива `#endif` ещё проще:

`#endif`

Когда препроцессор сталкивается с директивой `#if`, он вычисляет константное выражение. Если значение выражения равно нулю, строки между `#if` и `#endif` будут удалены из программы во время предварительной обработки. В противном случае строки между `#if` и `#endif` останутся в программе, которая будет обработана компилятором:

`#if` и `#endif` не будут влиять на программу. Следует отметить, что директива `#if` рассматривает неустановленные идентификаторы как макросы, которые имеют значение 0. Таким образом, если мы пренебрегаем определением `DEBUG`, тест

`#if DEBUG`

не удастся (но не сгенерирует сообщение об ошибке), в то время как тест

`#if !DEBUG`

преуспеет.

Оператор `defined`

Мы столкнулись с операторами `#` и `##` в Разделе 14.3. Есть ещё один оператор, `defined`, характерный для препроцессора. При применении к идентификатору `defined` генерирует значение 1, если идентификатор является текущим определяемым макросом, в противном случае он образует 0. Оператор `defined` обычно используется вместе с директивой `#if`; это позволяет нам писать

```
#if defined (DEBUG)
```

```
...
```

```
#endif
```

Строки между директивами `#if` и `#endif` будут включены в программу только в том случае, если `DEBUG` определяется как макрос. Круглые скобки вокруг `DEBUG` не требуются; мы могли бы просто написать

```
#if defined DEBUG
```

Поскольку `defined` проверяет только определён ли `DEBUG` или нет, нет необходимости давать `DEBUG` значение:

```
# defined DEBUG
```

Директивы `#ifdef` и `#ifndef`

Директива `#ifdef` проверяет, определён ли идентификатор в качестве макроса:

```
#ifdef identifier
```

Использование `#ifdef` аналогично использованию `#if`:

```
#ifdef identifier
```

Здесь строки, которые должны быть включены, если идентификатор определяется как макрос

```
#endif
```

Строго говоря, нет необходимости в `#ifdef`, так как мы можем объединить директиву `#if` с оператором `defined`, чтобы получить тот же эффект. Другими словами, директива

```
#ifdef identifier
```

эквивалентна

`#if defined (identifier)`

Директива `#ifndef` похожа на `# ifdef`, но проверяет, является ли идентификатор не определённым как макрос:

`ifndef identifier`

Запись

`#ifndef identifier`

есть то же самое, что и

`#if !defined (identifier)`

Директивы `#elif` и `#else`

Блоки `#if`, `#ifdef` и `#ifndef` могут быть вложены так же, как обычные инструкции `if`. Когда происходит вложение, рекомендуется использовать увеличивающееся количество отступов по мере роста уровня вложений. Некоторые программисты помещают комментарий по поводу каждого закрытия `#endif`, чтобы указать, какое условие соответствующего `#if` проверяется:

```
# if DEBUG
```

```
...
```

```
#endif /* DEBUG */
```

Этот способ облегчает чтение читателем начала блока `#if`. Для дополнительного удобства препроцессор поддерживает директивы `#elif` и `#else`:

```
#elif constant-expression
```

```
#else
```

`#elif` и `#else` могут использоваться в сочетании с `#if`, `#ifdef` или `#ifndef` для проверки ряда условий:

```
# if expr1
```

*Строки, которые должны быть включены, если expr1
отличен от нуля*

```
#elif expr2
```

*Строки, которые должны быть включены, если expr1 равно
нулю, но expr2 отличен от нуля*

```
#else
```

Строки, которые должны быть включены иначе

```
#endif
```

Хотя показана выше директива `#if`, вместо нее может использоваться директива `#ifdef` или `#ifndef`. Любое количество директив `#elif`, но не более одного `else`, может появляться между `#if` и `#endif`.

Использование условной компиляции

Условная компиляция, безусловно, удобна для отладки, но её использование на этом не кончается. Вот несколько других распространённых приложений:

Написание программ, переносимых на несколько машин или операционных систем. Следующий пример включает одну из трёх групп строк в зависимости от того, определён ли `WIN32`, `MAC_OS` или `LINUX` как макрос:

```
#if defined(WIN32)
...
#elif defined(MAC_OS)
...
#elif defined(LINUX)
...
#endif
```

Программа может содержать многие из этих блоков `#if`. В начале программы будет определён один (и только один) макрос, выбирающий конкретную операционную систему. Например, определение макроса `LINUX` может означать, что программа должна запускаться под операционной системой `Linux`.

■ **Написание программы, которая может быть скомпилирована с различными компиляторами.** Различные компиляторы часто распознают несколько разных версий Си. Некоторые принимают стандартную версию Си, некоторые - нет. Некоторые из них предоставляют расширения для машинного языка; некоторые - нет или предоставляют другой набор расширений. Условная компиляция может позволить программе настраиваться на разные компиляторы. Рассмотрим проблему написания программы, которая может быть скомпилирована с использованием более старого нестандартного компилятора. Макрос `__STDC__` позволяет препроцессору определить, соответствует ли компилятор стандарту (C89 или C99); если это не так, нам может потребоваться изменить некоторые аспекты программы. В частности, нам может потребоваться использовать объявления функций старого стиля (обсуждаемые в Вопросах и ответах в конце Главы 9) вместо прототипов функций. В каждой точке, где объявляются функции, мы можем поместить следующие строки:

```
#if __STDC__
```

Прототипы функций

#else Декларации функций в старом стиле

#endif

■ ***Предоставление определения макроса по умолчанию.***

Условная компиляция позволяет проверять, определён ли макрос в настоящее время, а если нет, задать его по умолчанию. Например, следующие строки будут определять макрос BUFFER_SIZE, если он ранее не был определён:

```
#ifndef BUFFER_SIZE
```

```
#define BUFFER_SIZE 256
```

```
#endif
```

■ ***Временное отключение кода, содержащего комментарии.***

Мы не можем использовать /*... */ комментариев чтобы "закомментировать" код, который уже содержит /*... */ комментарии. Вместо этого мы можем использовать директиву #if:

```
#if 0
```

Строки содержащие комментарий

```
#endif
```

Отключение кода таким образом часто называют "обусловливание выхода".

В Разделе 15.2 обсуждается другое общее использование

условной компиляции: защита файлов заголовков от множественного включения.

14.5 Прочие директивы

Чтобы завершить эту главу, мы кратко рассмотрим директивы `#error`, `#line` и `#pragma`.

Эти директивы более специализированные, чем те, которые мы уже изучили и они используются гораздо реже.

Директива `#error`

Директива `#error` имеет вид

`#error message`

где `message`-любая последовательность токенов.

Если препроцессор обнаруживает директиву `#error`, он выводит сообщение об ошибке, которое должно содержать сообщение. Точная форма сообщения об ошибке может

варьироваться от одного компилятора к другому; это может быть что-то вроде

Error directive: *message*

или, возможно, просто

#error message

Появление директивы *#error* указывает на серьезную ошибку в программе; некоторые компиляторы немедленно завершают компиляцию не пытаясь найти другие ошибки.

Директивы *#error* часто используются в сочетании с условной компиляцией для проверки ситуаций, которые не должны возникать во время обычной компиляции. Например, предположим, что мы хотим убедиться, что программа не может быть скомпилирована на компьютере, тип `int` который не способен хранить числа до 100000. Наибольшее возможное значение `int` представлено макросом `INT_MAX`, поэтому все, что нам нужно сделать, это вызвать директиву *#error*, если `INT_MAX` менее 100000:

```
#if INT_MAX < 100000
```

```
#error int type is too small /* Tun int слишком мал*/
```

```
#endif
```

751

Попытка скомпилировать программу на машине, целые числа которой хранятся в 16 битах, создаст сообщение, такое как

Error directive: int type is too small

Директива `#error` часто находится в части `# else` серии `# if - #elif - #else`:

```
#if defined(WIN32)
...
#elif defined(MAC_OS)
...
#elif defined(LINUX)
...
#else
```

```
#error No operating system specified
#endif
```

Директива `#line`

Директива `#line` используется для изменения способа нумерации строк программы. (Строки обычно пронумерованы 1, 2, 3, как и следовало ожидать.) Мы также можем использовать эту директиву, чтобы заставить компилятор думать, что он читает программу из файла с другим именем. Директива `#line` имеет две формы. В одной форме, мы указываем номер строки :

`#line n`

это должна быть последовательность цифр, представляющая целое число от 1 до 32767(2147483647 в C99). Эта директива заставляет нумеровать последующие строки в программе:

n , $n + 1$, $n + 2$ и так далее.

Во второй форме директивы `#line` указывается номер строки и имя файла:

`#line n "file"`

Предполагается, что строки, следующие за этой директивой, берутся из файла с номерами строк, начинающимися с ***n***. Значения ***n*** и/или строки файла можно задать с помощью макросов.

Одним из эффектов директивы `#line` является изменение значения макроса `__LINE__` (и, возможно, макроса `__FILE__`). Что ещё более важно, большинство компиляторов будут использовать информацию из директивы `#line` при генерации сообщений об ошибках.

Например, предположим, что следующая директива появляется в начале файла `foo.c`:

```
#line 10 "bar.c"
```

предположим, что компилятор обнаруживает ошибку в строке 5 `foo.c`. Сообщение об ошибке будет ссылаться на строку 13 файла `bar.c`, а не на строку 5 файла `foo.c`. (Почему на строку 13? Директива занимает строку 1 `foo.c`, поэтому перенумерация `foo.c` начинается со строки 2, которая рассматривается как строка 10 `bar.c`.)

На первый взгляд, директива `#line` загадочна. Почему мы хотим, чтобы сообщения об ошибках ссылались на другую строку и, возможно, на другой файл? Не затруднит ли это отладку программ? На самом деле, директива `#line` не очень часто используется программистами. Вместо этого она используется в основном программами, которые генерируют код Си в качестве выходных данных. Самым известным примером такой программы является уасс (Yet Another Compiler-Compiler), утилита UNIX, которая автоматически генерирует часть компилятора. (GNU-версия уасс называется `bison`.) Перед использованием уасс программист подготавливает файл, содержащий информацию для уасс, а также фрагменты Си-кода. Из этого файла уасс генерирует программу на Си, `y.tab.c`, который включает в себя код, предоставленный программистом. Затем программист компилирует `y.tab.c` обычным способом. Вставляя директивы `#line` в `y.tab.c`, уасс обманывает компилятор, полагая, что код

исходит из исходного файла—того, который написан программистом. В результате любые сообщения об ошибках, произведённые во время компиляции `y.tab.c` будут ссылаться на строки в исходном файле, а не на строки в `y.tab.c`, это облегчает отладку, потому что сообщения об ошибках относятся к файлу, написанному программистом, а не к (более сложному) файлу, сгенерированному уасс.

Директива `#pragma`

Директива `#pragma` предоставляет способ запроса специального поведения компилятора. Эта директива наиболее полезна для программ, которые имеют необычно большой размер или которым необходимо использовать возможности конкретного компилятора. Директива `#pragma` имеет вид:

`#pragma tokens`

где *tokens* являются произвольными токенами, директивы `#pragma` могут быть очень простыми (один токен), или они могут быть гораздо более сложными:

`#pragma data (heap_size => 1000, stack_size => 2000)`

Неудивительно, что набор команд, которые могут отображаться в директивах `#pragma`, различен для каждого

компилятора: вам придётся обратиться к документации компилятора, чтобы узнать, какие команды он разрешает и что делают эти команды. Кстати, препроцессор должен игнорировать любую директиву `#pragma`, содержащую нераспознанную команду; не разрешается выводить сообщение об ошибке.

В C89 нет стандартных прагм - все они определены реализациями. C99 имеет три стандартные прагмы, все из которых используют STDC в качестве первого токена после `#pragma`. Этими прагмами являются `FP_CONTRACT` (рассмотрен в Разделе 23.4). `CX_LIMITED_RANGE` (Раздел 27.4) и `FENV_ACCESS` (Раздел 27.6).

Оператор `_Pragma`

C99 вводит оператор `_Pragma`, который используется в сочетании с директивой `#pragma`. Выражение `_Pragma` имеет вид:

`_Pragma ( string-literal )`

Когда препроцессор встречает такое выражение, он "destringizes" (дестрингифицирует) *string-literal* (строковый литерал) (да, это термин, используемый в стандарте C99!)

удалив двойные кавычки вокруг строки и заменив escape-последовательности \" и \\ символами " и \ соответственно. Результатом является серия токенов, которые затем обрабатываются так, как будто они появляются в директиве #pragma. Например, написание

```
_Pragma("data(heap_size => 1000, stack_size => 2000)")
```

есть то же самое, что и

```
#pragma data(heap_size => 1000, stack_size => 2000)
```

Оператор _Pragma позволяет обойти ограничение препроцессора: тот факт, что директива препроцессора не может генерировать другую директиву. _Pragma, однако, является оператором. не является директивой и поэтому может отображаться в определении макроса. Это позволяет при расширении макроса оставить директиву #pragma. Давайте рассмотрим пример из руководства GCC. Следующий макрос использует оператор _Pragma :

```
#define DO_PRAGMA(x) _Pragma (#x)
```

Макрос будет вызываться следующим образом:

```
DO_PRAGMA (GCC dependency "parse.y")
```


После расширения, результат будет

```
#pragma GCC dependency "parse.y"
```

это одна из прагм, поддерживаемых GCC. (Выдаёт предупреждение, если дата указанного файла - parse.y в данном примере - является более свежей, чем дата текущего компилируемого файла.) Обратите внимание, что аргумент вызова DO_PRAGMA является серией токенов. Оператор # в определении DO_PRAGMA заставляет токены быть stringized (стрингифицированными) в "GCC dependency \"parse.y\""; затем эта строка передаётся оператору _Pragma, который её дестрингует, создавая директиву #pragma, содержащую исходные токены.

Вопросы и ответы

Вопрос: Я видел программу, которая содержит только один символ # в строке. Это допустимо?

Ответ: Да. Это директива null; она не даёт никаких последствий. Некоторые программисты используют директивы null для интервалов в блоках условной компиляции:

```
#if INT_MAX < 100000  
#  
#error int type is too small  
#  
#endif
```

Пустые строки также будут работать, конечно, но `#` помогает читателю увидеть размер блока.

Вопрос: Я не уверен, какие константы в программе должны быть определены как макросы. Есть ли какие рекомендации?

Ответ: Одно эмпирическое правило гласит, что каждая числовая константа, кроме 0 или 1, должна быть макросом. Символьные и строковые константы будут проблематичны, так как замена символьной или строковой константы макросом не всегда улучшает читаемость. Я рекомендую использовать макрос вместо символьной константы или строкового литерала при условии, что (1) константа используется более одного раза и (2) существует вероятность того, что константа может когда-нибудь быть изменена. Из-за правила (2) я не использую такие макросы как

```
#define NUL '\0'
```

хотя некоторые программисты используют.

Вопрос: Что делает оператор #, если аргумент, который он должен "stringize"(стрингифицировать) содержит символ " или \? [р. 324]

Ответ: Он преобразует " в \" и \ в\\. Рассмотрим следующий макрос:

```
#define STRINGIZE (x) #x
```

Препроцессор заменит STRINGIZE ("foo") на " \ "foo \ " ".

Вопрос: Я не могу заставить следующий макрос работать правильно:

```
#define CONCAT (x,y) x##y
```

CONCAT (a, b) даёт ab, как и ожидалось, но

CONCAT (a, CONCAT (b, c)) даёт странный результат.

Что происходит?

Ответ: Благодаря правилам, которые Керниган и Ричи называют "причудливыми", макросы, чьи списки замен зависят от ##, обычно не могут быть вызваны будучи вложенными. Проблема заключается в том, что

CONCAT (a, CONCAT (b, c)) не расширяется в нормальном режиме, с CONCAT (b, c), дающим bc, потом CONCAT (a, bc), дающим abc. Параметры макроса, перед или после ## в списке замен не раскрываются во время подстановки.

В результате CONCAT (a, CONCAT (b, c)) расширяется до aCONCAT (b, c), который не может быть расширен дальше, так как нет макроса с именем aCONCAT. Есть способ решить

проблему, но это **некрасиво**. Трюк состоит в том, чтобы определить второй макрос, который просто вызывает первый:

```
#define CONCAT2(x,y) CONCAT(x,y)
```

написание

```
CONCAT2 (a, CONCAT2 (b, c) )
```

теперь даёт желаемый результат. Поскольку препроцессор расширяет внешний вызов CONCAT2, он также расширяет CONCAT2 (b, c); разница в том, что список замены CONCAT2 не содержит ##. Если это сложно для восприятия - не волнуйтесь - это не та проблема, которая возникает часто. У оператора # такая же сложность, кстати. Если #x появляется в списке замен, где x - параметр макроса, соответствующий аргумент не расширяется. Таким образом, если N является макросом, представляющим 10, а STR (x) имеет список замещения #x, расширение STR (N) даёт "N", а не "10". Решение аналогично тому, которое мы использовали с CONCAT: определение второго макроса, задачей которого является вызов STR.

***Вопрос:** Предположим, что препроцессор обнаруживает исходное имя макроса во время сканирования, как показано в следующем примере:

```
#define N (2*M)
```

```
#define M (N+1)
```

```
i= N; /*бесконечный цикл? */
```

Препроцессор заменит N на $(2 \cdot M)$, затем заменит M на $(N+1)$.
Заменит ли препроцессор N снова, перейдя таким образом в бесконечный цикл?

Ответ: Некоторые старые препроцессоры действительно попадают в бесконечный цикл, но новые - нет. В соответствии со стандартом Си, если исходное имя макроса появляется снова во время расширения макроса, имя не заменяется снова. Вот как будет выглядеть присваивание после препроцессорной обработки:

```
i = (2*(N+1));
```

Некоторые предприимчивые программисты используют это поведение, написав макросы, имена которых соответствуют зарезервированным словам или функциям в стандартной библиотеке. Рассмотрим библиотечную функцию sqrt.

sqrt вычисляет квадратный корень из своего аргумента, возвращая значение, определяемое реализацией, если аргумент отрицательный. Возможно, мы предпочли бы, чтобы sqrt возвращал 0, если его аргумент отрицательный.

Поскольку sqrt является частью стандартной библиотеки, мы не можем легко изменить его. Мы можем, однако, определить макрос sqrt, который вычисляет значение 0 при отрицательном аргументе:

```
#undef sqrt
```

```
#define sqrt: (x) ((x) > 0 ? sqrt (x) : 0)
```

В дальнейшем вызов `sqrt` будет перехвачен препроцессором, который развернёт его в условное выражение, показанное здесь. Вызов `sqrt` внутри условного выражения не будет заменён во время повторного сканирования, поэтому он останется для обработки компилятором. (Обратите внимание на использование `#undef` для отмены определения `sqrt` перед определением макроса `sqrt`. Как мы увидим в Разделе 21.1, стандартная библиотека может иметь как макрос так и функцию с таким же именем. Отмена определения перед определением собственного макроса `sqrt` является защитной мерой, если библиотека уже определила `sqrt` как макрос.)

Вопрос: Я получаю ошибку при попытке использовать предопределённые макросы, такие как `__LINE__` и `__FILE__`. Есть ли специальный заголовок, который мне нужно включить?

Ответ: Нет. Эти макросы автоматически распознаются препроцессором. Убедитесь, что у вас есть два символа подчёркивания в начале и конце каждого имени макроса, а не один.

Вопрос: В чем цель различия "хостинговой реализации" и "автономной реализации"? Если "автономная реализация" не поддерживает заголовок `<stdio.h>`, какая польза от него?

Ответ: Хостинговая реализация необходима для большинства программ (в том числе в этой книге), которые полагаются на базовую операционную систему для ввода / вывода и других важных услуг. Автономная реализация Си будет использоваться для программ, для которых не требуется

операционная система (или только минимальная операционная система). Например, для написания ядра операционной системы (которая не требует традиционного ввода / вывода и не нуждается в `<stdio.h>`), потребуется отдельная реализация. Автономные реализации также полезны для написания программного обеспечения для встроенных систем.

Вопрос: Я думал, что препроцессор является просто редактором. Как он может вычислять константные выражения?

Ответ: Препроцессор более сложен, чем можно было бы ожидать; он знает достаточно о Си, чтобы иметь возможность вычислять константные выражения, хотя он делает это не совсем так, как компилятор. (С одной стороны, препроцессор рассматривает любое неопределённое имя как имеющее значение `0`. Другие различия слишком эзотеричны, чтобы вдаваться в подробности.) На практике операнды в константном выражении препроцессора обычно являются константами, макросами, представляющими константы и приложениями оператора `defined`.

Вопрос: Почему Си предоставляет директивы `#ifdef` и `#ifndef`, так как мы можем получить тот же эффект, используя директиву `#if` и оператор `defined`?

Ответ: Директивы `#ifdef` и `#ifndef` были частью Си с 1970-х годов. С другой стороны, оператор `defined` был добавлен в Си в 1980-х годах во время стандартизации. Итак, реальный вопрос: почему к языку добавили `defined` ? Ответ заключается

в том, что `defined` добавляет гибкость. Вместо того, чтобы просто проверить существование одного макроса с помощью `#ifdef` или `#ifndef`, теперь мы можем протестировать любое количество макросов, используя `#if` вместе с `defined`. Например, следующая директива проверяет, определены ли `FOO` и `BAR`, но `BAZ` не определён:

```
#if defined(FOO) && defined(BAR) && !defined(BAZ)
```

Вопрос: Я хотел скомпилировать программу, которую я не закончил писать, поэтому я "обусловил" незавершённую часть:

```
#if
```

```
...
```

```
#endif
```

Когда я скомпилировал программу, я получил сообщение об ошибке, относящееся к одной из строк между `#if` и `#endif`. Разве препроцессор не игнорирует эти строки?

Ответ: Нет, строки не полностью игнорируются.

Комментарии обрабатываются перед выполнением директив препроцессорной обработки, а исходный код разделяется на токены препроцессорной обработки. Таким образом, незаконченный комментарий между `#if` и `#endif` может вызвать сообщение об ошибке. Кроме того, непарные

одинарные кавычки или двойные кавычки могут привести к неопределённому поведению.

Упражнения

1. Напишите параметризованные макросы, вычисляющие следующие значения.

a) Куб x .

b) Остаток, когда x делится на 4.

c) 1, если произведение x и y меньше 100, 0 в противном случае

Всегда ли работают ваши макросы? Если нет, опишите, какие аргументы заставят их потерпеть неудачу.

W2. Приведите макрос NELEMS (a), который вычисляет количество элементов в одномерном массиве **a**. Подсказка: см. Обсуждение оператора sizeof в Разделе 8.1.

3. Пусть DOUBLE будет следующим макросом:

```
#define DOUBLE (x) 2 * x
```

(a) Каково значение DOUBLE (1 + 2)?

(б) Каково значение 4 / DOUBLE (2)?

(c) Исправьте определение DOUBLE.

W4. Для каждого из следующих макросов дайте пример, который иллюстрирует проблему с макросом и покажет, как её исправить.

(a) `#define AVG (x, y) (x-y) / 2`

(b) `#define AREA (x, y) (x) * (y)`

W * 5. Пусть TOUPPER будет следующим макросом:

```
#define TOUPPER(c) ('a'<=(c) &&(c) <='z'? (c) - 'a'+'A' : (c))
```

Пусть s - строка и пусть i - переменная int. Покажите результат, полученный каждым из следующих фрагментов программы.

(a) `strcpy(s, "abcd");`

`i = 0;`

`putchar(TOUPPER(s[++i]));`

(b) `strcpy(s, "0123");`

`i = 0;`

`putchar(TOUPPER(s[++i]));`

6. (a) Напишите макрос DISP (f, x), который расширяется в вызове printf, который отображает значение функции *f* при вызове с аргументом x. Например,

```
DISP (sqrt, 3.0);
```

должны развернуться в

```
printf ("sqrt (% g) =%g \ n", 3.0, sqrt (3.0));
```

(b) Напишите макрос DISC2(f,x,y), аналогичный DISC, но работающий для функций с двумя аргументами.

W*7 пусть GENERIC_MAX будет следующим макросом:

```
#define GENERIC_MAX (type) \
type type##_max(type x,type y) \
{ \
    return x > y ? x : y; \
}
```

(a) Показать расширение препроцессора в
GENERIC_MAX (long).

768

(b) объясните, почему `GENERIC_MAX` не работает для базовых типов, таких как `unsigned long`.

(c) опишите метод, который позволит нам использовать `GENERIC_MAX` с базовыми типами, такими как `unsigned long`. Подсказка: не изменяйте определение `GENERIC_MAX`.

****8.**Предположим, нам нужен макрос, который расширяется в строку, содержащую номер текущей строки и имя файла, другими словами, мы хотели бы написать

```
const char *str = LINE_FILE ;
```

и она разрастается

```
const char *str = " Line 10 of file foo.c";
```

где `foo.c` - файл, содержащий программу, а 10 - строка, в которой отображается вызов файла `LINE_FILE`. Это упражнение предназначено только для экспертов. Перед попыткой внимательно прочитайте Раздел вопросов и ответов.

9.Напишите следующие параметризованные макросы:

CHECK (x,y,n) - имеет значение 1, если x и y находятся в диапазоне от 0 до n - 1 включительно..

(b) MEDIAN (x,y,z) - находит среднее x,y, и z.

(c) POLYNOMIAL (x) - вычисляет полином

$$3x^5 + 2x^4 - 5x^3 - x^2 + 7x - 6$$

10.Функции часто, - но не всегда-могут быть записаны как параметризованные макросы. Обсудите, какие характеристики функции сделают её неприемлемой в качестве макроса.

11.(C99) Си программисты часто используют функцию fprintf для записи сообщений об ошибках:

```
fprintf (stderr, "Range error: index = %d\n", index);
```

stderr-это "стандартный поток ошибок " языка Си; остальные аргументы такие же, как и для printf, начиная с форматной строки. Напишите макрос с именем ERROR, который генерирует вызов fprintf, показанный выше, если задана строка формата и элементы для отображения:

```
ERROR("Range error: index = %d\n", index);
```

770

W 12.Предположим, что макрос определён следующим образом:

```
#define M 10
```

Какие из следующих тестов завершатся неудачно?

- (a) `#if M`
- (b) `#ifdef M`
- (c) `#ifndef M`
- (d) `#if defined(M)`
- (e) `#if !defined(M)`

13. (a) Покажите, как будет выглядеть следующая программа после препроцессорной обработки. Вы можете игнорировать любые строки, добавленные в

программу в результате включения заголовка `<stdio.h>`.

```
#include <stdio.h>
```

```
#define N 100
```

```
void f(void);
```

771

```
int main(void)
```

```
{
```

```
    f();
```

```
#ifdef N
```

```
#undef N
```

```
#endif
```

```
    return 0;
```

```
}
```

```
void f (void)
```

```
{
```

```
#if defined(N)
```

```
    printf("N is %d\n",N);
```

```
#else
```

```
    printf("N is undefined\n");
```

```
#endif
```

```
}
```

(б) Каким будет результат этой программы?

W*14.Покажите, как будет выглядеть следующая программа после препроцессорной обработки. Некоторые строки

программы могут вызвать ошибки компиляции, найдите все такие ошибки.

```
#define N = 10
```

```
#define INC(x) x+1
```

```
#define SUB (x,y) X-Y
```

```
#define SQR(x) ((x)*(x))
```

```
#define CUBE (x) (SQR (x)*(x))
```

```
#define M1(x,y) x##y
```

```
#define M2 (x,y) #x #y
```

```
int main(void)
```

```
{
```

```
    int a[N], i,j,k,m;
```

```
#ifdef N
```

```
    i=j;
```

```
#else
```

```
    j=i;
```

```
#endif
```

```
    i=10*INC(j);
```

```
    i = SUB(j,k);
```


773

```
i = SQR(SQR(j));
```

```
i = CUBE(j);
```

```
i = M1(j, k);
```

```
puts(M2(i, j));
```

```
#undef SQR
```

```
i = SQR(j);
```

```
#define SQR
```

```
i = SQR(j);
```

```
return 0;
```

```
}
```

15. Предположим, что программа должна выводить сообщения на английском, французском или испанском языках. С помощью условной компиляции напишите фрагмент программы, отображающий одно из следующих трёх сообщений, в зависимости от того, определён ли указанный макрос:

Insert Disk 1 (if ENGLISH is defined)

Inserez Le Disque 1 (if FRENCH is defined)

Inserte E1 Disco 1 (if SPANISH is defined)

*16.(C99) Предположим, что действуют следующие определения макросов:

```
#define IDENT(x) PRAGMA(ident #x)
```

```
#define PRAGMA(x) _Pragma(#x)
```

Как будет выглядеть следующая строка после расширения макроса?

```
IDENT (foo)
```

15 Написание больших программ

В программировании трудно найти правильную единицу времени для измерения прогресса. Некоторые соборы строились веками. Можно ли вообразить грандиозность и размер программы, на которую затратили столько времени?

Хотя некоторые программы на Си достаточно малы, чтобы поместить их в один файл, большинство из них - нет. Программы, состоящие из нескольких файлов, являются правилом, а не исключением. В этой главе мы увидим, что типичная программа состоит из нескольких исходных файлов и, как правило, некоторых заголовочных файлов. Исходные файлы содержат определения функций и внешних переменных; заголовочные файлы содержат информацию, которая будет совместно использоваться исходными файлами. Раздел 15.1 обсуждает исходные файлы, в то время как Раздел 15.2 рассматривает заголовочные файлы. Раздел 15.3 описывает, как разделить программу на исходные файлы и файлы заголовков. Раздел 15.4 затем показывает, как "собрать" (скомпилировать и связать) программу, состоящую из нескольких файлов, и как "перестроить" программу после изменения её части.

15.1 Исходные файлы

До этого момента мы предполагали, что программа на Си состоит из одного файла. На самом деле, программа может быть разделена между любым количеством исходных файлов.

По соглашению, исходные файлы имеют расширение .c. Каждый исходный файл содержит часть

программы, в основном определения функций и переменных. В одном исходном файле должна содержаться функция `main`, служащая точкой отсчёта для программы. Например, предположим, что мы хотим написать простую программу-калькулятор, которая вычисляет целочисленные выражения, введённые в обратной польской нотации (RPN), в которой операторы следуют за операндами. Если пользователь вводит выражение типа `30 5 - 7 *`, мы хотим, чтобы программа выводила его значение (175 в этом случае.)

Вычисление выражения RPN будет лёгким, если программа читает операнды и операторы один за другим, используя стёк для отслеживания промежуточных результатов. Если программа прочитает число, мы отправим его в стёк. Если она читает оператор, мы заставим её вытащить два числа из стёка, выполнить операцию, а затем вернуть результат обратно в стёк. Когда программа достигает конца пользовательского ввода, значение выражения будет находиться в стёке. Например, программа вычислит выражение `30 5 - 7 *` следующим образом:

1. Поместите 30 в стёк.
2. Поместите 5 в стёк.
3. Вытолкните два верхних числа из стёка, вычтите 5 из 30, получите 25, а затем вставьте результат обратно в стёк.
4. Поместите 7 в стёк.

5. Вытащите два верхних числа из стёка, умножьте их, а затем вставьте результат обратно в стёк.

После выполнения этих действий стёк будет содержать значение выражения (175).

Превратить эту стратегию в программу несложно. Основная функция программы будет содержать цикл, который выполняет следующие действия:

Прочитать "токен" (число или оператор). Если токен представляет собой число, вставьте его в стёк.

Если токен является оператором, извлеките его операнды из стёка, выполните операцию, а затем вставьте результат обратно в стёк. При разделении такой программы на файлы имеет смысл поместить связанные функции и переменные в один и тот же файл. Функция, которая считывает токены, может перейти в один исходный файл (скажем `token.c`), вместе с любыми функциями, которые имеют отношение к токенам. Связанные с стёком функции, такие как `push`, `pop`, `make__empty`, `is_empty`, и `is_full` может перейти в другой файл, `stack.c`. Переменные, представляющие стёк, также попадают в `stack.c`. Функция `main` войдёт в ещё один файл `calc.c`. Разделение программы на несколько исходных файлов имеет значительные преимущества:

■ Группировка связанных функций и переменных в один файл помогает прояснить структуру программы.

- Каждый исходный файл может быть скомпилирован отдельно - отличная экономия времени, если программа большая и должна часто изменяться (что часто встречается во время разработки программы).
- Функции легче повторно использовать в других программах, если они сгруппированы в отдельные исходные файлы. В нашем примере отделение функций `stack.c` и `token.c` от функции `main` упрощает повторное использование функций `stack` и функций `token` в будущем.

15.2 Файлы заголовков

Когда мы делим программу на несколько исходных файлов, возникают проблемы:

Как функция в одном файле вызывает функцию, определённую в другом файле? Как функция может получить доступ к внешней переменной в другом файле? Как два файла могут иметь одно и то же макроопределение или определение типа?

Ответ кроется в директиве `# include`, которая позволяет обмениваться информацией - прототипами функций, определениями макросов, определениями типов и многим другим - между любым количеством исходных файлов.

Директива `#include` указывает препроцессору, чтобы открыть указанный файл и вставить его содержимое в текущий файл.

Таким образом, если мы хотим, чтобы несколько исходных файлов имели доступ к одной и той же информации, мы поместим эту информацию в файл, а затем используем `#include` для переноса содержимого файла в каждый из исходных файлов. Файлы, включённые в этот способ, называются файлами заголовков (или иногда включаемыми файлами). Более подробно я расскажу о них позже в этом разделе. По соглашению, заголовочные файлы имеют расширение ***.h***.

Примечание: стандарт Си использует термин "исходный файл" для обозначения всех файлов, написанных программистом, включая оба файла ***.c*** и ***.h***. Я буду использовать "исходный файл" для ссылки только на ***.c*** файлы.

Директива `#include` имеет две основные формы. Первая форма используется для заголовочных файлов, принадлежащих собственной библиотеке языка Си:

код `#include <имя_файла>`.

Вторая форма: используется для всех остальных заголовочных файлов, включая те, которые мы пишем:

`#include "имя_файла"`.

Разница между ними - тонкая, связанная с тем, как компилятор находит файл заголовка.

Вот правила, которым следуют большинство компиляторов:

■ `#include <Имя файла>`:

Просматривает каталог (или каталоги), в котором находятся системные файлы заголовков. (Например, в системах UNIX системные заголовочные файлы обычно хранятся в каталоге `/usr/include`.)

■ `#include "filename"`:

Просматривает текущий каталог, затем смотрит каталог (или каталоги), в котором находятся системные файлы заголовков.

Места для поиска заголовочных файлов обычно могут быть изменены, часто с помощью опции командной строки, такой как `-Ipath`.

Не используйте скобки при включении написанных файлов заголовков:

```
include <myheader.h> *** WRONG ***
```

Препроцессор, вероятно, будет искать `myheader.h` там, где хранятся системные заголовочные файлы (и, конечно же, не найдёт его).

Имя файла в директиве `# include` может содержать информацию, которая помогает найти файл, например путь к каталогу или спецификатор диска:

```
#include "c:\cprogs\utils.h" /*Windows path */
```

```
#include "/cprogs/utils.h" /* UNIX path */
```

Хотя кавычки в директиве `#include` делают имена файлов похожими на строковые литералы, препроцессор не обрабатывает их таким образом. (Это повезло, так как `\с` и `\и`, которые появляются в примере Windows, будут рассматриваться как escape-последовательности в строковом литерале.) Обычно лучше не включать информацию о пути или диске в директивы `#include`. Такая информация затрудняет компиляцию программы при её переносе на другую машину или, что ещё хуже, на другую операционную систему.

Например, следующие директивы `#include` Windows указывают сведения о диске и/или пути, которые не всегда могут быть допустимыми:

```
#include "d: utils.h"
```

```
#include "\cprogs\include\utils.h"
```

```
#include "d:\cprogs\include\utils.h"
```

Следующие директивы лучше: они не упоминают конкретные диски, а пути являются относительными, а не абсолютными:

```
#include "utils.h"
```

```
#include "..\include\utils.h"
```

Директива `#include` имеет третью форму, которая используется реже, чем две другие:

```
#include tokens
```

где *tokens* - это любая последовательность препроцессорных токенов. Препроцессор сканирует токены и заменяет любые макросы, которые он находит. После замены макроса результирующая директива должна соответствовать одной из других форм `#include`. Преимущество третьего типа `#include` заключается в том, что имя файла может быть определено макросом, а не "жёстко закодировано" в самой директиве, как показано в следующем примере:

```
#if defined(IA32)
    #define CPU_FILE "ia32.h"
#elif defined(IA64)
    #define CPU_FILE "ia64.h"
#elif defined(AMD64)
    #define CPU_FILE "amd64.h"
#endif

#include CPU_FILE
```

Совместное использование макроопределений и типов

Большинство крупных программ содержат определения макросов и типов, которые должны совместно использоваться несколькими исходными файлами (или, в крайнем случае, всеми исходными файлами). Эти определения должны входить в заголовочные файлы. Например, предположим, что мы пишем программу, использующую макросы с именами `BOOL`, `TRUE` и `FALSE`. (Конечно, в C99 они не нужны, потому что заголовок `<stdbool.h>` определяет похожие макросы) вместо того, чтобы повторять определения этих макросов в каждом исходном файле, который нуждается в них, имеет смысл поместить определения в файл заголовка с именем вроде `boolean.h` :

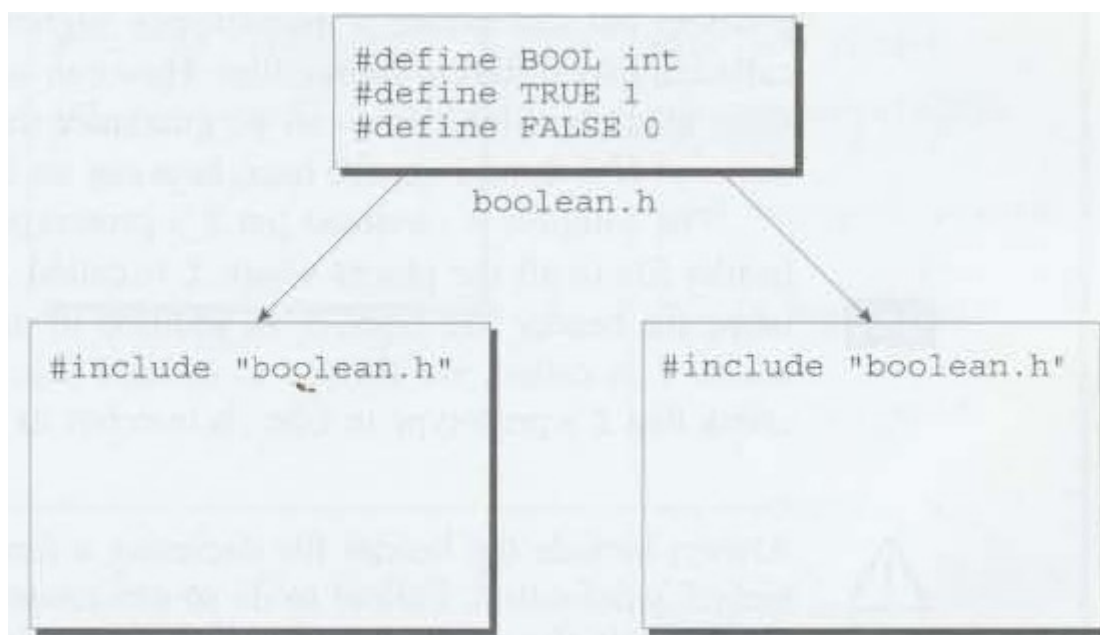
```
#define BOOL int
```

```
#define TRUE 1
```

```
#define FALSE 0
```

Любой файл источника, который требует этих макросов, просто будет содержать строку `#include "boolean.h"`.

На следующем рисунке два файла включают `boolean.h`:



Определения типов также часто встречаются в заголовочных файлах. Например, вместо определения макроса `BOOL` можно использовать `typedef` для создания типа `BOOL`. Если мы сделаем это файл `boolean.h` будет иметь следующий вид:

```
#define
```

```
TRUE 1
```

```
#define FALSE 0
```

```
typedef int Bool;
```

Размещение определений макросов и типов в заголовочных файлах имеет ряд явных преимуществ.

Во-первых, мы экономим время, не копируя определения в исходные файлы там, где они нужны.

Во-вторых, программу становится проще модифицировать. Изменение определения макроса или типа требует только редактирования одного файла заголовка: нам не нужно изменять множество исходных файлов, в которых используется макрос или тип.

В-третьих, нам не нужно беспокоиться о несоответствиях, вызванных исходными файлами, содержащими различные определения одного и того же макроса или типа.

Прототипы функций совместного использования

Предположим, что исходный файл содержит вызов функции f , которая определена в другом файле, foo.c. Вызов f , без предваряющего объявления, является рискованным. Без использования прототипа компилятор вынужден предположить, что тип возврата f - int и что число параметров соответствует числу аргументов в вызове f . Сами аргументы автоматически преобразуются в своего рода "стандартную форму" по умолчанию. Предположения компилятора могут быть ошибочными, но у них нет возможности проверить их, поскольку он компилирует файлы по одному. Если предположения неверны, программа, вероятно, не будет работать и не будет никаких подсказок относительно того, почему это не так.

(По этой причине C99 запрещает вызов функции, для которой компилятор ещё не видел объявление или определение).

При вызове функции f , определённой в другом файле, всегда убедитесь, что компилятор видел прототип функции f до вызова.

Наш первый импульс состоит в том, чтобы объявить f в файле, где он вызывается. Это решает проблему, но может создать кошмар для обслуживания. Предположим, что функция вызывается в пятидесяти различных исходных файлах. Как мы можем гарантировать, что прототипы f одинаковы во всех файлах? Как мы можем гарантировать, что они соответствуют определению f в `foo.c`? Если позже я должен буду изменить, как мы можем найти все файлы там, где они (прототипы) используются? Решение очевидно: поместите прототип f в файл заголовка, а затем включите файл заголовка во все места, где вызывается f . Поскольку f определяется в `foo.c`, давайте назовём файл заголовка `foo.h`. и включим `foo.h` в исходные файлы, где вызывается f , мы должны также включить его в `foo.c`, позволяя компилятору проверить прототип f в `foo.h` на соответствие определению `foo.c`.

Всегда включайте файл заголовка, объявляющий функцию f , в исходный файл, содержащий определение f . Невыполнение

этого требования может привести к труднодоступным ошибкам, так как вызовы в другом месте программы могут не соответствовать определению *f*.

Если *foo.c* содержит другие функции, большинство их должны быть объявлены в том же заголовочном файле, что и *f*. В конце концов, другие функции в *foo.c* предположительно связаны с *f*; любой файл, содержащий вызов *f*, вероятно, нуждается в некоторых других функциях в *foo.c*.

Функции, предназначенные для использования только в *foo.c* однако не следует объявлять в заголовочном файле: это может ввести в заблуждение. Чтобы проиллюстрировать использование прототипов функций в заголовочных файлах, вернёмся к калькулятору RPN раздела 15.1. Файл *stack.c* будет содержать определения функций *make_empty*, *is_empty*, *is_full*, *push* и *pop*.

Следующие прототипы для этих функций должны находиться в файле заголовка *stack.h*:

```
void make_empty (void);
```

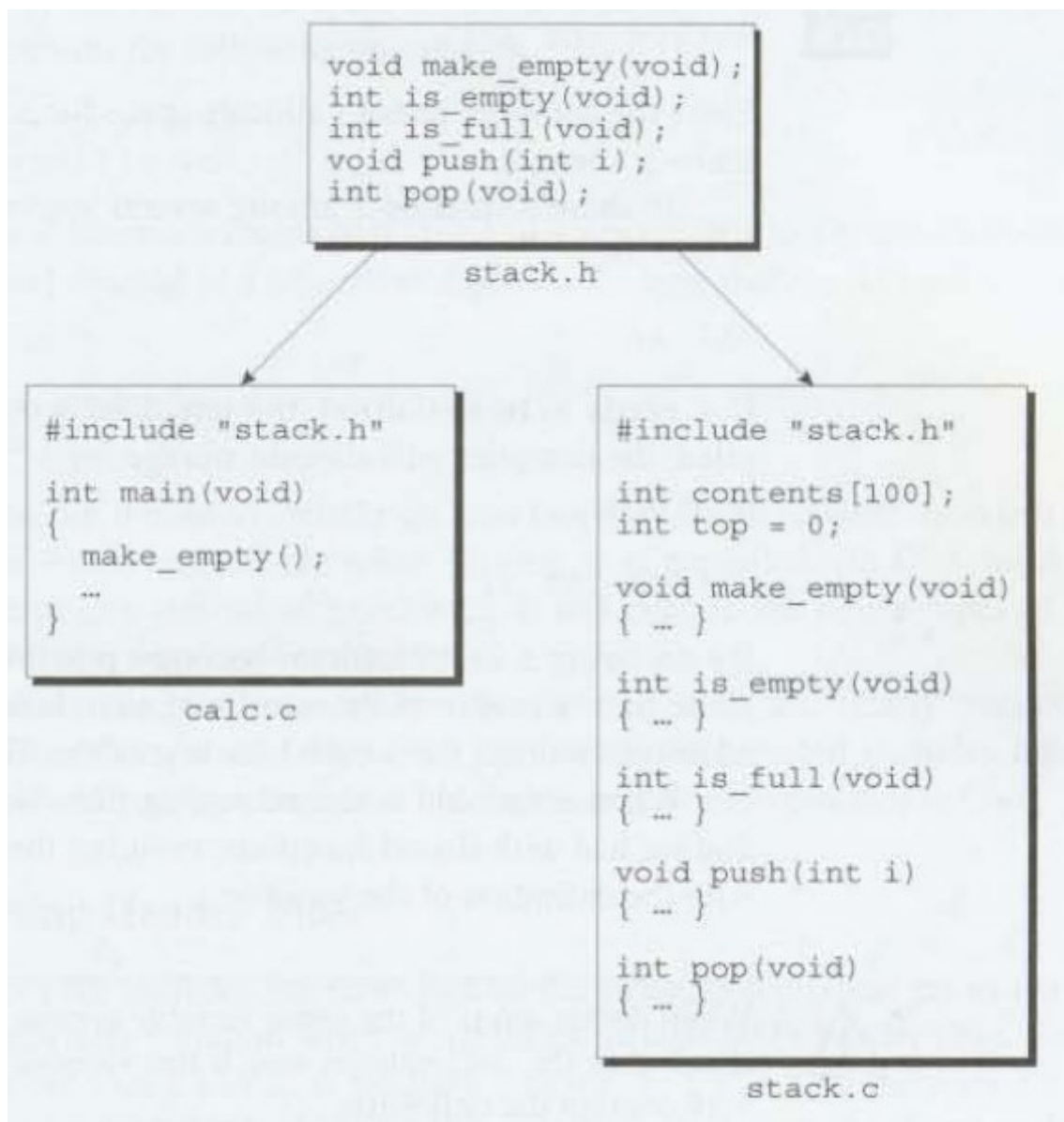
```
int is_empty (void);
```

```
is_full(void);
```

```
void push(int i);
```

```
int pop(void);
```

(Чтобы избежать усложнения примера, `is_empty` и `is_full` вернут значения `int` вместо логических значений.) Мы будем включать `stack.h` в `calc.c`, чтобы позволить компилятору проверять любые вызовы функций `stack`, которые появляются в последнем файле. Мы также будем включать `stack.h` в `stack.c`, чтобы компилятор мог проверить, что прототипы в `stack.h` соответствуют определениям в `stack.c`. На следующем рисунке показаны `stack.h`, `stack.c` и `calc.c`



Совместное использование объявлений переменных

Внешние переменные могут быть используемы в разных файлах во многом так же, как функции. Чтобы совместно

использовать функцию, мы помещаем её определение в один исходный файл, а затем помещаем её объявление в другие файлы, которые должны вызывать функцию. Совместное использование внешней переменной выполняется примерно таким же образом.

До этого момента нам не нужно было различать объявление переменной и её определение. Чтобы объявить переменную *i*, мы писали

```
int i; /* объявляет i и также определяет её */
```

это не только объявляет, что *i* является переменной типа `int`, но также определяет *i*, заставляя компилятор выделить пространство для *i*. Чтобы объявить *i* без его определения, мы должны поместить ключевое слово `extern` в начале его объявления:

```
extern int i; /* объявляет i без определения */
```

`extern` сообщает компилятору, что *i* определён в другом месте программы (скорее всего, в другом исходном файле), поэтому нет необходимости выделять пространство для него, `extern` работает с переменными всех типов. Когда мы используем его в объявлении массива можно не указывать длину массива:

```
extern int a[];
```

Поскольку в настоящее время компилятор не выделяет место для *a*, ему не нужно знать длину *a*. Чтобы совместно

использовать переменную *i* в нескольких исходных файлах, мы сначала помещаем определение *i* в один файл:

```
int i;
```

Если *i* должен быть инициализирован, инициализатор будет использован здесь. При компиляции этого файла компилятор выделит место для хранения *i*, остальные файлы будут содержать объявления *i*:

```
extern int i;
```

Объявив *i* в каждом файле, можно получить доступ и/или изменить *i* в этих файлах. Однако из-за слова `extern` компилятор не выделяет дополнительное хранилище для *i* каждый раз при компиляции одного из файлов.

Когда переменная совместно используется файлами, мы столкнёмся с проблемой, подобной той, которая была у нас с общими функциями: обеспечением того, чтобы все объявления переменной согласовывались с определением переменной. Когда объявления одной и той же переменной появляются в разных файлах, компилятор не может проверить, что объявления соответствуют определению переменных. Например, один файл может содержать определение

```
int i;
```

в то время как другой файл содержит объявление

```
extern long i;
```

Ошибка такого рода может привести к непредсказуемому поведению программы. Чтобы избежать несогласованности, объявления общих переменных обычно помещаются в файлы заголовков. Исходный файл, которому требуется доступ к определённой переменной, может включать соответствующий файл заголовка, кроме того, каждый файл заголовка, содержащий объявление переменной, включается в исходный файл, содержащий определение переменной, что позволяет компилятору проверить совпадают ли они.

Хотя обмен переменными между файлами является давней практикой в мире Си, он имеет существенные недостатки. В Разделе 19.2, мы увидим, в чем проблемы и научимся разрабатывать программы, которым не нужны общие переменные.

Вложенные включения

Файл заголовка может содержать директивы `#include`. Хотя эта практика может показаться немного странной, на практике она может быть весьма полезна. Рассмотрим файл `stack.h`, который содержит следующие прототипы:

```
int is_empty (void);  
int is_full (void);
```

Поскольку эти функции возвращают только 0 или 1, рекомендуется объявить их возвращаемый тип как Bool вместо int, где Bool - это тип, который мы определили ранее в этом разделе:

```
Bool is_empty (void);  
Bool is_full (void);
```

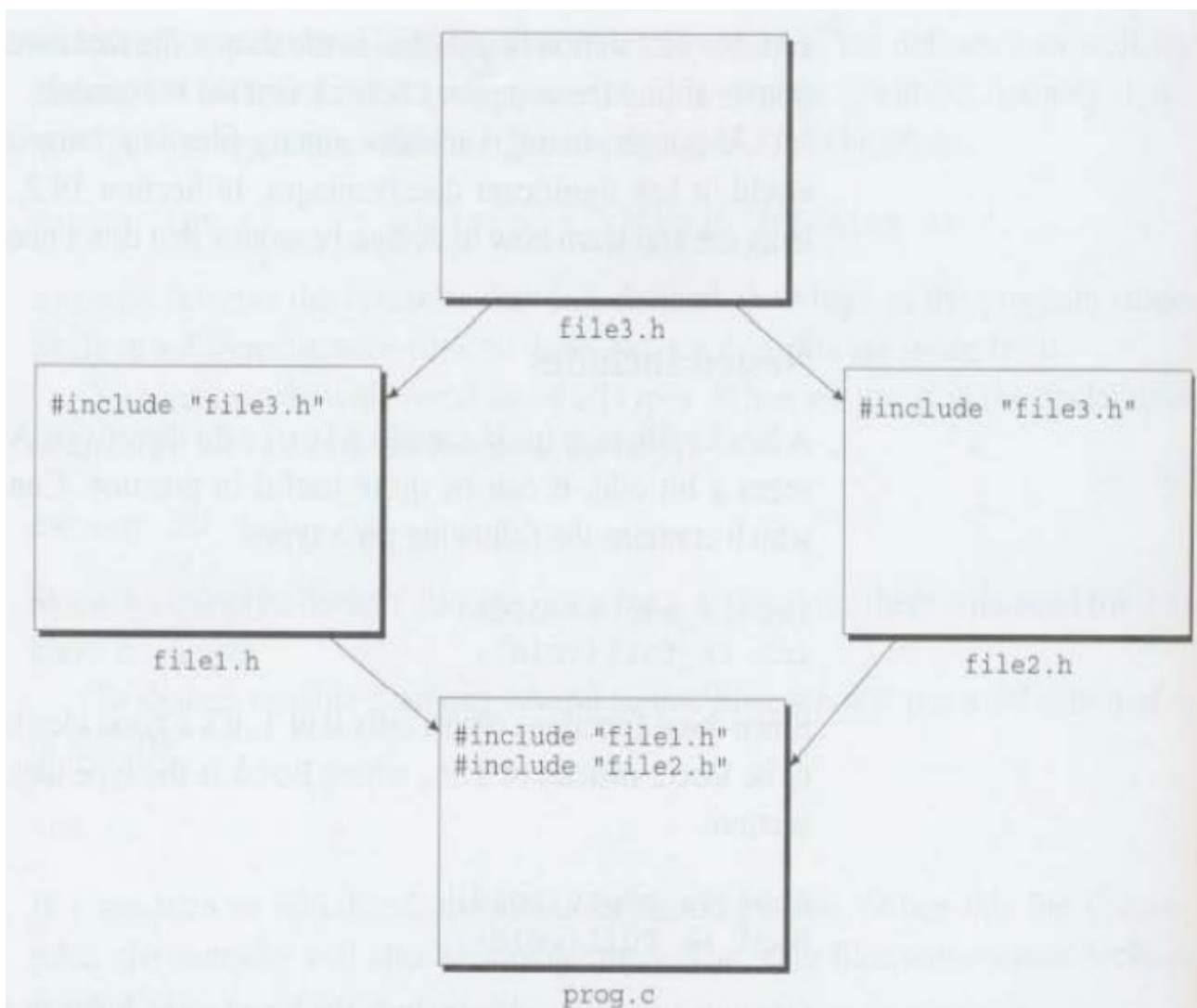
Конечно, нам нужно включить файл boolean.h в stack.h, чтобы определение Bool было доступно, когда stack.h скомпилирован. (В C99 мы включили бы <stdbool.h> вместо boolean.h и объявили бы, что возвращаемые типы двух функций будут bool, а не Bool). Традиционно Си - программисты избегают вложенных включений. (Ранние версии Си не допускали их вообще) однако предубеждение относительно вложенных включений в значительной степени исчезло, отчасти потому, что вложенные включения являются обычной практикой в C++.

Защита файлов заголовков

Если исходный файл содержит один и тот же файл заголовка дважды, могут возникнуть ошибки компиляции. Эта проблема часто возникает, когда файлы заголовков содержат другие файлы заголовков.

Например, предположим, что file1.h включает file3.h, file2.h включает file3.h, а prog.c включает файлы file1.h и file2.h

(см. рисунок в верхней части следующей страницы). Когда компилируется `prog.c`, `file3.h` будет скомпилирован дважды. Включение одного и того же файла заголовка дважды не всегда приводит к ошибке компиляции. Если файл содержит только определения макросов, прототипы функций и / или объявления переменных, не будет никаких трудностей. Однако если файл содержит определение типа, мы получим ошибку компиляции. На всякий случай рекомендуется защитить все заголовочные файлы от многократного включения; таким образом, мы можем добавить определения типов в файл позже без риска того, что мы забудем защитить файл. Кроме того, мы могли бы сэкономить некоторое время во время разработки программы, избегая ненужной перекompиляции того же файла заголовка.



Для защиты заголовочного файла мы заключим его содержимое в пару `#ifndef` - `#endif`.

Например, файл `boolean.h` может быть защищён следующим образом:

```
#ifndef BOOLEAN_H  
define BOOLEAN_H  
#define TRUE 1  
#define FALSE 0  
typedef int BOOL;  
#endif
```

Когда этот файл включается в первый раз, макрос `BOOLEAN_H` не будет определён, поэтому препроцессор позволит линиям между `#ifndef` и `#endif` остаться.

Но если файл будет включен во второй раз, препроцессор удалит строки между `#ifndef` и `#endif`.

Имя макроса (`BOOLEAN_H`) не имеет значения. Однако использование имени файла заголовка (`boolean.h`) позволяет избежать конфликтов с другими макросами. Так как мы не можем назвать макрос `BOOLEAN.H` (идентификаторы не могут содержать точки), имя типа `BOOLEAN_H` является хорошей альтернативой.

Директивы `#error` часто помещаются в заголовочные файлы для проверки условий, при которых файл заголовка не должен быть включён.

Например, предположим, что файл заголовка использует функцию, которая не существовала до исходного стандарта C89. Чтобы предотвратить использование файла заголовка с более старыми, нестандартными компиляторами, он может содержать директиву `ifndef`, которая проверяет наличие макроса `__STDC__`:

```
#ifndef __STDC__
```

```
#error This header requires a Standard C compiler
```

```
#endif
```

15.3. Разделение программы на файлы

Давайте теперь используем то, что знаем о файлах заголовков и файлах источников, чтобы разработать простой способ деления программы на файлы. Мы сосредоточимся на функциях, но те же принципы применимы и к внешним переменным. Будем считать, что программа уже разработана, то есть мы решили, какие функции потребуются программе и как расположить функции в логически связанные группы (дизайн программы мы обсудим в Главе 19).

Вот как мы продолжим. Каждый набор функций будет идти в отдельный исходный файл (давайте использовать имя `foo.c`

для одного такого файла).

Кроме того, мы создадим файл заголовка с тем же именем, что и исходный файл, но с расширением *.h* (*foo.h*, в нашем случае). В *foo.h* мы помещаем прототипы для функций, определённых в *foo.c*. (Функции, предназначенные для использования только внутри *foo.c*, не обязательно - и не должны объявляться в *foo.h*. Функция *read_char* в следующей программе является примером.) Мы будем включать *foo.h* в каждый файл источника, который должен вызвать функцию, определённую в *foo.c*. Более того, мы будем включать *foo.h* в *foo.c*: так, чтобы компилятор мог проверить, что прототипы функций в *foo.h* соответствуют определениям в *foo.c*. Функция *main* войдёт в файл, имя которого соответствует имени программы - если мы хотим, чтобы программа была известна как *bar*, то *main* должна быть в *bar.c*, возможно, что есть другие функции в том же файле, что и *main*, если они не вызываются из других файлов в программе.

Форматирование текста

Чтобы проиллюстрировать технику, о которой мы только что говорили, давайте применим её к небольшой программе форматирования текста с именем *justify*. В качестве примера ввода для обоснования мы будем использовать файл с именем *quote*, который содержит следующую (плохо отформатированную) цитату из "Разработка языка программирования C" Денниса М. Ричи (в "Истории языка программирования": II edited by T. J.

Bergin, Jr., and R. Gr Gibson. Jr. Addison—Wesley. Reading. M. 1996, pages 67 L687):

C is quirky, flawed, and an enormous success. Although accidents of history surely helped, it evidently satisfied a need for a system implementation language efficient enough to displace assembly language, yet sufficiently abstract and fluent to describe algorithms and interactions in a wide variety of environments

— Denis M. Ritchie

Чтобы запустить программу из приглашения UNIX или Windows, мы должны ввести команду

`justify < quote`

Символ < информирует операционную систему о том, что `justify` будет считывать из файла, а не принимать ввод с клавиатуры. Эта функция, поддерживаемая UNIX, Windows и другими операционными системами, называется переназначением ввода.

При задании файла `quote` в качестве ввода программа `justify` выдаст следующий результат:

C is quirky, flawed, and an anemone success. Although accidents of history surely helped, it evidently satisfied a need for a system implementation language efficient enough to displace assembly language, yet sufficiently abstract and fluent to describe algorithms and interactions in a wide variety of environments. -- Dennis M. Ritchie

Результат justify обычно появляется на экране, но мы можем сохранить его в файле с помощью перенаправления вывода: `justify <quote> newquote`

Результат justify (*выравнивания*) войдёт в файл newquote. В общем случае вывод justify должен быть идентичен вводу, за исключением того, что пробелы и пустые строки удаляются, а строки заполняются и выравниваются. "Заполнение" строки означает добавление слов до тех пор, пока ещё одно слово не вызовет переполнение строки. "Выравнивание" строки означает добавление дополнительных пробелов между словами, так что каждая строка имеет точно такую же длину (60 символов). Выравнивание должно быть сделано так, чтобы пробелы между словами в строке были равны (или почти равны). Последняя строка вывода не будет выровнена. Будем считать, что длина слова не превышает 20 символов (знак препинания считается частью слова, к которому оно примыкает). Конечно,

это немного ограничительно, но как только программа написана и отлажена, мы сможем легко увеличить этот предел до такой степени, чтобы он никогда не превышался. Если программа встречает более длинное слово. она должна игнорировать все символы после первых 20, заменяя их одной звёздочкой, например, должно быть напечатано слово `antidisestablishmentarianism` как `antidisestablishment*`

Теперь, когда мы поняли, что должна делать программа, пришло время подумать о дизайне. Начнём с того, что программа не должна писать слова по одному, когда их читает. Вместо этого она должна будет хранить их в "буфере строк", пока не накопится достаточно, чтобы заполнить строку. После дальнейших размышлений мы решаем, что сердцевиной программы будет цикл, который проходит примерно так:

```
for (;;) {  
    прочитать слово;  
    If (не удаётся прочитать слово) {  
        записать содержимое буфера строк без  
        выравнивания;  
        завершить программу;  
    }  
}
```

```
If (слово не помещается в буфер строк){  
    записать содержимое буфера строк с  
    выравниванием;  
    очистить буфер строк;  
}  
добавить слово в буфер строк;  
}
```

Так как нам понадобятся функции, которые имеют дело со словами и функции, которые имеют дело с буфером строк, давайте разделим программу на три исходных файла, поместив все функции, связанные со словами, в один файл (word.c) и все функции, связанные с буфером строк в другой файл (line.c.). Третий файл (justify.c) будет содержать функцию main.

В дополнение к этим файлам нам понадобятся два файла заголовка, word.h и line.h. Файл word.h будет содержать прототипы для функций в word.c; line.h будет играть аналогичную роль для line.c.

Изучая главный цикл, мы видим, что единственная связанная с словами функция, которая нам понадобится, - это read_word.

(Если read_word не может прочитать слово, потому что она достигло конца входного файла, мы подадим сигнал главному циклу, делая вид, что читаем "пустое" слово.)

803

Следовательно, файл word.h является небольшим:

```
//word.h
```

```
#ifndef WORD_H
```

```
#define WORD_H
```

```
/* read_word: читает следующее слово из ввода и сохраняет *  
его в word. Делает слово пустым, если ни одно слово не  
может быть прочитано из-за конца файла.
```

```
Обрезает слово, если его длина превышает len.
```

```
/*
```

```
void read_word(char *word, int len);
```

```
#endif
```

Обратите внимание, что макрос WORD_H защищает word.h от включения более одного раза.

Хотя word.h на самом деле не нуждается в этом, рекомендуется защищать все файлы заголовков таким образом.

Файл `line.h` не будет таким коротким, как `word.h`. Наша схема основного цикла показывает необходимость в функциях, которые выполняют следующие операции:

Запись содержимого буфера строк без выравнивания

Определить, сколько символов осталось в буфере строк

Записать содержимое буфера строк с выравниванием

Очистить буфер строк

Добавить слово в буфер строк

Мы назовём эти функции `flush_line`, `space_remaining`, `write_line`, `clear_line` и `add_word`.

Вот как выглядит файл заголовка `line.h`:

```
// line.h
```

```
#ifndef LINE_H
```

```
#define LINE_H
```

```
// clear_line: Очищает текущую строку.
```

```
/ * add_word: Добавляет слово в конец текущей строки.
```

```
Если это не первое слово в строке, перед словом ставится  
один пробел.
```

```
*/
```

```
void add_word(const char *word);
```



```
/* space_remaining: Возвращает количество символов,  
оставшихся в текущей строке.
```

```
*/
```

```
int space_remaining(void);
```

```
//write_line: Записывает текущую строку с выравниванием.
```

```
void write_line(void);
```

```
/* flush_line: Записывает текущую строку без выравнивания.  
Если строка пуста, ничего не делает.
```

```
* /
```

```
void flush_line(void);
```

```
#endif
```

Прежде чем писать файлы word.c и line.c, мы можем использовать функции, объявленные в word.h и line.h, для записи justify.c основной программы. Написание этого файла в основном связано с переводом нашего дизайна исходного цикла на Си.

//justify.c

/ Форматирование текстового файла */*

#include <string.h>

#include "line.h"

#include "word.h"

#define MAX_WORD_LEN 20

int main(void){

 char word[MAX_WORD_LEN+2];

 int word_len;

 clear_line();

 for (;;) {

 read_word(word, MAX_WORD_LEN+1);

 word_len = strlen(word);

 if (word_len == 0) {

 flush_line();

 return 0;

```

    }
    if (word_len > MAX_WORD_LEN)
        word[MAX_WORD_LEN] = '*';
    if (word_len + 1 > space_remaining()) {
        write_line();
        clear_line();
    }
    add_word(word);
}
}

```

Как `line.h`, так и `word.h` предоставляют компилятору доступ к прототипам функций в обоих файлах при компиляции `justify.c`.

`main` использует трюк для обработки слов, которые превышают 20 символов. Когда она вызывает `read_word`, `main` говорит ей усечь любое слово, которое превышает 21 символ. После того, как `read_word` возвращается, `main` проверяет, содержит ли `word` строку длиной более 20 символов. Если это так, прочитанное слово должно содержать не менее 21 символа (до усечения), поэтому `main` заменяет 21-й символ слова звёздочкой. Теперь пришло время написать `word.c`. Хотя файл заголовка `word.h` имеет прототип только для одной функции, `read_word`, мы можем добавить дополнительные функции в `word.c`, если нам нужно. Как оказалось, `read_word` легче писать, если добавить

небольшую "вспомогательную" функцию `read_char`. Мы назначаем `read_char` задачу чтения единственного символа и, если это будет символ новой строки или табуляции, конвертировать его в пробел. Наличие `read_word` вызывающую `read_char` вместо `getchar`, решает проблему обработки символов новой строки и табуляции как пробелов. Вот `word.c` file:

```
//word.c
```

```
#include <stdio.h>
```

```
#include "word.h"
```

```
int read_char(void){
```

```
    int ch = getchar();
```

```
    if (ch == '\n' || ch == '\t')
```

```
        return ' ';
```

```
    return ch;
```

```
}
```

```
void read_word(char *word, int len){
```

```
    int ch, pos = 0;
```

```
    while ((ch = read_char()) != ' ')
```

```
        ;
```

```
while (ch != ' ' && ch != EOF) {  
    if (pos < len)  
        word[pos++] = ch;  
    ch = read_char();  
}  
word[pos] = '\0';  
}
```

Прежде чем мы обсудим `read_word`, пару замечаний относительно использования `getchar` в функции `read_char`.

Во-первых, `getchar` возвращает значение `int` вместо значения `char`: вот почему переменная `ch` в `read_char` объявлена с типом `int` и почему возвращаемый тип `read_char` — `int`.

Кроме того, `getchar` возвращает значение `EOF`, когда он не может продолжить чтение (обычно потому, что он достиг конца входного файла).

`read_word` состоит из двух циклов.

Первый цикл пропускает пробелы, останавливаясь на первом непустом символе (`EOF` не является пробелом, поэтому цикл останавливается, если он достигает конца входного файла.)

Второй цикл считывает символы до появления пробела или `EOF`. Тело цикла сохраняет символы в `word` до достижения предела `len` после этого цикл продолжает чтение символов, но не сохраняет их.

Конечная инструкция в `read_word` заканчивает слово нулевым символом, тем самым делая его строкой. Если `read_word` встречает EOF до определения непустого символа, `pos` будет 0 в конце, делая слово пустой строкой. Остался только один файл - `line.c`, который предоставляет определения функций, объявленных в файле `line.h`.

`line.c` также потребуются переменные, чтобы отслеживать состояние буфера строк. Одна переменная, `line`, сохранит символы в текущей строке. Строго говоря, `line` - единственная переменная, в которой мы нуждаемся. Однако для скорости и удобства мы будем использовать две другие переменные: `line_len` (количество символов в текущей строке) и `num_words` (количество слов в текущей строке). Вот файл `line.c`:

```
//line.c
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include "line.h"
```

```
#define MAX_LINE_LEN 60
```

```
char line[MAX_LINE_LEN+1];
```

```
int line_len = 0;
```

```
int num_words = 0;
```

811

```
void clear_line(void){
    line[0] = '\0';
    line_len = 0;
    num_words = 0;
}
```

```
void add_word(const char *word){
    if (num_words > 0) {
        line[line_len] = ' ';
        line[line_len+1] = '\0';
        line_len++;
    }
    strcat(line, word);
    line_len += strlen(word);
    num_words++;
}
```

```
int space_remaining(void){
    return MAX_LINE_LEN - line_len;
}
```

```
void write_line(void){
    int extra_spaces, spaces_to_insert, i, j;
```

812

```
extra_spaces = MAX_LINE_LEN - line_len;
for (i = 0; i < line_len; i++) {
    if (line[i] != ' ')
        putchar(line[i]);
    else {
        spaces_to_insert = extra_spaces / (num_words - 1);
        for (j = 1; j <= spaces_to_insert + 1; j++)
            putchar(' ');
        extra_spaces -= spaces_to_insert;
        num_words--;
    }
}
putchar('\n');
}

void flush_line(void){
    if (line_len > 0)
        puts(line);
}
```

Большинство функций в line.c написать легко. Единственный замысловатый случай - write_line, которая пишет строку с выравниванием.

write_line записывает символы в строке один за другим, делая паузу на пробелах между каждой парой слов, чтобы написать

дополнительные пробелы, если это необходимо. Количество дополнительных пробелов хранится в `spaces_to_insert`, который имеет значение $\text{extra_spaces} / (\text{num_words} - 1)$, где `extra_spaces` изначально является разницей между максимальной длиной строки и фактической длиной строки. Поскольку `extra_spaces` и `num_words` меняются после каждого напечатанного слова, `spaces_to_insert` также изменится. Если `extra_spaces` равно 10 изначально и `num_words` равно 5, то за первым словом последуют 2 дополнительных пробела, за вторым-2, за третьим-3, а за четвертым-3.

15.4 Создание многофайловой программы

В Разделе 2.1 мы рассмотрели процесс компиляции и компоновки программы, которая помещается в один файл. Давайте расширим это обсуждение, чтобы охватить программы с несколькими файлами. Создание большой программы требует тех же основных шагов, что и создание маленькой:

■ **Компиляция.** Каждый файл источника в программе должен быть скомпилирован отдельно. (Файлы заголовков не нужно компилировать: содержимое файла заголовка автоматически скомпилируется всякий раз, когда скомпилирован исходный файл, который включает его.) Для каждого исходного файла, компилятор создаёт файл, содержащий объектный код. Эти файлы, известные как объектные файлы - имеют расширение `.o` в UNIX и `.obj` в Windows.

■ **Связывание.** Компоновщик объединяет объектные файлы, созданные на предыдущем шаге, вместе с кодом библиотечных функций для создания исполняемого файла. Помимо прочего, компоновщик отвечает за разрешение внешних ссылок, оставленных компилятором.

(Внешняя ссылка возникает, когда функция в одном файле вызывает функцию, определённую в другом файле, или обращается к переменной, определённой в другом файле.)

Большинство компиляторов позволяют построить программу за один шаг. Например, с помощью компилятора GCC мы используем следующую команду для построения программы `justify` из раздела 15.3:

```
gcc -o justify justify.c line.c word.c
```

Три исходных файла сначала компилируются в объектный код. Затем объектные файлы автоматически передаются компоновщику, который объединяет их в один файл. Опция `-o` указывает, что мы хотим, чтобы исполняемый фрагмент был назван `justify`.

Makefiles

Размещение имён всех исходных файлов в командной строке быстро становится утомительным. В любом случае, мы

можем потратить много времени на восстановление программы, если мы перекомпилируем все исходные файлы, а не только те, которые были затронуты нашими последними изменениями.

Чтобы упростить создание больших программ, UNIX создал концепцию makefile, файла, содержащего информацию, необходимую для построения программы. Makefile не только перечисляет файлы, которые являются частью программы, но также описывает зависимости между файлами.

Предположим, что файл foo.c включает в себя файл bar.h. Мы говорим, что foo.c "зависит" от bar.h, потому что изменения в bar.h потребуют от нас перекомпилировать foo.c.

Вот файл makefile UNIX для программы justify. makefile использует GCC для компиляции и компоновки:

```
justify: justify.o word.o line.o
    gcc -o justify justify.o word.o line.o
justify.o: justify.c word.h line.h
    gcc -c justify.c
word.o: word.c word.h
    gcc -c word.c
line.o: line.c line.h
    gcc -c line.c
```

Существует четыре группы строк; каждая группа именуется

правилом. Первая строка каждого правила содержит целевой файл, за которым следуют файлы, от которых он зависит. Вторая строка - это команда, которая должна быть выполнена, если цель должна быть перестроена из-за изменения одного из зависимых файлов. Давайте посмотрим на первые два файла; последние два похожи.

В первом правиле целью является `justify` (исполняемый файл):
`justify: justify.o word.o line.o`

`gcc -o justify justify.o word.o line.o`

В первой строке указывается, что `justify` зависит от файлов `justify.o`, `word.o` и `line.o`, если какой-либо из этих трёх файлов изменился с момента последней сборки программы, то `justify` нужно перестроить.

Команда в следующей строке показывает, как должна выполняться перестройка (с помощью команды `gcc` для связывания трёх объектных файлов). Во втором правиле `justify.o` является целью:

`justify.o: justify.c word.h line.h`

`gcc -c justify.c`

Первая строка указывает на то, что `justify.o` необходимо перестроить, если произошли изменения для `justify.c`, `word.h` или `line.h`.

(Причиной упоминания `word.h` и `line.h` является то, что `justify.c` включает оба этих файла, поэтому на него потенциально влияет изменение любого из них.) Следующая

строка показывает, как обновить justify.o (путём перекомпиляции justify.c).

Параметр -o указывает компилятору скомпилировать файл justify.c, но не пытаться связать его. После того, как мы создали makefile для программы, мы можем использовать утилиту make для сборки (или перестроения) программы. Проверив время и дату, связанные с каждым файлом в программе, make может определить, какие файлы устарели.

Затем она вызывает команды, необходимые для перестройки программы. Если вы хотите попробовать make, имейте ввиду следующее:

- Каждой команде в makefile должен предшествовать символ табуляции, а не ряд пробелов. (В нашем примере команды имеют отступ в восемь пробелов, но на самом деле это один символ табуляции.)

- makefile обычно хранится в файле с именем Makefile (или makefile). При использовании утилиты make она автоматически проверяет текущий каталог на наличие файла с одним из этих имён.

- Чтобы вызвать make, используйте команду

`make target`

где *target* - одна из целей, перечисленных в файле makefile.

Для сборки исполняемого файла `justify` с помощью `makefile` мы будем использовать команду `make justify`.

■ Если цель не указана при вызове `make`, она создаст цель первого правила. Например, команда

`make`

построит исполняемый файл `justify`, так как `justify` является первой целью в нашем `makefile`. За исключением этого специального свойства первого правила порядок правил в `makefile` произволен.

`make` достаточно сложна, о ней написаны целые книги, поэтому мы не будем пытаться углубляться в её тонкости. Скажем так, настоящие `makefile` обычно не так просты для понимания, как наш пример. Существует множество методов, которые уменьшают избыточность в файлах `makefile` и упрощают их изменение; в то же время эти методы значительно ухудшают их читаемость.

Кстати, не все используют `makefile`. Другие инструменты обслуживания программы также популярны, включая "файлы проекта", поддерживаемые некоторыми интегрированными средами разработки.

Ошибки при компоновке

Некоторые ошибки, которые не могут быть обнаружены во время компиляции, будут найдены во время компоновки. В частности, если определение функции или переменной отсутствует в программе, компоновщик не сможет разрешить внешние ссылки на неё, в результате чего появится сообщение "неопределённый символ" или "неопределённая ссылка".

Обнаруженные ошибки линкера (компоновщика) обычно легко исправить. Вот некоторые из наиболее распространённых причин:

- Орфографическая ошибка. Если имя переменной или функции указано с ошибкой, компоновщик сообщит о ней как об отсутствующей. Например, если функция `read_char` определена в программе, но вызвана как `read_cahe`, компоновщик сообщит, что `read_sahr` отсутствует.

- Отсутствующие файлы. Если компоновщик не может найти функции, находящиеся в файле `foo.c`, он может не знать о файле. Проверьте файл `makefile` или файл проекта, чтобы убедиться, что в нем указан `foo.c`.

- Отсутствующие библиотеки. Возможно, компоновщик не сможет найти все библиотечные функции, используемые в программе.

Классический пример встречается в программах UNIX, которые используют заголовок `<math.h>`. Простое включение заголовка в программу может быть недостаточным: во многих версиях UNIX необходимо указать опцию `-lm` при

компоновке программы, в результате чего компоновщик будет искать системный файл, содержащий скомпилированные версии функций `<math.h>`.

Не использование этой опции, может вызвать сообщение "undefed reference" (неопределённые ссылки) во время связывания.

Пересборка программы

Во время разработки программы редко возникает необходимость компилировать все её файлы. Большую часть времени мы будем тестировать программу, вносить изменения, а затем снова строить программу. Чтобы сэкономить время, процесс пересборки должен перекомпилировать только те файлы, которые могут быть затронуты последними изменениями. Давайте предположим, что мы разработали нашу программу так, как описано в Разделе 15.3, с заголовочным файлом для каждого исходного файла. Чтобы увидеть, сколько файлов потребуется перекомпилировать после изменения, необходимо рассмотреть две возможности. Первая возможность заключается в том, что изменение затрагивает один исходный файл. В этом случае необходимо перекомпилировать только этот файл. (После этого, конечно, вся программа должна быть скомпонована.) Рассмотрим программу `justify`. Предположим, мы решили уплотнить функцию `read_char` в `word.c` (изменения выделены жирным шрифтом):


```
int read_char (void){
    int ch = getchar();
    return (ch == '\n' || ch == '\t') ? '': ch;
}
```

Эта модификация не влияет на word.h, поэтому нам нужно только перекомпилировать word.c и повторно скомпоновать программу.

Вторая возможность заключается в том, что изменение влияет на заголовок файла. В этом случае мы должны перекомпилировать все файлы, которые содержат файл заголовка, так как они потенциально могут быть затронуты изменением. (Некоторые из них могут и не быть затронуты, но они платят за консерватизм.) В качестве примера рассмотрим функцию read_word в программе justify.

Обратите внимание, что main вызывает strlen сразу после вызова read_word, чтобы определить длину только что прочитанного слова. Поскольку read_word уже знает длину слова (переменная pos read_word отслеживает длину), кажется глупым использовать strlen. Изменить read_word, чтобы вернуть длину слова не сложно. Сначала мы меняем прототип read_word в word.h:

```
/*
 * read_word: Reads the next word from the input and
 * stores it in word. Makes word empty if no
 * word could be read because of end-of-file.
 */
```

822

* Truncates the word if its length exceeds *
* len. **Returns the number of characters stored.** *

*****/

```
int read_word(char *word, int len);
```

Конечно, мы стараемся изменить комментарий, который сопровождает `read_word`. Затем мы изменим определение `read_word` в `word.c`:

```
int read_word(char *word, int len){
    int ch, pos = 0;

    while ((ch = read_char()) == ' ')
        ;
    while (ch != ' ' && ch != EOF) {
        if (pos < len)
            word[pos++] = ch;
        ch = read_char();
    }
    word[pos] = '\0';
    return pos;
}
```

823

Наконец, мы модифицируем justify, а, удалив include <string.h> и изменив main следующим образом:

```
int main(void){
    char word[MAX_WORD_LEN+2];
    int word_len;
    clear_line();
    for (;;) {
        word_len = read_word(word, MAX_WORD_LEN+1);
        if (word_len == 0) {
            flush_line();
            return 0;
        }
        if (word_len > MAX_WORD_LEN)
            word[MAX_WORD_LEN] = '*';
        if (word_len + 1 > space_remaining()) {
            write_line();
            clear_line();
        }
        add_word(word);
    }
}
```

```
}
```

После того, как мы внесём эти изменения, мы перестроим программу `justify`, перекомпилируем `word.c` и `justify.c`, а затем перекомпонуюем. Нет необходимости перекомпилировать `line.c`, который не включает `word.h` и поэтому не будет заражён изменениями. С компилятором `GCC` мы могли бы использовать следующую команду для перестроения программы:

```
gcc -o justify justify.c word.c line.o
```

Обратите внимание на упоминание `line.o` вместо `line.c`. Одним из преимуществ использования `make`-файлов является то, что перестройка обрабатывается автоматически.

Изучая дату каждого файла, утилита `make` может определить, какие файлы изменились с момента последней компоновки программы. Затем она перекомпилирует эти файлы вместе со всеми файлами, которые зависят от них, прямо или косвенно. Например, если мы сделаем указанные изменения в `word.h`, `word.c` и `justify.c`, а затем перестроим программу `justify`, `make` выполнит следующие действия:

1. Создаст `justify.o`, скомпилировав `justify.c` (потому что изменены `justify.c` и `word.h`).
2. Создаст `word.o` путём компиляции `word.c` (потому что были изменены `word.c` и `word.h`).
3. Создаст `justify`, связав `justify.o`, `word.o` и `line.o` (потому что изменены `justify.o` и `word.o`)

Определение макросов вне программы

Компиляторы C обычно предоставляют некоторый метод указания значения макроса во время компиляции программы. Эта способность позволяет легко изменять значение макроса без редактирования каких-либо файлов программы. Это особенно ценно, когда программы создаются автоматически, используя make-файлы. Большинство компиляторов (включая GCC) поддерживают параметр `-D`, который позволяет указать значение макроса в командной строке:

```
gcc -DDEBUG = 1 foo.c
```

В этом примере макрос `DEBUG` определяется как значение 1 в программе `foo.c` так же, как если бы строка

```
#define DEBUG 1
```

появилась в начале `foo.c`.

Если параметр `-D` называет макрос без указания его значения, то принимается значение 1. Многие компиляторы также поддерживают опцию `-U`, которая "открепляет" макрос, как будто с помощью `#undef`. Мы можем использовать её, чтобы отменить определение предопределённого макроса или макроса, который был определён ранее в командной строке с помощью `-D`.

Вопросы и ответы

Вопрос: У вас нет примеров, использующих директиву `#include` для включения исходного файла.

Что произойдёт, если мы это сделаем?

Ответ: Это не очень хорошая практика, хотя она не является недопустимой. Вот пример проблемы, с которой вы можете столкнуться. Предположим, что `foo.c` определяет функцию *f*, которая нам понадобится в `bar.c` и `baz.c`, поэтому мы ставим директиву

```
#include "foo.c"
```

как в `bar.c`, так и в `baz.c`. Каждый из этих файлов будет хорошо компилироваться. Проблема возникает позже, когда компоновщик обнаруживает две копии объектного кода для *f*, конечно, нам бы сошло с рук включение `foo.c`, только в один из файлов, например, в `bar.c`, но не в `baz.c`; чтобы избежать проблем, лучше использовать `#include` только с заголовочными файлами, а не с исходными файлами.

Вопрос: Каковы точные правила поиска для директивы `#include`?

Ответ: Это зависит от вашего компилятора. Стандарт Си преднамеренно расплывчат в описании `#include`.

Если имя файла заключено в скобки, препроцессор просматривает "последовательность мест, определённых реализацией", как уклончиво выражается стандарт.

Если имя файла заключено в кавычки, "выполняется поиск в порядке определённом реализацией" и, если он не найден, выполняется поиск так, как если бы его имя было заключено в скобки.

Причина этой неуверенности проста:

не все операционные системы имеют иерархические (древовидные) файловые системы. А чтобы всё стало ещё интересней, стандарт не требует, чтобы имена, заключённые в скобки, вообще были именами файлов, оставляя открытой возможность того, что директивы `#include`, использующие `<>`, могут обрабатываться полностью в компиляторе.

Вопрос: Я не понимаю, почему каждый исходный файл нуждается в своём собственном файле заголовков. Почему бы не иметь один большой файл заголовков, содержащий определения макросов, определения типов и прототипы функций? Включив этот файл, каждый исходный файл будет иметь доступ ко всей необходимой общей информации.

Ответ: Конечно, подход "один большой заголовок" работает, его используют многие программисты, и у него есть

преимущество:

с одним только файлом заголовков, меньше файлов для управления. Однако для крупных программ недостатки этого подхода, как правило, перевешивают его преимущества. Использование одного файла заголовков не даёт никакой полезной информации кому-либо, читающему программу позже. С несколькими файлами заголовков читатель может быстро увидеть, какие части программы используются конкретным файлом источника. Но это не все. Поскольку каждый файл источника зависит от большого файла заголовков, его изменение приведёт к перекомпиляции всех исходных файлов - значительному недостатку в большой программе. Хуже того, файл заголовков, вероятно, будет часто меняться из-за большого количества информации, которую он содержит.

Вопрос: В главе говорится, что общий массив должен быть объявлен следующим образом:

```
extern int a[] ;
```

Поскольку массивы и указатели тесно связаны, будет ли правильным писать

```
extern int *a ;
```

вместо этого?

Ответ: Нет. При использовании в выражениях массивы "превращаются" в указатели (Мы заметили это поведение, когда имя массива используется как аргумент в вызове функции.) Однако в объявлениях переменных массивы и указатели представляют собой разные типы.

Вопрос: Будет ли ущерб от того, что исходный файл содержит заголовки, которые ему действительно не нужны?

Ответ: Нет, если заголовок не содержит объявление или определение, которое конфликтует с таковым в исходном файле. В противном случае, худшее, что может произойти, - это незначительное увеличение времени компиляции исходного файла.

Вопрос: Мне нужно было вызвать функцию в файле foo.c, поэтому я включил соответствующий файл заголовка, foo.h. Моя программа компилируется, но не связывается. Почему?

Ответ: Компиляция и компоновка полностью разделены в Си. Файлы заголовков существуют для предоставления информации компилятору, а не компоновщику: если вы хотите вызвать функцию в foo.c, тогда вы должны убедиться, что foo.c скомпилирован и что линкер знает, что он должен искать

файл объекта для foo.c, чтобы найти функцию. Обычно это означает имя foo.c в программе makefile или файле проекта.

Вопрос: Если моя программа вызывает функцию в <stdio.h>, означает ли это, что все функции в <stdio.h> будут связаны с программой?

Ответ: Нет. Включение <stdio.h> (или любого другого заголовка) не влияет на привязку. В любом случае большинство компоновщиков свяжут только те функции, которые действительно нужны вашей программе.

Вопрос: Где я могу взять утилиту make?

Ответ: make - стандартная утилита UNIX. Версия GNU, известная как GNU Make, включена в большинство дистрибутивов Linux. Она также доступна прямо из Free Software Foundation (www.gnu.org/software/make/).

Упражнения

К Разделу 15.1

1. В Разделе 15.1 перечислены некоторые преимущества деления программы на несколько исходных файлов
- (а) Опишите несколько других преимуществ.

(b) Опишите некоторые недостатки.

К Разделу 15.2

W2. Что из нижеследующего не должно быть помещено в файл заголовка? Почему нет?

- (a) Прототипы функций
- (b) Определения функций
- (c) Макроопределения
- (d) Определения типов

3. Мы видели, что запись `#include <file>` вместо `#include "file"` может не работать, если файл написан нами. Будут ли какие-либо проблемы с написанием `#include "file"` вместо `include <file>`, если файл является системным заголовком?

4. Предположим, что `debug.h` - это файл заголовка со следующим содержимым:

```
ifdef DEBUG
define PRINT_DEBUG(n) printf("Value of "#n": %d\n",
n);
#else
#define PRINT_DEBUG(n)
#endif
```

832

Пусть testdebug.c является следующим исходным файлом:

```
#include <stdio.h>

#define DEBUG

#include "debug.h"

int main (void){
    int i=1,j=2,k=3;
#ifdef DEBUG
    printf"Output if DEBUG is defined: \n");
#else
    printf("Output if DEBUG is not defined:\n");
#endif
    PRINT_DEBUG (i);
    PRINT_DEBUG (j);
    PRINT_DEBUG (k);
    PRINT_DEBUG (i + j);
    PRINT_DEBUG (2 * i + j - k);
    return 0;
}
```

(a) Каков результат, когда программа выполняется?

(b) Что такое вывод, если "define директива удалена из testdebug.c?

(c) Объясните, почему выбросы отличаются в частях (a) и (b).

(d) Требуется ли определение макроса DEBUG до того, как debug.h будет включён для того, чтобы PRINT_DEBUG имел желаемый эффект? Обосновать ответ.

5. Предположим, что программа состоит из трёх исходных файлов - main.c, f1.c и f2.c - плюс два файла заголовка, f1.h и f2.h. Все три исходных файла включают f1.h, но только f1.c и f2.c включают f2.h.

Напишите make-файл для этой программы, предполагая, что компилятор является gcc и что исполняемый файл должен называться demo.

W6. Следующие вопросы относятся к программе, описанной в упражнении 5.

(a) Какие файлы необходимо скомпилировать, когда программа будет построена в первый раз?

(b) Если f1.c изменяется после сборки программы, какие файлы необходимо перекомпилировать?

(c) Если f1.h изменяется после сборки программы, какие файлы необходимо перекомпилировать?

(d) Если f2.c изменяется после сборки программы, какие

файлы необходимо перекомпилировать?

(е) Если `f2.h` изменяется после сборки программы, какие файлы необходимо перекомпилировать?

Проекты по программированию

1. Программа `justify` Раздела 15.3 выравнивает строки, вставляя лишние пробелы между словами. Функция `write_line` работает так что слова ближе к концу строки, как правило, имеют несколько более широкие промежутки между ними, чем слова в начале. (Например, слова ближе к концу могут иметь три пробела между ними, в то время как слова, близкие к началу, могут быть разделены только двумя пробелами.) Улучшите программу, используя `write_line`, чередующую большие промежутки в конце и в начале строки.

2. Измените программу `justify` в Разделе 15.3, если функция `read_word` (а не `main`) сохранит символ `*` в конце слова, которое было усечено.

3. Измените программу `qsort.c` в Разделе 9.6, чтобы функции `quicksort` и `split` были в отдельном файле с именем `quicksort.c`. Создайте файл заголовка с именем `quicksorc.h`, который содержит прототипы для наших функций, а оба `qsorc.c` и `quicksort.c` включают этот файл.

4. Измените программу `remind.c` в Разделе 13.5, чтобы функция `read_line` находилась в отдельном файле с именем `readline.c`. Создайте файл заголовка с именем `readline.h`, который содержит прототип функции, и оба файла `remind.c` и `read_line.c` включают этот файл.

5. Измените Проект по программированию 6 из Главы 10, чтобы он имел отдельные файлы `stack.h` и `stack.c`, как описано в Разделе 15.2.

16 Структуры, объединения и перечисления

Функции задерживают связь, структуры данных стимулируют связь. Мораль: Структурируйте данные как можно позднее в процессе программирования.

В этой главе представлены три новых типа: структуры, объединения и перечисления.

Структура представляет собой набор значений (членов), возможно, различных типов. Объединение похоже на

структуру, за исключением того, что его члены используют одно и то же хранилище: в результате объединение может хранить один член одновременно, но не все члены одновременно. Перечисление - это целочисленный тип, значениям которого программист даёт имена. Из этих трёх типов, структуры являются самыми важными, поэтому я посвящу им большую часть главы, Раздел 16.1 показывает, как объявлять структурные переменные и выполнять над ними основные операции. Раздел 16.2 затем объясняет, как определить типы структуры, которые—среди прочего—позволяют нам писать функции, которые принимают аргументы структуры или возвращают структуры. Раздел 16.3 исследует, как массивы и структуры могут быть вложенными. Последние два раздела посвящены объединениям (Section 16.4) и перечислениям (Раздел 16.5).

16.1 Переменные структуры

Единственная структура данных, которую мы рассмотрели, - это массив. Массивы имеют два важных свойства. Во-первых все элементы массива имеют одинаковый тип. Во-вторых, для выбора элемента массива мы указываем его позицию (в виде целого индекса). Свойства структуры сильно отличаются от

свойств массива. Элементы структуры (её члены, выражаясь языком Си) не обязательно должны иметь один и тот же тип. Кроме того, члены структуры имеют имена: для выбора конкретного члена мы указываем его имя, а не позицию. Структуры могут показаться знакомыми, так как большинство языков программирования предоставляют аналогичную функцию. В некоторых языках структуры называются записями, а члены - полями.

Объявление переменных структуры

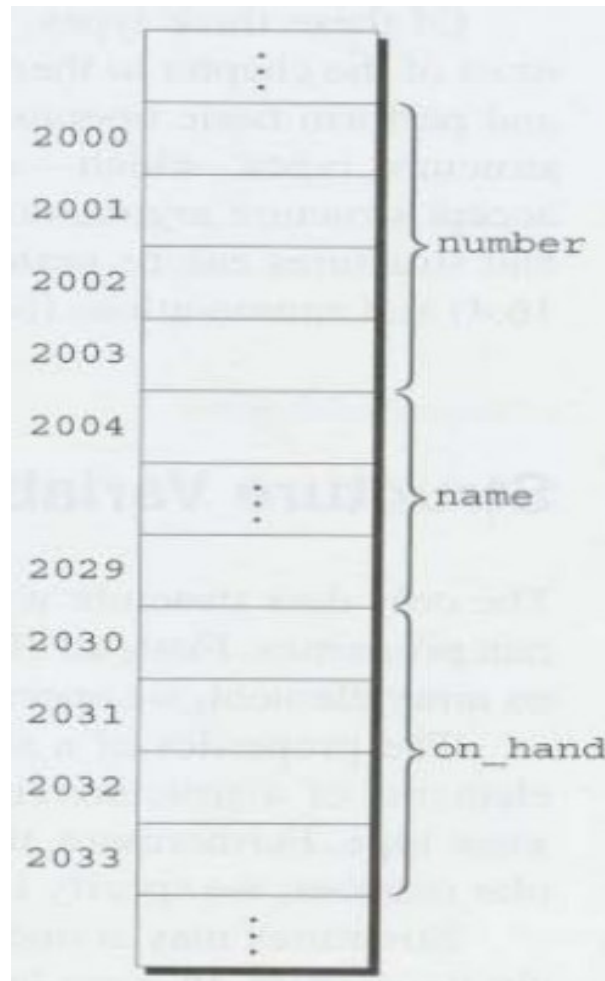
Когда нам нужно хранить коллекцию связанных элементов данных, структура является логичным выбором. Например, предположим, что нам нужно отслеживать детали на складе. Информация, которую нам нужно будет хранить для каждой детали, может включать номер детали (целое число), имя детали (строка символов) и количество деталей в наличии (целое число). Для создания переменных, которые могут хранить все три элемента данных, можно использовать следующее объявление:

```
struct{  
    int number;  
    char name [NAME_LEN+1];
```

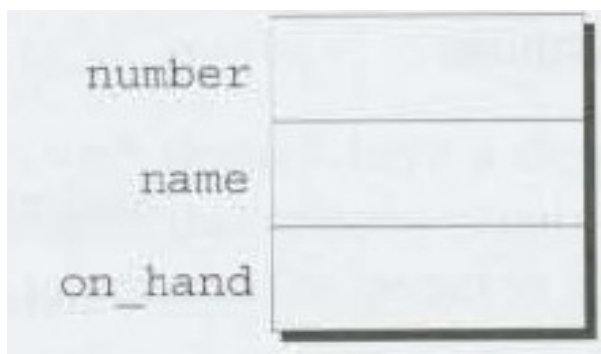
```
    int on_hand;  
} part1, part2;
```

Каждая структурная переменная имеет три члена: `number` (номер детали), `name` (название детали) и `on_hand` (количество в наличии). Обратите внимание, что это объявление имеет ту же форму, что и другие объявления переменных в Си: `struct {...}`, специфицирует тип, а `part1` и `part2` - переменные этого типа. Члены структуры хранятся в памяти в том порядке, в котором они объявлены. Чтобы показать, как выглядит переменная `part1` в памяти, предположим, что (1) `part1` находится по адресу 2000, (2) целые числа занимают четыре байта, (3) `NAME_LEN` имеет значение 25, и (4) между членами нет пробелов.

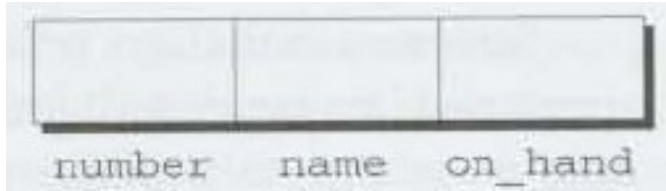
С этими предположениями часть 1 будет иметь внешний вид:



Обычно нет необходимости рисовать структуры в таких деталях. Я обычно показываю их более абстрактно, как серию ящиков:



Иногда я могу рисовать прямоугольники горизонтально, а не вертикально:



Значения членов появятся в ячейках позже; на данный момент я оставил их пустыми. Каждая структура представляет новую область видимости: любые имена, объявленные в этой области, не будут конфликтовать с другими именами в программе (в терминологии Си мы говорим, что каждая структура имеет отдельное пространство имён для своих членов.) Например, в одной программе могут отображаться следующие объявления:

```
struct {  
    int number;  
    char name[NAME_LEN+1];  
    int on_hand;  
}part1, part2;
```

```
struct {  
    char name[NAME_LEN+1];
```

841

```
int number;  
char sex;  
} employee1, employee2;
```

Члены number и name в структурах part1 и part2 не конфликтуют с членами number и name в employee1 и employee2.

Инициализация структурных переменных

Как и массив, структурная переменная может быть инициализирована в момент её объявления. Для инициализации структуры мы подготавливаем список значений, которые будут храниться в структуре, и заключаем его в фигурные скобки:

```
struct{  
    int number;  
    char name [NAME_LEN+1];  
    int on_hand;  
} part1 = {525,"Disk drive", 10},
```

```
part2 = {914, "Printer cable", 5};
```

Значения в инициализаторе должны отображаться в том же порядке, что и члены структуры. В нашем примере с `part1` член `number` будет равен 528, член `name` будет "Disk drive" и т. д. Вот как `part1` будет выглядеть после инициализации:

number	528
name	Disk drive
on_hand	10

Инициализаторы структуры следуют правилам, аналогичным правилам инициализаторов массива. Выражения, используемые в инициализаторе структуры, должны быть константами: например, мы не могли использовать переменную для инициализации члена `on_hand` `part1`, (в C99 это ограничение ослаблено, как мы увидим в Разделе 18.5.)

У инициализатора может быть меньше членов, чем у структуры, которую он инициализирует; как и у массивов, любые "оставшиеся" члены получают 0 в качестве начального значения. В частности, байты в оставшемся символьном массиве будут равны нулю, что представляет собой пустую строку.

Назначенные инициализаторы

Инициализаторы для C99, которые были рассмотрены в Разделе 8.1 в контексте массивов, также могут быть использованы со структурами. Рассмотрим инициализатор для `part1`, показанный в предыдущем примере:

```
{523, "Disk drive", 10}
```

Назначенный инициализатор выглядит подобным образом, только каждое значение будет помечено именем инициализируемого элемента:

```
{.number = 523,.name = "Disk thrive",.in_hand = 10}
```

Сочетание точки и имени члена называется десигнатором.

(Десигнаторы элементов массива имеют другую форму.)

Назначенные инициализаторы имеют ряд преимуществ. Во-первых, их легче читать и проверять на правильность, потому что читатель может чётко видеть соответствие между членами структуры и значениями, перечисленными в инициализаторе. Во-вторых, значения в инициализаторе не должны располагаться в том же порядке, в котором перечислены члены в структуре. Наш пример инициализатора может быть записан следующим образом:

```
{.on_hand = 10,.name = "Disk drive",.number= 525}
```

Так как порядок не имеет значения, программист не должен запоминать порядок, в котором члены были первоначально объявлены. Кроме того, порядок членов может быть изменён в будущем без изменения назначенных инициализаторов.

Не все значения, перечисленные в назначенном инициализаторе, должны иметь префикс десигнатора. (Это верно и для массивов, как мы видели в Разделе 8.1.)

Рассмотрим следующий пример:

```
{.number = 525, "Disk drive",.on_hand = 10}
```

Значение "Disk drive" не имеет десигнатора, поэтому компилятор предполагает, что он инициализирует член, следующий за number в структуре. Все члены, для которых инициализатор не может выполнить присваивание, обнуляются.

Операции над структурами

Так как наиболее распространённая операция массива - это индексация - выбор элемента по позиции - неудивительно, что наиболее распространённая операция в структуре - выбор одного из её членов. К элементам структуры обращаются по имени, но не по позиции. Для доступа к члену внутри структуры сначала, мы записываем имя структуры, затем точку, затем имя члена. Например, следующие инструкции будут отображать значения элементов part1:


```
printf ("Номер детали: d% \n, part1.number);  
printf ("Название части: s% \n, part1.name);  
printf ("Количество на руках: d% \n", part1.on_hand);
```

Элементы структуры являются lvalues, поэтому они могут отображаться в левой части присваивания или в качестве операнда в выражении инкремента или декремента:

```
part1.number = 258, - /* меняет номер детали part1 */  
part1.on_hand ++; /*увеличивает количество в наличии part1*/
```

Точка, которую мы используем для доступа к члену структуры, на самом деле является оператором Си. Она имеет тот же приоритет, что и постфиксные операторы ++ и --, поэтому она имеет приоритет почти перед всеми другими операторами. Рассмотрим следующий пример:

```
scanf ( "%d", &part1.on_hand);
```

Выражение &part1.on_hand содержит два оператора (& и.). Оператор. имеет приоритет над оператором &, поэтому & вычисляет адрес part1.on_hand, как мы и хотели.

Другой важной операцией структуры является присваивание.

```
part2 = part1;
```

Результатом этой инструкции является копирование `part1.number` в `part2.number`, `part1.name` в `part2.name`, и так далее. Поскольку массивы не могут быть скопированы с помощью оператора `=`, удивительно обнаружить, что структуры это могут. Это ещё более удивительно, если учесть, что массив, встроенный в структуру, копируется при копировании объемлющей структуры. Некоторые программисты используют это свойство, создавая "фиктивные" структуры для включения массивов, которые будут скопированы позже:

```
struct { int a[10]; } a1, a2;
```

```
a1 = a2;
```

```
/* Это допустимо, так как a1 и a2 являются структурами */
```

Оператор `=` может использоваться только со структурами совместимых типов. Две структуры, объявленные одновременно (как `part1` и `part2`) совместимы. Как мы увидим в следующем разделе, структуры, объявленные с использованием того же "тега структуры" или того же имени типа, также совместимы. Кроме присваивания, Си не предоставляет никаких операций над целыми структурами, в частности, мы не можем использовать операторы `==` и `!=` для проверки, равны ли две структуры.

Тип структуры

Хотя в предыдущем разделе показано, как объявить структурные переменные, не удалось обсудить важный вопрос: именование типов структур. Предположим, что программе нужно объявить несколько структурных переменных с одинаковыми членами. Если все переменные могут быть объявлены одновременно, нет никаких проблем.

Но если нам нужно объявить эти переменные в разных точках программы, то жизнь становится сложнее. Если

мы пишем:

```
struct {  
    int number;  
    char name [NAME_LEN+1]  
    int on_hand;  
}part1;
```

в одном месте и

```
struct {
```

```
int number;  
char name [NAME_LEN+1]  
int on_hand;  
}part2;
```

в другом, то мы быстро столкнёмся с проблемами. Повторение информации о структуре раздувает программу. Изменение программы позже будет рискованным, поскольку мы не можем легко гарантировать, что декларации будут оставаться согласованными. Но это не самые большие проблемы. Согласно правилам Си, `part1` и `part2` не имеют совместимых типов. В результате `part1` не может быть присвоена `part2`, и наоборот. Кроме того, поскольку у нас нет имени для типа `part1` или `part2`, мы не можем использовать их в качестве аргументов в вызовах функций. Чтобы избежать этих трудностей, мы должны иметь возможность определить имя, представляющее тип структуры, а не конкретную структурную переменную. Как оказалось, Си предоставляет два способа именования структур:

мы можем либо объявить "тег структуры",
либо использовать `typedef` для определения имени типа.

Объявление тега структуры

Тег структуры - это имя, используемое для идентификации определённого типа структуры. В следующем примере объявляется тег структуры с именем `part`:

```
struct part{  
    int number;  
    char name [NAME_LEN+1]  
    int on_hand;  
};
```

Обратите внимание на точку с запятой, которая следует за правой скобкой - она должна присутствовать, чтобы прервать объявление.

Случайное исключение точки с запятой в конце объявления структуры может вызвать удивительные ошибки. Рассмотрим следующий пример:

```
struct part{  
    int number;  
    char name [NAME_LEN+1]  
    int on_hand;
```

```
}          /*** WRONG: отсутствует точка с запятой** */  
  
f(void) {  
    ...  
    return 0; /* ошибка определяется в этой строке */  
}
```

Программист не определил тип возвращаемого значения функции **f** (кусочек неаккуратного программирования). Поскольку объявление предыдущей структуры не было завершено должным образом, компилятор предполагает, что **f** возвращает значение типа `struct parts`. Ошибка не будет обнаружена до тех пор, пока компилятор не достигнет первой инструкции `return` в функции. Результат: критическое сообщение об ошибке.

Как только мы создали тег `part`, мы можем использовать его для объявления переменных:

```
struct part part1, part2;
```

К сожалению, мы не можем сокращать это объявление, отбросив слово `struct`:

```
part part1, part2; /***WRONG***/
```

part не является именем типа; без слова struct оно не имеет смысла.

Поскольку теги структуры распознаются только после слова struct, они не конфликтуют с другими именами, используемыми в программе. Было бы совершенно законно (хотя и способствуя путанице) иметь переменную с именем part. Кстати, объявление тега структуры может быть объединено с объявлением переменных структуры:

```
struct part{  
    int number;  
    char name [NAME_LEN+1]  
    int on_hand;  
}; part1, part2;
```

Здесь мы объявили структурный тег part (что позволяет использовать part позже для объявления большего количества переменных), а также переменные part1 и part2.

Все конструкции объявленные типом struct part совместимы друг с другом:

```
struct part part1 = {528, "Disk drive", 10};
```

```
struct part part2;  
part2 = part1; /* legal: both parts have the same type */
```

Определение типа структуры

В качестве альтернативы объявлению тега структуры мы можем использовать `typedef` для определения подлинного имени типа. Например, мы могли бы определить тип с именем `Part` следующим образом:

```
typedef struct {  
    int number;  
    char name [NAME_LEN+1]  
    int on_hand;  
} Part;
```

Обратите внимание, что имя типа `Part` должно быть в конце, а не после слова `struct`. Мы можем использовать `Part` так же, как и встроенные типы. Например, мы можем использовать его для объявления переменных:

Part part1, part2;

Поскольку Part является именем typedef, нам не разрешается писать struct Part. Все переменные Part, независимо от того, где они объявлены, совместимы. Когда приходит время именовать структуру, мы обычно можем либо объявить тег структуры, либо использовать typedef. Однако, как мы увидим ниже, объявление структурного тега является обязательным, если структура должна использоваться в связанном списке. В большинстве моих примеров я буду использовать теги структуры, а не имена typedef.

Структуры как аргументы и возвращаемые значения

Функции могут иметь структуры как аргументы и возвращаемые значения. Давайте рассмотрим два примера. Наша первая функция, когда в качестве аргумента задана структура part, печатает элементы структуры:

```
void print_part(struct part p)
```

854

```
printf("Part number: %d\n", p.number);  
printf("Part name: %s\n", p.name);  
printf("Quantity on hand: %d\n", p.on_hand);
```

Вот как можно было бы вызвать `print_part`:

```
print_part (part1);
```

Наша вторая функция возвращает структуру `part`, которую она строит из своих аргументов:

```
struct part build_part(int number,const char name,int  
on_hand){  
    struct part p;  
    p.number = number;  
    strcpy(p.name, name);  
    p.on_hand = on_hand;  
    return p;  
}
```

Обратите внимание, что для параметров `build_part` допустимо иметь имена, которые соответствуют членам структуры `part`,

поскольку структура имеет собственное пространство имён. Вот как можно было бы вызвать `build_part`:

```
part1 = build_part(528, "Disk drive", 10);
```

Передача структуры в функцию и возврат структуры из функции требуют создания копии всех элементов структуры. В результате эти операции накладывают на программу изрядную нагрузку, особенно если структура большая. Чтобы избежать этих накладных расходов, иногда рекомендуется передать указатель на структуру вместо передачи самой структуры. Аналогично, функция может возвращать указатель на структуру вместо возврата фактической структуры. В Разделе 17.5 приведены примеры функций, которые имеют указатель на структуру в качестве аргумента и / или возвращают указатель на структуру. Есть и другие причины избегать копирования структур помимо эффективности. Например, заголовок `<stdio.h>` определяет тип с именем `FILE`, который обычно является структурой. Каждая структура `FILE` хранит информацию о состоянии открытого файла и поэтому должна быть уникальной в программе. Каждая функция в `<stdio.h>`, которая открывает файл, возвращает указатель на структуру `FILE` и каждая функция, выполняющая операцию в открытом файле, требует указателя на `FILE` в качестве аргумента. Иногда нам может понадобиться

инициализировать структурную переменную внутри функции для соответствия другой структуре, возможно, поставляемой в качестве параметра функции. В следующем примере инициализатором для `part2` является параметр, переданный функции *f*:

```
void f(struct part part1){  
    struct part part2 = part1;  
    ...  
}
```

Си разрешает инициализаторы такого рода, при условии, что структура, которую мы инициализируем (`part2`, в данном случае) имеет автоматическую продолжительность хранения (она локальна для функции и не была объявлена статической). Инициализатором может быть любое выражение правильного типа, включая вызов функции, который возвращает структуру.

Составные литералы

В Разделе 9.3 введена функция `C99`, известная как составной литерал. В этом разделе составные литералы использовались

для создания безымянных массивов, обычно с целью передачи массива функции. Составной литерал также можно использовать для создания структуры "на лету" без предварительного сохранения её в переменной. Полученная структура может быть передана в качестве параметра, возвращена функцией или присвоена переменной. Давайте рассмотрим пару примеров. Во-первых, мы можем использовать составной литерал для создания структуры, которая будет передана функции. Например, можно вызвать функцию `print_part` следующим образом:

```
print_part((struct part) {523, "Disk drive", 10});
```

Составной литерал (выделен жирным шрифтом) создаёт структуру с членами 523, **"Disk drive"**, и 10 в таком порядке. Затем эта структура передаётся в `print_part`, которая её отображает.

Вот как составной литерал может быть присвоен переменной:

```
part1 = (struct part) {523, "Disk drive", 10};
```

Эта инструкция напоминает объявление, содержащее инициализатор, но это не то же самое - инициализаторы могут отображаться только в объявлениях, а не в инструкциях, подобных этой. В общем случае составной литерал состоит из

имени типа в скобках, за которым следует набор значений, заключённых в фигурные скобки. В случае составного литерала, представляющего структуру, имя типа может быть тегом структуры, которому предшествует слово `struct` (как в наших примерах), или именем `typedef`. Составной литерал может содержать десигнаторы, как и назначенный инициализатор:ъ

```
print_part((struct part) {.on_hand = 10,.name = "Disk  
drive",.number = 528}));
```

Составной литерал может не обеспечить полной инициализации, в этом случае любые неинициализированные члены по умолчанию равны нулю.

16.3 Вложенные массивы и структуры

Структуры и массивы можно комбинировать без ограничений. Массивы могут иметь структуры в качестве элементов, а структуры могут содержать массивы и структуры в качестве членов. Мы уже видели пример массива, вложенного внутрь структуры (член `name` структуры `part`).

Рассмотрим другие ВОЗМОЖНОСТИ: структуры, члены которых являются структурами и массивы, элементы которых являются структурами.

Вложенная структура

Вложенность одного вида структуры внутри другого часто полезна. Например, предположим, что мы объявили следующую структуру, в которой могут храниться имя человека, инициал отчества и фамилия:

```
struct person_name {  
    char first [FIRST_NAME_LEN+1];  
    char middle_initial;  
    char last [LAST_NAME_LEN+1];  
};
```

Мы можем использовать структуру `person_name` как часть более крупной структуры:

```
struct student{  
    struct person_name name;
```

```
    int id,age;  
    char sex;  
} student1, student2;
```

Доступ к имени, отчеству или фамилии студента требует двух применений оператора точка:

```
strcpy (student1.name.first, " Fred");
```

Одним из преимуществ создания структуры `person_name` (вместо того, чтобы иметь `first`, `middle_initial` и `last` членами структуры `student`) является то, что мы можем более легко рассматривать имена как единицы данных. Например, если бы мы написали функцию, которая отображает имя, мы могли бы передать ей только один аргумент - структуру `person_name` - вместо трёх аргументов:

```
display_name( student1.name);
```

Аналогично, копирование информации из структуры `person_name` в элемент `name` структуры `student` займёт одно присваивание вместо трёх:

```
struct person_name new_name;
```

```
...
```



```
scudent.name = new_name;
```

Массивы структур

Одной из наиболее распространённых комбинаций массивов и структур является массив, элементами которого являются структуры. Массив такого типа может служить простой базой данных. Например, следующий массив структур `part` способен хранить информацию о сотне деталей:

```
struct part inventory[100];
```

Чтобы получить доступ к одной из структур `part` в массиве, мы будем использовать индекс. Например, чтобы напечатать `part`, хранящуюся в позиции *i*, мы могли бы написать

```
print_part (inventory[i]);
```

Доступ к члену в структуре `part` требует сочетания индекса и выбора члена. Чтобы присвоить 883 члену `number` `inventory` [1], мы могли бы написать

```
inventory[i].number = 883;
```

Доступ к одному символу в `part name` требует индекса (для выбора конкретной `part`), за которым следует выбор (для выбора члена `name`), за которым следует индекс (для выбора символа в `part name`). Чтобы изменить имя, сохранённое `inventory[i]`, на пустую строку, мы могли бы написать

```
inventory[i].name[0] = '\0';
```

Инициализация массива структур

Инициализация массива структур выполняется почти так же, как инициализация многомерных массивов. Каждая структура имеет свой собственный инициализатор, заключенный в скобки: инициализатор для массива просто обёртывает другой набор фигурных скобок вокруг инициализаторов структуры.

Одной из причин инициализации массива структур является то, что мы планируем рассматривать его как базу данных информации, которая не будет меняться во время выполнения программы. Например, предположим, что мы работаем над программой, которая будет нуждаться в доступе к кодам стран, используемых при международных телефонных звонках. Сначала мы создадим структуру, которая может хранить название страны вместе с её кодом:

```
struct dialing_code {  
    char *country;  
    int code;  
}
```

Обратите внимание, что `country` - это указатель, а не массив символов. Это может быть проблемой, если мы планируем использовать структуры `dialing_code` как переменные, но мы здесь не планируем этого. Когда мы инициализируем структуру `dialing_code`, `country` в конечном итоге укажет на строковый литерал. Далее мы объявим массив этих структур и инициализируем его, чтобы он содержал коды для некоторых самых густонаселённых стран мира:

```
const struct dialing_code country codes[] =  
{ {"Argentina", 54}, {"Bangladesh", 880},  
  {"Brazil", 55}, {"Bunna (Myanmar)", 95},  
  {"China", 56}, {"Colombia", 57},  
  {"Congo, Dem. Rep. of", 243}, {"Egypt", 20},  
  {"Ethiopia", 251}, {"France", 33},  
  {"Germany", 49}, {"India", 91},  
  {"Indonesia", 62}, {"Iran", 95},  
  {"Italy", 39}, {"Japan", 51},  
  {"Mexico", 52}, {"Nigeria", 234},
```

```
{ "Pakistan", 92 }, { "Philippines", 63 },
{ "Poland", 45 }, { "Russia", 7 },
{ "South Africa", 27 }, { "South Korea", 52 },
{ "Spain", 34 }, { "Sudan", 249 },
{ "Thailand", 66 }, { "Turkey", 90 },
{ "Ukraine", 330 }, { "United Kingdom", 44 },
{ "United States", 1 }, { "Vietnam", 54 } };
```

Внутренние фигурные скобки вокруг каждого значения структуры необязательны. Однако в плане стиля я предпочитаю не опускать их. Поскольку массивы структур (и структуры, содержащие массивы) настолько распространены, назначенные инициализаторы C99 позволяют элементу иметь более одного десигнатора.

Предположим, что мы хотим инициализировать массив `inventory`, чтобы он содержал одну деталь (`part`). Номер детали (`number`) равен 528, а количество в наличии (`on_hand`) равно 10, но теперь `name` остаётся пустым:

```
struct part inventory[100]
{ [0]. number = 525, [0]. on_hand = 10, [0]. name[0] = '\0' };
```

В первых двух элементах списка используются два десигнатора (один для выбора элемента массива 0 - структуры

part - и один для выбора члена в структуре). В последнем элементе используются три указателя: один для выбора элемента массива, один для выбора члена name внутри этого элемента и один для выбора элемента 0 из name.

ПРОГРАММА Обслуживание базы данных деталей

Чтобы проиллюстрировать, как на практике используются вложенные массивы и структуры, мы разработаем довольно длинную программу, которая поддерживает базу данных о деталях, хранящихся на складе. Программа построена вокруг массива структур, каждая из которых содержит информацию об одной детали -номер детали, название и количество. Наша программа будет поддерживать следующие операции:

- Добавление нового номера детали, названия детали и исходного количества в наличии. Программа должна напечатать сообщение об ошибке, если деталь уже находится в базе данных или если база данных заполнена.
- По данному номеру детали напечатать имя детали и текущее количество. Программа должна распечатать

866

сообщение об ошибке, если номер детали не указан в базе данных.

■ По данному номеру детали, изменить количество в наличии. Если номер детали отсутствует в базе данных, программа должна напечатать сообщение об ошибке.

■ Печать таблицы со всей информацией в базе данных. Детали должны отображаться в том порядке, в котором они были введены.

■ Завершить выполнение программы.

Мы будем использовать коды *i* (insert), *s* (search), *u* (update), *p* (print) и *q* (quit) для представления этих операций.

Сеанс работы с программой может выглядеть следующим образом:

Enter operation code i

Enter part number: 528

Enter part name: Disk drive

Enter quantity on hand: 10

867

Enter operation code:s

Enter part number:528

Part name: Disk drive

Quantity on hand: 10

Enter operation code: s

Enter part number: 914

Part not found.

Enter operation code: i

Enter part number: 914

Enter part name: Printer cable

Enter quantity on hand: 5

Enter operation code: u

Enter part number: 528

Enter change in quantity on hand: -2

Enter operation code: s

Enter part number: 528

868

Part name: Disk drive

Quantity on hand: 8

Enter operation code: p

Part Number	Part Name	Quantity on Hand
-------------	-----------	------------------

528	Disk drive	8
-----	------------	---

914	Printer cable	5
-----	---------------	---

Enter operation code: q

Программа будет хранить информацию о каждой детали в структуре. Мы ограничим размер базы данных 100-ней деталей (если это ограничение окажется слишком маленьким, мы всегда можем изменить его позже.); база делает возможным хранение структур в массиве, который я буду называть `inventory`. Для отслеживания количества деталей, хранящихся в массиве, мы будем использовать переменную `num_parts`. Поскольку эта программа управляется по меню, довольно легко изобразить главный цикл:

```
for (;;) {
```



```
Prompt user to operation code;
read code;
switch (code){
    case 'i': perform insert operation; break;
    case 's':perform search operation; break;
    case 'u':perform update operation; break;
    case 'p':perform print operation; break;
    case 'q': terminate program;
        default : print error message;
    }
}
```

Удобно иметь отдельные функции, выполняющие операции вставки, поиска, обновления и печати. Поскольку эти функции будут нуждаться в доступе к `inventory` и `num_parts`, мы можем захотеть сделать эти переменные внешними. В качестве альтернативы мы могли бы объявить переменные внутри `main`, а затем передать их функциям в качестве аргументов. С точки зрения дизайна обычно лучше делать переменные локальными для функции, а не делать их внешними (см. Раздел 10.2, если вы забыли почему). В этой программе, однако, включение `inventory` и `num_parts` внутрь `main` просто усложнит ситуацию. По причинам, которые объясню позже, я решил разделить программу на три файла:

870

inventory.c, в котором содержится основная часть программы; read_line.h, который содержит прототип функции read_line; и read_line.c, в котором содержится определение read_line. Мы обсудим последние два файла далее в этом разделе. Пока давайте сосредоточимся на inventory.c.

```
// inventory.c
```

```
/* Поддержка базы данных деталей (версия с массивом) */
```

```
#include <stdio.h>
```

```
#include "readline.h"
```

```
#define NAME_LEN 25
```

```
#define MAX_PARTS 100
```

```
struct part {
```

```
    int number;
```

```
    char name[NAME_LEN+1];
```

```
    int on_hand;
```

```
} inventory[MAX_PARTS];
```

871

```
int num_parts = 0; /* количество деталей, хранящихся в  
                    настоящее время */
```

```
int find_part(int number);
```

```
void insert(void);
```

```
void search(void);
```

```
void update(void);
```

```
void print(void);
```

```
/******
```

main: Предлагает пользователю ввести код операции, затем вызывает функцию для выполнения запрошенного действия. Повторяется, пока пользователь не введет команду «q». Печатает сообщение об ошибке, если пользователь вводит недопустимый код.

```
*****/
```

```
int main(void){
```

```
    char code;
```

```
    for (;;) {
```

```
        printf("Enter operation code: ");
```

```
        scanf(" %c", &code);
```

```
while (getchar() != '\n') /* skips to end of line */
    ;
switch (code) {
case 'i': insert();
    break;
case 's': search();
    break;
case 'u': update();
    break;
case 'p': print();
    break;
case 'q': return 0;
default: printf("Illegal code\n");
}
printf("\n");
}
}

/*****
```

873

find_part: ищет номер детали в массиве inventory. Возвращает индекс массива, если номер детали найден; в противном случае возвращает -1. *

```

*****/

int find_part(int number){
    int i;

    for (i = 0; i < num_parts; i++)
        if (inventory[i].number == number)
            return i;

    return -1;
}
```

```

/*****
```

insert: запрашивает у пользователя информацию о новой детали, а затем вставляет деталь в базу данных. Печатает сообщение об ошибке и преждевременно возвращается, если деталь уже существует или база данных заполнена.

```

*****/

void insert(void){
    int part_number;
```

```
if (num_parts == MAX_PARTS) {  
    printf("Database is full; can't add more parts.\n");  
    return;  
}
```

```
printf("Enter part number: ");  
scanf("%d", &part_number);  
if (find_part(part_number) >= 0) {  
    printf("Part already exists.\n");  
    return;  
}
```

```
    inventory[num_parts].number = part_number;  
    printf("Enter part name: ");  
    read_line(inventory[num_parts].name, NAME_LEN);  
    printf("Enter quantity on hand: ");  
    scanf("%d", &inventory[num_parts].on_hand);  
    num_parts++;  
}
```

```

/*****

```

search: Предлагает пользователю ввести номер детали, затем ищет деталь в базе данных. Если деталь существует, печатает название и количество в наличии; если нет, выводит сообщение об ошибке. *

```

*****/

```

```

void search(void){
    int i, number;

    printf("Enter part number: ");
    scanf("%d", &number);
    i = find_part(number);
    if (i >= 0) {
        printf("Part name: %s\n", inventory[i].name);
        printf("Quantity on hand: %d\n", inventory[i].on_hand);
    } else
        printf("Part not found.\n");
}

```

```

/*****

```

876

update: предлагает пользователю ввести номер детали.

Печатает сообщение об ошибке, если деталь не существует; в противном случае пользователю предлагается ввести изменение количества и обновить базу данных.

```

*****/

void update(void){
    int i, number, change;

    printf("Enter part number: ");
    scanf("%d", &number);
    i = find_part(number);
    if (i >= 0) {
        printf("Enter change in quantity on hand: ");
        scanf("%d", &change);
        inventory[i].on_hand += change;
    } else
        printf("Part not found.\n");
}

/*****
```


print: распечатывает список всех деталей в базе данных с указанием номера детали, названия детали и количества в наличии. Детали печатаются в том порядке, в котором они были введены в базу данных. *

```

*****/

void print(void){
    int i;

    printf("Part Number  Part Name  "
    "Quantity on Hand\n");
    for (i = 0; i < num_parts; i++)
        printf("%7d   %-25s%11d\n", inventory[i].number,
        inventory[i].name, inventory[i].on_hand);
}

```

В функции main строка формата "% c" позволяет scanf пропускать пробел перед чтением кода операции. Пробел в строке формата является критическим; без него scanf иногда считывает символ новой строки, который завершал предыдущую строку ввода. Программа содержит одну функцию find_part, которая не вызывается из main. Эта "вспомогательная" функция помогает нам избежать избыточного кода и упростить более важные функции. Вызывая find_part, функции insert, search и update могут найти

деталь в базе данных (или просто определить, существует ли эта деталь). Осталось только одна тонкость: функция `read_line`, которую программа использует для чтения имени детали. В Разделе 13.3 обсуждались вопросы, связанные с написанием такой функции. К сожалению, версия `read_line` в том разделе не будет работать должным образом в текущей программе. Посмотрите, что произойдёт, когда пользователь вставляет деталь: `

Enter part number: 525

Enter part name: Disk drive

Пользователь нажимает клавишу Enter после ввода номера детали и снова после ввода имени детали, каждый раз оставляя невидимый символ новой строки, который программа должна читать. Для обсуждения давайте притворимся, что эти символы видны:

Enter part number: 528 ☼

Enter part name: Disk driven ☼

Когда мы вызываем `scanf` для чтения номера детали, она потребляет 5, 2 и 8, но оставляет символ ☼ непрочитанным. Если мы попытаемся прочитать название детали, используя нашу оригинальную функцию `read_line`, она немедленно

встретит символ $\backslash$ и прекратит чтение. Эта проблема распространена, когда числовой ввод сопровождается символьным вводом. Наше решение будет состоять в том, чтобы написать версию `read_line`, которая пропускает символы пробела перед тем, как начнёт сохранять символы. Мало того, что это решит проблему новой строки, но также позволяет избежать хранения пробелов, предшествующих имени детали. Поскольку `read_line` не имеет отношения к другим функциям в `inventory.c` и, поскольку она потенциально может использоваться повторно в других программах, я решил отделить её от `inventory.c`. Прототип `read_line` войдет в файл заголовка `read_line.h`:

```
//readline.h
```

```
#ifndef READLINE_H
```

```
#define READLINE_H
```

```
/******
```

`read_line`: пропускает начальные символы пробела, затем считывает оставшуюся часть строки ввода и сохраняет её в `str`. Обрезает строку, если её длина превышает *n*. Возвращает количество сохранённых символов.

```
*****/
```

```
int read_line(char str[], int n);
```

880

#endif

Мы поместим определение read_line в файл readline.c:

//readline.c

#include <ctype.h>

#include <stdio.h>

#include "readline.h"

int read_line(char str[], int n){

int ch, i = 0;

while (isspace(ch = getchar()))

;

while (ch != '\n' && ch != EOF) {

if (i < n)

str[i++] = ch;

ch = getchar();

}

881

```
    str[i] = '\0';  
    return i;  
}
```

Выражение

`isspace(ch = getchar())`

управляет первой инструкцией `while`. Это выражение вызывает `getchar` для чтения символа, сохраняет символ в `ch`, а затем использует функцию `isspace`, чтобы проверить, является ли `ch` пробелом. Если нет, то цикл завершается с `ch`, содержащим не пробельный символ. Раздел 15.3 объясняет, почему `ch` имеет тип `int` вместо `char` и почему полезно протестировать на EOF.

16.4 Объединения

Объединение, как и структура, состоит из одного или нескольких членов, возможно, разных типов. Однако компилятор выделяет достаточно места только для самого большого из членов, которые накладываются друг на друга в пределах этого пространства. В результате присвоение нового значения одному члену также изменяет значения других членов. Чтобы проиллюстрировать основные свойства

882

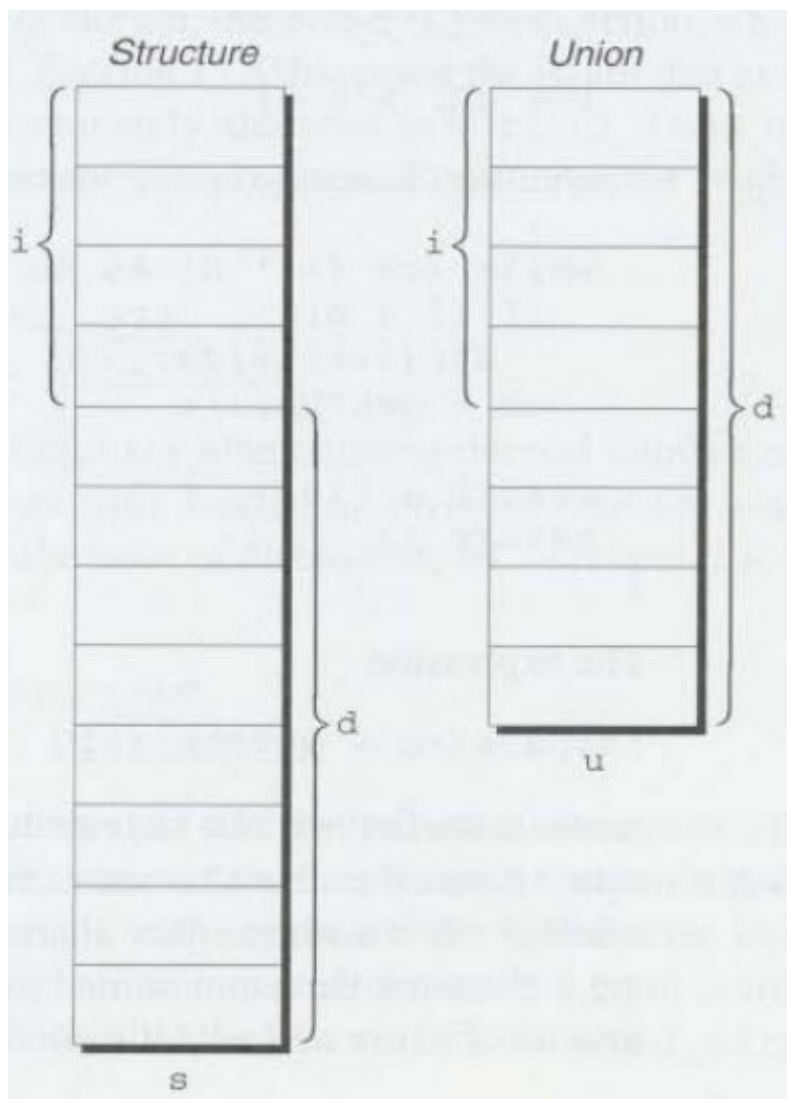
объединений, объявим переменную объединения ***u*** с двумя членами:

```
union {  
    int i;  
    double d;  
} u;
```

Обратите внимание как декларация объединения очень похожа на объявление структуры:

```
struct {  
    int i;  
    double d;  
} s;
```

На самом деле структура ***s*** и объединение ***u*** отличаются только одним фактором: члены ***s*** хранятся в разных адресах в памяти, а члены ***u*** хранятся по одному и тому же адресу. Вот как выглядят ***s*** и ***u*** в памяти (если предположить, что значения `int` требуют четырёх байтов, а значения `double` занимают восемь байтов):



В структуре ***i*** и ***d*** занимают разные ячейки памяти: общий размер ***s*** равен 12 байтам. В объединении ***u*** ***i*** и ***d*** перекрываются (*поочерёдно, не одновременно*); ***i***, это действительно первые четыре байта ***d***, поэтому ***u*** занимает всего восемь байтов. Кроме того, ***i*** и ***d*** имеют одинаковый адрес. Доступ к членам объединения осуществляется так же, как и к членам структуры. Чтобы сохранить число 82 в *i*-члене ***u***, мы будем писать

$u.i = 82;$

Для сохранения значения 74.8 в d -члене.
мы будем писать

$u.d = 74.8;$

Поскольку компилятор накладывает память для членов объединения, изменение одного члена изменяет любое значение, ранее сохранённое в любом из других членов. Таким образом, если мы сохраним значение в $u.d$, любое значение, ранее сохранённое в $u.i$, будет потеряно. (Если мы рассмотрим значение $u.i$, оно окажется бессмысленным.) Аналогично, изменение $u.i$ искажает $u.d$. Из-за этого свойства мы можем рассматривать u как место для хранения или i или d , но не для обоих. (Структура s позволяет нам хранить i и d). Свойства объединений почти идентичны свойствам структур. Мы можем объявлять теги объединения и типы объединения так же, как мы объявляем теги и типы структуры. Подобно структурам, объединения могут быть скопированы с помощью оператора $=$, переданы в функции и возвращены функциями. Допускаются даже инициализации аналогично структурам. Однако начальное значение можно дать только первому члену объединения. Например, мы можем инициализировать элемент i объединения u нулём следующим образом:


```
union{  
    int i;  
    double d;  
} u = {0};
```

Обратите внимание на наличие фигурных скобок, которые требуются. Выражение внутри фигурных скобок должно быть константным. (В C99 правила немного отличаются, как мы увидим в Разделе 18.5.) Назначенные инициализаторы, в C99, которые мы ранее обсуждали в контексте массивов и структур, также могут быть использованы с объединениями. Назначенный инициализатор позволяет нам указать, какой член объединения должен быть изменён.

Например, мы можем инициализировать d-элемент *u* следующим образом:

```
union {  
    int i;  
    double d;  
} u = {.d = 10.0};
```

Только один член может быть инициализирован, но он не обязательно должен быть первым. Существует несколько приложений для объединений. Мы обсудим два из них. ещё одно приложение - просмотр хранилища разными способами - сильно зависит от машины, поэтому я отложу его до Раздела 20.3.

Использование объединений для экономии места

Мы часто будем использовать объединения как способ сэкономить место в структурах. Предположим, что мы разрабатываем структуру, которая будет содержать сведения о номенклатуре, продаваемой через каталог подарков. В каталоге представлены только три вида товаров: книги, кружки и рубашки. Каждый товар имеет номер склада и цену, а также другую информацию, которая зависит от типа товара:

Books: Title, author, number of pages

Mugs: Design

Shirts: Design, colors available, sizes available

(Книги: название, автор, количество страниц

Кружки: дизайн

Рубашки: дизайн, доступные цвета, доступные размеры)

Наша первая попытка проектирования может привести к следующей структуре:

```
struct catalog_item {  
    int stock_number;
```

```
double price;  
int item_type;  
char title [TITLE_LEN+1];  
char author [AUTOR_LEN+1];  
int num_pages;  
char design [DESIGN_LEN+1];  
int colors;  
int sizes;  
};
```

Элемент `item_type` будет иметь одно из значений `BOOK`, `MUG` или `SHIRT`. Члены `colors` and `size` будут хранить закодированные сочетания цветов и размеров.

Хотя эта структура идеально подходит для использования, она тратит впустую пространство, так как только часть информации в структуре является общей для всех элементов в каталоге. Например, если элемент является книгой, нет необходимости хранить дизайн, цвета и размеры. Поместив объединение в структуру `catalog_item`, мы можем уменьшить пространство, необходимое для структуры. Членами объединения будут структуры, каждая из которых содержит данные, необходимые для определённого типа `catalog_item`:

888

```
struct catalog_item {  
    int stock_number;  
    double price;  
    int item_type;  
    union {  
        struct {  
            char title[TITLE_LEN+1];  
            char author [AUTHOR_LEN+1];  
            int num_pages;  
        } book;  
  
        struct {  
            char design [DESIGN_LEN+1];  
        } mug;  
        struct {  
            char design [DESIGN_LEN+1];  
            int colors;  
            int sizes;  
        } shirt;  
    }item;  
};
```

```
};
```

Обратите внимание, что объединение (с именем `item`) является членом структуры `catalog_item`, а структуры `book`, `mug` и `shirt` являются членами `item`, если `c` является структурой `catalog_item`, представляющей книгу, мы можем напечатать название книги следующим образом:

```
printf("%s", c.item.book.title);
```

Как показано в этом примере, доступ к объединению, вложенному внутрь структуры, может быть неудобным: чтобы найти название книги, нам нужно было указать имя структуры (`c`), имя члена структуры объединение (`item`), имя члена объединения структуру (`book`), а затем имя члена этой структуры (`title`).

Мы можем использовать структуру `catalog_item`, чтобы проиллюстрировать интересный аспект объединений. Обычно не рекомендуется хранить значение в одном члене объединения, а затем обращаться к данным через другой член, так как присваивание одному члену объединения приводит к тому, что значения других членов не определены. Однако в стандарте Си упоминается особый случай: два или более членов объединения являются структурами, а структуры начинаются с одного или нескольких совпадающих членов.

(Эти члены должны следовать в том же порядке и иметь совместимые типы, но не обязательно иметь одинаковые имена).

Если одна из структур в настоящее время действующая, то соответствующие члены в других структурах также будут действующими. Рассмотрим объединение, встроенное в структуру `catalog_item`. Оно содержит три структуры в качестве членов, два из которых (`mug` и `shirt`) начинаются с соответствующего члена (`design`). Теперь предположим, что мы присваиваем значение одному из членов `design`:

```
strcpy(c.item.mug.design, "Cats");
```

Член `design` в другой структуре будет определён и иметь то же самое значение:

```
printf("%s", c.item.shirt.design); /* prints "Cats" */
```

Использование объединений для построения смешанных структур данных

Объединения имеют ещё одно важное применение: создание структур данных, содержащих смесь данных различных типов. Допустим, нам нужен массив, элементы которого

представляют собой смесь значений `int` и `double`. Поскольку элементы массива должны быть одного типа, кажется, невозможно создать такой массив. Однако с помощью объединений это сделать достаточно легко. Во-первых, мы определяем тип объединения, члены которого представляют различные типы данных, которые должны храниться в массиве:

```
typedef union {  
    int i;  
    double d;  
}Number;
```

Затем мы создаём массив, элементами которого являются значения `Number`:

```
Number number_array [1000];
```

Каждый элемент `number_array` является объединением `Number`. Объединение `Number` может хранить либо значение `int`, либо значение `double`, что позволяет хранить смесь значений `int` и `double` в массиве чисел. Например, предположим, что мы хотим, чтобы элемент 0 из `number_array` сохранял 5, а в элементе 1 хранилось 8.395. Желаемый эффект дадут следующие присваивания:

```
number_array[0].i = 5;  
number_array[1].d = 8.395;
```

Добавление "поля тег" в объединение

Объединения страдают от серьёзной проблемы: нет простого способа узнать, который из членов объединения был изменён последним и поэтому содержит значимое значение.

Рассмотрим проблему написания функции, отображающей значение, которое в настоящее время хранится в объединении `Number`. Эта функция может иметь следующую схему:

```
void print_number(Number n){  
    if (n contains an integer)  
        printf("%d", n.i);  
    else  
        printf("%g", n.d);  
}
```

К сожалению, `print_number` не может определить, содержит ли *n* целое число или число с плавающей запятой. Чтобы отслеживать эту информацию, мы можем встроить объединение в структуру, которая имеет ещё один член: "поле

тега" или "дискриминант" (определитель), цель которого - напомнить нам, что в настоящее время хранится в объединении. В структуре `catalog_item`, рассмотренной ранее в этом разделе, этой цели служил `item_type`. Преобразуем тип `Number` в структуру со встроенным объединением:

```
#define INT_KIND 0
#define DOUBLE_KIND 1

typedef struct {
    int kind; /* tag field */
    union {
        int i;
        double d;
    } u;
} Number;
```

`Number` имеет два члена, `kind` и ***u***. Значение `kind` будет либо `INT_KIND`, либо `DOUBLE_KIND`. Каждый раз, когда мы присваиваем значение члену ***u***, мы также изменяем `kind`, чтобы напомнить нам, какой член ***u*** мы изменили. Например,

894

если ***n*** - переменная Number, присвоение *i*-члену ***u*** будет иметь следующий вид:

```
n.kind = INT_KIND;
```

```
n.u.i = 82;
```

Обратите внимание, что для присвоения ***i*** необходимо сначала выбрать элемент ***u*** из ***n***, а затем элемент ***i*** из ***u***. Когда нам нужно получить число, хранящееся в числовой переменной, *kind* сообщит нам, какому члену объединения было присвоено значение последним. Функция `print _number` может воспользоваться этой возможностью:

```
void print_number(Number n){  
    if (n.kind == INT_KIND)  
        printf("%d", n.u.i);  
    else  
        printf("%g", n.u.d);  
}
```

Обязанностью программы является изменение поля тега при каждом присваивании члену объединения.

16.5 Перечисления

Во многих программах нам понадобятся переменные, которые имеют только небольшой набор значимых значений.

Например, логическая переменная должна иметь только два возможных значения: "true" и "false".

Переменная, в которой хранится масть игровой карты, должна иметь только четыре потенциальных значения:

"трефы" ("clubs"), "бубны" ("diamond"), "червы" ("hearts") и "пики" ("spades"). Очевидный способ работы с такой переменной - объявить её целым числом и иметь набор кодов, представляющих возможные значения переменной:

```
int s; /* s будет хранить масть (suit) */  
s = 2; /* 2 обозначает "червы" ("hearts") */
```

Хотя эта техника работает, она оставляет желать лучшего. Некто, читающий программу, не может сказать, что *s* имеет только четыре возможных значения, и смысл 2 не сразу очевиден. Использование макросов для определения "типа масти" и имён для различных мастей является шагом в правильном направлении:

896

```
#define SUIT int  
#define CLUBS 0  
#define DIAMONDS 1  
#define HEARTS 2  
#define SPADES 3
```

Наш предыдущий пример, теперь будет легче читать:

```
SUIT s;  
  
...  
s = HEARTS;
```

Этот метод лучше, но все равно не самое хорошее решение. Для читающего программу нет никаких признаков того, что макросы представляют значения одного и того же "типа." Если количество возможных значений больше малого числа, определение отдельного макроса для каждого из них будет утомительным. Более того, определённые нами имена - CLUBS, DIAMONDS, HEARTS, и SPADES - будут удалены препроцессором, поэтому они не будут доступны во время отладки. Си обеспечивает особый вид, разработанный специально для переменных, которые имеют небольшое количество возможных значений. Перечисляемый тип - это

тип, значения которого перечисляются ("enumerated") программистом, который должен создать имя (константу перечисления) для каждого из значений. В следующем примере перечислены значения (CLUBS, DIAMONDS, HEARTS, и SPADES), которые можно присвоить переменным s1 и s2:

```
enum (CLUBS, DIAMONDS, HEARTS, SPADES) s1, s2;
```

Хотя перечисления имеют мало общего со структурами и объединениями, они объявляются аналогичным образом. Однако, в отличие от членов структуры или объединения, имена констант перечисления должны отличаться от других идентификаторов, объявленных в охватывающей области. Константы перечисления аналогичны константам, созданным с помощью директивы `#define`, но они не эквивалентны. Во-первых, константы перечисления подчиняются правилам области C: если перечисление объявляется внутри функции, его константы не будут видны вне функции.

Теги перечисления и имена типов

Нам часто придётся давать имена перечислениям по тем же причинам, по которым мы именуем структуры и объединения.

Как и для структур и объединений, существует два способа назвать перечисление: объявлением тега или использованием typedef для создания реального имени типа. Теги перечисления напоминают теги структуры и объединения. Например, чтобы определить тег `suit`, мы могли бы написать

```
enum suit {CLUBS, DIAMONDS, HEARTS, SPADES};
```

переменные `suit` будут объявлены следующим образом:

```
enum suit s1, s2;
```

В качестве альтернативы мы могли бы использовать typedef, чтобы сделать `Suit` именем типа:

```
typedef enum (CLUBS,DIAMONDS, HEARTS, SPADES) Suit;  
Suit s1, s2;
```

В C89 использование typedef для именования перечисления - отличный способ создания логического типа:

```
typedef enum (FALSE, TRUE) B00L;
```

Конечно, C99 имеет встроенный логический тип, поэтому программисту с C99 нет необходимости определять тип BOOL таким образом.

Перечисления как целые числа

За кулисами Си обрабатывает переменные перечисления и константы как целые числа. По умолчанию компилятор присваивает целые числа 0, 1, 2 константам в конкретном перечислении. Например, в нашем перечислении `suit`, `CLUBS`, `DIAMONDS`, `HEARTS` и `SPADES` представляют 0, 1, 2 и 3 соответственно. Если хотите, мы можем выбирать различные значения для констант перечисления. Предположим, что мы хотим, чтобы `CLUBS`, `DIAMONDS`, `HEARTS` и `SPADES` обозначали 1, 2, 3 и 4.

Мы можем указать эти числа при объявлении перечисления:

```
enum suit {CLUBS = 1, DIAMONDS = 2, HEARTS  
= 3, SPADES = 4};
```

Значения констант перечисления могут быть произвольными целыми числами, перечисленными в произвольном порядке:

900

```
enum dept {RESEARCH = 20, PRODUCTION = 10, SALES = 25};
```

Даже допустимо, чтобы две или более константы перечисления имели одинаковое значение. Если для константы перечисления не указано значение, её значение на единицу больше значения предыдущей константы. (Первая константа перечисления по умолчанию имеет значение 0.)

В следующем перечислении BLACK имеет значение 0, LT_GRAY равен 7, DK_GRAY равен 8, а WHITE-15:

```
enum EGA_colors (BLACK, LT_GRAY = 7, DK_GRAY, WHITE = 15};
```

Поскольку значения перечисления представляют собой не что иное, как тонко замаскированные целые числа, С позволяет смешивать их с обычными целыми числами:

```
int i;

enum (CLUBS, DIAMONDS, HEARTS, SPADES}s;

i = DIAMONDS; /* i is now 1 */

s = 0; /* s is now 0 (CLUBS) */

s++; /* s is now 1 (DIAMONDS) */
```



```
i=s+2; /*is now 3 */
```

Компилятор рассматривает `s` как переменную некоторого целочисленного типа; `CLUBS`, `DIAMONDS`, `HEARTS` и `SPADES` - это просто имена целых чисел 0, 1, 2 и 3.

Хотя удобно использовать значение перечисления в качестве целого числа, опасно использовать целое число в качестве значения перечисления. Например, мы можем случайно сохранить число 4, которое не соответствует ни одной масти (suit) в `s`.

Использование перечислений для объявления " полей тегов"

Перечисления идеально подходят для решения проблемы, с которой мы столкнулись в Разделе 16.4: определение члена объединения, которому последним было присвоено значение. Например, в структуре чисел мы можем сделать член `kind` перечислением вместо `int`:

```
typedef struct {  
    enum(INT_KIND, DOUBLE_KIND) kind;  
    union {  
        int i;  
        double d;  
    }u;  
}Number;
```

Новая структура используется точно так же, как и старая. Преимущества в том, что мы покончили с макросами INT_KIND и DOUBLE_KIND (теперь они константы перечисления), и мы прояснили значение kind - теперь очевидно, что kind имеет только два возможных значения: INT_KIND и DOUBLE_KIND.

Вопросы и ответы

Вопрос: Когда я попытался с помощью оператора sizeof определить количество байтов в структуре, я получил число, которое было больше, чем размеры членов, сложенных вместе. Как такое может быть?

Ответ: Давайте рассмотрим пример:

```
struct {  
    char a;  
    int b;  
}s;
```

Если значения `char` занимают один байт, а значения `int` - четыре байта, то насколько велика `s`? Очевидный ответ — пять байт — может быть неправильным. Некоторые компьютеры требуют, чтобы адрес определённых элементов данных был кратен некоторому числу байтов (обычно два, четыре или восемь, в зависимости от типа элемента). Чтобы удовлетворить это требование, компилятор будет "выравнивать" члены структуры, оставляя "дыры" (неиспользуемые байты) между соседними членами. Если предположить, что элементы данных должны начинаться с размера, кратного четырём байтам, то за элементом структуры ***a*** будет следовать трёхбайтовая дыра. В результате `sizeof (S)` будет 8. Кстати, структура может иметь дыру в конце, а также дыры между элементами. Например, структура

```
struct {
```

```
    int a;  
    char b;
```

} s;

может иметь трёхбайтовую дыру после члена **b**.

Вопрос: Может ли быть "дыра" в начале структуры?

Ответ: Нет, стандарт Си указывает, что дыры разрешены только между членами или после последнего члена. Одним из следствий является то, что указатель на первый член структуры гарантированно будет таким же, как указатель на всю структуру. (Обратите внимание, однако, что два указателя не будут иметь один и тот же тип.)

Вопрос: Почему нельзя использовать оператор == для проверки равенства двух структур?

Ответ: Эта операция была исключена из Си, потому что не существует способа её реализации, который бы соответствовал философии языка. Сравнение членов структуры один за другим было бы слишком неэффективным. Сравнение всех байтов в структурах было бы лучше (на многих компьютерах есть специальные инструкции, которые могут быстро выполнить такое сравнение). Однако, если структуры содержат дыры, сравнение байтов может дать неверный ответ; даже если соответствующие члены имеют одинаковые значения, оставшиеся данные, хранящиеся в

дырах, могут быть разными. Проблема может быть решена с помощью компилятора, который гарантирует, что дыры всегда содержат одно и то же значение (скажем, нуль). Однако инициализация дыр приведёт к снижению производительности всех программ, использующих структуры, поэтому это невозможно.

Вопрос: Почему Си предоставляет два способа именования типов структуры (теги и имена typedef)?

Ответ: Первоначально в Си не было typedef, поэтому теги были единственным доступным методом для именования типов структур. Когда был добавлен typedef, было слишком поздно удалять теги. Кроме того, тег все ещё необходим, когда член структуры указывает на структуру того же типа (см. структуру узла Раздел 17.5).

Вопрос: Может ли структура иметь тег и имя typedef?

Ответ: Да. На самом деле, тег и имя typedef могут быть одинаковыми, хотя это и не обязательно:

```
typedef struct part {  
    int number;  
    char name [NAME_LEN+1]  
    int on_hand;
```

```
}part;
```

Вопрос: Как поделиться типом структуры между несколькими файлами в программе?

Ответ: Поместите объявление тега структуры (или typedef, если вы предпочитаете) в файл заголовка, а затем включите файл заголовка, где структура необходима. Например, чтобы совместно использовать структуру part мы поместим следующие строки в файл заголовка:

```
struct part {  
    int number;  
    char name [NAME_LEN+1];  
    int on_hand ;  
};
```

Обратите внимание, что мы объявляем только тег структуры, а не переменные этого типа.

Кстати, файл заголовка, содержащий объявление тега структуры или типа структуры, может нуждаться в защите от многократного включения. Объявление тега или имени typedef дважды в одном файле является ошибкой.

Аналогичные замечания касаются объединений и перечислений.

Вопрос: Если я включу объявление структуры `part` в два разных файла, будут ли переменные `part` в одном файле иметь тот же тип, что и переменные `part` в другом файле?

Ответ: Технически, нет. Однако стандарт Си говорит, что переменные `part` в одном файле имеют тип, совместимый с типом переменных `part` в другом файле. Переменные с совместимыми типами могут быть присвоены друг другу, поэтому между типами, являющимися "совместимыми" и "одинаковыми", практически нет разницы.

Правила совместимости структуры в C89 и C99 немного отличаются. В C89 структуры, определённые в разных файлах, совместимы, если их члены имеют одинаковые имена и отображаются в том же порядке, а соответствующие члены имеют совместимые типы. C99 идёт на один шаг дальше: он требует, чтобы обе структуры имели один и тот же тег или все не имеют тегов. Аналогичные правила совместимости применяются к объединениям и перечислениям (с той же разницей между C89 и C99).

Вопрос: Допустимо ли иметь указатель на составной литерал?

Ответ: Да. Рассмотрим функцию `print_part` раздела 16.2. В настоящее время параметром этой функции является структура `part`. Функция была бы более эффективной, если бы модифицировался вместо этого указатель на структуру `part`. Использование функции для печати составного литерала

будет выполняться путём предварительной настройки аргумента с помощью оператора & (адрес):

```
print_part(&(struct part) {528,"Disk drive", 10});
```

Вопрос: Разрешение указателя на составной литерал, по-видимому, позволяет изменять литерал. Это так?

Ответ: Да. Составные литералы являются lvalues, которые могут быть изменены, хотя это имеет место лишь в редких случаях.

Вопрос: Я видел программу, в которой последняя константа в перечислении сопровождалась запятой, например:

```
enum gray_values {  
    BLACK = 0,  
    DARK_GRAY = 64,  
    GRAY = 129,  
    LIGHT_GRAY = 192,  
};
```

Это допустимо?

Ответ: Эта практика действительно допустима в C99 (и поддерживается некоторыми компиляторами до C99). Разрешение "конечной запятой" упрощает изменение

перечислений, поскольку мы можем добавить константу в конец перечисления без изменения существующих строк кода. Например, может потребоваться добавить белый цвет (WHITE) в перечисление:

```
enum gray_values {  
    BLACK = 0,  
    DARK_GRAY = 64,  
    GRAY = 129,  
    LIGHT_GRAY = 192,  
    WHITE = 255,  
};
```

Запятая после определения LIGHT_GRAY облегчает добавление WHITE в конец списка. Одна из причин этого изменения заключается в том, что C89 позволяет заключительные запятые в инициализаторах, поэтому казалось непоследовательным не допускать такую же гибкость в перечислениях. Кстати, C99 также позволяет заключительные запятые в составных литералах.

Вопрос: Можно ли использовать значения перечисляемого типа в качестве индексов?

Ответ: Да, действительно. Они являются целыми числами и имеют — по умолчанию - значения, которые начинаются с 0 и

отсчитываются вверх, поэтому они делают отличные индексы. Кроме того, в C99 константы перечисления могут использоваться в качестве индексов в назначенных инициализаторах.

Вот пример:

```
enum weekdays {MONDAY, TUESDAY, WEDNESDAY,  
THURSDAY, FRIDAY};
```

```
const char *daily_specials[] = {  
[MONDAY] = "Beef ravioli",  
[TUESDAY] = "BLTs",  
[WEDNESDAY] = "Pizza",  
[THURSDAY] = "Chicken fajitas",  
[FRIDAY] = "Macaroni and cheese"  
};
```

Упражнения

К Разделу 16.1

1. В следующих объявлениях структуры *x* и *y* имеют члены с именами *x* и *y*:

911

```
struct{ int x, y; } x;
```

```
struct{ int x, y; } y;
```

Являются ли эти декларации допустимыми индивидуально ?
Могут ли обе декларации появляться, как показано в программе? Обосновать ответ.

W 2 (a) Объявите структурные переменные с именами *s1*, *s2* и *s3*, каждая из которых имеет действительные и мнимые члены типа *double*.

(b) Измените объявление в части (a) таким образом, чтобы члены *s1* изначально имели значения 0.0 и 1.0, а члены *s2*-1.0 и 0.0. (*s3* не инициализируется.)

(c) Написать инструкции, которые копируют члены *s2* в *s1*.
Можно ли это сделать в одной инструкции или требуется две?

(d) Написать инструкции, которые складывают соответствующие члены *s1* и *s2*, сохраняя результат в *s3*.

3.(a) Показать, как объявить тег *complex* для структуры с двумя членами типа *double*, действительным и мнимым.

(b) Используйте тег `complex` для объявления переменных с именами `c1`, `c2` и `c3`.

(c) Напишите функцию с именем `make_complex`, которая сохранит два аргумента (оба типа `double`) в структуре `complex`, а затем возвращает структуру.

(d) Напишите функцию с именем `add_complex`, которая складывает соответствующие члены своих аргументов (обе структуры `complex`), а затем возвращает результат (другая структура `complex`).

W 4. Повторите упражнение 3, но на этот раз используйте тип с именем `Complex`.

5. Запишите следующие функции, считая, что структура `date` содержит три члена: `month`, `day` и `year` (все типы `int`).

(a) `int day_of_year (struct date d)`

Возвращает день года (целое число от 1 до 366), которое соответствует дате `d`.

(b) `int compare_dates (struct date d1, struct date d2);`

Возвращает -1, если `d1` - более ранняя дата, чем `d2`, +1, если `d1` - более поздняя дата, чем `d2`, и 0, если `d1` и `d2` совпадают.

6. Напишите следующую функцию, предполагая, что структура `time` содержит три члена: `hours`, `minutes` и `seconds` (все типы `int`).

`struct time split_time (long total_seconds);`

`total_seconds` - это время, представленное как количество секунд с полуночи. Функция возвращает структуру, содержащую эквивалентное время в часах (0-23), минутах (0-59) и секундах (0-59).

7. Предположим, что структура `fraction` содержит два члена: `numerator` (числитель) и `denominator` (знаменатель) (оба типа `int`). Напишите функции, которые выполняют следующие операции над дробями:

(a) Приводит f к несократимому виду. Подсказка. Чтобы привести дробь к несократимому виду, сначала вычислите наибольший общий делитель (GCD) числителя и знаменателя. Затем разделите как числитель, так и знаменатель на GCD.

(b) Складывает дроби $f1$ и $f2$.

(e) Вычитает дробь $f2$ из дроби $f1$.

(d) Умножает дроби $f1$ и $f2$.

(e) Делит дробь $f1$ на дробь $f2$.

Дроби f , f_1 и f_2 будут аргументами типа `struct fraction`; каждая функция возвращает значение типа `struct fraction`. Дроби, возвращаемые функциями в разделах (b) - (e), должны быть приведены к несократимой дроби. Подсказка. Вы можете использовать функцию из раздела (a), чтобы помочь записать функции в разделах (b) — (e).

8. Пусть `color` - следующая структура:

```
struct color {  
    int red;  
    int green;  
    int blue;  
};
```

(a) напишите объявление для переменной `const MAGENTA` типа `struct color`, члены которой имеют значения 255, 0 и 255 соответственно.

(b) (C99) повторите часть (a), но используйте назначенный инициализатор, который не задает значение `green`, позволяя ему по умолчанию быть 0.

9. Напишите следующие функции. (Структура `color` определена в Упражнении 8.)

(a) `struct: color make_color (int red, int green, int blue);`

Возвращает структуру `color`, содержащую заданные значения красного, зелёного и синего цветов.

Если любой аргумент меньше нуля, соответствующий член структуры будет вместо этого содержать нуль. Если какой-либо аргумент больше 255, соответствующий член структуры будет содержать 255.

(b) `int getRed (struct color c);`

Возвращает значение члена `c` `red`.

(c) `bool equal_color (struct color color1, struct color color2);`

Возвращает `true`, если соответствующие члены `color1` и `color2` равны.

(d) `struct color brighter (struct color c);`

Возвращает структуру `color`, которая представляет собой более яркую версию цвета `c`. Структура идентична `c`, за исключением того, что каждый член делится на 0,7 (с усечением результата до целого числа). Однако есть три особых случая:

(1) Если все члены `c` равны нулю, функция возвращает цвет, члены которого имеют значение 3.

(2) Если какой-либо член `c` больше 0, но меньше 3, он заменяется на 3 перед делением на 0,7.

(3) Если деление на 0,7 приводит к тому, что член превышает 255, он уменьшается до 255.

(e) `struct color darker (struct color c);`

Возвращает цветовую структуру, которая представляет более тёмную версию цвета *c*. Структура идентична *c*, за исключением того, что каждый член умножен на 0,7 (с усечением результата до целого числа).

10. Следующие структуры предназначены для хранения информации об объектах на графическом экране:

```
struct point { int x, y; };
```

```
struct rectangle { struct point upper_left, lower_right; };
```

Структура `point` хранит координаты *x* и *y* точки на экране. Структура `rectangle` хранит координаты верхних левых и нижних правых углов прямоугольника. Напишите функции, которые выполняют следующие операции над структурой `rectangle` *r*, переданной в качестве аргумента:

(a) Вычисляют площадь *r*.

(b) Вычисляют центр *r*, возвращая его как значение `point`.

Если координата *x* или *y* центра не является целым числом, сохраните её усечённое значение в структуре `point`.

(c) Перемещают *r* на *x* единиц в направлении *x* и *y* единиц в направлении *y*, возвращая модифицированную версию *r*, (*x* и *y* являются дополнительными аргументами функции).

(d) Определите, находится ли точка *p* в пределах *r*, возвращая

917

true или false. (*p* является дополнительным аргументом типа struct point).

К Разделу 16.4.

W 11. Предположим, что *s* - следующая структура:

```
struct {  
    double a;  
    union {  
        char b [4];  
        double c;  
        int d;  
    }e;  
    char f[4];  
} s;
```

Если значения char занимают один байт, значения int занимают четыре байта, а значения double занимают восемь байтов, сколько пространства будет выделяться компилятором Си для *s*? (Предположим, что компилятор не оставляет "дыр" между членами).

12. Предположим, что **u** является следующим объединением:

```
union {  
    double a;  
    struct {  
        char b[4];  
        double c;  
        int d;  
    } e;  
    char f[4]  
}u;
```

Если значения `char` занимают один байт, значения `int` занимают четыре байта, а значения `double` занимают восемь байт, сколько пространства будет выделяться компилятором Си для **u**?

(Предположим, что компилятор не оставляет "дыр" между членами).

13. Предположим, что **s** является следующей структурой (point является структурным тегом, объявленным в Упражнении 10):

```
struct shape {  
    int shape_kind; /* RECTANGLE or CIRCLE */
```

```
struct point center; /* coordinates of center */  
union {  
    struct {  
        int height,width;  
    } rectangle;  
    struct {  
        int radius;  
    } circle;  
}u;  
}s;
```

Если значение `shape_kind` равно `RECTANGLE`, элементы `height` и `width` сохраняют размеры прямоугольника. Если значение `shape_kind` равно `CIRCLE`, член `radius` сохраняет радиус круга. Укажите, какие из следующих утверждений являются допустимыми, и покажите, как исправить те, которые не являются допустимыми:

- (a) `s.shape_kind = RECTANGLE;`
- (b) `e.center.x = 10;`
- (c) `s.height = 25;`
- (d) `s.u.rectangle.widt = 8`
- (e) `s.u.circ1e = 5;`
- (f) `s.u.radius = 5;`

W 14. Пусть `shape` - тег структуры, объявленный в упражнении 13. Запишите функции, которые выполняют следующие операции над структурой `shape s`, переданной в качестве аргумента:

- (a) Вычисляет площадь `s`.
- (b) Перемещает `s` на `x` единиц в направлении `x` и `y` единиц в направлении `y`, возвращая модифицированную версию `s` (`x` и `y` - дополнительные аргументы функции).
- (c) Масштабирует `s` с коэффициентом `c` (значение `double`), возвращающее изменённую версию `s`. (`c` - дополнительный аргумент функции.)

W 15.

- (a) Объявить тег для перечисления, значения которого представляют семь дней недели.
- (b) Использовать `typedef` для определения имени для перечисления части (a).

16. Какие из следующих утверждений о константах перечисления верны?

- (a) Константа перечисления может представлять любое целое число, указанное программистами.
- (b) Константы перечисления имеют те же свойства, что и

константы, созданные с помощью `#define`.

(с) По умолчанию константы перечисления имеют значения 0, 1, 2.

(d) Все константы в перечислении должны иметь разные значения.

(е) Константы перечисления могут использоваться как целые числа в выражениях.

W 17. Предположим, что ***b*** и ***i*** объявлены следующим образом:

```
enum (FALSE, TRUE) b;
```

```
int i;
```

Какие из следующих утверждений являются допустимыми?

Какие из них "безопасны" (всегда дают значимый результат)?

(a) `b = FALSE;`

(b) `b = i;`

(c) `b ++;`

(d) `i - = b;`

(e) `i = 2 * b + 1;`

18. (a) Каждый квадрат шахматной доски может содержать одну фигуру - пешку, коня, слона, ладью, королеву или короля - или может быть пустым. Каждая фигура либо чёрная, либо белая. Определите два типа: `Piece`, который имеет семь возможных значений (одно из которых "пустое") и `Color`,

который имеет два значения.

(b) Используя типы из части (a), определите тип структуры с именем `Square`, который может хранить как тип фигуры, так и её цвет.

(c) Используя тип `Square` из части (b), объявите 8 x 8 массив с именем `board`, который может хранить все содержимое шахматной доски.

(d) Добавить инициализатор в декларацию в части (c), так чтобы первоначальное значение `board` соответствует обычному расположению фигур в начале шахматной игры. Квадрат, который не занят фигурой, должен иметь значение "пустой" и цвет чёрный.

19. Объявить структуру, тег которой `pinball_machine`, следующем составе :

`name` - строка длиной до 40 символов

`year` - целое число (год выпуска)

`type` - перечисление со значениями EM (электромеханические) и SS (твердотельные)

`players` - целое число (представляющее максимальное количество средств воспроизведения).

20. Предположим, что переменная `direction` объявлена следующим образом:

`enum (NORTH, SOUTH, EAST, WEST);`

Пусть x и y - переменные `int`. Напишите инструкцию `switch`, которая проверяет значение `direction`, увеличивая x , если направление `EAST`, уменьшая x , если направление `WEST`, увеличивая y , если направление `SOUTH` и уменьшая y , если направление `NORTH`.

21. Каковы целочисленные значения констант перечисления в каждой из следующих деклараций?

(a) `enum (NUL, SON, STX, ETX);`

(b) `enum (VT = 11, FF, CR);`

(c) `enum (so = 14, SI, DLE, CAN = 24, EM);`

(d) `enum (ENQ = 45, ACK, BEL, LF = 37, ETB, ESC);`

22. Пусть `chess_pieces` будет следующим перечислением:

```
enum chess_pieces (KING, QUEEN, ROCK, BISHOP, KNIGHT,
FAWN);
```

(a) Напишите декларацию (включая инициализатор) для константного массива из целых чисел с именем `piece_value`, который хранит числа 200, 9, 5, 3, 3 и 1, представляющие значение каждой фигуры, от короля до пешки. (Значение короля на самом деле бесконечно, так как "захват" короля (Checkmate) заканчивает игру, но некоторые шахматные программы присваивают королю большое значение, такое как 200.)

(b) (C99) Повторите часть (a), но используйте инициализатор

для инициализации массива. Используйте константы перечисления в `chess_pieces` как индексы в указателях. (Подсказка: см. для примера последний вопрос в Q & A.)

Проекты по программированию

W1. Напишите программу, которая запрашивает у пользователя ввод телефонного международного кода, а затем просматривает его в массиве `country_codes` (см. Раздел 16.3). Если она найдёт код, программа должна отобразить имя соответствующей страны; если нет, программа должна распечатать сообщение об ошибке.

2. Измените программу `inventory.c` раздела 16.3, чтобы операция `p` (`print`) отображала детали, отсортированные по номеру детали.

W 3. Измените программу `inventory.c` раздела 16.3, сделав `inventory` и `num_parts` локальными для функции `main`.

4. Измените программу `inventory.c` в Разделе 16.3, добавив член `price` в структуру `part`. Функция `insert` должна запрашивать у пользователя цену за новый предмет. Функции поиска и печати должны отображать цену. Добавьте новую команду, которая позволяет пользователю изменять цену детали.

5. Измените Проект по программированию 8 из Главы 5, чтобы время хранилось в отдельном массиве. Элементы массива будут структурами, каждая из которых содержит время отправления и соответствующее время прибытия. (Каждый раз будет целое число, представляющее количество минут с полуночи.) Программа будет использовать цикл для поиска в массиве времени отправления, ближайшего к времени, введённому пользователем.

6. Модифицируйте программу Проекта по программированию 9 из Главы 5, чтобы каждый дата, введённая пользователем, сохраняется в структуре `date` (см. упражнение 5). Включите функцию `compare_dates` упражнения 5 в вашу программу.

17 Расширенное использование указателей

Мысленно человек может только воспроизвести сложную информацию. Движение, или течение, или изменение перспективы важнее, чем статическое изображение, каким бы красивым оно не было.

В предыдущих главах мы видели два важных использования указателей. В Главе 11 показано, как использование указателя на переменную в качестве аргумента функции позволяет функции изменять переменную. В Главе 12 показано, как обрабатывать массивы, выполняя арифметику на указателях на элементы массива. Эта глава завершает наш обзор указателей, рассматривая два дополнительных приложения: динамическое выделение памяти и указатели на функции.

Используя динамическое выделение памяти, программа может получать блоки памяти по мере необходимости во время выполнения. Раздел 17.1 объясняет основы динамического распределения памяти. Раздел 17.2 обсуждает динамически выделенные строки, которые обеспечивают большую гибкость, чем обычные символьные массивы. Раздел 17.3 охватывает динамическое выделение памяти для массивов в целом. В Разделе 17.4 рассматривается проблема

освобождения блоков динамически выделяемой памяти при освобождении памяти, когда она больше не нужна.

17.1 Динамическое распределение памяти

Структуры данных Си обычно фиксированы по размеру. Например, количество элементов в массиве фиксируется после компиляции программы. (В C99 длина массива переменной длины определяется во время выполнения, но остаётся фиксированной до конца времени существования массива.) Фиксированный размер данных может быть проблемой, так как мы вынуждены выбирать их размеры при написании программы; мы не можем изменить размеры, не изменив программу и не скомпилировав её снова. Рассмотрим программу `inventory` Раздела 16.3, которая позволяет пользователю добавлять детали в базу данных. База данных хранится в массиве длиной 100. Для увеличения ёмкости базы данных можно увеличить размер массива и перекомпилировать программу. Но независимо от размера массива, всегда есть вероятность, что он заполнится. К счастью, не все потеряно. Си поддерживает динамическое выделение памяти: возможность выделения памяти во время выполнения программы. Используя динамическое выделение

памяти, мы можем проектировать структуры данных, которые растут (и сжимаются) по мере необходимости. Хотя динамическое выделение памяти доступно для всех типов данных, оно чаще всего используется для строк, массивов и структур. Динамически распределенные структуры представляют особый интерес, так как мы можем связать их вместе, чтобы сформировать списки, деревья и другие структуры данных.

Функции выделения памяти

Для динамического выделения памяти необходимо вызвать одну из трёх функций выделения памяти, объявленных в заголовке `<stdlib.h>`:

функция `malloc` выделяет блок памяти, но не инициализирует его.

`calloc` - выделяет блок памяти и очищает его.

`realloc` - изменяет размер ранее выделенного блока памяти.

Из трёх, `malloc` является наиболее используемым. Она более эффективна, чем `calloc`, так как ей не нужно очищать блок памяти, который она выделяет. Когда мы вызываем функцию выделения памяти для запроса блока памяти, функция не

имеет представления, какой тип данных мы планируем хранить в блоке, поэтому она не может вернуть указатель на обычный тип, такой как `int` или `char`. Вместо этого функция возвращает значение типа `void *`. Значение `void *` является "универсальным" указателем - по сути, просто адресом памяти.

Нулевой (пустой) указатель

Когда вызывается функция выделения памяти, всегда есть вероятность, что она не сможет найти блок памяти, достаточно большой, чтобы удовлетворить наш запрос. Если это произойдёт, функция вернёт нулевой указатель. Нулевой указатель - это "указатель на ничто" - специальное значение, которое можно отличить от всех допустимых указателей. После того, как мы сохранили возвращаемое значение функции в переменной указателе, мы должны проверить, является ли она нулевым указателем.

Это обязанность программиста, проверять возвращаемое значение любой функции выделения памяти и принимать соответствующие меры, если это нулевой указатель. Результат попытки доступа к памяти через нулевой указатель не

определён; программа может аварийно завершить работу или вести себя непредсказуемо.

Указатель null представлен макросом с именем NULL, поэтому мы можем проверить возвращаемое значение malloc следующим образом:

```
p = malloc (10000);  
if (p == NULL) { /* не удалось выполнить распределение;  
принять соответствующие меры */  
}
```

Некоторые программисты объединяют вызов malloc с тестом на NULL:

```
if ((p = malloc (10000)) == NULL) {  
/* не удалось выполнить распределение; принять  
соответствующие меры */  
}
```

Макрос NULL определяется в шести заголовках: <locale.h>, <stddef.h>, <stdio.h>, <stdlib.h>, <string.h> и <time.h>. (Заголовок C99 <wchar.h> также определяет NULL.) Пока один из этих заголовков включён в программу, компилятор распознает NULL. Программа, которая использует любую из функций выделения памяти, будет включать <stdlib.h>, конечно, делая доступным NULL. В Си указатели проверяют true или false так же, как и числа. Все непустые указатели при

931

проверке дают true; только нулевые указатели являются ложными (false). Таким образом, вместо написания

```
if (p == NULL)...
```

мы могли бы написать

```
if (!p)...
```

а вместо написания

```
if (p != NULL)...
```

мы могли бы написать

```
if (p)...
```

В качестве стиля я предпочитаю явное сравнение с NULL.

17.2 Динамически выделяемые строки

Динамическое выделение памяти часто полезно для работы со строками. Строки хранятся в символьных массивах и может быть трудно предвидеть, сколь велики должны быть эти массивы. Динамическое выделение строк позволяет отложить принятие решения до запуска программы.

Использование malloc для выделения памяти для строки

Функция malloc имеет следующий прототип:

```
void *malloc (size_t size);
```

функция malloc выделяет блок размером size байтов и возвращает указатель на него. Обратите внимание, что size имеет тип size_t, целочисленный тип без знака, определённый в библиотеке C.

Если мы не выделяем очень большой блок памяти, мы можем считать size обычным целым числом. Использование malloc для выделения памяти для строки достаточно просто, потому что Си гарантирует, что значение char требует точно одного байта хранения (sizeof (char) равняется 1, другими словами).

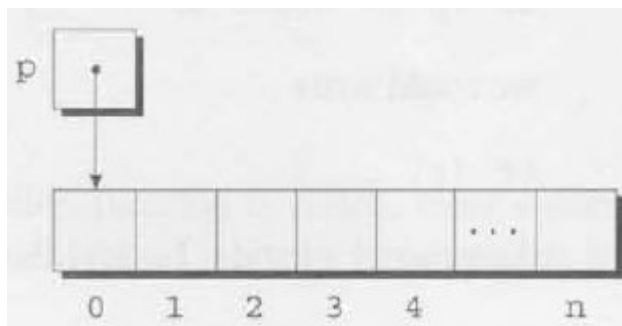
933

Чтобы выделить место для строки из n символов, мы напишем

```
p = (char *) malloc(n + 1);
```

При использовании `malloc` для выделения места для строки не забудьте указать место для символа `null`.

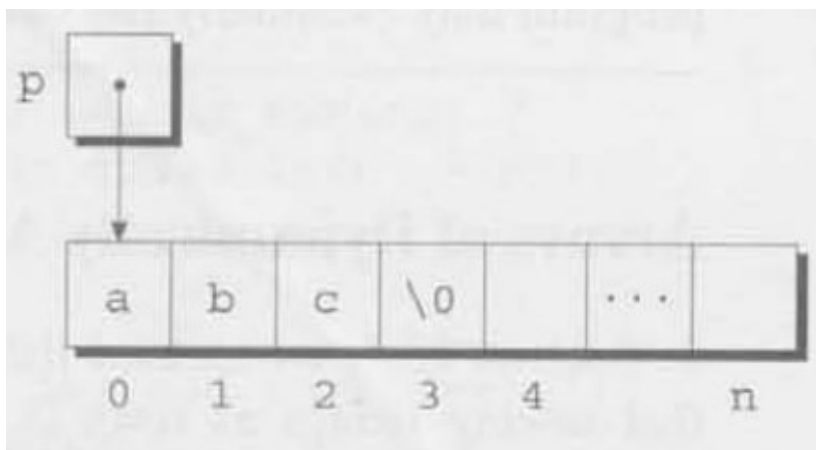
Память, выделенная с помощью `malloc`, никак не очищается и не инициализируется, поэтому p будет указывать на неинициализированный массив из $n + 1$ символов:



Вызов `strcpy` является одним из способов инициализации этого массива:

```
strcpy(p, "abc");
```

Первые четыре символа в массиве теперь будут `a`, `b`, `c` и `\0`.



Использование динамического выделения памяти в строковых функциях

Динамическое выделение памяти позволяет писать функции, возвращающие указатель на "новую" строку - строку, которая не существовала до вызова функции.

Рассмотрим задачу написания функции, которая объединяет две строки, не изменяя ни одну из них. Стандартная библиотека Си не включает такую функцию (`strcat` не совсем то, что мы хотим, так как она изменяет одну из строк, переданных ей), но мы можем легко написать нашу собственную.

Наша функция измерит длину двух строк, которые будут объединены, а затем вызовет `malloc`, чтобы выделить нужное количество места для результатов. Затем функция копирует

первую строку в новое пространство и вызовет `strcat` для объединения со второй строкой.

```
char *concat(const char *s1, const char *s2){
    char *result;
    result = malloc(strlen(s1) + strlen(s2) + 1);
    if (result == NULL){
        printf("Error: malloc failed in concat\n");
        exit (EXIT_FAILURE);
    }
    strcpy(result, s1);
    strcat (result, s2);
    return result;
}
```

Если `malloc` возвращает нулевой указатель, `concat` выводит сообщение об ошибке и завершает работу программы. Это не всегда правильное действие: некоторые программы должны восстанавливаться после сбоев выделения памяти и продолжать работу.

Вот как можно вызвать функцию `concat`:

```
p = concat ("abc", " def");
```

После вызова **p** будет указывать на строку "abcdef", которая хранится в динамически выделяемом массиве. Массив состоит из семи символов, включая нулевой символ в конце.

Функции, такие как `concat`, которые динамически распределяют память, должны использоваться с осторожностью. Когда строка, которая возвращает `concat`, больше не нужна, мы должны вызвать функцию `free`, чтобы освободить пространство, которое занимает строка. Если мы этого не сделаем, программа может в конечном итоге исчерпать память

Массивы динамически выделяемых строк

В Разделе 13.7 мы рассмотрели проблему хранения строк в массиве. Мы обнаружили, что хранение строк в виде строк в двумерном массиве символов может привести к потере места, поэтому мы попытались создать массив указателей на строковые литералы. Методы раздела 13.7 работают так же хорошо, если элементы массива являются указателями на динамически выделенные строки.

Чтобы проиллюстрировать это, перепишем программу `remind.c` Раздела 13.5, которая печатает ежемесячный список ежедневных напоминаний.

ПРОГРАММА печати списка напоминаний за один месяц (Пересмотренная)

Оригинальная программа `remind.c` хранит строки напоминания в двумерном массиве символов, причём каждая строка массива содержит одну текстовую строку. После того, как программа прочитает день и связанное с ним напоминание, она будет просматривать массив, чтобы определить, куда поместить день, используя для сравнения `strcmp`. Затем она использует `strcpy` для перемещения всех строк ниже этой точки вниз на одну позицию. Наконец, программа копирует день в массив и вызывает `strcat`, чтобы добавить напоминание в день. (См. *Пояснение к программе `remind.c`*). В новой программе `remind2.c` массив будет одномерным, его элементы будут указателями на динамически выделяемые строки. Переключение на динамически выделяемые строки в этой программе будет иметь два основных преимущества.

Во-первых, мы можем использовать память более эффективно, выделяя точное количество символов, необходимых для хранения напоминания, вместо того, чтобы хранить напоминание в фиксированном количестве символов, как это делает оригинальная программа. Во-вторых, нам не нужно будет вызывать `strcpy` для перемещения существующих строк напоминания, чтобы освободить место для нового

напоминания. Вместо этого мы просто переместим указатели на строки. Вот новая программа, с изменениями жирным шрифтом. Переход от двумерного массива к массиву указателей оказывается на удивление простым: нам нужно всего лишь изменить восемь строк программы.

```
//remind2.c
```

```
/* Печать списка напоминаний на один месяц (версия  
динамическими строками)*/
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_REMIND 50 /* максимальное число  
напоминаний */
```

```
#define MSG_LEN 60 /* максимальная длина напоминания*/
```

```
int read_line(char str[], int n);
```

```
int main(void){
```

```
    char *reminders[MAX_REMIND];
```

939

```
char day_str[3], msg_str[MSG_LEN+1];
int day, i, j, num_remind = 0;

for (;;) {
    if (num_remind == MAX_REMIND) {
        printf("-- No space left --\n");
        break;
    }

    printf("Enter day and reminder: ");
    scanf("%2d", &day);
    if (day == 0)
        break;

    sprintf(day_str, "%2d", day);
    read_line(msg_str, MSG_LEN);

    for (i = 0; i < num_remind; i++)
        if (strcmp(day_str, reminders[i]) < 0)
            break;

    for (j = num_remind; j > i; j--)
```

```
reminders[j] = reminders[j-1];
```

```
reminders[i] = malloc(2 + strlen(msg_str) + 1);
```

```
if (reminders[i] == NULL) {
```

```
    printf("-- No space left --\n");
```

```
    break;
```

```
}
```

```
strcpy(reminders[i], day_str);
```

```
strcat(reminders[i], msg_str);
```

```
num_remind++;
```

```
}
```

```
printf("\nDay Reminder\n");
```

```
for (i = 0; i < num_remind; i++)
```

```
    printf(" %s\n", reminders[i]);
```

```
return 0;
```

```
}
```



```
int read_line(char str[], int n){  
    int ch, i = 0;  
  
    while ((ch = getchar()) != '\n')  
        if (i < n)  
            str[i++] = ch;  
    str[i] = '\0';  
    return i;  
}
```

17.3 Динамически выделяемые массивы

Динамически выделяемые массивы имеют те же преимущества, что и динамически выделяемые строки (неудивительно, что строки являются массивами). Когда мы пишем программу, часто бывает трудно оценить правильный размер массива; было бы удобнее подождать, пока программа запустится, чтобы решить, насколько большим должен быть массив. C решает эту проблему, позволяя программе выделять пространство для массива во время выполнения, а затем получить доступ к массиву через указатель на его первый

элемент. Тесная связь между массивами и указателями, которую мы рассмотрели в главе 12, делает динамически выделяемый массив таким же простым в использовании, как и обычный массив. Хотя `malloc` может выделять пространство для массива, иногда используется функция `calloc`, так как она инициализирует выделяемую память. Функция `realloc` позволяет нам заставить массив "вырасти" или "сжаться" по мере необходимости.

Использование `malloc` для выделения памяти для массива

Мы можем использовать `malloc` для выделения пространства памяти для массива так же, как мы использовали его для выделения пространства для строки. Основное отличие состоит в том, что элементы произвольного массива не обязательно будут иметь длину в один байт, каковыми они являются в строке. В результате нам потребуется использовать оператор `sizeof` для вычисления объёма пространства, необходимого для каждого элемента. Предположим, мы пишем программу, которой нужен массив из n целых чисел, где n должно быть вычислено во время выполнения программы. Сначала объявим переменную указателя:

```
int *a;
```

Как только значение `n` станет известно, программа вызовет `malloc`, чтобы выделить место для массива:

```
a = malloc(n * sizeof(int));
```

Всегда используйте `sizeof` при вычислении, сколько места требуется для массива. Неспособность выделить достаточно памяти может иметь серьёзные последствия. Рассмотрим следующую попытку выделения памяти для массива из ***n*** целых чисел:

```
a = malloc(n * 2);
```

Если значения `int` больше двух байт (как у большинства компьютеров), `malloc` не выделит достаточно большой блок памяти. Когда мы позже попытаемся получить доступ к элементам массива, программа может аварийно завершить работу или вести себя хаотично.

Как только она указывает на динамически выделяемый блок памяти, мы можем игнорировать тот факт, что ***a*** является указателем и использовать его вместо имени массива, благодаря связи между массивами и указателями в Си. Например, можно использовать следующий цикл для инициализации массива, на который указывает ***a***:

```
for (i = 0; i < n; i++)  
    a[i] = 0;
```

У нас также есть возможность использовать арифметику указателя вместо индексирования для доступа к элементам массива.

Функция `calloc`

Хотя функция `malloc` может использоваться для выделения памяти для массива, Си предоставляет альтернативу - функцию `calloc` - это иногда лучше, `calloc` имеет следующий прототип в `<stdlib.h>` :

```
void *calloc(size_t nmemb, size_t size);
```

`calloc` выделяет пространство для массива с элементами `nmemb`, каждый из которых имеет размер `size` в байтах; она возвращает нулевой указатель, если запрошенное пространство недоступно.

После выделения памяти `calloc` инициализирует её, установив для всех битов значение 0. Например, следующий вызов `calloc` выделяет пространство для массива из *n* целых чисел, которые изначально гарантированно равны нулю:

```
a = calloc(n, sizeof(int));
```

Поскольку `calloc` очищает выделяемую память, а `malloc` этого не делает, мы можем иногда выделять пространство для

объекта, отличного от массива. Вызывая `calloc` с **1** в качестве первого аргумента, мы можем выделить пространство для элемента данных любого типа:

```
struct point {int x, y;}*p;  
p = calloc(1, sizeof(struct point));
```

После выполнения этой инструкции **p** будет указывать на структуру, члены которой **x** и **y** установлены в ноль.

Функция `realloc`

После выделения памяти для массива мы можем обнаружить, что она слишком велика или слишком мала. Функция `realloc` может изменить размер массива, чтобы лучше удовлетворить нашим потребностям. Следующий прототип `realloc` отображается в `<stdlib.h>`

```
void *realloc(void *ptr, size_t size);
```

`ptr` при вызове `realloc` должен указывать на блок памяти, полученный предыдущим вызовом `malloc`, `calloc` или `realloc`. Параметр `size` представляет новый размер блока, который

может быть больше или меньше исходного размера. Хотя `realloc` не требует, чтобы `ptr` указывал на память, используемую в качестве массива, на практике обычно это и происходит.

Убедитесь, что указатель, переданный в `realloc`, получен из предыдущего вызова `malloc`, `calloc` или `realloc`. Если это не так, вызов `realloc` вызывает неопределённое поведение.

Стандарт Си излагает ряд правил, касающихся поведения `realloc`:

- При расширении блока памяти `realloc` не инициализирует байты, добавленные в блок.
- Если `realloc` не может увеличить блок памяти в соответствии с запросом, он возвращает нулевой указатель; данные в старом блоке памяти остаются неизменными.
- Если `realloc` вызывается с нулевым указателем в качестве первого аргумента, он ведёт себя как `malloc`.

■ Если `realloc` вызывается с 0 в качестве второго аргумента, блок памяти освобождается.

Стандарт Си не указывает, каким именно образом работает `realloc`. Тем не менее, мы ожидаем, что она будет достаточно эффективна. При запросе уменьшить размер блока памяти `realloc` должна уменьшить блок "на месте", не перемещая данные, хранящиеся в блоке.

Точно так же `realloc` всегда должна пытаться расширить блок памяти, не перемещая его. Если она не может увеличить блок (потому что байты, следующие за блоком, уже используются для других целей), `realloc` выделит новый блок в другом месте, а затем скопирует содержимое старого блока в новый.

Как только `realloc` возвратился, обязательно обновите все указатели на блок памяти, так как возможно, что `realloc` переместила блок в другое место.

17.4 Освобождение памяти

`malloc` и другие функции выделения памяти получают блоки памяти из пула хранения, известного как куча.

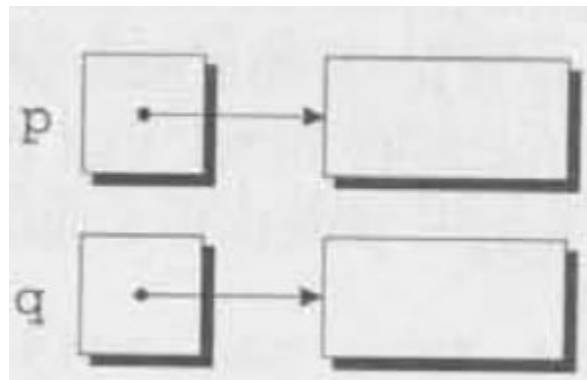
Слишком частый вызов этих функций - или запросы ими больших блоков памяти - могут исчерпать кучу, заставляя функции возвращать нулевой указатель. Что ещё хуже, программа может выделять блоки памяти, а затем терять их, растрачивая память впустую. Рассмотрим следующий пример:

```
p = malloc(...);
```

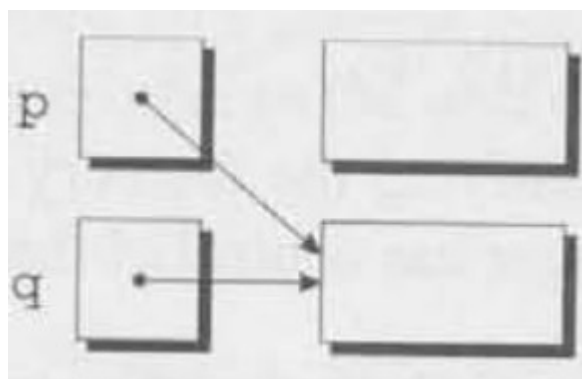
```
q = malloc(...);
```

```
p = q;
```

После выполнения первых двух инструкций ***p*** указывает на один блок памяти, а ***q***-на другой:



После того как ***q*** присваивается ***p***, обе переменные указывают на второй блок памяти :



Нет указателей на первый блок (потерянный), поэтому мы никогда не сможем использовать его снова. Блок памяти, который больше не доступен программе, называется мусором. Программа, которая оставляет после себя мусор, имеет утечку памяти.

Некоторые языки предоставляют сборщик мусора, который автоматически находит и перерабатывает мусор, но в Си его нет.

Вместо этого, каждая программа на языке Си отвечает за утилизацию собственного мусора, вызывая функцию `free` для освобождения ненужной памяти.

Функция `free`

Функция `free` имеет следующий прототип в `<stdlib.h>`:

```
void free (void * ptr);
```

Использовать `free` легко; мы просто передаём ей указатель на блок памяти, который больше не нужен:

```
p = malloc(...);
```

```
q = malloc(...);
```

```
free (p);
```

```
p = q;
```

Вызов `free` освобождает блок памяти, на который указывает ***p***. Этот блок теперь доступен для повторного использования при последующих вызовах функции `malloc` или других функций распределения памяти.

Аргумент `free` должен быть указателем, который ранее возвращался функцией распределения памяти.

(Следовательно, аргумент может быть нулевым указателем и в этом случае вызов `free` не имеет эффекта.)

Передача `free` указателя на любой другой объект (такой как переменная или элемент массива) вызывает неопределённое поведение.

Проблема "висячего указателя"

Хотя функция `free` позволяет нам освобождают память, которая больше не нужна, её использование приводит к новой проблеме: висячие указатели.

Вызов `free (p)` освобождает блок памяти, на который указывает ***p***, но не изменяет сам ***p***. Если мы забудем, что ***p*** больше не указывает на допустимый блок памяти, может начаться хаос:

```
char *p = malloc(4);
```

```
...
```

```
free (p); /*Чтобы избежать проблем с висячим указателем  
установите p = NULL;*/
```

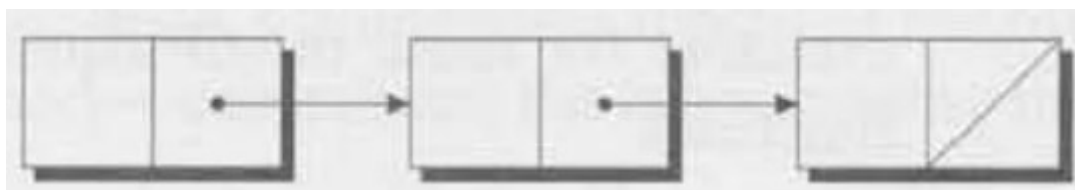
```
strcpy(p, "abc"); /*** WRONG ***/
```

Изменение памяти, на которую указывает ***p***, является серьёзной ошибкой, так как наша программа больше не контролирует эту память. Попытка доступа или изменения освобождённого блока памяти приводит к неопределённому поведению. Попытка изменить освобождённый блок памяти, вероятно, будет иметь катастрофические последствия, которые могут включать сбой программы. Висячие указатели может быть трудно обнаружить, так как несколько указателей могут указывать на один и тот же блок памяти. Когда блок освобождается, все указатели остаются висящими.

17.5 Связанные списки

Динамическое выделение памяти особенно полезно для построения списков, деревьев, графов и других связанных структур данных. Мы рассмотрим связанные списки в этом разделе; обсуждение других связанных структур данных выходит за рамки данной книги. Для получения дополнительной информации обратитесь к книге, такой как Роберт Седжвик "Фундаментальные алгоритмы на С" Части 1-5. Анализ, Структуры данных, Сортировка, Поиск, Алгоритмы на графах. Третья редакция. Издательство DiaSoft. 2003.

Связанный список состоит из цепочки структур (называемых узлами), каждый узел содержит указатель на следующий узел в цепочке:



Последний узел в списке содержит нулевой указатель, показанный здесь в виде диагональной линии. В предыдущих главах мы использовали массив всякий раз, когда нам нужно было хранить коллекцию элементов данных; связанные списки дают нам альтернативу.

Связанный список более гибок, чем массив: мы можем легко вставлять и удалять узлы в связанном списке, позволяя списку расти и сжиматься по мере необходимости. С другой стороны, мы теряем возможность "произвольного доступа" массива. К любому элементу массива можно получить доступ за один и тот же промежуток времени; доступ к узлу в связанном списке осуществляется быстрее, если узел близок к началу списка и медленнее, если он близок к концу.

Этот раздел описывает, как настроить связанный список в Си. Он также показывает, как выполнять некоторые общие операции над связанными списками: вставка узла в начало списка, поиск узла и удаление узла.

Объявление типа узла

Чтобы настроить связанный список, первое, что нам потребуется, это структура, представляющая один узел (node) в списке.

Для простоты предположим, что узел не содержит ничего, кроме целого числа (данных узла) плюс указатель на следующий узел в списке.

Вот как будет выглядеть наша структура узлов:

```
struct node {  
    int value; /* данные, хранящиеся в узле */  
    struct node *next; /*указатель на следующий узел */  
};
```

Обратите внимание, что следующий член имеет тип `struct node*`, что означает, что он может хранить указатель на структуру `node`. Кстати, в имени `node` нет ничего особенного; это просто обычный тег структуры. Один из аспектов структуры `node` заслуживает особого упоминания.

Как поясняется в разделе 16.2, мы обычно имеем возможность использовать либо тег, либо имя `typedef` для определения имени для определённого типа структуры. Однако, когда структура имеет член, который указывает на тот же тип структуры, что и узел, мы должны использовать тег структуры. Без тега `node` у нас не было бы способа объявить тип `next`.

(В структуре, членом которой является указатель на этот же тип, этот член необходимо объявлять явно с использованием служебного слова `struct`

```
typedef struct node {  
    int value;  
    struct node *next;  
} node;)
```

Теперь, когда у нас объявлена структура `node`, нам нужен способ отслеживать, где начинается список. Другими словами, нам нужна переменная, которая всегда указывает на первый узел в списке. Назовём переменную `first`:

```
struct node *first = NULL;
```

Установка `first` в `NULL` означает, что список изначально пуст.

Создание узла

При создании связанного списка мы будем создавать узлы по одному, добавляя каждый из них в список.

Создание узла требует трёх шагов:

1. Выделяем память для узла.
2. Сохраняем данные в узле.
3. Вставляем узел в список.

Сейчас мы сосредоточимся на первых двух шагах. Когда мы создадим узел, нам понадобится переменная, которая может временно указывать на узел, пока он не будет вставлен в список.

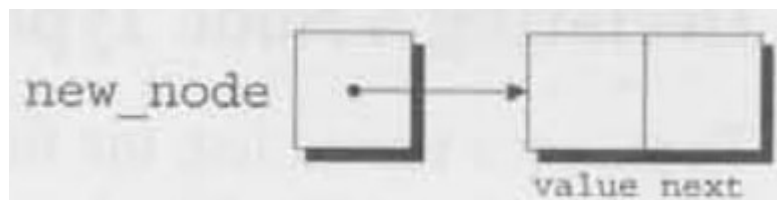
Назовём эту переменную `new_node`:

```
struct node *new_node;
```

Мы будем использовать malloc для выделения памяти для нового узла и сохраним возвращаемое значение в new_node:

```
new_node = malloc(sizeof(struct node));
```

new_node теперь указывает на блок памяти, достаточно большой, чтобы содержать структуру node:



Будьте внимательны, задавая sizeof имя выделяемого типа, а не имя указателя на этот тип:

```
new_node = malloc (sizeof (new_node)); /***WRONG ***/  
( new_node = malloc (sizeof *new_node);
```

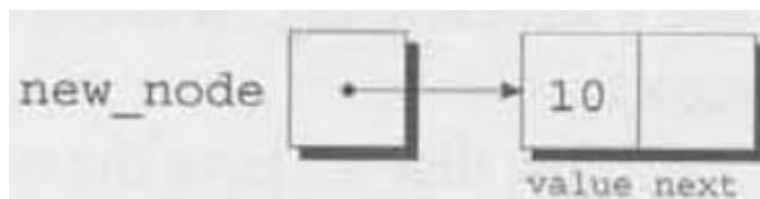
Программа все равно будет компилироваться, но malloc выделит память только для указателя на структуру узла. Вероятный результат - сбой позже, когда программа попытается сохранить данные в узле, на который предположительно указывает new_node.

957

Далее мы будем хранить данные в элементе value нового узла:

```
(*new_node).value = 10;
```

Вот как будет выглядеть картинка после этого присваивания:



Чтобы получить доступ к члену `value` узла, мы применили оператор косвенного обращения `*` (для ссылки на структуру, на которую указывает `new_node`), а затем оператор выбора. (для выбора члена структуры). Скобки вокруг `*new_node` являются обязательными, поскольку в противном случае оператор `.` будет иметь приоритет над оператором `*`.

Оператор ->

Прежде чем перейти к следующему шагу- вставке нового узла в список- давайте рассмотрим полезную пиктограмму для быстрого доступа к объекту.

Доступ к члену структуры с помощью указателя настолько распространён, что Си предоставляет специальный оператор

именно для этой цели. Этот оператор, известный как выбор-стрелка вправо - является знаком минуса, за которым следует `>`. Используя оператор `->`, мы можем написать

```
new_node -> value = 10;
```

вместо

```
(*new_node) . value = 10;
```

Оператор `->` представляет собой комбинацию операторов `*` и `.`; он выполняет косвенное обращение к `new_node`, чтобы найти структуру, на которую он указывает, затем выбирает элемент `value` структуры. Оператор `->` создаёт `lvalue`, поэтому мы можем использовать его везде, где разрешена обычная переменная.

Мы только что видели пример, в котором `new_node -> value` отображается в левой части присваивания.

Он может так же легко появиться в вызове `scanf`:

```
scanf ( "%i",&new_node -> value);
```

Обратите внимание, что оператор `&` все ещё требуется, хотя `new_node` является указателем. Без `&` мы передадим `scanf` значение `new-node->value`, которое имеет тип `int`.

Вставка узла в начало связанного списка

Одним из преимуществ связанного списка является возможность добавления узлов в любой точке списка: в начале, в конце или в любом месте в середине.

Однако начало списка - это самое простое место для вставки узла, поэтому давайте сосредоточимся на этом случае. Если `new_node` указывает на узел, который нужно вставить, а `first` указывает на первый узел в связанном списке, то нам понадобятся две инструкции для вставки узла в список.

Сначала мы изменим член `next` нового узла, чтобы он указывал на узел, который ранее был в начале списка:

```
new_node->next = first;
```

Во-вторых, мы сделаем `first` указывающим на новый узел:

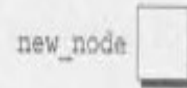
```
first = new_node;
```

Будут ли эти инструкции работать, если список пуст при вставке узла?

Да, к счастью. Чтобы убедиться, что это правда, давайте проследим процесс установка двух узлов в пустой список.

Сначала мы вставим узел, содержащий число 10, а затем узел, содержащий 20. На следующих рисунках нулевые указатели отображаются в виде диагональных линий.

```
first = NULL;
```



```
new_node->value = 10;
```



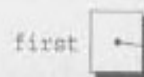
```
new_node->next = first;
```



```
first = new_node;
```



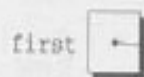
```
new_node = malloc(sizeof(struct node));
```



```
new_node->value = 20;
```



```
new_node->next = first;
```



```
first = new_node;
```



Вставка узла в связанный список является настолько распространённой операцией, что мы, вероятно, захотим написать функцию для этой цели. Назовём функцию `add_to_list`. Она будет иметь два параметра: `list` (указатель на первый узел в старом списке) и *n* (целое число, которое будет храниться в новом узле)

```
struct node *add_to_list(struct node *list, int n){
    struct node *new_node;
    new_node = malloc(sizeof (struct node));
    if (new_node == NULL){
        printf("Error: malloc failed in add_to_list\n");
        exit (EXIT_FAILURE);
    }
    new_node->value =n;
    new_node->next = list;
    return new_node;
}
```

Обратите внимание, что `add_to_list` не изменяет указатель `list`. Вместо этого, она возвращает указатель на вновь созданный узел (теперь в начале списка).

Когда мы вызовем `add_to_list`, нам нужно будет сохранить его возвращаемое значение в `first`:

```
first = add_to_list(first, 10);
```

```
first = add_to_list(first, 20);
```

Эти инструкции добавляют узлы, содержащие 10 и 20, в список, на который указывает `first`. Сделать `add_to_list` обновляющей `first` напрямую, а не возвращающей новое значение для `first`, оказывается сложно.

Мы вернёмся к этой проблеме в разделе 17.6. Следующая функция использует `add_to_list` для создания связанного списка, содержащего числа, введённые пользователем

```
struct node *read_numbers (void){
    struct node *first = NULL;
    int n;
    printf("Enter a series of integers (0 to terminate");

    for (;;) {
        scanf("%d", &n);
        if (n == 0)
            return first;
        first = add_to_list(first, n);
    }
}
```

Номера в списке будут расположены в обратном порядке, так как `first` всегда указывает на узел, содержащий последний введенный номер.

Поиск в связанном списке

Как только мы создали связанный список, нам, возможно, придется искать в нем определённый фрагмент данных.

Хотя цикл `while` можно использовать для поиска в списке, инструкция `for` часто превосходит его. Мы привыкли использовать инструкцию `for` при написании циклов, включающих подсчёт, но её гибкость делает инструкцию `for` подходящей и для других задач, включая операции со связанными списками.

Вот обычный способ посетить узлы в связанном списке, используя переменную указателя ***p*** для отслеживания "текущего" узла:

идиома: `for (p = first; p != NULL; p = p -> next)`

...

Присваивание: `p = p -> next`

перемещает указатель ***p*** от одного узла к другому. Присвоение этой формы неизменно используется в Си при написании

цикла, проходящего через связанный список. Напишем функцию `search_list`, которая ищет в списке (на который указывает параметр `list`) целое число *n*. Если она находит *n*, `search_list` вернёт указатель на узел, содержащий *n*; в противном случае она вернёт нулевой указатель. Наша первая версия `search_list` основана на идиоме "обход списка":

идиома:

```
struct node *search_list(struct node *list, int n){
    struct node *p;
    for (p = list; p != NULL; p = p -> next)
        if (p -> value == n)
            return p;
    return NULL;
}
```

Конечно, есть много других способов написать `search_list`. Одним из вариантов было бы исключить переменную *p*, вместо этого используя сам список для отслеживания текущего узла:

```
struct node *search_list(struct node *list, int n){
    for(; list != NULL; list = list -> next)
        if (list -> value == n)
```



```

        return list;

    return NULL;
}

```

Поскольку `list` является копией исходного указателя списка, нет никакого вреда в изменении его внутри функции. Другой альтернативой является объединение теста `list->value == n` с тестом `list != NULL`:

```

struct node *search_list(struct node *list, int n){
    for (; list != NULL && list->value != n ; list= list->next)
        ;
    return list;
}

```

Так как `list` равен `NULL`, если мы дойдём до конца списка, то возврат `list` будет правильным, даже если мы не найдём ***n***. Эта версия `search_list` может быть немного яснее, если мы используем инструкцию `while`:

```

struct node *search_list(struct node *list, int n){
    while (list != NULL && list->value != n)
        list = list->next;
    return list;
}

```

Удаление узла из связанного списка

Большим преимуществом хранения данных в связанном списке является то, что мы можем легко удалить

узлы, которые нам больше не нужны.

Удаление узла, как и создание узла, состоит из трёх шагов:

1. Найдите узел, который необходимо удалить.
2. Измените предыдущий узел так, чтобы он "обходил" удаляемый узел.
3. Вызовите free, чтобы освободить место, занимаемое удаленным узлом.

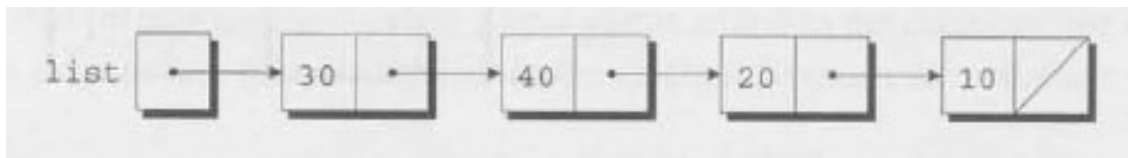
Шаг 1 сложнее, чем кажется. Если мы будем просматривать список обычным способом, мы получим указатель на удаляемый узел. К сожалению, мы не сможем выполнить шаг 2, который требует изменения предыдущего узла.

Существуют различные решения этой проблемы. Мы будем использовать метод "завершающего указателя": при поиске в списке на шаге 1 мы будем иметь указатель на предыдущий узел (prev), а также указатель на текущий узел (cur). Если list указывает на список для поиска и *n*-целое число для удаления, следующий цикл реализует шаг 1:

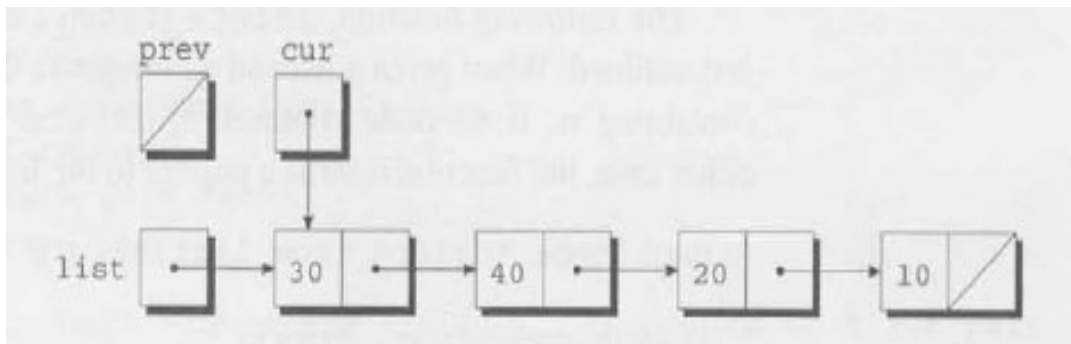
```
for (cur = list, prev = NULL; cur != NULL && cur -> value != n;  
    prev = cur, cur = cur -> next)  
    ;
```

Здесь мы видим силу инструкции `for` языка Си. Этот довольно экзотический пример, с его пустым телом и либеральным использованием оператора запятая, выполняет все действия, необходимые для поиска *n*.

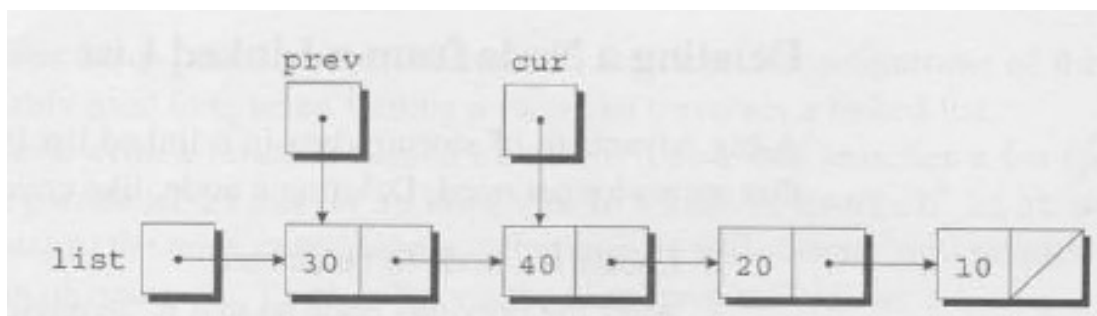
Когда цикл завершается, `cur` указывает на узел, который нужно удалить, в то время как `prev` указывает на предыдущий узел (если он есть). Чтобы увидеть, как работает этот цикл, предположим, что `list` указывает на список, содержащий 30, 40, 20 и 10, в таком порядке.



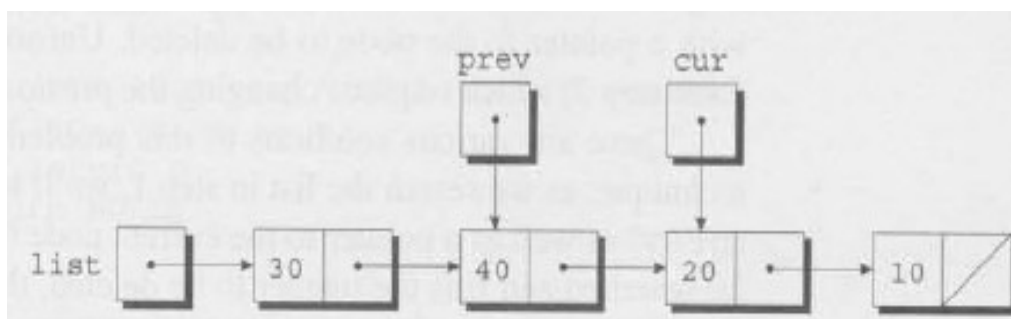
Предположим, что *n* равно 20, поэтому наша цель-удалить третий узел в списке. После выполнения `cur = list`, `prev = NULL`, `cur` указывает на первый узел в списке:



Тест $cur \neq \text{NULL} \ \& \ \& \ cur \rightarrow value \neq n$ даёт true, так как cur указывает на узел, а узел не содержит 20. После того, как будет выполнено $prev = cur$, $cur = cur \rightarrow next$, мы увидим, как указатель $prev$ будет следовать за cur :



Опять тест $cur \neq \text{NULL} \ \& \ \& \ cur \rightarrow value \neq n$ даёт true, поэтому $prev = cur$, $cur = cur \rightarrow next$ выполняется ещё раз:

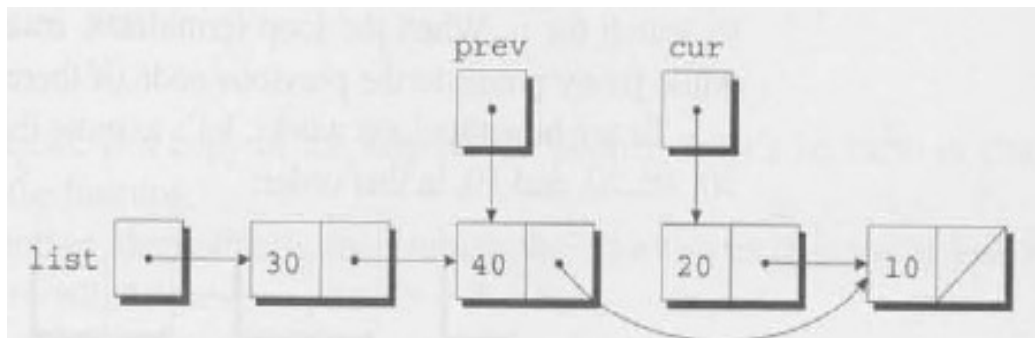


969

Так как `cur` теперь указывает на узел, содержащий 20, условие `cur -> value != n` равно `false` и цикл завершается.

Далее мы выполним обход, требуемый на шаге 2. Инструкция `prev -> next = cur -> next;`

делает указатель предыдущего узла указывающим на узел после текущего узла:



Теперь мы готовы к шагу 3, освобождающему память, занятую текущим узлом:

`free (cur);`

Следующая функция, `delete_from_list`, использует стратегии, которые мы только что изложили. Когда дан список и целое число ***n***, функция удаляет первый узел, содержащий ***n***. Если узел не содержит ***n***, `delete_from_list` ничего не делает. В любом случае, функция возвращает указатель на список.

```
struct node *delete_from_list(struct node *list, int n){
    struct node *cur, *prev;
    for (cur = list, prev = NULL;
        cur != NULL && cur -> value != n;
        prev = cur, cur = cur -> next) ;
        ;
    if (cur == NULL)
        return list; /* n was not found */
    if (prev == NULL)
        list = list -> next; /* n is in the first node */
    else
        prev -> next = cur -> next; /* n is in some other node */
    free (cur);
    return list;
}
```

Удаление первого узла в списке - особый случай, который требует другого обходного шага. Для этого случая выполняется тест `prev == NULL`.

Упорядоченный список

Когда узлы списка упорядочены - отсортированы по данным, хранящимся внутри узлов - мы говорим, что список упорядочен.

Вставить узел в упорядоченный список сложнее (узел не всегда помещается в начало списка), но поиск быстрее (мы можем прекратить просмотр после достижения места, в котором расположен нужный узел).

Следующая программа иллюстрирует как повышенную сложность вставки узла, так и более быстрый поиск.

ПРОГРАММА Ведение базы данных деталей (пересмотренная)

Давайте переделаем программу базы данных деталей раздела 16.3. на этот раз база данных хранится в связанном списке.

Использование связанного списка вместо массива имеет два основных преимущества:

1) Нам не нужно устанавливать предварительный размер базы данных; он может расти, пока не исчерпается память для хранения деталей.

2) Мы можем легко сохранить базу данных, отсортированную по номеру детали - когда новая деталь добавляется в базу данных, мы просто вставляем её в нужное место в списке.

В оригинальной программе база данных не сортировалась. В новой программе структура `part` будет содержать дополнительный член (указатель на следующий узел в связанном списке), а переменная `inventory` будет указателем на первый узел в списке:

```
struct part {  
    int number;  
    char name(NAME_LEN+1);  
    int on_hand;  
    struct part *next;  
};
```

```
struct part *inventory = NULL; /* points to first part */
```

Большинство функций в новой программе будут очень похожи на свои аналоги в оригинальной программе. Однако функции `find_part` и `insert` будут более сложными, так как мы будем

сохранять узлы в списке `inventory` отсортированными по номеру детали.

В исходной программе `find_part` возвращает индекс массива `inventory`. В новой программе `find_part` вернёт указатель на узел, содержащий искомый номер детали.

Если номер детали не найден, `find_part` возвращает нулевой указатель.

Поскольку список `inventory` сортируется по номеру детали, новая версия `find_part` может сэкономить время, остановив поиск, когда она находит узел, содержащий номер детали, который больше или равен искомому номеру детали.

Цикл поиска `find_part` будет иметь вид:

```
for (p = inventory;  
p != NULL && number > p->number;  
p = p → next)
```

Цикл завершится, когда ***p*** станет `NULL` (что указывает на то, что номер детали не найден) или когда `number > p->number` будет `false` (что указывает на то, что номер детали, который мы ищем, меньше или равен номеру, уже сохранённому в узле).

В последнем случае мы все ещё не знаем, действительно ли нужное число находится в списке, поэтому нам понадобится ещё один тест:

```
if (p != NULL && number == p -> number)
    return p;
```

В исходной версии `insert` новая деталь хранится в следующем доступном элементе массива.

Новая версия должна определить место новой детали в списке и вставить её туда. У нас также будет проверка в `insert`, присутствует ли номер детали уже в списке, `insert` может выполнить обе задачи, используя цикл, аналогичный тому, который находится в `find_part`:

```
for (cur = inventory, prev = NULL;
     cur != NULL && new_node -> number > cur -> number;
     prev = cur, cur = cur -> next)
```

Этот цикл основан на двух указателях:

`cur`, который указывает на текущий узел и `prev`, который указывает на предыдущий узел. Когда цикл завершается, `insert` будет проверять, не является ли `cur` `NULL`, а `new_node->number` равно `cur->number`; если это так, номер детали уже находится в списке. В противном случае `insert` вставит новый

узел между узлами, на которые указывают prev и cur, используя стратегию, подобную той, которую мы использовали для удаления узла.

(Эта стратегия работает, даже если новый номер детали больше, чем любой в списке; в этом случае наш будет нулевым, но prev будет указывать на последний узел в списке.)

Вот новая программа. Как и оригинальная программа, эта версия требует функции read_line, описанной в разделе 16.3; я предполагаю, что readline.h содержит прототип этой функции.

```
//inventory2.c
/* Поддерживает базу данных деталей (версия со связным
списком) */
#include <stdio.h>
#include <stdlib.h>
#include "readline.h"

#define NAME_LEN 25

struct part {
    int number;
    char name[NAME_LEN+1];
    int on_hand;
    struct part *next;
};
```

```
struct part *inventory = NULL; /* points to first part */
```

```
struct part *find_part(int number);
```

```
void insert(void);
```

```
void search(void);
```

```
void update(void);
```

```
void print(void);
```

```
/******
```

Предлагает пользователю ввести код операции, затем вызывает функцию для выполнения запрошенного действия. Повторяется, пока пользователь не введет команду «q». Печатает сообщение об ошибке, если пользователь вводит недопустимый код.

```
*****/
```

```
int main(void){
```

```
    char code;
```

```
    for (;;) {
```

```
        printf("Enter operation code: ");
```

```
        scanf(" %c", &code);
```

```
        while (getchar() != '\n') /* skips to end of line */
```

```
            ;
```

```
        switch (code) {
```

```
        case 'i': insert();
            break;
        case 's': search();
            break;
        case 'u': update();
            break;
        case 'p': print();
            break;
        case 'q': return 0;
        default: printf("Illegal code\n");
    }
    printf("\n");
}

}

/*****

find_part: ищет номер детали в списке inventory. Возвращает
указатель на узел, содержащий номер детали; если номер
детали не найден, возвращает NULL.

*****/

struct part *find_part(int number) {
    struct part *p;
    for (p = inventory;
        p != NULL && number > p->number; p = p->next)
        ;
}
```

```

    if (p != NULL && number == p->number)
        return p;
    return NULL;
}

```

```

/*****

```

* insert: запрашивает у пользователя информацию о новой детали и затем вставляет деталь в список inventory; список остаётся отсортированным по номеру детали. Печатает сообщение об ошибке и возвращается досрочно, если деталь уже существует или для детали не удалось выделить место.

```

*****/

```

```

void insert(void){
    struct part *cur, *prev, *new_node;

    new_node = malloc(sizeof(struct part));
    if (new_node == NULL) {
        printf("Database is full; can't add more parts.\n");
        return;
    }

```

```

    printf("Enter part number: ");
    scanf("%d", &new_node->number);

```

```

    for (cur = inventory, prev = NULL;

```

```
cur != NULL && new_node->number > cur->number;
prev = cur, cur = cur->next)
    ;
if (cur != NULL && new_node->number == cur->number){
    printf("Part already exists.\n");
    free(new_node);
    return;
}

printf("Enter part name: ");
read_line(new_node->name, NAME_LEN);
printf("Enter quantity on hand: ");
scanf("%d", &new_node->on_hand);

new_node->next = cur;
if (prev == NULL)
    inventory = new_node;
else
    prev->next = new_node;
}
```

/******

search: Предлагает пользователю ввести номер детали, затем ищет деталь в базе данных. Если деталь существует, печатает

название и количество в наличии; если нет, выводит сообщение об ошибке.

```
*****/
void search(void){
    int number;
    struct part *p;

    printf("Enter part number: ");
    scanf("%d", &number);
    p = find_part(number);
    if (p != NULL) {
        printf("Part name: %s\n", p->name);
        printf("Quantity on hand: %d\n", p->on_hand);
    } else
        printf("Part not found.\n");
}

/*****
```

update: предлагает пользователю ввести номер детали.

Печатает сообщение об ошибке, если деталь не существует; в противном случае пользователю предлагается ввести изменение количества и обновить базу данных.

```
*****/
void update(void){
    int number, change;
```


981

```
struct part *p;

printf("Enter part number: ");
scanf("%d", &number);
p = find_part(number);
if (p != NULL) {
    printf("Enter change in quantity on hand: ");
    scanf("%d", &change);
    p->on_hand += change;
} else
    printf("Part not found.\n");
}
```

/******

print: распечатывает список всех деталей в базе данных с указанием номера детали, названия детали и количества в наличии. Номера деталей будут отображаться в порядке возрастания.

*****/

```
void print(void){
    struct part *p;

    printf("Part Number  Part Name  Quantity on Hand\n");
    for (p = inventory; p != NULL; p = p->next)
```

```
printf("%7d  %-25s%11d\n", p->number, p->name,  
p->on_hand);  
}
```

Обратите внимание на использование `free` в функции `insert`, `insert` выделяет память для детали перед проверкой, существует ли деталь. Если это так, `insert` освобождает пространство, чтобы избежать утечки памяти.

17.6 Указатели на указатели

В разделе 13.7 мы столкнулись с понятием указателя на указатель. В этом разделе мы использовали массив, элементы которого имели тип `char *`; указатель на один из элементов массива имел тип `char **`. Понятие "указатели на указатели" также часто всплывает в контексте связанных структур данных. В частности, когда аргумент функции является переменной-указателем, мы иногда хотим, чтобы функция могла изменять переменную, заставляя её указывать на другое место. Для этого необходимо использовать указатель на указатель. Рассмотрим функцию `add_to_list` раздела 17.5, которая вставляет узел в начало связанного списка. Когда мы вызываем `add_to_list`, мы передаём ему указатель на первый

983

узел в исходном списке; затем он возвращает указатель на первый узел в обновлённом списке:

```
struct node *add_to_list(struct node *list, int n)
{
    struct node *new_node;
    new_node = malloc(sizeof(struct node));

    if (new_node == NULL){
        printf("Error:malloc failed in add_to_list\n");
        exit (EXIT_FAILURE);
    }
    new_node->value = n;
    new_node->next = list;
    return new_node;
}
```

Предположим, что мы модифицируем функцию так, чтобы она присваивала `new_node` `list`-у вместо возврата `new_node`. Другими словами, давайте удалим инструкцию `return` из `add_to_list` и заменим её на `list = new_node`; К сожалению, эта идея не работает. Предположим, что мы вызываем `add_to_list` следующим образом:

```
add_to_list (first, 10);
```

В точке вызова `first` копируется в `list`. (Указатели, как и все аргументы передаются по значению.)

Последняя строка в функции изменяет значение `list` (*копии first*), указывающего теперь на новый узел. Однако это присваивание не влияет на `first`. Получить `add_to_list` изменяющую `first` возможно, но это требует передачи `add_to_list` указателя на `first`. Вот правильная версия функции:

```
void add_to_list(struct node **list, int n){
    struct node *new_node;
    new_node = malloc(sizeof(struct node));
    if (new_node == NULL) {
        printf("Error: malloc: failed in add_to_list\n");
        exit (EXIT_FAILURE);
    }
    new_node -> value = n;
    new_node -> next = *list;
    *list = new_node;
}
```

Когда мы называем новую версию `add_to_list`, первый аргумент должен быть адресом `first`

```
add_to_list (&first, 10);
```

Поскольку `list` присваивается адрес `first`, мы можем использовать `*list` в качестве псевдонима для `first`. В частности, присвоение `new_node *list` изменит `first`.

17.7 Указатели на функции

Мы видели, что указатели могут указывать на различные типы данных, включая переменные, элементы массивов и динамически выделенные блоки памяти. Но Си не требует, чтобы указатели указывали только на данные; возможно также иметь указатели на функции.

Указатели на функции не такие странные, как вы думаете. В конце концов, функции занимают места в памяти, так что каждая функция имеет адрес, так же, как каждая переменная имеет адрес.

Указатели на функции в качестве аргументов

Указатели на функции можно использовать почти так же, как указатели на данные. В частности, передача указателя на функцию в качестве аргумента, является достаточно распространённой в Си. Предположим, что мы пишем функцию с именем `integrate`, которая интегрирует

математическую функцию f между точками a и b . Мы хотим сделать `integrate` как можно более общей, передав ей f в качестве аргумента. Для достижения этого эффекта в Си мы объявим f как указатель на функцию

Предполагая, что мы хотим интегрировать функции, которые имеют параметр `double` и возвращают результат `double`, прототип для `integrate` будет выглядеть следующим образом:

```
double integrate(double (*f) (double), double a, double b);
```

Скобки вокруг `* f` указывают, что f является указателем на функцию, а не функцией, которая возвращает указатель. Также допустимо объявлять f , как если бы это была функция:

```
double integrate(double f(double), double a, double b);
```

С точки зрения компилятора этот прототип идентичен предыдущему. Когда мы вызываем `integrate`, мы будем указывать имя функции в качестве первого аргумента. Например, следующий вызов будет интегрировать функцию `sin` (синус) от 0 до $\pi / 2$:

```
result = integrate(sin, 0.0, PI/2);
```

Обратите внимание, что после `sin` нет скобок. Когда имя функции не сопровождается скобками, компилятор Си создаёт указатель на функцию вместо генерации кода для вызова функции.

В нашем примере мы не вызываем `sin`; вместо этого мы передаём указатель на `sin`. Если это кажется запутанным, подумайте о том, как Си обрабатывает массивы.

Если ***a*** - это имя массива, то `a[i]` представляет один элемент массива, ***a*** само по себе служит указателем на массив.

Аналогичным образом, если ***f*** является функцией, Си рассматривает `f (x)` как вызов функции, но ***f*** саму по себе как указатель на функцию. В теле `integrate` мы можем вызвать функцию, на которую указывает ***f***:

```
y = (*f) (x);
```

* `f` представляет функцию, на которую указывает ***f***; `x` - аргумент вызова. Таким образом, во время выполнения

```
integrate (sin, 0.0, PI / 2);
```

каждый вызов `* f` на самом деле является вызовом `sin`. Как альтернативу `(* f) (x)`, Си позволяет нам написать `f (x)`, чтобы вызвать функцию, на которую указывает ***f***. Хотя `f (x)` выглядит более естественно, я буду придерживаться записи `(*f) (x)` в качестве напоминания о том, что ***f*** является указателем на функцию, а не именем функции.

Функция `qsort`

Хотя может показаться, что указатели на функции не имеют отношения к программисту средней руки, это вовсе не так.

На самом деле, некоторые из наиболее полезных функций в библиотеке Си требуют указателя функции в качестве аргумента. Одной из них является `qsort`, которая принадлежит заголовку `<stdlib.h>`.

`qsort` - универсальная функция сортировки, способная сортировать любой массив на основе любых критериев, которые мы задаём. Поскольку элементы массива, которые она сортирует, могут быть любого типа - даже типа структуры или объединения - `qsort` необходимо указать, как определить, какой из двух элементов массива "меньше."

Мы предоставим эту информацию `qsort`, написав функцию сравнения. При задании двух указателей *p* и *q* на элементы массива функция сравнения должна возвращать целое число, которое является отрицательным, если **p* "меньше" **q*, ноль, если **p* "равно" **q*, и положительное, если **p* "больше, чем" **q*. Термины "меньше" "равно" и "больше" указаны в кавычках, потому что это наша обязанность определить, как сравниваются **p* и **q*.

`qsort` имеет следующий прототип:

```
void qsort(void *base, size_t nmemb, size_t size, int (*compar)
(const void *, const void *));
```

`base` должен указывать на первый элемент массива. (Если нужно отсортировать только часть массива, то `base` будет указывать на первый элемент в этой части.) В простейшем случае `base` - это просто имя массива.

`nmemb` - количество элементов для сортировки (не обязательно количество элементов в массиве).

`size` - размер каждого элемента массива, измеряемый в байтах.

`compar` - указатель на функцию сравнения. Когда вызывается `qsort`, она сортирует массив по возрастанию, вызывая функцию сравнения всякий раз, когда ей нужно сравнить элементы массива. Чтобы отсортировать массив `inventory` раздела 16.3, мы используем следующий вызов `qsort`:

```
qsort (inventory, num_parts, sizeof (struct part), compare_parts);
```

Обратите внимание, что вторым аргументом является `num_parts`, а не `MAX_PARTS`; мы хотим сортировать не весь массив `inventory`, а только часть, в которой хранятся детали. Последний аргумент, `compare_parts`, является функцией, сравнивающей две структуры `part`. Написать функцию `compare_parts` не так просто, как можно было бы ожидать. `qsort` требует, чтобы её параметры имели тип `void *`, но мы не можем получить доступ к членам структуры `part` через указатель `void *` вместо этого нам нужен указатель типа `struct part*`. Чтобы решить эту проблему, нам нужно чтобы `compare_parts` присваивала свои параметры, ***p*** и ***q***, переменным типа `struct part*`, тем самым преобразовав их в нужный тип, `compare_parts` теперь может использовать эти переменные для доступа к членам структур, на которые указывают ***p*** и ***q***. Предполагая, что мы хотим отсортировать

массив `inventory` в порядке возрастания по номеру детали, вот как может выглядеть функция `compare_parts`:

```
int compare_parts(const void *p, const void *q){  
    const struct part *p1 = p;  
    const struct part *q1 = q;  
    if (p1 -> number < q1 -> number);  
        return -1;  
    else if (p1 -> number == q1 -> number)  
        return 0;  
    else  
        return 1;  
}
```

Объявления `p1` и `q1` включают слово `const`, чтобы избежать предупреждения от компилятора. Поскольку ***p*** и ***q*** являются указателями `const` (указывающими, что объекты, на которые они указывают, не должны быть модифицированы), они должны быть присвоены только переменным-указателям, которые также объявлены как `const`.

Хотя эта версия `compare_parts` работает, большинство программистов на Си написали бы функцию более лаконично. Сначала заметим, что мы можем заменить `p1` и `q1` выражениями приведения:

```
int compare_parts(const void *p, const void *q){  
  
    if (((struct part *) p) -> number < ((struct part *) q) -> number)  
        return -1;  
    else if (((struct part *) p)->number ==((struct part *)q)->number)  
        return 0;  
    else  
        return 1;  
}
```

Круглые скобки ((struct part *)p) необходимы; без них компилятор попытался бы привести p->number к типу struct part*.

Мы можем сделать compare_parts ещё короче, удалив инструкцию if:

```
int compare_parts(const void *p, const void *q){  
    return ((struct part *) p)->number - ((struct part *) q)->number;  
}
```

Вычитание номера детали, на которую указывает ***q***, из номера детали, на которую указывает ***p***, даёт отрицательный результат, если ***p*** имеет меньший номер детали, ноль, если номера деталей равны, и положительный результат, если ***p*** имеет больший номер детали. (Обратите внимание, что вычитание двух целых чисел потенциально опасно из-за опасности переполнения. Я предполагаю, что номера деталей - это целые положительные числа, поэтому этого не должно происходить здесь.) Чтобы отсортировать массив `inventory` с помощью `part name` (именам деталей) вместо `part number` (номеров деталей), мы будем использовать следующую версию `compare_parts`:

```
int compare_parts(const void *p, const void *q){
return strcmp(((struct part *)p)->name,((struct part*) q)->name);
}
```

Все, что `compare_parts` должна сделать, это вызвать `strcmp`, которая без труда возвращает отрицательный, нулевой или положительный результат.

Другое использование указателей на функции

Хотя я подчеркнул полезность указателей на функции в качестве аргументов для других функций, это ещё не все, на

что они способны. Си обрабатывает указатели на функции так же, как указатели на данные; мы можем хранить указатели на функции в переменных или использовать их как элементы массива или как члены структуры или объединения. Мы даже можем написать функции, которые возвращают указатели на функции.

Вот пример переменной, которая может хранить указатель на функцию:

```
void (*pf)(int);
```

pf может указывать на любую функцию с параметром int и возвращаемым типом void. Если *f* - такая функция, то pf может указывать на *f* следующим образом:

```
pf = f;
```

Обратите внимание, что нет амперсанда, предшествующего *f*. Поскольку pf указывает на *f*, мы можем вызвать *f*, написав либо

```
(* pf) (i),
```

либо

```
pf (i);
```

Массивы, элементы которых являются указателями на функции, имеют удивительное количество приложений.

Например, предположим, что мы пишем программу, которая отображает меню команд для выбора пользователем. Мы можем написать функции, которые реализуют эти команды, а затем хранить указатели на функции в массиве:

```
void (*file_cmd[]) (void) =  
{new_cmd,  
open_cmd,  
close_cmd,  
close_all_cmd,  
save_cmd,  
save_as_cmd,  
save_all_cmd,  
print_cmd,  
exit_cmd};
```

Если пользователь выбирает команду *n*, где *n* находится в диапазоне от 0 до 8, мы можем по индексу массива `file_cmd` вызвать соответствующую функцию:

```
(*file_cmd[n]) (); /* or file_cmd[n](); */
```

Конечно, мы могли бы получить аналогичный эффект с помощью инструкции `switch`. Однако использование массива указателей на функции даёт нам большую гибкость, поскольку элементы массива могут быть изменены по мере выполнения программы.

ПРОГРАММА табуляция тригонометрических функций

Следующая программа печатает таблицы, показывающие значения функций `cos`, `sin` и `tan` (все три принадлежат `<math.h>` (*не забудьте в конец строки компиляции добавить опцию `-lm`*). Программа построена вокруг функции с именем `tabulate`, которая при передаче указателя на функцию ***f*** выводит таблицу, показывающую значения - ***f***.

```
//tabulate.c
```

```
/* Табулирует значения тригонометрических функций*/
```

```
#include <math.h>
```

```
#include <stdio.h>
```

```
void tabulate(double (*f)(double), double first,  
             double last, double incr);
```

```
int main(void){  
    double final, increment, initial;  
  
    printf("Enter initial value: ");  
    scanf("%lf", &initial);  
  
    printf("Enter final value: ");  
    scanf("%lf", &final);  
  
    printf("Enter increment: ");  
    scanf("%lf", &increment);  
  
    printf("\n  x   cos(x)"  
          "\n  -----  ----- \n");  
    tabulate(cos, initial, final, increment);
```


997

```
printf("\n x sin(x)"  
      "\n ----- \n");  
tabulate(sin, initial, final, increment);
```

```
printf("\n x tan(x)"  
      "\n ----- \n");  
tabulate(tan, initial, final, increment);
```

```
return 0;
```

```
}
```

```
void tabulate(double (*f)(double), double first,  
              double last, double incr)
```

```
{
```

```
double x;
```

```
int i, num_intervals;
```

```
num_intervals = ceil((last - first) / incr);
```

```
for (i = 0; i <= num_intervals; i++) {
```

```
    x = first + i * incr;
```

998

```
printf("%10.5f %10.5f\n", x, (*f)(x));  
}  
}
```

tabulate использует функцию `ceil`, которая также находится в `<math.h>`. Когда задан аргумент `x` типа `double`, `ceil` возвращает наименьшее целое число, которое больше или равно `x`. Вот как выглядит сеанс с `tabulate.c`:

Enter initial value: 0

Enter final value:.5

Enter increment:.1

x	cos(x)
---	--------

0.00000	1.00000
---------	---------

0.10000	0.99500
---------	---------

0.20000	0.98007
---------	---------

0.30000	0.95534
---------	---------

0.40000	0.92106
---------	---------

0.50000	0.87758
---------	---------

999

x sin(x)

0.00000 0.00000

0.10000 0.09983

0.20000 0.19867

0.30000 0.29552

0.40000 0.38942

0.50000 0.47943

x tan(x)

0.00000 0.00000

0.10000 0.10033

0.20000 0.20271

0.30000 0.30934

0.40000 0.42279

0.50000 0.54630

17.8 Ограниченные указатели (C99)

В этом и следующем разделах рассматриваются две функции, связанные с указателями в C99. Обе они в первую очередь представляют интерес для продвинутых программистов C: большинство читателей захотят пропустить эти разделы.

В C99 ключевое слово `restrict` может появиться в объявлении указателя:

```
int * restrict p;
```

Указатель, объявленный с помощью `restrict`, называется ограниченным указателем.

`restrict` решает следующую задачу: если ***p*** указывает на объект, который позже модифицируется, то к этому объекту обращаются только через ***p***. (Альтернативные способы получить доступ к объекту включают наличие другого указателя на тот же самый объект, или имея ***p***, который указывает на именованную переменную.) Наличие нескольких способов доступа к объекту часто называется использованием псевдонимов (*aliasing*). Давайте рассмотрим пример поведения, которому ограниченные указатели должны воспрепятствовать.

Предположим, что ***p*** и ***q*** были объявлены следующим образом:

1001

```
int * restrict p;
```

```
int * restrict q;
```

Теперь предположим, что ***p*** указывает на динамически выделяемый блок памяти:

```
p = malloc (sizeof (int));
```

Обычно было бы законно скопировать ***p*** в ***q***, а затем изменить целое число через ***q***:

```
q = p;
```

```
* q = 0; /* вызывает неопределённое поведение */
```

потому что ***p*** является ограниченным указателем и эффект выполнения инструкции `*q = 0;` неопределён.

Делая ***p*** и ***q*** указывающими на тот же самый объект, мы заставили `*p` и `*q` быть псевдонимами (алиасами). Если ограниченный указатель ***p*** объявлен как локальная переменная без класса хранения `extern`, ограничение применяется только к ***p***, когда выполняется блок, в котором ***p*** объявлен. (Обратите внимание, что тело функции является блоком.)

Ключевое слово `restrict` может использоваться с функциональными параметрами типа указателя и в этом случае `restrict` применяется только при выполнении функции.

Когда ограничение применяется к переменной указателя с областью видимости файла, ограничение длится для всего выполнения программы. Точные правила использования `restrict` довольно сложны; подробности см. в стандарте C99.

Есть даже ситуации, когда псевдоним, созданный с помощью ограниченного указателя, является допустимым. Например, ограниченный указатель ***p*** может быть законно скопирован в другую ограниченную указательную переменную ***q***, при условии, что ***p*** является локальным для функции, а ***q*** определяется внутри блока, вложенного в тело функции. Чтобы проиллюстрировать использование `restrict`, давайте посмотрим на `memcpy` и `memmove`, которые принадлежат заголовку `<string.h>`, `memcpy` имеет следующий прототип в C99:

```
void *memcpy (void * restrict s1, const void * restrict s2, size_t n);
```

`memcpy` похожа на `strcpy`, за исключением того, что она копирует байты из одного объекта в другой (`strcpy` копирует символы из одной строки в другую).

`s2` указывает на данные, подлежащие копированию, `s1` указывает на место назначения копии, а ***n*** - количество байтов, которые нужно скопировать.

Использование `restrict` с `s1` и `s2` указывает на то, что источник копии и место назначения не должны перекрываться.

(Однако это не гарантирует, что они не перекрываются.)

Напротив, `restrict` не появляется в прототипе `memmove`:

1003

```
void *memmove(void s1, const void *s2, size_t n);
```

memmove делает то же самое, что memcpy: она копирует байты из одного места в другое.

Разница в том, что memmove гарантированно работает, даже если источник и место назначения перекрываются. Например, мы могли бы использовать memmove для смещения элементов массива на одну позицию:

```
int a [100];
```

```
...
```

```
memmove(&a[0],&a[1], 99 * sizeof(int));
```

До C99 не было возможности документировать разницу между memcpy и memmove. Прототипы для двух функций были почти одинаковыми:

```
void *memcpy(void *s1, const void *s2, size_t n);
```

```
void *memmove (void *s1, const void s2, size_t n );
```

Использование restrict в версии C99 прототипа memcpy позволяет программисту знать, что s1 и s2 должны указывать на объекты, которые не перекрываются, иначе функция не будет работать. Хотя использование restrict в прототипах функций является полезной фиксацией, это не основная

причина его существования, `restrict` предоставляет компилятору информацию, которая может позволить ему создавать более эффективный код - процесс, известный как оптимизация. ((Класс хранения `register` служит той же цели. Однако не каждый компилятор пытается оптимизировать программы, а те, которые делают это, обычно позволяют программисту отключить оптимизацию.

В результате стандарт C99 гарантирует, что `restrict` не влияет на поведение программы, которая соответствует стандарту: если все использования `restrict` удалить из такой программы, она должна вести себя точно также. Большинство программистов не используют `restrict`, если они не настраивают программу для достижения наилучшей производительности. Тем не менее, стоит знать о `restrict`, потому что оно появляется в прототипах C99 для ряда стандартных библиотечных функций.

19.9 Элементы гибкого массива (C99)

Время от времени нам нужно определить структуру, содержащую массив неизвестного размера.

Например, возможно нам придётся сохранить строки в форме, отличной от обычной. Обычно строка представляет собой массив символов с нулевым символом, обозначающим конец.

1005

Однако есть преимущества для хранения строк другими способами. Одним из вариантов является сохранение длины строки вместе с символами строки (но без нулевого символа). Длина и символы могут храниться в такой структуре, как эта:

```
struct vstring {  
    int len;  
    char chars [N];  
}
```

Здесь N - макрос, представляющий максимальную длину строки. Однако использование массива фиксированной длины, такого как этот, нежелательно, поскольку это вынуждает нас ограничивать длину строки, а также тратить память (поскольку большинству строк не нужны все N символов в массиве). Программисты на Си традиционно решали эту проблему, объявляя длину символов равной 1 (фиктивное значение), а затем динамически выделяя каждую строку:

```
struct vstring {  
    int len;  
    char chars [1];  
};  
  
...
```

1006

```
struct vstring *str = malloc(sizeof(struct vstring) + n - 1);  
str->len = n;
```

Мы "обманиваем", выделяя больше памяти, чем объявленная структура (в данном случае, дополнительные $n - 1$ символ), а затем используем память для хранения дополнительных элементов массива `chars`. Этот метод стал настолько распространенным на протяжении многих лет, что он имеет название: "struct hack." Взлом структуры не ограничивается массивами символов; он имеет множество применений. Со временем он стал достаточно популярным, чтобы его поддерживали многие компиляторы. Некоторые (включая GCC) даже позволяют массиву символов иметь нулевую длину, что делает этот трюк немного более явным. К сожалению, стандарт C89 не гарантирует, что struct hack будет работать, и не допускает массивов нулевой длины. В знак признания полезности struct hack, C99 имеет особенность, известную как гибкий член массива, который служит той же цели. Если последний член структуры является массивом, его длина может быть опущена:

```
struct vstring {  
    int len;  
    char chars[]; /* flexible array member — C99 only */  
}
```

1007

Длина массива `chars` не определяется, пока не будет выделена память для структуры `vstring`, обычно с помощью вызова `malloc`:

```
struct vstring *str = malloc(sizeof(struct vstring)+ n);
```

```
str->len = n;
```

В этом примере `str` указывает на структуру `vstring`, в которой массив символов занимает ***n*** символов. Оператор `sizeof` игнорирует член `chars` при вычислении размера структуры. (Гибкий элемент массива необычен тем, что он не занимает места в структуре.) К структуре, содержащей гибкий элемент массива, применяется несколько специальных правил. Гибкий элемент массива должен отображаться последним в структуре, и структура должна иметь по крайней мере один другой член.

Копирование структуры, содержащей гибкий элемент массива, приведёт к копированию других элементов, но не самого гибкого массива. Структура, содержащая гибкий элемент массива, является неполным типом. Неполный тип - это отсутствие части информации, необходимой для определения объёма памяти. Неполные типы, которые обсуждаются далее в одном из вопросов в конце этой главы и в разделе 19.3, подвержены различным ограничениям.

1008

В частности, неполный тип (и, следовательно, структура, содержащая гибкий член массива) не может быть членом другой структуры или элементом массива. Однако, массив может содержать указатели на структуры, которые имеют гибкий элемент массива; проект 7 в конце этой главы построен вокруг такого массива.

Вопросы и ответы

Вопрос: Что представляет макрос NULL?

Ответ: NULL на самом деле означает (*целое число*) 0. Когда мы используем 0 в контексте, где требуется указатель, компиляторы Си рассматривают его как нулевой указатель вместо целого числа 0. Макрос NULL предоставляется только для того, чтобы избежать путаницы. Присваивание

`p = 0;`

может присваивать значение 0 числовой переменной или присваивать нулевой указатель переменной указателя; мы не видим, что именно присваивается. Напротив, присваивание

`p = NULL;`

даёт понять, что *p* является указателем.

1009

***Вопрос:** В заголовочных файлах, поставляемых с моим компилятором, значение NULL определяется следующим образом:

```
#define NULL (void *) 0
```

В чем преимущество приведения 0 к void *?

Ответ: Этот приём, разрешенный стандартом Си, позволяет компиляторам выявлять некорректное использование нулевого указателя. Например, предположим, что мы пытаемся присвоить NULL целочисленной переменной:

```
i=NULL;
```

если NULL определяется как 0, то это присваивание совершенно законно.

Но если NULL определяется как (void *) 0, компилятор может сообщить нам, что мы присваиваем указатель целочисленной переменной. Определение NULL как (void *) 0 имеет второе, более важное, преимущество. Предположим, что мы вызываем функцию со списком аргументов переменной длины и передаём NULL в качестве одного из аргументов. Если NULL определяется как 0, компилятор неправильно передаст нулевое целое значение. (В обычном вызове функции NULL работает нормально, поскольку компилятор знает из прототипа функции, что он ожидает указатель. Однако если функция имеет список аргументов переменной длины, компилятору не хватает этих знаний.) Если NULL

1010

определён как (void *) 0, компилятор передаст нулевой указатель. Чтобы сделать ещё более запутанным, некоторые заголовочные файлы определяют NULL как 0L (long версия 0). Это определение, как и определение NULL как 0, является пережитком предыдущих лет Си, когда указатели и целые числа были совместимы.

Для большинства целей, однако, это действительно не имеет значения, как определяется NULL; просто думайте об этом как имени для нулевого указателя.

Вопрос: Поскольку 0 используется для представления нулевого указателя, я предполагаю, что нулевой указатель - это всего лишь адрес со всеми нулевыми битами, правильно?

Ответ: Необязательно. Каждый компилятор C может представлять нулевые указатели по-разному, и не все компиляторы используют нулевой адрес.

Например, некоторые компиляторы используют несуществующий адрес памяти для нулевого указателя; таким образом, попытка доступа к памяти через нулевой указатель может быть обнаружена оборудованием. То, как нулевой указатель хранится внутри компьютера, не должно нас беспокоить; это деталь, о которой должны беспокоиться эксперты компилятора. Важно то, что при использовании в контексте указателя, 0 преобразуется компилятором в соответствующую внутреннюю форму.

Вопрос: Допустимо ли использовать NULL в качестве нулевого символа ?

Ответ: Определённо нет. NULL - это макрос, представляющий нулевой указатель, а не нулевой символ. Использование NULL в качестве нулевого символа будет работать с некоторыми компиляторами, но не со всеми (так как некоторые определяют NULL как (void *) 0). В любом случае, использование нулевого символа в качестве указателя может привести к большой путанице.

Если требуется имя для нулевого символа, определите следующий макрос:

```
#define NUL '\0'
```

***Вопрос:** Когда моя программа завершается, я получаю сообщение "Null pointer assignment". Что это означает?

Ответ: Это сообщение, которое генерируется программами, скомпилированными с некоторыми более старыми компиляторами Си на основе DOS, указывает, что программа хранила данные в памяти, используя неверный указатель (но не обязательно нулевой указатель).

К сожалению, сообщение не отображается до тех пор, пока программа не завершится, поэтому нет никакой подсказки о том, какая инструкция вызвала ошибку. Сообщение

1012

"Присвоение нулевого указателя" может быть вызвано отсутствующим & в scanf:

```
scanf("%d",i); /*должно было быть scanf("%d", &i); */
```

Другой возможностью является присваивание, включающее неинициализированный или нулевой указатель:

```
*p = i; /* p is uninitialized or null */
```

***Вопрос:** Как программа узнает, что произошло "Присваивание нулевого указателя"?

Ответ: Сообщение зависит от того, что в моделях малой и средней памяти данные хранятся в одном сегменте с адресами, начинающимися с 0. Компилятор оставляет "дыру" в начале сегмента данных—небольшой блок памяти, который инициализируется 0, но в остальном не используется программой. Когда программа завершает работу, она проверяет, не являются ли какие-либо данные в области "дыры" ненулевыми. Если это так, то он (блок) был изменён через плохой указатель.

Вопрос: Есть ли преимущество в приведении возвращаемого значения malloc или других функций выделения памяти?

Ответ: Обычно нет. Приведение указателя void *, возвращаемого этими функциями, не требуется, так как

указатели типа `void *` автоматически преобразуются в любой тип указателя при присваивании.

Привычка приведения возвращаемого значения является пережитком из более старых версий Си, в котором функции выделения памяти возвращали значение `char *`, что делало приведение необходимым. Программы, разработанные для компиляции в виде кода C++, могут извлечь выгоду из приведения, но это единственная причина для этого. В C89 есть на самом деле небольшое преимущество при не выполнении приведения. Предположим, что мы забыли включать `<stdlib.h>` заголовок в нашу программу. При вызове `malloc` компилятор будет считать, что его возвращаемый тип `int` (возвращаемое значение по умолчанию для любой функции Си). Если мы не приведём возвращаемое значение `malloc`, компилятор C89 выдаст ошибку (или, по крайней мере, предупреждение), так как мы пытаемся присвоить целочисленное значение переменной указателя. С другой стороны, если мы приведём возвращаемое значение к указателю, программа может скомпилироваться, но, скорее всего, не будет работать должным образом. Начиная с C99 это не так. Не включение заголовка `<stdlib.h>` вызовет ошибку при вызове `malloc`, поскольку C99 требует, чтобы функция была объявлена перед вызовом.

1014

Вопрос: Функция `callos` инициализирует блок памяти, устанавливая его биты в нуль. Означает ли это, что все элементы данных в блоке становятся нулевыми?

Ответ: Обычно, но не всегда.

Установка целого числа в нулевые биты всегда делает целое число нулевым. Установка числа с плавающей запятой в нулевые биты обычно делает число нулевым, но это не гарантируется - это зависит от того, как хранятся числа с плавающей запятой. История такая же как для указателей; указатель, чьи биты равны нулю, не обязательно нулевой указатель.

***Вопрос:** Я вижу, как механизм тега структуры позволяет структуре содержать указатель на себя. Но что, если у каждой из двух структур есть член, указывающий на другую?

Ответ: Вот как мы справимся с этой ситуацией:

```
struct s1; /* incomplete declaration of s1 */  
  
struct s2 {  
    ...  
    struct s1 *p;  
    ...  
};
```

1015

```
struct s1 {  
    ...  
    struct s2 *q;  
    ...  
}
```

Первое объявление `s1` создаёт неполный тип структуры, так как мы не указали члены `s1`. Второе объявление `s1` "завершает" тип, описывая члены структуры. Неполные объявления типа структуры разрешены в Си, хотя их использование ограничено. Одним из таких применений является создание указателя на такой тип (как мы делали при объявлении *p*).

Вопрос: Вызов `malloc` с неправильным аргументом - в результате чего он выделяет слишком много памяти или слишком мало памяти—кажется распространённой ошибкой. Есть ли более безопасный способ использовать `malloc`?

Ответ: Да, есть. Некоторые программисты используют следующую идиому при вызове `malloc` для выделения памяти для одного объекта:

идиома `p = malloc (sizeof (*p) );`

Поскольку `sizeof (*p)` - это размер объекта, на который указывает *p*, этот оператор гарантирует, что будет выделен

1016

правильный объем памяти. На первый взгляд эта идиома выглядит подозрительно: вероятно, что **p** неинициализирован, что делает значение *p неопределённым. Однако sizeof не вычисляет *p, он просто вычисляет его размер, поэтому идиома работает, даже если **p** неинициализирован или содержит нулевой указатель. Чтобы выделить память для массива с **n** элементами, можно использовать слегка изменённую версию идиомы:

```
p = malloc (n * sizeof (*p));
```

Вопрос: Почему функция qsort не называется просто sort?

Ответ: Название qsort происходит от алгоритма Quicksort, опубликованного Ч. Э. Р. Хоаром в 1962 году (и обсуждаемого в разделе 9.6). По иронии судьбы, стандарт Си не требует, чтобы qsort использовал алгоритм Quicksort, хотя многие версии qsort это делают.

Вопрос: Не нужно ли приводить первый аргумент qsort к типу void *, как в следующем примере?

```
qsorts((void *) inventory, num_parts, sizeof(struct  
part),compare_parts);
```

Ответ: Нет. Указатель любого типа может быть автоматически преобразован в void *.

***Вопрос:** Я хочу использовать qsort для сортировки массива целых чисел, но у меня возникли проблемы с написанием функции сравнения. В чем заключается секрет?

Ответ: Вот версия, которая работает:

```
int compare_ints(const void *p, const void *q){  
    return *(int*)p - *(int *)q;  
}
```

Необычно, не правда ли? Выражение (int *) p приводит **p** к типу int *, поэтому * (int *) p будет целым числом, на которое указывает **p**.

***Вопрос:** Мне нужно было отсортировать массив строк, поэтому я решил, что просто использую strcmp в качестве функции сравнения. Однако, когда я передал её qsort, компилятор выдал мне предупреждение.

Я попытался решить проблему, вставив strcmp в функцию сравнения:

```
int compare_strings(const void *p, const void *q){  
    return strcmp(p, q);  
}
```

Теперь моя программа компилируется, но qsort не сортирует массив. Что я делаю неправильно?

Ответ: Во-первых, вы не можете передать `strcmp` в `qsort`, так как для `qsort` требуется функция сравнения с двумя параметрами `const void*`. Функция `compare_strings` не работает, так как неправильно предполагает, что ***p*** и ***q*** являются строками (указателями `char*`).

Фактически, ***p*** и ***q*** указывают на элементы массива, содержащие `char *` указатели.

Чтобы исправить `compare_strings`, мы приведем ***p*** и ***q*** к типу `**char`, а затем используем оператор `*` для удаления одного уровня косвенного обращения:

```
int compare_strings(const void *p, const void *q){  
    return strcmp(*(char **)p, *(char **)q);  
}
```

Упражнения

К разделу 17.1

1.Необходимость проверять возвращаемое значение `malloc` (или любой другой функции выделения памяти) каждый раз, когда мы вызываем её, раздражает. Напишите функцию с именем `my_malloc`, которая служит "оболочкой" для `malloc`. Когда мы вызываем `my_malloc` и просим выделить ***n*** байт. она

1019

вызывает `malloc`, проверяет, что `malloc` не возвращает нулевой указатель, а затем возвращает указатель из `malloc`.

`my_malloc` должна вывести сообщение об ошибке и завершить работу программы, если `malloc` вернёт нулевой указатель.

К разделу 17.2

W2. Напишите функцию `duplicate`, которая использует динамическое выделение памяти для создания копии строки. Например, вызов

```
p = duplicate (str)
```

выделяет место для строки той же длины, что и `str`, копирует содержимое `str` в новую строку и возвращает указатель на нее. `duplicate` возвращает нулевой указатель, если выделение памяти завершается неудачей.

К разделу 17.2

3. Напишите следующую функцию:

```
int *create_array(int n, int initial_value);
```

Функция должна возвращать указатель на динамически выделенный массив `int` с `n` членами, каждый из которых инициализируется `initial_value`. Возвращаемое значение

1020

должно быть NULL, если массив не может быть выделен.
Предположим, что действуют следующие объявления:

```
struct point { int x, y; };  
struct rectangle { struct point upper_left, lower_right; };  
struct rectangle *p;
```

Предположим, что мы хотим, чтобы *p* указывала на структуру rectangle, верхний левый угол которой находится в (10, 25), а нижний правый угол - в (20, 15). Напишите серию инструкций, которые выделяют такую структуру и инициализируют её, как указано.

W5. Предположим, что *f* и *p* объявлены следующим образом:

```
struct{  
    union {  
        char a, b;  
        int c;  
    }d;  
    int e[5];  
} f, *p = &f;
```


1021

Какие из следующих инструкций являются допустимыми?

(a) `p->b = ' ';`

(b) `p->e[3] = 10;`

(c) `(*p).d.a = '*';`

(d) `p->d->c = 20;`

6. Измените функцию `delete_from_list` так, чтобы она использовала только одну переменную указателя вместо двух (`cur` и `prev`).

W7. Следующий цикл должен удалить все узлы из списка и освободить память, которую они занимают. К сожалению, цикл неверен. Объясните, что с ним не так и покажите, как исправить ошибку.

```
for (p = first; p != NULL; p = p->next)
    free (p);
```

8. Раздел 15.2 описывает файл, `stack.c`, который предоставляет функции для хранения целых чисел в стёке, в этом разделе стёк был реализован как массив. Изменить `stack.c` таким образом, чтобы стёк теперь хранился в виде связанного списка. Замените переменные `contents` и `top` одной переменной, указывающей на первый узел в списке ("верх")

1022

стёка). Запишите функции в `stack.c` тем, чтобы они использовали этот указатель. Удалите функцию `is_full`, вместо этого `push` будет возвращать либо `true` (если память будет доступна для создания узла), либо `false` (если нет).

9. `True` или `false`: если `x` является структурой, а `a` - членом этой структуры, то `(&x) -> a` совпадает с `x.a`. Обоснуйте свой ответ.

10. Измените функцию `print_part` раздела 16.2 так, чтобы её параметр указывал на структуру `part`. В ответе используйте оператор `->`.

11. Напишите следующую функцию:

```
int count_occurrences (struct node *list, int n);
```

Параметр `list` указывает на связанный список; функция должна возвращать количество раз, которое `n` появляется в этом списке. Предположим, что структура `node` определена в разделе 17.5.

12. Напишите следующую функцию:

```
struct node *find_last(struct node *list, int n);
```

1023

Параметр `list` указывает на связанный список. Функция должна вернуть указатель на последний узел, содержащий `n`, она должна вернуть `NULL`, если `n` не отображается в списке. Предположим, что структура `node` определена в разделе 17.5.

13. Следующая функция должна вставить новый узел в его правильное место в упорядоченном списке, возвращая указатель на первый узел в списке. К сожалению, функция работает не во всех случаях. Объясните, что с ней не так, и покажите, как это исправить. Предположим, что структура `node` определена в разделе 17.5.

```
struct node *insert_into_ordered_list(struct node *list, struct node
*new_node)
{
    struct node *cur = list, *prev = NULL;
    while (cur->value <= new_node->value)
    {
        prev= cur;
        cur = cur->next;
    }
    prev->next= new_node;
    new_node->next = cur;
```

1024

```
    return list;  
}
```

14. Измените функцию `delete_from_list` (раздел 17.5) так, чтобы её первый параметр имел тип `struct node **` (указатель на указатель на первый узел в списке) и возвращаемый тип `void`.

`delete_from_list` должна изменить свой первый аргумент, чтобы он указывал на список после удаления нужного узла.

W15. Покажите вывод следующей программы и объясните, что она делает.

```
#include <stdio.h>  
  
int f1(int (*f) (int));  
int f2(int i);  
  
int main (void){  
    printf("Answer %d\n",f1(f2));  
    return 0;  
}
```

```
int f1(int (*f) (int))
```

1025

```
{  
    int n = 0;  
    while ((*f) (n)) n++;  
    return n;  
}
```

```
int f2 (int i)  
{  
    return i * i + i -12;  
}
```

16. Напишите следующую функцию. Вызов `sum(g, i, j)` должен вернуть $g(i)+\dots+g(j)$.

```
int sum(int (*f) (int), int start,int end);
```

W17. Пусть ***a*** - массив из 100 целых чисел. Напишите вызов сортировки, который сортирует только последние 50 элементов в ***a***. (Вам не нужно писать функцию сравнения).

18. Измените функцию `compare_parts` таким образом, чтобы детали сортировались по номерам в порядке убывания.

19. Напишите функцию, которая при задании строки в качестве аргумента ищет в следующей структуре соответствующее имя команды, а затем вызывает функцию, связанную с этим именем.

```
struct {  
    char *cmd_name;  
    void (*cmd_pointer)(void);  
} file_cmd[] =  
{ {"new",    new_cmd}  
  { "open ",  open_cmd}  
  {"close",   clcse_cmd}  
  {"close all", close_all_cmd}  
  { " save ",  save_cmd}  
  {" save as", save_as_cmd}  
  { "save all", savew_all_cmd}  
  {"print",   print_cmd}  
  {"exit",    exit_cmd}  
};
```

Проекты по программированию

W1. Модифицируйте `inventory.c`, программу раздела 16.3 так, чтобы массив `inventory` выделялся динамически и позже перераспределялся, когда он заполнится. Используйте `malloc` изначально, чтобы выделить достаточно места для массива из 10 структур `part`. Если в массиве больше нет места для новых деталей, используйте `realloc`, чтобы удвоить его размер.

Повторяйте шаг удвоения каждый раз, когда массив заполняется.

W2. Измените программу `inventory.c` раздела 16.3, чтобы команда `p` (`print`) вызывала `qsort` для сортировки массива `inventory` до того, как она распечатает детали.

3. Измените программу `inventory2.c` раздела 17.5, добавив команду `e` (`erase`), которая позволяет пользователю удалять детали из базы данных.

4. Модифицируйте программу `justify` раздела 15.3, переписав файл `line.c`, чтобы он сохранял текущую строку в связанном списке. Каждый узел в списке будет хранить одно слово. Массив строки будет заменён переменной, указывающей на узел, содержащий первое слово. Эта переменная будет хранить нулевой указатель всякий раз, когда строка пуста.

5. Запишите программу, которая сортирует ряд слов, введённых пользователем:

1028

Enter word: foo

Enter word: bar

Enter word: baz

Enter word: quux

Enter word:

In sorted order: bar baz foo quux

Предположим, что каждое слово длиной не более 20 символов. Прекратите чтение, когда пользователь вводит пустое слово (в, нажимает Enter, не вводя слово). Храните каждое слово в динамически выделенной строке, используя массив указателей, чтобы отслеживать строки, как в программе remind.c (раздел 17.2). После того, как все слова будут прочитаны, отсортируйте массив (используя любую технику сортировки), а затем используйте цикл для печати слов в отсортированном порядке. Подсказка: используйте функцию `read_line` для чтения каждого слова, как в remind.2c.

6. Измените Проект по программированию 5 так, чтобы он использовал `qsort` для сортировки массива указателей.

7. (C99) Измените программу remind2.c раздела 17.2, чтобы каждый элемент массива `reminders` был указателем на структуру `vstring` (см. раздел 17.9), а не указателем на обычную строку.

18 Объявления (декларации)

Легко сделать что-то переменным. Хитрость в том, чтобы измерять продолжительность постоянства.

Объявления играют центральную роль в программировании на Си. Объявляя переменные и функции, мы предоставляем важную информацию, необходимую компилятору для проверки программы на наличие потенциальных ошибок и преобразования её в объектный код. В предыдущих главах приводились примеры объявлений, не вдаваясь в подробности; эта глава заполняет пробелы. Она исследует сложные параметры, которые можно использовать в объявлениях и показывает, что объявления переменных и объявления функций имеют довольно много общего. Она также обеспечивает твёрдую основу в важных концепциях продолжительности хранения, области видимости и компоновки (linkage). В разделе 18.1 рассматривается синтаксис деклараций в их наиболее общем виде, чего мы до сих пор избегали. Следующие четыре раздела посвящены элементам, которые отображаются в объявлениях: классы хранения (раздел 18.2), квалификаторы типа(раздел 18.3), деклараторы (раздел 18.4) и инициализаторы (раздел 18.5). В

раздел 18.6 обсуждается ключевое слово `inline`, которое может использоваться в объявлениях функций C99.

18.1 Синтаксис объявления

Объявления предоставляют компилятору сведения о смысле идентификаторов. Когда мы пишем

`int i;`

мы сообщаем компилятору, что в текущей области имя *i* представляет переменную типа `int`. Объявление

`float f (float);`

сообщает компилятору, что *f*-это функция, возвращающая значение `float` и имеющая один аргумент, также типа `float`. В общем случае декларация имеет следующий вид:

спецификаторы-объявления деклараторы;

Спецификаторы объявления описывают свойства объявляемых переменных или функций. **Деклараторы** дают свои имена и могут предоставить дополнительную информацию об их свойствах. Спецификаторы объявления делятся на три категории:

■ **Классы хранения.** Существует четыре класса хранения: auto, static, extern и register. В объявлении может содержаться не более одного класса хранения; если он присутствует, он должен быть первым.

■ **Квалификаторы типов.** В C89 существует только два квалификатора типа: const и volatile. C99 имеет третий квалификатор типа, restrict. Объявление может содержать ноль или более квалификаторов типов.

■ **Спецификаторы типов.** Ключевые слова void, char, short, int, long, float, double, signed и unsigned являются спецификаторами типов. Эти слова могут быть объединены, как описано в главе 7; порядок, в котором они появляются, не имеет значения (int unsigned long совпадает с long unsigned int). Спецификаторы типа также включают спецификации структур, объединений и перечислений (например, struct point { int x,y; }, struct {int x,y;} или struct point). Имена типов, созданные с помощью typedef, также являются спецификаторами типов.

(C99 имеет четвёртый вид спецификатора объявления, спецификатор функции, который используется только в объявлениях функций. В этой категории есть только один член, ключевое слово inline.) Квалификаторы типов и спецификаторы типов должны следовать за классом хранения, но нет других ограничения на их порядок.

1032

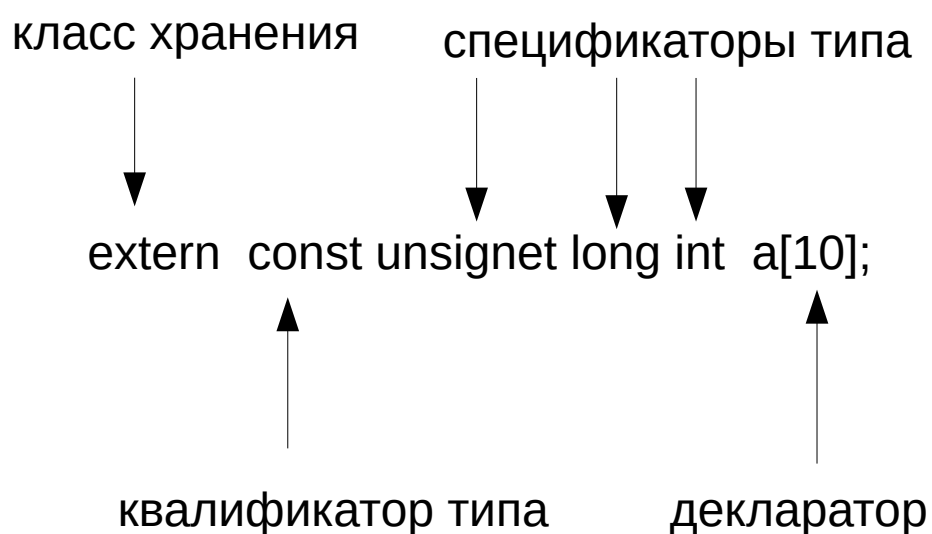
С точки зрения стиля стиля я поместил бы квалификаторы типов перед спецификаторами типов. Деклараторы включают идентификаторы (имена простых переменных), идентификаторы, а затем [] (имена массивов). идентификаторы, которым предшествуют * (имена указателей), и идентификаторы, за которыми следуют () (имена функций). Деклараторы разделяются запятыми. За декларатором, представляющим переменную, может следовать инициализатор. Давайте рассмотрим несколько примеров, иллюстрирующих эти правила. Вот объявление с классом хранения и тремя деклараторами:

класс хранения деклараторы
 ↓ ↓ ↓ ↓
static float x, y, *p;
 ↑
 спецификатор типа

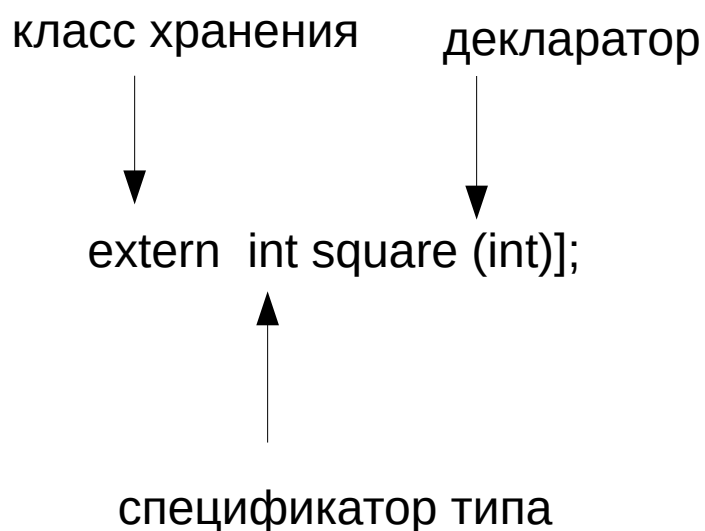
Следующее объявление имеет квалификатор типа, но не класс хранения. Оно также имеет инициализатор:

квалификатор типа декларатор инициализатор
 ↓ ↓ ↓
const char month [] = "January";
 ↑
 спецификатор типа

Следующее объявление содержит как класс хранения, так и квалификатор типа. Он также имеет три спецификатора типа; их порядок не важен:



Объявления функций, также как как объявления переменных, могут иметь класс хранения, квалификаторы типов и спецификаторы типов. Следующее объявление содержит класс хранения и спецификатор типа:



Следующие четыре раздела охватывают классы хранения, квалификаторы типов, деклараторы и инициализаторы.

18.2 Классы хранения

Классы хранения могут быть указаны для переменных и - в меньшей степени - для функций и параметров. Сейчас мы сосредоточимся на переменных. Напомним из раздела 10.3, что термин блок относится к телу функции (части,

заключённой в фигурные скобки) или составной инструкции, возможно содержащий объявления. В С99 инструкции выбора (if и switch) и инструкции итерации (while, do и for) - наряду с "внутренними" инструкции, которыми они управляют, - также считаются блоками, хотя это в основном формальность.

Свойства переменных

Каждая переменная в программе С имеет три свойства:

■ ***Длительность хранения.*** Длительность хранения переменной определяет, когда память резервируется для переменной и когда эта память освобождается.

Хранилище для переменной с автоматической продолжительностью хранения выделяется при выполнении окружающего блока; хранилище освобождается при завершении блока. в результате чего переменная теряет своё значение. Переменная со статической продолжительностью хранения остаётся в том же месте хранения, пока работает программа, что позволяет ей неограниченно сохранять своё значение.

■ **Область видимости.** Область видимости переменной - это часть текста программы, в которой на переменную можно ссылаться. Переменная может либо иметь область видимости блок (переменная видима от точки объявления до конца заключающего её блока) или область видимости файл (переменная видима от точки объявления до конца заключающего её файла).

■ **Связывание.** Связывание переменной определяет степень, в которой она может совместно использоваться различными частями программы. Переменная с внешним связыванием может совместно использоваться несколькими (возможно, всеми) файлами в программе. Переменная с внутренним связыванием ограничена одним файлом, но может совместно использоваться функциями в этом файле. (Если переменная с тем же именем появляется в другом файле, она рассматривается как другая переменная.) Переменная без связывания принадлежит одной функции и не может использоваться совместно.

Длительность хранения по умолчанию, область видимости и связывание переменной зависят от того, где она объявлена:

■ Переменные, объявленные внутри блока (включая тело функции), имеют автоматическую длительность хранения, область видимости блока и не имеют связывания.

■ Переменные, объявленные вне любого блока на самом внешнем уровне программы, имеют статическую

1037

продолжительность хранения, область видимости файла и внешнее связывание.

В следующем примере показаны свойства переменных i и j по умолчанию:

- int i;
- статическая длительность хранения
- файловая область видимости
- внешнее связывание

```
void f(void){
```

- intj;
 - автоматическая длительность хранения
 - блоковая область видимости
 - нет связывания

}

Для многих переменных продолжительность хранения, область видимости и связывание по умолчанию являются адекватными. Если это не так, мы можем изменить эти свойства, указав явный класс хранения: `auto`, `static`, `extern` или `register`.

Класс хранения auto (автоматический)

Класс хранения `auto` допустим только для переменных, принадлежащих блоку. Переменная `auto` имеет автоматическую продолжительность хранения (что неудивительно), блоковую область видимости и отсутствие связывания. Класс хранения `auto` почти никогда не указывается явно, так как он используется по умолчанию для переменных, объявленных внутри блока.

Статический класс хранения (`static`)

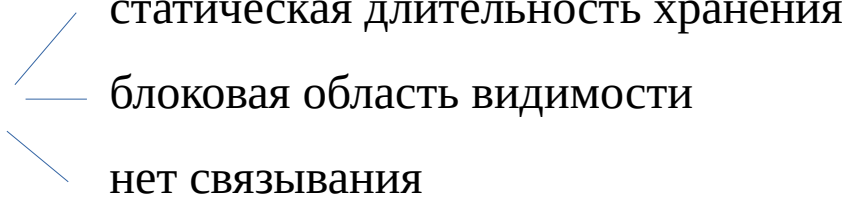
Статический класс хранения может использоваться со всеми переменными, независимо от того, где они объявлены, но он имеет другое влияние на переменную, объявленную вне блока, чем на переменную, объявленную внутри блока. При использовании вне блока слово `static` указывает, что переменная имеет внутреннее связывание. При использовании внутри блока `static` изменяет длительность хранения переменной с `automatic` на `static`. На следующем рисунке показан эффект объявления *i* и *j* статическими:

<code>static int i;</code>		статическая длительность хранения
		файловая область видимости
		внешнее связывание

1039

```
void f (void){
```

```
    static int i;
```



- статическая длительность хранения
- блоковая область видимости
- нет связывания

```
}
```

При использовании объявления вне блока `static` скрывает переменную в файле, в котором она объявлена: переменную могут видеть только функции, находящиеся в том же файле. В следующем примере функции `f1` и `f2` имеют доступ к `i`, но функции в других файлах - нет:

```
static int i;
```

```
void f1 (void){
```

```
/* has access to i */
```

```
}
```

```
void f2 (void){
```

```
/* has access to i */
```

```
}
```

Такое использование `static` может помочь реализовать метод, известный как сокрытие информации.

Статическая переменная, объявленная в блоке, находится в том же месте хранения во время выполнения программы. В отличие от автоматических переменных, которые теряют свои значения каждый раз, когда программа покидает блок, статическая переменная будет сохранять своё значение бесконечно долго, статические переменные имеют некоторые интересные свойства:

- статическая переменная в блоке инициализируется только один раз перед выполнением программы. Автоматическая переменная инициализируется каждый раз, когда она начинает существование (при условии, конечно, что у неё есть инициализатор).

- каждый раз, когда функция вызывается рекурсивно, она получает новый набор автоматических переменных. Если у неё есть статическая переменная, с другой стороны, эта переменная разделяется всеми вызовами функции.

- Не смотря на то, что функция не должна возвращать указатель на автоматическую переменную, нет ничего плохого в том, что она возвращает указатель на статическую переменную. Объявление одной из переменных статической позволяет функции сохранять информацию между вызовами в "скрытой" области, к которой остальная часть программы не

1041

может получить доступ. Однако чаще мы будем использовать `static` для повышения эффективности программ.

Рассмотрим следующую функцию:

```
char digit_to_hex_char(int digit){  
    const char hex_chars[16] = "0123456789ABCDEF";  
    return hex_chars [digit];  
}
```

Каждый раз, когда вызывается функция `digit_to_hex_char`, символы `0123456789ABCDEF` копируются в массив `hex_chars`, чтобы инициализировать его. Теперь сделаем массив статичным:

```
char digit_to_hexchar (int digit){  
    static const char hex_chars[16] = "0123456789ABCDEF";  
    return hex_chars [digit];  
}
```

Поскольку статические переменные инициализируются только один раз, мы повысили скорость `digit_to_hex_char`.

Класс хранения extern

Класс хранения extern позволяет нескольким исходным файлам совместно использовать одну и ту же переменную.

Раздел 15.2 охватывал основы использования extern, поэтому я не буду посвящать ему здесь много места. Напомним, что декларация

```
extern int i;
```

информирует компилятор о том, что *i* является переменной int, но не заставляет его выделять память для *i*. В

терминологии Си эта декларация не является определением *i*; она просто сообщает компилятору, что нам нужен доступ к переменной, которая определена в другом месте (возможно, позже в этом же файле, или - чаще всего - в другом файле).

Переменная может иметь много объявлений в программе, но должна иметь только одно определение. Есть одно исключение из правила, что объявление extern переменной не является определением. Объявление extern, инициализирующее переменную, служит определением переменной. Например, объявление

```
extern int i = 0;
```

фактически совпадает с

```
int i = 0;
```

1043

Это правило предотвращает инициализацию переменной несколькими объявлениями `extern` различными способами. Переменная в объявлении `extern` всегда имеет статическую продолжительность хранения. Область видимости переменной зависит от размещения объявлений. Если объявление находится внутри блока, переменная имеет область видимости блока; в противном случае - область видимости файла:

extern int i; статическая длительность хранения
 — файловая область видимости
 — ?связывание

```
void f(void)
```

$$\{$$

extern int i; статическая длительность хранения
 — блоковая область видимости
 — ?связывание

$$\}$$

Определение связывания переменной `extern` немного сложнее. Если переменная была объявлена статической ранее в файле (вне определения функции), то она имеет внутреннее связывание. В противном случае (обычный случай) переменная имеет внешнее связывание.

Класс хранения `register`

Использование класса хранения `register` в объявлении переменной требует от компилятора хранить переменную в регистре, а не хранить её в основной памяти, как другие переменные. (Регистр - это область хранения, расположенная в ЦП компьютера. Данные, хранящиеся в регистре, могут быть доступны и обновлены быстрее, чем данные, хранящиеся в обычной памяти.) Указание класса хранения переменной `register` является запросом, а не командой. Компилятор, если он выбирает, может свободно хранить переменную `register` в памяти. Класс хранения `register` допустим только для переменных, объявленных в блоке. Переменная `register` имеет ту же продолжительность хранения, область и связывание, что и переменная `auto`. Однако переменной `register` не хватает одного свойства, которое есть у переменной `auto`: так как регистры не имеют адресов, недопустимо использовать оператор `&` для

1045

получения адреса переменной регистра. Это ограничение применяется, даже если компилятор решил сохранить переменную в памяти, register лучше всего использовать для переменных, к которым часто обращаются и/или которые часто обновляются. Например, переменная управления циклом в инструкции for является хорошим кандидатом для обращения к register:

```
int sum_array(int a[], int n){  
    register int i;  
    int sum = 0;  
    for (i = 0; i < n; i++)  
        sum += a[i];  
    return sum;  
}
```

register не так популярен среди Си -программистов, как когда-то. Современные компиляторы гораздо более сложны, чем ранние Си-компиляторы; многие могут автоматически определить, какие переменные выиграют больше всего от хранения в регистрах. Тем не менее, использование register предоставляет полезную информацию, которая может помочь компилятору оптимизировать производительность программы. В частности, компилятор знает, что переменная

register не может принимать свой адрес и поэтому не может быть изменена с помощью указателя. В этом отношении ключевое слово register связано с ключевым словом restrict в C99.

Класс хранения функций

Объявления функций (и определения), как и объявления переменных, может включать в себя класс хранения, но только варианты extern и static. Слово extern в начале объявления функции указывает, что функция имеет внешнее связывание, позволяющее ей вызываться из других файлов, static указывает на внутреннее связывание, ограничивая использование имени функции файлом, в котором она определена. Если класс хранения не указан, предполагается, что функция имеет внешнее связывание. Рассмотрим следующие объявления функций:

```
extern int f(int i);
```

```
static int g(int i);
```

```
int h(int i);
```

f имеет внешнее связывание, ***g*** имеет внутреннее связывание, а ***h*** (по умолчанию) имеет внешнее связывание. Поскольку ***g*** имеет внутреннее связывание, её нельзя вызывать

непосредственно из-за пределов файла, в котором она определена.

(Объявление **g** статической не полностью предотвращает её вызов в другом файле; косвенный вызов через указатель функции все ещё возможен.) Объявление функций как `extern` подобно объявлению переменных как `auto` - это не имеет смысла. По этой причине я не использую `extern` в объявлениях функций. Однако имейте в виду, что некоторые программисты широко используют `extern`, что, безусловно, не наносит вреда. С другой стороны, объявление функций статическими весьма полезно. На самом деле я рекомендую использовать `static` при объявлении любой функции, которая не предназначена для вызова из других файлов. Преимущества этого включают:

■ **Простота обслуживания.** Объявление функции **f** статической гарантирует, что функция **f** не видна вне файла, в котором находится её определение. В результате, некто, проводя позднее модификацию программы, будет знать, что изменения в **f** не повлияют на функции в других файлах. (Одно исключение: функция в другом файле, которой передаётся указатель на **f**, могла бы быть затронута изменениями в **f**. К счастью, эту ситуацию просто определить, исследуя файл, в котором определена **f**, так как функция, которой передаётся указатель на **f**, **должна также быть определена там.**)

■ Уменьшенное "загрязнение пространства имён." Поскольку функции, объявленные статическими, имеют внутреннее связывание, их имена могут быть повторно использованы в других файлах. Хотя мы, вероятно, не будем намеренно использовать имя функции для какой-либо другой цели, в больших программах этого может быть трудно избежать. Чрезмерное количество имён с внешним связыванием может привести к тому, что программисты на Си называют "загрязнением пространства имён": имена в разных файлах случайно конфликтуют друг с другом. Использование `static` помогает предотвратить эту проблему.

Параметры функции имеют те же свойства, что и автоматические переменные: длительность автоматического хранения, блоковую область видимости и отсутствие связывания. Единственный класс хранения, который может быть указан для параметров - `register`.

Резюме

Теперь, когда мы рассмотрели различные классы хранения, давайте подведём итог тому, что мы знаем.

1049

В следующем фрагменте программы показаны все возможные способы включения или не включения классов хранения в объявлениях переменных и параметров.

```
int a;  
extern int b;  
static int c;  
  
void (int d, register int e){  
    auto int g;  
    int h;  
    static int i;  
    extern int j;  
    register int k;  
}
```

В таблице 18.1 показаны свойства каждой переменной и параметра в этом примере.

Имя	Длительность хранения	Область видимости	Связывание
a	статическая	файловая	внешнее
b	статическая	файловая	†
c	статическая	файловая	внутреннее
d	автоматическая	блоковая	нет
e	автоматическая	блоковая	нет
g	автоматическая	блоковая	нет
h	автоматическая	блоковая	нет
i	статическая	блоковая	нет
j	статическая	блоковая	†
k	автоматическая	блоковая	нет

† Определения b и j не показаны, потому что невозможно определить связывание этих переменных. В большинстве случаев, переменные будут определены в другом файле и с внешним связыванием.

Из четырёх классов хранения наиболее важными являются static и extern, auto ни как не влияет, а современные компиляторы сделали register менее важным.

18.3 Квалификаторы типов

Существует два квалификатора типа: `const` и `volatile`. (C99 имеет третий квалификатор типа, `restrict`, который используется только с указателями.) Поскольку использование `volatile` ограничено низкоуровневым программированием, я отложу его обсуждение до раздела 20.3.

`const` используется для объявления объектов, похожих на переменные, но доступных только для чтения: программа может получить доступ к значению объекта `const`, но не может его изменить.

Например, объявление

```
const int n = 10;
```

создаёт объект `const` с именем ***n***, значение которого равно 10.

Объявление

```
const int tax_brackets[] = {750, 2250, 3750, 5250, 7000};
```

создаёт массив `const` с именем `tax_brackets`.

Объявление объекта с квалификатором `const` имеет ряд преимуществ:

Это форма документирования: она предупреждает всех, кто читает программу, о том, что объект доступен только для чтения.

■ Компилятор может проверить, что программа случайно пытается изменить значение объекта.

■ Когда программы пишутся для определённых типов приложений (в частности, для встраиваемых систем), компилятор может использовать слово `const` для идентификации данных, которые будут храниться в ПЗУ (только для чтения).

На первый взгляд может показаться, что `const` выполняет ту же роль, что и директива `#define`, которую мы использовали в предыдущих главах для создания имён констант. Однако между `#define` и `const` существуют значительные различия:

■ Мы можем использовать `#define` для создания имени числовой, символьной или строковой константы. `const` может использоваться для создания объектов только для чтения любого типа, включая массивы, указатели, структуры и объединения. ■

■ Объекты `const` подчиняются тем же правилам области видимости, что и переменные; константы, созданные с помощью `#define`, - нет. В частности, мы не можем

1053

использовать `#define` для создания константы с областью видимости блока.

■ Значение объекта `const`, в отличие от значения макроса, можно просмотреть в отладчике.

■ В отличие от макросов, объекты `const` нельзя использовать в константных выражениях. Например, мы не можем писать

```
const int n = 10;
```

```
int a[n]; /** WRONG ***/
```

так как границы массива должны быть константными выражениями. (В C99 этот пример будет допустимым, если **a** имеет автоматическую длительность хранения—он будет рассматриваться как массив переменной длины - но не допустим, если он имеет статическую длительность хранения.)

■ законно применять оператор адреса (`&`) к объекту `const`, так как он имеет адрес. У макроса нет адреса.

Нет никаких абсолютных правил, которые диктуют, когда использовать `#define` и когда использовать `const`. Я рекомендую использовать `#define` для констант, представляющих числа или символы. Таким образом, вы

сможете использовать константы в качестве размеров массивов, в инструкциях `switch` и в других местах, где требуются константные выражения.

18.4 Деклараторы

Декларатор состоит из идентификатора (имени объявляемой переменной или функции), которому, возможно, предшествует символ `*` или за которым следует `[]` или `()`. Комбинируя `*`, `[]` и `()`, мы можем создавать деклараторы ошеломляющей сложности. Прежде чем мы рассмотрим более сложные деклараторы, давайте рассмотрим деклараторы, которые мы видели в предыдущих главах. В простейшем случае Декларатор - это просто идентификатор, как показано в следующем примере:

```
int i;
```

Деклараторы также могут содержать символы `*`, `[]`, и `()`

■ Декларатор, начинающийся с `*`, представляет указатель:

```
int *p;
```

■ Декларатор, заканчивающийся символом `[]`, представляет массив:

```
int a [10];
```

1055

Скобки можно оставить пустыми, если массив является параметром, если он имеет инициализатор или если его класс хранения `extern`:

```
extern int a[];
```

Поскольку ***a*** определяется в другом месте программы, компилятору не нужно знать его длину здесь. (В случае многомерного массива пустым может быть только первый набор скобок.) C99 предоставляет две дополнительные опции для того, что находится между скобками в объявлении параметра массива. Одним из вариантов является ключевое слово `static`, за которым следует выражение, указывающее минимальную длину массива. Другой - символ `*`, который может использоваться в прототипе функции для указания аргумента массива переменной длины. Раздел 9.3 обсуждает обе опции C99.

■ Декларатор, заканчивающийся на `()`, представляет функцию:

```
int abs (int i);
```

```
void swap(int *a, int *h);
```

```
int find_largest(int a[], int n);
```

C позволяет опустить имена параметров в объявлении функции:

1056

```
int abs(int);  
void swap(int*, int *);  
int find_largest(int[], int);
```

Скобки можно даже оставить пустыми:

```
int abs();  
void swap();  
int find_largest();
```

Объявления в последней группе определяют типы возвращаемых значений функций `abs`, `swap` и `find_largest`, но не предоставляют сведений об их аргументах. Оставить скобки пустыми - это не то же самое, что поместить между ними слово `void`, которое указывает на отсутствие аргументов. Стиль объявления функции с пустыми скобками в значительной степени исчез. Он уступает стилю прототипа, представленному в C89, так как не позволяет компилятору проверить, имеют ли вызовы функций правильные аргументы.

Если бы все деклараторы были такими же простыми, как эти, программирование на Си было бы несложным. К сожалению, деклараторы в реальных программах часто объединяют обозначения `*`, `[]` и `()`.

Примеры таких комбинаций мы уже видели. Известно, что

1057

```
int * ap [10];
```

объявляет массив из 10 указателей на целые. Мы знаем, что `float *fp (float);` объявляет функцию, которая имеет аргумент `float` и возвращает указатель на `float`. И, в разделе 17.7 мы узнали, что

```
void (*pf) (int);
```

объявляет указатель на функцию с аргументом `int` и возвращаемым типом `void`.

Расшифровка сложных объявлений

До сих пор у нас не было особых проблем с пониманием деклараторов. Но как насчёт деклараторов, подобных приведённому в следующей декларации?

```
int * (*x [10]) (void) ;
```

Этот декларатор объединяет `*`, `[]` и `()`, поэтому не очевидно, является ли `x` указателем массива или функции.

К счастью, есть два простых правила, которые позволят нам понять любую декларацию, какой бы запутанной она ни была:

1058

■ Всегда читайте декларацию изнутри наружу. Иначе говоря, найдите объявленный идентификатор и начните расшифровку объявления оттуда.

В ситуации выбора, всегда в приоритете будут [] и () по отношению к *. Если * предшествует идентификатору и [] следует за ним, идентификатор представляет массив, а не указатель. Аналогично, если * предшествует идентификатору и () следует за ним, идентификатор представляет функцию, а не указатель. (Конечно, мы всегда можем использовать скобки, чтобы переопределить обычный приоритет [] и () над *.) Давайте сначала применим эти правила к нашим простым примерам в объявлении

```
int*ap [10];
```

идентификатором является ар. Поскольку * предшествует ар и [] следует за ним, мы отдаём предпочтение [], поэтому ар является массивом указателей.

В декларации

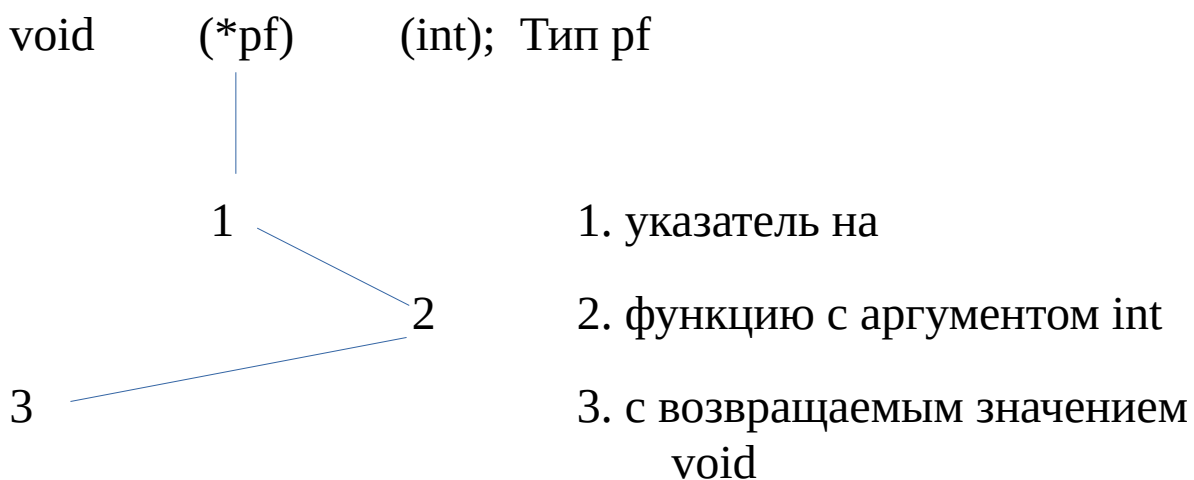
```
float * fp(float);
```

идентификатор - fp. Так как * предшествует fp и () следует за ним, мы отдаём предпочтение (), поэтому fp - это функция, которая возвращает указатель. Декларация

```
void (*pf) (int);
```

1059

немного сложнее. Поскольку *pf заключён в круглые скобки, pf должен быть указателем. Но за (*pf) следует (int), поэтому pf должен указывать на функцию с аргументом int. Слово void представляет возвращаемый тип этой функции. Как показывает последний пример, понимание сложного декларатора часто включает движение зигзагом с одной стороны идентификатора на другую:

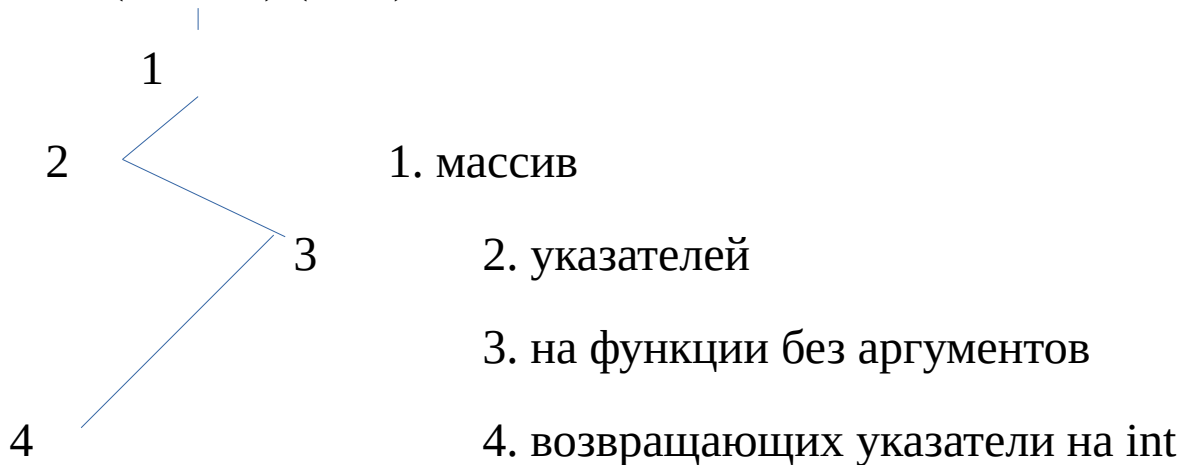


Давайте используем этот метод движения зигзагом, чтобы расшифровать объявление, данное ранее:

```
int * (*x[10]) (void);
```

1060

```
int * (* x[10]) (void);
```



Освоение объявления Си требует времени и практики.

Единственная хорошая новость заключается в том, что есть определённые вещи, которые не могут быть объявлены в Си.

Функции не могут возвращать массивы:

```
int f(int) []; /*** WRONG По правилу зигзага: функция  
принимающая int и возвращающая массив из int ***/
```

Функции не могут возвращать функции:

```
int g(int) (int); /*** WRONG По правилу зигзага: функция  
принимающая int и возвращающая функцию от int  
возвращающую int ***/
```

Массивы функций также невозможны:

```
int a[10] (int); /*** WRONG ***/
```


(Пропробуем

\$ cat p.c

extern int f(int) [];

extern int a[10] (int);

extern int g(int) (int);

int main(void) {

;

return 0;

}

\$ gcc -Wall -Wextra -Werror -o p p.c

p.c:2:13: error: ‘f’ declared as function returning an array

extern int f(int) [];

^

p.c:3:12: error: declaration of ‘a’ as array of functions

extern int a[10] (int);

^

p.c:4:12: error: ‘g’ declared as function returning a function

extern int g(int) (int);

^

)

В каждом случае мы можем использовать указатели, чтобы получить желаемый эффект. Функция не может возвращать массив, но может возвращать указатель на массив. Функция не может возвращать функцию, но может возвращать указатель на функцию.

Массивы функций не допускаются, но массив может содержать указатели на функции. (В разделе 17.7 приведён пример такого массива)

Использование определений типов для упрощения объявлений

Некоторые программисты используют определения типов для упрощения сложных объявлений. Рассмотрим объявление *x*, которое мы рассмотрели ранее в этом разделе:

```
int>(*x[10])(void);
```

Чтобы упростить понимание типа *x*, можно использовать следующую серию определений типов:

```
typedef int *Fcn(void);
```

```
typedef Fcn *Fcn_ptr;
```

```
typedef Fcn_ptr Fcn_ptr_array[10];  
Fcn_ptr_array x;
```

Если мы прочитаем эти строки в обратном порядке, мы увидим, что `x` имеет тип `Fcn_ptr_array`, `Fcn_ptr_array` - это массив значений `Fcn_ptr`, а `Fcn_ptr` - это указатель на тип `Fcn_ptr`, а `Fcn_ptr` - это функция, которая не имеет аргументов и возвращает указатель на значение `int`.

18.5 Инициализаторы

Для удобства Си позволяет задавать начальные значения переменных по мере их объявления. Чтобы инициализировать переменную, мы пишем символ `=` после её декларатора, а затем следует инициализатор. (Не путайте символ `=` в объявлении с оператором присваивания; инициализация - это не то же самое, что присваивание.) Мы видели различные виды инициализаторов в предыдущих главах. Инициализатор простой переменной - это выражение того же типа, что и переменная:

```
int i = 5 / 2; /* i изначально 2 */
```

Если типы не совпадают, Си преобразует инициализатор, используя те же правила, что и для присваивания:

```
int j = 5.5; /* преобразовано в 5 */
```

1064

Инициализатор переменной указателя должен быть выражением указателя того же типа, что и переменная или типа void

```
int *p = &i;
```

Инициализатор массива, структуры или объединения обычно представляет собой последовательность значений, заключённых в фигурные скобки:

```
int a[5] = {1, 2, 3, 4,5};
```

В C99 инициализаторы с фигурной скобкой могут иметь другие формы благодаря назначенным инициализаторам. Чтобы завершить наше освещение объявлений, давайте взглянем на некоторые дополнительные правила, которые управляют инициализаторами:

■ Инициализатор для переменной со статической продолжительностью хранения должен быть константным выражением:

```
#define FIRST 1
```

```
#define LAST 100
```

```
static int i = LAST - FIRST + 1;
```

1065

Поскольку LAST и FIRST являются макросами, компилятор может вычислить начальное значение i ($100 - 1 + 1 = 100$).

Если бы LAST и FIRST были переменными, инициализатор был бы недопустим.

■ Если переменная имеет автоматическую продолжительность хранения, её инициализатор не обязательно должен быть константой:

```
int f(int n){  
    int last = n -1;  
}
```

■ Инициализатор с фигурными скобками для массива, структуры или объединения должен содержать только константные выражения, но ни в коем случае, переменные или вызовы функций:

```
#define N 2  
  
int powers[5] = {1, N, N*N, N*N*N, N*N*N*N};
```

Поскольку N является константой, инициализатор для powers является законным; если бы N была переменной, программа

не компилировалась бы, в C99 это ограничение применяется только в том случае, если переменная имеет статическую продолжительность хранения.

■ Инициализатором автоматической структуры или объединения может быть другая структура или объединение:

```
void g(struct part part1){  
    struct part part2 = part1;  
  
    ...  
}
```

Инициализатор не обязательно должен быть именем переменной или параметра, хотя он должен быть выражением соответствующего типа. Например, инициализатором `part2` может быть `*p`, где *p* имеет тип `struct part *`, или `f (part1)`, где *f* - функция, возвращающая структуру `part`.

Неинициализированная переменная

В предыдущих главах, мы подразумевали, что неинициализированные переменные имеют неопределённые

значения. Это не всегда верно; начальное значение переменной зависит от длительности её хранения :

■ Переменные с автоматической продолжительностью хранения не имеют начального значения по умолчанию. Начальное значение автоматической переменной не может быть предсказано и может различаться при каждом появлении переменной.

■ Переменные со статической длительностью хранения по умолчанию имеют нулевое значение. В отличие от памяти, выделяемой `callos`, которая просто устанавливается в ноль бит, статическая переменная правильно инициализируется на основе её типа: целочисленные переменные инициализируются в 0, плавающие переменные инициализируются в 0.0, а переменные указателя содержат нулевой указатель. С точки зрения стиля лучше предоставить инициализаторы для статических переменных, чем полагаться на то, что они гарантированно равны нулю. Если программа обращается к переменной, которая не была инициализирована явно, кто-то, читающий программу позже, не сможет легко определить, предполагается ли, что переменная равна нулю или инициализирована присваиванием где-то в программе.

18.6 Функции inline (C99)

Объявления функций C99 имеют дополнительную опцию, которая отсутствует в C89: они могут содержать ключевое слово `inline`. Это ключевое слово является новым типом спецификатора объявления, отличным от классов хранения, квалификаторов типов и спецификаторов типов. Чтобы понять влияние `inline`, нам нужно визуализировать машинные инструкции, которые генерируются компилятором Си для обработки процесса вызова функции и возврата из функции.

На уровне компьютера может потребоваться выполнение нескольких инструкций для подготовки к вызову, сам вызов требует перехода к первой инструкции в функции, а также могут существовать дополнительные инструкции, выполняемые самой функцией, которая приступает к выполнению. Если у функции есть аргументы, их нужно скопировать (поскольку Си передаёт аргументы по значению). Возврат из функции требует аналогичного количества усилий как со стороны вызванной функции, так и со стороны вызвавшей её. Совокупная работа, необходимая для вызова функции и последующего возврата из неё, часто называется "накладными расходами", поскольку это дополнительная работа сверх того, что действительно должна выполнять функция. Хотя накладные расходы на вызов функции замедляют работу программы суммарно всего на крошечную

величину, в определённых ситуациях она может складываться, например, когда функция вызывается миллионы или миллиарды раз, когда используется более старый медленный процессор (как может быть в случае со встроенной системой) или когда программа должна соответствовать очень строгим срокам (как в системе реального времени). В C89 единственным способом избежать накладных расходов при вызове функции является использование параметризованного макроса.

Однако параметризованные макросы имеют определённые недостатки. C99 предлагает лучшее решение этой проблемы: создайте встроенную (inline) функцию. Слово "inline" предполагает стратегию имплементации, в которой компилятор заменяет каждый вызов функции машинными инструкциями для функции.

Данная методика позволяет избежать обычного вызова функции, хотя это может привести к незначительному росту размера скомпилированной программы.

Однако объявление функции inline фактически не заставляет компилятор "встроить" функцию. Оно просто предполагает, что компилятор должен попытаться сделать вызовы функции как можно быстрее, возможно, путём выполнения встроенного расширения, когда функция вызывается компилятор может и проигнорировать это предложение. В этом отношении inline похоже на ключевые слова register и

1070

restriction, которые компилятор может использовать для повышения производительности программы, но также может и проигнорировать.

Определения inline

Встроенная функция имеет ключевое слово inline в качестве одного из её спецификаций объявления:

```
inline double average (double a, double b){  
  
    return (a + b) / 2;  
  
}
```

Здесь ситуация немного усложняется, average имеет внешнее связывание, поэтому другие исходные файлы могут содержать вызовы average. Однако определение average не рассматривается компилятором как внешнее определение (вместо этого оно является встроенным определением), поэтому попытка вызвать average из другого файла будет считаться ошибкой. Существует два способа избежать этой ошибки. Одним из вариантов является добавление слова static в определение функции:

1071

```
static inline double average(double a, double b){  
    return (a + b) / 2;  
}
```

Теперь `average` имеет внутреннее связывание, поэтому её нельзя вызывать из других файлов. Другие файлы могут содержать свои собственные определения `average`, которые могут быть такими же, как это определение, или могут отличаться.

Другой вариант - предоставить внешнее определение для `average`, чтобы вызовы разрешались из других файлов.

Один из способов сделать это - записать функцию `average` во второй раз (без использования `inline`) и поместить второе определение в другой исходный файл. Это допустимо, но не лучшая идея иметь две версии одной и той же функции, потому что мы не можем гарантировать, что они останутся неизменными, когда программа будет изменена. Вот лучший подход. Во-первых, мы поместим встроенное определение `average` в файл заголовка (назовём его `average.h`)

```
#ifndef AVERAGE_H  
#define AVERAGE_H  
inline double average(double a, double b){  
    return (a + b) / 2;  
}
```

```
#endif
```

Далее мы создадим соответствующий исходный файл, average.c:

```
#include "average.h"
```

```
extern double average(double a, double b);
```

Теперь в любой файл, который должен вызывать функцию average, нужно просто включить заголовок average.h, в котором содержится встроенное определение average. Файл average.c содержит прототип для average, в котором используется ключевое слово extern, что приводит к тому, что определение average, включённое в average.h, рассматривается как внешнее определение в average.c.

Общее правило в C99 состоит в том, что если все объявления верхнего уровня функции в определённом файле включают inline, но не extern, то определение функции в этом файле является inline. Если функция используется где-либо в программе (в том числе и в файле, содержащем её встроенное определение), то внешнее определение функции нужно будет ему предоставить из какого-то другого файла. При вызове функции компилятор может выполнить обычный вызов (используя внешнее определение функции) или встроенное расширение (используя встроенное определение функции). Невозможно определить, какой выбор сделает компилятор, поэтому очень важно, чтобы оба определения были согласованными. Техника, которую мы только что обсуждали

(используя файлы `average.h` и `average.c`) гарантирует, что определения совпадают.

Ограничения на встроенные функции

Поскольку встроенные функции реализуются совершенно иначе, чем обычные функции, на них распространяются другие правила и ограничения. Переменные со статической продолжительностью хранения представляют собой особую проблему для встроенных функций с внешним связыванием. Следовательно, C99 налагает следующие ограничения на встроенную функцию с внешним связыванием (но не на функцию с внутренним связыванием):

- ➔ Функция не может определять модифицируемую статическую переменную
- ➔ Функция не может содержать ссылки на переменные с внутренним связыванием.

Такая функция может определять переменную как `static`, так и `const`, но каждое встроенное определение функции может создавать собственную копию переменной.

Использование встроенных функций с GCC

Некоторые компиляторы, включая GCC, поддерживали встроенные функции до стандарта C99. В результате их правила использования встроенных функций могут отличаться от стандартных. В частности, схема, описанная ранее (с использованием файлов `average.h` и `average.c`) могут не работать с этими компиляторами. Ожидается, что версия 4.3 GCC (недоступная на момент написания этой книги) будет поддерживать встроенные функции так, как описано в стандарте C99. Функции, указанные как статические, так и встроенные, должны работать нормально, независимо от версии GCC. Эта стратегия легальна и в C99, так что это самая безопасная ставка. Статическая встроенная функция может использоваться в одном файле или помещаться в файл заголовка и включаться в любой исходный файл, который должен вызывать функцию. Есть ещё один способ поделиться встроенной функцией с несколькими файлами, которая работает с более старыми версиями GCC, но конфликтует с C99. Этот метод включает размещение определения функции в файле заголовка, указание того, что функция является как внешней, так и встроенной, а затем включение файла заголовка в любой исходный файл, содержащий вызов функции.

Вторая копия определения—без слов `extern` и `inline` - помещается в один из исходных файлов. (Таким образом, если компилятор не может встроить функцию по какой-либо причине, он все равно будет иметь определение.) Последнее

замечание о GCC: функции "встроены" только в том случае, если оптимизация запрашивается через параметр командной строки -O.

Вопросы и ответы

Вопрос: Почему инструкции выбора и итерации (и их "внутренние" инструкции) считаются блоками в C99?

Ответ: Это довольно удивительное правило связано с проблемой, которая может возникнуть, когда составные литералы используются в инструкциях выбора и инструкциях итерации. Проблема связана с продолжительностью хранения составных литералов, поэтому давайте сначала обсудим эту проблему. Стандарт C99 утверждает, что объект, представленный составным литералом, имеет статическую продолжительность хранения, если составной литерал находится вне тела функции. В противном случае, он имеет автоматическую длительность хранения; в результате память, занимаемая объектом, освобождается в конце блока, в котором появляется составной литерал. Рассмотрим следующую функцию, которая возвращает структуру point, созданную с помощью составного литерала:

```
struct point create_point(int x, int y){  
    return (struct point) {x, y};
```

1076

```
}
```

Эта функция работает правильно, потому что объект, созданный составным литералом, будет скопирован при возврате функции. Исходный объект больше не будет существовать, но копия останется. Предположим теперь, что мы немного изменим функцию:

```
struct point *create_point(int x, int y){  
    return &(struct point) {x, y};  
}
```

Эта версия `create _point` имеет неопределённое поведение, поскольку она возвращает указатель на объект, который имеет автоматическую продолжительность хранения и не будет существовать после возврата функции.

Теперь вернёмся к вопросу, с которого мы начали: почему инструкции выбора и итерации считаются блоками?

Рассмотрим следующий пример:

```
/* Пример 1- инструкция if без фигурных скобок */  
double *coefficients,value;  
if (polynomial_selected == 1)  
    coefficients = (double[3]) {1.5, -3.0, 6.0};  
else  
    coefficients = (double[3]) {4.5, 1.0, -3.5};
```


1077

```
value = evaluate_polynomial(coefficients);
```

Этот фрагмент программы, по-видимому, ведёт себя желаемым образом (но читайте дальше), `coefficients` укажет на один из двух объектов, созданных составными литералами, и этот объект все ещё будет существовать в момент вызова `evaluate_polynomial`. Теперь рассмотрим, что произойдёт, если мы поместим фигурные скобки вокруг "внутренних" инструкций - тех, которые контролируются выражением `if`:

```
/* Пример 2 - else инструкция с фигурными скобками */
```

```
double *coefficients, value;
```

```
if (polynomial_selected == 1) {  
    coefficients = (double[3]) {1.5, -3.0, 6.0};  
}else  
{  
    coefficients = (double[3]) {4.5, 1.0, -3.5};  
}  
value = evaluate_polynomial(coefficients);
```

Теперь у нас проблемы. Каждый составной литерал вызывает создание объекта, но этот объект существует только внутри блока, образованного фигурными скобками, которые заключают инструкцию, в которой появляется литерал. К тому времени, когда вызывается `evaluate_polynomial, coefficients` указывает на объект, который больше не существует. Результат: неопределённое поведение.

Создатели C99 были недовольны этим положением вещей, потому что программисты вряд ли ожидали, что простое добавление фигурных скобок внутри инструкции `if` вызовет неуверенное поведение. Чтобы избежать проблемы, они решили, что внутренние инструкции всегда будут считаться блоками. В результате примеры 1 и 2 эквивалентны, причём оба демонстрируют неопределённое поведение.

Подобная проблема может возникнуть, когда составной литерал является частью управляющего выражения инструкции выбора или итерации. По этой причине каждая инструкция выбора и итерации целиком считаются блоком (как будто невидимый набор фигурных скобок окружает всю инструкцию). Так, например, инструкция `if` с предложением `else` состоит из трёх блоков: каждая из двух внутренних инструкций является блоком, как и вся инструкция `if`.

Вопрос: Вы сказали, что хранилище для переменной с автоматической продолжительностью хранения выделяется при выполнении окружающего блока. Это верно для массивов переменной длины C99?

Ответ: Нет. Хранилище для массива переменной длины не выделяется в начале окружающего блока, потому что длина массива ещё не известна. Вместо этого оно выделяется, когда достигается объявление массива во время выполнения блока. В этом отношении массивы переменной длины отличаются от всех других автоматических переменных.

Вопрос: Какая разница между "областью видимости" и "связанностью"?

Ответ: Область видимости работает на компилятор, в то время как связывание использует компоновщик.

Компилятор использует область видимости идентификатора, чтобы определить, допустимо ли ссылаться на идентификатор в заданной точке файла. Когда компилятор преобразует исходный файл в объектный код, он отмечает, какие имена имеют внешнее связывание, в конечном итоге сохраняя эти имена в таблице внутри объектного файла.

Таким образом, компоновщик имеет доступ к именам с внешней связанностью; имена с внутренней связанностью или без связанности невидимы для компоновщика.

Вопрос: Я не понимаю, как имя может иметь блоковую область видимости блока, но внешнее связывание. Не могли бы Вы уточнить?

Ответ: Конечно. Предположим, что один исходный файл определяет переменную *i*:

```
int i;
```

Предположим, что определение *i* лежит вне какой-либо функции, поэтому по умолчанию *i* имеет внешнее связывание. В другом файле есть функция *f*, которая должна получить доступ к *i*, поэтому тело *f* объявляет *i* как extern:

```
void f (void ){  
    extern int i;  
    ...  
}
```

В первом файле *i* имеет файловую область видимости. В *f*, однако, *i* имеет блоковую область видимости, если другие функции, кроме *f*, нуждаются в доступе к *i*, им нужно будет объявить её отдельно. (Или мы можем просто переместить объявление *i* за пределы *f*, чтобы *i* имела файловую область видимости.) Что сбивает с толку во всей этой истории, так это то, что каждое объявление или определение *i* устанавливает

другую область видимости; иногда это файловая область, а иногда это область блока.

Вопрос: Почему нельзя использовать `const` объекты в константных выражениях? `const` означает "константа", да?

Ответ: В Си, `const` означает "только для чтения", а не "константа". Давайте рассмотрим несколько примеров, которые иллюстрируют, почему `const` объекты не могут быть использованы в константных выражениях. Начнём с того, что объект `const` может быть постоянным только в течение своего жизненного цикла, а не во время выполнения программы. Предположим, что объект `const` объявлен внутри функции:

```
void f (int n){  
    const int m = n / 2;  
    ...  
}
```

Когда вызывается ***f***, ***m*** будет инициализировано значением $n / 2$. Значение ***m*** будет оставаться постоянным до тех пор, пока ***f*** не возвратится. Когда ***f*** вызовется в следующий раз, ***m***, вероятно, будет иметь другое значение. Вот в чем проблема. Предположим, что ***m*** появляется в инструкции `switch`:

1082

```
void f(int n){  
  
    const int m = n/2;  
    ...  
    switch (...){  
    ...  
    case m:... /*** WRONG ***/  
    ...  
    }  
    ...  
}
```

Значение ***m*** не будет известно до тех пор, пока не будет вызвано ***f***, что нарушает правило Си о том, что значения меток case должны быть константными выражениями. Далее рассмотрим const - объекты, объявленные вне блоков. Эти объекты имеют внешнее связывание и могут совместно использоваться файлами. Если бы Си позволял использовать const объекты в константных выражениях, мы могли бы легко оказаться в следующей ситуации:

```
extert const int n;  
  
int a[n]; /*** WRONG ***/
```

n, вероятно, определена в другом файле, что делает невозможным определение длины ***a*** компилятором. (Я

предполагаю, что **a** является внешней переменной, поэтому она не может быть массивом переменной длины.)

Если этого недостаточно, чтобы убедить вас, подумайте об этом: если объект `const` также объявлен как `volatile`, его значение может измениться в любое время во время выполнения. Вот пример из стандарта C :

```
extern const volatile int real_time_clock;
```

Переменная `real_time_clock` не может быть изменена программой (потому что она объявлена `const`), но её значение может измениться через какой-либо другой механизм (потому что она объявлена `volatile`).

Вопрос: Почему синтаксис деклараторов такой странный?

Ответ: Объявления нацелены имитировать использование. Декларатор указателя имеет форму `*p`, которая соответствует способу, которым оператор косвенного обращения будет позже применён к **p**. Декларатор массива имеет вид `a [...]`, что соответствует тому, как массив будет позже индексироваться. Декларатор функции имеет форму `f (...)`, что соответствует синтаксису вызова функции. Это рассуждение распространяется даже на самые сложные деклараторы. Рассмотрим массив `file_cmd` раздела 17.7, элементы которого являются указателями на функции.

Декларатор `file_cmd` имеет вид `(*file_cmd[]) (void)`

1084

и вызов функции имеет вид (*file_cmd [n]) ();

Скобки круглые, квадратные и * находятся в одинаковых позициях.

Упражнения

К разделу 18.1

1. Для каждого из следующих объявлений определите класс хранения, квалификаторы типов, спецификаторы типов, деклараторы и инициализаторы.

(a) static char **lookup (int level);

(b) volatile unsigned long io_flags;

(c) extern char *file_name [MAX_FILES], path[];

(d) static const char token_buf[] = "";

К разделу 18.2

W2. Ответьте на каждый из следующих вопросов с помощью auto, extern, register и / или static.

1085

(a) какой класс хранения используется в первую очередь для указания на то, что переменная или функция может быть разделена несколькими файлами?

(b) предположим, что переменная x должна совместно использоваться несколькими функциями в одном файле, но скрыта от функций в других файлах. Какой класс хранения должен быть объявлен?

3. Перечислите продолжительность хранения (статическое или автоматическое), область видимости (блок или файл) и связывание (внутреннее, внешнее или нет) каждой переменной и параметра в следующем файле:

```
extern float a;
```

```
void f(register double b){
```

```
    static int c;
```

```
    auto char d;
```

```
}
```

W4. Пусть f - произвольная функция. Каким будет значение $f(10)$, если f никогда не вызывали раньше? Каково будет значение $f(10)$, если f была ранее вызвана пять раз?

```
int f (int i){
```

1086

```
static int j = 0;  
return i* j++;  
}
```

5. Укажите, является ли каждое из следующих утверждений является истинным или ложным. Обоснуйте каждый ответ.

(a) каждая переменная со статической продолжительностью хранения имеет файловую область видимости.

(B) каждая переменная, объявленная внутри функции не имеет связанности.

с) каждая переменная с внутренней связанностью имеет статическую продолжительность хранения.

(d)каждый параметр имеет область видимости блока.

6. Следующая функция должна выводить сообщения об ошибках. Каждому сообщению предшествует целое число, указывающее, сколько раз была вызвана функция. К сожалению, функция всегда отображает 1 как номер сообщения об ошибке. Найдите ошибку и покажите, как её исправить, не внося никаких изменений за пределами функции.

```
void print_error(const char *message){
```

1087

```
int n = 1;  
printf("Error %d: %s\n",n++,message);  
}
```

К разделу 18.3

7. Предположим, что мы объявляем `x` объектом `const`. Какое из следующих утверждений о `x` является ложным?

(a) если `x` имеет тип `int`, его можно использовать в качестве значения метки `case` в инструкции `switch`.

(b) компилятор проверит, что `x` не присваивается.

© `x` подчиняется тем же правилам области видимости, что и переменные.

d) `x` может быть любого типа.

Раздел 18.3

W8. Напишите полное описание типа `x`, как указано в каждой из следующих деклараций

(a) `char (*x[10] ) (int);`

1088

- (b) `int (*x(int)) [5];`
- (c) `float * (*x(void)) (int);`
- (d) `void (*x(int, void (*y) (int))) (int);`

9. Используйте ряд определений типов для упрощения каждого объявления в упражнении 8.

W10. Напишите объявления для следующих переменных и функций:

- (a) *p* - указатель на функцию с аргументом символьного указателя, который возвращает символьный указатель.
- (b) *f*- функция с двумя аргументами: *p*, указатель на структуру с тегом *t* и *n*, целое число типа `long`.
f возвращает указатель на функцию, которая не имеет аргументов и ничего не возвращает.
- (c) *a* - это массив из четырёх указателей на функции, которые не имеют никаких аргументов и ничего не возвращает.
Элементы изначально указывают на функции `insert`, `search`, `update` и `print`.
- (d) *b* - массив из 10 указателей на функции с двумя аргументами `int`, возвращающими структуры с тегом *t*.

1089

11. В разделе 18.4, мы увидели, что следующие объявления являются недопустимыми:

```
int f(int) []; /* функции не могут возвращать массивы */  
int g (int) (int); /* функции не могут возвращать функции */  
int a[10] (int); /*элементы массива не могут быть функциями */
```

Однако мы можем добиться аналогичных эффектов с помощью указателей: функция может возвращать указатель на первый элемент массива, функция может возвращать указатель на функцию, а элементы массива могут быть указателями на функции. Соответствующим образом пересмотреть каждое из этих заявлений.

* 12. (a) Напишите полное описание типа функции *f*, считая, что она объявлена следующим образом:

```
int (* f (float (*) (long), char *)) (double);
```

(b) Приведите пример, показывающий, как будет вызываться *f*.

К разделу 18.5

W13. Какие из следующих объявлений являются допустимыми? (Предположим, что PI является макросом, который представляет 3.14159.)

1090

- (a) `char c = 65;`
- (b) `static int i = 5, j = i * i;`
- (c) `double d = 2 * PI;`
- (d) `double angles[] = {0, PI/2, PI, 3*PI/2};`

14. Какие переменные нельзя инициализировать?

- (a) Переменные массива
- (b) Перечислимые переменные
- (c) Структурные переменные
- (d) Переменные объединения
- (e) Ни одно из вышеуказанных

W15. Какое свойство переменной определяет, имеет ли она начальное значение по умолчанию?

W15. Какое свойство переменной определяет, имеет ли начальное значение по умолчанию?

- (a) Длительность хранения
- (b) Область видимости
- (c) Связывание
- (d) Тип

19 Разработка программы

*Там, где есть модульность, возможно непонимание:
Соккрытие информации предполагает необходимость
проверки связи.*

Очевидно, что программы реального мира больше, чем примеры в этой книге, но вы не можете понять, насколько они больше.

Более быстрые процессоры и большая оперативная память позволили писать программы, которые были бы непрактичными всего несколько лет назад.

Популярность графических пользовательских интерфейсов значительно увеличила среднюю длину программы.

Большинство полнофункциональных программ сегодня составляют не менее 100 000 строк. Программы в миллион строк являются обычным явлением, и это не такая уж и редкость программы в 10 миллионов строк и более. Хотя Си не был предназначен для написания больших программ, многие крупные программы фактически были написаны на Си. Это сложно и это требует большой осторожности, но это можно сделать.

В этой главе я расскажу о методах, которые оказались полезными при написании больших программ, и покажу, какие возможности Си (например, статический класс хранения) особенно полезны. Написание больших программ (часто называемое "programming—in—the—large") сильно отличается от написания маленьких - это похоже на разницу между написанием курсовой работы (10 страниц с двойным

интервалом, конечно) и 1000—страничной книгой. Большая программа требует большего внимания к стилю, так как над ней будет работать много людей. Это требует тщательной документации. Она требует планирования обслуживания, поскольку она, вероятно, будет изменяться много раз. Прежде всего, большая программа требует тщательного проектирования и гораздо большего планирования, чем небольшая программа. Как выразился Алан Кей, разработчик языка программирования Smalltalk, "вы можете построить конуру из чего угодно". Собачью будку можно построить без особой конструкции из любых подручных материалов. С другой стороны, дом для людей слишком сложен, чтобы просто собрать его на коленке". В главе 15 обсуждалось написание больших программ на С, но основное внимание уделялось деталям языка.

В этой главе мы ещё раз вернёмся к этой теме, на этот раз сосредоточившись на методах хорошего дизайна программы. Полное обсуждение вопросов разработки программ явно выходит за рамки этой книги.

Тем не менее, я постараюсь кратко осветить некоторые важные концепции в разработке программ и показать, как их использовать для создания программ на языке С, которые можно читать и поддерживать.

В разделе 19.1 обсуждается, как рассматривать программу на языке Си в качестве набора модулей, предоставляющих

услуги друг другу. Затем мы увидим, как концепции сокрытия информации (раздел 19.2) и абстрактные типы данных (раздел 19.3) могут улучшить модули. Сосредоточившись на одном примере (тип данных стёка), раздел 19.4 иллюстрирует, как абстрактный тип данных может быть определён и реализован в С. раздел 19.5 описывает некоторые ограничения С для определения абстрактных типов данных и показывает, как обойти их.

19.1 Модули

При разработке программы на языке Си (или программы на любом другом языке, если уж на то пошло) часто бывает полезно рассматривать её как ряд независимых модулей. Модуль - это совокупность сервисов, некоторые из которых доступны другим частям программы (клиентам). Каждый модуль имеет интерфейс, описывающий доступные сервисы. Подробная информация о модуле, включая исходный код самих сервисов, хранится в реализации модуля. В контексте С "услугами" являются функции. Интерфейс модуля представляет собой файл заголовка, содержащий прототипы функций, которые будут доступны клиентам (исходным файлам). Реализация модуля - это исходный файл, содержащий определения функций модуля. Чтобы проиллюстрировать эту терминологию, давайте рассмотрим программу калькулятора, описанную в разделах 15.1 и 15.2.

Эта программа состоит из файла `calc.c`, который содержит функцию `main`, и модуля стёка, который хранится в файлах `filesstack.h` и `stack.c` и (см. рисунок вверху следующей страницы).

`calc.c` является клиентом модуля стёка.

`stack.h` - интерфейс модуля стёка; он предоставляет все, что клиенту нужно знать о модуле.

`stack.c` - реализация модуля; он содержит определения функций стёка, а также объявления переменных, составляющих стёк.

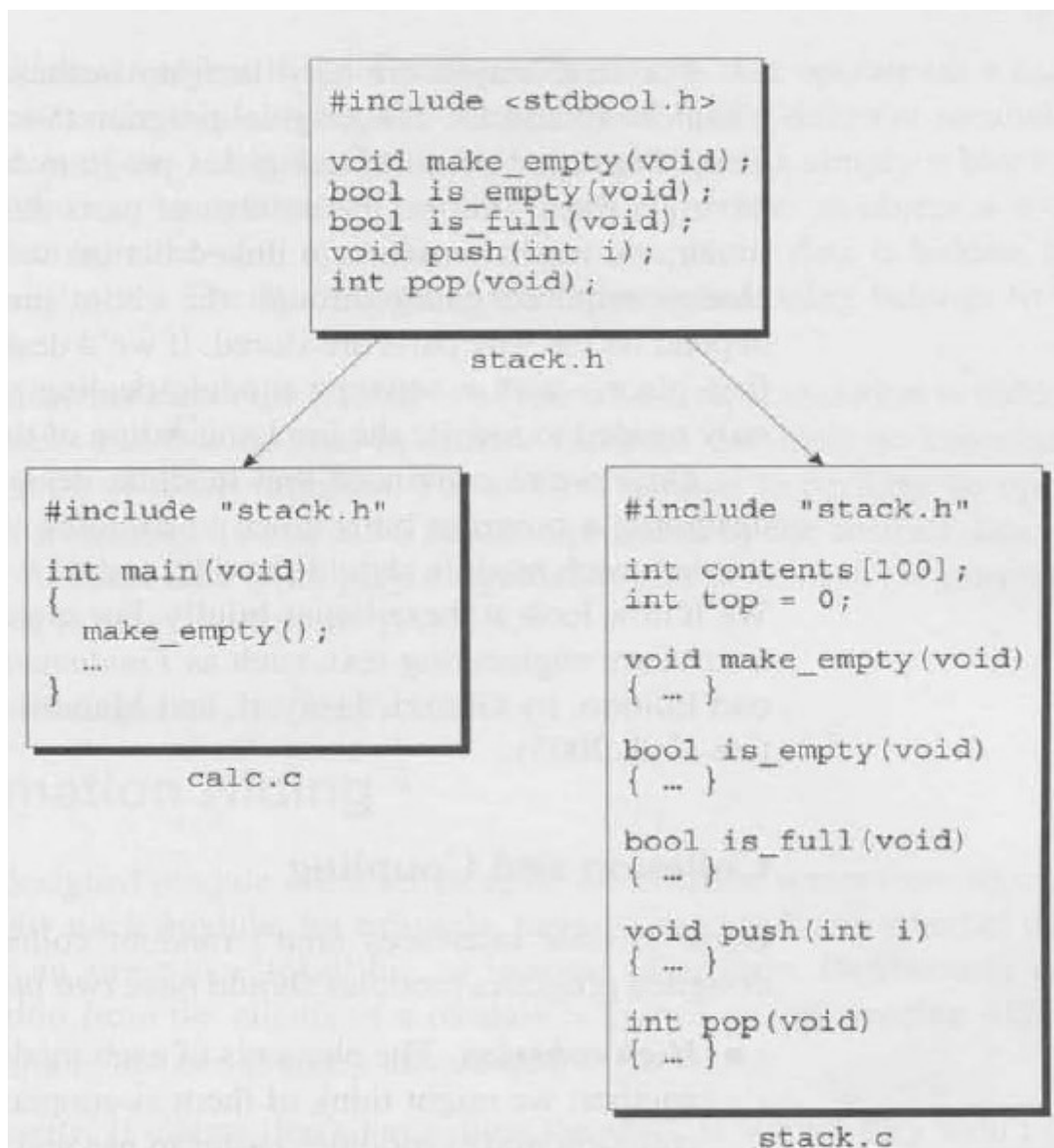
Библиотека `си` сама по себе представляет собой набор модулей. Каждый заголовок в библиотеке служит интерфейсом к модулю.

(`stdio.h`), например, является интерфейсом к модулю, содержащему функции ввода-вывода, в то время как

`<string.h>` - это интерфейс к модулю, содержащему функции обработки строк. Разделение программы на модули имеет ряд преимуществ:

- ***Абстракция.***

Если модули правильно спроектированы, мы можем рассматривать их как абстракции; мы знаем, что они делают, но нас не волнуют детали того, как они это делают. Благодаря абстракции нет необходимости понимать, как работает вся программа, чтобы внести изменения в одну её часть. Более того, абстракция



облегчает нескольким членам команды работу над одной и той же программой. После согласования интерфейсов для модулей ответственность за реализацию каждого модуля может быть делегирована конкретному лицу. Члены команды могут работать в значительной степени независимо друг от друга.

- ***Возможность повторного использования.***

Любой модуль, предоставляющий услуги, потенциально может быть повторно использован в других программах. Например, наш модуль стёка можно использовать повторно. Поскольку часто бывает трудно предвидеть будущее использование модуля, рекомендуется разрабатывать модули для повторного использования.

- ***Ремонтопригодность.***

Небольшая ошибка обычно затрагивает только реализацию одного модуля, упрощая поиск и исправление ошибки. После исправления ошибки для перекомпиляции программы требуется только перекомпиляция модуля (с последующей компоновкой всей программы). В более крупном масштабе мы могли бы заменить реализацию модуля целиком, возможно, для повышения производительности или при переносе программы на другую платформу.

Хотя все эти преимущества важны, удобство сопровождения является наиболее критичным. Большинство реальных программ эксплуатируются в течение многих лет, в течение которых обнаруживаются ошибки, вносятся улучшения и модификации для удовлетворения меняющихся требований. Модульное проектирование программы значительно упрощает обслуживание. Поддержание программы должно быть похоже на поддержание автомобиля — восстановление

спущенной шины не должно требовать капитального ремонта двигателя.

Для примера нам не нужно искать дальше программы inventory из глав 16 и 17. Исходная программа (раздел 16.3) хранила записи деталей в массиве.

Предположим, что после использования этой программы в течение некоторого времени клиент возражает против наличия фиксированного ограничения на количество деталей, которые могут храниться. Чтобы удовлетворить клиента, мы можем переключиться на связанный список (как мы сделали в разделе 17.5). Чтобы сделать это изменение, нужно было пройти через всю программу, ищем все места, которые зависят от способа хранения деталей. Если бы мы изначально разработали программу по-другому - с отдельным модулем, занимающимся хранением деталей, - нам нужно было бы переписать только реализацию этого модуля, а не всю программу. Как только мы убедимся, что модульное проектирование - это правильный путь, процесс разработки программы сводится к решению, какие модули она должна иметь, какие услуги должен предоставлять каждый модуль и как модули должны быть взаимосвязаны. Теперь мы кратко рассмотрим эти вопросы. Для получения дополнительной информации о дизайне обратитесь к тексту software engineering, такому как Fundamentals of Software Engineering,

Second Edition, by Ghezzi, Jazayeri and Mandrioli (Upper Saddle River, N. J.: Prentice-Hall. 2003).

Согласованность и связанность

Хорошие интерфейсы модулей - это не случайные наборы объявлений, в хорошо спроектированной программе модули должны иметь два свойства:

- ➔ **Высокая согласованность.** Элементы каждого модуля должны быть тесно связаны друг с другом; мы можем думать о них как о сотрудничающих для достижения общей цели. Высокая согласованность упрощает использование модулей и упрощает понимание всей программы.
- ➔ **Слабая связанность.** Модули должны быть как можно более независимыми друг от друга. Слабая связь облегчает изменение программы и повторное использование модулей.

Обладает ли программа калькулятора этими свойствами? Модуль `стёка` чётко согласован: его функции представляют операции в `стёке`. В программе небольшая связанность. Файл `calc.c` зависит от `stack.h` (а `stack.c`, конечно, зависит от `stack.h`), но других явных зависимостей нет.

Типы модулей

Из-за потребности в высокой степени согласованности и низкой взаимосвязанности модули, как правило, попадают в определённые типичные категории:

- **Пул данных** - это совокупность связанных переменных и/или констант. В С модуль этого типа часто является просто файлом заголовка. С точки зрения дизайна размещение переменных в файлах заголовков обычно не является хорошей идеей, но сбор связанных констант в файле заголовков часто может быть полезным. В библиотеке С `<float.h>` и `<limits.h>` являются пулами данных.
- **Библиотека** - это набор связанных функций. Заголовок `<string.h>`, например, является интерфейсом к библиотеке функций обработки строк.
- **Абстрактный объект** - это набор функций, которые работают со скрытой структурой данных. (В этой главе термин "объект" имеет другое значение, чем в остальной части книги. В терминологии языка Си объект - это просто блок памяти, который может хранить значение. В этой главе. однако объект - это набор данных, связанных с операциями над данными. Если данные скрыты, то объект "абстрактный".) модуль стёка, который мы обсуждали, относится к этой категории.

- **Абстрактный тип данных (ADT)** - это тип, представление которого скрыто. Клиентские модули могут использовать этот тип для объявления переменных, но не имеют представления о структуре этих переменных. Чтобы клиентский модуль мог выполнить операцию над такой переменной, он должен вызвать функцию, предоставляемую модулем абстрактного типа данных. Абстрактные типы данных играют значительную роль в современном программировании; мы вернёмся к ним в разделах 19.3-19.5.
-

19.2 Соккрытие информации

Хорошо спроектированный модуль часто хранит некоторую информацию в секрете от клиентов. Клиентам нашего модуля стёка, например, не нужно знать, хранится ли стёк в массиве, в связанном списке или в какой-либо другой форме.

Умышленное соккрытие информации от клиентов модуля называется соккрытием информации. У соккрытия информации есть два основных преимущества:

- **Безопасность.** Если клиенты не знают, как хранится стёк, они не смогут повредить его, вмешиваясь в его внутреннюю работу. Для выполнения операций со стёком, им придётся вызывать функции,

предоставляемые самим модулем, - функции, которые мы написали и протестировали.

- **Гибкость.** Внесение изменений - независимо от того, насколько они велики - во внутреннюю работу модуля, не составит труда. Например, мы могли бы сначала реализовать стёк в виде массива, а затем перейти к связанному списку или другому представлению. Конечно, нам придётся переписать реализацию модуля, но—если модуль был спроектирован правильно—нам не придётся изменять внешний вид модуля.

В языке Си основным инструментом для принудительного сокрытия информации является статический класс хранения. Объявление переменной как `static` с областью действия файла даёт ей внутреннюю связь, тем самым предотвращая доступ к ней из других файлов, включая клиентов модуля. (Объявление функции статической также полезно - функция может быть непосредственно вызвана только другими функциями в том же файле.)

Модуль стёка

Чтобы увидеть преимущества сокрытия информации, давайте рассмотрим две реализации стёкового модуля: одна

1102

использует массив, а другая - связанный список. Файл заголовка модуля будет иметь следующий вид:

```
//stack.h

#ifndef STACK_H
#define STACK_H

#include <stdbool.h> /* C99 only */

void make_empty (void) ;
bool is_empty (void);
bool is_full (void);
void push(int i);
int pop (void);

#endif
```

Я включил заголовок с99 <stdbool.h>, чтобы функции is_empty и is_full могли возвращать результат bool, а не значение int. Давайте сначала используем массив для реализации стёка:

1103

```
//stack.c

#include <stdio.h>
#include <stdlib.h>
#include "stack.h"

#define STACK_SIZE 100

static int contents [STACK_SIZE] ;
static int top = 0;
static void terminate(const char *message){
    printf("%s\n", message) ;
    exit (EXIT_FAILURE) ;
}

void make_empty (void){
    top = 0;
}

bool is_empty (void){
    return top == 10;
}
```

1104

```
bool is_full(void){  
    return top == STACK_SIZE;  
}  
  
void push(int i){  
    if (is_full())  
        terminate("Error in push: stack is full.");  
    contents [top++] = i;  
}
```

Переменные, составляющие стёк (contents и top), объявляются статическими, так как у остальной части программы нет причин обращаться к ним напрямую. Функция terminate также объявляется статической. Эта функция не является частью интерфейса модуля; вместо этого она предназначена для использования исключительно в рамках реализации модуля. Некоторые программисты используют макросы для указания того, какие функции и переменные являются "общедоступными - public" (доступны в любом месте программы), а какие — "частными - private " (ограничены одним файлом):

```
#define PUBLIC /* empty */  
  
#define PRIVATE static
```

1105

Причина написания PRIVATE вместо static заключается в том, что последний имеет более одного использования в С;

PRIVATE даёт понять, что мы используем его для принудительного сокрытия информации. Вот как бы выглядела реализация стёка, если бы мы использовали PUBLIC и PRIVATE:

```
PRIVATE int contents[STACK_SIZE];
```

```
PRIVATE int top = 0;
```

```
PRIVATE void terminate(const char *message) {...}
```

```
PUBLIC void make_empty(void) {...}
```

```
PUBLIC bool is_empty(void) {...}
```

```
PUBLIC bool is_full(void) {...}
```

```
PUBLIC void push(int i) {...}
```

```
PUBLIC int pop (void) {...}
```

Теперь мы перейдем к реализации модуля стёка в виде связанного списка:

```
//stack2.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include "stack.h"
```

1106

```
struct node {  
    int data;  
    struct node *next;  
}  
  
static struct node *top = NULL;  
  
static void terminate(const char *message{  
    printf ("%s\n", message);  
    exit (EXIT_FAILURE);  
  
void make_empty (void){  
    while (!is_empty())  
        pop( );  
}  
  
bool is_empty (void){  
    return top== NULL;  
}  
  
bool is_full (void){  
    return false;  
}
```

1107

```
void push(int i){
    struct node *new_node = malloc(sizeof(struct node) );
    if (new_node == NULL)
        terminate("Error in push: stack is full.");
    new_node->data = i;
    new_node->next = top;
    top = new_node;
}

int pop (void){
    struct node *old_top;
    int i;
    if (is_empty())
        terminate("Error in pop: stack is empty.");
    old_top = top;
    i = top->data;
    top = top->next;
    free (old_top);
    return i;
}
```

Обратите внимание, что функция `is_full` возвращает `false` каждый раз, когда она вызывается.

Связанный список не имеет ограничений по размеру, поэтому стёк никогда не будет полным.

Возможно (но маловероятно), что у программы может закончиться память, что приведёт к сбою функции `push`, но нет простого способа проверить это условие заранее. Наш пример стёка ясно показывает преимущество сокрытия информации: не имеет значения, используем ли мы `stack1.c` или `stack2.c` для реализации модуля стёка.

Обе версии соответствуют интерфейсу модуля, поэтому мы можем переключаться с одной на другую, не внося изменений в другие части программы.

19.3 Абстрактные типы данных

Модуль, который служит абстрактным объектом, как модуль стёка в предыдущем разделе, имеет серьёзный недостаток:

нет возможности иметь несколько экземпляров объекта (в данном случае более одного стёка). Для этого нам нужно сделать ещё один шаг и создать новый тип. Как только мы определим тип стёка, мы сможем иметь столько стёков, сколько захотим. Следующий фрагмент иллюстрирует, как мы могли бы иметь два стёка в одной программе:

```
Stack s1, s2;
```


1109

```
make_empty(&s1) ;  
make_empty (&s2);  
push(&s1, 1);  
push(&s2, 2);  
if (!is_empty(&s1))  
    printf("%d\n", pop(&s1)) ; /* prints "1" */
```

Мы на самом деле не уверены, что `s1` и `s2` являются (структурами? указателями?), но это не имеет значения. Для клиентов `s1` и `s2` - это абстракции, которые отвечают на определённые операции (`make_empty`, `is_ernpty`, `is_full`, `push` и `pop`). Давайте преобразуем наш `stack.h` так, чтобы он обеспечивал тип `Stack`, где `Stack` - это структура. Для этого потребуется добавить параметр `Stack` (или `Stack *`) к каждой функции.

Теперь заголовок будет выглядеть следующим образом (изменения в `stack.h` выделены жирным шрифтом; неизменные части заголовка не отображаются):

```
# define STACK_SIZE 100  
typedef struct {  
    int contents [STACK_SIZE];  
    int top;
```

1110

} Stack;

void make-empty(Stack *s);

bool is_empty(const Stack *s);

bool is_full(const Stack *s);

void push(Stack *s, int i);

int pop(Stack *s);

Параметры стёка для `make_empty`, `push` и `pop` должны быть указателями, так как эти функции изменяют стёк.

Параметр `is_empty` и `is_full` не обязательно должен быть указателем, но я все равно сделал его таковым. Передача этим функциям указателя `Stack` вместо значения `Stack` более эффективна, так как последнее приведёт к копированию структуры.

Инкапсуляция

К сожалению, `Stack` не является абстрактным типом данных, поскольку `stack.h` показывает, что на самом деле представляет собой тип `Stack`.

Ничто не мешает клиентам использовать переменную `Stack` в качестве структуры:

1111

```
Stack s1;
```

```
s1.top = 0;
```

```
s1.contents [top ++] = 1;
```

Предоставление доступа к членам `top` и `contents` позволяет клиентам повредить стёк. Хуже того, мы не сможем изменить способ хранения стёков, без необходимости оценивать влияние изменений на клиентов. Нам нужен способ, при котором клиенты не знают, как представлен тип стёка. С имеет только ограниченную поддержку для инкапсуляции типов таким образом. Более новые языки на основе С, включая С++, Java и С#, лучше приспособлены для этой цели.

Неполные типы

Единственный инструмент, который даёт нам С для инкапсуляции, - это неполный тип. (Неполные типы были кратко упомянуты в разделе 17.9 и в разделе вопросов и ответов в конце главы 17.) Стандарт С описывает неполные типы как "типы, которые описывают объекты, но не имеют информации, необходимой для определения их размеров". Например, объявление

```
struct t; /* неполное объявление t */
```

1112

сообщает компилятору, что *t* является тегом структуры, но не описывает элементы структуры. В результате компилятор не имеет достаточно информации для определения размера такой структуры. Намерение состоит в том, чтобы неполный тип был завершён в другом месте программы. Пока тип остаётся неполным, его использование ограничено. Поскольку компилятор не знает размер неполного типа, его нельзя использовать для объявления переменной:

```
struct t s; /*" WRONG ***/
```

Однако вполне законно определять тип указателя, который ссылается на неполный тип:

```
typedef struct t * T;
```

Это определение типа утверждает, что переменная типа *T* является указателем на структуру с тегом *t*. Теперь мы можем объявлять переменные типа *T*, передавать их в качестве аргументов функциям и выполнять другие допустимые операции для указателей.

(Размер указателя не зависит от того, на что он указывает, что объясняет, почему C допускает такое поведение.) Однако мы не можем применить оператор *->* к одной из этих переменных, поскольку компилятор ничего не знает о членах структуры *t*.

19.4 Абстрактный тип данных Stack

Чтобы проиллюстрировать, как абстрактные типы данных могут быть инкапсулированы с использованием неполных типов, мы разработаем ADT (Abstract Data Type) стёк на основе модуля стёка, описанного в Разделе 19.2. В процессе мы исследуем три различных способа реализации стёка.

Определение интерфейса для ADT Stack

Во-первых, нам понадобится файл заголовка, который определяет тип ADT стёка и даёт прототипы функций, представляющих операции со стёком. Назовём этот файл `stackADT.h`. Тип `Stack` будет указателем на структуру `stack_type`, в которой хранится фактическое содержимое стёка. Эта структура является неполным типом, который будет завершён в файле, реализующем стёк. Члены этой структуры будут зависеть от того, как реализован стёк. Вот как будет выглядеть файл `stackADT.h`:

//stackADT.h (1-я версия)

```
# ifndef STACKADT_H
```

```
# define STACKADT_H
```

```
#include <stdbool.h> /* C99 only */
```

1114

```
typedef struct stack_type *Stack;
```

```
stack create(void);
```

```
void destroy(Stack s);
```

```
void make_empty(Stack s);
```

```
bool is_empty(Stack s);
```

```
bool is_full(Stack s);
```

```
void push(Stack s, int i);
```

```
int pop(Stack s);
```

```
#endif
```

Клиенты, которые включают `stackADT.h`, смогут объявлять переменные типа `Stack`, каждая из которых может указывать на структуру `stack_type`. Затем клиенты могут вызывать функции, объявленные в `stackADT.h`, для выполнения операций с переменными стёка. Однако, клиенты не могут получить доступ к членам структуры `stack_type`, так как эта структура будет определена в отдельном файле. Обратите внимание, что каждая функция имеет параметр `Stack` или возвращает значение `Stack`. Функции стёка в Разделе 19.3

1115

имели параметры типа `Stack *`. Причина различия в том, что переменная `Stack` теперь является указателем; он указывает на структуру `stack_type`, в которой хранится содержимое стёка. Если функции необходимо изменить стёк, она изменяет саму структуру, а не указатель на структуру.

Также обратите внимание на наличие функций `create` и `destroy`. Модулю эти функции обычно не нужны, но ADT нужны, `create` будет динамически выделять память для стёка (включая память, необходимую для структуры `stack_type`), а также инициализировать стёк в его состояние "empty", `destroy` освободит динамически выделенную память стёка.

Следующий клиентский файл можно использовать для тестирования ADT стёка. Он создаёт два стёка и выполняет над ними различные операции.

```
//stackclient.c
```

```
#include <stdio.h>
```

```
#include "StackADT.h"
```

```
int main(void){
```

```
    Stack s1, s2;
```

```
    int n;
```

```
    s1 = create();
```

```
    s2= create ();
```

1116

```
push(s1,1);
push(s1, 2);
n= pop(s1);
printf("Вытолкнут %d из стёка s1\n",n);
push(s2, n);
n= pop(s1);
printf("Вытолкнут %d из стёка s1\n",n);
push(s2, n);
destroy(s1);
while (!is_empty(s2))
    printf("Вытолкнут %d из стёка s2\n",pop(s2));
push(s2,3);
make_empty(s2);
if (is_empty(s2))
    printf("s2 is empty\n");
else
    printf("s2 is not empty\n");
destroy(s2);
return 0;
}
```


Если стёк ADT реализован правильно, то программа должна выдавать следующие выходные данные:

Вытолкнут 2 из стёка s1

Вытолкнут 1 из стёка s1

Вытолкнут 1 из стёка s2

Вытолкнут 2 из стёка s1

s2 is empty

Реализация стёкового ADT с использованием массива фиксированной длины

Существует несколько способов реализации стёка ADT. Наш первый подход самый простой. У нас будет файл `stackADT.c`, определяющий структуру `stack_type` так, чтобы он содержал массив фиксированной длины (для хранения содержимого стёка) вместе с целым числом, которое отслеживает вершину стёка:

```
struct stack_type {  
    int contents [STACK_SIZE];  
    int top;
```

1118

};

Вот как будет выглядеть stackADT.c:

```
//stackADT.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include " stackADT.h"
```

```
#define STACK_SIZE 100
```

```
struct stack_type {
```

```
    int contents [STACK_SIZE];
```

```
    int top;
```

```
}
```

```
static void terminate(const char *message){
```

```
    printf ("%s\n". message) ;
```

```
    exit (EXIT_FAILURE) ;
```

```
}
```

```
Stack create (void){
```

1119

```
Stack s = malloc(sizeof(struct stack_type));
if (s == NULL)
    terminate("Error in create: stack could not be created.");
s->top = 0;
return s;
}

void destroy (Stack s){
    free(s);
}

void make_empty (Stack s){
    s->top = 0;
}

bool is_empty (stack s){
    return s->top == 0;
}

bool is_full (Stack s){
    return s->top == STACK_SIZE;
```

1120

}

```
void push(Stack s, int i){
    if (is_full(s))
        terminate("Error in push: stack is full.");
    s->contents[s->top++] = i;
}

int pop (Stack s){
    if (is_empty(s))
        terminate("Error in pop: stack is empty.");
    return s->contents [--s->top];
}
```

Следует особо отметить то, что в функциях в этом файле используется оператор `->`, а не оператор `.` (точка) для доступа к членам `contents` и `top` структуры `stack_type`. Параметр `s` является указателем на структуру `stack_type`, а не самой структурой, поэтому использование оператора `.` (точка) будет недопустимым.

Изменение типа элемента в Stack ADT

Теперь, когда у нас есть рабочая версия стёкового ADT, давайте попробуем улучшить её. Во-первых, обратите внимание, что элементы в стёке должны быть целыми числами. Это слишком ограничительно: на самом деле тип

1121

элемента не имеет значения. Элементами стёка также могут быть другие базовые типы (float, double, long и т. д.) или даже структуры, объединения или указатели, если на то пошло. Чтобы упростить изменение ADT стёка для разных типов элементов, давайте добавим определение типа в заголовок stackADT.h. Он определит тип с именем Item, представляющий тип данных, которые будут храниться в стёке.

//stackADT.h (версия 2)

```
#ifndef STACKADT_H
```

```
#define STACKADT_H
```

```
#include <stdbool.h> /* C99 only */
```

```
typedef int Item;
```

```
typedef struct stack_type *Stack;
```

```
Stack create(void);
```

```
void deetroy(Stack s);
```

```
void make_empty(Stack s);
```

```
bool is_empty(Stack s);
```

```
bool is_full(stack s);
```

```
void push(stack s, Item i;
```

1122

Item pop(stack s);

#end if

Изменения в файле выделены жирным шрифтом. Помимо добавления типа **Item**, функции **push** и **pop** были изменены, **push** теперь имеет параметр типа **Item**, а **pop** возвращает значение типа **Item**. С этого момента мы будем использовать эту версию **stackADT.h**; он заменяет более раннюю версию. Файл **stackADT.c** необходимо изменить, чтобы он соответствовал новому **stackADT.h**. Однако изменения минимальны. Структура **stack_type** теперь будет содержать массив, элементы которого имеют тип **Item** вместо **int**:

```
struct stack_type {  
    Item contents [STACK_SIZE];  
    int top;  
};
```

Единственные другие изменения - **push** (второй параметр теперь имеет тип **Item**) и **pop** (который возвращает значение типа **Item**). Тела **push** и **pop** не меняются. Файл **stack client.c** можно использовать для тестирования нового **stackADT.h** и **stackADT.c**, чтобы убедиться, что тип **Stack** по-прежнему работает (работает!). Теперь мы можем изменить тип

элемента в любое время, когда захотим, просто изменив определение типа элемента в `stackADT.h`. (Хотя нам не нужно менять файл `stackADT.c`, нам все равно придётся его перекомпилировать.)

Реализация стёка ADT с использованием динамического массива

ещё одна проблема со стёком ADT в его нынешнем виде заключается в том, что каждый стёк имеет фиксированный максимальный размер, который в настоящее время установлен на уровне 100 элементов. Конечно, этот предел может быть увеличен до любого числа, но все стёки, созданные с использованием типа `Stack`, будут иметь предел. Невозможно иметь стёки разной ёмкости или устанавливать размер стёка во время работы программы. Есть два решения этой проблемы. Один из них - реализовать стёк в виде связанного списка, и в этом случае нет фиксированного ограничения на его размер. Мы рассмотрим это решение через минуту. Но сначала давайте попробуем другой подход, который включает хранение элементов стёка в динамически выделяемом массиве.

Суть последнего подхода состоит в том, чтобы изменить структуру `stack_type` так, чтобы член `contents` был указателем на массив, в котором хранятся элементы, а не сам массив:

1124

```
struct stack_type {  
    Item * contents;  
    int top;  
    int size;  
};
```

Я также добавил новый член `size`, в котором хранится максимальный размер стёка (длина массива, на который указывает `contents`). Мы будем использовать этот член для проверки условия "стёк заполнен". Функция `create` теперь будет иметь параметр, определяющий желаемый максимальный размер стёка:

```
Stack create(int size);
```

При вызове `create` создаётся структура `stack_type` плюс массив размера `length`. Член структуры `contents` будет указывать на этот массив. Файл `stackADT.h` будет таким же, как и раньше, за исключением того, что нам нужно добавить параметр `size` в функцию `create`. (Назовём новую версию `stackADT2.h`).

Однако файл `stackADT.c` потребует более обширной модификации. Новая версия отображается ниже, а изменения выделены жирным шрифтом.

```
//stackADT.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```


1125

```
#include "stackADT2.h"
```

```
struct stack_type {
```

```
    Item *contents;
```

```
    int top;
```

```
    int size;
```

```
}
```

```
static void terminate(const char *message){
```

```
    printf ( "%s\n", message);
```

```
    exit (EXIT_FAILURE);
```

```
}
```

```
Stack create(int size){
```

```
    stack s= malloc(sizeof(struct stack_type));
```

```
    if (s == NULL)
```

```
        terminate("Error in create: stack could not be created.");
```

```
    s->contents = malloc(size * sizeof(Item));
```

```
    if (s->contents == NULL){
```

```
        free(s);
```

```
        terminate("error in create: stack could not be created.");
```

```
}
```

1126

```
s->top = 0;
s->size = size;
return 0
}

void destroy (stack s){
    free(s->contents);
    free(s);
}

void make_empty ( Stack s){
    (s->top = 0;
}

bool is_empty (Stack s){
    return s->top == 0;
}

bool is_full(Stack s){
    return s->top == s->size;
}
```

1127

```
void push(Stack s, Item i){
    if (is_full(s))
        terminate("Error in push: Stack is full.");
    s->contents[s->top++] = i;
}
```

```
Item pop (Stack s)
    if (is_empty(s))
        terminate("Error in pop: Stack is empty.");
    return s->contents [--s->top];
}
```

Теперь функция `create` вызывает `malloc` дважды: один раз для выделения структуры `stack_type` и один раз для выделения массива, который будет содержать элементы стёка. Любой вызов `malloc` может завершиться неудачно, что приведёт к вызову `terminate`. Функция `destroy` должна дважды вызвать `free`, чтобы освободить всю память, выделенную функцией `create`. Файл `stackclient.c` снова можно использовать для тестирования ADT стёка. Однако вызовы `create` необходимо будет изменить, поскольку `create` теперь требует аргумента. Например, мы могли бы заменить инструкции

```
s1 = create();
```

1128

```
s2 = create();
```

следующими инструкциями:

```
s1 = create(100);
```

```
s2 = create(200);
```

Реализация стёкового ADT с использованием связного списка

Реализация стёкового ADT с использованием динамически выделяемого массива даёт нам большую гибкость, чем использование массива фиксированного размера. Однако от клиента по-прежнему требуется указать максимальный размер стёка во время его создания. Если вместо этого мы будем использовать реализацию связанного списка, предустановленного ограничения на размер стёка не будет.

Наша реализация будет аналогична той, что содержится в файле `stack2.c` из Раздела 19.2. Связанный список будет состоять из узлов, представленных следующей структурой:

```
struct node {  
    Item data;  
    struct node *next;
```

```
}
```

Типом члена данных теперь является `Item`, а не `int`, но структура в остальном такая же, как и раньше. Структура `stack_type` будет содержать указатель на первый узел в списке:

```
struct stack_type{  
    struct node *top;  
}
```

На первый взгляд структура `stack_type` кажется излишней; мы могли бы просто определить `Stack` как `struct node *` и позволить значению `Stack` быть указателем на первый узел в списке. Однако нам все ещё нужна структура `stack_type`, чтобы интерфейс к стёку оставался неизменным.

(Если бы мы отказались от этого, любой функции, которая изменяла стёк, потребовался бы параметр `Stack *` вместо параметра `Stack`.) Более того, наличие структуры `stack_type` упростит изменение реализации в будущем, если мы решим сохранить дополнительную информацию.

Например, если позже мы решим, что структура `stack_type` должна содержать счётчик того, сколько элементов в настоящее время хранится в стёке, мы можем легко добавить член в структуру `stack_type` для хранения этой информации. Нам не нужно вносить какие-либо изменения в заголовок `stackADT.h`. (Мы будем использовать этот файл заголовка, а не `stackADT2.h`.) Мы также можем использовать исходный файл

1130

stackclient.c для тестирования. Все изменения будут в файле stackADT.c. Вот новая версия:

```
//stackADT3.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include "stackAD.h"
```

```
struct node {
```

```
    Item data;
```

```
    struct node *next;
```

```
struct stack_type {
```

```
    struct node *top;
```

```
}
```

```
static void terminate(const char *message){
```

```
    printf("%a\n", message);
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
Stack create (void){
```

```
    Stack s = malloc(sizeof (struct stack_type));
```

```
    if (s == NULL)
```

```
        terminate("Error in create: stack could not be created.");
```

1131

```
    s->top = NULL; return s;  
}
```

```
void destroy (Stack s){  
    make_empty(s);  
    free(s);  
}
```

```
void make empty (Stack s){  
    while (!is_empty(s) )  
        pop (s);  
}
```

```
bool is empty(Stack s){  
    return s->top == NULL;  
}
```

```
bool is_full (Stack s){  
    return false;  
}
```

```
void push(Stack s, Item i) {
```

1132

```
struct node *new_node = malloc(sizeof (struct node) );
if (new_node == NULL)
    terminate("Error in push: stack is full.");
new_node->data = i;
new_node->next = s->top;
s->top = new_node;
}
```

```
Item pop (Stack s){
    struct node *old_top;
    Item i;
    if (is_empty(s))
        terminate("Error in pop: stack is empty.");
    old_top = s->top;
    i = old_top->data;
    s->top = old_top->next;
    free (old_top) ;
    return i;
}
```

Обратите внимание, что функция destroy вызывает make_empty (для освобождения памяти, занятой узлами в

связанном списке), прежде чем она вызывает `free` (чтобы освободить память для структуры `stack_type`).

19.5. Проблемы проектирования абстрактных типов данных

В разделе 19.4 описан ADT стёка и показано несколько способов его реализации. К сожалению, у этого ADT есть несколько проблем, которые не позволяют ему стать пригодным для широкого употребления. Давайте рассмотрим каждую из этих проблем и обсудим возможные решения.

Соглашения об именах

В настоящее время функции стёка ADT имеют короткие и понятные имена: `create`, `destroy`, `make_empty`, `is_empty`, `is_full`, `push` и `pop`. Если в программе более одного ADT, вероятны конфликты имён, поскольку функции в двух модулях имеют одинаковое имя. (Для каждого ADT, например, потребуется своя собственная функция `create`.) Следовательно, нам, вероятно, потребуется использовать имена функций, которые включают имя самого ADT, например, `stack_create` вместо `create`.

Обработка ошибок

ADT стёк обрабатывает ошибки, отображая сообщение об ошибке и завершая программу. Это неплохо. Программист может избежать выталкивания из пустого стёка или проталкивания данных в полный стёк, заботясь о вызове `is_empty` перед каждым вызовом `pop` и `is_full` перед каждым вызовом `push`, так что теоретически нет причин для отказа вызова `push` или `pop`. (Однако в реализации связанного списка вызов `is_full` не является надёжным; последующий вызов `push` все равно не сработает.) Тем не менее, мы можем предоставить программе способ восстановления после этих ошибок, а не завершение. Альтернативный вариант - вернуть функции `push` и `pop` значение типа `bool`, чтобы указать, были ли они успешными, `push` в настоящее время имеет тип возврата `void`, поэтому было бы легко изменить его так, чтобы он возвращал `true`, если операция `push` завершилась успешно, и `false`, если стёк заполнен. Изменить функцию `pop` было бы труднее, поскольку `pop` в настоящее время возвращает значение, которое было вытолкнуто. Однако, если `pop` должен вернуть указатель на это значение, а не на само значение, тогда `pop` может вернуть `NULL`, чтобы указать, что стёк пуст. Заключительный комментарий об обработке ошибок: стандартная библиотека C содержит параметризованный

макрос с именем `assert`, который может завершить программу, если указанное условие не выполнено.

Мы могли бы использовать вызовы этого макроса в качестве замены для инструкции `if` и вызовов `terminate`, которые в настоящее время появляются в ADT стёке.

Общие ADT

В середине раздела 19.4 мы улучшили ADT стёка, упростив изменение типа элементов, хранящихся в стёке - все, что нам нужно было сделать, это изменить определение типа элемента. Это все ещё неудобно, было бы лучше, если бы стёк мог вмещать элементы любого типа без необходимости изменять файл `stack.h`. Также обратите внимание, что наш стёк ADT страдает серьёзным недостатком: программа не может создать два стёка, элементы которых имеют разные типы. Создать несколько стёков легко, но в этих стёках должны быть элементы одного типа. Чтобы использовать стёки с разными типами элементов, нам нужно было бы сделать копии файла заголовка и исходного файла ADT стёка и изменить один набор файлов так, чтобы у типа `Stack` и связанных с ним функций были разные имена. Мы хотели бы иметь один "общий" тип стёка, из которого мы могли бы создать стёк целых чисел, стёк строк или любой другой стёк, который нам может понадобиться. Существуют различные

способы создания такого типа в С, но ни один из них не является полностью удовлетворительным. В наиболее распространённом подходе в качестве типа элемента используется `void *`, что позволяет использовать произвольные указатели, которые затапливаются и выталкиваются. При использовании этого метода файл `stackADT.h` будет похож на нашу исходную версию; однако прототипы функций `push` и `pop` будут иметь следующий вид:

```
void push(stack s, void *p);
```

```
void *pop(stack s);
```

`pop` возвращает указатель на элемент, извлечённый из стёка; если стёк пуст, он возвращает нулевой указатель.

Использование `void *` в качестве типа элемента имеет два недостатка. Во-первых, этот подход не работает для данных, которые нельзя представить в виде указателя. Элементами могут быть строки (которые представлены указателем на первый символ в строке) или динамически выделяемые структуры, но не базовые типы, такие как `int` и `double`. Другой недостаток - невозможность проверки ошибок. стёк, в котором хранятся элементы `void *`, с радостью допускает сочетание указателей разных типов; нет способа обнаружить ошибку, вызванную проталкиванием указателя неправильного типа.

ADT в новых языках

Проблемы, которые мы только что обсудили, решаются гораздо более чётко в новых языках на основе С, таких как С++, Java и С#. Конфликты имён предотвращаются путём определения имён функций внутри класса. ADT стёка будет представлен классом Stack; функции стёка будут принадлежать этому классу и будут распознаваться компилятором только при применении к объекту Stack. В этих языках есть функция, известная как обработка исключений, которая позволяет таким функциям, как push и pop, "генерировать" исключение, когда они обнаруживают состояние ошибки. Затем код в клиенте может обработать ошибку, "перехватывая" исключение. С++, Java и С# также предоставляют специальные возможности для определения общих ADT. в С++, например, мы определили бы шаблон стёка, не указав тип элемента.

Q&A (Вопросы и ответы)

Вопрос: Вы сказали, что язык Си не предназначен для написания больших программ. Разве UNIX - это не большая программа?

Ответ: В 1978 году Кен Томпсон подсчитал, что ядро UNIX состояло примерно из 10 000 строк кода на языке Си (плюс небольшое количество ассемблера). Другие компоненты UNIX были сопоставимы по размеру; в другой статье 1978 года. Деннис Ричи и его коллеги оценили размер компилятора PDP-11 C в 9660 строк. По сегодняшним меркам это действительно небольшие программы.

Вопрос: Существуют ли какие-либо абстрактные типы данных в библиотеке C?

Ответ: Технически их нет, но некоторые из них близки к этому, включая тип FILE (определённый в `<stdio.h>`). Перед выполнением операции с файлом мы должны объявить переменную типа FILE *:

```
FILE *fp;
```

Затем переменная `fp` будет передана различным функциям обработки файлов. Программисты должны рассматривать файл как абстракцию.

Нет необходимости знать, что такое FILE, чтобы использовать этот тип файла. Предположительно, FILE - это тип структуры, но стандарт C даже не гарантирует этого. На самом деле, лучше не слишком много знать о том, как хранятся значения FILE, поскольку определение типа FILE может (и часто бывает) варьироваться от одного компилятора C к другому. Конечно, мы всегда можем заглянуть в файл `stdio.h` и увидеть,

1139

что такое FILE. В этом случае ничто не помешает нам написать код для доступа к внутренним компонентам FILE. Например, мы можем обнаружить, что FILE - это структура с членом по имени bsize (размер буфера файла):

```
typedef struct {  
    ...  
    int bsize; /* размер буфера */  
    ...  
} FILE;
```

Как только мы узнаем о члене bsize, ничто не помешает нам получить доступ к размеру буфера для конкретного файла:

```
printf ("Размер буфера:% d \ n", fp-> bsize);
```

Однако это не очень хорошая идея, потому что другие компиляторы C могут хранить размер буфера под другим именем или отслеживать его совершенно другим способом. Смена значения члена bsize - ещё более худшая идея:

```
p-> size = 1024;
```

Это опасно, если мы не знаем всех подробностей того, как хранятся файлы. Даже если нам известны подробности, они могут измениться с использованием другого компилятора или следующей версии того же компилятора.

Вопрос: Какие ещё неполные типы существуют, кроме неполных структурных типов?

Ответ: Один из наиболее распространённых неполных типов возникает, когда массив объявляется без указанного размера:

```
extern int a [];
```

После этого объявления (с которым мы впервые столкнулись в Разделе 15.2) **a** имеет неполный тип, потому что компилятор не знает длины **a**. Предположительно, **a** определено в другом файле программы; это определение предоставит недостающую длину. Другой неполный тип встречается в объявлениях, которые не указывают длину массива, но предоставляют инициализатор:

```
int a [] = {1, 2, 3};
```

В этом примере массив **a** изначально имеет неполный тип, но тип завершается инициализатором. Объявление тега объединения без указания членов объединения также создаёт неполный тип. Гибкие элементы массива (функция C99) имеют неполный тип. Наконец, **void** - неполный тип. Тип **void** имеет необычное свойство - он никогда не может быть завершён, что делает невозможным объявление переменной этого типа.

Вопрос: Какие ещё есть ограничения на использование неполных типов?

Ответ: Оператор sizeof не может применяться к неполному типу (что неудивительно, поскольку размер неполного типа неизвестен).

Член структуры или объединения (кроме элемента гибкого массива) не может иметь неполный тип. Точно так же элементы массива не могут иметь неполный тип. Наконец, параметр в определении функции не может иметь неполный тип (хотя это разрешено в объявлении функции). Компилятор "настраивает" каждый параметр массива в определении функции так, чтобы он имел тип указателя, таким образом предотвращая его неполный тип.

Упражнения

К Разделу 19.1.

1. Очередь похожа на стёк, за исключением того, что элементы добавляются с одного конца, но удаляются с другого по методу FIFO (первый вошёл, первый вышел). Операции с очередью могут включать:

Вставка элемента в конец очереди

Удаление элемента из начала очереди

Возврат первого элемента в очереди (без изменения очереди)

1142

Возврат последнего элемента в очереди (без изменения очереди)

Проверка, пуста ли очередь

Напишите интерфейс для модуля очереди в виде файла заголовка с именем queue.h.

К Разделу 19.2.

W 2.Измените файл stack2.c, чтобы использовать макросы PUBLIC и PRIVATE.

W 3.(a) напишите реализацию модуля очереди на основе массива, описанную в упражнении 1.

3.(a)Используйте три целых числа для отслеживания состояния очереди, причём одно целое число хранит позицию первого пустого слота в массиве (используется при вставке элемента), второе - позицию следующего элемента, подлежащего удалению, а третье - количество элементов в очереди. Вставка или удаление, которые привели бы к увеличению любого из первых двух целых чисел после конца массива, должны вместо этого сбросить переменную в ноль, тем самым заставив её "обернуться" к началу массива.

(b) написать реализацию связанного списка модуля очереди, описанного в упражнении 1. Используйте два указателя, один

1143

из которых указывает на первый узел в списке, а другой - на последний узел. Когда элемент вставляется в очередь, добавьте его в конец списка. Когда элемент удаляется из очереди. удалите первый узел в списке.

К Разделу 19.3.

в 4.(a) напишите реализацию типа Stack, предполагая, что стёк - это структура, содержащая массив фиксированной длины.

(b) Повторите тип Stack, на этот раз используя представление связного списка вместо массива (покажите как `stack.h`, так и `stack.c`).

5. Измените заголовок `queue.h` в упражнении 1 так, чтобы он определял тип Queue, где Queue - это структура, содержащая массив фиксированной длины (см. Упражнение 3 (a)).

Измените функции в `queue.h`, чтобы они принимали параметр Queue *.

К Разделу 19.4.

6 (a) Добавьте функцию просмотра в `stackADT.c`. Эта функция будет иметь параметр типа Stack. При вызове она возвращает верхний элемент в стёке, но не модифицирует стёк.

(b) Повторите часть (a), на этот раз изменив `stackADT2.c`.

(c) Повторите часть (a), изменив `stackADT3.c` в этом файле.

7. Измените `stackADT2.c` так, чтобы размер стёка автоматически удваивался при заполнении. Пусть функция `push` динамически выделяет новый массив, который вдвое больше старого, а затем копирует содержимое стёка из старого массива в новый. Не забудьте, что после копирования данных старый массив будет освобождён.

Проекты по программированию

1. Измените проект по программированию 1 из главы 10 так, чтобы он использовал стёк ADT, описанный в разделе 19.4. Вы можете использовать любую из реализаций ADT, описанных в этом разделе.

2. Измените проект по программированию 6 из главы 10 так, чтобы он использовал стёк ADT, описанный в разделе 19.4. Вы можете использовать любую из реализаций ADT, описанных в этом разделе.

3. Измените файл `stackADT2.c` раздела 19.4, добавив в структуру `stack_type` элемент `int` с именем `len`. Этот член будет отслеживать, сколько элементов в настоящее время хранится в стёке. Добавьте новую функцию с именем `length`, которая имеет параметр `Stack` и возвращает значение члена `len`. (Некоторые из существующих функций в `stackADT3.c` также

необходимо будет изменить.) Измените `stackclient.c` так, чтобы он вызывал функцию `length` (и отображал возвращаемое ею значение) после каждой операции, изменяющей стёк.

4. Измените файлы `stackADT.h` и `stackADT3.c` из Раздела 19.4 так, чтобы в стёке хранились значения типа `void *`, как описано в Разделе 19.5: тип `Item` больше не будет использоваться. Измените `stackclient.c` так, чтобы он хранил указатели. для строк в стёках `s1` и `s2`.

5. Начиная с заголовка `queue.h` упражнения 1, создайте файл с именем `queueADT.h`, который определяет следующий тип `Queue`:

```
typedef struct queue_type * Queue;
```

`queue_type` - это неполный структурный тип. Создайте файл с именем `queueADT.c`, который содержит полное определение `queue_type`, а также определения всех функций в `queue.h`.

Используйте фиксированную длину, чтобы хранить элементы в очереди (см. Упражнение 3 (а)). Создайте файл с именем `queueclient.c` (похожий на файл `stackclient.c` из Раздела 19.4), который создаёт две очереди и выполняет над ними операции. Не забудьте предоставить функции создания и удаления для вашего ADT.

6. Измените проект по программированию 5 так, чтобы элементы в очереди хранились в динамически выделяемом массиве, длина которого передаётся в функцию create.

7. Измените проект по программированию 5 так, чтобы элементы в очереди хранились в связанном списке (см. Упражнение 3 (b)).

20. Низкоуровневое программирование

Язык программирования имеет низкий уровень, если в программах приходится уделять внимание несущественному.

В предыдущих главах были описаны высокоуровневые, машинно-независимые функции языка Си. Хотя эти функции подходят для многих приложений, некоторые программы должны выполнять операции на уровне битов.

Битовые манипуляции и другие низкоуровневые операции особенно полезны для написания системных программ (включая компиляторы и операционные системы), программ шифрования, графических программ и программ, для которых решающее значение имеет быстрое выполнение и/или эффективное использование пространства. Раздел 20.1 охватывает побитовые

операторы языка Си, которые обеспечивают лёгкий доступ как к отдельным битам, так и к битовым полям. Затем в разделе 20.2 показано, как объявлять структуры, содержащие битовые поля.

Наконец, в разделе 20.3 описывается, как некоторые обычные функции языка Си (определения типов, объединения и указатели) могут помочь в написании низкоуровневых программ. Некоторые из методов, описанных в этой главе, зависят от знаний о том, как данные хранятся в памяти, которые могут варьироваться в зависимости от машины и компилятора.

Полагаясь на эти методы, скорее всего, вы сделаете программу непереносимой, поэтому лучше всего избегать их без крайней необходимости. Если они вам действительно нужны, постарайтесь ограничить их использование определёнными модулями в вашей программе; не распространяйте их повсюду. И, прежде всего, обязательно документируйте то, что вы делаете!

29.1 Побитовые операторы

С предоставляет шесть битовых операторов, которые работают с целочисленными данными на битовом уровне. Сначала мы обсудим два побитовых оператора сдвига, а затем четыре других побитовых оператора (побитовое дополнение, побитовое И, побитовое исключающее ИЛИ и побитовое включающее ИЛИ).

Операторы побитового сдвига

Побитовые операторы shift могут преобразовывать двоичное представление целого числа, сдвигая его биты влево или вправо. C предоставляет два оператора сдвига, которые показаны в таблице 20.1.

Символ	Означает
<<	Сдвиг влево
>>	Сдвиг вправо

Таблица 20.1 Побитовые операторы сдвига

Операнды для << и >> могут быть любого целочисленного типа (включая char). Целочисленные продвижения выполняются для обоих операндов; результат имеет тип левого операнда после продвижения. Значение $i \ll j$ - это результат, когда биты в i сдвигаются влево на j позиций. Для на место каждого бита, который "сдвинут" с левого конца i , приходит нулевой бит справа. Значение $i \gg j$ является результатом сдвига i вправо на j позиций. Если i имеет беззнаковый тип или если значение i неотрицательно, нули добавляются слева по мере необходимости. Если i - отрицательное число, результат определяется реализацией; некоторые реализации добавляют нули в левый конец, в то время как другие сохраняют бит знака, добавляя единицы. Для переносимости лучше выполнять сдвиги только беззнаковых чисел. Следующие примеры иллюстрируют эффект применения операторов сдвига к числу 13.

(Для простоты в этих и других примерах в этом разделе используются целые числа типа short, которые обычно составляют 16 бит.)

unsigned short i, j,

1149

```
i = 13; /* i is now 13 (binary 0000000000001101) */
```

```
j = i << 2; /* j is now 52 (binary 0000000000110100) */
```

```
j = i >> 2; /* j is new 3 (binary 0000000000000011) */
```

Как показывают эти примеры, ни один из операторов не изменяет свои операнды. Чтобы изменить переменную путём сдвига её битов, мы будем использовать составные операторы присваивания `<< =` и `>> =`:

```
i = 13; /* i is now 13 (binary 0000000000001101) */
```

```
i <<= 2; /* i is now 52 (binary 0000000000110100) */
```

```
i >>= 2; /* i is now 13 (binary 0000000000001101) */
```

Операторы побитового сдвига имеют более низкий приоритет, чем арифметические операторы, что может вызвать сюрпризы. Например, `i << 2 + 1` означает `i << (2 + 1)`, а не `(i << 2) + 1`.

Побитовое дополнение, И, ИЛИ, исключающее ИЛИ

В таблице 20.2 перечислены остальные побитовые операторы.

Символ	Что означает
~	Побитовое НЕ (отрицание, дополнение к 1)
&	Побитовое И
^	Побитовое исключающее ИЛИ
	Побитовое (включающее) ИЛИ

Таблица 20.2 Остальные побитовые операции

Оператор $\sim$ унарный; для его операнда выполняются целочисленные продвижения. Остальные операторы являются бинарными; для их операндов выполняются обычные арифметические преобразования. Операторы $\sim$, $\&$, $\wedge$ и $|$ выполняют логические операции со всеми битами в своих операндах. Оператор $\sim$ создаёт дополнение своего операнда, в котором нули заменяются единицами, а единицы заменяются нулями. Оператор $\&$ выполняет логическую операцию И для всех соответствующих битов в своих двух операндах.

Операторы $\wedge$ и $|$ аналогичны (оба выполняют логическую операцию или над битами в своих операндах); однако $\wedge$ производит 0 всякий раз, когда оба операнда имеют бит равный 1, тогда как $|$ производит 1. Не путайте побитовые операторы $\&$ и $|$ с логическими операторами $\&\&$ и $||$.

Побитовые операторы иногда дают те же результаты, что и логические операторы, но они не эквивалентны. Следующие примеры иллюстрируют действие операторов $\sim$, $\&$, $\wedge$ и $|$:

```
unsigned short i, j, k;
```

```
i = 21; /* i is now 21 (binary 0000000000010101) */
```

```
j = 56; /* j is now 56 (binary 0000000000111000) */
```

```
k = ~i; /* k is now 65514 (binary 111111111101010) */
```

```
k = i & j; /* k is now 16 (binary 0000000000010000) */
```

```
k = i ^ j; /* k is now 45 (binary 0000000000101101) */
```

1151

`k = i | j; /* k is now 61 (binary 0000000000111101) */`

Значение, показанное для $\sim i$, основано на предположении, что значение `unsigned short` занимает 16 бит.

Оператор $\sim$ заслуживает особого упоминания, поскольку с его помощью мы можем сделать даже низкоуровневые программы более переносимыми. Предположим, что нам нужно целое число, все биты которого равны 1.

Предпочтительный метод - записать ~ 0 , который не зависит от количества битов в целом числе. Точно так же, если нам нужно целое число, все биты которого равны 1, за исключением последних пяти, мы можем написать $\sim 0x1f$.

Каждый из операторов $\sim$, $\&$, $\wedge$ и $|$ имеет свой приоритет:

Наивысший: $\sim$

$\&$

$\wedge$

Самый низкий: $|$

В результате мы можем комбинировать эти операторы в выражениях без использования скобок.

Например, мы могли бы написать `i & ~j | k` вместо

`(i & (~j)) | k` и `i ^ j & ~k` вместо `i ^ (j & (~k))`.

Конечно, не повредит использовать круглые скобки, чтобы избежать путаницы. Приоритет $\&$, $\wedge$ и $|$ ниже, чем приоритет

1152

операторов отношения и равенства. Следовательно, утверждения, подобные следующему, не будут иметь желаемого эффекта:

если (status & 0x4000 != 0)...

Вместо проверки того, что status & 0x4000 не равен нулю, этот оператор оценит 0x400 (который имеет значение 1), а затем проверит, не равно ли значение status & 1 нулю.

Операторы составного присваивания & =, ^ = и | = соответствуют поразрядным операторам &, ^ и |.

i =21; /* i is now 21 (binary 0000000000010101) */

j =56 /* j is now 56 (binary 0000000000111000) */

i &= j /* i is now 16 (binary 0000000000010000) */

i ^= j /* i is now 40 (binary 0000000000101000) */

i |= j /* i is now 56 (binary 0000000000111000) */

Использование побитовых операторов для доступа к битам

При низкоуровневом программировании нам часто приходится хранить информацию в виде отдельных битов или наборов битов. В графическом программировании, например, мы можем захотеть сжать два или более пикселей в один байт. Используя побитовые операторы, мы можем извлекать или

изменять данные, хранящиеся в небольшом количестве битов. Предположим, что *i* - это 16 - битовая переменная unsigned short. Давайте посмотрим, как выполнять наиболее распространённые однобитовые операции над *i*:

- **Установка бита.** Предположим, что мы хотим установить 4-й бит в *i*, (мы предположим, что самый левый - или самый значительный - бит имеет номер 15, а самый младший - номер 0.) Самый простой способ установить 4-й бит - это установить значение *i* с помощью константы 0x0010 ("маска", содержащая 1 бит в позиции 4):

```
i = 0x0000; /* i is now 0000000000000000 */
```

```
i |= 0x0010; /* i is now 00000000000010000 */
```

В более общем случае, если позиция бита хранится в переменной *j*, мы можем использовать оператор сдвига для создания маски:

идиома `i |= 1 << j; /* устанавливает бит j */`

Например, если *j* имеет значение 3, то `1 << j` равно 0x0008.

- **Очистка бита.**

Чтобы очистить 4-й бит из *i*, мы использовали бы маску с битом 0 в позиции 4 и битом 1 везде:

```
i = 0x00ff; /* i теперь 0000000011111111 */
```

`i &= ~ x0010; /* i теперь 0000000011101111 */`

Используя ту же идею, мы можем легко написать инструкцию, который очищает бит, позиция которого сохраняется в переменной:

идиома `i & ~(1 << j); /* очистка бита j */`

- **Тестирование бит.** Следующий оператор `if` проверяет, установлен ли 4-й бит в *i*:

`if (i & 0x0010)... /* tests bit: 4 */`

- Чтобы проверить, установлен ли бит *j*, мы используем следующую инструкцию:

идиома `if (i & 1 << j) /* tests bit j */`

Чтобы облегчить работу с битами, мы часто даём им имена. Например, предположим, что мы хотим, чтобы битовые числа 0, 1 и 2 соответствовали цветам синий, зелёный и красный соответственно. Во-первых, мы определяем имена, представляющие три битовые позиции:

```
#define BLUE 1
```

```
#define GREEN 2
```

```
#define RED 4
```

Установка, очистка и проверка СИНЕГО бита будут выполняться следующим образом:

```
i | BLUE; /* sets BLUE bit */
```

1155

```
i &= ~BLUE; /* clears BLUE bit */
```

```
if (i & BLUE).../* tests BLUE bit */
```

Также легко установить, очистить или протестировать несколько бит одновременно:

```
i | BLUE | GREEN; /* sets BLUE and GREEN bits */
```

```
i & ~(BLUE | GREEN); /* clears BLUE and GREEN bits */
```

```
if (i & (BLUE | GREEN)) ; /* tests BLUE and GREEN bits */
```

Инструкция if проверяет, установлен ли бит BLUE или бит GREEN.

Использование побитовых операторов для доступа к битовым полям

Работа с группой из нескольких последовательных совпадений (битовое поле) немного сложнее, чем работа с одним битовым значением. Вот примеры двух наиболее распространённых операций с битовым полем:

Изменение битового поля.

Для изменения битового поля требуется поразрядное И (для очистки битового поля), за которым следует поразрядное ИЛИ (для сохранения новых битов в битовом поле). Следующая инструкция показывает, как мы можем сохранить двоичное значение 101 в битах 4-6 переменной.

1156

```
i = i & ~0x0070 | 0x0050; /* stores 101 in bits 4-6 */
```

Оператор `&` очищает биты 4-6 из *i*; затем оператор `|` устанавливает биты 6 и 4. Обратите внимание, что `i | = 0x0050` сам по себе не всегда будет работать: он установит биты 6 и 4, но не изменит бит 5. Чтобы немного обобщить пример, предположим, что переменная *j* содержит значение, которое должно быть сохранено в битах 4-6 в *i*. Нам нужно будет сдвинуть *j* в позицию перед выполнением побитового ИЛИ:

```
i = (i & ~0x0070) | (j << 4); /* stores j in bits 4-6 */
```

Оператор `|` имеет более низкий приоритет, чем `&` и `<<`, поэтому, если захотим, можем опустить скобки:

```
i=i & ~0x0070 | j << 4;
```

Получение битового поля.

Когда битовое поле находится в правом конце числа (в наименее значимых битах), получить его значение легко. Например, следующая инструкция извлекает биты 0 - 2 в переменной *i*:

```
j = i & 0x0007; /* извлекает биты 0 - 2 */
```

Если битовое поле не находится в правом конце *i*, то мы можем сначала сдвинуть битовое поле в конец, прежде чем извлекать поле с помощью оператора `&`. Чтобы извлечь биты

1157

4 - 6 из i , например, мы могли бы использовать следующее утверждение:

$j = (i \gg 4) \& 0x0007$; /* извлекает биты 4-6 */

ПРОГРАММА Шифрование XOR

Один из простейших способов шифрования данных заключается в применении исключающего или (XOR) для каждого символа с помощью секретного ключа.

Предположим, что ключ - это символ `&`: если мы применим XOR к этому ключу и символу `z`, мы получим символ `\` (предполагая, что мы используем набор символов ASCII):

00100110 (ASCII code for `&`)

XOR 01111010 (ASCII code for `z`)

01011100 (ASCII code for `\`)

Чтобы расшифровать сообщение, мы просто применяем тот же алгоритм, другими словами, шифруя уже зашифрованное сообщение, мы восстановим исходное сообщение. Например, если мы заменим символ `&` символом `\`, то получим исходный символ:

1158

00100110 (ASCII code for &)

XOR 01011100 (ASCII code for \)

01111010 (ASCII code for z)

Следующая программа, xor.c, шифрует сообщение, применяя XOR к каждому символу с символом &. Исходное

сообщение может быть введено пользователем или прочитано из файла с помощью перенаправления ввода; зашифрованное сообщение может быть просмотрено на экране или сохранено в файле с помощью перенаправления вывода. Например, предположим, что файл msg содержит следующие строки:

Trust not him with your secrets, who, when left alone in your room, turns over your papers.

--Johann Kaspar Lavater (1741—1801)

Чтобы зашифровать файл msg, сохранив зашифрованное сообщение в newmsg, мы должны использовать следующую команду:

```
xor < msg > newmsg
```

newmsg теперь будет содержать следующие строки:

rTSUR HIR NOR QORN _IST UCETCRU, QNI, QNCH JC@R
GJIHC OH _IST TIHK, RSTHU IPCT _IST VGVCTU.

--1INGHH mGUVGT jGPGRCT (1741-1801)

Чтобы восстановить исходное сообщение, мы воспользуемся командой:

```
xor < newmsg
```

которая отобразит его на экране. Как показывает пример, наша программа не изменит некоторые символы, в том числе цифры. Выполнение XOR на этих символах с помощью & приведёт к появлению невидимых управляющих символов, что может вызвать проблемы с некоторыми операционными системами. В главе 22 мы увидим, как избежать проблем при чтении и записи файлов, содержащих управляющие символы. А пока мы будем осторожны, используя функцию isprint, чтобы убедиться, что и исходный, и новый (зашифрованный) символ печатные символы (т. е. не управляющие символы). Если какой-либо из символов не прошёл этот тест, программа запишет исходный символ вместо нового. Вот готовая программа, которая очень короткая:

```
//xor.c
```

```
//выполнить xor шифрование
```

```
#include <ctype.h>
```

```
#include <stdio.h>
```

```
#define KEY '&'
```

```
int main(void){
```

```
    int orig_char, new_char;
```

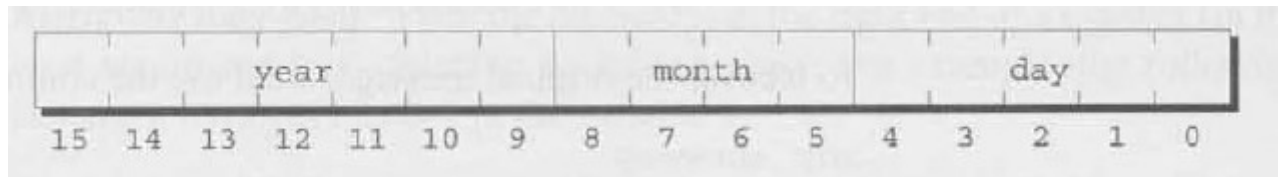
1160

```
while ((orig_char = getchar()) != EOF) {  
    new_char = orig_char ^ KEY;  
    if (isprint(orig_char) && isprint(new_char))  
        putchar(new_char);  
    else  
        putchar(orig_char);  
}  
return 0;  
}
```

20.2 Битовые поля в структурах

Хотя методы из Раздела 20.1 позволяют нам работать с битовыми полями, эти методы могут быть сложными в использовании и потенциально сбивать с толку. К счастью, C предоставляет альтернативу: объявление структур, члены которых представляют битовые поля. В качестве примера давайте посмотрим, как операционная система MS-DOS (часто называемая DOS) хранит дату создания или последнего изменения файла. Поскольку дни, месяцы и годы - это небольшие числа, их хранение в виде обычных целых чисел приведёт к напрасной трате памяти. Вместо этого DOS выделяет только 16 бит для даты, 5 бит для дня, 4 бита для месяца и 7 бит для года:

1161



Используя битовые поля, мы можем определить структуру C с идентичной компоновкой:

```
struct file_date {  
    unsigned int day: 5;  
    unsigned int month: 4;  
    unsigned int year: 7;  
};
```

Число после каждого члена указывает его длину в битах.

Поскольку все члены имеют один и тот же тип, мы можем сжать объявление, если захотим:

```
struct file_date {  
    unsigned int day: 5, month: 4, year: 7;  
};
```

Тип битового поля должен быть `int`, `unsigned int` или `signed int`. Использование `int` (без `signed` или `unsigned`) неоднозначно: некоторые компиляторы обрабатывают бит старшего разряда поля как бит знака, а другие - нет. Объявляйте все битовые поля как `unsigned int` или `signed int`. В C99 битовые поля также могут иметь тип `_Bool`. Компиляторы C99 могут допускать

1162

дополнительные типы битовых полей. Мы можем использовать битовое поле так же, как любой другой член структуры, как показано в следующем примере:

```
struct file_date fd{  
    fd.day = 28;  
    fd.month = 12;  
    fd.year = 8; /* represents 1988 */  
};
```

Обратите внимание, что член `year` хранится отсчитанным относительно 1980 (года, когда мир начал своё существование, согласно Microsoft). После этих присваиваний переменная `fd` будет иметь следующий вид:



Мы могли бы использовать поразрядные операторы для достижения того же эффекта; использование этих операторов может даже сделать программу немного быстрее. Однако наличие доступной программы обычно важнее, чем выигрыш нескольких микросекунд. У битовых полей есть одно ограничение, которое не распространяется на другие

элементы структуры. Поскольку битовые поля не имеют адресов в обычном смысле, C не позволяет нам применять оператор адреса (&) к битовому полю. Из-за этого правила такие функции, как `scanf`, не могут сохранять данные непосредственно в битовом поле:

```
scanf("%d", &fd.day); /*** WRONG ***/
```

Как хранятся битовые поля

Давайте внимательно посмотрим, как компилятор обрабатывает объявление структуры, имеющей элементы с битовыми полями. Как мы увидим, стандарт C предоставляет компилятору значительную свободу в выборе способа хранения битовых полей.

Правила, касающиеся того, как компилятор обрабатывает битовые поля, зависят от понятия "единицы хранения". Размер единицы хранения определяется реализацией; типичные значения - 8, 16 и 32 бита. При обработке объявления структуры, компилятор упаковывает битовые поля одно за другим в блок памяти без промежутков между полями, пока не останется достаточного места для следующего поля. В этот момент некоторые компиляторы переходят к началу следующей единицы хранения, в то время как другие разделяют битовое поле по единицам хранения.

(То, что происходит, определяется реализацией.) Порядок, в котором распределяются битовые поля (слева направо или справа налево), также определяется реализацией. В нашем примере `file_date` предполагается, что единицы хранения имеют длину 16 бит. (8-битная единица хранения также была бы приемлемой при условии, что компилятор разделяет поле месяца между двумя единицами хранения.) Мы также предполагаем, что битовые поля распределяются справа налево (причём первое битовое поле занимает младшие биты). С позволяет нам опускать имя любого битового поля. Неименованные битовые поля полезны в качестве "дополнения", чтобы гарантировать, что другие битовые поля расположены должным образом. Рассмотрим время, связанное с файлом DOS, которое хранится следующим образом:

```
struct file_time {  
    unsigned int seconds:5;  
    unsigned int minutes:6;  
    unsigned int hours: 5;  
};
```

(Возможно, Вам интересно, как можно хранить секунды - число от 0 до 59 в поле всего с 5 битами. Ну, DOS обманывает: он делит количество секунд на 2, так что член

1165

seconds на самом деле находится между 0 и 29.) Если нас не интересует поле секунд, мы можем опустить его название:

```
struct file_time {  
    unsigned int: 5; /* not used */  
    unsigned int: minutes: 6;  
    unsigned int: hours: 5;  
};
```

Остальные битовые поля будут выровнены так, как если бы поле секунд все ещё присутствовало. ещё один трюк, который мы можем использовать для управления хранением битовых полей -это указание 0 в качестве длины безымянного битового поля;

```
struct s {  
    unsigned int a: 4;  
    unsigned int : 0; /* 0 - length bit-field */  
    unsigned int b: 8;
```

Битовое поле нулевой длины является сигналом для компилятора, чтобы выровнять следующее битовое поле с начала следующей единицы хранения. Если единицы хранения имеют длину 8 бит, компилятор выделит 4 бита для члена **a**, пропустит 4 бита, выйдет на следующую единицу хранения, а затем выделит 8 бит для **b**. Если единицы

хранения имеют длину 16 бит, компилятор выделит 4 бита для *a*, пропустит 12 бит, а затем выделит 8 бит для *b*.

20.3 Другие низкоуровневые техники

Некоторые языковые функции, которые мы рассмотрели в предыдущих главах, часто используются в низкоуровневом программировании. Чтобы завершить эту главу, мы рассмотрим несколько важных примеров: определение типов, представляющих единицы хранения, использование объединений для обхода обычной проверки типов и использование указателей в качестве адресов. Мы также рассмотрим квалификатор типа `volatile`, который мы избегали обсуждать в Разделе 18.3 из-за его низкоуровневой природы.

Определение машинно-зависимых типов

Поскольку тип `char` - по определению - занимает один байт, мы иногда будем рассматривать символы как байты, используя их для хранения данных, которые не обязательно имеют символьную форму. Когда мы это делаем, будет хорошей идеей определить тип `BYTE`:

```
typedef unsigned char BYTE;
```

В зависимости от машины мы можем определить дополнительные типы. В архитектуре x86 широко используются 16-битные слова, поэтому для этой платформы было бы полезно следующее определение:

```
typedef unsigned short WORD;
```

В следующих примерах мы будем использовать типы BYTE и WORD.

Использование объединений для предоставления нескольких представлений данных

Кроме использования объединений показанного на примерах в Разделе 16.4 - они часто используются в C для совершенно другой цели: просмотра блока памяти двумя или более различными способами. Вот простой пример, основанный на структуре `file_date`, описанной в разделе 20.2. Поскольку структура `file_date` уместается в два байта, мы можем рассматривать любое двухбайтовое значение как структуру `file_date`. В частности, мы могли бы рассматривать значение `short unsigned` как структуру `file_date` (предполагая, что целые числа `short` имеют длину 16 бит). Следующее

1168

объединение позволяет нам легко преобразовать короткое целое число в `file_date` или наоборот:

```
union int_date {  
    unsigned short i;  
    struct file_date fd;  
};
```

С помощью этого объединения мы могли бы извлечь `file_date` с диска в виде двух байтов, а затем извлечь его поля `month`, `day` и `year`. И наоборот, мы могли бы составить дату как структуру `file_date`, а затем записать её на диск в виде пары байтов. В качестве примера того, как мы могли бы использовать объединение `int_date`, определим функцию, которая при передаче аргумента `unsigned short` печатает его как дату файла:

```
void print_date(unsigned short n){  
    union int_date u;  
    u.i = n;  
    printf("%d/%d/td\n",u.fd.month, u.fd.day, u.fd.year + 1980);
```

Использование объединений для обеспечения нескольких представлений данных особенно полезно при работе с регистрами, которые часто делятся на более мелкие блоки, например, процессоры x86 имеют 16-разрядные регистры с именами AX, BX, CX и DX. Каждый из этих регистров можно

рассматривать как два 8-битных регистра. Например, АХ разделен на регистры с именами АН и АL. (Н и L обозначают "высокий" и "низкий".) При написании низкоуровневых приложений для компьютеров на базе x86 нам могут понадобиться переменные, которые представляют содержимое регистров АХ, ВХ, СХ и ДХ. Нам нужен доступ как для 16-, так и для 8-битных регистров; в то же время мы должны принять во внимание их взаимосвязь (изменение АХ влияет как на АН, так и на АL; изменение АН или АL модифицирует АХ). Решение состоит в том, чтобы создать две структуры, одна из которых содержит элементы, соответствующие 16-разрядным регистрам, а другая - элементы, соответствующие 8-разрядным регистрам. Затем мы создаём объединение, которое включает в себя две структуры:

```
union {  
    struct {  
        WORD ax, bx, cx, dx;  
    }word;  
    struct{  
        BYTE al, ah, bl, bh, cl, ch, dl, dh;  
    }byte;  
}regs;
```

1170

Элементы структуры WORD будут накладываться на элементы структуры BYTE; например, ax будет занимать ту же память, что и al и ah. И это, конечно, именно то, чего мы хотели. Вот пример, показывающий, как можно использовать объединение regs:

```
regs.byte.ah = 0x12;
```

```
regs.byte.al = 0x34;
```

```
printf("AX: %hx\n", regs.word.ax);
```

Изменение ah и al влияет на ax, поэтому выход будет

AX: 1234

Обратите внимание, что в структуре byte al расположен перед ah, даже несмотря на то, что регистр AL является "младшей" половиной AX, а AH - "старшей" половиной. Не без причины. Если элемент данных состоит из более чем одного байта, есть два логических способа сохранить его в памяти: байты в "естественном" порядке (с крайним левым байтом, сохраняемым первым "big-endian") или байты в обратном порядке (крайний левый байт хранится последним "little-endian"). Первая альтернатива называется обратным порядком байтов ("big-endian" - "тупоконечник"); второй известен как прямой порядок байтов ("little-endian" - "остроконечник"). С не требует особого порядка следования байтов, поскольку это зависит от ЦП, на котором будет

выполняться программа. Некоторые процессоры используют подход с прямым порядком байтов, а некоторые - с обратным порядком байтов. При чем здесь структура `byte`? Оказывается, процессоры x86 предполагают, что данные хранятся в обратном порядке, поэтому первый байт `regs.word.ax` - это младший байт. Обычно нам не нужно беспокоиться о порядке байтов. Однако программы, работающие с памятью на низком уровне, должны знать порядок, в котором хранятся байты (как показывает пример `regs`). Это также актуально при работе с файлами, которые содержат несимвольные данные. Будьте осторожны при использовании объединений для предоставления нескольких представлений данных. Данные, которые действительны в исходном формате, могут оказаться недопустимыми, если рассматривать их как данные другого типа, что вызовет непредвиденные проблемы.

Использование указателей в качестве адресов

В Разделе 11.1 мы видели, что указатель на самом деле является своего рода адресом памяти, хотя обычно нам не нужно знать подробности. Однако, когда мы занимаемся низкоуровневым программированием, важны детали.

Адрес часто имеет то же количество битов, что и целое число (или длинное целое число). Создать указатель, представляющий конкретный адрес, легко: мы просто преобразуем целое число в указатель. Например, вот как мы

1172

можем сохранить адрес 1000 (шестнадцатеричный) в переменной-указателе:

```
BYTE * p;
```

```
p = (BYTE *) 0x1000; /* p содержит адрес 0x1000 */
```

Квалификатор типа volatile

На некоторых компьютерах определённые ячейки памяти являются "изменчивыми" ("volatile"); значение, хранящееся в таком месте, может изменяться во время работы программы, даже если сама программа не сохраняет там новые значения.

Например, некоторые области памяти могут содержать данные, поступающие непосредственно от устройств ввода. Квалификатор типа volatile позволяет нам информировать компилятор о том, что если какие-либо данные, используемые в программе, являются изменчивыми, volatile обычно появляется в объявлении переменной-указателя, которая будет указывать в область изменчивой памяти:

```
volatile BYTE * p; /* p будет указывать на изменчивый байт */
```

Чтобы понять, зачем нужен квалификатор volatile, предположим, что p указывает на ячейку памяти, которая содержит самый последний символ, введённый с клавиатуры пользователя. Это местоположение изменчиво: его значение

1173

меняется каждый раз, когда пользователь вводит символ. Мы могли бы использовать следующий цикл для получения символов с клавиатуры и сохранения их в массиве буферов:

```
while (buffer not full){  
    wait for input;  
    buffer[i] = *p;  
    if (buffer[i++] == '\n')  
        break;  
}
```

Нетривиальный компилятор может заметить, что этот цикл не изменяет ни p , ни $*p$, поэтому он может оптимизировать программу, изменив её так, чтобы $*p$ выбирался только один раз:

```
store *p in a register;  
while (buffer not full){  
    wait for input;  
    buffer[i] = value stored in register;  
    if (buffer[i++] == '\n')  
        break;  
}
```

Оптимизированная программа заполнит буфер множеством копий одного и того же символа — это не совсем то, что мы

имели в виду. Объявление того, что *r* указывает на изменчивые данные, позволяет избежать этой проблемы, сообщая компилятору, что **r* должен извлекаться из памяти каждый раз, когда это необходимо.

Вопросы и ответы

Вопрос: Что вы имеете в виду, говоря, что операторы *&* и *|* иногда дают те же результаты, что и операторы *&&* и *||*, но не всегда?

Ответ: Давайте сравним *i & j* с *i && j* (аналогичные замечания применимы к *|* и *||*). Пока *i* и *j* имеют значение 0 или 1 (в любой комбинации), оба выражения будут иметь некоторое значение. Однако если *i* и *j* должны иметь другие значения, то выражения могут не всегда совпадать. Например, если *i* равно 1, а *j* равно 2, то *i & j* имеет значение 0 (*i* и *j* не имеют соответствующих битов 1), а *i && j* имеет значение 1. Если *i* равно 3, а *j* равно 2, то *i & j* имеет значение 2, в то время как *i && j* имеет значение 1. Побочные эффекты - это ещё одно различие. Вычисление *i & j ++* всегда увеличивает *j* как побочный эффект, тогда как вычисление *i && j ++* иногда увеличивает *j*.

Вопрос: Кого волнует, как DOS хранит даты файлов? Разве DOS не умер?

Ответ: Для большей части, да. Однако до сих пор существует множество файлов, созданных много лет назад, даты которых хранятся в формате DOS. В любом случае даты файлов DOS - хороший пример того, как используются битовые поля.

Вопрос: Откуда берутся термины "прямой порядок байтов"("little-endian") и "обратный порядок байтов"("big-endian")?

Ответ: В романе Джонатана Свифта "Путешествия Гулливера" жители вымышленных государств Лилипутии и Блефуску постоянно спорят, открывать ли варёные яйца с тупого конца (big end) или с острого конца (little end). Выбор, конечно, произвольный, точно так же, как порядок байтов в элементе данных.

Упражнения

К Разделу 20.1

*1. Покажите результат работы каждого из следующих фрагментов программы. Предположим, что *i*, *j* и *k* - переменные unsigned short.

(a) *i* = 8; *j* = 9;

1176

```
printf("%d", i >> 1 + j >> 1);
```

(b) $i = 1;$

```
printf("%d", i & ~i);
```

(c) $i=2;j=1;k=9;$

```
printf("%d", ~i & j ^ k);
```

(d) $i=7;j=8 ;k= 9;$

```
printf("%d", i ^ j & k);
```

W2. Опишите простой способ "переключить" бит (изменить его с 0 на 1 или с 1 на 0). Проиллюстрируйте эту технику, написав оператор, который переключает бит 4 переменной i .

*3. Объясните, какое влияние следующий макрос оказывает на его аргументы. Вы можете предположить, что аргументы имеют один и тот же тип

```
# define M (x, y) ((x) ^ = (y), (y) ^ = (x), (x) ^ (y))
```

w4. В компьютерной графике цвета часто хранятся в виде трёх чисел, представляющих интенсивность красного, зелёного и синего цветов. Предположим, что для каждого числа требуется восемь бит и мы хотим сохранить все три

значения в одном целом числе long. Запишите макрос с именем MK_COLOR с тремя параметрами (интенсивностью красного, зелёного и синего). MK_COLOR должен возвращать long, в котором последние три байта содержат значения интенсивности красного, зелёного и синего цветов, значение красного в последнем байте, зелёного в предпоследнем байте.

5. Напишите макросы с именами GET_RED, GET_GREEN и GET_BLUE, которые при задании цвета в качестве аргумента (см. Упражнение 4) возвращают его 8-битные значения интенсивности красного, зелёного и синего цветов.

W6. (a) Используйте побитовые операторы, чтобы написать следующую функцию:

```
unsigned short swap_bytes (unsigned short i);
```

swap_bytes должна возвращать число, полученное в результате перестановки двух байтов в i (целые числа short занимают два байта на большинстве компьютеров). Например, если i имеет значение 0x1234 (00010010 00110100 в двоичном формате), тогда swap_bytes должен вернуть 0x3412 (001 10100 00010010 в двоичном формате).

Протестируйте свою функцию, написав программу, которая считывает число в шестнадцатеричном формате, а затем записывает число с заменёнными байтами:

Введите шестнадцатеричное число (до четырёх цифр): 1234

Число с переставленными байтами: 3412

1178

Подсказка: используйте преобразование % hx для чтения и записи шестнадцатеричных чисел.

(b) Уплотните функцию `swap_bytes`, чтобы её тело было одним оператором.

7. Запишите следующие функции:

```
unsigned int rotate_left(unsigned int i, int n);
```

```
unsigned int rotate_right(unsigned int i, int n);
```

`rotate_left` должна восстановить результат сдвига битов в *i* влево на *n* позиций, при этом биты, которые были "сдвинуты", перемещаются в правый конец *i*. (Например, вызов `rotate_left(0x12345678, 4)` должен возвращать `0x23456781`, если целые числа имеют длину 32 бита.) `rotate_right` аналогична, но она должна "вращать" биты вправо, а не влево.

W8. Пусть *f* будет следующей функцией:

```
unsigned int f(unsigned int i, int m, int n) {  
    return (1 >> (m + 1 - n)) & ~ (~ 0 << n);  
}
```

(a) Какое значение имеет `~ (~ 0 << n)`?

(б) Что делает эта функция?

9(a). Напишите следующую функцию:

1179

```
int count_ones (unsigned char ch);
```

count_ones должен возвращать число из 1 бит в ch.

(b) напишите функцию в части (a) без использования цикла.

10. Напишите такую функцию:

```
unsigned int reverse_bits(unsigned int n);
```

reverse_bits должна возвращать целое число без знака, биты которого совпадают с битами в n, но в обратном порядке.

11. Каждый из следующих макросов определяет положение одного бита внутри целого числа.

```
#define SHIFT_BIT 1
```

```
#define CTRL_BIT 2
```

```
#define ALT_BIT 4
```

Следующая инструкция должна проверить, установлен ли какой-либо из трёх битов, но она никогда не отображает указанное сообщение. Объясните, почему эта инструкция не работает и покажите, как её исправить. Предположите, что key_code является переменной int.

```
if (key_code & (SHIFT_BIT | CTRL_BIT | ALT_BIT) == 0)
```

```
    printf ("Клавиши - модификаторы не нажаты\n");
```

1180

12. Следующая функция предположительно объединяет два байта для формирования целого числа unsigned short.

Объясните, почему функция не работает, и покажите, как её исправить.

```
unsigned short create_short(unsigned char high_byte,unsigned
char low_byte){
    return high_byte << 8 + low_byte;
}
```

13 *. Если n - это переменная типа unsigned int, какое влияние оказывает следующий оператор на биты в n ?

$n \&= n - 1;$

Подсказка: рассмотрите влияние на n , если этот оператор выполняется более одного раза.

К Разделу 20.2

w14. При хранении в соответствии со стандартом IEEE floating-point, значение с плавающей точкой состоит из 1-бита знака (крайний левый или самый значимый бит), 8-бит показателя степени и 23 бит мантииссы в указанном порядке. Разработайте тип структуры, занимающий 32 бита, с членами битового поля, конкурирующими за знак, показатель степени и дробь.

1181

Объявите битовые поля, чтобы они имели тип `unsigned int`. Обратитесь к руководству вашего компилятора, чтобы определить порядок битовых полей.

* 15. (a) Предположим, что переменная `s` была объявлена следующим образом:

```
struct {  
    int flag: 1;  
}s;
```

В некоторых компиляторах выполнение следующих инструкций приводит к отображению 1, но в других компиляторах результат равен -1. Объясните причину такого поведения.

```
s.flag = 1;  
printf ("%d \n", s.flag);
```

(b) как можно избежать этой проблемы?

К Разделу 20.2

16. Начиная с процессора 386, процессоры x86 имеют 32-разрядные регистры с именами EAX, EBX, ECX и EDX. Вторая половина (наименее значимые биты) этих регистров совпадает с AX, BX, CX и DX, соответственно. Измените объединение `re1gs` так, чтобы оно включало эти регистры, а также более старые. Ваше объединение должно быть

настроено так, чтобы изменение EAX изменяло AX, а изменение AX изменяло вторую половину EAX.

(Другие новые регистры будут работать аналогичным образом.) Вам нужно будет добавить несколько "фиктивных" членов в структуры word и byte, соответствующие другой половине EAX, EBX, ECX и EDX. Объявите тип новых регистров как DWORD (двойное слово, которое должно быть определено как unsigned long. Не забывайте, что архитектура x86-это little-endian-"остроконечник").

Проекты по программированию

1. Создайте объединение, позволяющее рассматривать 32-разрядное значение либо как float, либо как структуру, описанную в упражнении 14. Напишите программу, которая хранит 1 в поле знака структуры, 128 в поле экспоненты и 0 в поле мантииссы, а затем печатает значение float, хранящееся в объединении. (Ответ должен быть -2.0, если вы правильно настроили битовые поля.)

21 Стандартная библиотека

Каждая программа является частью другой программы и редко соответствует ей.

В предыдущих главах мы рассматривали библиотеку C по частям; в этой главе рассматривается библиотека в целом. В Разделе 21.1 приведены общие рекомендации по использованию библиотеки. В нем также описывается трюк, используемый в заголовках некоторых библиотек: использование макроса для "сокрытия" функции. Раздел 21.2 даёт обзор каждого заголовка в библиотеке C89; Раздел 21.3 делает то же самое для новых заголовков в библиотеке C99. В последующих главах подробно рассматриваются заголовки библиотеки, а связанные заголовки сгруппированы по главам. Заголовки `<stddef.h>` и `<stdbool.h>` очень краткие, поэтому я решил обсудить их в этой главе (в разделах 21.4 и 21.5 соответственно).

21.1 Использование библиотеки

Стандартная библиотека C89 разделена на 15 частей, каждая из которых описывается заголовком. C99 имеет ещё девять заголовков. всего 24 (см. Таблицу 21.1).

<assert.h>	<inttypes.h>*	<signal.h>	<stdlib.h>
<complex.h>*	<iso646.h>*	<stdarg.h>	<string.h>
<ctype.h>	<limits.h>	<stdbool.h>*	<tgmath.h>*
<errno.h>	<locale.h>	<stddef.h>	<time.h>
<fenv.h>*	<math.h>	<stdint.h>*	<wchar.h>*
<float.h>	<setjmp.h>	<stdio.h>	<wctype.h>*

C99-*

Таблица 21.1

Большинство компиляторов поставляются с более обширной библиотекой, которая может иметь много заголовков, не представленных в Таблице 21.1. Разумеется, дополнительные заголовки не являются стандартными, поэтому мы не можем рассчитывать на то, что они будут доступны в других компиляторах.

Эти заголовки часто предоставляют функции, специфичные для конкретного компьютера или операционной системы (что объясняет, почему они не являются стандартными).

Они могут предоставлять функции, позволяющие лучше контролировать экран и клавиатуру.

Заголовки, которые поддерживают графику или оконный пользовательский интерфейс, также распространены.

Стандартные заголовки состоят в основном из прототипов функций, определений типов и определений макросов.

Если один из наших файлов содержит вызов функции, объявленной в заголовке, или использует один из определённых в нем типов или макросов, нам нужно будет включить заголовок в начало файла.

Если файл включает несколько стандартных заголовков, порядок директив `#include` значения не имеет. Также разрешено включать стандартный заголовок более одного раза.

Ограничения на имена, используемые в библиотеке

Любой файл, содержащий стандартный заголовок, должен подчиняться нескольким правилам. Во-первых, он не может использовать имена макросов, определённых в этом заголовке, для каких-либо других целей. Например, если файл включает `<stdio.h>`, он не может повторно использовать `NULL`, поскольку макрос с таким именем уже определён в `<stdio.h>`. Во-вторых, имена библиотек с файловой областью (в частности, имена `typedef`) не могут быть переопределены на уровне файла. Таким образом, если файл включает `<stdio.h>`, он не может определить `size_t` как идентификатор с

областью видимости файла, поскольку `<stdio.h>` определяет `size_t` как имя typedef.

Хотя эти ограничения довольно очевидны, у C есть другие ограничения, которых вы не могли ожидать:

Идентификаторы, начинающиеся с символа подчёркивания, за которым следует заглавная буква или второй символ подчёркивания, зарезервированы для использования в библиотеке; программы никогда не должны использовать имена в этой форме для каких-либо целей.

Идентификаторы, начинающиеся с подчёркивания, зарезервированы для использования в качестве идентификаторов и тегов с областью видимости файла. Никогда не используйте такое имя в своих целях, если оно не объявлено внутри функции.

Каждый идентификатор с внешней связью в стандартной библиотеке зарезервирован для использования в качестве идентификатора с внешней связью. В частности, зарезервированы имена всех стандартных библиотечных функций. Таким образом, даже если файл не включает `<stdio.h>`, он не должен определять внешнюю функцию с именем `printf`, поскольку в библиотеке уже есть функция с таким именем.

Эти правила применимы к каждому файлу в программе, независимо от того, какие заголовки содержит файл.

Хотя эти правила не всегда соблюдаются, несоблюдение их может привести к тому, что программа не будет переносимой. Перечисленные выше правила применимы не только к именам, используемым в настоящее время в библиотеке, но и к именам, отложенным для дальнейшего использования. Полное описание того, какие имена зарезервированы, довольно длинное; вы найдёте его в стандарте C в разделе "Будущие направления библиотек". Например, C резервирует идентификаторы, которые начинаются с `str`, за которым следует строчная буква, так что функции с такими именами могут быть добавлены в заголовок `<string.h>`.

Функции, скрытые макросами

Программисты на C часто заменяют небольшие функции параметризованными макросами. Такая практика встречается даже в стандартной библиотеке. Стандарт C позволяет заголовкам определять макросы, имена которых совпадают с именами библиотечных функций, но защищает программиста, требуя, чтобы также была доступна настоящая функция.

В результате для заголовка библиотеки нередко объявлять функцию и определять макрос с тем же именем. Мы уже видели пример макроса, дублирующего библиотечную функцию: `getchar` - это библиотечная функция, объявленная в `<stdio.h>` в заголовке. Она имеет следующий прототип:

1188

```
int getchar (void);
```

<stdio.h> обычно также определяет getchar как макрос:

```
#define getchar() getc (stdin)
```

По умолчанию вызов `getchar` будет рассматриваться как вызов макроса (поскольку макроимена заменяются во время предварительной обработки). В большинстве случаев нам удобнее использовать макрос вместо истинной функции, потому что это, вероятно, заставит нашу программу работать быстрее. Однако иногда нам нужна настоящая функция, возможно, чтобы минимизировать размер исполняемого кода. Если возникает необходимость, мы можем удалить определение макроса (таким образом получив доступ к истинному действию) с помощью директивы `#undef`.

Например, мы можем отменить определение макроса `getchar` после включения `stdio.h`:

```
#include <stdio.h>
```

```
#undef getchar
```

Если `getchar` не является макросом, никакого вреда не будет; `#undef` не действует, если ему присвоено имя, которое не определено как макрос. В качестве альтернативы мы можем отключить индивидуальное использование макроса, заключив его имя в круглые скобки:

```
ch = (getchar) (); /* вместо ch = getchar(); */
```


Препроцессор не может обнаружить параметризованный макрос, если за его именем не стоит левая скобка. Однако компилятор не так легко обмануть; он все ещё может распознавать `getchar` как функцию.

21.2 Обзор библиотеки C 89

Теперь мы кратко рассмотрим заголовки в стандартной библиотеке C89. Этот раздел может служить "дорожной картой", которая поможет вам определить, какая часть библиотеки вам нужна. Каждый заголовок подробно описывается далее в этой главе или в следующей главе.

<assert.h> Диагностика

Содержит только макрос `assert`, который позволяет нам вставлять самопроверки в программу. Если какая-либо проверка не удалась, программа завершается.

<ctype.h> Обработка символов

Предоставляет функции для классификации символов и преобразования букв из нижнего в верхний регистр и наоборот.

<errno.h> Ошибки

Предоставляет `errno` ("номер ошибки"), `lvalue`, которое можно проверить после вызова определённых библиотечных

функций, чтобы узнать, произошла ли ошибка во время вызова.

<float.h> Характеристики плавающих типов

Предоставляет макросы, описывающие характеристики плавающих типов, включая их диапазон и точность.

<limits.h> Размеры целочисленных типов

Предоставляет макросы, описывающие характеристики целочисленных типов (включая типы символов), включая их максимальные и минимальные значения.

<locale.h> Локализация

Предоставляет функции, помогающие программе адаптировать своё поведение к стране или другому географическому региону. Специфическое для локали поведение включает способ печати чисел (например, символ, используемый в качестве десятичной точки), формат денежных значений (например, символ валюты), набор символов и внешний вид даты и времени.

<math.h> Математика

Предоставляет общие математические функции, в том числе тригонометрические, гиперболические, экспоненциальные, логарифмические, степенные, ближайшие целые, абсолютные значения и функции остатка.

<setjmp.h> Нелокальные прыжки

Предоставляет функции `setjmp` и `longjmp`.

`setjmp` "помечает" место в программе; `longjmp` затем может быть использована для возврата к этому месту позже. Эти функции позволяют переходить от одной функции к другой, все ещё активной функции, минуя обычный механизм функции `return`.

`setjmp` и `longjmp` используются в основном для решения серьёзных проблем, возникающих при выполнении программы.

< *signal.h* > Обработка сигналов

Предоставляет функции, которые имеют дело с исключительными условиями (сигналами), включая прерывания и ошибки во время выполнения.

Функция `signal` устанавливает функцию, которая будет вызвана, если позже должен появиться данный сигнал.

Функция `raise` вызывает появление сигнала.

< *stdarg.h* > Переменное число аргументов

Предоставляет инструменты для написания функций, которые, подобно `printf` и `scanf`, могут иметь переменное количество аргументов.

<stddef.h> Общие определения

Предоставляет определения часто используемых типов и макросов.

<stdio.h> Ввод / вывод

Предоставляет большой набор функций ввода / вывода, включая операции с файлами как с последовательным, так и с произвольным доступом.

<stdlib.h> Общие утилиты

"Сборный" заголовок для функций, которые не подходят ни к одному из других заголовков. Функции в этом заголовке могут преобразовывать строки в числа, генерировать псевдослучайные числа, выполнять задачи управления памятью, взаимодействовать с операционной системой, выполнять поиск и сортировку. и выполнять преобразования между многобайтовыми символами и широкими символами.

<stdio.h> Обработка строк

Предоставляет функции, выполняющие строковые операции, включая копирование, объединение, сравнение и поиск, а также функции, которые работают с произвольными блоками памяти.

<time.h> Дата и время

Предоставляет функции для определения времени (и даты, управления временем и форматирования времени для отображения.

21.3 Изменения библиотеки C99

Некоторые из самых больших изменений в C99 коснулись стандартной библиотеки. Эти изменения делятся на три группы:

- ***Дополнительные заголовки.*** В стандартной библиотеке C99 есть девять заголовков, которых нет

в C89. Три из них (<isos46.h>, <wchar.h> и <wctype.h>) были фактически добавлены в C в 1995 году, когда в стандарт C89 были внесены поправки. Остальные шесть (<complex.h>, <fenv.h>, <inttypes.h>, <stdbool.h>, <stdint.h> и <tgmath.h>) являются новыми в C99.

- ***Дополнительные макросы и функции.*** Стандарт C99 добавляет макросы и функции к нескольким существующим заголовкам, в первую очередь (<float.h>, <math.h> и <stdio.h>). Дополнения к заголовку <math.h> настолько обширны, что они рассматриваются в отдельном разделе (раздел 23.4).
- ***Улучшенные версии существующих функций.*** Некоторые существующие функции, включая printf и scanf, имеют дополнительные возможности в C99.

Теперь мы кратко рассмотрим девять дополнительных заголовков в стандартной библиотеке C99, точно так же, как мы сделали в Разделе 21.2 для заголовков в библиотеке C89.

< complex.h> *Комплексная арифметика*

Определяет макросы `complex` и `I`, которые полезны при работе с комплексными числами.

Также предусмотрены функции для выполнения математических операций над комплексными числами.

<fenv.h> Среда с плавающей точкой

Обеспечивает доступ к флагам состояния с плавающей запятой и режимам управления. Например, программа может проверить флаг, чтобы увидеть, произошло ли переполнение во время операции с плавающей точкой, или установить режим управления, чтобы указать, как должно выполняться округление.

<inttypes.h> преобразование формата целочисленных типов

Определяет макросы, которые могут использоваться в строках формата для ввода/вывода целочисленных типов, объявленных в `<stdint.h>`. Также предоставляет функции для работы с целыми числами наибольшей ширины.

<iso646.h> Альтернативное написание

1196

Определяет макросы, которые представляют определённые операторы (те, которые содержат символы &, |, ~, ! и ^).

Эти макросы полезны для написания программ в среде, где эти символы могут не быть частью локального набора символов.

<stdbool.h> Логический тип и значения

Определяет макросы типа bool, true и false, а также макрос, который можно использовать для проверки того, были ли определены эти макросы.

<stdint.h> Целочисленные типы

Объявляет целочисленные типы с указанной шириной и определяет связанные макросы (например, макросы, которые определяют максимальные и минимальные значения каждого типа). Также определяет параметризованные макросы, которые создают целочисленные константы с определёнными типами.

<tgmath.h> Типовая математика

В C99 есть несколько версий многих математических функций в заголовках <math.h> и <complex.h>. Макросы

1197

"универсального типа" в `<tgmath.h>` могут определять типы переданных им аргументов и заменять вызов соответствующей функции `<math.h>` или `<complex.h>`.

<wchar.h> Утилиты расширенных многобайтовых и широких символов

Предоставляет функции для ввода-вывода широких символов и манипулирования широкими строками.

<wctype.h> Утилиты классификации и сопоставления широких символов

Версия `<ctype.h>` для расширенных символов. Предоставляет функции для классификации и изменения регистра широких символов.

21.4 заголовок `<stddef.h>`: Общие определения

Заголовок `<stddef.h>` содержит определения часто используемых типов и макросов; он не объявляет никаких функций. Эти типы таковы:

`ptrdiff_t`. Тип результата при вычитании двух указателей.

`size_t`. Тип, возвращаемый оператором `sizeof`.

`wchar_t`. Тип, достаточно большой, чтобы представлять все возможные символы во всех поддерживаемых локалях.

Все три являются именами целочисленных типов: `ptrdiff_t` должен быть знаковым типом, а `size_t` - беззнаковым.

Дополнительные сведения о `wchar_t` см. В разделе

25.2. Заголовок `<stddef.h>` также определяет два макроса.

Один из них - `NULL`, который представляет нулевой указатель. Другой макрос, `offsetof`, требует

двух аргументов: *type* (тип структуры) и *member-designator* (член структуры).

`offsetof` вычисляет количество байтов между началом структуры и указанным членом. Рассмотрим следующую структуру:

```
struct s{  
    char a;  
    int b [2] ;  
    float c;  
}
```

Значение `offsetof (struct s, a)` должно быть равно 0; C гарантирует, что первый член структуры имеет тот же адрес, что и сама структура. Мы не можем точно сказать, каковы смещения ***b*** и ***c***. Одна из возможностей состоит в том, что `offsetof (struct, b)` равно 1 (так как ***a*** имеет длину в один байт), а `offsetof(struct s, c)` равно 9 (предполагая 32-битные целые числа). Однако некоторые компиляторы оставляют "дыры" - неиспользуемые байты - в структурах (см. раздел Вопросы и ответов в конце Главы 16), которые могут повлиять на значение, создаваемое `offsetof`. Если компилятор должен оставить трехбайтовую дыру после ***a***, например, то смещения ***b*** и ***c*** будут равны 4 и 12 соответственно.

Но в этом и заключается прелесть `offsetof`: она производит правильные смещения для любого компилятора, позволяя нам писать переносимые программы. Существуют различные способы использования `offsetof`. Например, предположим, что мы хотим сохранить первые два члена структуры ***s*** в файле, игнорируя член ***c***, вместо того, чтобы функция `fwrite` записывала байты `sizeof (struct s)`, которая сохранила бы всю структуру, мы скажем ей записать только байты `offsetof ( struct s, c)`.

Последнее замечание:

Некоторые типы и макросы, определённые в `<stddef.h>`, также отображаются в других заголовках. (Макрос `NULL`, например, также определён в `<locale.h>`, `<stdio.h>`, `<stdlib.h>`, `<string.h>` и

<time.h>, а также в заголовке C99 <wchar.h>.) в результате немногие программы должны включать <stddef.h>.

21.5 заголовок <stdbool.h> (C99): ЛОГИЧЕСКИЙ ТИП И ЗНАЧЕНИЯ

Заголовок <stdbool.h> определяет четыре макроса:

- `bool`, (определяется как `_Bool`)
- `true` (определяется как `1`)
- `false` (определяется как `0`)
- `_bool_true_false_are_defined` (определено как `1`)

Мы видели много примеров того, как используются `bool`, `true` и `false`. Потенциальные возможности использования макроса `_bool_true_false_are_defined` более ограничены.

Программа может использовать директиву предварительной обработки (например, `#if` или `#ifdef`) для проверки этого макроса, прежде чем пытаться определить свою собственную версию `bool`, `true` или `false`.

Вопросы и ответы

Вопрос: Я заметил, что вы используете термин "стандартный заголовок", а не "стандартный файл заголовка". Есть ли какая-то причина не использовать слово "файл"?

Ответ: Да. Согласно стандарту C "стандартный заголовок" не обязательно должен быть файлом. Хотя большинство компиляторов действительно хранят стандартные заголовки в виде файлов, на самом деле они могут быть встроены в сам компилятор.

Вопрос: В разделе 14.3 описаны некоторые недостатки использования параметризованных макросов вместо функций. В свете этих проблем не опасно ли предоставлять макрос - замену стандартной библиотечной функции?

Ответ: Согласно стандарту C, параметризованный макрос, заменяющий библиотечную функцию, должен быть "полностью защищён" круглыми скобками и должен вычислять свои аргументы ровно один раз. Эти правила позволяют избежать большинства проблем, упомянутых в разделе 14.3.

Упражнения

1. Найдите, где хранятся заголовочные файлы в вашей системе. Найдите нестандартные заголовки и определите назначение каждого из них.

2. Найдя файлы заголовков в вашей системе (см. Упражнение 1), найдите стандартный заголовок, в котором макрос скрывает функцию.

3. Когда макрос скрывает функцию, что должно быть первым в заголовочном файле: определение макроса или прототип функции? Обоснуйте свой ответ.

4. Составьте список всех зарезервированных идентификаторов в разделе "направления будущей библиотеки" стандарта C99. Различайте идентификаторы, зарезервированные для использования только при включении конкретного заголовка, и идентификаторы, зарезервированные для использования в качестве внешних имён.

* 5. Функция `islower`, принадлежащая `<ctype.h>`, проверяет, является ли символ строчной буквой (низкого регистра).

Почему следующая макрос-версия `islower` не будет допустимой в соответствии со стандартом C? (Вы можете предположить, что набор символов - это ASCII.)

```
# define islower(c) ((c) >= 'a' && (c) <= 'z')
```

6. Заголовок `<ctype.h>` обычно определяет большинство своих функций также как макросы. Эти макросы полагаются на

1203

статический массив, объявленный в <ctype.h>, но определённый в отдельном файле. Ниже отображается часть типичного заголовка <ctype.h>. Используйте этот пример, чтобы ответить на следующие вопросы.

(а) почему имена "битовых" макросов (таких как _UPPER) и массива _ctype начинаются с подчёркивания?

(b) объясните, что будет содержать массив _ctype. Предполагая, что набор символов является ASCII, покажите значения элементов массива в позициях 9 (горизонтальный символ табуляции). 32 (символ пробела). 65 (буква A). и 94 (символ ^). См. раздел 23.5 для описания того, что должен возвращать каждый макрос.

(с) в чем преимущество использования массива для реализации этих макросов?

```
#define _UPPER 0x01 /* uppercase letter */
#define _LOWER 0x02 /* lowercase letter */
#define _DIGIT 0x04 /* decimal digit */
#define _CONTROL 0x08 /* control character */
#define _PUNCT 0x10 /* punctuation character */
#define _SPACE 0x20 /* white-space character */
#define _HEX 0x40 /* hexadecimal digit */
#define _BLANK 0x80 /* space character */
#define isalnum(c) (_ctype[c] & (_UPPER | _LOWER | _DIGIT))
#define isalpha(c) (_ctype[c] & (_UPPER | _LOWER))
```

1204

```
#define iscntrl(c) (_ctype[c] & _CONTROL)
#define isdigit(c) (_ctype[c] & _DIGIT)
#define isgraph(c) (_ctype[c] & (_BLANK | _PUNCT | _UPPER |
_LOWER | _DIGIT) )
#define islower(c) (_ctype[c] & _LOWER)
#define isprint(c) (_ctype [c] & (_BLANK | _PUNCT | _UPPER |
_LOWER | _DIGIT) )
#define ispunct(c) (_ctype[c] & _PUNCT)
#define isspace(c) (_ctype[c] & _SPACE)
#define isupper(c) (_ctype[c] s: _UPPER)
#define isxdigit(c) (_ctype[c] & (_DIGIT | _HEX))
```

W 7. В каком стандартном заголовке вы ожидаете найти каждое из следующего?

- (a) Функция, определяющая текущий день недели.
- (b) Функция, которая проверяет, является ли символ цифрой.
- (c) Макрос, который даёт наибольшее значение типа `unsigned int`
- (d) Функция округления числа с плавающей запятой до следующего большего целого числа.
- (e) Макрос, который определяет количество бит в символе.
- (f) Макрос, который определяет количество значащих цифр в значении `double`
- (g) Функция, которая ищет в строке определённый символ.
- (h) Функция, открывающая файл для чтения.

Проекты по программированию

1. Напишите программу, которая объявляет структуру `s` (см. Раздел 21.4) и выводит размеры и смещения членов `a`, `b` и `c`.
2. (Используйте `sizeof` для поиска размеров; используйте `offsetof` для поиска смещений.)
3. Попросите программу также распечатать размер всей конструкции. По этой информации определите, есть ли в конструкции дыры. Если да, опишите расположение и размер каждой.

22 Ввод/вывод

В симбиозе человек-машина приспособляться должен человек; машины этого не могут

Библиотека ввода-вывода `C` - самая большая и важная часть стандартной библиотеки. В соответствии с его высоким статусом, мы посвятим целую главу (самую длинную в книге) заголовку `<stdio.h>`, основному хранилищу функций

ввода/вывода. Мы использовали `<stdio.h>` с главы 2, и у нас есть опыт работы с функциями `printf`, `scanf`, `putchar`, `getchar`, `puts` и `gets`. В этой главе представлена дополнительная информация об этих шести функциях, а также представлен ряд новых функций, большинство из которых имеют дело с файлами. К счастью, многие из новых функций тесно связаны с функциями, с которыми мы уже знакомы, например, `fprintf` - это "файловая версия" функции `printf`. Мы начнём главу с обсуждения некоторых основных вопросов: концепция потока, тип `FILE`, перенаправление ввода и вывода, а также различие между текстовыми и двоичными файлами (раздел 22.1). Затем мы обратимся к функциям, специально разработанным для использования с файлами, включая функции, которые открывают и закрывают файлы (Раздел 22.2). После рассмотрения `printf`, `scanf` и связанных с ними функций для "форматированного" ввода / вывода (Раздел 22.3) мы рассмотрим функции, которые читают и записывают неформатированные данные:

- `getc`, `putc` и связанные с ними функции. которые читают и записывают по одному символу за раз (раздел 22.4).
- `gets`, `puts` и связанные функции. которые читают и записывают по одной строке за раз (раздел 22.5).
- `fread` и `fwrite`, которые читают и записывают блоки данных (раздел 22.6).

В разделе 22.7 затем показано, как выполнять операции произвольного доступа к файлам. Наконец, в разделе 22.8

описываются `sprintf`. Функции `snprintf` и `sscanf`, варианты `printf` и `scanf`, которые записывают в строку или читают из нее. В этой главе рассматриваются все функции, кроме восьми, в `<stdio.h>`. Одна из этих восьми, функция `perror`, тесно связана с заголовком `<errno.h>`, поэтому я отложу её до Раздела 24.2, в котором обсуждается этот заголовок.

Раздел 26.1 охватывает остальные функции (`vfprintf`, `vprintf`, `vsprintf`, `vsnprintf`, `vfscanf`, `vscanf` и `vsscanf`). Эти функции полагаются на тип `va_list`, который представлен в этом разделе. В C89 все стандартные функции ввода/вывода принадлежат `<stdio.h>`, но это не так в C99, где некоторые функции ввода/вывода объявлены в заголовке `<wchar.h>`. Функции `<wchar.h>` работают с широкими символами, а не с обычными символами. Хорошая новость заключается в том, что большинство этих функций очень похожи на функции `<stdio.h>`. Функции в `<stdio.h>`, которые читают или записывают данные, известны как функции ввода / вывода байтов; аналогичные функции в `<wchar.h>` называются функциями ввода/вывода широких символов.

22.1 Потоки

В языке C термин поток означает любой источник ввода или любое место назначения для вывода. Многие небольшие программы, подобные программам в предыдущих главах,

получают весь свой ввод из одного потока (обычно связанного с клавиатурой) и записывают весь свой вывод в другой поток. (обычно связаны с экраном). Более крупным программам могут потребоваться дополнительные потоки. Эти потоки часто представляют собой файлы, хранящиеся на различных носителях (например, на жёстких дисках, компакт-дисках, DVD-дисках и флэш-памяти), но их можно так же легко связать с устройствами, которые не хранят файлы: сетевыми портами, принтерами и т.п.

Мы сконцентрируемся на файлах, поскольку они обычны и легки для понимания. (Я могу даже иногда использовать термин "файл", когда должен сказать "поток".) Имейте в виду, однако, что многие функции в `<stdio.h>` одинаково хорошо работают со всеми потоками, а не только с теми, которые представляют файлы.

Указатели файлов

Доступ к потоку в программе на С осуществляется через указатель файла, который имеет тип `FILE *` (тип `FILE` объявлен в `<stdio.h>`). Некоторые потоки представлены указателями на файлы со стандартными именами; при необходимости мы можем объявить дополнительные указатели на файлы. Например, если программе нужны два

1209

потока в дополнение к стандартным, она может содержать следующее объявление:

```
FILE * fp1, * fp2;
```

Программа может объявлять любое количество переменных FILE *, хотя операционные системы обычно ограничивают количество потоков, которые могут быть открыты одновременно.

Стандартные потоки и перенаправление

<stdio.h> предоставляет три стандартных потока (таблица 22.1). Эти потоки готовы к использованию - мы их не объявляем. и мы их не открываем и не закрываем.

Указатель файла	Поток	Значение по умолчанию
stdin	Стандартный ввод	Клавиатура
stdout	Стандартный вывод	Экран
stderr	Стандартная ошибка	Экран

Таблица 22.1 Стандартные потоки

Функции, которые мы использовали в предыдущих главах, - printf, scanf, putchar, getchar, puts и gets - получают входные данные из стандартного ввода и отправляют выходные данные в стандартный вывод. По умолчанию stdin представляет

1210

клавиатуру; stdout и stderr представляют экран. Однако многие операционные системы позволяют изменять эти значения по умолчанию с помощью механизма, известного как перенаправление. Как правило, мы можем заставить программу получать ввод из файла, а не с клавиатуры, помещая имя файла в командную строку, которому предшествует символ < :

```
demo <in.dat
```

Этот метод, известный как перенаправление ввода, по существу заставляет поток stdin представлять файл (in.dat, в данном случае) вместо клавиатуры. Прелесть перенаправления заключается в том, что демонстрационная программа не понимает, что она читает из in.dat; насколько ей известно, любые данные, которые она получает из stdin, вводятся с клавиатуры.

Перенаправление вывода аналогично. Перенаправление потока stdout обычно выполняется путём ввода имени файла в командную строку, перед которым стоит символ > :

```
demo >out.dat
```

Все данные, записанные в stdout, теперь будут поступать в файл out.dat вместо того, чтобы появляться на экране. Кстати, мы можем комбинировать перенаправление вывода с перенаправлением ввода:

1211

demo <in.dat >out.dat

Символы < и > не обязательно должны быть рядом с именами файлов, и порядок, в котором перечислены перенаправленные файлы, не имеет значения, поэтому следующие примеры будут работать так же хорошо:

demo < in.dat > out.dat

demo > out.dat < in.dat

Одна из проблем с перенаправлением вывода заключается в том, что все, записанное в stdout, помещается в файл. Если программа сойдёт с рельсов и начнет писать сообщения об ошибках, мы не увидим их, пока не просмотрим файл. Здесь на помощь приходит stderr. Написав сообщения об ошибках в stderr вместо stdout, мы можем гарантировать, что эти сообщения будут отображаться на экране даже при перенаправлении stdout. (Операционные системы часто позволяют перенаправлять сам stderr.)

Текстовые файлы против двоичных файлов

< stdio.h> поддерживает два вида файлов: текстовые и двоичные, байты в текстовом файле представляют собой символы, что позволяет человеку просматривать файл или редактировать его. Например, исходный код программы на

языке Си хранится в текстовом файле. С другой стороны, в двоичном файле байты не обязательно представляют символы; группы байтов могут представлять другие типы данных, такие как целые числа и числа с плавающей запятой. Исполняемая программа на языке Си хранится в двоичном файле, вы это быстро поймёте, если попытаетесь заглянуть в его содержимое.

Текстовые файлы имеют две характеристики, которыми не обладают двоичные файлы:

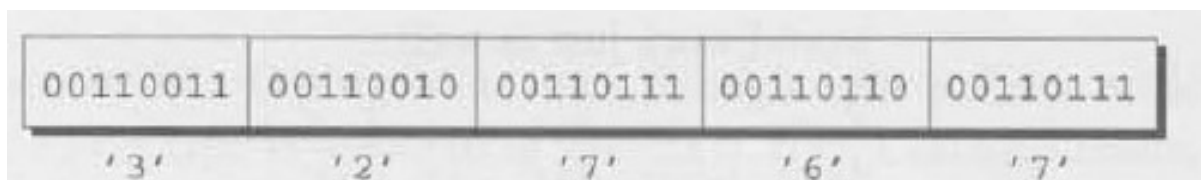
- ***Текстовые файлы делятся на строки.*** Каждая строка в текстовом файле обычно заканчивается одним или двумя специальными символами; выбор символов зависит от операционной системы. В Windows маркер конца строки представляет собой символ возврата каретки ('\x0d'), за которым сразу же следует символ перевода строки ('\x0a'). В UNIX и более новых версиях операционной системы Macintosh (Mac OS) маркер конца строки представляет собой один символ перевода строки. В более старых версиях Mac OS используется один символ возврата каретки.
- ***Текстовые файлы могут содержать специальный маркер "конец файла".*** Некоторые операционные системы позволяют использовать специальный байт в качестве маркера в конце текстового файла. В Windows

1213

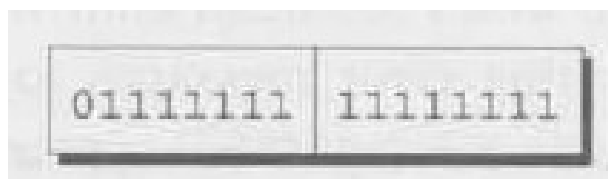
маркер '\x1a' (Ctrl-Z). Нет никакого требования, чтобы Ctrl-Z присутствовал, но если это так, то он отмечает конец файла; любые байты после Ctrl-Z должны быть проигнорированы. Соглашение Ctrl-Z является пережитком DOS, которая, в свою очередь, унаследовала его от CP/M, ранней операционной системы для персональных компьютеров. Большинство других операционных систем, включая UNIX, не имеют специального символа конца файла.

Двоичные файлы не делятся на строки. В двоичном файле нет маркеров конца строки или конца файла; все байты обрабатываются одинаково.

Когда мы записываем данные в файл, нам нужно будет решить, хранить ли их в текстовой или двоичной форме, чтобы увидеть разницу, подумайте, как мы могли бы хранить число 32767 в файле. Одним из вариантов было бы записать число в текстовой форме в виде символов 3, 2, 7, 6 и 7. Если бы набор символов был ASCII, то у нас было бы следующие пять байтов:



Другой вариант - хранить число в двоичном формате, который займёт всего два байта:



(Байты будут перевёрнуты в системах, которые хранят данные в прямом порядке - "little-endian".) Как показывает этот пример, хранение чисел в двоичном формате часто может сэкономить довольно много места.

Когда мы пишем программу, которая читает из файла или записывает в файл, мы должны учитывать, является ли он текстовым или двоичным файлом. Программа, которая воспроизводит содержимое файла на экране, вероятно, предположит, что это текстовый файл. Программа копирования файлов, с другой стороны, не может предполагать, что копируемый файл является текстовым. Если это так, двоичные файлы, содержащие символ конца файла, не будут скопированы полностью. Когда мы не можем точно сказать, является ли файл текстовым или двоичным, безопаснее предположить, что он двоичный.

22.2 Файловые операции

Простота - одно из преимуществ перенаправления ввода и вывода; нет необходимости открывать файл, закрывать файл или выполнять какие-либо другие явные файловые операции. К сожалению, для многих приложений перенаправление слишком ограничено. Когда программа полагается на перенаправление, она не может контролировать свои файлы; он даже не знает их имён. Что ещё хуже, перенаправление не помогает, если программе нужно читать из двух файлов или записывать в два файла одновременно. Когда перенаправления недостаточно, мы будем использовать файловые операции, которые предоставляет `<stdio.h>`. В этом разделе мы рассмотрим эти операции, которые включают открытие файла, закрытие файла, изменение способа буферизации файла, удаление файла и переименование файла.

Открытие файла

`FILE *fopen(const char * restrict filename, const char * restrict mode) ;`

Открытие файла для использования в качестве потока требует вызова функции `fopen`. Первый аргумент `fopen` - это строка, содержащая имя файла, который нужно открыть. ("Имя файла" может включать информацию о расположении файла, такую как спецификатор диска или путь.) Второй аргумент - это "строка режима", которая указывает, какие операции мы

1216

собираемся выполнить с файлом. Например, строка "r" указывает, что данные будут считаны из файла, но в него не будут записаны никакие данные.

Обратите внимание, что `restrict` появляется дважды в прототипе функции `fopen`. `restrict`, которое является ключевым словом C99, указывает, что имя файла и режим должны указывать на строки, которые не разделяют области памяти. Прототип C89 для `fopen` не содержит `restrict`, но в остальном идентичен, `restrict` не влияет на поведение `fopen`, поэтому обычно его можно просто игнорировать.

Программисты Windows: будьте осторожны, когда имя файла при вызове `fopen` включает символ `\`, поскольку C рассматривает `\` как начало escape-последовательности.

Вызов

```
fopen ("c: \project \test1.dat", "r" )
```

завершится ошибкой, потому что компилятор рассматривает `\t` как escape-символ. (`\p` не является допустимым escape-символом, но похож на него. Стандарт C утверждает, что его значение не определено.) Есть два способа избежать проблемы. Один из них - использовать `\\` вместо `\`

`fopen ("c: \\ project \\ test1.dat", "r")` Другой способ ещё проще - просто используйте символ `/` вместо `\`:

```
fopen ("c: /project/test1.dat", "r ")
```

Windows с радостью примет / вместо \ в качестве разделителя каталогов.

`fopen` возвращает указатель файла, который программа может сохранить в переменной (и обычно сохраняет) и использовать позже всякий раз, когда ей нужно выполнить операцию с файлом. Вот типичный вызов `fopen`, где `fp` - это переменная типа `FILE *`:

```
fp = fopen ("in.dat", "r"); /* открывает in.dat для чтения */
```

Когда программа вызывает функцию ввода для последующего чтения из `in.dat`, она предоставит `fp` в качестве аргумента.

Когда не удаётся открыть файл, `fopen` возвращает нулевой указатель. Возможно, файла не существует, или он находится не в том месте, или у нас нет разрешения на его открытие.

Никогда не предполагайте, что файл можно открыть; всегда проверяйте возвращаемое значение `fopen`, чтобы убедиться, что это не нулевой указатель.

Режимы (Modes)

Какую строку `mode` мы передадим в `fopen`, зависит не только от того, какие операции мы планируем выполнить с файлом позже, но и от того, содержит ли файл текстовые или двоичные данные. Чтобы открыть текстовый файл, мы должны использовать одну из строк `mode` в таблице 22.2.

Режим Означает

1218

"r"	Открыть для чтения
"w"	Открыть для записи (файл не должен существовать)
"a"	Открыть для добавления (файл не должен существовать)
"r+"	Открыть для чтения и записи начиная с самого начала
"w+"	Открыть для чтения и записи (усечь, если файл существует)
"a+"	Открыть для чтения и записи (добавить если файл существует)

Таблица 22.2 Строки режимов для текстовых файлов

Когда мы используем `for` для открытия двоичного файла, нам нужно будет включить букву `b` в строку режима. В таблице 22.3 перечислены строки режима для двоичных файлов.

Режим	Означает
"rb"	Открыть для чтения
"wb"	Открыть для записи (файл не должен существовать)
"ab"	Открыть для добавления (файл не должен существовать)
"r+b" или "rb+"	Открыть для чтения и записи начиная с самого начала
"w+b" или "wb+"	Открыть для чтения и записи (усечь, если

файл существует)

"a+b" или "ab+" Открыть для чтения и записи (добавить если файл существует)

Таблица 22.3 Строки режимов для бинарных файлов

Из таблиц 22.2 и 22.3 мы видим, что `<stdio.h>` различает запись данных и добавление данных. Когда данные записываются в файл, они обычно перезаписывают то, что было ранее. Однако, когда файл открывается для добавления, данные, записанные в файл, добавляются в конец, таким образом сохраняя исходное содержимое файла. Кстати, при открытии файла как для чтения, так и для записи применяются особые правила (строка режима содержит символ +). Мы не можем переключиться с чтения на запись без предварительного вызова функции позиционирования файла, если только операция чтения не встретила конец файла. Кроме того, мы не можем переключиться с записи на чтение без вызова функции `fflush` (рассматривается далее в этом разделе) или вызова функции позиционирования файлов.

Заккрытие файла

```
int fclose (FILE * stream);
```

Функция `fclose` позволяет программе закрыть файл, который больше не используется. Аргументом для `fclose` должен быть указателем на файл, полученный в результате вызова `fopen` или `freopen` (обсуждается далее в этом разделе). `fclose` возвращает ноль, если файл был успешно закрыт; в противном случае он возвращает код ошибки EOF (макрос, определённый в `<stdio.h>`).

Чтобы показать, как `fopen` и `fclose` используются на практике, вот схема программы, которая открывает файл `example.dat` для чтения, проверяет, успешно ли он был открыт, а затем закрывает его перед завершением:

```
#include <stdio.h>

#include <stdlib.h>

#define FILE_NAME "example.dat"

int main(void){
    FILE *fp;
    fp = fopen(FILE_NAME, "r");
    if (fp == NULL) {
        printf ("Can't open %s\n", FILE_NAME) ;
        exit (EXIT_FAILURE) ;
    }
}
```


1221

```
    }  
    fclose (fp);  
    return 0;  
}
```

Конечно, поскольку программисты С таковы, каковы они есть, для них нет ничего необычного в том, чтобы увидеть вызов `fopen` в сочетании с объявлением `fp`:

```
FILE *fp = fopen(FILE_NAME, "r");
```

или тест на NULL:

```
if ((fp = fopen (FILE NAME, "r")) == NULL)
```

Присоединение файла к открытому потоку

```
FILE *freopen(const char * restrict filename, const char * restrict  
mode, FILE * restrict stream) ;
```

`freopen` присоединяет другой файл к уже открытому потоку. Наиболее распространенным использованием `freopen` является связывание файла с одним из стандартных потоков (`stdin`, `stdout` или `stderr`), чтобы заставить программу начать запись в файл `foo`, мы могли бы, например, использовать следующий вызов `freopen`:

```
if (freopen("foo", "w", stdout) == NULL) {
```

```
/*error; foo не может быть открыт */  
}
```

После закрытия любого файла, ранее связанного с stdout (путём перенаправления командной строки или предыдущего вызова freopen), freopen откроет foo и свяжет его со stdout.

Нормальное возвращаемое значение freopen - это его третий аргумент (указатель на файл). Если он не может открыть новый файл, freopen возвращает нулевой указатель. (freopen игнорирует ошибку, если старый файл не может быть закрыт.)

C99 добавляет новый поворот. Если filename является нулевым указателем, freopen пытается изменить режим потока на тот, который задан параметром mode. Однако реализации не обязаны поддерживать эту функцию; если они это делают, они могут накладывать ограничения на то, какие изменения режима разрешены.

Получение имён файлов из командной строки

Когда мы пишем программу, которая должна будет открыть файл, вскоре становится очевидной одна проблема: как сообщить имя файла программе? Создание имён файлов в

1223

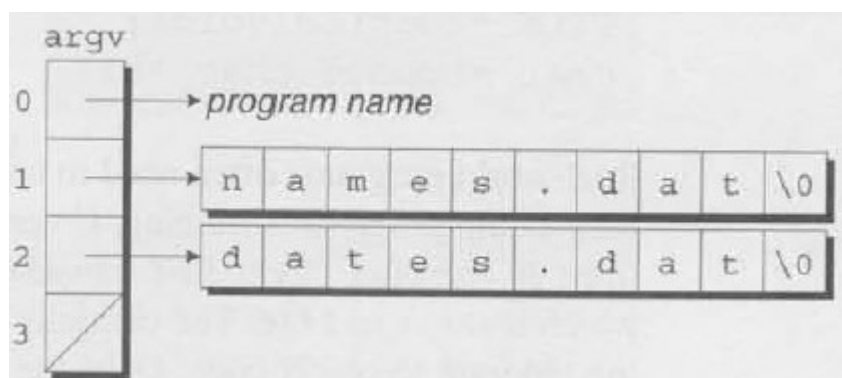
самой программе не обеспечивает большой гибкости, а запрос пользователя на ввод имён файлов может быть неудобным. Часто лучшим решением является получение программой имён файлов из командной строки. Например, когда мы выполняем программу с именем `demo`, мы можем снабдить её именами файлов, поместив их в командную строку:

```
demo names.dat dates.dat
```

В разделе 13.7 мы видели, как получить доступ к аргументам командной строки, определив `main` как функцию с двумя параметрами:

```
int main(int argc, char *argv[]){  
    ...  
}
```

`argc` - это число аргументов командной строки; `argv` - массив указателей на строки аргументов. `argv [0]` указывает на имя программы, `argv [1]` через `argv [argc-1]` указывает на остальные аргументы, а `argv [argc]` является нулевым указателем. В приведённом выше примере `argc` равен 3, `argv [0]` указывает на строку, содержащую имя программы, `argv [1]` указывает на строку "names. dat", а `argv [2]` указывает на строку " dates.dat":



ПРОГРАММА Проверка возможности открытия файла

Следующая программа определяет, существует ли файл и может ли он быть открыт для чтения. Когда программа будет запущена, пользователь даст ей имя файла для проверки:

`canopen file`

Затем программа напечатает либо *file* можно открыть, либо *file* нельзя открыть. Если пользователь вводит неправильное количество аргументов в командной строке, программа напечатает сообщение `usage: canopen filename`, чтобы напомнить пользователю, что для `canopen` требуется одно имя файла.

```
//canopen.c
```

```
#include <stdio.h>
```

1225

```
#include <stdlib.h>

int main(int argc, char *argv[]){
    FILE *fp;
    if (argc != 2) {
        printf("usage: canopen filename\n");
        exit(EXIT_FAILURE);
    }
    if ((fp = fopen(argv[1], "r")) == NULL) {
        printf("%s can't be opened\n", argv[1]);
        exit(EXIT_FAILURE);
    }
    printf("%s можно открыть\n", argv[1]);
    fclose(fp);
    return 0;
}
```

Обратите внимание, что мы можем использовать перенаправление, чтобы отбросить вывод сапорен и просто проверить значение статуса, которое он возвращает.

Временные файлы

```
FILE *tmpfile (void);
```

```
char *tmpnam(char *s) ;
```

Реальным программам часто требуется создавать временные файлы - файлы, которые существуют только до тех пор, пока программа работает. Компиляторы С, например, часто создают временные файлы. Компилятор может сначала перевести программу на С в некоторую промежуточную форму, которую он хранит в файле. Затем компилятор прочитает файл позже, когда он программу в объектный код. После полной компиляции программы нет необходимости сохранять файл, содержащий промежуточную форму программы. <stdio.h> предоставляет две функции,

tmpfile и tmpnam, для работы с временными файлами.

tmpfile создаёт временный файл (открытый в режиме "wb +"), который будет существовать до его закрытия или завершения программы. Вызов tmpfile возвращает указатель файла, который можно использовать для доступа к файлу позже:

```
FILE * tempptr;
```

```
tempptr = tmpfile (); /* создаёт временный файл */
```

Если создать файл не удастся, tmpfile возвращает нулевой указатель.

Хотя tmpfile прост в использовании, у него есть несколько недостатков: (1) мы не знаем имени файла, который создаёт

tmpfile, и (2) мы не можем позже решить, сделать ли файл постоянным.

Если эти ограничения представляют проблему, можно создать временный файл с помощью fopen. Конечно, мы не хотим, чтобы этот файл имел то же имя, что и уже существующий файл, поэтому нам нужен способ сгенерировать новые имена файлов; Вот тут и пригодится функция tmpnam.

tmpnam генерирует имя временного файла. Если его аргумент является нулевым указателем, tmpnam сохраняет имя файла в статической переменной и возвращает указатель на него:

```
char *filename;
```

```
filename = tmpnam (NULL); /* создаёт имя временного файла */
```

В противном случае tmpnam копирует имя файла в массив символов, предоставленный программистом:

```
char filename [L_tmpnam];
```

```
tmpnam (filename) ; /* создаёт имя временного файла */
```

В последнем случае tmpnam также возвращает указатель на первый символ этого массива. L_tmpnam - это макрос в <stdio.h>, который указывает, какого минимального размера нужно создать массив символов, который будет содержать имя временного файла.

Убедитесь, что аргумент `tmpnam` указывает на массив, состоящий как минимум из `L_tmpnam` символов. Также будьте осторожны, чтобы не вызывать `tmpnam` слишком часто; макрос `TMP_MAX` (определённый в `<stdio.h>`) определяет максимальное количество имён временных файлов, которые потенциально могут быть сгенерированы `tmpnam` во время выполнения программы. Если не удаётся сгенерировать имя файла, `tmpnam` возвращает нулевой указатель.

Буферизация файлов

```
int fflush(FILE *stream);
```

```
void setbuf(FILE * restrict stream, char * restrict buf);
```

```
int setvbuf(FILE * restrict stream, char * restrict buf, int mode;  
size_t size);
```

Передача данных на диск или с диска - относительно медленная операция. В результате программа не может напрямую обращаться к файлу на диске каждый раз, когда она хочет прочитать или записать байт. Секрет достижения приемлемой производительности - это буферизация: данные, записанные в поток, фактически хранятся в буферной области памяти; когда он заполнен (или поток закрыт), буфер "очищается" (записывается в фактическое устройство

вывода). Аналогичным образом можно буферизовать входные потоки: буфер содержит данные с устройства ввода; ввод читается из этого буфера, а не из самого устройства.

Буферизация может привести к огромному увеличению эффективности, поскольку чтение байта из буфера или сохранение байта в буфере практически не занимает времени. Конечно, для передачи содержимого буфера на диск или с диска требуется время, но одно большое "перемещение блока" происходит намного быстрее, чем перемещение множества крошечных байтов. Функции в `<stdio.h>` автоматически выполняют буферизацию, когда это кажется целесообразным. Буферизация происходит за кулисами, и мы обычно не беспокоимся об этом. Однако в редких случаях нам может потребоваться более активная роль. Если это так, мы можем использовать функции `fflush`, `setbuf` и `setvbuf`.

Когда программа записывает вывод в файл, данные обычно сначала попадают в буфер. Буфер очищается автоматически, когда он заполняется или файл закрывается. Однако, вызывая `fflush`, программа может очищать буфер файла так часто, как пожелает. Вызов

```
fflush (fp); /* очищает буфер для fp */
```

очищает буфер для файла, связанного с `fp`. Вызов

```
fflush (NULL); /* очищает все буферы */
```

очищает все выходные потоки, `fflush` возвращает ноль в случае успеха и EOF в случае ошибки.

1230

setvbuf позволяет нам изменять способ буферизации потока и управлять размером и расположением буфера. Третий аргумент функции, определяющий желаемый тип буферизации, должен быть одним из следующих макросов:

`_IOBF` (полная буферизация). Данные считываются из потока, когда буфер пуст, или записываются в поток, когда он заполнен.

`_IOLBF` (буферизация строки). Данные считываются из потока или записываются в поток по одной строке за раз.

`_IONBF` (без буферизации), данные читаются из потока или записываются в поток напрямую, без буфера.

(Все три макроса определены в `<stdio.h>`.) Полная буферизация - это значение по умолчанию для потоков, которые не подключены к интерактивным устройствам.

Второй аргумент `setvbuf` (если это не нулевой указатель) - это адрес нужного буфера. Буфер может иметь статическую продолжительность хранения, автоматическую продолжительность хранения или даже выделяться динамически. Автоматизация буфера позволяет автоматически освободить его пространство при выходе из блока; его динамическое выделение позволяет нам освободить буфер, когда он больше не нужен, последний аргумент `setvbuf` - это количество байтов в буфере. Буфер большего размера

1231

может дать лучшую производительность; меньший буфер экономит место. Например, следующий вызов `setvbuf` изменяет буферизацию потока на полную, используя `N` байтов в массиве `buffer` в качестве буфера:

```
char buffer [N];
```

...

```
setvbuf (stream, buffer, _IOFBF, N);
```

`setvbuf` должен вызываться после открытия потока, но до того, как над ним будут выполнены какие-либо другие операции.

Также допустимо вызывать `setvbuf` с нулевым указателем в качестве второго аргумента, который запрашивает у `setvbuf` создание буфера с указанным размером. `setvbuf` возвращает ноль в случае успеха. Он возвращает ненулевое значение, если аргумент режима недопустим или запрос не может быть выполнен.

`setbuf` - это более старая функция, которая принимает значения по умолчанию для режима буферизации и размера буфера. Если `buf` является нулевым указателем, вызов

```
setbuf (stream, buf)
```

эквивалентен

```
(void) setvbuf(stream, NULL, _IONBF, 0);
```

В противном случае это эквивалентно

```
(void) setvbuf (stream, buf, _IOFBF, BUFSIZ);
```

1232

где BUFSIZ - это макрос, определённый в <stdio.h>. Функция setbuf считается устаревшей; её не рекомендуется использовать в новых программах.

При использовании setvbuf или setbuf обязательно закройте поток до того, как его буфер будет освобождён. В частности, если буфер является локальным для функции и имеет автоматическую продолжительность хранения, не забудьте закрыть поток до возврата из функции.

Разные файловые операции

```
int remove (const char * filename);
```

```
int rename (const char * old, const char * new);
```

Функции удаления и переименования позволяют программе выполнять основные операции по управлению файлами. В отличие от большинства других функций в этом разделе, удаление и переименование работают с именами файлов, а не с указателями файлов. Обе функции возвращают ноль в случае успеха и ненулевое значение в случае неудачи.

remove удаляет файл:

```
remove ("foo") ; /* удаляет файл с именем "foo" */
```

1233

Если программа использует `fopen` (вместо `tmpfile`) для создания временного файла, она может использовать команду `remove` для удаления файла до завершения работы программы. Убедитесь, что удаляемый файл закрыт; эффект удаления файла, который в данный момент открыт, определяется реализацией.

`rename` меняет имя файла:

```
rename ("foo", "bar") ;/* переименовывает "foo" в "bar" */
```

`rename` удобен для переименования временного файла, созданного с помощью `fopen`, если программа решит сделать его постоянным. Если файл с новым именем уже существует, эффект определяется реализацией.

Если файл, который нужно переименовать, открыт, обязательно закройте его перед вызовом метода переименования; функция может не работать, если её попросят переименовать открытый файл.

22.3 Форматированный ввод/вывод

В этом разделе мы рассмотрим библиотечные функции, которые используют строки формата для управления чтением и записью. Эти функции, к которым относятся наши старые

1234

друзья `printf` и `scanf`, имеют возможность преобразовывать данные из символьной формы в числовую во время ввода и из числовой формы в символьную во время вывода. Ни одна из других функций ввода-вывода не может выполнять такие преобразования.

Функции...`printf`

```
int fprintf (FILE * restrict stream, const char * restrict format,....);
```

```
int printf(const char * restrict format, ...);
```

Функции `fprintf` и `printf` записывают переменное количество элементов данных в поток вывода, используя строку формата для управления внешним видом вывода. Прототипы для обеих функций оканчиваются символом... (многоточием), который указывает переменное количество дополнительных аргументов. Обе функции возвращают количество записанных символов; отрицательное возвращаемое значение указывает, что произошла ошибка. Единственная разница между `printf` и `fprintf` заключается в том, что `printf` всегда записывает в `stdout` (стандартный поток вывода), тогда как `fprintf` записывает в поток, указанный первым аргументом:

```
printf("Total: %d\n", total); /* записывает в stdout */
```

```
fprintf(fp, "Total: %d\n", total); /* записывает в fp */
```

1235

Вызов `printf` эквивалентен вызову `fprintf` с `stdout` в качестве первого аргумента.

Однако не думайте о `fprintf` как о функции, которая записывает данные в файлы на диске. Как и многие функции в `<stdio.h>`, `fprintf` отлично работает с любым выходным потоком. Фактически, одно из наиболее распространённых применений `fprintf` - запись сообщений об ошибках в `stderr`, стандартный поток ошибок - не имеет ничего общего с дисковыми файлами. Вот как может выглядеть такой вызов:

```
fprintf(stderr, "Error: data file can't be opened.\n");
```

В `<stdio.h>` есть ещё две функции, которые могут записывать форматированный вывод в поток. Эти функции, названные `vfprintf` и `vprintf`, довольно непонятны. Обе они используют тип `va_list`, который объявлен в `<stdarg.h>`, поэтому они обсуждаются вместе с этим заголовком.

... printf спецификации преобразования

И для `printf`, и для `fprintf` требуется строка формата, содержащая обычные символы и/или спецификации преобразования. Обычные символы печатаются как есть;

1236

спецификации преобразования описывают, как остальные аргументы должны быть преобразованы в символьную форму для отображения. В разделе 3.1 кратко описаны спецификации преобразования, а в последующих главах мы добавили больше деталей. Теперь мы рассмотрим то, что нам известно о спецификациях преобразования и восполним оставшиеся пробелы.

Спецификация преобразования... printf состоит из символа %, за которым следуют до пяти отдельных элементов:

%	#0	12	.5	L	g
	Флаг	Мини- мальная ширина поля	Точность	Модифика- тор длины	Специ- фикатор преобразо- вания

Вот подробное описание этих предметов, которые должны появиться в указанном порядке:

- **Флаги** (необязательно; допускается более одного). Флаг - вызывает выравнивание по левому краю поля; другие флаги влияют на способ отображения чисел. В таблице 22.4 приведён полный список флагов.

Флаг	Что означает
-	Выровнять по левому краю внутри поля. (По

умолчанию используется выравнивание по правому краю.)

- + Числа со знаком, полученные в результате преобразования, всегда начинаются с + или -. (Обычно только отрицательным числам предшествует знак.)
- пробел Перед неотрицательными числами, полученными в результате преобразования со знаком, ставится пробел. (Флаг + заменяет флаг пробела.)
- # Восьмеричные числа начинаются с 0, ненулевые шестнадцатеричные числа с 0x или 0X. Для чисел с плавающей точкой всегда используется десятичная точка. Завершающие нули не удаляются из чисел, напечатанных с преобразованием g или G.
- 0 Числа дополняются ведущими нулями до ширины поля. Флаг 0 (ноль) игнорируется, если спецификатор преобразования равен d, i, o, u, x или X и указана точность. (Флаг - отменяет флаг 0.)

Таблица 22.4 Флаги для функций ... printf

- **Минимальная ширина поля** (необязательно). Элемент, который слишком мал для этого количества символов, будет дополнен. (По умолчанию пробелы добавляются слева от элемента, таким образом, выравнивая его по правому краю внутри поля.) Элемент, который слишком велик для ширины поля, все равно будет отображаться полностью. Ширина поля может быть целым числом или

символом *. Если * присутствует, ширина поля получается из следующего аргумента. Если этот аргумент отрицательный, он обрабатывается как положительное число, которому предшествует флаг -.

- **Точность** (необязательно). Значение точности зависит от преобразования:

d, i, o, u, x, X: минимальное количество цифр (начальные нули добавляются, если число имеет меньшее количество цифр)

a, A, e, E, f, F: число цифр после десятичной точки

g, G : количество значащих цифр

s: максимальное количество байтов

Точность - это точка (.), За которой следует целое число или символ *. Если присутствует *, точность получается из следующего аргумента. (Если этот аргумент отрицательный, эффект такой же, как и отсутствие указания точности.) Если присутствует только точка, точность равна нулю.

- **Модификатор длины** (необязательно). Наличие модификатора длины указывает на то, что отображаемый элемент имеет тип, который длиннее или короче, чем обычно для конкретной спецификации преобразования. (Например, %d обычно относится к значению int; %hd используется для отображения short int, а %ld

используется для отображения long int.) Таблица 22.5 перечисляет каждый модификатор длины, спецификаторы преобразования, с которыми он может использоваться, и тип, обозначенный их комбинацией. (Любая комбинация модификатора длины и спецификатора преобразования, не показанная в таблице, вызывает неопределённое поведение.)

Модифика тор длины	Спецификатор преобразования	Значение
hh	d, i, o, u, x, X	signed char, unsigned char
	n	signed char
h	d, i, o, u, x, X	short int, unsigned short int
	n	short int
l (ell)	d, i, o, u, x, X	long int, unsigned long int
	n	long int
	c	wint_t
	s	wchar_t
	a, A, e, E, f, F, g, G	не действует
ll (ell-ell)	d, i, o, u, x, X	long long int, unsigned long long int
	n	long long int

j	d, i, o, u, x, X	intmax_t, uintmax_t
	n	intmax_t
z	d, i, o, u, x, X	size_t
	n	size_t
t	d, i, o, u, x, X	ptrdiff_t
	n	ptrdiff_t
L	a, A, e, E, f, F, g, G	long double

Таблица 22.5 Модификаторы длины для функций ...printf

- **Спецификатор преобразования.** Спецификатор преобразования должен быть одним из символов, перечисленных в таблице 22.6. Обратите внимание, что f, F, e, E, g, G, а и А предназначены для записи значений double. Однако они также отлично работают со значениями float. Благодаря продвижению аргументов по умолчанию, аргументы float автоматически преобразуются в double при передаче в функцию с переменным числом аргументов. Аналогично, символ, переданный в...printf, автоматически преобразуется в int, поэтому преобразование с работает правильно.

1241

Будьте внимательны и соблюдайте описанные здесь правила; эффект от использования недопустимой спецификации преобразования не определён.

Спецификатор преобразования	Назначение
-----------------------------	------------

d, i	Преобразует значение типа <code>int</code> в десятичную форму.
------	--

o, u, x, X	Преобразует значение типа <code>int</code> без знака в основание 8 (o), основание 10 (u) или основание 16 (x, X). x отображает шестнадцатеричные цифры a - f в нижнем регистре: X отображает их в верхнем регистре.
------------	---

f, F	Преобразует значение <code>double</code> в десятичную форму, помещая десятичную точку в правильную позицию. Если точность не указана, отображается шесть цифр после
------	---

десятичной точки.

- e, E Преобразует значение типа double в экспоненциальное представление. Если точность не указана, отображается шесть цифр после десятичной точки. Если выбрано e, перед экспонентой стоит буква e; если выбрано E, показателю степени предшествует E.
- g, G g преобразует значение double либо в форму *f*, либо в форму *e*. Форма *e* выбирается, если показатель степени меньше -4 или больше или равен точности. Нули в конце не отображаются (если не используется флаг #); десятичная точка появляется только тогда, когда за ней следует цифра. G выбирает между формами F и E.
- a, A Преобразует значение типа double в шестнадцатеричное научное представление с использованием формы [-] 0xh. hhhhpr + d, где [-] - необязательный знак минус, h представляет шестнадцатеричные цифры, + - знак плюс или минус, а d - показатель степени. d - десятичное число, представляющее степень 2. Если точность не указана, после десятичной точки отображается достаточное количество цифр, чтобы представить точное значение числа (если возможно). a отображает шестнадцатеричные цифры a - f в нижнем регистре; A отображает их в верхнем регистре. Выбор a или A также влияет на регистр букв x

1243

	и р.
c	Отображает значение типа int как беззнаковый символ.
s	Записывает символы, на которые указывает аргумент. Останавливает запись, когда достигается количество байтов, указанное точностью (если она есть), или встречается нулевой символ.
p	Преобразует значение void * в форму для печати.
n	Соответствующий аргумент должен указывать на объект типа int. Сохраняет в этом объекте количество символов, записанных на данный момент этим вызовом... printf; не производит никакого вывода.
%	Пишет символ %.

С99 Изменения в спецификациях преобразования..printf

Спецификации преобразования для printf и fprintf претерпели ряд изменений в С99:

- **Дополнительные модификаторы длины.** С99 добавляет модификаторы длины hh, ll, j, z и t. hh и ll предоставляют дополнительные параметры длины, j позволяет записывать целые числа наибольшей ширины, а z и t

упрощают запись значений типа `size_t` и `ptrdiff_t` соответственно.

- **Дополнительные спецификаторы преобразования.** C99 добавляет спецификаторы преобразования `F`, `a` и `A`. `F` - то же самое, что и `f`, за исключением способа записи бесконечности и `NaN` (см. Ниже). Спецификации преобразования `a` и `A` используются редко. Они связаны с шестнадцатеричными константами с плавающей запятой, которые обсуждаются в разделе вопросов и ответов в конце главы 7.
- **Возможность писать бесконечность и NaN.** Стандарт IEEE 754 с плавающей запятой позволяет результату операции с плавающей запятой быть бесконечностью, отрицательной бесконечностью или `NaN` ("не числом"). Например, деление 1,0 на 0,0 даёт положительную бесконечность, деление -1,0 на 0,0 даёт отрицательную бесконечность, а деление 0,0 на 0,0 даёт `NaN` (поскольку результат математически не определён). В C99 спецификаторы преобразования `a`, `A`, `e`, `E`, `f`, `F`, `g` и `G` могут преобразовывать эти специальные значения в форму, которая может отображаться. `a`, `e`, `f` и `g` преобразуют положительную бесконечность в `inf` или `infinity` (любой из них допустим), отрицательную бесконечность в `-inf` или `-infinity`, а `NaN` в `nan` или `-nan` (возможно, за которыми следует ряд символов,

заключённых в круглые скобки). А, Е, F и G эквивалентны а, е, f и g, за исключением того, что используются заглавные буквы (INF, INFINITY, NAN).

- **Поддержка широких символов.** ещё одна особенность C99 - это способность `fprintf` писать широкие символы. Спецификация преобразования `%lc` используется для записи одного широкого символа; `%ls` используется для строки широких символов.
- **Теперь разрешены ранее неопределённые спецификации преобразования.** В C89 эффект от использования `%le`, `%lE`, `%lf`, `%lg` и `%lG` не определён. Эти спецификации преобразования допустимы в C99 (модификатор длины `l` просто игнорируется).

Примеры спецификаций преобразования... `printf`

Ух! Пришло время привести несколько примеров. Мы видели множество повседневных спецификаций преобразования в предыдущих главах, поэтому мы сконцентрируемся здесь на иллюстрации некоторых из наиболее сложных. Как и в предыдущих главах, я буду использовать ● для обозначения символа пробела.

Давайте начнём с изучения влияния флагов на конверсию `%d` (они оказывают аналогичное влияние на другие конверсии). В первой строке Таблицы 22.7 показан эффект `8d` без каких-

1246

либо флагов. Следующие четыре строки показывают действие флагов -, +, пробела и 0 (флаг # никогда не используется с %d). Остальные строки показывают эффект комбинации флагов.

<i>Conversion Specification</i>	<i>Result of Applying Conversion to 123</i>	<i>Result of Applying Conversion to -123</i>
%8d	•••••123	•••••-123
%-8d	123•••••	-123•••••
%+8d	•••••+123	•••••-123
% 8d	•••••123	•••••-123
%08d	00000123	-0000123
%-+8d	+123•••••	-123•••••
%- 8d	•123•••••	-123•••••
%+08d	+0000123	-0000123
% 08d	•0000123	-0000123

Таблица 22.7 Влияние флагов на конверсию %d

В таблице 22.8 показано влияние флага # на преобразования o, x, X, g и G.

<i>Conversion Specification</i>	<i>Result of Applying Conversion to 123</i>	<i>Result of Applying Conversion to 123.0</i>
%8o	•••••173	
%#8o	•••••0173	
%8x	•••••7b	
%#8x	•••••0x7b	
%8X	•••••7B	
%#8X	•••••0X7B	
%8g		•••••123
%#8g		•123.000
%8G		•••••123
%#8G		•123.000

Таблица 22.8 Влияние флага #

В предыдущих главах мы использовали минимальную ширину поля и точность при отображении чисел, поэтому здесь нет смысла приводить больше примеров. Вместо этого в таблице 22.9 показано влияние минимальной ширины поля и точности на преобразование %s.

<i>Conversion Specification</i>	<i>Result of Applying Conversion to "bogus"</i>	<i>Result of Applying Conversion to "buzzword"</i>
%6s	•bogus	buzzword
%-6s	bogus•	buzzword
%.4s	bogu	buzz
%6.4s	••bogu	••buzz
%-6.4s	bogu••	buzz••

Таблица 22.9 Влияние минимальной ширины поля и точности на конверсию %s

В таблице 22.10 показано, как преобразование %g отображает одни числа в форме %e, а другие - в форме %f. Все числа в таблице были записаны с использованием спецификации преобразования %.4g. Первые два числа имеют степень не менее 4, поэтому они отображаются в форме %e. Следующие восемь чисел отображаются в форме %f. Последние два числа имеют показатель степени меньше -4, поэтому они отображаются в форме %e.

<i>Number</i>	<i>Result of Applying %.4g Conversion to Number</i>
123456.	1.235e+05
12345.6	1.235e+04
1234.56	1235
123.456	123.5
12.3456	12.35
1.23456	1.235
.123456	0.1235
.0123456	0.01235
.00123456	0.001235
.000123456	0.0001235
.0000123456	1.235e-05
.00000123456	1.235e-06

Таблица 22.10 Примеры конверсии %g

Раньше мы предполагали, что минимальная ширина поля и точность были константами, встроенными в строку формата. Размещение символа * вместо любого числа позволяет нам указать его как аргумент после строки формата. Например, все следующие вызовы printf производят одинаковый вывод:

```
printf("%6.4d", i);
```

```
printf("%*.4d", 6, i);
```

```
printf("%6.*d", 4, i);
```

```
printf("%*.*d", 6, 4, i);
```

Обратите внимание, что значения, которые необходимо заполнить для *, идут непосредственно перед отображаемым значением. Между прочим, главное преимущество * в том,

1249

что он позволяет нам использовать макрос для указания ширины или точности:

```
printf ("%*d", WIDTH, i);
```

Мы даже можем вычислить ширину или точность во время выполнения программы:

```
printf ("%*d", page_width/num_cols, i);
```

Самые необычные спецификации — это %p и %n.

Преобразование %p позволяет нам распечатать значение указателя:

```
printf ("%p", (void *) ptr); // отображает значение ptr
```

Хотя %p иногда бывает полезен во время отладки, это не та функция, которую большинство программистов используют ежедневно. Стандарт C не определяет, как выглядит указатель при печати с использованием %p, но он, вероятно, будет отображаться как восьмеричное или шестнадцатеричное число.

Преобразование %n используется, чтобы узнать, сколько символов было напечатано на данный момент с помощью вызова... printf. Например, после вызова

```
printf ("%d%n\n", 123, &len);
```

значение len будет равно 3, так как printf записал 3 символа (123) к тому времени, когда достиг %n. Обратите внимание,

1250

что & должен предшествовать len (поскольку %n требует указателя), а сама len не печатается.

Функции...scanf

```
int fscanf(FILE * restrict stream, const char * restrict format, ...);  
int scanf(const char * restrict format,...);
```

fscanf и scanf считывают элементы данных из входного потока, используя строку формата для указания расположения входных данных. После строки формата в качестве дополнительных аргументов следует любое количество указателей, каждый из которых указывает на объект. Элементы ввода преобразуются (согласно спецификациям преобразования в строке формата) и сохраняются в этих объектах. scanf всегда читает из stdin (стандартный входной поток), тогда как fscanf читает из потока, указанного его первым аргументом:

```
scanf("%d%d", &i, &j); /* читает из stdin */
```

```
fscanf(fp, "%d%d", &i, &j); /* читает из fp */
```

Функции... scanf возвращаются преждевременно, если происходит сбой ввода (больше не удаётся прочитать входные символы) или если происходит сбой сопоставления (входные

символы не совпадают со строкой формата), (В С99 сбой ввода также может произойти из-за ошибки кодирования, что означает, что была предпринята попытка прочитать многобайтовый символ, но входные символы не соответствовали никакому допустимому многобайтовому символу.) Обе функции возвращают количество элементов данных, которые были прочитаны и присвоены объектам; они возвращают EOF, если ошибка ввода происходит до того, как какие-либо элементы данных могут быть прочитаны. Циклы, в которых проверяется возвращаемое значение scanf, обычны в программах на языке С. Следующий цикл, например, считывает серию целых чисел одно за другим, останавливаясь при первых признаках проблемы:

```
идиома: while (scanf("%d", &i) == 1) {  
    ...  
}
```

Строки формата...scanf

Вызов функций..scanf аналогичен вызовам функций...printf. Однако это сходство может вводить в заблуждение; Функции... scanf работают совершенно иначе, чем функции ...printf. Стоит думать о scanf и fscanf как о функциях "сопоставления с образцом". Строка формата представляет

собой образец, которому функция...scanf пытается сопоставить данные при чтении ввода. Если ввод не соответствует строке формата, функция вернётся, как только обнаружит несоответствие; введённый символ, который не соответствует, "отодвигается" для чтения в будущем. Строка формата... scanf может содержать три вещи:

- **Спецификации преобразования.** Спецификации преобразования в строке формата ...scanf аналогичны спецификациям в строке формата ...printf. Большинство спецификаций преобразования пропускают символы пробела в начале элемента ввода (за исключением %[,%c и %n). Однако спецификации преобразования никогда не пропускают завершающие символы пробела. Если ввод содержит ●123☼, спецификация преобразования %d потребляет ●, 1, 2 и 3, но оставляет непрочитанным ☼ (я использую ● для обозначения символа пробела и ☼ для представления символа новой строки).
- **Символы пробела.** Один или несколько последовательных символов пробела в строке формата... scanf соответствуют нулю или нескольким символам пробела во входном потоке.
- **Символы, отличные от пробелов.** Символ, не являющийся пробелом, отличный от %, соответствует тому же символу во входном потоке.

1253

Например, строка формата "ISBN%d-%d-%ld-%d" указывает, что ввод будет состоять из:

букв ISBN

возможно, некоторых символов пробела

целого числа

символа -

целого числа (возможно, ему предшествуют символы пробела)

символа -

длинного целого числа (long) (возможно, которому предшествуют символы пробела)

символа -

целого числа (возможно, ему предшествуют символы пробела)

Спецификации преобразования ...scanf

Спецификации преобразования для функций ... scanf на самом деле немного проще, чем для функций ...printf. Спецификация преобразования ...scanf состоит из символа %, за которым следуют элементы, перечисленные ниже (в указанном порядке).

- х * (необязательно), наличие * означает подавление присвоения: элемент ввода считывается, но не присваивается объекту. Элементы, сопоставленные с помощью *, не включаются в счётчик, возвращаемый ... scanf.
- х **Максимальная ширина поля** (необязательно). Максимальная ширина поля ограничивает количество символов в элементе ввода; преобразование элемента заканчивается, если это число достигнуто. Пробелы, пропущенные в начале преобразования, не учитываются.
- х **Модификатор длины** (необязательно). Наличие модификатора длины указывает на то, что объект, в котором будет храниться элемент ввода, имеет тип, который длиннее или короче, чем обычно для конкретной спецификации преобразования. Таблица 22.11 перечисляет каждый модификатор длины, спецификаторы преобразования, с которыми он может использоваться, и тип, указанный комбинацией этих двух (Любая комбинация модификатора длины и спецификатора преобразования, не показанная в таблице, вызывает неопределённое поведение).

Модифи- катор длины	Спецификатор преобразования	Значение
---------------------------	--------------------------------	----------

1255

hh	d, i, o, u, x, X, n	signed char, unsigned char
h	d, i, o, u, x, X, n	short int, unsigned short int
l (<i>ell</i>)	d, i, o, u, x, X, n	long int, unsigned long int
	a, A, e, E, f, F, g, G	double
	c, s, или [	wchar_t
ll (<i>ell-ell</i>)	d, i, o, u, x, X, n	long long int, unsigned long long int
j	d, i, o, u, x, X, n	intmax_t, uintmax_t
z	d, i, o, u, x, X, n	size_t
t	d, i, o, u, x, X, n	ptrdiff_t
L	a, A, e, E, f, F, g, G	long double

Таблица 22.11 Модификатор длины для функций...scanf

Спецификатор преобразования. Спецификатор преобразования должен быть одним из символов, перечисленных в таблице 22.12.

Спецификатор	Значение
--------------	----------

d	Соответствует десятичному целому числу; предполагается, что соответствующий аргумент имеет тип int
i	Соответствует целому числу; предполагается, что соответствующий аргумент имеет тип int,

предполагается, что целое число находится в базе 10, если оно не начинается с 0 (указывающее восьмеричное число) или с 0x или 0X (шестнадцатеричное число).

- o Соответствует восьмеричному целому числу; предполагается, что соответствующий аргумент имеет тип `unsigned int`
- u Соответствует восьмеричному целому числу; предполагается, что соответствующий аргумент имеет тип `unsigned int`
- x, X Соответствует шестнадцатеричному целому числу; предполагается, что соответствующий аргумент имеет тип `unsigned int`
- a, A, e, E, f, F, g, G Соответствует числу с плавающей точкой; предполагается, что соответствующий аргумент имеет тип `float`. В C99 число может быть бесконечным или NaN.
- c Соответствует `n` символам, где `n` - максимальная ширина поля, или одному символу, если ширина поля не указана. Предполагается, что соответствующий аргумент является указателем на массив символов (или символьный объект, если ширина поля не указана). Не добавляет в конец нулевой символ.
- s Соответствует последовательности символов, отличных от пробельных, а затем добавляет символ `NULL` в конце. Соответствующий аргумент предполагается указателем на символьный массив.
- [Соответствует непустой последовательности

символов из набора сканирования, а затем добавляет в конец нулевой символ.

Предполагается, что соответствующий аргумент является указателем на массив символов.

- | | |
|---|---|
| p | Соответствует значению указателя в форме, в которой... printf записала бы его. Соответствующий аргумент предполагается указателем на объект void *. |
| n | Соответствующий аргумент должен указывать на объект типа int. Сохраняет в этом объекте количество символов, прочитанных на данный момент этим вызовом -.. scanf. Никакой ввод не используется, и на возвращаемое значение... scanf не влияет. |
| % | Соответствует символу % |

Таблица 22.12 Спецификаторы преобразования для функции...scanf.

Элементы числовых данных всегда могут начинаться со знака (+ или -). Однако спецификаторы o, u, x и X преобразуют элемент в беззнаковую форму, поэтому обычно они не используются для чтения отрицательных чисел.

Спецификатор *[* - это более сложная (и более гибкая) версия спецификатора *s*. Полная спецификация преобразования с использованием *[* имеет вид %[set] или %[[^] set], где set может

быть любым набором символов. (Если `[` является одним из символов в наборе, то он должен стоять первым.)

`%[set]` соответствует любой последовательности символов в наборе (наборе сканирования), `%[^ set]` соответствует любой последовательности символов, не входящей в набор (другими словами, набор сканирования состоит из всех символов, не входящих в набор). Например, `%[abc]` соответствует любой строке, содержащей только буквы `a`, `b` и `c`, а `%[^ abc]` соответствует любой строке, которая не содержит `a`, `b` или `c`.

Многие из спецификаторов преобразования... `scanf` тесно связаны с функциями преобразования чисел в `<stdlib.h>`. Эти функции преобразуют строки (например, `"-297"`) в их эквивалентные числовые значения (`-297`). Спецификатор ***d***, например, ищет необязательный знак `+` или `-`, за которым следует последовательность десятичных цифр; это точно такая же форма, которую требует функция `strtol`, когда её просят преобразовать строку в десятичное число. В таблице 22.13 показано соответствие между спецификаторами преобразования и функциями числового преобразования.

<i>Conversion Specifier</i>	<i>Numeric Conversion Function</i>
<code>d</code>	<code>strtol</code> with 10 as the base
<code>i</code>	<code>strtol</code> with 0 as the base
<code>o</code>	<code>strtoul</code> with 8 as the base
<code>u</code>	<code>strtoul</code> with 10 as the base
<code>x, X</code>	<code>strtoul</code> with 16 as the base
<code>a, A, e, E, f, F, g, G</code>	<code>strtod</code>

Таблица 22.13 Соответствие между спецификаторами преобразования и функциями числового преобразования.

Будьте осторожны при написании вызовов `scanf`.

Недопустимая спецификация преобразования в строке формата `scanf` так же плоха, как и спецификация в строке формата `printf`; любая из них вызывает неопределённое поведение.

C99 Изменения в спецификациях преобразования ... scanf

Спецификации преобразования для `scanf` и `fscanf` претерпели некоторые изменения в C99, но список не такой обширный, как для функций...`printf`:

- ✓ **Дополнительные модификаторы длины.** C99 добавляет модификаторы длины `hh`, `ll`, `j`, `z` и `t`. Они соответствуют модификаторам длины в спецификациях преобразования... `printf`.
- ✓ **Дополнительные спецификаторы преобразования.** C99 добавляет спецификаторы преобразования `F`, `a` и `A`. Они предусмотрены для симметрии с... `printf`; функции... `scanf` обрабатывают их так же, как `e`, `E`, `f`, `g` и `G`.
- ✓ **Умение читать бесконечность и NaN.** Так же, как функции... `printf` могут записывать бесконечность и NaN,

функции... `scanf` могут читать эти значения. Для правильного чтения они должны иметь такой же вид, как значения, записанные функциями `--printf`, с игнорированием регистра. (Например, `INF` или `inf` будут читаться как бесконечность.)

- ✓ **Поддержка широких символов.** Функции... `scanf` могут читать многобайтовые символы, которые затем преобразуются в широкие символы для хранения. Спецификация преобразования `%lc` используется для чтения одного многобайтового символа или последовательности многобайтовых символов; `%ls` используется для чтения строки многобайтовых символов (в конце добавляется нулевой символ). Спецификации преобразования `%l[set]` и `%l[^set]` также могут считывать строку многобайтовых символов.

Примеры scanf

Следующие три таблицы содержат примеры вызовов `scanf`. Каждый вызов применяется к входным символам, показанным справа от него. Символы, напечатанные ~~зачёркиванием~~, потребляются вызовом. Значения переменных после вызова отображаются справа от входных данных.

Примеры в Таблице 22.14 показывают эффект объединения спецификаций преобразования, символов пробела и символов, отличных от пробелов. В трёх случаях `j` не присваивается никакого значения, поэтому оно сохраняет своё значение до

1261

вызова `scanf`. Примеры в Таблице 22.15 показывают эффект подавления присваивания и указания ширины поля. Примеры в Таблице 22.16 иллюстрируют более скрытые спецификаторы преобразования (`i`, `[` и `n`).

<i>scanf Call</i>	<i>Input</i>	<i>Variables</i>
<code>n = scanf("%d%d", &i, &j);</code>	<code>12*,*34</code>	n: 1 i: 12 j: unchanged
<code>n = scanf("%d,%d", &i, &j);</code>	<code>12*,*34</code>	n: 1 i: 12 j: unchanged
<code>n = scanf("%d ,%d", &i, &j);</code>	<code>12*,*34</code>	n: 2 i: 12 j: 34
<code>n = scanf("%d, %d", &i, &j);</code>	<code>12*,*34</code>	n: 1 i: 12 j: unchanged

Таблица 22.14 Примеры `scanf` (Группа1)

<i>scanf Call</i>	<i>Input</i>	<i>Variables</i>
<code>n = scanf("%*d%d", &i);</code>	<code>12*34</code>	n: 1 i: 34
<code>n = scanf("%*s%s", str);</code>	<code>My*Fair*Lady</code>	n: 1 str: "Fair"
<code>n = scanf("%1d%2d%3d", &i, &j, &k);</code>	<code>12345</code>	n: 3 i: 1 j: 23 k: 45
<code>n = scanf("%2d%2s%2d", &i, str, &j);</code>	<code>123456</code>	n: 3 i: 12 str: "34" j: 56

Таблица 22.15 Примеры scanf (Группа 2)

<i>scanf Call</i>	<i>Input</i>	<i>Variables</i>
<code>n = scanf("%i%i%i", &i, &j, &k);</code>	<code>12 0 12 0 x12</code>	n: 3 i: 12 j: 10 k: 18
<code>n = scanf("%[0123456789]", str);</code>	<code>123abc</code>	n: 1 str: "123"
<code>n = scanf("%[0123456789]", str);</code>	<code>abc123</code>	n: 0 str: unchanged
<code>n = scanf("%[^0123456789]", str);</code>	<code>abc123</code>	n: 1 str: "abc"
<code>n = scanf("%*d%d%n", &i, &j);</code>	<code>10 20 30</code>	n: 1 i: 20 j: 5

Таблица 22.16 Примеры scanf (Группа 3)

Обнаружение условий окончания файла и ошибок

```
void clearerr(FILE *stream);
```

```
int feof(FILE *stream);
```

```
int ferror(FILE *stream);
```

Если мы попросим функцию...scanf прочитать и сохранить ***n*** элементов данных, мы ожидаем, что её возвращаемое

значение будет *n*. Если возвращаемое значение меньше *n*, что-то пошло не так. Есть три возможности:

- ➔ **Конец файла.** Функция обнаружила конец файла до полного совпадения строки формата.
- ➔ **Ошибка чтения.** Функция не смогла прочитать символы из потока.
- ➔ **Ошибка соответствия.** Элемент данных был в неправильном формате. Например, функция могла встретить букву при поиске первой цифры целого числа.

Но как мы можем определить, какая именно неисправность произошла? Во многих случаях это не имеет значения; что-то пошло не так, и мы должны отказаться от программы. Однако могут быть моменты, когда нам потребуется определить причину сбоя.

Каждый поток имеет два связанных с ним индикатора: индикатор ошибки и индикатор конца файла. Эти индикаторы сбрасываются при открытии потока. Неудивительно, что обнаружение конца файла устанавливает индикатор конца файла, а ошибка чтения устанавливает индикатор ошибки. (Индикатор ошибки также устанавливается при возникновении ошибки записи в потоке вывода.) Ошибка сопоставления не меняет ни один из индикаторов.

После того, как индикатор ошибки или конца файла установлен, он остаётся в этом состоянии до тех пор, пока не будет явно очищен, возможно, с помощью вызова функции `clearerr`. `clearerr` очищает индикаторы конца файла и ошибки:

```
clearerr (fp); /* очищает индикаторы eof и error для fp */
```

`clearerr` требуется нечасто, поскольку некоторые другие функции библиотеки сбрасывают один или оба индикатора как побочный эффект. Мы можем вызвать функции `feof` и `ferror`, чтобы проверить индикаторы потока, чтобы определить, почему предыдущая операция над потоком не удалась. Вызов `feof (fp)` возвращает ненулевое значение, если индикатор конца файла установлен для потока, связанного с `fp`. Вызов `ferror (fp)` возвращает ненулевое значение, если установлен индикатор ошибки. В противном случае обе функции возвращают ноль. Когда `scanf` возвращает значение меньше ожидаемого, мы можем использовать `feof` и `ferror`, чтобы определить причину. Если `feof` возвращает ненулевое значение, мы достигли конца входного файла. Если `ferror` возвращает ненулевое значение, во время ввода произошла ошибка чтения. Если ни один из них не возвращает ненулевое значение, значит, произошла ошибка сопоставления. Независимо от того, в чем была проблема, возвращаемое значение `scanf` сообщает нам, сколько элементов данных было прочитано до возникновения проблемы.

1265

Чтобы увидеть, как можно использовать `feof` и `ferror`, давайте напишем функцию, которая ищет в файле строку, начинающуюся с целого числа. Вот как мы собираемся вызвать функцию:

```
n = find_int ("foo");
```

"foo" - это имя файла для поиска. Функция возвращает значение найденного целого числа, которое затем присваивается *n*. Если возникает проблема - файл не открывается, возникает ошибка чтения или ни одна строка не начинается с целого числа - `find_int` вернёт код ошибки (-1, -2 или -3 соответственно). Я предполагаю, что ни одна строка в файле не начинается с отрицательного целого числа.

```
int find_int(const char *filename){  
    FILE *fp = fopen(filename, "r");  
    int n;  
    if (fp == NULL)  
        return -1; /* не могу открыть файл */
```

1266

```
while (fscanf(fp, "%d", &n) != 1) {
    if (ferror(fp)) {
        fclose (fp);
        return -2; /* ошибка чтения*/
    }
    if (feof(fp)) {
        fclose (fp) ;
        return -3; /* целое число не найдено */
    }
    fscanf(fp, "%*[^\\n]"); /* пропускает остаток строки */

}

fclose (fp);

return n;

}
```

Управляющее выражение цикла while вызывает fscanf, пытаясь прочитать целое число из файла. Если попытка не удалась (fscanf возвращает значение, отличное от 1), find_int вызывает ferror и feof, чтобы узнать, была ли проблема ошибкой чтения или концом файла. В противном случае fscanf, должно быть, завершился неудачно из-за ошибки сопоставления, поэтому find_int пропускает остальные

символы в текущей строке и пытается снова. Обратите внимание на использование преобразования `%*[\n]` для пропуска всех символов до следующей новой строки. (Теперь, когда мы знаем о наборах сканирования, пора похвастаться!)

22.4 Ввод/вывод символов

В этом разделе мы рассмотрим библиотечные функции, которые читают и записывают одиночные символы. Эти функции одинаково хорошо работают с текстовыми потоками и двоичными потоками.

Вы заметите, что функции в этом разделе обрабатывают символы как значения типа `int`, а не `char`. Одна из причин заключается в том, что входные функции указывают на условие конца файла (или ошибки), возвращая EOF, который является отрицательной целочисленной константой.

Функции вывода

```
int fputc(int c, FILE *stream);
```

```
int putc(int c, FILE *stream);
```

```
int putchar(int c);
```

1268

putchar записывает один символ в поток stdout:

```
putchar (ch); /* записывает ch в стандартный вывод */
```

fputc и putc - более общие версии putchar, которые записывают символ в произвольный поток:

```
fputc (ch, fp); /* записывает ch в fp */
```

```
putc (ch, fp); /* записывает ch в fp */
```

Хотя put и fputc делают одно и то же, putc обычно реализуется как макрос (а также как функция), а fputc реализуется только как функция. Сам putchar обычно представляет собой макрос, определённый следующим образом:

```
#define putchar(c) putc((c), stdout)
```

Может показаться странным, что в библиотеке есть как putc, так и fputc. Но, как мы видели в Разделе 14.3, у макросов есть несколько потенциальных проблем. Стандарт C позволяет макросу putc оценивать аргумент потока более одного раза, что не разрешено делать fputc. Хотя программисты обычно предпочитают putc, который даёт более быструю программу, в качестве альтернативы доступен fputc.

Если происходит ошибка записи, все три функции устанавливают индикатор ошибки для потока и возвращают EOF; в противном случае они возвращают написанный символ.

Функции ввода

```
int fgetc(FILE *stream);
```

```
int getc(FILE *stream);
```

```
aint getchar (void);
```

```
int ungetc(int c, FILE *stream);
```

getchar считывает символ из потока stdin:

```
ch = getchar (); /* читает символ из stdin */
```

fgetc и getc читает символ из произвольного потока:

```
fgetc (fp); /* считывает символ из fp */
```

```
getc (fp); /* считывает символ из fp */
```

Все три функции обрабатывают символ как беззнаковое значение char (которое затем преобразуется в тип int перед его возвращением). В результате они никогда не возвращают отрицательное значение, отличное от EOF.

Отношения между getc и fgetc аналогичны отношениям между puts и fputc. getc обычно реализуется как макрос (а также как функция), в то время как fgetc реализуется только как функция. getchar обычно также является макросом:

```
#define getchar() getc(stdin)
```

Для чтения символов из файла программисты обычно предпочитают `getc` вместо `fgetc`. Поскольку `getc` обычно доступен в форме макроса, он, как правило, работает быстрее. `fgetc` можно использовать в качестве резерва, если `getc` не подходит. (Стандарт позволяет макросу `getc` вычислять свой аргумент более одного раза, что может быть проблемой.)

Функции `fgetc`, `getc` и `getchar` ведут себя одинаково, если возникает проблема. В конце файла они устанавливают индикатор конца файла и возвращают EOF. Если происходит ошибка чтения, они устанавливают индикатор ошибки потока и возвращают EOF. Чтобы различать эти две ситуации, мы можем вызвать либо `feof`, либо `ferror`.

Одним из наиболее распространённых применений `fgetc`, `getc` и `getchar` является чтение символов из файла, один за другим, до появления конца файла. Для этой цели обычно используется следующий цикл `while`:

идиома:

```
while ((ch = getc(fp)) != EOF) {  
    ...  
}
```

После чтения символа из файла, связанного с `fp`, и сохранения его в переменной `ch` (которая должна иметь тип `int`), тест `while` сравнивает `ch` с `EOF`. Если `ch` не равен `EOF`, мы ещё не в конце файла, поэтому выполняется тело цикла. Если `ch` равно `EOF`, цикл завершается.

Всегда сохраняйте возвращаемое значение `fgetc`, `getc` или `getchar` в переменной типа `int`, а не в переменной типа `char`. Тестирование переменной типа `char` на соответствие `EOF` может дать неверный результат.

Есть ещё одна функция ввода символов, `ungetc`, которая "отодвигает" символ, считанный из потока, и очищает индикатор конца файла потока. Эта возможность может быть удобна, если нам нужен "опережающий" символ во время ввода. Например, чтобы прочитать серию цифр, останавливаясь на первой нецифровой, мы могли бы написать

```
while (isdigit (ch = getc (fp))) {
```

```
    ...
```

```
}
```

```
ungetc (ch, fp); //возвращает последний прочитанный символ
```

Число символов, которые могут быть возвращены последовательными вызовами `ungetc` - без промежуточных

операций чтения - зависит от реализации и типа задействованного потока; только первый вызов гарантированно будет успешным. Вызов функции позиционирования файла (fseek, fsetpos или rewind) приводит к потере возвращаемых символов.

ungetc возвращает символ, который был запрошен для возврата, однако он возвращает EOF, если предпринимается попытка вернуть EOF или вернуть больше символов, чем позволяет реализация.

ПРОГРАММА Копирование файла

Следующая программа делает копию файла. Имена исходного файла и нового файла будут указаны в командной строке при выполнении программы. Например, чтобы скопировать файл f1.c в f2.c, мы использовали бы команду

```
fcopy f1.c f2.c;
```

fcopy выдаст сообщение об ошибке, если в командной строке не указано ровно два имени файла или если любой файл не может быть открыт.

```
//fcopy.c
```

1273

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
    FILE *source_fp, *dest_fp;
    int ch;
    if (argc != 3) {
        fprintf(stderr, "usage: fcopy source dest\n");
        exit(EXIT_FAILURE);
    }
    if ((source_fp = fopen(argv[1], "rb")) == NULL) {
        fprintf(stderr, "Can't open %s\n", argv[1]);
        exit(EXIT_FAILURE);
    }

    if ((dest_fp = fopen(argv[2], "wb")) == NULL) {
        fprintf(stderr, "Can't open %s\n", argv[2]);
        fclose(source_fp);
        exit(EXIT_FAILURE);
    }

    while ((ch = getc(source_fp)) != EOF)
```

1274

```
        putc(ch, dest_fp);  
fclose(source_fp);  
fclose(dest_fp);  
return 0;  
}
```

Использование "rb" и "wb" в качестве файловых режимов позволяет копировать как текстовые, так и двоичные файлы. Если бы мы использовали вместо этого "r" и "w", программа не обязательно могла бы копировать двоичные файлы.

22.5 Ввод/вывод строк

Теперь мы обратимся к библиотечным функциям, которые читают и пишут строки. Эти функции используются в основном с текстовыми потоками, хотя вполне законно использовать их и с двоичными потоками.

Функции вывода

```
int fputs(const char * restrict s, FILE * restrict stream);  
int puts(const char *s) ;
```

Мы столкнулись с функцией `puts` в разделе 13.3; она записывает строку символов в `stdout`:

```
puts ("Эй, вы, там!"); /* пишет в stdout*/
```

После того как он записывает символы в строку, `puts` всегда добавляет символ новой строки.

`fputs` - это более общая версия `puts`. её второй аргумент указывает на поток, в который должен быть записан вывод:

```
fputs("Эй, вы, там!", fp); /* пишет в fp */
```

В отличие от `puts`, функция `fputs` не записывает символ новой строки, если он не присутствует в строке.

Обе функции возвращают EOF в случае ошибки записи; в противном случае они возвращают неотрицательное число.

Функции ввода

```
char * fgets (char * restrict s, int n, FILE * restrict stream);
```

```
char * gets (char * s);
```

Функция `gets`, с которой мы впервые столкнулись в Разделе 13.3, считывает строку ввода из `stdin`:

```
gets(str); /* читает строку из стандартного ввода */
```

gets читает символы один за другим, сохраняя их в массиве, на который указывает str, до тех пор, пока не прочитает символ новой строки (который она отбрасывает).

fgets - это более общая версия gets, которая может читать из любого потока. fgets также безопаснее, чем gets, поскольку ограничивает количество сохраняемых символов. Вот как мы можем использовать fgets, предполагая, что str - это имя массива символов:

```
fgets (str, sizeof(str), fp); /* читает строку из fp */
```

Этот вызов заставит fgets читать символы до тех пор, пока она не достигнет первого символа новой строки или не будет прочитан sizeof (str)-1 символ, в зависимости от того, что произойдет раньше. Если она читает символ новой строки, gets сохраняет его вместе с другими символами. (Таким образом, gets никогда не сохраняет символ новой строки, но fgets иногда это делает.)

И get, и fgets возвращают нулевой указатель, если происходит ошибка чтения или они достигают конца входного потока до сохранения каких-либо символов. (Как обычно, мы можем вызвать feof или ferror, чтобы определить, какая ситуация возникла.) В противном случае оба возвращают свой первый аргумент, который указывает на массив, в котором был сохранен ввод. Как и следовало ожидать, обе функции сохраняют нулевой символ в конец строки.

Теперь, когда вы знаете о `fgets`, я бы посоветовал использовать его вместо `gets` в большинстве ситуаций. С помощью `gets` всегда есть возможность выйти за пределы принимающего массива, поэтому безопасно использовать только тогда, когда гарантировано, что читаемая строка помещается в массив. Когда нет гарантии (а обычно её нет), гораздо безопаснее использовать `fgets`. Обратите внимание, что `fgets` будет читать из стандартного входного потока, если в качестве третьего аргумента передаётся `stdin`:

```
fgets (str, sizeof (str), stdin);
```

22.6 Ввод / вывод блока

```
size_t fread(void * restrict ptr, size_t size, size_t nmemb, FILE *  
restrict stream);
```

```
size_t fwrite(const void * restrict ptr, size_t size, size_t nmemb,  
FILE * restrict stream) ;
```

Функции `fread` и `fwrite` позволяют программе читать и записывать большие блоки данных за один шаг. `fread` и `fwrite` используются в основном с двоичными потоками, хотя - осторожно - их можно использовать и с текстовыми потоками.

`fwrite` предназначена для копирования массива из памяти в поток. Первый аргумент в вызове `fwrite` - это адрес массива, второй аргумент - это размер каждого элемента массива (в байтах), а третий аргумент - это количество элементов для записи. Четвёртый аргумент - это указатель файла, указывающий, куда должны быть записаны данные.

Например, чтобы записать все содержимое массива ***a***, мы могли бы использовать следующий вызов `fwrite`:

```
fwrite(a, sizeof(a[0]), sizeof(a) / sizeof(a[0]), fp);
```

Нет никакого правила обязывающего записать весь массив; мы могли бы так же легко записать любую его часть. `fwrite` возвращает количество фактически записанных элементов (не байт). Это число будет меньше третьего аргумента, если произойдёт ошибка записи.

*(Функция `fread` считывает массив размером `sizeof(a) / sizeof(a[0])` элементов, каждый из которых имеет размер `sizeof(a[0])` байт, из потока, и сохраняет его в блоке памяти, на который указывает ***a***.)*

`fread` будет читать элементы массива из потока. Аргументы `fread` аналогичны аргументам `fwrite`: адрес массива, размер каждого элемента (в байтах), количество элементов для чтения и указатель файла. Чтобы прочитать содержимое файла (***и сохранить***) в массиве ***a***, мы могли бы использовать следующий вызов `fread`:

```
n = fread(a, sizeof(a[0]), sizeof(a) / sizeof(a[0]), fp);
```

Важно проверить возвращаемое значение `fread`, которое указывает фактическое количество прочитанных элементов (не байтов). Это число должно быть равно третьему аргументу, если не был достигнут конец входного файла или не произошла ошибка чтения. Можно использовать функции `ferror` или `feof` для определения проблемы - произошла ошибка или был достигнут конец файла.

Будьте осторожны, чтобы не перепутать второй и третий аргументы. Рассмотрим следующий вызов `fread`:

```
fread (a, 1, 100, fp)
```

Мы просим `fread` прочитать 100 однобайтовых элементов, поэтому он вернёт значение от 0 до 100. Следующий вызов просит `fread` прочитать один блок размером 100 байт:

```
fread (a, 100, 1, fp)
```

Возвращаемое значение `fread` в этом случае будет либо 0, либо 1.

`fwrite` удобна для программы, которой необходимо сохранить данные в файле перед завершением работы. Позже программа (или другая программа, если на то пошло) может использовать `fread` для чтения данных обратно в память.

Несмотря на внешний вид, данные не обязательно должны быть в виде массива; `fread` и `fwrite` одинаково хорошо работают с переменными всех типов. В частности, структуры можно читать с помощью `fread` или писать с помощью `fwrite`.

Например, чтобы записать структурную переменную *s* в файл, мы могли бы использовать следующий вызов `fwrite`:

```
fwrite (&s, sizeof (s), 1, fp);
```

Будьте осторожны при использовании `fwrite` для записи структур, содержащих значения указателей; не гарантируется, что эти значения будут действительными при повторном считывании.

22.7 Позиционирование файлов

```
int fgetpos(FILE * restrict stream, fpos_t * restrict pos);
```

```
int fseek(FILE *stream, long int offset, int whence) ;
```

```
int fsetpos(FILE *stream, const fpos_t *pos) ;
```

```
long int ftell(FILE *stream) ;
```

```
void rewind(FILE *stream) ;
```

Каждый поток имеет соответствующую позицию файла. При открытии файла позиция файла устанавливается в начале файла. (Однако, если файл открыт в режиме "добавить", начальная позиция файла может находиться в начале или в конце файла, в зависимости от реализации.) Затем, когда выполняется операция чтения или записи, позиция файла автоматически перемещается, позволяя нам последовательно

перемещаться по файлу. Хотя последовательный доступ подходит для многих приложений, некоторым программам требуется возможность перемещаться по файлу, получая доступ к некоторым данным здесь и другим данным там. Например, если файл содержит серию записей, нам может потребоваться перейти непосредственно к определённой записи и прочитать или обновить её. `<stdio.h>` поддерживает эту форму доступа, предоставляя пять функций, которые позволяют программе определять текущую позицию файла или изменять её.

Функция `fseek` изменяет позицию файла, связанную с первым аргументом (указателем файла). Третий аргумент указывает, должна ли вычисляться новая позиция относительно начала файла, текущей позиции или конца файл. `<stdio.h>` определяет для этой цели три макроса:\

`SEEK_SET` Начало файла

`SEEK_CUR` Текущая позиция файла

`SEEK_END` Конец файла

Второй аргумент - это (возможно, отрицательное) количество байтов. Например, чтобы перейти к началу файла, направление поиска будет равно `SEEK_SET`, а количество байтов будет равно нулю:

1282

`fseek (fp, 0L, SEEK_SET); /* перемещается в начало файла */`
Чтобы перейти в конец файла, направление поиска должно быть `SEEK_END`:

`fseek (fp, 0L, SEEK_END); /* перемещается в конец файла */`
Чтобы переместиться на 10 байт назад, направление поиска должно быть `SEEK_CUR`, а количество байтов -10:

`fseek (fp, -10L, SEEK_CUR); /* перемещается на 10 байтов назад */`

Обратите внимание, что счётчик байтов имеет тип `long int`, поэтому я использовал `0L` и `-10L` в качестве аргументов. (0 и -10, конечно, также будут работать, поскольку аргументы автоматически преобразуются в правильный тип.)

Обычно `fseek` возвращает ноль. Если происходит ошибка (например, запрошенная позиция не существует), `fseek` возвращает ненулевое значение.

Кстати, функции позиционирования файлов лучше всего использовать с двоичными потоками. С не запрещает программам использовать их с текстовыми потоками, но требуется осторожность из-за различий в операционных системах. В частности, `fseek` чувствителен к тому, является ли поток текстовым или двоичным. Для текстовых потоков либо (1) `offset` (второй аргумент `fseek`) должен быть равен нулю, либо (2) `whence` (его третий аргумент) должен быть `SEEK_SET`, а `offset` - значение, полученное предыдущим

вызовом `ftell`. (Другими словами, мы можем использовать `fseek` только для перехода к началу или концу текстового потока или для возврата к месту, которое было посещено ранее.) В двоичном режиме реализации не обязаны поддерживать режим `SEEK_END`.

Функция `ftell` возвращает текущую позицию файла как целое число `long`. (Если происходит ошибка, `ftell` возвращает `-1L` и сохраняет код ошибки в `errno`.) Значение, возвращаемое `ftell`, может быть сохранено и позже передано при вызове `fseek`, что позволяет вернуться к предыдущей позиции файла:

```
long file pos;
```

```
filepos = ftell(fp) ; /*сохраняет текущую позицию*/
```

```
...
```

```
fseek(fp, filepos, SEEK_SET) ; /* возвращается в старое  
положение*/
```

Если `fp` - двоичный поток, вызов `ftell (fp)` возвращает текущую позицию файла в виде счётчика байтов, где ноль представляет начало файла. Однако, если `fp` - это текстовый поток, `ftell (fp)` не обязательно является счётчиком байтов. В результате лучше не выполнять арифметические действия со значениями, возвращаемыми `ftell`. Например, не рекомендуется вычитать значения, возвращаемые `ftell`, чтобы увидеть, насколько далеко друг от друга находятся две позиции файла.

Функция `rewind` устанавливает позицию файла в начале.

Вызов `rewind (fp)` почти эквивалентен

`fseek (fp, 0L, SEEK_SET)`

Различия? `rewind` не возвращает значение, но очищает индикатор ошибки для `fp`.

У `fseek` и `ftell` есть одна проблема: они ограничены файлами, позиции которых могут быть сохранены в виде длинных целых чисел (`long`). Для работы с очень большими файлами C предоставляет две дополнительные функции: `fgetpos` и `fsetpos`. Эти функции могут обрабатывать большие файлы, поскольку они используют значения типа `fpos_t` для представления позиций файлов. Значение `fpos_t` не обязательно является целым числом; это может быть, например, структура.

Вызов `fgetpos (fp, &file_pos)` сохраняет позицию файла, связанную с `fp`, в переменной `file_pos`. Вызов

`fsetpos (fp, &file_pos)` устанавливает позицию файла для `fp` как значение, хранящееся в `file_pos`. (Это значение должно было быть получено предыдущим вызовом `fgetpos`.) Если вызов `fgetpos` или `fsetpos` завершился неудачно, код ошибки сохраняется в `errno`. Обе функции возвращают ноль при успешном выполнении и ненулевое значение при неудаче.

Вот как мы можем использовать `fgetpos` и `fsetpos`, чтобы сохранить позицию в файле и вернуться к ней позже:

1285

```
fpos_t file_pos;
```

```
...
```

```
fgetpos (fp, &file_pos); /* сохраняет текущую позицию */
```

```
...
```

```
fsetpos (fp, &file_pos); /* возвращается в старую позицию */
```

ПРОГРАММА Изменение файла записей деталей

Следующая программа открывает двоичный файл, содержащий структуры part, считывает структуры в массив, устанавливает член on_hand каждой структуры в 0, а затем записывает структуры обратно в файл. Обратите внимание, что программа открывает файл в режиме "rb +", что позволяет читать и писать.

```
// invclear.c
```

```
/*Изменяет файл записей деталей, устанавливая количество on_hand равным нулю для всех записей*/
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define NAME_LEN 25
```

```
#define MAX_PARTS 100
```

```
struct part {
```

1286

```
    int number;
    char name[NAME_LEN+1];
    int on_hand;
} inventory[MAX_PARTS];

int num_parts;

int main(void){
    FILE *fp;
    int i;
    if ((fp = fopen("inventory.dat", "rb+")) == NULL) {
        fprintf(stderr, "Can't open inventory file\n");
        exit(EXIT_FAILURE);
    }
    num_parts = fread(inventory, sizeof(struct part),
        MAX_PARTS, fp);

    for (i = 0; i < num_parts; i++)
        inventory[i].on_hand = 0;
    rewind(fp);
    fwrite(inventory, sizeof(struct part), num_parts, fp);
    fclose(fp);
}
```

```
    return 0;  
}
```

Кстати, вызов `rewind` критичен. После вызова `fread` позиция файла находится в конце файла. Если бы мы вызывали `fwrite` без предварительного вызова `rewind`, `fwrite` добавляла бы новые данные в конец файла вместо перезаписи старых.

22.8 Строковый ввод/вывод

Функции, описанные в этом разделе, немного необычны, так как они не имеют ничего общего с потоками или файлами. Вместо этого они позволяют нам читать и записывать данные, используя строку, как если бы это был поток. Функции `sprintf` и `snprintf` записывают символы в строку так же, как они были бы записаны в поток; функция `sscanf` считывает символы из строки, как если бы она считывала их из потока. Эти функции, которые очень похожи на `printf` и `scanf`, весьма полезны. `sprintf` и `snprintf` дают нам доступ к возможностям форматирования `printf` без фактической записи данных в поток. Точно так же `sscanf` предоставляет нам доступ к мощным возможностям `scanf` по сопоставлению шаблонов. В оставшейся части этого раздела подробно рассматриваются `sprintf`, `snprintf` и `sscanf`.

Три похожие функции (`vsprintf`, `vsnprintf` и `vsscanf`) также принадлежат `<stdio.h>`. Однако эти функции полагаются на тип `va_list`, который объявлен в `<stdarg.h>`. Я отложу их обсуждение до Раздела 26.1, в котором рассматривается этот заголовок.

ФУНКЦИИ ВЫВОДА

```
int sprintf(char * restrict s, const char * restrict format,...);
```

```
int snprintf(char * restrict s, size_t n, const char * restrict  
format,...);
```

Функция `sprintf` похожа на `printf` и `fprintf`, за исключением того, что она записывает вывод в массив символов (на который указывает его первый аргумент) вместо потока. Второй аргумент `sprintf` - это строка формата, идентичная той, что используется `printf` и `fprintf`. Например, вызов

```
sprintf (date, "%d/%d/%d", 9, 20, 2010);
```

запишет "20.09.2010" в `date`. По завершении записи в строку `sprintf` добавляет нулевой символ и возвращает количество сохраненных символов (не считая нулевого символа). Если возникает ошибка кодирования (широкий символ не может быть преобразован в допустимый многобайтовый символ), `sprintf` возвращает отрицательное значение. `sprintf` имеет множество применений. Например, иногда требуется

отформатировать данные для вывода, не записывая их. Мы можем использовать `sprintf` для форматирования, а затем сохранять результат в строке, пока не придёт время для вывода. `sprintf` также удобен для преобразования чисел в символьную форму.

Функция `snprintf` такая же, как `sprintf`, за исключением дополнительного параметра *n*. В строку будет записано не более *n*-1 символов, не считая завершающего нулевого символа, который всегда записывается, если *n* не равно нулю. (Точно так же можно сказать, что `snprintf` записывает в строку не более *n* символов, последний из которых является нулевым символом.) Например, вызов

```
snprintf(name, 13, "%s, %s", "Einstein", "Albert");
```

напишет "Einstein, Al" в `name`.

`snprintf` возвращает количество символов, которые были бы записаны (не включая нулевой символ), если бы не было ограничения на длину. Если происходит ошибка кодирования, `snprintf` возвращает отрицательное число. Чтобы узнать, есть ли у `snprintf` место для записи всех запрошенных символов, мы можем проверить, было ли возвращаемое значение неотрицательным и меньше *n*.

Функции ввода

```
int sscanf (const char * restrict s,  
            const char * restrict format,...);
```

Функция `sscanf` похожа на `scanf` и `fscanf`, за исключением того, что она читает из строки (на которую указывает её первый аргумент) вместо чтения из потока. Вторым аргументом `sscanf` - это строка формата, идентичная той, что используется `scanf` и `fscanf`.

`sscanf` удобна для извлечения данных из строки, которая была прочитана другой функцией ввода. Например, мы можем использовать `fgets` для получения строки ввода, а затем передать эту строку в `sscanf` для дальнейшей обработки:

```
fgets (str, sizeof(str), stdin); /* читает строку ввода */  
sscanf (str, "%d%d", &i, &j); /* извлекает два целых числа */
```

Одним из преимуществ использования `sscanf` вместо `scanf` или `fscanf` является то, что мы можем проверять строку ввода столько раз, сколько необходимо, а не один раз, что упрощает распознавание альтернативных форм ввода и восстановление после ошибок. Рассмотрим задачу чтения даты, записанной в форме `month/day/year` или `month-day-year`. Предполагая, что `str` содержит строку ввода, мы можем извлечь `month`, `day` и `year` следующим образом:

```
if (sscanf(str, "%d /%d /%d", &month, &day, &year) == 3)
    printf("Month: %d, day: %d, year: %d\n", month, day, year);
else if (sscanf(str, "%d-%d-%d", &month, &day, &year) == 3)
    printf("Month: %d, day: %d, year: %d\n", month, day, year);
else
    printf("Date not in the proper form\n") ;
```

Подобно функциям `scanf` и `fscanf`, `sscanf` возвращает количество успешно прочитанных и сохраненных элементов данных. `sscanf` возвращает EOF, если она достигает конца строки (отмеченного нулевым символом) до нахождения первого элемента.

Вопросы и ответы

Вопрос: Если я использую перенаправление ввода или вывода, будут ли имена перенаправленных файлов отображаться в качестве аргументов командной строки? \

Ответ: Нет; операционная система удаляет их из командной строки. Допустим, мы запускаем программу, введя

```
demo foo <in_file bar >out_file baz
```

Значение `argc` будет 4, `argv [0]` будет указывать на имя программы, `argv [1]` укажет на "foo", `argv [2]` будет указывать на "bar", а `argv [3]` будет указывать на " baz ".

Вопрос: Я думал, что конец строки всегда отмечен символом новой строки. Теперь вы говорите, что маркер конца строки меняется в зависимости от операционной системы. Чем вы объясняете это несоответствие?

Ответ: Функции библиотеки C создают впечатление, будто каждая строка заканчивается одним символом новой строки. Независимо от того, содержит ли входной файл символ возврата каретки, символ перевода строки или и то, и другое, библиотечная функция, такая как `getc`, вернёт один символ новой строки. Функции вывода выполняют обратное преобразование. Если программа вызывает библиотечную функцию для записи символа новой строки в файл, функция этот символ в соответствующий маркер конца строки. Подход C делает программы более переносимыми и простыми в написании; мы можем работать с текстовыми файлами, не беспокоясь о том, как на самом деле отображается конец строки. Обратите внимание, что ввод/вывод, выполняемый для файла, открытого в двоичном режиме, не подлежит преобразованию символов - возврат каретки и перевод строки обрабатываются так же, как и другие символы.

Вопрос: Я пишу программу, которая должна сохранять данные в файл, чтобы их позже прочитала другая программа. Что лучше хранить данные в текстовой или двоичной форме?

Ответ: Это зависит от обстоятельств. Если данные представляют собой только текст, особой разницы нет. Однако если данные содержат числа, решение усложняется.

Двоичная форма обычно предпочтительнее, так как её можно быстро читать и писать. Числа уже находятся в двоичной форме, когда они хранятся в памяти, поэтому их легко скопировать в файл. Запись чисел в текстовой форме происходит намного медленнее, поскольку каждое число должно быть преобразовано (обычно с помощью `fprintf`) в символьную форму. Чтение файла позже также займёт больше времени, поскольку числа должны быть преобразованы из текстовой формы обратно в двоичную. Более того, хранение данных в двоичной форме часто экономит место, как мы видели в Разделе 22.1.

Однако у двоичных файлов есть два недостатка. Их трудно читать людям, что может затруднить отладку. Кроме того, двоичные файлы обычно не переносятся из одной системы в другую, поскольку разные типы компьютеров хранят данные по-разному. Например, некоторые машины хранят значения типа `int`, используя два байта, а другие - четыре байта. Также существует проблема порядка следования байтов (прямой или обратный порядок байтов).

Вопрос: Программы C для UNIX, похоже, никогда не используют букву **b** в строке режима, даже если открываемый файл является двоичным. Как понять?

Ответ: В UNIX текстовые файлы и двоичные файлы имеют точно такой же формат, поэтому нет никакой необходимости использовать **b**. Однако программисты UNIX все равно должны включать **b**, чтобы для других их программы были более переносимыми.

Вопрос: Я видел программы, которые вызывают `foren` и помещают букву **t** в строку `mode`. Что это значит?

Ответ: Стандарт C допускает появление дополнительных символов в строке режима при условии, что они следуют за `g`, `w`, `a`, `b` или `+`. Некоторые компиляторы позволяют использовать **t**, чтобы указать, что файл должен быть открыт в текстовом режиме, а не в двоичном. Конечно, текстовый режим все равно используется по умолчанию, поэтому **t** ничего не добавляет. По возможности лучше избегать использования **t** и других непереносимых опций.

Вопрос: Зачем вообще использовать вызов функции `fclose` для закрытия файла? Разве не правда, что все открытые файлы автоматически закрываются при завершении программы?

Ответ: Обычно это так, но не в том случае, если программа вызывает `abort` для завершения. Однако даже когда `abort` не

используется, все равно есть веские причины вызвать `fclose`. Во-первых, это уменьшает количество открытых файлов. Операционные системы ограничивают количество файлов, которые программа может открыть одновременно; большие программы могут столкнуться с этим ограничением. (Макрос `FOPEN_MAX`, определённый в `<stdio.h>`, указывает минимальное количество файлов, которые реализация гарантирует одновременное открытие.) Во-вторых, программа становится легче для понимания и модификации; глядя на вызов `fclose`, читателю легче определить точку, в которой файл больше не используется. В-третьих, это вопрос безопасности. Заккрытие файла гарантирует, что его содержимое и запись в каталоге будут обновлены должным образом; если программа выйдет из строя позже, по крайней мере, файл останется нетронутым.

Вопрос: Я пишу программу, которая предложит пользователю ввести имя файла. Сколь велик должен быть массив символов, в котором будет храниться имя файла?

Ответ: Это зависит от вашей операционной системы. К счастью, вы можете использовать макрос `FILENAME_MAX` (определённый в `<stdio.h>`) для указания размера массива. `FILENAME_MAX` - это длина строки, которая будет содержать самое длинное имя файла, которое, как гарантирует реализация, может быть открыто.

Вопрос: Может ли fflush сбрасывать поток, открытый и для чтения, и для записи?

Ответ: Согласно стандарту C эффект вызова fflush определяется для потока, который (а) был открыт для вывода или (б) был открыт для обновления и последняя операция которого не была чтением. Во всех остальных случаях эффект вызова fflush не определён. Когда fflush передаётся нулевой указатель, он сбрасывает все потоки, удовлетворяющие либо (а), либо (б).

Вопрос: Может ли строка формата в вызове...printf или ...scanf быть переменной?

Ответ: Конечно; это может быть любое выражение типа char *. Это свойство делает функции...printf и...scanf ещё более универсальными, чем мы подозревали. Рассмотрим следующий классический пример из книги Кернигана и Ричи "Язык программирования C", который выводит аргументы командной строки программы, разделённые пробелами:

```
while (--argc > 0)
```

```
printf((argc > 1) ? "%s " : "%s", *++argv) ;
```

Строка формата - это выражение (argc > 1) ? "%s " : "%s", которое вычисляется как "%s " для всех аргументов командной строки, кроме последней.

Вопрос: Какие функции библиотеки, кроме clrerr, очищают индикаторы ошибок потока и конца файла?

Ответ: Вызов `rewind` очищает оба индикатора, как и открытие или повторное открытие потока. Вызов `ungetc`, `fseek` или `fsetpos` очищает только индикатор конца файла,

Вопрос: Я не могу заставить `feof` работать; кажется, что она возвращает ноль даже в конце файла. Что я делаю не так?

Ответ: `feof` вернёт ненулевое значение, только если предыдущая операция чтения не удалась; вы не можете использовать `feof` для проверки наличия конца файла перед попыткой чтения. Вместо этого вы должны сначала попытаться прочитать, а затем проверить возвращаемое значение из функции ввода. Если возвращаемое значение указывает, что операция была неудачной, вы можете затем использовать `feof`, чтобы определить, была ли ошибка связана с концом файла. Другими словами, лучше не думать о вызове `feof` как о способе обнаружения конца файла. Вместо этого думайте об этом как о способе подтвердить, что конец файла был причиной сбоя операции чтения.

Вопрос: Я все ещё не понимаю, почему библиотека ввода-вывода предоставляет макросы с именами `putc` и `getc` в дополнение к функциям с именами `fputc` и `fgetc`. Согласно разделу 21.1, уже существует две версии `putc` и `getc` (макрос и функция). Если нам нужна настоящая функция вместо макроса, мы можем демаскировать функцию `putc` или `getc`,

отменив определение макроса. Так почему существуют `fputc` и `fgetc`?

Ответ: Исторические причины. До стандартизации в С не было правила, согласно которому должна существовать настоящая функция для резервирования каждого параметризованного макроса в библиотеке. `putc` и `getc` традиционно реализовывались только как макросы; `fputc` и `fgetc` были реализованы только как функции.

Вопрос: Что не так с сохранением возвращаемого значения `fgetc`, `getc` или `getchar` в переменной типа `char`? Я не понимаю, как тестирование переменной типа `char` на EOF может дать неправильный ответ.

Ответ: Есть два случая, когда этот тест может дать неверный результат. Чтобы сделать нижеследующее обсуждение конкретным, я предполагаю арифметику с дополнением до двух.

Во-первых, предположим, что `char` - беззнаковый тип.

(Напомним, что некоторые компиляторы рассматривают `char` как тип со знаком, но другие рассматривают его как тип без знака.) Теперь предположим, что `getc` возвращает EOF, который мы сохраняем в переменной `char` с именем `ch`. Если EOF представляет -1 (его типичное значение), `ch` будет иметь значение 255. Сравнение `ch` (символ без знака) с EOF (целое число со знаком) требует преобразования `ch` в целое число со

знаком (255, в данном случае). Сравнение с EOF не выполняется, поскольку 255 не равно -1.

Теперь предположим, что вместо этого char является типом со знаком. Рассмотрим, что произойдёт, если getc прочитает байт, содержащий значение 255, из двоичного потока.

Сохранение 255 в переменной ch даёт ей значение -1, поскольку ch - знаковый символ. Проверка того, что ch равно EOF, даст (ошибочно) истинный результат.

Вопрос: Функции ввода символов, описанные в Разделе 22.4, требуют нажатия клавиши Enter, прежде чем они смогут прочитать то, что набрал пользователь. Как я могу написать программу, которая реагирует на отдельные нажатия клавиш?

Ответ: Как вы заметили, функции getc, fgetc и getchar буферизуются; они не начинают читать ввод, пока пользователь не нажмёт клавишу Enter. Чтобы читать символы по мере их ввода - что важно для некоторых типов программ, - вам потребуется нестандартная библиотека, адаптированная к вашей операционной системе. В UNIX, например, библиотека curses часто предоставляет такую возможность.

Вопрос: Когда я читаю пользовательский ввод, как я могу пропустить все символы, оставшиеся в текущей строке ввода?

Ответ: Одна из возможностей состоит в том, чтобы написать небольшую функцию, которая читает и игнорирует все

1300

символы вплоть до первого символа новой строки (и включая его) :

```
void skip_line (void){  
    while (getchar () != '\n')  
        ;  
};
```

Другая возможность - попросить `scanf` пропустить все символы до первого символа новой строки:

```
scanf ("%* [^\n]"); /* пропускает символы до новой строки */
```

`scanf` прочитает все символы до первого символа новой строки, но не сохранит их нигде (* указывает на подавление присваивания). Единственная проблема с использованием `scanf` заключается в том, что он оставляет непрочитанным символ новой строки, поэтому вам, возможно, придётся отбросить его отдельно.

Что бы вы ни делали, не вызывайте функцию `fflush`:

```
fflush (stdin); /* эффект не определён */
```

Хотя некоторые реализации позволяют использовать `fflush` для "очистки" непрочитанного ввода, предполагать, что все так делают, - плохая идея. `fflush` предназначен для очистки выходных потоков; в стандарте C указано, что его влияние на входные потоки не определено.

Вопрос: Почему не рекомендуется использовать `fread` и `fwrite` с текстовыми потоками?

Ответ: Одна из трудностей заключается в том, что в некоторых операционных системах символ новой строки становится парой символов при записи в текстовый файл (подробности см. В Разделе 22.1). Мы должны принять во внимание это расширение, иначе мы потеряем наши данные. Например, если мы используем `fwrite` для записи блоков из 80 символов, некоторые из блоков могут в конечном итоге занять более 80 байтов в файле из-за символов новой строки, которые были расширены.

Вопрос: Почему существует два набора функций позиционирования файлов (`fseek` / `ftell` и `fsetpos` / `fgetpos`)? Разве одного набора недостаточно?

Ответ: `fseek` и `ftell` были частью библиотеки C. Однако у них есть один недостаток: они предполагают, что позиция в файле уместится в `long int`. Поскольку `long int` обычно является 32-битным типом, это означает, что `fseek` и `ftell` могут не работать с файлами, содержащими более 2147483647 байтов.

Признавая эту проблему, при создании C89 в `<stdio.h>` были добавлены `fsetpos` и `fgetpos`. Эти функции не обязаны обрабатывать позиции файлов как числа, поэтому на них не распространяется ограничение `long int`. Но не думайте, что вам нужно использовать `fsetpos` и `fgetpos`; если ваша

реализация поддерживает 64-битный тип `long int`, `fseek` и `ftell` подходят даже для очень больших файлов.

Вопрос: Почему в этой главе не обсуждается управление экраном: перемещение курсора, изменение цвета символов на экране и т. д.?

Ответ: C не предоставляет стандартных функций для управления экраном. Стандарт C решает только те вопросы, которые можно разумно стандартизировать для широкого диапазона компьютеров и операционных систем; управление экраном находится за пределами этой области. Обычным способом решения этой проблемы в UNIX является использование библиотеки `curses`, которая поддерживает управление экраном независимо от терминала.

Точно так же нет стандартных функций для создания программ с графическим пользовательским интерфейсом.

Однако вы, скорее всего, можете использовать вызовы функций C для доступа к оконному API (интерфейсу прикладного программирования) для вашей операционной системы.

Упражнения

К Разделу 22.1

1303

1. Укажите, будет ли каждый из следующих файлов с большей вероятностью содержать текстовые или двоичные данные:

- (a) Файл объектного кода, созданный компилятором C
- (b) Список программ, созданный компилятором C
- (c) Электронное сообщение, отправленное с одного компьютера на другой.
- (d) Файл, содержащий графическое изображение

К Разделу 22.2

W 2. Укажите, какая строка `mode`, скорее всего, будет передана для открытия в каждой из следующих ситуаций:

- (a) Система управления базой данных открывает файл, содержащий записи, которые необходимо обновить.
 - (b) Почтовая программа открывает файл сохраненных сообщений, чтобы в конце можно было добавить дополнительные сообщения.
 - (c) Графическая программа открывает файл, содержащий изображение, которое будет отображаться на экране.
 - (d) Интерпретатор команд операционной системы открывает "сценарий оболочки" (или "пакетный файл"), содержащий команды для выполнения.
3. Найдите ошибку в следующем фрагменте программы и покажите, как её исправить.

1304

```
FILE * fp;  
if (fp = fopen (filename, "r")) {  
    читать символы до конца файла  
}  
fclose (fp);
```

К Разделу 22.3

W 4. Покажите, как будет выглядеть каждое из следующих чисел, если оно будет отображено `printf` с `%#012.5g` в качестве спецификации преобразования:

- (a) 83.7361
- (b) 29748.6607
- (c) 1054932234.0
- (d) 0.0000235218

5. Есть ли разница между спецификациями преобразования `printf` `%.4d` и `%04d`? Если да, объясните, какая.

W* 6. Напишите вызов `printf`, который печатает

1 widget

если переменная `widget` (типа `int`) имеет значение 1 и

n widget

в противном случае, где n - значение `widget`. Вам не разрешается использовать инструкцию `if` или любой другой инструкцию; ответ должен быть единственным вызовом `printf`.

*7. Предположим, что мы вызываем `scanf` следующим образом: `n = scanf ("%d%f%d", &i, &x, &j);`

(`i`, `j` и `n` - переменные типа `int`, а `x` - переменная `float`.)

Предполагая, что входной поток содержит показанные символы, задайте значения `i`, `j`, `n` и `x` после вызова. Кроме того, укажите, какие символы использовались при вызове.

(a) 10●20¢●30☼

(b) 1.0●2.0●3.0☼

(c) 0.1●0.2●0.3☼

(d).1e●.2●.3☼

W 8. В предыдущих главах мы использовали строку формата `scanf " %c"`, когда хотели пропустить символы пробела и прочесть непустой символ. Некоторые программисты вместо этого используют `"%1s"`. Эквивалентны ли эти две техники? Если нет, то в чем разница?

К Разделу 22.4

9. Какой из следующих вызовов не является допустимым способом чтения одного символа из стандартного входного потока?

1306

- (a) getch ()
- (б) getchar ()
- (c) getc (stdin)
- (г) fgetc (stdin)

W 10. У программы fсору.с есть один небольшой недостаток: она не проверяет наличие ошибок при записи в целевой файл. Ошибки во время записи случаются редко, но иногда случаются (например, диск может переполниться). Покажите, как добавить в программу недостающую проверку ошибок, предполагая, что мы хотим, чтобы она отображала сообщение и немедленно завершала свою работу в случае возникновения ошибки.

11. В программе fсору.с появляется следующий цикл:

```
while ((ch = getc (source_fp))!= EOF)
    putc (ch, dest_fp);
```

Предположим, что мы забыли заключить в скобки
ch = getc (source _fp):

```
while (ch = getc (source_fp) != EOF)
    putc (ch, dest_fp);
```

Скомпилируется ли программа без ошибок? Если да, то что будет делать программа при запуске?

1307

12. Найдите ошибку в следующей функции и покажите, как её исправить.

```
int count_periods(const char *filename){
    FILE * fp;
    int n = 0;
    if ((fp = fopen(filename, "r")) != NULL) {
        while (fgetc(fp) != EOF)
            if (fgetc(fp) == '.')
                n++;
        fclose (fp) ;
    }
    return n;
}
```

13. Напишите следующую функцию:

```
int line_length(const char *filename, int n);
```

Функция должна возвращать длину строки ***n*** в текстовом файле с именем `filename` (при условии, что первая строка в файле - это строка 1). Если такой строки нет, функция должна вернуть 0.

К Разделу 22.5

W 14. (a) Напишите свою собственную версию функции `fgets`. Сделайте так, чтобы она вела себя как можно больше как

1308

настоящая функция `fgets`; в частности, убедитесь, что он имеет правильное возвращаемое значение. Чтобы избежать конфликтов со стандартной библиотекой, не называйте функцию `fgets`.

(b) Напишите свою собственную версию `fputs`, следуя тем же правилам, что и в части (a).

К Разделу 22.7

W 15. Напишите вызовы `fseek`, которые выполняют следующие операции позиционирования файлов в двоичном файле, данные которого расположены в 64-байтовых "записях". В каждом случае используйте `fr` в качестве указателя файла.

- a) перейти к началу записи *n*. (Предположим, что первой записью в файле является запись 0.)
- (b) переместитесь в начало последней записи в файле.
- (c) продвинуться вперед одну запись.
- (d) передвинуться назад на две записи.

К Разделу 22.8

16. Предположим, что `str` - это строка, содержащая "рейтинг продаж", которому непосредственно предшествует символ `#`. (другие символы могут предшествовать `#` и/или следовать за

1309

рейтингом продаж). Рейтинг продаж - это ряд десятичных цифр, возможно, содержащих запятые, например, в следующих примерах:

989

24,675

1,162,620

Напишите вызов `sscanf`, который извлекает рейтинг продаж (но не символ `#`) и сохраняет его в строковой переменной `sales_rank`.

Проекты по программированию

1. Расширьте программу `canopen.c` из раздела 22.2 так, чтобы пользователь мог поместить любое количество имён файлов в командную строку:

```
canopen foo bar baz
```

Программа должна выводить для каждого файла отдельное сообщение можно ли его открыть или нельзя. Завершить программу со статусом `EXIT_FAILURE`, если один или несколько файлов не могут быть открыты.

W 2. Напишите программу, которая преобразует все буквы в файле в верхний регистр. (Символы, отличные от букв, не должны изменяться.) Программа должна получить имя файла из командной строки и записать его вывод в `stdout`.

3. Напишите программу с именем `fcats`, которая "объединяет" любое количество файлов, записывая их в стандартный поток вывода, один за другим, без перерывов между файлами.

Например, следующая команда отобразит на экране файлы `f1.c`, `f2.c` и `f3.c`:

`fcats f1.c f2.c f3.c`:

`fcats` должен выдать сообщение об ошибке, если какой-либо файл не может быть открыт. Подсказка: поскольку одновременно открыто не более одного файла, `fcats` требуется только одна переменная-указатель файла. После завершения работы с файлом `fcats` может использовать ту же переменную при открытии следующего файла.

W 4. (a) Напишите программу, подсчитывающую количество символов в текстовом файле.

(b) Напишите программу, которая считает количество слов в текстовом файле. ("Слово" - это любая последовательность символов, отличных от пробелов.)

(c) Напишите программу, которая подсчитывает количество строк в текстовом файле.

Пусть каждая программа получит имя файла из командной строки.

5. Программа `hcr` с раздела 20.1 отказывается шифровать байты, которые - в оригинальной или зашифрованной форме - являются управляющими символами. Теперь мы можем снять это ограничение. Измените программу так, чтобы имена

1311

входных и выходных файлов были аргументами командной строки. Откройте оба файла в двоичном режиме и удалите тест, который проверяет, являются ли исходные и зашифрованные символы печатными символами.

w 6. Напишите программу, которая отображает содержимое файла в виде байтов и символов. Попросите пользователя указать имя файла в командной строке. Вот как будет выглядеть вывод, когда программа будет использоваться для отображения файла run.c раздела 2.1:

Offset	Bytes										Characters
0	23	69	6E	63	6C	75	64	65	20	3C	#include <
10	73	74	64	69	6F	2E	68	3E	0D	0A	stdio.h>..
20	0D	0A	69	6E	74	20	6D	61	69	6E	..int main
30	28	76	6F	69	64	29	0D	0A	7B	0D	(void)..{.
40	0A	20	20	70	72	69	6E	74	66	28	. printf(
50	22	54	6F	20	43	2C	20	6F	72	20	"To C, or
60	6E	6F	74	20	74	6F	20	43	3A	20	not to C:
70	74	68	61	74	20	69	73	20	74	68	that is th
80	65	20	71	75	65	73	74	69	6F	6E	e question
90	2E	5C	6E	22	29	3B	0D	0A	20	20	.\n");..
100	72	65	74	75	72	6E	20	30	3B	0D	return 0;.
110	0A	7D									.}

Каждая строка показывает 10 байт из файла в виде шестнадцатеричных чисел и символов. Число в столбце Offset указывает на положение в файле первого байта строки. Отображаются только печатные символы (определённые

функцией `isprint`); другие символы отображаются в виде точек. Обратите внимание, что внешний вид текстового файла может варьироваться в зависимости от набора символов и операционной системы. В приведённом выше примере предполагается, что `run.c` является файлом Windows, поэтому байты `0D` и `0A` (символы возврата каретки ASCII и перевода строки) появляются в конце каждой строки. Подсказка: обязательно откройте файл в режиме `"rb"`.

7. Из многих методов сжатия содержимого файла один из самых простых и быстрых известен как последовательное кодирование. Этот метод сжимает файл, заменяя последовательности идентичных байтов парой байтов: число повторений, за которым следует байт, подлежащий повторению. Например, предположим, что сжатый файл начинается со следующей последовательности байтов (показанной в шестнадцатеричном формате):

46 6F 6F 20 62 61 72 21 21 21 20 20 20 20 20

сжатый файл будет содержать следующие байты:

01 46 02 6F 01 20 01 62 01 61 01 72 03 21 05 20

Последовательное кодирование длин серий хорошо работает, если исходный файл содержит много длинных последовательностей идентичных байтов. В худшем случае

1313

(файл без повторяющихся байтов) последовательное кодирование может фактически удвоить длину файла.

(a) Напишите программу с именем `compress_file`, которая использует последовательное кодирование для сжатия файла. Чтобы запустить файл `compress_file`, мы использовали бы команду вида

```
compress_file original-file
```

`compress_file` запишет сжатую версию `original-file` в `original-file.c`. Например, команда

```
compress_file foo.txt
```

заставит `compress_file` записать сжатую версию `foo.txt` в файл с именем `foo.txt.rle`. Подсказка: программа, описанная в Проекте по программированию 6, может быть полезна для отладки.

(b) Напишите программу с именем `uncompress_file`, которая отменяет сжатие, выполняемое программой сжатия файла. Команда `uncompress-file` будет иметь вид

```
uncompress file compressed-file
```

сжатый-файл должен иметь расширение. `rle`. Например, команда распаковать файл `foo.txt.rle`

1314

вызовет `uncompress_file` для открытия файла `foo.txt.rle` и записи несжатой версии его содержимого в `foo.txt`. `uncompress_file` должен отображать сообщение об ошибке, если его аргумент командной строки не заканчивается расширением `.rle`.

8. Измените программу `inventory.c` из Раздела 16.3, добавив две новые операции:

Сохранить базу данных в указанном файле.

Загрузить базу данных из указанного файла.

Используйте коды `d` (`dump`) и `r` (`restore`), соответственно, для представления этих операций. Взаимодействие с пользователем должно иметь следующий вид:

Введите код операции: d

Введите имя выходного файла: inventory.dat

Введите код операции: r

Введите имя входного файла: inventory.dat

Подсказка: используйте `fwrite`, чтобы записать массив, содержащий части, в двоичный файл. Используйте `fread` для восстановления массива, прочитав его из файла,

W 9. Напишите программу, которая объединяет два файла, содержащие записи деталей, хранящиеся в программе `inventory.c` (см. Проект по программированию 8).

Предположим, что записи в каждом файле отсортированы по номеру детали, и что мы хотим, чтобы полученный файл также был отсортирован. Если в обоих файлах есть детали с одинаковым номером, объедините количества, хранящиеся в записях. (В качестве проверки согласованности попросите программу сравнить названия деталей и напечатать сообщение об ошибке, если они не совпадают.) Пусть программа получит имена входных файлов и объединённого файла из командной строки.

* 10. Измените программу `inventory2.c` из Раздела 17,5, добавив операции `d` (дамп) и `r` (восстановление), описанные в Проекте по программированию 8. Поскольку структуры `part` не хранятся в массиве, операция `d` не может их сохранить. все с помощью одного вызова `write`. Вместо этого ей нужно будет посетить каждый узел в связанном списке, записав номер детали, название детали и количество в наличии в файл. (Не сохраняйте следующий указатель; он станет недействительным после завершения программы.) При чтении частей из файла операция `r` будет перестраивать список по одному узлу за раз.

11. Напишите программу, которая считывает дату из командной строки и отображает её в следующей форме:

September 13, 2010

Разрешить пользователю вводить дату как 9-13-2010 или 9/13/2010; вы можете предположить, что в дате нет пробелов. Распечатать сообщение об ошибке, если для даты нет ни одной из указанных форм. Подсказка: используйте `sscanf` для извлечения месяца, дня и года из аргумента командной строки.

12. Измените Проект по программированию 2 из главы 3, чтобы программа считывала серию элементов из файла и отображала данные в столбцах. Каждая строка файла будет иметь следующий вид:

item, price, mm/dd/yyyy

Например, предположим, что файл содержит следующие строки:

583,13.5,10/24/2005

3912, 599.99, 07/27/2008

Результат программы должен иметь следующий вид:

item,	Unit Purchase,	Purchase Date
583	\$ 13.5	10/24/2005
3912	\$ 599,99	07/27/2008

1317

Пусть программа получит имя файла из командной строки.

13. Измените Проект по программированию 8 из главы 5, чтобы программа получала время вылета и прибытия из файла с именем `flights.dat`. Каждая строка файла будет содержать время отправления, за которым следует время прибытия, с одним или несколькими пробелами, разделяющими их. Время будет выражено в 24-часовом формате. Например, вот как мог бы выглядеть `flights.dat`, если бы он содержал информацию о рейсе, указанную в исходном проекте:

```
8:00 10:16
9:43 11:52
11:19 13:31
12:47 15:00
14:00 16:08
15:45 17:55
19:00 21:20
21:45 23:58
```

14. Измените Проект по программированию 15 из главы 8 так, чтобы программа предлагала пользователю ввести имя файла, содержащего сообщение, которое нужно зашифровать:

Enter name of file to be encrypted: message.txt

Enter shift amount (1-25): 3

Затем программа записывает зашифрованное сообщение в файл с тем же именем, но с добавленным расширением .enc. В этом примере исходное имя файла - message.txt, поэтому зашифрованное сообщение будет сохранено в файле с именем message.txt.enc. Нет ограничений на размер файла, который нужно зашифровать, или на длину каждой строки в файле.

15. Измените программу justify из Раздела 15.3, чтобы она считывала из одного текстового файла и записывала в другой. Программа должна получить имена обоих файлов из командной строки.

16. Измените программу fcopy.c из Раздела 22.4 так, чтобы она использовала fread и fwrite для копирования файла блоками по 512 байт. (Конечно, последний блок может содержать меньше 512 байт.)

17. Напишите программу, которая считывает серию телефонных номеров из файла и отображает их в стандартном формате. Каждая строка файла будет содержать один номер телефона, но номера могут быть в разных форматах. Вы можете предположить, что каждая строка содержит 10 цифр, возможно, смешанных с другими символами (которые следует игнорировать). Например, предположим, что файл содержит следующие строки:

404.817.6900

1319

(215) 686-1776

312-746-6000

877 275 5273

6173434200

Результат программы должен иметь следующий вид:

(404) 817-6900

(215) 686-1776

(312) 746-6000

(877) 275-5273

(617) 343-4200

Пусть программа получит имя файла из командной строки.

18. Напишите программу, которая считывает целые числа из текстового файла, имя которого указано в качестве аргумента командной строки. Каждая строка файла может содержать любое количество целых чисел (включая ни одного), разделённых одним или несколькими пробелами. Программа должна отображать наибольшее число в файле, наименьшее число и медиану (число, ближайшее к середине, если целые числа были отсортированы). Если файл содержит чётное число целых чисел, в середине будут два числа; программа должна отображать их среднее значение (с округлением в меньшую сторону). Вы можете предположить, что файл содержит не более 10 000 целых чисел. Подсказка: сохраните целые числа в массиве, а затем отсортируйте массив.

19. (a) Напишите программу, преобразующую текстовый файл Windows в текстовый файл UNIX. (См. Раздел 22.1 для обсуждения различий между текстовыми файлами Windows и UNIX.) (b) Напишите программу, которая преобразует текстовый файл UNIX в текстовый файл Windows.

В каждом случае пусть программа получит имена обоих файлов из командной строки. Подсказка: откройте входной файл в режиме "rb" и выходной файл в режиме "wb".

23 Библиотека поддержки чисел и символьных данных

Продолжительный контакт с компьютером превращает математиков в клерков и наоборот.

24 Обработка ошибок

Существует два способа написания безошибочных программ; работает только третий.

Хотя студенческие программы часто терпят неудачу при неожиданном вводе данных, коммерческие программы должны быть "пуленепробиваемыми" - способными изящно восстанавливаться после ошибок, а не терпеть сбои. Чтобы сделать программы пуленепробиваемыми, мы должны предвидеть ошибки, которые могут возникнуть во время выполнения программы, включить проверку для каждой из них и обеспечить подходящее действие для выполнения программы, если ошибка произойдёт.

В этой главе описаны два способа проверки программ на наличие ошибок: с помощью макроса `assert` и путём тестирования переменной `errno`. Раздел 24.1 охватывает заголовок `<assert.h>`, где определяется `assert`. В разделе 24.2 обсуждается заголовок `<errno.h>`, к которому принадлежит переменная `errno`. Этот раздел также включает в себя освещение функций `perror` и `strerror`. Эти функции, исходящие из `<stdio.h>` и `<string.h>` соответственно, тесно связаны с переменной `errno`.

В разделе 24.3 объясняется, как программы могут обнаруживать и обрабатывать условия, известные как сигналы, некоторые из которых представляют собой ошибки. Функции, работающие с сигналами, объявляются в заголовке `<signal.h>`.

Наконец, в разделе 24.4 рассматривается механизм `setjmp/longjmp`, который часто используется для реагирования на ошибки. И `setjmp`, и `longjmp` принадлежат заголовку `<setjmp.h>`.

Обнаружение и обработка ошибок не входят в число сильных сторон Си. Си указывает на ошибки во время выполнения различными способами, а не одним единообразным способом. Кроме того, ответственность за включение кода для проверки на наличие ошибок лежит на программисте. Легко переоценить потенциальные ошибки; если одна из них действительно должна произойти, программа часто продолжает работать, хотя и не очень хорошо. Более новые языки, такие как C++, Java и C#, имеют функцию "обработки исключений", которая облегчает обнаружение ошибок и реагирование на них.

24.1 Заголовок `<assert.h>`: Диагностика

```
void assert (scalar expression);
```

`assert`, который определён в заголовке `<assert.h>`, позволяет программе отслеживать собственное поведение и обнаруживать возможные проблемы на ранней стадии.

Хотя `assert` на самом деле является макросом, он предназначен для использования в качестве функции. У него есть один аргумент, который должен быть "утверждением" - выражением, которое, как мы ожидаем, будет истинным при нормальных обстоятельствах. Каждый раз, когда выполняется `assert`, он проверяет значение своего аргумента. Если аргумент имеет ненулевое значение, `assert` ничего не делает. Если значение аргумента равно нулю, `assert` записывает сообщение в `stderr` (стандартный поток ошибок) и вызывает функцию `abort` для прекращения выполнения программы.

Например, предположим, что файл `demo.c` объявляет массив ***a*** длиной 10. Нас беспокоит, что инструкция `a[i] = 0;` в `demo.c` может привести к сбою программы, потому что ***i*** не находится между 0 и 9. Мы можем использовать `assert` для проверки этого условия перед выполнением присваивания `a[i]`:

```
assert (0 <= i && i <10); /* сначала проверяет индекс */
```

```
a[i] = 0; // теперь выполняет присваивание
```

Если значение ***i*** меньше 0 или больше или равно 10, программа завершится после отображения сообщения, подобного следующему:

Ошибочное утверждения: `0 <= i & i <10`, файл `demo.c`, строка 109

C99 изменяет `assert` несколькими незначительными способами. Стандарт C89 гласит, что аргумент `assert` должен иметь тип `int`. Стандарт C99 ослабляет это требование, позволяя аргументу иметь любой скалярный тип (отсюда и слово `scalar` в прототипе для `assert`). Это изменение позволяет, например, использовать в качестве аргумента число с плавающей запятой или указатель. Кроме того, C99 требует, чтобы неудачный `assert` отображал имя функции, в которой он появляется. (C89 требует только, чтобы `assert` отображал аргумент - в текстовой форме - вместе с именем исходного файла и номером исходной строки). Предлагаемая форма сообщения :

`Assertion failed: expression, function abc, file xyz, line nnn.`

(Ошибка утверждения: выражение, функция `abc`, файл `xyz`, строка `nnn`.)

Точная форма сообщения, создаваемого `assert`, может варьироваться от одного компилятора к другому, хотя всегда должна содержать информацию, требуемую стандартом. Например, компилятор GCC выдает следующее сообщение в ситуации, описанной ранее:

`a.out: demo.c:109: main: Assertion 0 <= i && i < 10 failed.`

У `assert` есть один недостаток: он немного увеличивает время работы программы из-за дополнительной проверки, которую она выполняет. Время от времени использование `assert`, вероятно, не окажет большого влияния на скорость работы

программы, но даже это небольшое сокращение времени может быть неприемлемо для критически важных приложений. В результате многие программисты используют `assert` во время тестирования, а затем отключают его по завершении программы. Отключить `assert` просто: нам нужно только определить макрос `NDEBUG` перед включением заголовка `<assert.h>`:

```
#define NDEBUG  
#include <assert.h>
```

Значение `NDEBUG` не имеет значения, просто тот факт, что он определён. Если позже программа потерпит неудачу, мы можем повторно активировать `assert`, удалив определение `NDEBUG`.

Старайтесь не помещать выражение, имеющее побочный эффект, включая вызов функции, внутри `assert`; если позже `assert` будет отключён, выражение не будет вычисляться.

Рассмотрим следующий пример:

```
assert ((p = malloc(n)) != NULL);
```

Если `NDEBUG` определён, `assert` будет проигнорирован, а `malloc` не будет вызываться.

24.2 Заголовок `<errno.h>`: ошибки

Некоторые функции в стандартной библиотеке указывают на сбой, сохраняя код ошибки (положительное целое число) в

errno, переменной типа int, объявленной в <errno.h>. (errno на самом деле может быть макросом. Если это так, стандарт C требует, чтобы он представлял lvalue, что позволяет нам использовать его как переменную.) Большинство функций, которые полагаются на errno, принадлежат <math.h>, но есть несколько в других частях библиотеки.

Допустим, нам нужно использовать библиотечную функцию, которая сигнализирует об ошибке, сохраняя значение в errno. После вызова функции мы можем проверить, не равно ли значение errno нулю; если не равно, то при вызове функции произошла ошибка. Например, предположим, что мы хотим проверить, не завершился ли вызов функции sqrt (квадратный корень). Вот как будет выглядеть код:

```
errno = 0;
```

```
y = sqrt (x);
```

```
if (errno != 0) {  
    fprintf(stderr, "sqrt error; program terminated.\n");  
    exit (EXIT_FAILURE) ;  
}
```

Когда errno используется для обнаружения ошибки при вызове библиотечной функции, важно сохранить ноль в errno перед вызовом функции. Хотя в начале выполнения

программы значение `errno` равно нулю, оно могло быть изменено более поздним вызовом функции. Библиотечные функции никогда не сбрасывают `errno`; это ответственность программы.

Значение, сохраняемое в `errno` при возникновении ошибки, часто бывает либо `EDOM`, либо `ERANGE`. (Оба являются макросами, определёнными в `<errno.h>`.) Эти макросы представляют два типа ошибок, которые могут возникнуть при вызове математической функции:

- Ошибки домена (`EDOM`): аргумент, переданный функции, находится за пределами домена функции. Например, передача отрицательного числа в `sqrt` вызывает ошибку домена.
- Ошибки диапазона (`ERANGE`): возвращаемое значение функции слишком велико, чтобы его можно было представить в типе возвращаемого значения функции. Например, передача 1000 в функцию `exp` обычно вызывает ошибку диапазона, потому что e^{1000} слишком велико, чтобы представлять его как `double` на большинстве компьютеров.

Некоторые функции могут иметь оба вида ошибок; сравнивая `errno` с `EDOM` или `ERANGE`, мы можем определить, какая ошибка произошла.

C99 добавляет макрос `EILSEQ` в `<errno.h>`. Библиотечные функции в определённых заголовках, особенно заголовке

<wchar.h>, сохраняют значение EILSEQ в errno при возникновении ошибки кодирования.

Функции perror и strerror

```
void perror (const char * s); //из <stdio.h>  
char * strerror (int errnum); //из <string.h>
```

Теперь мы рассмотрим две функции, которые связаны с переменной errno, хотя ни одна из них не принадлежит <errno.h>.

Когда библиотечная функция сохраняет ненулевое значение в errno, нам может потребоваться отобразить сообщение, указывающее на природу ошибки. Один из способов сделать это - вызвать функцию perror (объявленную в <stdio.h>), которая печатает следующие элементы в указанном порядке: (1) её аргумент, (2) двоеточие, (3) пробел, (4) сообщение об ошибке, определяемое значением errno, и (5) символ новой строки. perror записывает в поток stderr, а не в стандартный вывод.

Вот как мы можем использовать perror:

```
errno = 0;  
y = sqrt (x);
```

1329

```
if (errno != 0) {  
    perror ("sqrt error");  
    exit (EXIT_FAILURE) ;  
}
```

Если вызов `sqrt` завершится неудачно из-за ошибки домена, `perror` выдаст следующий результат:
`sqrt error: Numerical argument out of domain`
(Числовой аргумент вне домена)

Сообщение об ошибке, которое отображается после ошибки `sqrt`, определяется реализацией. В этом примере числовой аргумент вне домена - это сообщение, соответствующее ошибке `EDOM`. Ошибка `ERANGE` обычно приводит к другому сообщению, например, `Numerical result out of range` ("Числовой результат вне допустимого диапазона").

Функция `strerror` принадлежит `<string.h>`. При передаче кода ошибки `strerror` возвращает указатель на строку, описывающую ошибку. Например, вызов `puts (strerror (EDOM) ) ;` может распечатать
`Numerical argument out of domain`

Аргумент `strerror` обычно является одним из значений `errno`, но `strerror` вернёт строку для любого переданного ему целого числа. `strerror` тесно связан с функцией `perror`. Сообщение об

ошибке, которое отображает `errno`, - это то же сообщение, которое вернёт `strerror`, если в качестве аргумента ему будет передано `errno`.

24.3 Заголовок `<signal.h>`: обработка сигналов

Заголовок `<signal.h>` предоставляет средства для обработки исключительных условий, известных как сигналы. Сигналы делятся на две категории: ошибки времени выполнения (например, деление на ноль) и события, вызванные вне программы. Например, многие операционные системы позволяют пользователям прерывать или прекращать работу запущенных программ; эти события обрабатываются как сигналы в С. Когда происходит ошибка или внешнее событие, мы говорим, что был подан сигнал. Многие сигналы асинхронны: они могут возникать в любой момент выполнения программы, а не только в определённых точках, которые известны программисту. Поскольку сигналы могут возникать в неожиданное время, с ними нужно обращаться уникальным образом. В этом разделе рассматриваются сигналы, описанные в стандарте Си. Сигналы играют более заметную роль в UNIX, чем можно было бы ожидать от их ограниченного освещения здесь. Для получения информации

о сигналах UNIX обратитесь к одной из книг по программированию UNIX, перечисленных в библиографии.

Сигнальные макросы

<signal. h > определяет ряд макросов, представляющих сигналы; в таблице 24.1 перечислены эти макросы и их значения. Значение каждого макроса является положительной целочисленной константой. Реализации C могут предоставлять другие сигнальные макросы, если их имена начинаются с STG, за которым следует заглавная буква. ((Реализации UNIX, в частности, предоставляют большое количество дополнительных сигнальных макросов.)

Стандарт C не требует, чтобы сигналы в таблице 24.1 поднимались автоматически, поскольку не все из них могут быть значимыми для конкретного компьютера и операционной системы. Большинство реализаций поддерживают по крайней мере некоторые из этих сигналов.

Имя	Что означает
SIGABRT	Аномальное завершение работы (возможно, вызванное вызовом abort)
SIGFPE	Ошибка при арифметической операции (возможно деление на ноль или переполнение)
SIGILL	Неверная инструкция

1332

SIGINT	Прерывание
SIGSEGV	Недопустимый доступ к хранилищу
SIGTERM	Запрос на завершение

Функция **signal**

```
void (*signal(int sig, void (*func) (int))) (int);
```

<signal.h> предоставляет две функции: `raise` и `signal`. Мы начнём с `signal`, которая устанавливает функцию обработки сигнала для использования позже, если данный сигнал должен появиться. `signal` намного проще в использовании, чем можно было ожидать от его довольно устрашающего прототипа. Его первый аргумент - это код конкретного сигнала; второй аргумент - это указатель на функцию, которая будет обрабатывать сигнал, если он будет вызван позже в программе. Например, следующий вызов `signal` устанавливает обработчик для сигнала `SIGINT`:

```
signal(SIGINT, handler);
```

`handler` - это имя функции обработки сигналов. Если сигнал `SIGINT` появится позже во время выполнения программы, обработчик будет вызван автоматически.

Каждая функция обработки сигналов должна иметь параметр `int` и возвращаемый тип `void`. Когда возникает конкретный сигнал и вызывается его обработчик, обработчику будет передан код для сигнала. Знание того, какой сигнал вызвал его вызов, может быть полезно для обработчика сигналов; в частности, это позволяет нам использовать один и тот же обработчик для нескольких разных сигналов.

Функция обработки сигналов может делать множество вещей. Возможны следующие варианты: игнорирование сигнала, выполнение некоторого рода исправления ошибок или завершение программы. Однако, если она не вызывается `abort` или `raise`, обработчик сигнала не должен вызывать библиотечную функцию или пытаться использовать переменную со статической продолжительностью хранения. (Однако из этих правил есть несколько исключений.)

Если функция обработки сигналов возвращается, программа возобновляет выполнение с точки, в которой возник сигнал, за исключением двух случаев: (1) Если сигнал был `SIGABRT`, программа завершится (ненормально), когда обработчик вернётся. (2) Эффект возврата из функции, обработавшей `SIGFPE`, не определён. (Другими словами, не делайте этого.)

Хотя `signal` имеет возвращаемое значение, от него часто отказываются. Возвращаемое значение, указатель на предыдущий обработчик указанного сигнала, при желании может быть сохранено в переменной. В частности, если мы

1334

планируем позже восстановить исходный обработчик сигнала, нам нужно сохранить возвращаемое значение сигнала:

```
void (*orig_handler) (int) ;/* переменная указатель функции */  
...  
orig_handler = signal(SIGINT, handler) ;
```

Эта инструкция устанавливает обработчик в качестве обработчика для SIGINT, а затем сохраняет указатель на исходный обработчик в переменной обработчика orig_handler. Чтобы позже восстановить исходный обработчик, мы напишем

```
signal (SIGINT, orig_handler); /* восстанавливает  
                               оригинальный обработчик */
```

Предопределённые обработчики сигналов

Предопределённые обработчики сигналов

Вместо написания собственных обработчиков сигналов у нас есть возможность использовать один из предопределённых обработчиков, которые предоставляет <signal.h>. Их два, каждый из которых представлен макросом:

- SIG_DFL. SIG_DFL обрабатывает сигналы "по умолчанию". Чтобы установить SIG_DFL, мы использовали бы такой вызов, как
`signal (SIGINT, SIG_DFL); /* использовать обработчик по умолчанию */`
Эффект от вызова SIG_DFL определяется реализацией, но в большинстве случаев он вызывает завершение программы.
- SIG_IGN. Вызов `signal (SIGINT, SIG_IGN); /* игнорировать сигнал SIGINT */` указывает, что SIGINT должен быть проигнорирован, если он позже будет получен.

Помимо SIG_DFL и SIG_IGN, заголовок `<signal.h>` может предоставлять другие обработчики сигналов; их имена должны начинаться с `STG_`, за которым следует заглавная буква. В начале выполнения программы обработчик каждого сигнала инициализируется значением SIG_DFL или SIG_IGN, в зависимости от реализации.

`<signal.h>` определяет другой макрос, `SIG_ERR`, который, похоже, должен быть обработчиком сигнала. `STG_ERR` фактически используется для проверки ошибки при установке обработчика сигнала. Если вызов сигнала завершился неудачно - он не может установить обработчик для указанного сигнала - он возвращает `SIG_ERR` и сохраняет положительное

1336

значение в errno. Таким образом, чтобы проверить, не прошёл ли сигнал, мы могли бы написать

```
if (signal(SIGINT, handler) == SIG_ERR) {  
    perror ("signal (SIGINT, handler) failed");  
    ...  
}
```

У всего механизма обработки сигналов есть один сложный аспект: что произойдёт, если сигнал будет инициирован функцией, которая его обрабатывает? Чтобы предотвратить бесконечную рекурсию, стандарт C89 предписывает двухэтапный процесс, когда возникает сигнал, для которого программистом установлена функция обработки сигналов. Во-первых, либо обработчик этого сигнала сбрасывается на SIG_DFL (обработчик по умолчанию), либо сигнал блокируется от появления во время выполнения обработчика. (STGILL - особый случай; при вызове SIGILL никаких действий не требуется.) Только после этого вызывается обработчик, предоставленный программистом.

После обработки сигнала необходимость переустановки обработчика определяется реализацией. Реализации UNIX обычно оставляют обработчик сигналов установленным после того, как он был использован, но другие реализации могут сбрасывать обработчик на SIG_DFL. В последнем случае обработчик может переустановить себя, вызвав signal перед возвратом.

C99 меняет процесс обработки сигналов несколькими незначительными способами. При возникновении сигнала реализация может отключить не только этот сигнал, но и другие. Если функция обработки сигнала возвращается из обработки сигнала SIGILL или SIGSEGV (а также сигнала SIGFPE), эффект не определён. C99 также добавляет ограничение, что если сигнал возникает в результате вызова функции abort или raise, сам обработчик сигнала не должен вызывать raise.

Функция raise

```
int raise (int sig);
```

Хотя сигналы обычно возникают из-за ошибок времени выполнения или внешних событий, иногда бывает удобно, если программа вызовет появление сигнала. Именно это и делает функция raise. Аргумент raise определяет код желаемого сигнала:

```
raise (SIGABRT); /* вызывает сигнал SIGABRT */
```

Возвращаемое значение raise может использоваться для проверки того, был ли вызов успешным: ноль указывает на успех, а ненулевое значение указывает на неудачу.

Тестовые сигналы

Следующая программа иллюстрирует использование сигналов. Сначала она устанавливает специальный обработчик для сигнала SIGINT (тщательно сохраняя исходный обработчик), затем вызывает `raise_sig`, чтобы поднять этот сигнал. Затем она устанавливает SIG_IGN в качестве обработчика сигнала SIGINT и снова вызывает `raise_sig`. Наконец, она переустанавливает исходный обработчик для SIGINT, а затем в последний раз вызывает `raise_sig`.

```
// tsignal.c
```

```
/* Tests signals */
```

```
#include <signal.h>
```

```
#include <stdio.h>
```

```
void handler(int sig);
```

```
void raise_sig(void);
```

```
int main(void){
```

```
    void (*orig_handler)(int);
```

```
    printf("Installing handler for signal %d\n", SIGINT);
```

```
    orig_handler = signal(SIGINT, handler);
```

```
    raise_sig();
```

```
printf("Changing handler to SIG_IGN\n");  
signal(SIGINT, SIG_IGN);  
raise_sig();
```

```
printf("Restoring original handler\n");  
signal(SIGINT, orig_handler);  
raise_sig();
```

```
printf("Program terminates normally\n");  
return 0;
```

```
}
```

```
void handler(int sig){  
    printf("Handler called for signal %d\n", sig);  
}
```

```
void raise_sig(void){  
    raise(SIGINT);  
}
```

Кстати, вызов `raise` не обязательно должен быть отдельной функцией. Я определил `raise_sig` просто, чтобы подчеркнуть: независимо от того, где возникает сигнал - будь то в `main` или в какой-либо другой функции, - он будет перехвачен последним установленным обработчиком этого сигнала.

Результат этой программы может несколько отличаться. Вот одна возможность:

Установка обработчика для сигнала 2

Обработчик вызван для сигнала 2

Изменение обработчика на SIG_IGN

Восстановление исходного обработчика

Из этого вывода мы видим, что наша реализация определяет SIGINT равным 2 и что исходным обработчиком для SIGINT должен быть SIG_DFL. (Если бы это был SIG_IGN, мы бы также увидели сообщение "Программа завершается нормально".) Наконец, мы замечаем, что SIG_DFL привел к завершению программы без отображения сообщения об ошибке.

24.4 Заголовок `<setjmp.h>`: нелокальные переходы

```
int setjmp (jmp_buf env);  
void longjmp (jmp_buf env, int val);
```

Обычно функция возвращается в точку, из которой она была вызвана. Мы не можем использовать инструкцию `goto`, чтобы заставить её перейти в другое место, потому что `goto` может

перейти только к метке в той же функции. Заголовок `<setjmp.h>`, однако, позволяет одной функции переходить непосредственно к другой функции без возврата.

Наиболее важными элементами в `<setjmp.h>` являются макрос `setjmp` и функция `longjmp`. `setjmp` "отмечает" место в программе; Затем `longjmp` можно использовать, чтобы вернуться в это место позже. Хотя этот мощный механизм имеет множество потенциальных приложений, он используется в основном для обработки ошибок.

Чтобы отметить цель будущего прыжка, мы вызываем `setjmp`, передавая ему переменную типа `jmp_buf` (объявленную в `<setjmp.h>`). `setjmp` сохраняет текущее "окружение" (включая указатель на расположение самого `setjmp`) в переменной для последующего использования при вызове `longjmp`; затем он возвращает ноль.

Чтобы вернуться к точке `setjmp`, нужно вызвать `longjmp`, передав ей ту же переменную `jmp_buf`, которую мы передали в `setjmp`. После восстановления среды, представленной переменной `jmp_buf`, `longjmp` вернётся - вот где это может быть сложно - из вызова `setjmp`. На этот раз возвращаемое значение `setjmp` - `val`, второй аргумент `longjmp`. (Если `val` равно 0, `setjmp` возвращает 1.)

Убедитесь, что аргумент `longjmp` был ранее инициализирован вызовом `setjmp`. Также важно, чтобы функция, содержащая исходный вызов `setjmp`, не должна была возвращаться до

1342

вызова `longjmp`. Если какое-либо ограничение нарушается, вызов `longjmp` приводит к неопределённому поведению.

(Программа, вероятно, выйдет из строя.)

Подводя итог, `setjmp` возвращает ноль при первом вызове; позже `longjmp` передает управление обратно исходному вызову `setjmp`, который на этот раз возвращает ненулевое значение. Понятно? Возможно, нам понадобится пример ...

ПРОГРАММА Тестирование `setjmp` / `longjmp`

Следующая программа использует `setjmp` для отметки места в `main`; функция `f2` позже возвращается на это место, вызывая `longjmp`.

```
// tsetjmp.c
```

```
/* Tests setjmp/longjmp */
```

```
#include <setjmp.h>
```

```
#include <stdio.h>
```

```
jmp_buf env;
```

```
void f1(void);
```

```
void f2(void);
```

```
int main(void){
```

1343

```
    if (setjmp(env) == 0)
        printf("setjmp returned 0\n");
    else {
        printf("Program terminates: longjmp called\n");
        return 0;
    }
    f1();
    printf("Program terminates normally\n");
    return 0;
}
```

```
void f1(void){
    printf("f1 begins\n");
    f2();
    printf("f1 returns\n");
}
```

```
void f2(void){
    printf("f2 begins\n");
    longjmp(env, 1);
    printf("f2 returns\n");
}
```

Результатом этой программы будет:

setjmp returned 0

f1 begins

1344

f2 begins

Program terminates: longjmp called

Исходный вызов `setjmp` возвращает 0, поэтому `main` вызывает `f1`. Затем `f1` вызывает `f2`, которая использует `longjmp` для передачи управления обратно в `main` вместо возврата к `f1`. Когда выполняется `longjmp`, управление возвращается к вызову `setjmp`. На этот раз `setjmp` возвращает 1 (значение, указанное в вызове `longjmp`).

Вопросы и ответы

Вопрос: Вы сказали, что важно сохранить ноль в `errno` перед вызовом библиотечной функции, которая может его изменить, но я видел программы UNIX, которые проверяют `errno`, даже не устанавливая его в ноль. В чем дело?

Ответ: Программы UNIX часто содержат вызовы функций, принадлежащих операционной системе. Эти системные вызовы полагаются на `errno`, но они используют его немного иначе, чем описано в этой главе. Когда такой вызов терпит неудачу, он возвращает специальное значение (например, 1 или нулевой указатель) в дополнение к сохранению значения в `errno`. Программам не нужно сохранять ноль в `errno` перед таким вызовом, потому что возвращаемое значение функции указывает на то, что произошла ошибка. Некоторые функции

в стандартной библиотеке C работают таким же образом, используя `errno` не столько для того, чтобы сигнализировать об ошибке, сколько для указания, какая это была ошибка.

Вопрос: Моя версия `<errno.h>` определяет другие макросы, кроме `EDOM` и `ERANGE`. Законна ли такая практика?

Ответ: Да. Стандарт C допускает макросы, которые представляют другие состояния ошибки, при условии, что их имена начинаются с буквы `E`, за которой следует цифра или заглавная буква. Реализации UNIX обычно определяют огромное количество таких макросов.

Вопрос: Некоторые макросы, представляющие сигналы, имеют загадочные имена, например `SIGFPE` и `SIGSEGV`. Откуда эти имена?

Ответ: Названия этих сигналов восходят к ранним компиляторам C, работавшим на DEC PDP-11. Аппаратное обеспечение PDP-11 могло обнаруживать ошибки с такими названиями, как: "Floating Point Exception" ("Исключение с плавающей точкой") и "Segmentation Violation" (:Нарушение сегментации:).

Вопрос: Ладно, мне любопытно. Если он не вызывается `abort` или `raise`, обработчик сигналов не должен вызывать стандартную библиотечную функцию, но вы сказали, что есть исключения из этого правила. Каковы они?

Ответ: Обработчику сигнала разрешено вызывать функцию `signal` при условии, что первый аргумент - это сигнал, который он обрабатывает в данный момент. Эта оговорка важна, потому что она позволяет обработчику сигналов переустановить себя. В C99 обработчик сигнала также может вызывать функцию `abort` или функцию `_Exit`.

Вопрос: Продолжая предыдущий вопрос, обработчик сигналов обычно не предназначен для доступа к переменным со статической продолжительностью хранения. Что является исключением из этого правила?

Ответ: Это немного сложнее. Ответ включает тип с именем `sig_atomic_t`, объявленный в заголовке `<signal.h>`. `sig_atomic_t` - это целочисленный тип, к которому можно получить доступ "как к атомарному объекту" в соответствии со стандартом C. Другими словами, ЦП может извлечь значение `sig_atomic_t` из памяти или сохранить его в памяти с помощью одной машинной инструкции, вместо того, чтобы использовать две или более машинных инструкций. `sig_atomic_t` часто определяется как `int`, поскольку большинство процессоров могут загружать или сохранять значение `int` за одну инструкцию.

Это приводит нас к исключению из правила, согласно которому функция обработки сигналов не должна иметь доступ к статическим переменным. Стандарт C позволяет обработчику сигналов сохранять значение в переменной

`sig_atomic_t` - даже со статической продолжительностью хранения - при условии, что она объявлена `volatile`. Чтобы понять причину этого загадочного правила, подумайте, что могло бы произойти, если бы обработчик сигнала изменил статическую переменную, имеющую тип, более широкий, чем `sig_atomic_t`. Если программа извлекла часть переменной из памяти непосредственно перед появлением сигнала, а затем завершила выборку после того, как сигнал был обработан, она могла бы получить значение мусора. Переменные `sig_atomic_t` можно получить за один шаг, поэтому этой проблемы не возникает. Объявление переменной `volatile` предупреждает компилятор о том, что значение переменной может измениться в любой момент. (Внезапно может возникнуть сигнал, вызывающий обработчик сигнала, который изменяет переменную.)

Вопрос: Программа `tsignal.c` вызывает `printf` из обработчика сигналов. Разве это не незаконно?

Ответ: Функция обработки сигнала, вызванная в результате `raise` или `abort`, может вызывать библиотечные функции. `tsignal.c` использует `raise` для вызова обработчика сигнала.

Вопрос: Почему `setjmp` изменяет переданный ему аргумент? Я думал, что C всегда передает аргументы по значению.

Ответ: Стандарт C говорит, что `jmp_buf` должен быть типом массива, поэтому `setjmp` фактически передаёт указатель.

Вопрос: У меня проблемы с `setjmp`. Есть ли ограничения на то, как его можно использовать?

Ответ: Согласно стандарту C, есть только два допустимых способа использовать `setjmp`:

- Как выражение в выражении-инструкции (возможно, приведённое к `void`).
- Как часть управляющего выражения в инструкциях `if`, `switch`, `while`, `do` или `for`. Все управляющее выражение должно иметь одну из следующих форм, где `constexpr` - это целочисленное постоянное выражение, а `op` - оператор отношения или равенства:

```
set jmp (...)  
!setjmp (...)  
constexpr op setjmp (...)  
setjmp (...) op constexpr
```

Использование `setjmp` любым другим способом вызывает неопределённое поведение.

Вопрос: Каковы значения переменных в программе после того, как программа выполнила вызов `longjmp`?

Ответ: Большинство переменных сохраняют значения, которые были у них во время `longjmp`. Однако автоматическая переменная внутри функции, которая содержит `setjmp`, имеет

1349

неопределённое значение, если только она не была объявлена `volatile` или не была изменена с момента выполнения `setjmp`.

Вопрос: Допустимо ли вызывать `longjmp` внутри обработчика сигналов?

Ответ: Да, при условии, что обработчик сигнала не был вызван из-за сигнала, поднятого во время выполнения обработчика сигнала. (C99 снимает это ограничение.)

Упражнения

К разделу 24.1

1. (а) `assert`-трование можно использовать для проверки двух типов проблем: (1) проблемы, которые никогда не должны возникать, если программа правильная, и (2) проблемы, которые не зависят от программы. Объясните, почему `assert` лучше всего подходит для задач первой категории.

(б) Приведите три примера проблем, не зависящих от программы.

2. Напишите вызов `assert`, который заставляет программу завершаться, если переменная с именем `top` имеет значение `NULL`.

1350

3.Измените файл `stackADT2.c` из Раздела 19.4, чтобы он использовал `assert` для проверки ошибок вместо использования инструкции `if`. (Обратите внимание, что функция `terminate` больше не нужна и её можно удалить.)

К разделу 24.2

W 4. (a) Напишите функцию- "оболочку" с именем `try_math_fcn`, которая вызывает математическую функцию (предполагается, что она имеет аргумент `double` и возвращает значение типа `double`), а затем проверяет, был ли вызов успешным. Вот как мы можем использовать `try_math_fcn`:

```
y = try_math_fcn (sqrt, x, «Ошибка при вызове sqrt»);
```

Если вызов `sqrt (2)` успешен, `try_math_fcn` возвращает значение, вычисленное `sqrt`. Если вызов завершился неудачно, `try_math_fcn` вызывает `error` для вывода сообщения «Ошибка при вызове sqrt», а затем вызывает `exit` для завершения программы.

(b) Напишите макрос, который имеет тот же эффект, что и `try_math_fcn`, но строит сообщение об ошибке из имени функции:

```
y = TRY_MATH_FCN (sqrt, x);
```

1351

Если вызов `sqrt` завершится неудачно, появится сообщение "Ошибка при вызове `sqrt`". Подсказка: пусть `TRY_MATH_FCN` вызывает `tr_math_fcn`.

К разделу 24.4

W 5. В программе `inventory.c` (см. Раздел 16.3) функция `main` имеет цикл `for`, который предлагает пользователю ввести код операции, считывает код, а затем вызывает операции вставки, поиска, обновления или печати. Добавьте вызов `setjmp` в `main` таким образом, чтобы последующий вызов `longjmp` возвращался в цикл `for`. (После `longjmp` пользователю будет предложено ввести код операции, и программа продолжит работу в обычном режиме.) `setjmp` потребуются переменная `jmp_buf`; где это должно быть объявлено?

25 Интернациональные ВОЗМОЖНОСТИ

*Если ваш компьютер говорит по-английски, он,
вероятно, был сделан в Японии.*

26 Разные библиотечные функции

Параметризацию процедур должен выполнять пользователь, а не их создатели.

<stdarg.h>, <stdlib.h> и <time.h> - единственные заголовки C89, которые не были рассмотрены в предыдущих главах - не похожи ни на какие другие в стандартной библиотеке.

Заголовок <stdarg.h> (раздел 26.1) позволяет писать функции с переменным числом аргументов. <stdlib.h> (раздел 26.2) - это набор функций, которые не помещаются ни в один из других заголовков. Заголовок <time.h> (раздел 26.3) позволяет программам работать с датой и временем.

26.1 Заголовок <stdarg.h>: переменное число аргументов

```
type va_arg(va_list ap, type);
```

```
void va_copy(va_list dest, va_list src);
```

1353

```
void va_end(va_list ap);
```

```
void va_start (va_list ap, parmN);
```

Такие функции, как `printf` и `scanf`, обладают необычным свойством: они допускают любое количество аргументов. Как оказалось, способность обрабатывать переменное количество аргументов не ограничивается библиотечными функциями. Заголовок `<stdarg.h>` предоставляет инструменты, которые нам понадобятся для написания наших собственных функций с переменной длины списками аргументов. `<stdarg.h>` объявляет один тип (`va_list`) и определяет несколько макросов. В C89 есть три макроса с именами `va_start`, `va_arg` и `va_end`, которые можно рассматривать как функции с прототипами, показанными выше. C99 добавляет похожий на функцию макрос с именем `va_copy`.

Чтобы увидеть, как работают эти макросы, мы воспользуемся ими для написания функции с именем `max_int`, которая находит максимальное из любого числа целочисленных аргументов. Вот как мы можем вызвать функцию:

```
max_int (3, 10, 30, 20)
```

Первый аргумент указывает, сколько дополнительных аргументов последует. Этот вызов `max_int` вернёт 30

1354

(наибольшее из чисел 10, 30 и 20). Вот определение функции `max_int`:

```
int max_int(int n,...){ /* n должно быть не менее 1 */

    va_list ap;
    int i, current, largest;
    va_start(ap, n);
    largest = va_arg(ap, int);

    for (i = 1; i < n; i++) {
        current = va_arg(ap, int);
        if (current > largest)
            largest = current;
    }
    va_end (ap) ;
    return largest;
}
```

Символ... в списке параметров (известный как многоточие) указывает, что за параметром ***n*** следует переменное количество дополнительных параметров.

Тело `max_int` начинается с объявления переменной типа `va_list`:

1355

`va_list ap;`

Объявление такой переменной обязательно для `max_int`, чтобы иметь доступ к аргументам, следующим за ***n***.

Инструкция

`va_start (ap, n);`

указывает, где начинается часть переменной длины в списке аргументов (в данном случае после ***n***). Функция с переменным количеством аргументов должна иметь хотя бы один "нормальный" параметр; многоточие всегда идёт в конце списка параметров после последнего нормального параметра.

Инструкция

`largest = va_arg (ap, int);`

выбирает второй аргумент `max_int` (тот, который стоит после ***n***), присваивает ему наибольшее значение и автоматически переходит к следующему аргументу. Слово `int` указывает, что мы ожидаем, что второй аргумент `max_int` будет иметь тип `int`.

Инструкция

`current = va_arg (ap, int);`

выбирает оставшиеся аргументы `max_int`, один за другим, по мере выполнения внутри цикла.

Не забывайте, что `va_arg` всегда переходит к следующему аргументу после получения текущего. Из-за этого свойства мы не могли бы написать цикл `max_int` следующим образом:

```
for (i = 1; i < n; i ++)  
    if (va_arg (ap, int) > largest) /*** НЕПРАВИЛЬНО ***/  
        largest = va_arg (ap, int);
```

Инструкция

va_end (ap);

требуется для "очистки" перед возвратом функции. (Или вместо возврата функция может вызвать va_start и снова просмотреть список аргументов.)

(Перед тем как выйти из функции, необходимо вызвать функцию va_end(), что гарантирует правильное восстановление стека. Если функция va_end() не вызвана, то возникает аварийная ситуация.)

Макрос va_copy копирует src (значение va_list) в dest (также va_list). Полезность va_copy заключается в том, что несколько вызовов va_arg могут быть сделаны с использованием src до того, как он будет скопирован в dest, таким образом обрабатывая некоторые аргументы. Вызов va_copy позволяет функции запомнить, где она находится в списке аргументов, чтобы позже она могла вернуться к той же точке, чтобы повторно проверить аргумент (и, возможно, аргументы, следующие за ним).

Каждый вызов va_start или va_copy должен сочетаться с вызовом va_end, и эти вызовы должны появляться в одной и

той же функции. Все вызовы `va_arg` должны появляться между вызовом `va_start` (или `va_copy`) и соответствующим вызовом `va_end`.

Когда вызывается функция со списком переменных аргументов, компилятор выполняет продвижение аргументов по умолчанию для всех аргументов, которые соответствуют многоточию. В частности, аргументы `char` и `short` повышаются до `int`, а значения `float` повышаются до `double`. Следовательно, нет смысла передавать в `va_arg` такие типы, как `char`, `short` или `float`, поскольку аргументы - после продвижения - никогда не будут иметь один из этих типов.

Вызов функции со списком переменного числа аргументов

Вызов функции со списком переменного числа аргументов является рискованным по своей сути. ещё в главе 3 мы видели, насколько опасно передавать неверные аргументы в `printf` и `scanf`. Другие функции со списками переменных аргументов также чувствительны. Основная трудность состоит в том, что функция со списком переменных аргументов не имеет возможности определить количество аргументов или их типы. Эта информация должна быть передана в функцию и/или принята функцией. `max_int`

полагается на первый аргумент, чтобы указать, сколько дополнительных аргументов следует за ним; предполагается, что аргументы имеют тип `int`. Такие функции, как `printf` и `scanf`, полагаются на строку формата, которая описывает количество дополнительных аргументов и тип каждого из них.

Другая проблема связана с передачей `NULL` в качестве аргумента. `NULL` обычно определяется для представления 0. Когда 0 передаётся функции со списком аргументов переменной, компилятор предполагает, что он представляет собой целое число - он никак не может сказать, что мы хотим, чтобы он представлял нулевой указатель. Решение состоит в том, чтобы добавить приведение, записывая `(void *) NULL` или `(void *) 0` вместо `NULL`. (Более подробное обсуждение этого вопроса см. В разделе "Вопросы и ответы" в конце главы 17.)

Функции `v...printf`

```
int vfprintf (FILE * restrict stream, const char * restrict format,  
va_list arg); //from <stdio.h>
```

```
int vprintf(const char * restrict format, va_list arg);  
// from <stdio.h>
```

```
int vsnprintf (char * restrict s, size_t n, const char * restrict format,
```

```
va_list arg); //from <stdio.h>
```

```
int vsprintf (char * restrict s, const char * restrict format,
```

```
va_list arg); //from <stdio.h>
```

Функции `fprintf`, `vprintf` и `vsprintf` ("функции `v...printf`") принадлежат `<stdio.h>`. Мы обсуждаем их в этом разделе, потому что они неизменно используются вместе с макросами в `<stdarg.h>`. C99 добавляет функцию `vsnprintf`.

Функции `v... printf` тесно связаны с `fprintf`, `printf` и `sprintf`. Однако, в отличие от этих функций, функции `v... printf` имеют фиксированное количество аргументов. Последним аргументом каждой функции является значение `va_list`, что означает, что она будет вызываться функцией со списком переменного числа аргументов. На практике функции `v... printf` используются в основном для написания функций-оболочек, принимающих переменное количество аргументов, которые затем передаются функции `v... printf`.

В качестве примера предположим, что мы работаем над программой, которая должна время от времени отображать сообщения об ошибках. Мы бы хотели, чтобы каждое сообщение начиналось с префикса формы

```
** Error n:
```

где `n` равно 1 для первого сообщения об ошибке и увеличивается на единицу для каждой последующей ошибки. Чтобы упростить создание сообщений об ошибках, мы

1360

напишем функцию с именем `errorf`, которая похожа на `printf`, но добавляет

**** Error n:**

в начало вывода и всегда записывает в `stderr` вместо `stdout`. У нас будет `errorf`, вызывающая `vfprintf`, чтобы сделать большую часть фактического вывода. Вот как может выглядеть `errorf`:

```
int errorf(const char *format,...){
    static int num_errors = 0;
    int n;
    va_list ap;
    num_errors++;
    fprintf(stderr, "*** Error %d: ", num_errors);
    va_start(ap, format) ;
    n = vfprintf(stderr, format, ap);
    va_end(ap) ;
    fprintf(stderr, "\n") ;
    return n;
}
```

Функция-оболочка - в нашем примере - `errorf` - отвечает за вызов `va_start` перед вызовом функции `v...printf` и за вызов `va_end` после возврата из функции `v...printf`. Функция-

оболочка может вызывать `va_arg` один или несколько раз перед вызовом функции `v...printf`.

Функция `vsnprintf` была добавлена в версию `<stdio.h>` для C99. Он соответствует `snprintf` (обсуждается в Разделе 22.8), который также является функцией C99.

Функции **`v...scanf`**

```
int vfscanf(FILE * restrict stream, const char * restrict format,  
va_list arg); //from <stdio.h>
```

```
int vscanf(const char * restrict format, va_list arg);  
//from <stdio.h>
```

```
int vsscanf(const char * restrict s, const char * restrict format,  
va_list arg); //from <stdio.h>
```

C99 добавляет набор "функций `v...scanf`" в заголовок `<stdio.h>`. `vfscanf`, `vscanf` и `vsscanf` эквивалентны `fscanf`, `scanf` и `sscanf` соответственно, за исключением того, что у них есть параметр `va_list`, через который может быть передан список переменного числа аргументов. Как и функции `v...printf`, каждая функция `v..scanf` предназначена для вызова функцией-оболочкой, которая принимает переменное количество аргументов, которые затем передаются в функцию `v...scanf`.

Функция-оболочка отвечает за вызов `va_start` перед вызовом функции `v...scanf` и за вызов `va_end` после возврата из функции `v...scanf`.

26.2 Заголовок `<stdlib.h>`: Общие утилиты

`<stdlib.h>` служит сборником для функций, которые не подходят ни к одному из других заголовков. Функции в `<stdlib.h>` делятся на восемь групп:

- ✓ Числовые функции преобразования
- ✓ Функции генерации псевдослучайной последовательности
- ✓ Функции управления памятью
- ✓ Связь с окружающей средой
- ✓ Утилиты поиска и сортировки
- ✓ Целочисленные арифметические функции
- ✓ Функции преобразования многобайтовых/широких символов
- ✓ Функции преобразования многобайтовых/широких строк

Мы рассмотрим каждую группу по очереди, за тремя исключениями: функции управления памятью, функции

преобразования многобайтовых/широких символов и функции преобразования многобайтовых/широких строк.

Функции управления памятью (`malloc`, `calloc`, `realloc` и `free`) позволяют программе выделить блок памяти, а затем освободить его или изменить его размер. В главе 17 довольно подробно описаны все четыре функции.

Функции преобразования многобайтовых символов в широкие используются для преобразования многобайтовых символов в широкие символы и наоборот. Функции преобразования многобайтовых строк в широкие выполняют аналогичные преобразования между многобайтовыми строками и широкими строками. Обе группы функций обсуждаются в Разделе 25.2.

Числовые функции преобразования

```
int atoi(const char *nptr) ;
```

```
long int atol(const char *nptr) ;
```

```
long long int atoll(const char *nptr);
```

```
double strtod(const char * restrict nptr, char ** restrict endptr) ;
```

```
float strtof(const char * restrict nptr, char ** restrict endptr);
```

```
long double strtold(const char * restrict nptr, char ** restrict endptr);
```

1364

```
long int strtol(const char * restrict nptr, char ** restrict endptr, int  
base);
```

```
long long int strtoll(const char * restrict nptr, char ** restrict  
endptr, int base);
```

```
unsigned long int strtoul( const char * restrict nptr, char ** restrict  
endptr, int base);
```

```
unsigned long long int strtoull( const char * restrict nptr, char **  
restrict endptr, int base);
```

Функции числового преобразования (или "функции преобразования строк", как они называются в C89) преобразуют строки, содержащие числа в символьной форме, в их эквивалентные числовые значения. Три из этих функций довольно старые, ещё три были добавлены при создании стандарта C89 и ещё пять были добавлены в C99.

Все функции числового преобразования - новые или старые - работают во многом одинаково. Каждая функция пытается преобразовать строку (указанную параметром *nptr*) в число. Каждая функция пропускает символы пробела в начале строки, обрабатывает последующие символы как часть числа (возможно, начиная со знака плюс или минус) и останавливается на первом символе, который не может быть частью числа. Кроме того, каждая функция возвращает ноль, если преобразование невозможно (строка пуста или символы,

следующие за любым начальным пробелом, не имеют формы, которую ищет функция).

Старые функции (`atof`, `atoi` и `atol`) преобразуют строку в значение типа `double`, `int` или `long int` соответственно. К сожалению, эти функции не имеют возможности указать, какая часть строки была использована во время преобразования. Более того, функции не имеют возможности указать, что преобразование было неудачным. (Некоторые реализации этих функций могут изменять переменную `errno` при сбое преобразования, но это не гарантируется.)

Функции C89 (`strtod`, `strtol` и `strtoul`) более сложные. Во-первых, они указывают, где остановилось преобразование, изменяя переменную, на которую указывает `endptr`. (Второй аргумент может быть нулевым указателем, если нас не интересует, где закончилось преобразование.) Чтобы проверить, может ли функция использовать всю строку целиком, мы можем просто проверить, указывает ли эта переменная на нулевой символ. Если преобразование не может быть выполнено, переменной, на которую указывает `endptr`, присваивается значение `nptr` (если `endptr` не является нулевым указателем), более того, `strtol` и `strtoul` имеют аргумент `base`, указывающий основание преобразуемого числа. Поддерживаются все базы от 2 до 36 (включительно). Помимо того, что `strtod`, `strtol` и `strtoul` более универсальны, чем старые функции, они лучше обнаруживают ошибки.

Каждая функция сохраняет `ERANGE` в `errno`, если преобразование даёт значение, выходящее за пределы диапазона возвращаемого функцией типа. Кроме того, функция `strtod` возвращает плюс или минус `HUGE_VAL`; функции `strtol` и `strtoul` возвращают наименьшее или наибольшее значение соответствующих возвращаемых типов. (`strtol` возвращает либо `LONG_MIN`, либо `LONG_MAX`, а `strtoul` возвращает `ULONG_MAX`.)

C99 добавляет функции `atoll`, `strtof`, `strtold`, `strtoll` и `strtoull`. `atoll` - это то же самое, что и функция `atol`, за исключением того, что она преобразует строку в значение типа `long long int`. `strtof` и `strtold` аналогичны `strtod`, за исключением того, что они преобразуют строку в значение типа `float` или `long double` соответственно. `strtoll` - это то же самое, что и `strtol`, за исключением того, что он преобразует строку в значение `long long int`. `strtoull` - это то же самое, что и `strtoul`, за исключением того, что он преобразует строку в значение типа `unsigned long long int`. C99 также вносит небольшое изменение в функции преобразования чисел с плавающей запятой: строка, передаваемая в `strtod` (а также его более новые родственники, `strtof` и `strtold`), может содержать шестнадцатеричное число с плавающей запятой, бесконечность или NaN.

ПРОГРАММА Тестирование функций числового преобразования

Следующая программа преобразует строку в числовую форму, применяя каждую из шести функций числового преобразования, существующих в C89. После вызова функций strtod, strtol и strtoul программа также показывает, дало ли каждое преобразование допустимый результат и смогло ли оно использовать всю строку. Программа получает входную строку из командной строки.

```
//tnumconv.c
```

```
#include <errno.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define CHK_VALID printf("  %s  %s\n", \
    errno != ERANGE ? "Yes" : "No ", \
    *ptr == '\0' ? "Yes" : "No")
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    char *ptr;
```

1368

```
if (argc != 2) {
    printf("usage: tnumconv string\n");
    exit(EXIT_FAILURE);
}

printf("Function Return Value\n");
printf("-----\n");
printf("atof  %g\n", atof(argv[1]));
printf("atoi %d\n", atoi(argv[1]));
printf("atol  %ld\n\n", atol(argv[1]));

printf("Function Return Value    Valid?  "
       "String Consumed?\n"
       "-----  "
       "-----\n");

errno = 0;
printf("strtod  %-19g", strtod(argv[1], &ptr));
CHK_VALID;
```

1369

```
errno = 0;
printf("strtol  %-19ld", strtol(argv[1], &ptr, 10));
CHK_VALID;

errno = 0;
printf("strtoul  %-19lu", strtoul(argv[1], &ptr, 10));
CHK_VALID;

return 0;
}
```

Если 9223372036854775809 - аргумент командной строки, вывод программы может иметь следующий вид:

Function	Return Value
----------	--------------

-----	-----
-------	-------

atof	9.22337e+18
------	-------------

atoi	-1
------	----

atol	9223372036854775807
------	---------------------

Function	Return Value	Valid?	String Consumed?
----------	--------------	--------	------------------

-----	-----	-----	-----
-------	-------	-------	-------

1370

strtod	9.22337e+18	Yes	Yes
strtol	9223372036854775807	No	Yes
strtoul	9223372036854775809	Yes	Yes

На моей машине число 9223372036854775809 слишком велико для представления в виде длинного целого числа, хотя оно допустимо как длинное целое число без знака. Функции `atoi` и `atol` не могли указать, что число, представленное их аргументом, выходит за пределы допустимого диапазона. В показанном выводе они вернули: `atoi` $\rightarrow$ -1, поскольку для `atoi` если преобразованное значение выходит за пределы диапазона соответствующего типа, возвращаемое значение не определено, а для `atol` возвращено 9223372036854775807 (наибольшее длинное целое число). Функция `strtoul` выполнила преобразование правильно; `strtol` вернула 9223372036854775807 (стандарт требует, чтобы она возвращала наибольшее длинное целое число) и сохранила `ERANGE` в `errno`. Если 123.456 - это аргумент командной строки, вывод будет

Function	Return Value
----------	--------------

<code>atof</code>	123.456
-------------------	---------

<code>atoi</code>	123
-------------------	-----

1371

atol 123

Function	Return Value	Valid?	String Consumed?
----------	--------------	--------	------------------

-----	-----	-----	-----
-------	-------	-------	-------

strtod	123.456	Yes	Yes
--------	---------	-----	-----

strtol	123	Yes	No
--------	-----	-----	----

strtoul	123	Yes	No
---------	-----	-----	----

Все шесть функций рассматривали эту строку как допустимое число, хотя целочисленные функции останавливались на десятичной точке. Функции strtol и strtoul смогли указать, что они не полностью потребляют строку.

Если foo является аргументом командной строки, то выводом будет

Function	Return Value
----------	--------------

-----	-----
-------	-------

atof	0
------	---

atoi	0
------	---

atol	0
------	---

Function	Return Value	Valid?	String Consumed?
----------	--------------	--------	------------------

-----	-----	-----	-----
-------	-------	-------	-------

strtod	0	Yes	No
--------	---	-----	----

strtol	0	Yes	No
--------	---	-----	----

strtoul	0	Yes	No
---------	---	-----	----

Все функции посмотрели на букву *f* и тут же вернули ноль. Функции str... не изменили errno, но мы можем сказать, что что-то пошло не так из-за того, что функции не потребляли строку.

Функции генерации псевдослучайных последовательностей

```
int rand(void);
```

```
void srand(unsigned int seed);
```


Функции `rand` и `srand` поддерживают генерацию псевдослучайных чисел. Эти функции полезны в программах моделирования и игровых программах (например, для имитации броска костей или раздачи в карточной игре).

Каждый раз, когда она вызывается, `rand` возвращает число от 0 до `RAND_MAX` (макрос, определённый в `<stdlib.h>`), числа, возвращаемые `rand`, на самом деле не являются случайными; они генерируются из "начального" значения.

Невнимательному наблюдателю, однако, кажется что `rand` производит несвязанную последовательность чисел.

Вызов `srand` обеспечивает начальное значение для `rand`. Если `rand` вызывается до `srand`, то начальное значение принимается равным 1. Каждое начальное значение определяет определённую последовательность псевдослучайных чисел; `srand` позволяет нам выбрать, какую последовательность мы хотим. Программа, которая всегда использует одно и то же начальное значение, всегда будет получать одну и ту же последовательность чисел от `rand`. Это свойство иногда может быть полезно: программа ведёт себя одинаково при каждом запуске, что облегчает тестирование. Тем не менее, мы обычно хотим, чтобы `rand` создавал другую последовательность при каждом запуске программы.

(Программа игры в покер, которая всегда сдаёт одни и те же карты, вряд ли будет популярной.) Самый простой способ "рандомизировать" начальные значения - вызвать функцию `time`, которая возвращает число, кодирующее текущую

дату и время. Передача возвращаемого значения времени `srand` приводит к тому, что поведение `rand` меняется от одного запуска к другому. Примеры этой техники см. в программах `guess.c` и `guess2.c` (раздел 10.2).

ПРОГРАММА Тестирование функций генерации псевдослучайной последовательности

Следующая программа отображает первые пять значений, возвращаемых функцией `rand`, а затем позволяет пользователю выбрать новое начальное значение. Процесс повторяется до тех пор, пока пользователь не введет ноль в качестве начального числа.

```
// trand.c

#include <stdio.h>
#include <stdlib.h>

int main(void){
    int i, seed;

    printf("This program displays the first five values of "
        "rand.\n");
    for (;;) {
        for (i = 0; i < 5; i++)
            printf("%d ", rand());
```

1375

```
printf("\n\n");
printf("Enter new seed value (0 to terminate): ");
scanf("%d", &seed);
if (seed == 0)
    break;
srand(seed);
}
return 0;
}
```

Вот как может выглядеть сеанс с программой:

This program displays the first five values of rand.

1804289383 846930886 1681692777 1714636915 1957747793

Enter new seed value (0 to terminate): 100

677741240 611911301 516687479 1039653884 807009856

Enter new seed value (0 to terminate): 1

1804289383 846930886 1681692777 1714636915 1957747793

Enter new seed value (0 to terminate): 0

Существует множество способов написания функции rand, поэтому нет никакой гарантии, что каждая версия rand будет генерировать числа, показанные здесь. Обратите внимание,

1376

что выбор 1 в качестве семени даёт ту же последовательность чисел, что и отсутствие указания семени вообще.

Есть много способов написать функцию rand, поэтому нет гарантии, что каждая версия rand будет генерировать числа, показанные здесь. Обратите внимание, что при выборе семени равным 1 выдаётся та же последовательность чисел, что и при не указании семени вообще.

Связь с окружающей средой

```
void abort (void);
```

```
int atexit(void (*func) (void));
```

```
void exit(int status);
```

```
void _Exit (int status);
```

```
char *getenv(const char *name);
```

```
int system(const char *string) ;
```

Функции этой группы обеспечивают простой интерфейс операционной системы, позволяя программам (1) завершать работу, как обычно, так и ненормально, и возвращать код состояния операционной системе, (2) извлекать информацию из среды пользователя и (3) выполнять команды

операционной системы. Одна из функций, `_Exit`; является добавлением C99.

Выполнение вызова `exit(n)` в любом месте программы обычно эквивалентно выполнению инструкции `return n`; в `main`: программа завершается, и *n* возвращается операционной системе в виде кода состояния. `<stdlib.h>` определяет макросы `EXIT_FAILURE` и `EXIT_SUCCESS`, которые могут использоваться в качестве аргументов для выхода.

Единственным другим переносимым аргументом для выхода является 0, который имеет то же значение, что и `EXIT_SUCCESS`. Возврат кодов состояния, отличных от этих, является допустимым, но не обязательно переносимым для всех операционных систем.

Когда программа завершается, она обычно выполняет несколько заключительных действий за кулисами, включая очистку выходных буферов, содержащих незаписанные данные, закрытие открытых потоков и удаление временных файлов. У нас могут быть другие действия по "очистке", которые, как хотелось бы нам, программа выполняла при завершении. Функция `atexit` позволяет нам "зарегистрировать" функцию, которая будет вызываться при завершении программы. Например, чтобы зарегистрировать функцию с именем `cleanup`, мы могли бы вызвать `atexit` следующим образом:

```
atexit (cleanup) ;
```

Когда мы передаём `atexit` указатель на функцию, она сохраняет указатель для использования в будущем. Если позже программа завершится нормально (посредством вызова `exit` или инструкции `return` в функции `main`), любая функция, зарегистрированная с помощью `atexit`, будет вызываться автоматически. (Если две или более функции были зарегистрированы, они вызываются в порядке, обратном порядку, в котором они были зарегистрированы.)

Функция `_Exit` аналогична функции `exit`. Однако `_Exit` не вызывает функции, которые были зарегистрированы с помощью `atexit`, и не вызывает никаких обработчиков сигналов, ранее переданных функции сигнала. Кроме того, `_Exit` не обязательно очищает выходные буферы, закрывает открытые потоки или удаляет временные файлы - выполнение этих действий определяется реализацией.

`abort` также похожа на `exit`, но её вызов вызывает аварийное завершение программы. Функции, зарегистрированные в `atexit`, не вызываются. В зависимости от реализации может случиться так, что буферы вывода, содержащие незаписанные данные, не сбрасываются, потоки не закрываются, а временные файлы не удаляются. `abort` возвращает определяемый реализацией код состояния, указывающий на неудачное завершение.

Многие операционные системы предоставляют "среду": набор строк, описывающих характеристики пользователя. Эти

строки обычно включают путь, по которому будет выполняться поиск, когда пользователь запускает программу, тип пользовательского терминала (в случае многопользовательской системы) и так далее. Например, путь поиска UNIX может выглядеть примерно так:

```
PATH = /usr/local/bin:/bin:/usr/bin:.
```

`getenv` предоставляет доступ к любой строке в среде пользователя. Чтобы найти текущее значение строки `PATН`, например, мы могли бы написать

```
char *p = getenv("PATН") ;
```

p теперь указывает на строку `"/usr/local/bin:/bin:/usr/bin:."`.

Будьте осторожны с `getenv`: она возвращает указатель на статически выделенную строку, которая может быть изменена последующим вызовом функции.

Системная функция позволяет программе на С запускать другую программу (возможно, команду операционной системы). Аргумент `system` - это строка, содержащая команду, аналогичную той, которую мы вводим в командной строке операционной системы. Например, предположим, что мы пишем программу, которой нужен список файлов в текущем каталоге. Программа UNIX будет вызывать систему следующим образом:

```
system("ls >myfiles") ;
```

Этот вызов вызывает команду UNIX `ls` и просит её записать список текущего каталога в файл с именем `myfiles`.

Возвращаемое значение системы определяется реализацией. `system` обычно возвращает код статуса завершения из программы, которую мы просили запустить; проверка этого значения позволяет нам проверить, правильно ли работает программа. Вызов системы с нулевым указателем имеет особое значение: функция возвращает ненулевое значение, если доступен командный процессор.

Утилиты поиска и сортировки

```
void *bsearch(const void *key, const void *base, size_t nmemb,  
size_t size, int (*compar) (const void *, const void *));
```

```
void qsort (void *base, size_t nmemb, size_t size, int (*compar)  
(const void *, const void *));
```

Функция `bsearch` ищет в отсортированном массиве определённое значение ("ключ"). Когда вызывается `bsearch`, параметр `key` указывает на ключ, `base` указывает на массив, `nmemb` - количество элементов в массиве, `size` - это размер

каждого элемента (в байтах), а `compare` - указатель на функцию сравнения, функция сравнения аналогична функции, требуемой `qsort`: при передаче указателей на ключ и элемент массива (в этом порядке) функция должна возвращать отрицательное, нулевое или положительное целое число в зависимости от того, является ли ключ меньше, равен или больше элемента массива. `bsearch` возвращает указатель на элемент, соответствующий ключу; если совпадения не найдено, `bsearch` возвращает нулевой указатель.

Хотя стандарт C не требует этого, `bsearch` обычно использует алгоритм двоичного поиска для поиска в массиве. `bsearch` сначала сравнивает ключ с элементом в середине массива; если есть совпадение, функция вернётся. Если ключ меньше среднего элемента, `bsearch` ограничивает поиск первой половиной массива; если ключ больше, `bsearch` ищет только последнюю половину массива. `bsearch` повторяет эту стратегию до тех пор, пока не найдёт ключ или не закончатся элементы для поиска. Благодаря этой технике `bsearch` выполняется довольно быстро - поиск в массиве из 1000 элементов требует не более 10 сравнений; для поиска в массиве из 1 000 000 элементов требуется не более 20 сравнений. В разделе 17.7 обсуждается функция `qsort`, которая может сортировать массив. `bsearch` работает только для отсортированных массивов, но мы всегда можем использовать `qsort` для сортировки массива перед тем, как запросить поиск у `bsearch`.

ПРОГРАММА Определение длины воздушного пути в милях

Наша следующая программа вычисляет расстояние от Нью-Йорка до различных зарубежных городов. Программа сначала просит пользователя ввести название города, а затем отображает пробег до этого города:

Enter city name: Shanghai

Shanghai is 7371 miles from New York City.

Программа сохранит пары город/расстояние в массиве. Используя bsearch для поиска в массиве названия города, программа может легко найти соответствующее расстояние. (Мили с сайта Infoplease.com.)

```
//airmiles.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct city_info {
```

```
    char *city;
```

```
    int miles;
```

```
};
```

```
int compare_cities(const void *key_ptr,
```

1383

```
    const void *element_ptr);

int main(void){
    char city_name[81];
    struct city_info *ptr;
    const struct city_info mileage[] =
        {"Berlin", 3965}, {"Buenos Aires", 5297},
        {"Cairo", 5602}, {"Calcutta", 7918},
        {"Cape Town", 7764}, {"Caracas", 2132},
        {"Chicago", 713}, {"Hong Kong", 8054},
        {"Honolulu", 4964}, {"Istanbul", 4975},
        {"Lisbon", 3364}, {"London", 3458},
        {"Los Angeles", 2451}, {"Manila", 8498},
        {"Mexico City", 2094}, {"Montreal", 320},
        {"Moscow", 4665}, {"Paris", 3624},
        {"Rio de Janeiro", 4817}, {"Rome", 4281},
        {"San Francisco", 2571}, {"Shanghai", 7371},
        {"Stockholm", 3924}, {"Sydney", 9933},
        {"Tokyo", 6740}, {"Warsaw", 4344},
        {"Washington", 205}};
```

1384

```
printf("Enter city name: ");
scanf("%80[^\n]", city_name);
ptr = bsearch(city_name, mileage,
              sizeof(mileage) / sizeof(mileage[0]),
              sizeof(mileage[0]), compare_cities);
if (ptr != NULL)
    printf("%s is %d miles from New York City.\n",
          city_name, ptr->miles);
else
    printf("%s wasn't found.\n", city_name);

return 0;
}

int compare_cities(const void *key_ptr,
                  const void *element_ptr)
{
    return strcmp((char *) key_ptr,
                  ((struct city_info *) element_ptr)->city);
}
```

Целочисленные арифметические функции

```
int abs(int j);
```

```
long int labs(long int j);
```

```
long long int llabs(long long int j);
```

```
div_t div(int numer, int denom);
```

```
ldiv_t ldiv(long int numer, long int denom) ;
```

```
lldiv_t lldiv(long long int numer, long long int denom) ;
```

Функция `abs` возвращает абсолютное значение типа `int`;

функция `labs` возвращает абсолютное значение типа `long int`.

Функция `div` делит свой первый аргумент на второй, возвращая значение `div_t`. `div_t` - это структура, которая содержит как частный член (с именем `quot`), так и остаточный член (`rem`). Например, если `ans` - это переменная `div_t`, мы могли бы написать

```
ans = div (5, 2);
```

```
printf ("Частное: %d Остаток:% d \ n", ans.quot, ans.rem);
```

Функция `ldiv` аналогична, но работает с длинными целыми числами; она возвращает структуру `ldiv_t`, которая также имеет члены `quot` и `rem`. (Типы `div_t` и `ldiv_t` объявлены в `<stdlib.h>`.)

C99 предоставляет две дополнительные функции. Функция `llabs` возвращает абсолютное значение типа `long long int`. `lldiv` похожа на `div` и `ldiv`, за исключением того, что она делит два `long long int` значения и возвращает структуру `lldiv_t`. (Тип `lldiv_t` также был добавлен в C99.)

26.3 Заголовок `<time.h>`: Дата и время

Заголовок `<time.h>` предоставляет функции для определения времени (включая дату), выполнения арифметических операций над значениями времени и форматирования времени для отображения. Однако прежде чем мы исследуем эти функции, нам нужно обсудить, как хранится время. `<time.h>` предоставляет три типа, каждый из которых представляет свой способ хранения времени:

- `clock_t`: значение времени, измеряемое в "тактах системных часов".

- `time_t`: компактное закодированное время и дата (календарное время).
- `struct tm`: время, разделённое на секунды, минуты, часы и т. д.

Значение типа `struct tm` часто называют разбитым временем. В таблице 26.1 показаны члены структуры `tm`. Все члены имеют тип `int`.

Имя	Описание	Минимальное значение	Максимальное значение
<code>tm_sec</code>	Секунды после минуты	0	61↑
<code>tm_min</code>	Минуты после часа	0	59
<code>tm_hour</code>	Часы с полуночи	0	23
<code>tm_mday</code>	День месяца	1	31
<code>tm_mon</code>	Месяцы с января	0	11
<code>tm_year</code>	Год с 1900	0	-
<code>tm_wday</code>	Дней после воскресенья	0	6
<code>tm_yday</code>	Дней с 1 января	0	365
<code>tm_isdst</code>	Флаг перехода на летнее время	↓	↓

1388

↑ Позволяет получить две дополнительные "високосные секунды". В C99 максимальное значение - 60.

‡ Положительное, если действует летнее время, ноль, если оно не действует, и отрицательное, если эта информация неизвестна.

Таблица 26.1 Члены структуры tm

Эти типы используются для разных целей. Значение `clock_t` подходит только для представления интервалов времени; Значения `time_t` и `struct tm` могут хранить дату и время целиком. Значения `time_t` жестко закодированы, поэтому они занимают мало места. Значения `struct tm` требуют гораздо больше места, но с ними часто легче работать. В стандарте C указано, что `clock_t` и `time_t` должны быть "арифметическими типами", и это вся информация. Мы даже не знаем, хранятся ли значения `clock_t` и `time_t` как целые числа или числа с плавающей запятой.

Теперь мы готовы рассмотреть функции в `<time.h>`, которые делятся на две группы: функции управления временем и функции преобразования времени.

Функции управления временем

```
clock_t clock(void) ;
```


1389

```
double difftime(time_t time1, time_t time0);
```

```
time_t mktime(struct tm *timeptr) ;
```

```
time_t time(time_t *timer) ;
```

Функция `clock` возвращает значение `clock_t`, представляющее процессорное время, используемое программой с момента начала выполнения. Чтобы преобразовать это значение в секунды, мы можем разделить его на `CLOCKS_PER_SEC`, макрос, определённый в `<time.h>`.

Когда `clock` используются для определения того, как долго программа работает, обычно её вызывают дважды: один раз в начале `main` и один раз непосредственно перед завершением программы:

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main (void){
```

```
    clock_t start_clock = clock();
```

```
    printf ("Processor time used: %g sec. \n",
```

```
    (clock() - start_clock) / (double) CLOCKS _PER_SEC);
```

```
    return 0;
```

```
}
```

1390

Причина первоначального вызова `clock` заключается в том, что программа использует некоторое время процессора, прежде чем достигнет `main`, благодаря скрытому "запускающему" коду. Вызов `clock` в начале `main` определяет, сколько времени требуется для кода запуска, чтобы мы могли вычесть его позже. Стандарт C89 говорит только, что `clock_t` - это арифметический тип; тип `CLOCKS_PER_SEC` не указан. В результате тип выражения

$(\text{clock}() - \text{start_clock}) / \text{CLOCKS_PER_SEC}$

может отличаться от одной реализации к другой, что затрудняет отображение с помощью `printf`. Чтобы решить эту проблему, наш пример преобразует `CLOCKS_PER_SEC` в `double`, заставляя все выражение иметь тип `double`. В C99 тип `CLOCKS_PER_SEC` определён как `clock_t`, но `clock_t` по-прежнему является типом, определяемым реализацией.

Функция `time` возвращает текущее календарное время. Если его аргумент не является нулевым указателем, `time` также сохраняет календарное время в объекте, на который указывает аргумент. Способность `time` возвращать время двумя разными способами - историческая причуда, но она даёт нам возможность писать либо

```
cur_time = time (NULL) ;
```

или

1391

`time (&cur_time) ;`

где `cur_time` - это переменная типа `time_t`.

Функция `difftime` возвращает разницу между `time0` (более раннее время) и `time1`, измеренную в секундах. Таким образом, чтобы вычислить фактическое время выполнения программы (а не время процессора), мы могли бы использовать следующий код:

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main(void){
```

```
    time_t start_time = time (NULL) ;
```

```
    printf ("Running time: %g sec.\n",
```

```
           difftime (time (NULL), start_time));
```

```
    return 0;
```

```
}
```

Функция `mktime` преобразует время с разбивкой (хранящейся в структуре, на которую указывает её аргумент) в календарное время, которое затем возвращает. В качестве побочного эффекта `mktime` регулирует элементы структуры в соответствии со следующими правилами:

- ✗ `mktime` изменяет все члены, значения которых выходят за пределы допустимого диапазона (см. Таблицу 26.1). Эти изменения, в свою очередь, могут потребовать внесения изменений в других членов. Если `tm_sec` слишком велик, например, `mktime` сокращает его до нужного диапазона (0-59), добавляя дополнительные минуты к `tm_min`. Если `tm_min` теперь слишком велик, `mktime` уменьшает его и добавляет дополнительные часы к `tm_hour`. Если необходимо, процесс будет продолжен для членов `tm_mday`, `tm_mon` и `tm_year`.
- ✗ После настройки других членов структуры (при необходимости) `mktime` устанавливает правильные значения `tm_wday` (день недели) и `tm_yday` (день года). Никогда не нужно инициализировать значения `tm_wday` и `tm_yday` перед вызовом `mktime`; он игнорирует исходные значения этих членов.

Возможность `mktime` настраивать элементы структуры `tm` делает его полезным для арифметики, связанной со временем. В качестве примера давайте воспользуемся `mktime`, чтобы ответить на следующий вопрос: если Олимпийские игры 2012 года начнутся 27 июля и закончатся через 16 дней, какова дата окончания игр ? Начнём с сохранения 27 июля 2012 года в структуре `tm`:

```
struct tm t;
```

```
t.tm_mday = 27;
```

1393

```
t.tm_mon = 6 /* July */
```

```
t.tm_year = 112; /* 2012 от 1900*/
```

Мы также инициализируем другие члены структуры (кроме `tm_wday` и `tm_yday`), чтобы убедиться, что они не содержат неопределенных значений, которые могут повлиять на ответ:

```
t.tm_sec = 0;
```

```
t.tm_min = 0;
```

```
t.tm_hour = 0;
```

```
t.tm_isdst = -1;
```

Затем мы добавим 16 к члену `tm_mday`:

```
t.tm_mday += 16;
```

Остается 43 в `tm_mday`, что выходит за пределы допустимого диапазона для этого члена. Вызов `mktime` вернёт элементы структуры в их правильные диапазоны:

```
mktime (&t) ;
```

Мы отбросим возвращаемое значение `mktime`, поскольку нас интересует только влияние функции на *t*. Соответствующие члены *t* теперь имеют следующие значения:

Член	Значение	Имея в виду
<code>tm_mday</code>	12	12
<code>tm_mon</code>	7	August
<code>tm_year</code>	112	2012

1394

tm_wday	0	Sunday
tm_yday	224	225-й день в году

Функции преобразования времени

```
char *asctime(const struct tm *timeptr);
```

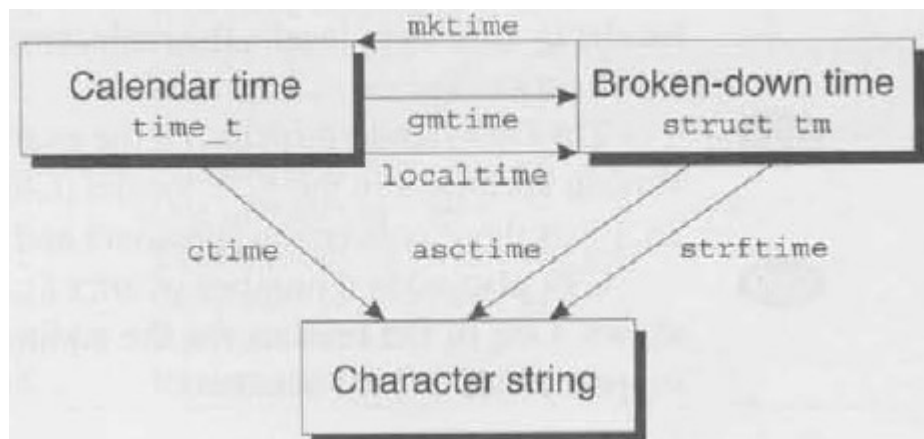
```
char *ctime(const time_t *timer);
```

```
struct tm *gmtime(const time_t *timer);
```

```
struct tm *localtime(const time_t *timer);
```

```
size_t strftime(char * restrict s, size_t maxsize, const char * restrict  
format, const struct tm * restrict timeptr) ;
```

Функции преобразования времени позволяют преобразовывать календарное время в сокращённое время. Они также могут преобразовывать время (календарное или с разбивкой) в строковую форму. На следующем рисунке показано, как связаны эти функции:



На рисунке показана функция `mktime`, которую стандарт C классифицирует как функцию "манипуляции", а не как функцию "преобразования".

Функции `gmtime` и `localtime` похожи. При передаче указателя на календарное время оба возвращают указатель на структуру, содержащую эквивалентное время с разбивкой. `localtime` производит местное время, а возвращаемое значение `gmtime` выражается в UTC (всемирное координированное время). Возвращаемое значение `gmtime` и `localtime` указывает на статически выделенную структуру, которая может быть изменена последующим вызовом любой функции.

Функция `asctime` (ASCII time) возвращает указатель на строку с нулевым завершением вида

Sun Jun 3 17:48:34 2007\n

построенный из разбитого времени, на которое указывает её аргумент. Функция `ctime` возвращает указатель на строку, описывающую местное время. Если `cur_time` является переменной типа `time_t`, то вызов

`time (&cur_time)`

эквивалентно

`asctime (localtime (&cur_time) )`

Возвращаемое значение `asctime` и `ctime` указывает на статически выделенную строку, которая может быть изменена более поздним вызовом любой из функций.

Функция `strftime`, как и функция `asctime`, преобразует разбитое время в строковую форму. Однако, в отличие от `asctime`, он даёт нам большой контроль над форматированием времени. Фактически, `strftime` напоминает `sprintf` тем, что записывает символы в строку `s` (первый аргумент) в соответствии с форматной строкой (третий аргумент). Строка формата может содержать обычные символы (которые копируются в `s` без изменений) вместе со спецификаторами преобразования, показанными в таблице 26.2 (которые заменены указанными строками). Последний аргумент указывает на структуру `tm`, которая используется как источник информации о дате и времени. Второй аргумент - это ограничение на количество символов, которые могут храниться в `s`.

Функция `strftime`, в отличие от других функций в `<time.h>`, чувствительна к текущему языку. Изменение категории `LC_TIME` может повлиять на поведение спецификаторов преобразования. Примеры в Таблице 26.2 предназначены строго для локали "C"; в немецком языке заменой `%A` может быть `Dienstag` вместо `Tuesday`.

Стандарт C99 определяет точные строки замены для некоторых спецификаторов преобразования в локали "C". (C89 не вдавался в подробности этого уровня.) В таблице 26.3 перечислены эти спецификаторы преобразования и строки, которыми они заменяются.

C99 также добавляет ряд спецификаторов преобразования `strftime`, как показано в таблице 26.2. Одной из причин использования дополнительных спецификаторов преобразования является желание поддержать стандарт ISO 8601.

Спецификатор преобразования	Замена
<code>%a</code>	Сокращенное название дня недели (например, Sun)
<code>%A</code>	Полное название дня недели (например, Sunday)
<code>%b</code>	Сокращенное название месяца (например, Jun)

1398

%B	Полное название месяца (например, June)
%c	Полностью день и время (например, Sun Jun 3 17:48:34 2007)
%C	Год разделен на 100 и усечен до целого числа (00–99)
%d	День месяца (01-31)
%D	Эквивалентно %m /%d /%y
%e	День месяца (1-31); перед одной цифрой стоит пробел
%F	Эквивалент %Y-%m-%d
%g	Последние две цифры года по стандарту ISO 8601 (00-99)
%G	ISO 8601 недельный год
%h	Эквивалент %b
%H	Час в 24-часовом формате (00-23)
%I	Час в 12-часовом формате (01-12)
%j	День года (001-366)
%m	Месяц (01-12)
%M	Минуты (00-59)
%n	Символ новой строки
%p	Обозначение AM / PM (AM или PM)
%r	12-часовое время (например, 17:48:34 PM)
%R	Эквивалент %H: %M
%S	Секунды (00-61); максимальное значение в C99 - 60

1399

%t	Символ горизонтальной табуляции
%T	Эквивалент %H: %M: %S
%u	TSO 8601 день недели (1-7); Понедельник 1
%U	Номер недели (00-53); первое воскресенье - начало недели 1
%V	Номер недели ISO 8601 (01-53)
%w	День недели (0-6); Воскресенье 0
%W	Номер недели (00-53); первый понедельник - начало недели 1
%x	Полная дата (например, 06/03/07)
%X	Полное время (например, 17:48:34)
%y	Две последние цифры года (00-99)
%Y	Год
%z	Смещение от UTC в формате ISO 8601 (например, -0530 или +0200)
%Z	Название часового пояса или аббревиатура (например, EST)
%%	%

Таблица 26.2 Спецификаторы преобразования для функции
strftime

Спецификатор преобразования	Замена
%A	Первые три символа

A	Одно из "Sunday", "Monday",..., "Saturday"
b	Первые три символа %B
B	Одно из "January", "February",..., "December"
c	Эквивалентно "%a %b %e %T %Y"
p	Одно из "AM" или "PM"
r	Эквивалентно "%I: %M: %S%p"
x	Эквивалентно "%m/%d/%y"
X	Эквивалентно %T
Z	Определяется реализацией

Таблица 26.2 Строки замены для спецификаторов
преобразования strftime для локали "C"

ISO 8601

ISO 8601 - это международный стандарт, описывающий способы представления даты и времени. Первоначально он был опубликован в 1988 году, а затем обновлён в 2000 и 2004 годах. В соответствии с этим стандартом даты и время полностью числовые (т. е. месяцы не представлены именами), а часы выражаются в 24-часовом формате.

1401

Существует ряд форматов даты и времени ISO 8601, некоторые из которых напрямую поддерживаются спецификаторами преобразования strftime в C99. Основной формат даты ISO 8601 (ГГГГ-ММ-ДД) (YYYY-MM-DD) и основной формат времени (чч: мм: сс) (hh:mm:ss) соответствуют спецификаторам преобразования %F и %T соответственно.

ISO 8601 имеет систему нумерации недель в году; эта система поддерживается спецификаторами преобразования %g, %G и %v. Недели начинаются в понедельник, а неделя 1 - это неделя, в которую входит первый четверг года.

Следовательно, первые несколько дней января (аж три) могут относиться к последней неделе прошлого года. Например, рассмотрим календарь на январь 2011 года:

January 2011							Year	Week
Mo	Tu	We	Th	Fr	Sa	Su		
					1	2	2010	52
3	4	5	6	7	8	9	2011	1
10	11	12	13	14	15	16	2011	2
17	18	19	20	21	22	23	2011	3
24	25	26	27	28	29	30	2011	4
31							2011	5

6 января - первый четверг года, поэтому неделя с 3 по 9 января - это неделя 1. 1 января и 2 января относятся к последней неделе (52 неделя) предыдущего года. Для этих двух дат strftime заменит %g на 10, %G на 2010 и %v на 52.

Обратите внимание, что последние несколько дней декабря иногда будут приходиться на первую неделю следующего года; это происходит всякий раз, когда 29, 30 или 31 декабря - понедельник.

Спецификатор преобразования %z соответствует спецификации часового пояса ISO 8601: -hhmm означает, что часовой пояс отстает от UTC на hh часов и mm минут; строка +hhmm указывает величину, на которую часовой пояс опережает UTC.

C99 позволяет использовать символ E или O для изменения значения определённых спецификаторов преобразования strftime. Спецификаторы преобразования, которые начинаются с модификатора E или O, вызывают замену с использованием альтернативного формата, который зависит от текущего языкового стандарта. Если альтернативное представление не существует в текущем языковом стандарте, модификатор не действует. (В языковом стандарте "C" E и O игнорируются.) В таблице 26.4 перечислены все спецификаторы преобразования, которым разрешено иметь модификаторы E или O.

ПРОГРАММА Отображение даты и времени

Допустим, нам нужна программа, которая отображает текущую дату и время. Первым шагом программы, конечно же, является вызов функции времени для получения

1403

календарного времени. Второй шаг - преобразовать время в строковую форму и распечатать её. Самый простой способ выполнить второй шаг - вызвать `ctime`, который возвращает указатель на строку, содержащую дату и время, а затем передать этот указатель в функции `puts` или `printf`.

Специфи
каторы

Замена

%Ec	Альтернативное представление даты и времени
%EC	Название базового года (периода) в альтернативном представлении
%Ex	Альтернативное представление даты
%EX	Альтернативное представление времени
%Ey	Смещение от %EC (только год) в альтернативном представлении
%EY	Полное альтернативное представление года
%Od	День месяца с использованием альтернативных цифровых символов (заполненных ведущими нулями или ведущими пробелами, если нет альтернативного символа для нуля)
%Oe	День месяца с использованием альтернативных цифровых символов (заполненных ведущими пробелами)
%OH	Час в 24-часовом формате с использованием альтернативных цифровых символов
%OI	Час в 12-часовом формате с использованием

1404

альтернативных цифровых символов

%Om Месяц с использованием альтернативных цифровых символов

%OM Минуты с использованием альтернативных цифровых символов

%OS Секунды с использованием альтернативных числовых символов

%Ou День недели ISO 8601 как число в альтернативном представлении, где понедельник 1

%OU Номер недели с использованием альтернативных цифровых символов

%OV Номер недели ISO 8601 с использованием альтернативных цифровых символов

%Ow День недели в виде числа с использованием альтернативных цифровых символов

%OW Номер недели с использованием альтернативных цифровых символов

%Oy Последние две цифры года с использованием альтернативных цифровых символов

Таблица 26.4 E- и O -модифицированные спецификаторы преобразования для функции strftime (только C99)

Все идёт нормально. Но что, если мы хотим, чтобы программа отображала дату и время определенным образом?

Предположим, нам нужен следующий формат, где 06 - это месяц, а 03 - день месяца:

06-03-2007 5: 48p

Функция `ctime` всегда использует один и тот же формат даты и времени, так что это бесполезно. Функция `strftime` лучше; используя её, мы можем почти добиться желаемого внешнего вида. К сожалению, `strptime` не позволяет отображать однозначный час без нуля в начале. Кроме того, `strftime` использует АМ и РМ вместо *a* и *p*.

Когда `strftime` недостаточно хороша, у нас есть другая альтернатива: преобразовать календарное время в разбитое время, затем извлечь соответствующую информацию из структуры `tm` и отформатировать её самостоятельно, используя `printf` или аналогичную функцию. Мы могли бы даже использовать `strftime` для выполнения некоторого форматирования, прежде чем другие функции завершат работу.

Следующая программа иллюстрирует варианты. Она отображает текущую дату и время в трёх форматах: в том, который используется `ctime`, во втором, близком к тому, что мы хотим (созданный с помощью `strftime`), и желаемом формате (созданным с помощью `printf`). Версия `ctime` проста в

1406

исполнении, версия strftime немного сложнее, а версия printf самая сложная.

```
// datetime.c
```

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main(void){
```

```
    time_t current = time(NULL);
```

```
    struct tm *ptr;
```

```
    char date_time[21];
```

```
    int hour;
```

```
    char am_or_pm;
```

```
    /* Печать даты и времени в формате по умолчанию*/
```

```
    puts(ctime(&current));
```

```
    /* Print date and time, using strftime to format */
```

```
    strftime(date_time, sizeof(date_time),
```

```
        "%m-%d-%Y %I:%M%p\n", localtime(&current));
```

```
    puts(date_time);
```

```
    /* Print date and time, using printf to format */
```

1407

```
ptr = localtime(&current);
hour = ptr->tm_hour;
if (hour <= 11)
    am_or_pm = 'a';
else {
    hour -= 12;
    am_or_pm = 'p';
}
if (hour == 0)
    hour = 12;
printf("%.2d-%.2d-%d %2d:%.2d%c\n", ptr->tm_mon + 1,
    ptr->tm_mday, ptr->tm_year + 1900, hour,
    ptr->tm_min, am_or_pm);

return 0;
}
```

Вопросы и ответы

Вопрос: Хотя `<stdlib.h>` предоставляет ряд функций, которые преобразуют строки в числа, похоже, что нет никаких функций, которые преобразуют числа в строки. Что же делать?

Ответ: Некоторые библиотеки C предоставляют функции с такими именами, как `itoa`, которые преобразуют числа в строки. Однако использование этих функций - не лучшая идея: они не являются частью стандарта C и не могут быть переносимы. Лучший способ выполнить такое преобразование - вызвать такую функцию, как `sprintf`, которая записывает форматированный вывод в строку:

```
char str [20];
```

```
int i;
```

```
...
```

```
sprintf (str, "%d", i); /* записывает i в строку str */
```

`sprintf` не только портативен, но и обеспечивает большой контроль над внешним видом числа.

Вопрос: В описании функции `strtod` говорится, что C99 позволяет строковому аргументу содержать шестнадцатеричное число с плавающей точкой, бесконечность или NaN. Какой формат у этих чисел?

Ответ: Шестнадцатеричное число с плавающей точкой начинается с 0x или 0X, за которыми следуют одна или несколько шестнадцатеричных цифр (возможно, включая символ десятичной точки), а затем, возможно, двоичный показатель степени. (См. Вопросы и ответы в конце главы 7 для обсуждения шестнадцатеричных плавающих констант, которые имеют похожий, но не идентичный формат.)

Бесконечность имеет форму INF или INFINITY; любая или все буквы могут быть строчными. NaN представлено строкой NAN (снова без учёта регистра), за которой, возможно, следует пара круглых скобок. Скобки могут быть пустыми или содержать серию символов, где каждый символ представляет собой букву, цифру или подчёркивание. Эти символы могут использоваться для указания некоторых битов в двоичном представлении значения NaN, но их точное значение определяется реализацией. Такой же вид символьной последовательности, которую стандарт C99 называет n-символьной последовательностью, также используется в вызовах функции nan.

Вопрос: Вы сказали, что выполнение вызова exit(n) в любом месте программы обычно эквивалентно выполнению инструкции return n; в main. Когда это не будет эквивалентно?

Ответ: Есть две проблемы. Во-первых, когда функция main возвращается, время жизни её локальных переменных

заканчивается (при условии, что они имеют автоматическую продолжительность хранения, а не объявлены статическими), что неверно, если вызывается функция `exit`. Проблема возникнет, если любое действие, которое происходит при завершении программы - например, вызов функции, ранее зарегистрированной с помощью `atexit`, или очистка буфера выходного потока - требует доступа к одной из этих переменных. В частности, программа могла вызвать `setvbuf` и использовать одну из основных переменных в качестве буфера. Таким образом, в редких случаях программа может вести себя неправильно, если пытается вернуться из `main`, но работает, если вместо этого вызывает `exit`.

Другая проблема возникает только в C99, что делает допустимым наличие у `main` типа возврата, отличного от `int`, если реализация явно разрешает это делать программисту. В этих условиях вызов `exit (n)` не обязательно эквивалентен выполнению `return n;` в `main`. Фактически, инструкция `return n;` может быть недопустимой (например, если объявлено, что `main` возвращает `void`).

Вопрос: Есть ли связь между функцией `abort` и сигналом `SIGABRT`?

Ответ: Да. Вызов `abort` фактически вызывает сигнал `SIGABRT`. Если для `SIGABRT` нет обработчика, программа завершается аварийно, как описано в Разделе 26.2. Если

обработчик был установлен для SIGABRT (вызовом функции signal), обработчик вызывается. Если обработчик возвращается, программа завершается ненормально. Однако, если обработчик не возвращается (например, вызывает longjmp), программа не завершается.

Вопрос: Почему существуют функции div и ldiv? Разве мы не можем просто использовать операторы / и %?

Ответ: div и ldiv - это не совсем то же самое, что / и %.

Вспомните из Раздела 4.1, что применение / и % к отрицательным операндам не даёт переносимого результата в С89. Если *i* или *j* отрицательны, то, округляется ли значение *i* / *j* в большую или меньшую сторону, определяется реализацией, как и знак *i* % 7. С другой стороны, ответы, вычисленные с помощью div и ldiv, не зависят по реализации. Частное округляется до нуля; остаток вычисляется по формуле $n = q \times d + r$, где *n* - исходное число, *q* - частное, *d* - делитель, а *r* - остаток. Вот несколько примеров:

<i>n</i>	<i>d</i>	<i>q</i>	<i>r</i>
7	3	2	1
-7	3	-2	-1
7	-3	-2	1
-7	-3	2	-1

1412

В C99 операторы / и % гарантированно дают тот же результат, что и div и ldiv.

Эффективность - это ещё одна причина, по которой существуют div и ldiv. Многие машины имеют инструкцию, которая может вычислять как частное, так и остаток, поэтому вызов div или ldiv может быть быстрее, чем использование операторов / и % по отдельности.

Вопрос: Откуда взялось название функции gmtime?

Ответ: Название gmtime означает среднее время по Гринвичу (GMT), относящееся к местному (солнечному) времени в Королевской обсерватории в Гринвиче, Англия. В 1884 году GMT было принято в качестве международного эталонного времени, а другие часовые пояса были выражены как часы "отстает от GMT" или "впереди GMT". В 1972 году всемирное координированное время (UTC) - система, основанная на атомных часах, а не на наблюдениях за Солнцем - заменило GMT в качестве международной системы отсчёта времени. Добавляя "дополнительную секунду" один раз в несколько лет, UTC синхронизируется с GMT с точностью до 0,9 секунды, поэтому для всех измерений времени, кроме самых точных, две системы идентичны.

Упражнения

К Разделу 26.1

1. Перепишите функцию `max_int` так, чтобы вместо передачи числа целых чисел в качестве первого аргумента мы должны были указать 0 в качестве последнего аргумента. Подсказка: `max_int` должен иметь хотя бы один "нормальный" параметр, поэтому вы не можете удалить параметр `n`. Вместо этого предположите, что оно представляет собой одно из сравниваемых чисел.

2. Напишите упрощённую версию `printf`, в которой единственной спецификацией преобразования является `%d`, и предполагается, что все аргументы после первого имеют тип `int`. Если функция встречает символ `%`, за которым не следует сразу символ `d`, она должна игнорировать оба символа.

Функция должна использовать вызовы `putchar` для получения всех выходных данных. Вы можете предположить, что строка формата не содержит escape-последовательностей.

3. Расширьте функцию упражнения 2 так, чтобы она позволяла использовать две спецификации преобразования: `%d` и `%s`.

Каждый `%d` в строке формата указывает аргумент `int`, а каждый `%s` указывает аргумент `char *` (string).

4. Напишите функцию с именем `display`, которая принимает любое количество аргументов. Первый аргумент должен быть целым числом. Остальные аргументы будут строками.

1414

Первый аргумент указывает, сколько строк содержит вызов. Функция будет печатать строки в одной линии, а соседние строки будут разделены одним пробелом. Например, вызов

```
display(4, "Special", "Agent", "Dale", "Cooper") ;
```

выдаст следующий результат:

```
Special Agent Dale Cooper
```

5. Напишите следующую функцию:

```
char * vstrcat (const char * first,...);
```

Предполагается, что все аргументы `vstrcat` являются строками, за исключением последнего аргумента, который должен быть нулевым указателем (приведённым к типу `char *`). Функция возвращает указатель на динамически выделяемую строку, содержащую объединение аргументов. `vstrcat` должна возвращать нулевой указатель, если недостаточно памяти. Подсказка: попросите `vstrcat` дважды перебрать аргументы: один раз, чтобы определить объем памяти, необходимый для возвращаемой строки, и один раз, чтобы скопировать аргументы в строку.

6. Напишите следующую функцию:

```
char * max_pair (int num_pairs,...);
```

Предполагается, что аргументы `max_pair` являются "парами" целых чисел и строк; значение `num_pairs` указывает, сколько пар последует. (Пара состоит из аргумента `int`, за которым следует аргумент `char *`). Функция ищет целые числа, чтобы найти наибольшее; Затем она возвращает следующий за ним строковый аргумент. Рассмотрим следующий вызов:
`max_pair(5, 180, "Seinfeld", 180, "I Love Lucy", 39, "The Honeymooners", 210, "All in the Family", 86, "The Sopranos")`
Самый большой аргумент типа `int` - 210, поэтому функция возвращает "Все в семействе", которое следует за ним в списке аргументов.

К Разделу 26.2

W7. Объясните значение следующей инструкции, предполагая, что `value` является переменной типа `long int`, а `p` - переменная типа `char *`:
`value = strtol (p, &p, 10);`

8. Напишите утверждение, в котором переменной `n` случайным образом присваивается одно из чисел 7, 11, 15 или 19.

W 9. Напишите функцию, которая возвращает случайное значение `double d` в диапазоне $0,0 \leq d \leq 1,0$.

1416

10. Преобразуйте следующие вызовы `atoi`, `atol` и `atoll` в вызовы `strtol`, `strtoul` и `strtoll` соответственно.

- (a) `atoi (str)`
- (b) `atol (str)`
- (c) `atoll(str)`

11. Хотя функция `bsearch` обычно используется с отсортированным массивом, иногда она работает правильно с массивом, который отсортирован только частично. Какому условию должен удовлетворять массив, чтобы гарантировать правильную работу `bsearch` для определённого ключа?

Подсказка: ответ содержится в стандарте C.

К Разделу 26.3

12. Напишите функцию, которая после принятия года возвращает значение `time_t`, представляющее 12:00 первого дня этого года.

13. В разделе 26.3 описаны некоторые форматы даты и времени ISO 8601. Вот ещё несколько:

- (a) Год, за которым следует день года: `YYYY-DDD`, где `DDD` - это число от 001 до 366.

1417

(b) Год, неделя и день недели: `YYYY-Www -D`, где `ww` - число от 01 до 53, а `D` - цифра между 1 по 7, начиная с понедельника и заканчивая воскресеньем

(c) Дата и время вместе: `YYYY-MM-DDThh:mm:ss`

Задайте строки `strftime`, соответствующие каждому из этих форматов.

Проекты по программированию

W 1.(a) Напишите программу, которая вызывает функцию `rand` 1000 раз, распечатывая младший бит каждого возвращаемого значения (0, если возвращаемое значение чётное, 1, если оно нечетное). Вы видите какие-либо закономерности? (Часто последние несколько бит возвращаемого значения `rand` не являются случайными.)

(b) Как мы можем улучшить случайность `rand` для генерации чисел в небольшом диапазоне?

2. Напишите программу, которая проверяет функцию `atexit`. Программа должна иметь две функции (помимо `main`), одна из которых выводит "Вот и все", а другая, "ребята!" Используйте функцию `atexit` для регистрации обеих, которые будут

вызываться при завершении программы. Убедитесь, что они вызываются в правильном порядке, чтобы мы увидели сообщение. Вот и все, ребята! на экране.

W 3. Напишите программу, которая использует функцию `clock` для измерения времени, необходимого `qsort` для сортировки массива из 1000 целых чисел, которые изначально находятся в обратном порядке. Запустите программу также для массивов из 10000 и 100000 целых чисел.

W 4. Напишите программу, которая запрашивает у пользователя дату (месяц, день и год) и целое число `n`, а затем печатает дату, которая будет через `n` дней.

5. Напишите программу, которая предложит пользователю ввести две даты, а затем напечатает разницу между ними, измеренную в днях. Подсказка: используйте функции `mktime` и `difftime`.

W 6. Напишите программы, которые отображают текущую дату и время в каждом из следующих форматов. Используйте `strftime` для выполнения всего или большей части форматирования.

(a) Sunday, June 3, 2007 05:48p

(b) Sun, 3 Jun 07 17:48

(c) 06/03/07 5:48:34 PM

27 Дополнительная поддержка C99 для математики

Простота не предшествует сложности, а следует за ней.

Приложение А Операторы C

Приоритет	Имя	Символ (ы)	Ассоциативность
1	Индекс массива	[]	Левая
	Вызов функции	()	
	Член структуры и союза	. ->	
	Инкремент (постфикс)	++	
	Декремент (постфикс)	--	
2	Инкремент (префикс)	++	Правая
	Декремент (префикс) --	--	
	Адрес	&	
	Разыменование	*	
	Унарный плюс	+	
	Унарный минус	-	
	Побитовое дополнение	~	
	Логическое отрицание	!	
	Размер	sizeof	
3	Приведение	()	Правая
4	Мультипликативные	* / %	Левая
5	Аддитивные	+ -	
6	Побитовый сдвиг	<< >>	
7	Отношения	< > <= >=	

1420

8	Равенство	!= ==	
9	Побитовое И	&	
10	Побитовое исключительное ИЛИ	^	
11	Побитовое ИЛИ		
12	Логическое И	&&	
13	Логическое ИЛИ		
14	Условный оператор	? :	Правая
15	Присваивание	= *= /= %= += -= <<= >>= &= ^= =	Правая
16	Запятая	,	Левая

Приложение Е Кодированный набор символов ASCII

Escape Sequence										
Decimal	Oct	Hex	Char	Character						
0	\0	\x00		<i>nul</i>	32		64	@	96	`
1	\1	\x01		<i>soh</i> (^A)	33	!	65	A	97	a
2	\2	\x02		<i>stx</i> (^B)	34	"	66	B	98	b
3	\3	\x03		<i>etx</i> (^C)	35	#	67	C	99	c
4	\4	\x04		<i>eot</i> (^D)	36	\$	68	D	100	d
5	\5	\x05		<i>enq</i> (^E)	37	%	69	E	101	e
6	\6	\x06		<i>ack</i> (^F)	38	&	70	F	102	f
7	\7	\x07	\a	<i>bel</i> (^G)	39	'	71	G	103	g
8	\10	\x08	\b	<i>bs</i> (^H)	40	(	72	H	104	h
9	\11	\x09	\t	<i>ht</i> (^I)	41	)	73	I	105	i
10	\12	\x0a	\n	<i>lf</i> (^J)	42	*	74	J	106	j
11	\13	\x0b	\v	<i>vt</i> (^K)	43	+	75	K	107	k
12	\14	\x0c	\f	<i>ff</i> (^L)	44	,	76	L	108	l
13	\15	\x0d	\r	<i>cr</i> (^M)	45	-	77	M	109	m
14	\16	\x0e		<i>so</i> (^N)	46	.	78	N	110	n
15	\17	\x0f		<i>si</i> (^O)	47	/	79	O	111	o
16	\20	\x10		<i>dle</i> (^P)	48	0	80	P	112	p
17	\21	\x11		<i>dc1</i> (^Q)	49	1	81	Q	113	q
18	\22	\x12		<i>dc2</i> (^R)	50	2	82	R	114	r
19	\23	\x13		<i>dc3</i> (^S)	51	3	83	S	115	s
20	\24	\x14		<i>dc4</i> (^T)	52	4	84	T	116	t
21	\25	\x15		<i>nak</i> (^U)	53	5	85	U	117	u
22	\26	\x16		<i>syn</i> (^V)	54	6	86	V	118	v
23	\27	\x17		<i>etb</i> (^W)	55	7	87	W	119	w
24	\30	\x18		<i>can</i> (^X)	56	8	88	X	120	x
25	\31	\x19		<i>em</i> (^Y)	57	9	89	Y	121	y
26	\32	\x1a		<i>sub</i> (^Z)	58	:	90	Z	122	z
27	\33	\x1b		<i>esc</i>	59	;	91	[	123	{
28	\34	\x1c		<i>fs</i>	60	<	92	\	124	
29	\35	\x1d		<i>gs</i>	61	=	93	]	125	}
30	\36	\x1e		<i>rs</i>	62	>	94	^	126	~
31	\37	\x1f		<i>us</i>	63	?	95	_	127	<i>del</i>